# THE OMNI-COMPASS OPERATOR'S MANUAL

*How to Embed, Wire, Tune and Run Omni-Compass on Your Stack*

**AJ Dubra**
Owner and Developer

**THE OMNI-COMPASS LLC**
October 2026

**The Omni-Compass Operator's Manual**
How to Embed, Wire, Tune and Run Omni-Compass on Your Stack
October 2026

Copyright © 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. No part of this book may be reproduced, stored or transmitted in any form or by any means, except as allowed by the Omni-Compass Evaluation License, without the written permission of The Omni-Compass LLC.

The Omni-Compass LLC reserves the right to change this book, the software it describes and their terms at any time, without notice. Every copy, export and printout of any part of this book carries this page, the LICENSE, the NOTICE and the Disclosures unchanged.

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by Dubra Software Inc. / The Omni-Compass LLC (no filing number is stated, and no grant, registration or approval is claimed). www.omni-compass.com

“Omni-Compass”, the Omni-Compass rose and the related names and marks are trademarks of The Omni-Compass LLC. Other names that appear in this book are the property of their owners and are used only to identify the systems Omni-Compass works with.

**License.** This book and the software it describes are proprietary and are not open source (SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0). They may be used only to evaluate Omni-Compass and to reproduce its published results, in simulation and on systems the reader owns or controls. Any commercial use, commercialization, monetization, production use, redistribution, hosted or managed service, or incorporation into any product or service requires a written Omni-Compass Enterprise License, signed by The Omni-Compass LLC and paid for. The full terms are printed in Appendix O.

**Disclaimer.** The software is provided “as is”, without warranty of any kind. This book is the wiring and operating manual and carries no benchmark results; every published result carries its evidence class. A modelled result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this book is a promise of a particular saving on a particular system: the only number that applies to a system is the one its own paired runs produce, on its own receipt.

Set in Liberation Serif and DejaVu Sans Mono, the text at 11 pt on a 16.5 pt line. Built from the repository by `docs/book/build_book.py`; the book and the code change in the same commit.

The Omni-Compass LLC · www.omni-compass.com

# Contents

**The Problem and the Answer** viii

**Foreword** x

**Preface** xi

**Prelude: A Night on the Cluster** xiii

**Introduction: The Idea and What This Manual Covers** xv

**The key: GAIN, STEADY and LOSS** xvii

**What You Are Looking At** xx

**Disclosures, Declarations and Disclaimers** xxii

**Quick Reference: The Card for the Glove Box** xxxiii

**Part One. The Governor, from the Outside** 1

1. What Omni-Compass Is . . . . . 1
2. Your Stack as One Body: Muscles and Organisms . . . . . 8
3. The Spine: the Base Every Stack Runs On . . . . . 13
4. Tuning: The Factory Tune, the Dial and the Settings You Write . . . . . 18

**Part Two. The Harness: Plugs, Wires and the Rules They Obey** 25

5. The Plug, the Adapters and the Wire Check . . . . . 25
6. The Rules the Wires Obey . . . . . 36

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Part Three. Wiring It onto Your Stack, Step by Step 45**

7. Before You Start, and the Eight Levels . . . 45
8. Stack by Stack . . . 55

**Part Four. Operating It 77**

9. The OFF Switch, the Rules, and the Log . . . 77
10. Maintenance, Upgrades and Security . . . 82

**Part Five. Checking It on Your Own System 84**

11. Paired Runs and Receipts on Your Own System . . . 84
12. Evidence Classes and How to Read a Receipt . . . 89
13. The Engine Version and the Three-Run Rule . . . 91

**Part Six. The Implementer's Guide: Omni-Compass on a Cloud 94**

14. The Implementer's Path: From an Empty Account to a Governed Cluster . . . 95
15. Identity, Permissions and Least Privilege on a Managed Cluster . . . 100
16. Azure Kubernetes Service, Step by Step . . . 106
17. Google Kubernetes Engine, Step by Step . . . 111
18. Amazon Elastic Kubernetes Service, Step by Step . . . 114
19. The Response-Time Feed and the Line . . . 118
20. Running in the Cloud: the Whole Engine, the Bill and the Alarm Minutes . . . 122
21. Day Two: Running Omni-Compass in Production . . . 130
22. The Graphics Processor: Putting the Governor on a Card . . . 136
23. Observability: Feeding the Governor from Your Monitoring, and Watching It . . . 140
24. Change Management, Risk and Compliance . . . 150
25. Planning a Cloud Run: Capacity, Cost and the Shape of Load . . . 153
26. On Your Own Racks and in Private Clouds . . . 161
27. Hardening and the Supply Chain . . . 166
28. Rolling Out Across a Fleet: Many Clusters, Many Services, GitOps . . . 178
29. Failure Modes and the Drill Book . . . 184

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

30. A Complete Installation, Start to Finish . . . 189
31. Questions an Implementer Asks . . . 195

**Part Seven. License and Your Copy** **197**

32. License and Commercial Terms . . . 197
33. Verifying Your Copy . . . 198

**Back Matter** **198**

Glossary . . . 198
Appendix A. Command Reference . . . 205
Appendix B. The Files an Installation Touches . . . 209
Appendix C. The Gauges . . . 210
Appendix D. Troubleshooting . . . 213
Appendix E. The Muscles: What Each Is For, and How It Is Wired . . . 217
Appendix F. The Azure Account, Click by Click . . . 543
Appendix G. The Google Cloud Account, Click by Click . . . 547
Appendix H. The Amazon Web Services Account, Click by Click . . . 550
Appendix I. The Install Manifests . . . 553
Appendix J. The Implementer's Checklists . . . 564
Appendix K. The Kubernetes Governor's Settings . . . 566
Appendix L. The Audit, Record by Record . . . 575
Appendix M. The Three Clouds Side by Side . . . 581
Appendix N. The Paired Run from GitHub: Every Input . . . 585
Appendix O. The License . . . 588
Appendix P. Licensing: Questions and Answers . . . 600
Appendix Q. Third-Party Notices . . . 602
Appendix R. The Enterprise License: Standard Terms . . . 603
Appendix S. Patents and Trademarks . . . 607
Appendix T. Security Policy . . . 609

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Index . . . . . 610

Contact . . . . . 624

# List of Figures

Every figure is drawn from the code it pictures, by one of two kits (`docs/book/figures.py`; `docs/book/schematics.py` for the wiring schematics, block diagrams, flowcharts, timing diagrams, state machines, the receipts charted from their own tables and the grids), and printed where the text first needs it; the same drawings stand beside the text on GitHub, in `docs/book/figures/`. A chart of a receipt is read by the reading key: it pictures the table and adds nothing to it.

Figure 1. Block diagram: the verdict's paired trial around a slow knob, read GAIN, STEADY or LOSS . . . . . 4

Figure 2. State machine: the governor's modes . . . . . 5

Figure 3. One pedal, no brake: the modes and the two switches . . . . . 7

Figure 4. Grid: the families of the spine and of the Physics / Robotics / Autonomous realm . . . . . 9

Figure 5. Grid: the families of the Compute / AI / Cloud realm . . . . . 11

Figure 6. The seven organisms on the spine . . . . . 12

Figure 7. Grid: the families of the Energy / Facility / Industrial realm . . . . . 14

Figure 8. Grid: the families of the Distribution / Specialized realm . . . . . 15

Figure 9. The compass band, its cushions, and the smoothing of the force . . . . . 22

Figure 10. The two-way plug: one wire out, one wire in . . . . . 26

Figure 11. State machine: the life of one muscle under the plug . . . . . 28

Figure 12. Flowchart: the wire check, steps 1 to 4 . . . . . 29

Figure 13. Flowchart: the wire check, steps 5 to 7 and the verdict . . . . . 30

Figure 14. The muscle SDK's runtime: what happens to one command . . . . . 33

Figure 15. One decision of the governor, from the senses to the audit . . . . . 37

Figure 16. Flowchart: the release gate, reasons 1 to 5 . . . . . 39

Figure 17. Flowchart: the release gate, reasons 6 to 9 and the release . . . . . 40

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Figure 18. Timing diagram: the governor's clocks against the native controllers' own . . . . . 43
Figure 19. The eight levels of an installation . . . . . 48
Figure 20. Flowchart: the verdict's trial on one step, from the reference to the reading . . . . . 53
Figure 21. Flowchart: the verdict's trial on one step, from the reading to the step allowed or refused . . . . 54
Figure 22. Timing diagram: a give-back trial from reference to verdict . . . . . 55
Figure 23. Wiring schematic: Kubernetes in the cluster, levels 2 to 4 . . . . . 58
Figure 24. Wiring schematic: a managed cloud cluster, the whole engine at the max . . . . . 60
Figure 25. Wiring schematic: a database behind its connection pooler . . . . . 62
Figure 26. Wiring schematic: three independent simulators, each on its own native controller . . . . . 63
Figure 27. Wiring schematic: a message broker's consumer group . . . . . 64
Figure 28. Wiring schematic: a cache's memory ceiling . . . . . 66
Figure 29. Wiring schematic: a drone swarm's cruise override . . . . . 67
Figure 30. Wiring schematic: a database's storage-engine cache . . . . . 69
Figure 31. Flowchart: the direction rule of a cache knob . . . . . 70
Figure 32. Wiring schematic: a buffer pool resized online in the server's own chunks . . . . . 72
Figure 33. Wiring schematic: the machine's frequency governor under the processor's meter . . . . . 73
Figure 34. Wiring schematic: the data center under the servers . . . . . 75
Figure 35. Flowchart: the three ways a governor stops . . . . . 79
Figure 36. Flowchart: one paired run, native against Omni-Compass on top, the pairs . . . . . 85
Figure 37. Flowchart: one paired run, from the pairs to the three-run table . . . . . 86
Figure 38. Flowchart: from the audit to the receipt, the table and the one number . . . . . 90
Figure 39. Flowchart: the three-run reading rule, a change of 0.5% or more a change . . . . . 92
Figure 40. The three-run rule: a change of 0.5% or more is a change, STEADY under it, and the runs that disagree . . . . . 93
Figure 41. Flowchart: the implementer's eight phases and their gates . . . . . 98
Figure 42. The load schedules of the cloud sets: two bursts and the real day . . . . . 110
Figure 43. Wiring schematic: the response-time feed and the line . . . . . 120
Figure 44. Flowchart: parking and waking a machine in the cloud . . . . . 123
Figure 45. Deployment diagram: the governor on a managed cluster . . . . . 127

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Figure 46. Modelled trace: one pedal over a burst, machines against demand . . . . . 129
Figure 47. Timing diagram: the hand-back after a reset, a kill and a death . . . . . 132
Figure 48. Wiring schematic: a graphics card, two wires . . . . . 137
Figure 49. Block diagram: the two wires of a card as an antagonist pair . . . . . 139
Figure 50. State machine: the two-wire card governor . . . . . 141

# The Problem and the Answer

**What the whole industry faces.** Every computing system in service, from one server to a cloud region, from a GPU to a building's chillers, a grid feeder or a robot cell, is run by a stack of native controllers: the Kubernetes autoscalers and scheduler, the cloud's node pools, the GPU's power and clock firmware, the database's cache and connection pools, the building's cooling loops, the battery's management system, the servo's loop. Each was designed and tuned alone, against its own setpoint, with margins sized for the worst case it might meet. Three costs follow, on every stack:

1. **Capacity held idle.** Worst-case margins stay in place through the common case. Machines, power caps, caches and cooling run far inside their limits, and the room between the operating point and the limit is paid for and seldom used.
2. **Late reaction.** Reactive loops act on averaged, lagging signals and wait out deliberate cooldowns, so queues and response-time tails form before capacity arrives.
3. **Loops at cross purposes.** The loops share power, cooling, connections and memory. One loop's correction is another loop's disturbance, and no loop sees the whole.

Demand for computing, above all for AI and the cloud, is growing faster than the power, cooling and hardware that can be built for it. The usual answer is to add: more machines, more power, more people watching thresholds. Replacing the native controllers is not an answer most operators will take: they are proven, often certified, and the people on call know them.

**What Omni-Compass does about it.** Omni-Compass is a supervisory governor. It sits on top of those native controllers and never replaces them. It never touches the data: no request, packet, file or byte of the work passes through it. It plugs into each setting a native controller already exposes, through a two-way plug (one wire in, the reading; one wire out, the setting), takes a snapshot of every setting before its first write, and governs the whole stack as one body, the organism: every muscle driven as far as its own limit and never past it, and, on the engine that stands, never below native; where it cannot bring a muscle a GAIN, that muscle stays native. It acts before a line is crossed instead of after, reads back every write, puts every give-back to its own trial, records every decision with its reason, walks every setting home if it stops (the lease) and stops at once on the kill switch. Its first rule is do no harm, judged at the organism: never less work or health than

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

native (`DISCLOSURES.md`, section 3). The one headline is an aim, not a result. The aim: more work, faster, on fewer machines, for the same energy and the same bill, from the hardware and the controllers already in place, every benefit made at the software end; in a cloud and at the chip, Omni-Compass holds. The record reads the work and the speed (the Omni index, `results/OMNI_INDEX.md`); the machines are not yet fewer: the whole organism's composition reads them more, power given (`results/WHOLE_SYSTEM.md`).

**Where the proof is.** This book shows how to wire, tune, run and check Omni-Compass. The mechanism, the mathematics and every result, each a paired comparison of native and Omni-Compass on top of native run three times, every row kept, LOSS included, are in the Owner's Manual and in `results/`. The only number that applies to your system is the one your own paired runs produce. The first thing the Owner's Manual, the README and the live benchmark engine page show is the bottom line: one light (green and round GAINING, yellow and diamond STEADY, red and octagonal REDLINING) and one number against native for each way to run Omni-Compass (native alone, Omni-Compass on native, Omni-Compass direct) and for each route a whole workload is bought by (on site, through the cloud). This book carries none of those numbers.

**What you install, and what it starts on.** Omni-Compass consists of a governing engine, its muscle adapters, a complete configuration profile and a validation harness. An installation starts on the **factory profile**, the complete, versioned set of Omni-Compass default settings shipped together, and every change from it makes a **custom profile** that is the installation's own (section 4.8). The words are written once, with their exact meanings and their mathematics, in `docs/FACTORY_PROFILE.md`; this book uses them as written there and carries the procedure only.

**The cloud is a wire.** A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center, on site or off site), and Omni-Compass can be wired in at all three, and in each it acts in one place, the software end, on top of the controllers already there. The wire makes no benefit of its own: the benefit is made where Omni-Compass sits, at the software end and at the chip, and what a cloud run must show is that the engine holds the same on a cloud's machines as on site (no judged gauge LOSS against native), the work per dollar read beside the bill (Part Six, chapter 20; `DISCLOSURES.md`, sections 9 and 10; the founder's words of 31 October, night: "We just need to show we hold"). A smaller bill is never the aim; how a company spends the gain, as more work or as a cut, is its own choice. Where Omni-Compass sits on neither end, a cloud's bill shows nothing of Omni-Compass.

**What is not claimed.** No new physics, no new model of computation, and no promise for any system Omni-Compass was not run on. It does not make a system immune to failure, delay or drift: it keeps each muscle inside its own limits and shows, row by row, what it did against native. A result on another system is that system's own, from its own paired runs; where it differs, the first presumption is the wiring (`DISCLOSURES.md`, section 4).

**What is protected, and how.** Omni-Compass belongs to The Omni-Compass LLC. Its source, manuals and records are its copyrighted work; all patents, copyrights and trademarks filed in the USA. The repository is licensed for Evaluation and Simulation use only (`LICENSE`); any commercial use, monetization or production

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

use requires a signed, paid Omni-Compass Enterprise License (`ENTERPRISE_LICENSE.md`). What a licensee acquires is the working whole: the engine, the wiring and the factory card of every muscle, the harness that measures it against native, and the receipts.

# Foreword

I built Omni-Compass to sit on top of what already exists. Kubernetes, the GPU driver, the building controller, the battery inverter, the robot's servo loop: those are muscles, and they are good muscles. What none of them has is one governor that reads all of them at once, holds each of them inside its safe range, and puts every setting back exactly where it found it when it stops. That governor is Omni-Compass, and this book is how you put it in.

**Every machine already has a controller.** Kubernetes has its autoscaler. A graphics card has its firmware. A building has its thermostat loop, a battery has its charge controller, a power grid has its tap changers, and a robot has its servo. Engineers spent decades making these controllers safe, and they are. But each one was built to watch one thing and to protect that one thing. None of them sees the whole body. None of them knows that the cooling loop is holding back the processor, or that the autoscaler is adding machines the database cannot feed. So each one plays it safe in the middle of its range, because playing it safe is its whole job. The headroom between the middle and the edge is paid for every hour and almost never used.

**That gap is what you are wiring it to close.** Omni-Compass is a **supervisory governor**: one supervisor over the muscles you already run. It never replaces them and it never competes with them. It reads them, it decides, and it writes one setting at a time through a two-way plug: one wire in, one wire out. Before it writes anything, it takes a snapshot. When it stops, every setting goes back to that snapshot.

**The rule I will not bend: do no harm.** Its real goal is the organism you wire it onto: to strengthen it and keep it healthy, working at the best state its own muscles can carry, turning what is fed into it (its work, its data, its energy) into the most and best output it can make. Harm is the organism turning out less than its native controllers make it turn out, or made less healthy: a muscle pushed past its own limit, a service over its own line, anything made to fail. Omni-Compass never does either, and it pushes a muscle only as far as that muscle itself can go (`DISCLOSURES.md`, Chapter 4). The engine that stands keeps the rule in its strictest form: where it cannot bring a muscle a gain, that muscle stays native, and before a slow setting moves, the governor tries the step on that muscle and keeps it only if the muscle's own work costs no more for it. A step that costs more is undone.

**The max is the factory setting.** Omni-Compass ships tuned the way I tuned it: every muscle runs up to its own limit, read live from the system, never past it and never below what native gives. The energy it takes to do more work is **power given**, shown beside the work, the speed and the wear it bought. It is never called a loss. Anyone can turn the dial down from the max, retune a muscle, or add one; nobody has to.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**One pedal.** I wanted it to drive like an electric car. Push the pedal for more, lift off for less. Lift off all the way and the settings drift back toward native. Reset hands everything back. If the governor ever crashes, its lease runs out and the plug walks every setting home, step by step. The kill switch is separate and instant, and it exists for security, not for daily driving.

**What this book holds.** This is the manual for the people who wire it: what the governor is from the outside, how to tune it, how the plug and the wires work, how to wire it onto each kind of stack one level at a time, how to run it, how to switch it off, and how to check on your own system, with your own paired runs, what it does there. It is the book you keep in the glove box. When the software changes, this manual changes in the same commit.

**How I want it judged.** On your own receipts. Every paired run sets native beside Omni-Compass on top of native, and, where native's own controller can be switched off, Omni-Compass direct beside them ("there's native, there's Omni and native, and then there's Omni by itself", 23 October). Run it three separate times under rules you wrote down first, show every row, losses included, and read each row as one word, **GAIN**, **STEADY** or **LOSS**, by what the gauge is for. Where a receipt reads LOSS, look at the wiring first; this book tells you where.

**Where to begin.** Read the front matter first. It tells you what you are looking at before it asks you to touch anything. Then wire it one level at a time, watch it before you let it write, and judge it by your own receipts.

**AJ Dubra** Founder, The Omni-Compass LLC

# Preface

I wrote this book for the engineers who put Omni-Compass into a running system: how to embed it, how to wire it to every muscle of a stacked, clustered system or to any one muscle, how to tune it, how to run it, how to switch it off, and how to check on their own system, with their own paired runs, what it does there. It is the manual for the glove box, not a description of how the engine is built.

The book is in six parts, after the front matter.

| You are | Read |
|---|---|
| CTO, architect, head of platform | the front matter, Part One, the eight levels in Part Three, then Part Five |
| The engineer wiring it | everything, in order. Do not skip the wire check or the watch level |
| Platform engineer for one stack | Part Two, the chapter of Part Three for your stack, and Part Four |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| You are | Read |
|---|---|
| Data-center, facility or energy operator | the Quick Reference card, Part One, Part Three (levels five to seven, the processor and the data center under the servers), Part Four |
| Site reliability engineer, on call | the Quick Reference card, Part Four (the switches and the log), the troubleshooting appendix |
| Security and compliance | Part Four, the Disclosures, the license in the back matter |
| Procurement and counsel | the Disclosures, Part Six, the license and its questions and answers in the back matter |

**Part One, the governor from the outside,** says what Omni-Compass is as an installer meets it, how the muscles of a stack become one body, and how it is tuned: the factory tune, the dial, each muscle's own card and the settings an operator writes. **Part Two, the harness,** is the universal plug, the adapters, the wire check and the rules every wire obeys. **Part Three** wires it onto a stack one level at a time and then stack by stack. **Part Four** is operating it: the OFF switch, the rules, the log, maintenance and security. **Part Five** is checking it on your own system: paired runs and receipts, evidence classes, the engine version and the three-run rule. **Part Six** is the license and how to verify a copy. The back matter holds the glossary, every command, the files an installation touches, the gauges, troubleshooting, and every muscle with its wiring and its own card.

**Conventions.** `Code` is a command, file or switch exactly as typed. "Native" means a system as it runs today, without Omni-Compass. "Muscle" means any machine, service or controller Omni-Compass can read and set. "Knob" or "lever" is one setting on a muscle. "The band" is a service reading's safe range. A step that writes to a system is marked **WRITES**. Every number in a receipt carries its evidence class: S for a model (modelled), L for live software, P for a physical meter.

**The standards every page of this manual keeps.** An installation wired by this manual should meet, and its receipts should show, the following discipline throughout:

1. **Native beside it, always.** Every paired run is read against native, on the same machines, with the same work at the same moments: omni (Omni-Compass on native) against native, and, where it is run, Omni-Compass direct against native and against omni. Omni-Compass on native is never compared as a competitor to the controller it sits on; the direct arm switches that controller off to show what Omni-Compass does as the only controller, and is never read in place of the arm on native.
2. **Written down, then run.** The arms, the knob, its cover, the gauges, their aims and the readings are written down before the first counted run. Rules are fixed on one tuning case that is shown and never counted, then applied unchanged to the cases that are counted. A change made after a result has been seen is written down with its time and reason, and the receipt that carries the result says so.
3. **Three separate runs, one rule.** A result is run three times (A, B and C) on the same engine version and profile, and every judged row reads by one rule: the change is the mean of the three runs' changes in percent of native; a mean under 0.5% reads STEADY, under the floor (STEADY, the same where nothing

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

moved); from 0.5% up the word comes from what the gauge is for, GAIN, LOSS or power given, **in all three of our runs** where every run moved the same way with its 95% interval clear of zero, otherwise **by the mean of our three runs** with the count of runs beyond the noise; runs that moved 0.5% or more in opposite directions read **the runs disagree** (chapter 13). Nobody outside The Omni-Compass LLC has reviewed or approved the published results, and every three-run table says so.

4. **Every row, losses included.** A row that went against Omni-Compass is in the receipt beside the rows that went for it. When Omni-Compass loses, the first suspect is the wiring, then native's setup, then the scoring; when that is fixed, the run is repeated and the loss stays in the record.
5. **Nothing read across engine versions.** Each engine version is fingerprinted (`python3 tools/omni_version.py`). A receipt names the version it ran on and is never read against a receipt of another.
6. **Do no harm.** The organism is never left turning out less than native, nor less healthy, and every muscle works only as far as its own limit (`DISCLOSURES.md`, section 3). On the engine that stands, Omni-Compass moves a slow knob only where a paired trial on the muscle itself shows the muscle never behind for it (section 1.2). Where it cannot bring a knob a gain, that knob stays native, and the receipt says "nothing for Omni to move".

# Prelude: A Night on the Cluster

This is a story, not a result. It follows one night on one cluster to show, in plain order, what each part of this book does. The shop, the times and the settings are made up for the story. A measured result belongs to a receipt, and chapter 11 says how to make one on your own system.

**9:00 p.m. The shop is quiet.** An online store runs on Kubernetes. Its native controllers are the ones almost everybody uses. The Horizontal Pod Autoscaler watches the average processor load and adds copies of the web service (pods) when that load passes its target. The cluster autoscaler adds machines when the pods no longer fit. A careful engineer picked their settings months ago, for the worst day she could imagine, and she picked them well. Tonight the cluster runs on its floor of two machines, and every customer gets an answer in a blink.

**A week earlier, Omni-Compass went on in watch mode.** For seven days it read every gauge and wrote nothing. It learned how fast each pod starts, how the response time moves when the load moves, and where each setting's safe band lies. Then the team ran the wire check: every reading came from the place it claimed, every lever moved the thing it claimed, and every write was read back. Only then did they turn it to govern. Every setting it would touch was saved in a snapshot first.

**9:40 p.m. The sale starts.** In two minutes the requests double. The native autoscaler does exactly what it was built to do, but it is built to wait: it needs the average load to climb, then it needs a decision, then each new pod needs time to start. While it waits, customers wait. With Omni-Compass on top, the compass sees the response

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

time slide toward the edge of its band and into the cushion. It pushes. It raises the autoscaler's floor of pods, a setting the autoscaler already respects, so new pods start before the average catches up. It does not fight the autoscaler and it does not go around it. It moves one setting, through the plug, and reads it back.

**10:15 p.m. The database starts to strain.** More web pods mean more connections to the database, and the database has a limit of its own. A controller that watches only the web service would keep adding pods and make the jam grow. Omni-Compass sees the whole body. One body, one brain: before it takes the next step for the web service, it checks what that step would do to every other part. The step that would push the database past its own cushion is refused. Nothing is pushed into a loss to buy a gain somewhere else. The web service holds where the database can still keep up, and the queue drains instead of growing.

**11:30 p.m. The rush fades.** The pedal lifts. Omni-Compass does not drop everything at once, because a sudden drop is how systems stumble. It eases each setting back toward native along a smooth curve. The extra machines go away as the work goes away. The energy the cluster used during the rush shows up in the report as power given, beside the work it bought and the minutes customers did not spend waiting.

**1:00 a.m. Idle.** The pedal is lifted all the way. The cluster sits on its floor of two machines again, exactly where the engineer left it. Omni-Compass keeps reading every second. Staying native is its live decision, made again each second, not a verdict from some earlier run.

**3:12 a.m. Something breaks.** An upgrade restarts the machine that runs the governor, and Omni-Compass stops in the middle of its work. Nobody is awake. Nothing bad happens. The governor's lease runs out, and the plug walks every setting back to its snapshot, step by step. The native controllers keep running as they always have, because they never stopped. When the governor comes back, it starts from what it reads, not from what it remembers.

**7:00 a.m. The morning report.** On a live system, the proof does not come from a feeling that the night went well. It comes from paired runs: the same work, run once on native and once with Omni-Compass on top, and compared gauge by gauge. Every gauge gets one word. Time waited that fell reads **GAIN**. Work done that rose reads **GAIN**. A change under half of one percent reads **STEADY**, and the report says so plainly; a change of 0.5% or more shows its word. A gauge that moved the wrong way reads **LOSS**, and it stays in the report until its cause is found and fixed. The energy spent on extra work reads **power given**, never a loss.

**What the night shows.** Every part of that night is a chapter of this book:

- the band, the cushions, the line and the factory tune are in chapter 4;
- the plug, the snapshot and the wire check are in chapter 5;
- one body, one brain, and blind means hold, are in chapter 6;
- the watch mode and the levels are in chapter 7, and the cluster's own wiring in section 8.1;
- the lease and the kill switch are in chapter 9;
- the paired runs and the three words are in chapters 11 to 13.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Read the rest of the front matter before you touch a wire. It tells you what you are looking at and how to read what it tells you.

# Introduction: The Idea and What This Manual Covers

**The problem.** Stated in brief at the front of this book ("The Problem and the Answer"); here in engineering terms. Production systems are governed by many local feedback loops: horizontal autoscalers, firmware power and clock governors, building-management loops, battery-management systems, connection pools, servo controllers. Each loop is tuned in isolation, against its own setpoint, with margins sized for a worst-case disturbance. Three costs follow, and every operator pays them:

- **Reaction latency.** Reactive loops act on lagging, averaged signals. A Kubernetes Horizontal Pod Autoscaler recomputes its desired replicas from averaged utilization on a sync period (15 seconds by default) and damps its decisions with stabilization windows; new pods then need time to schedule and start. Queues form before capacity arrives.
- **Static over-provisioning.** Worst-case margins persist through the far more common nominal regime. The operating point sits well inside the safe region, and the headroom between it and the constraint boundary is paid for and rarely used.
- **Uncoordinated coupling.** Loops share resources: power, cooling, connections, memory, network. One loop's correction is another loop's disturbance. More web replicas saturate the database's connections; more cooling pushes a power feed toward its limit.

Operators close these gaps by hand, with fixed settings and runbooks. That is safe, and it leaves the region between the nominal operating point and the safe boundary unused nearly all of the time.

**The idea.** Close one supervisory loop around the native loops. Omni-Compass treats each setting a native controller already exposes (a replica floor, a power cap, a clock ceiling, a temperature setpoint, a pool size) as an actuator and each native telemetry stream as a measurement, and governs the whole body from above. Seen from the outside, every decision period runs the same five steps, and those five steps are all an installation has to provide for:

1. **Sense.** It reads each muscle's own gauges through the plug: utilization, response-time percentiles, queue depth, temperature, power, state of charge.
2. **Place.** It places each reading in its band, from calm to the line, with a 5% cushion at each edge, so that every muscle's state is read on the same scale whatever its units.
3. **Decide.** It chooses at most one bounded step per actuator, smooth and never hammering, clipped to the actuator's cover, the hard range nothing can leave.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

4. **Actuate.** It writes the step through the plug, idempotently, and reads the value back to confirm the write landed.
5. **Verify.** It compares the muscle with and without the step in a paired trial on the muscle itself, judged on the muscle's own cost per unit of work, and checks every coupled muscle against its own cushion. A step that fails either test is undone.

The installation supplies the wires (sense and actuate) and the settings (the band, the line, the cover and the tune); the governor does the rest. Nothing in an installation re-implements the decision, and nothing in this manual asks the installer to.

**What Omni-Compass is not.**

- It is **not a replacement** for your controllers. Native keeps running underneath, always. Omni-Compass only moves settings that native already exposes and respects.
- It is **not a black box**. Every decision is written to a log, with the reading that caused it and the setting it changed (chapter 9).
- It is **not a one-time tuning script**. It decides again every second, and staying native is one of its decisions.
- It is **not a cost-cutter first**. It gives power where the power buys work or speed, and it shows that power beside what it bought.

**The promise, in three parts.**

1. **Do no harm.** The organism is never left turning out less than native, nor less healthy, and no muscle is pushed past its own limit. On the engine that stands, a step stays only where the muscle loses nothing for it, measured on the muscle itself; where Omni-Compass cannot bring a gain, the muscle stays native.
2. **Everything comes back.** Every setting is saved before the first write and restored when Omni-Compass stops. If the governor crashes, its lease runs out and the settings walk home on their own. A separate kill switch stops everything at once.
3. **Every claim is checked.** Each paired run compares two arms, native and Omni-Compass on top of native, on the same machines with the same work, three separate times. Each reading is one word, GAIN, STEADY or LOSS, decided by what the gauge is for, and each carries its evidence class: **P** for a physical meter, **L** for live software, **S** for our own models. A model is never shown as proof of hardware.

**What it governs.** The catalog names 1,107 muscles in 70 families, the settings that operators everywhere already turn by hand: machine pools and replicas, power caps and clock limits, cache sizes and connection pools, cooling setpoints and battery reserves, robot speeds and drone-swarm settings, and many more. Appendix E lists every one: what it is, what Omni-Compass reads from it, what it moves, and the factory setting it runs on; and for each of the 70 families how to wire it on your own stack (native controller, wire out, wire in, restore, run live or modelled). Chapter 2 shows how the muscles of one stack are grouped into one body.

**What you need before you wire it.**

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **A license for any use beyond evaluation.** Production use requires a signed, paid Omni-Compass Enterprise License.
- **Read access first.** Omni-Compass starts in watch mode: it reads and computes, and writes nothing.
- **The wire check.** Every reading must come from where it claims, every lever must move what it claims, and every write must read back. Nothing writes until the check passes.
- **Your own service line.** The response time or other limit your service must meet comes from your own service objectives, not from ours.
- **Paired runs on your own system.** The only number that applies to your system is the one your own paired runs produce, on your own receipt.

**What this manual covers, and what it does not.** It covers how to embed, wire, tune, run and check Omni-Compass on a stacked, clustered system or on any one muscle, the way a vehicle's manual covers its controls, its settings, its service and its warning lights. It does not cover how the engine is built, and it carries no benchmark results: those are kept with the code, each in its own table with its evidence class.

**How to read what follows.** Next comes the Quick Reference card, the page to keep beside the system. Part One describes the governor from the outside and how to tune it; Part Two, the plug and the rules its wires obey; Part Three, the wiring, level by level and stack by stack; Part Four, operating it; Part Five, checking it on your own system. Read in that order, nothing in the wiring chapters should surprise you.

# The key: GAIN, STEADY and LOSS

Every judged reading in Omni-Compass is one of three words: **GAIN**, **STEADY** or **LOSS**. Better, same and worse are not used anywhere. The word is decided by what the gauge is for, never by the plus or minus sign of its number: who sets the aim differs from one system to the next, and each gauge's aim is written in its preregistration before the runs.

| Word | What each letter stands for | Its sense in underwriting | What it means in a table |
|---|---|---|---|
| **GAIN** | **G** Growth (asset appreciation); **A** Advantage (risk reduction); **I** Improvement (margin expansion); **N** Net-positive (profitable outcome) | surplus and positive variance | the gauge moved toward its declared aim against native: more work done, a shorter wait, fewer misses or failures, or a resource cut |
| **STEADY** | **S** Stable (zero variance); **T** True to baseline (expected norm); **E** Equilibrium (balanced risk); **A** At par (no net change); **D** Drift-free (within the 0.5% floor); **Y** Yield held (nothing | the par baseline: equilibrium | native and Omni-Compass within 0.5% of each other: the mean of the three runs' changes under the 0.5% floor, or no change at all; a knob native already runs at the top of its |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Word | What each letter stands for | Its sense in underwriting | What it means in a table |
|---|---|---|---|
| | lost) | | range is STEADY, at its limit on native |
| **LOSS** | **L** Leakage (capital drain); **O** Off-track (adverse selection); **S** Setback (claim liability); **S** Subpar (negative variance) | actuarial deficit and exposure | the gauge moved away from its declared aim: less work done, a longer wait, more misses or failures |

How the sign of a number becomes a word:

- **Time waited, missed or failed** (a response time, time over the line, failed requests): a fall is GAIN, a rise is LOSS.
- **Work done** (work inside the line, throughput, a hit rate): a rise is GAIN, a fall is LOSS.
- **A resource spent or held** (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear on a part): a fall is a **cut**, and a cut is GAIN. A rise is **power given**: what Omni put in to buy the work and the speed shown beside it. Power given is always shown and never hidden, and it is never called a loss (the founder's order of 12 October 2026: the max rides every muscle's edge; it never shaves). A peak is power held, so a peak that rose is power given, in a data centre and in a building alike (17 October).
- **Work bought with each unit of a resource** (work per energy): a rise is a cut in what each unit of work took, a GAIN; a fall is power given, because it falls when more power is given for the same work.
- So a minus can read GAIN (a response time cut 65%, a bill cut 30%) and a plus can read LOSS (failed requests up).
- **The floor, and every change above it** (the founder's orders of 20 and 24 October 2026): a change of half of one percent (0.5%) or more either way is a change, and reads **GAIN** or **LOSS** (or **power given**) by what the gauge is for; STEADY is kept for a change under 0.5%. The change is the mean of the three runs' changes, each in percent of native. Where every run moved the same way with its 95% interval clear of zero, the word reads **in all three of our runs**; otherwise it reads **by the mean of our three runs** and says in how many of the three runs it stood beyond the noise, so a reader sees how firmly it stands. Where runs moved 0.5% or more in opposite directions, the reading is **the runs disagree**. A mean under 0.5% reads **STEADY, under the floor**, with its size.
- **Two numbers, never mixed** (the referee's work order of 25 October): every row shows native's value and Omni-Compass's, the **raw change** (Omni-Compass − native) / native with its true sign, + for a rise and − for a fall whatever the gauge is for, labelled raw, and apart from it the word with its size and no sign. A bill cut 3% reads raw −3.0%, GAIN 3.0%; failed requests up 2% read raw +2.0%, LOSS 2.0%; a pod start wait from 0.22 s to 0.83 s reads raw +275.0%, LOSS 275.0%; a change under the 0.5% floor reads STEADY 0.3%. No raw sign is ever turned to match a word.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Some preregistrations name their outcome with a label written before the runs (for example SUPERIOR WITHIN GUARDRAILS in the card protocol). The label stays as it was frozen, and every table shows it with its word first: TOWARD ITS AIMS reads GAIN; BOTH WAYS reads GAIN AND LOSS; AWAY FROM ITS AIMS reads LOSS; SAME AS NATIVE reads STEADY (the same as native); AT ITS LIMIT ON NATIVE reads STEADY (at its limit on native); SUPERIOR WITHIN GUARDRAILS reads GAIN (within the guardrails); ENERGY IMPROVEMENT WITH SERVICE TRADEOFF reads GAIN AND LOSS (energy cut, service traded for it); SERVICE IMPROVEMENT WITH ENERGY TRADEOFF reads GAIN, power given (service GAIN, energy given for it); NONINFERIOR / INCONCLUSIVE reads STEADY (inside the noise, within the guardrails); NOT ESTABLISHED reads STEADY (not established: inside the noise, a guardrail crossed on the mean). A run that breaks its own validity checks reads INVALID and carries no word.

Every reading is one of these. **GAIN, LOSS or power given in all three of our runs**: A, B and C moved the same way, every 95% interval clear of zero. **GAIN, LOSS or power given by the mean of our three runs**: the runs' mean moved 0.5% or more, with the count of runs that stood beyond the noise beside it. **STEADY, under the floor**: the mean moved less than 0.5%. **STEADY, the same**: no change at all. **The runs disagree**: runs moved 0.5% or more in opposite directions, so the test is unstable there and is looked into. **Shown, not judged**: a gauge declared in advance as shown only, such as the knob's own setting.

Why these three words: they are the words of underwriting and actuarial work, where every outcome is graded as surplus, par or deficit, and each letter spells out the same rule. Omni-Compass is there to add (GAIN), never to take (LOSS), and to hand a muscle back to its own native setting where it cannot add (STEADY). Its first rule is do no harm, in the founder's meaning of 19 October 2026: the organism Omni-Compass governs is never left turning out less than its native controllers make it turn out, nor less healthy, and every muscle works only as far as its own limit (`DISCLOSURES.md`, section 3). So **LOSS should never appear**: where it does, the row stays in the table, its cause is found and fixed (our own wiring first, then native's setup, then the scoring) and the test is run again.

Disclosure and disclaimer: GAIN, STEADY and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results. The underwriting senses explain the words; they are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. A reading states what our own runs measured against native under the declared aims, not a promise about any other system; every number stands beside its word, so a reader can judge it by any other convention. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# What You Are Looking At

**What Omni-Compass is.** Omni-Compass is a supervisory governor. It is never a controller of its own and never a replacement for one: it sits on top of the controllers a system already runs (the Kubernetes autoscaler, a database's connection pool, a GPU's firmware, a building's battery controller, a power grid's tap changer, a robot's servo) and moves only the settings those controllers already expose to an operator, through one wire in (what it reads) and one wire out (what it writes). Everything the native controller does, it still does. Pull the plug and every setting walks back to where the operator left it (the lease); the kill switch returns them at once.

**What "native" means.** Native is the same stack, the same muscles, the same machines and the same work, run the way everybody runs it today: the shipped controllers at their shipped or operator settings. Every paired run sets omni beside native, where omni (Omni-Compass on native) is that same native stack with Omni-Compass on top. Since the founder's order of 23 October a third arm is run beside them wherever a muscle's native is a controller that can be switched off: **Omni-Compass direct**, the same stack with that controller off and Omni-Compass the only controller moving the knob (on Kubernetes no pod autoscaler and no cluster autoscaler; the scheduler and the kubelet stay). The three are always shown together and each pair is read on its own (section 1.1). Nothing else differs between the arms, so whatever differs in the result is what Omni-Compass did.

**What a muscle and an organism are.** A muscle is one setting a native controller exposes (a pool's size, a power cap, a clock ceiling, a replica count, a battery's charge rate) together with the gauges that show what it does. An organism is a stack of muscles governed as one body: the muscles of one company's clusters, databases, caches, cards, cooling and power, read by one governor and judged together, so that no muscle is moved at another's expense. Chapter 2 sets out how a stack is grouped into one body; Appendix E lists every muscle Omni-Compass can be wired to, with what it reads and what it moves, and for every family the wiring on your own stack: the native controller that owns the muscle, where the reading is tapped, where the setting is written, how it is restored, and whether that wire has run live or on a model.

**What you see when it is hooked up.** Every paired run ends in a receipt with the same shape: the two arms side by side, every gauge of the system (the work done, the response times, the misses and failures, the machines and energy spent), the paired change with its 95% interval, and one word per gauge, GAIN, STEADY or LOSS (the key, above). A three-run table repeats the paired run three times, A, B and C, on the same engine version and profile, and reads each row by one rule: the change is the mean of the three runs' changes in percent of native; a mean under 0.5% reads STEADY, under the floor (STEADY, the same where nothing moved); from 0.5% up the word comes from what the gauge is for, GAIN, LOSS or power given, **in all three of our runs** where every run moved the same way with its 95% interval clear of zero, otherwise **by**

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**the mean of our three runs** with the count of runs beyond the noise; runs that moved 0.5% or more in opposite directions read **the runs disagree** (chapter 13).

**How to read a receipt, in order.**

1. **The engine and the evidence class.** The header names the engine version (`python3 tools/omni_version.py`) and the class (P, L or S; chapter 12). A receipt is never read across engine versions, and S is never read as proof about real equipment.
2. **The work and the service rows first**: work done, response times, time over the line, failed requests. These say what the system delivered. A GAIN here is the point of the governor; a LOSS here is the first thing to look for.
3. **Then the resource rows**: machines, energy, memory, connections. A cut is a GAIN. A rise is **power given**: what Omni-Compass put in to buy the work and speed above it. Read the two together: power given that bought nothing is a cost to find and fix, starting with the wiring.
4. **Then the floor and the noise.** STEADY, a mean change under 0.5% (under the floor) or none at all, is a result, not a failure. A change of 0.5% or more is a change and shows its word, with the count of runs beyond the noise beside it whenever not all three are. The runs disagree means runs that moved 0.5% or more pointed different ways: the test is unstable there and is looked into.
5. **Then the clock and the machine.** Where the test machine itself is the limit (its processor saturated, native alone failing a large share of requests), the receipt says so; a test machine's limit is never read as the governor's result, and the test is rebuilt so the machine no longer decides it.
6. **Every row is shown**, LOSS included. Nothing is cut from a receipt because it went against Omni-Compass.

**How Omni-Compass tells GAIN from LOSS.** It does not read a plus or a minus sign as good or bad. Before it moves anything, every gauge it reads is declared with its aim, and the aim decides the word: for time waited, missed or failed a fall is GAIN; for work done a rise is GAIN; for a resource spent or held a fall is a cut, and a cut is GAIN, while a rise is power given, kept only while it buys the work or the speed shown beside it. A slow knob moves only as far as a paired trial on that muscle has shown costs its own work nothing (section 1.2); a step that costs more is undone, and where no step passes the muscle stays native, which is STEADY. Inside a body, a step that would push another part past its cushion is refused, so no GAIN in one muscle is bought with a LOSS in another (chapter 6).

**Wiring it: who, what, when, where, why and how.** Read this before the wiring chapters; a governor wired without it may run and still not be the mechanism.

- **Why.** To get more work, faster, with fewer misses, from the machines and the energy already paid for, by letting one governor move the settings native controllers already expose, in the direction each gauge's aim asks, and only while no other gauge pays for it. If nothing can be gained, the purpose is met by leaving native alone.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **What.** Two things per muscle and nothing else: one wire in (the gauges that show the muscle's state and its work, read live from the system, never typed into code) and one wire out (the one setting native already exposes to an operator, with its hard range read from the system). The governor does the arithmetic; the wiring never re-implements it.
- **Where.** On top of the native controller, never in place of it: on the autoscaler's target and ceiling, not on the pods; on the pool's size, not on the queries; on the card's power limit, not on its firmware. Every muscle of the stack is wired into the same governor, so the body is judged as one.
- **Who.** The operator who owns the system grants each wire, decides which knobs Omni-Compass may write and which it only watches, and keeps the kill switch. The governor writes only what it was granted; the native controller keeps doing its own job.
- **When.** Watch first: the governor reads every wire and writes nothing until the wire check passes and paired runs on the operator's own system show the receipt. Then one level at a time (the eight levels, chapter 7). It reads and decides once a second, all the time; staying native is a live decision, made again every second, never a setting left on.
- **How.** The plug snapshots each setting once, at the start; the governor reads every gauge, places each muscle in its band, tries one step on one muscle at a time inside a body, keeps it only if the paired trial shows a GAIN and no LOSS, and hands every setting back to the snapshot on reset, on the lease after a crash, or at once on the kill switch. The proof is the receipt: native and Omni-Compass on the same work, every gauge, three runs, every row shown.

**What the end result is.** A system fully hooked up runs native exactly as before, with one governor reading every gauge of every muscle once a second and moving, muscle by muscle, only where a paired trial has shown a GAIN with no LOSS anywhere in the body; where it cannot help, the muscle stays native, which is STEADY, not a failure. The intended outcome, stated as the claim the receipts test: more work, faster, with fewer misses, on the same or fewer machines, with every unit of power given shown beside what it bought. Whether a given system gets there is answered only by its own receipts.

# Disclosures, Declarations and Disclaimers

This page is the one place every declaration about Omni-Compass is made. The README, the manual, the book, every guide and every report point here. Where any other page seems to say more than this page, this page governs.

## 1. Rights

1. Omni-Compass, its engine, its mathematics, its software and its documentation are the property of The Omni-Compass LLC. All patents, copyrights and trademarks covering them have been filed in the United

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

States by The Omni-Compass LLC. No filing number is stated here, and no grant, registration or approval is claimed.

2. The software is licensed for evaluation and simulation only (`LICENSE`). Running it in production, on any system beyond evaluation, or in any product or service, requires a signed, paid Omni-Compass Enterprise License.
3. Names of other companies and products (NVIDIA, Kubernetes, Red Hat OpenShift, Amazon EKS, Google GKE, Microsoft AKS, Karpenter, Lambda and others) are the property of their owners. They identify the systems Omni-Compass was tested with or connects to. No affiliation, endorsement or certification by any of them is stated or implied.
4. All rights reserved. The Omni-Compass LLC reserves the right to change the software, the documentation, the results and the terms at any time, without notice. Every copy, fork, export, report, archive or printout of any part of Omni-Compass carries the README's notice, `LICENSE`, `NOTICE` and this page unchanged. The Omni-Compass LLC, www.omni-compass.com

## 2. What a result is, and what it is not

1. Every result carries its evidence class: **T** a theorem, **V** verified in code, **S** our own models (modelled), **L** live software (real Kubernetes on GitHub's machines), **P** a physical meter (a real card's own power meter). A model is a statement about the model. A software benchmark is a statement about the software and machine it ran on. Only a meter speaks for hardware.
2. The models were written by the same people who wrote the law. They show how the mechanism behaves; they are not an independent test.
3. On Kubernetes in kind every machine stays powered, so energy there is a declared model, not a meter; on the data stores it is the host's processor seconds. Neither is scored in the Omni index: energy is scored only where a meter read it (the card's own meter), and every benefit is claimed at the software end (the founder's order of 31 October, evening).
4. Every comparison is paired: the same seed, load and clock with and without Omni-Compass, in an order rotated by repetition. Intervals are 95% intervals over the paired repetitions. Ratios across seeds are summarised as geometric means. Labels are assigned by code from rules written and committed before the run (`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`), never by hand.
5. Losses and failed runs are kept beside the wins (`docs/EVIDENCE_LEDGER.md`, `results/live/`, git history). The first real-card run, for example, read GAIN on its energy (the work per energy up 3.6%, its interval clear of zero) and LOSS on its slowest answers (the 95th percentile 58.5% slower); it is published as it happened (`results/gpu/run-20261002T082232Z/GPU_REPS.md`, `docs/EVIDENCE_LEDGER.md`).
6. No result is a promise of a saving on any other system. No dollar figure in this repository is a measured result. Nothing here is a vendor certification, a safety certification, legal advice or investment advice.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 3. Do no harm, and the rule Omni-Compass keeps on every muscle

**What "do no harm" means** (the founder's words of 19 October 2026; this is its meaning everywhere in Omni-Compass, and it has no other). Omni-Compass has one goal: to strengthen the organism it governs and keep it healthy, working at the best state its own muscles can carry. Whatever is fed into the organism (its work, its data, its energy: its nutrients) passes through its muscles and comes out as its output. Omni-Compass's brain drives the organism toward the most and the best output it can make from what it is fed (the most work, done fastest, with the fewest misses and failures) while it keeps every muscle healthy.

- **Harm is judged at the organism.** Harm is the organism turning out less than its native controllers make it turn out (less work done; more time waited, missed or failed), or the organism made less healthy: a muscle pushed past its own limit, a service pushed over its own line, or anything made to fail. Omni-Compass does neither.
- **Every muscle only as far as it can go.** Omni-Compass can push a muscle only as far as that muscle itself can be pushed: its own limit, read live from the system, never past it. It finds the room each muscle already has and uses all of it. Whether its mathematics can draw a little more out of a muscle than the muscle was built to give is an open question; a measurement answers it before anything is claimed.
- **The muscles work as one body.** Where one muscle's strength cannot be used without straining another, the brain offsets and engages the muscles together to bring the whole organism to its best state. It may use one muscle less, leave it native, or rest it, so that another can carry more, whenever the organism's whole output comes out ahead and every muscle stays healthy. Which muscles to drive and which to leave native is the brain's live decision, at every moment and in every run.
- **Reactive and proactive.** Like an immune system, the brain answers a breach at once, and it acts before a line is crossed.
- **Every row is still shown.** Each muscle's and each gauge's reading stays in every table beside the organism's, LOSS included. Nothing is cut.

**How the engine that stands keeps it.** The engine that stands keeps do no harm in its strictest form. By default it moves each muscle's settings toward more work, faster, up to the muscle's own limit and never past it (the max); on the dial anyone can tune it down toward the same work with less (the tune-down). Either way it keeps a step only where the muscle is never behind native for it: a step that costs the muscle more for each piece of work, or loses more than **2%** in any measure (work done, response time at the median, p95 or p99, time over the service line, or the machine's own cost of running Omni-Compass), is undone. A resource spent that rose is **power given**, shown beside the work, the speed and the wear it bought, never called a loss. Where no move passes that test, Omni-Compass leaves the muscle exactly as it runs alone. It takes nothing from one muscle to give to another: that form never harms the organism, but it leaves unused any gain that needs one muscle to give way to another. Letting one muscle give way so that the organism's whole output comes out ahead, judged on the organism and on every muscle's health, is the next engine version's work. It is built and proven in our own models first (the model gate), every benchmark is run again on it, and no result claims it before its runs exist.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**The factory tune: one tuned engine, and a tuning bench.** The engine in this repository is presented as The Omni-Compass LLC tuned and ships it: self-running, set at the factory. Every muscle in the catalog is wired and runs on the setting found for it (its own card where one was found on development seeds and kept under the strict rule, 364 muscles in `realms/muscle_cards_v5.csv`; the engine's own setting for every other), frozen and fingerprinted by version (`python3 tools/omni_version.py`); no setting is presented as universal. **Every published benchmark result is one engine at its one factory tune**: the engine its table names (`python3 tools/omni_version.py`), run again on each engine that stands. From omni-v17 that tune is held to a declared rule (`docs/THE_TUNE.md`, `tools/tune_study.py`): of the tunes that harm no gauge against native, the one against which no other tune tried reads GAIN on one service gauge and LOSS on none, searched on development seeds and read again on untouched ones; the claim is non-dominance among the tunes searched, never a global optimum. The live benchmark engine page and the control panel are a live tuning engine: every knob, button and monitor turns in real time (the beat, the dial, the pedal, each muscle driven or left native, up or down, in or out, minus or plus) to show the mechanism of action with native alone, with Omni-Compass on top, or both side by side; what it shows at any other setting is the engine's range, never a benchmark result. As on any engine, where the mixture, the intake, the exhaust, the firing or the pistons change what it delivers, **turning any knob of Omni-Compass can change every metric, up or down, in any direction a tuner chooses**: one engine for many vessels, tuned many ways, for many outputs. A licensee may add on, retune, upgrade or tune down any muscle or combination (the dial, the cards, the bands and the service lines are exposed); a changed tune is the licensee's own, and so are its results. The factory tune is published only where it reads no LOSS: a LOSS found in The Omni-Compass LLC's own runs is treated as a fault in the mechanism or its wiring, found and fixed before the result is published, never as a number to live with (the founder's order of 23 October 2026). The one place a LOSS is meant to be seen is the live benchmark engine, when a person turns its knobs away from the factory tune (the dial, the heartbeat, the sizes, the load) to watch the system react and the brain hold or recover: that showing is the proof that the factory tune is the best state found, and that a turned knob never breaks the engine: it changes its mechanism of action, how its muscles and the organism share the work, toward the purpose the new setting asks for, and a changed tune's readings are the licensee's own.

**The factory profile, and the profile that comes next.** The factory tune is one complete configuration profile, the factory profile: every governing rule, gain, margin, decision interval and default setting shipped together, versioned and fingerprinted. Today's is omni-v19's. The best validated factory profile for the declared operating conditions is not yet established: it is chosen only after every mechanism is verified, every knob combination is tested together on development workloads, and the choice is validated on fresh ones; then it is frozen, carried into every implementation (the C++ harness included) and released under its own identity. Until then no other profile is presented as the baseline; on the live benchmark engine a candidate is evaluated, never released, and a custom profile's readings are its own. The words, the mathematics and the bill's arithmetic are written once in `docs/FACTORY_PROFILE.md`.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 4. The wiring declaration

**THE PUBLISHED BENCHMARKS ARE THE RECORD OF WHAT OMNI-COMPASS DID ON THE SYSTEMS, LOADS AND ENGINE THEY NAME, EACH WITH ITS EVIDENCE CLASS. WHERE A RESULT ON ANY OTHER SYSTEM DIFFERS FROM THEM, OR ANYTHING LOOKS WRONG, THE FIRST PRESUMPTION IS THE WIRING, THE SETTINGS, OR HOW OMNI-COMPASS WAS IMPLEMENTED OR READ, NOT THE LAW. CHECK THOSE FIRST (THE MANUAL'S "WIRED RIGHT OR WIRED WRONG") BEFORE DRAWING ANY CONCLUSION ABOUT OMNI-COMPASS.**

**Omni-Compass acts only through the wires it is given.** It reads the meters it is pointed at and moves the settings it is allowed to move. If a reading is the wrong one, slow, or blind; if a lever is the wrong one, already owned by another controller, or given a range (cover) that is wrong for the machine; if a band or a service line is set for a different workload; then Omni-Compass will do exactly what its law says with the wrong information, and the result will not be the published one.

Therefore:

1. **It cannot be slapped on.** Every installation goes through the levels of the manual in order (chapter 6), starting read-only, and every writing level begins with the wire check (manual, section 5.4) and a watch arm that must equal native.
2. **If your paired receipts differ from the published benchmarks in direction** (service LOSS, or no saving where one was shown), **the first presumption is wiring, not the law.** Confirm the installation with the checks in the manual's section 5.5 (*Wired right or wired wrong*) before drawing any conclusion about Omni-Compass.
3. **This has happened to us, and is published.** On the first real card (NVIDIA A10, 2026-10-02) the governor read being busy as trouble, set its power lid under the card's own working draw and allowed the clock under the card's own working clock; the card served bursts more slowly and the slowest answers were 58.5% slower. The cause was found in the card's own samples and corrected (`docs/GPU_PREREGISTRATION.md`, amendments 6 and 7). That is what a wiring fault looks like, and how it is found.
4. Omni-Compass never writes outside a lever's cover, never fights another writer (it stops and leaves that lever alone), reads back every write, and returns every lever to the value it read before its first write when it stops or when the OFF switch is used. These protect the machine; they do not make a wrong wiring right.

## 5. Safety and responsibility

1. Run watch mode first; run the wire check before any write; keep the master OFF switch (`python3 tools/omni_switch.py off`, the whole harness at once) in reach at every level, and use it at the first sign of anything wrong, a suspected breach included.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

2. Omni-Compass is a supervisory governor. The machines' own controls (firmware, autoscalers, safety systems) stay in place and keep their own protections; Omni-Compass sets only values they already accept.
3. Do not connect Omni-Compass to safety-critical systems (vehicles, medical devices, grid protection, life safety) except as a modelled study, and never without the qualified review those systems require.
4. The software is provided "as is", without warranty of any kind, as stated in `LICENSE`. The operator is responsible for its use on their systems.

## 6. Changes to the work and to this page

1. The software, the law's settings, the benchmarks, the results, the documents, the licensing terms and this page may change at any time, without notice. The version in the repository's `main` branch at a given commit is the version that commit describes; `RELEASE_MANIFEST.json` and `python3 verify.py` identify it.
2. Statements about work in progress, planned features, expected results and future runs are expectations, not promises. Where a result has not yet been measured, the documents say so (`docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, section 8).
3. A signed Omni-Compass Enterprise License governs its own terms for its own term.

## 7. Privacy and data

Omni-Compass contains no telemetry. It sends nothing to The Omni-Compass LLC or anyone else, and makes no network connection of its own except to the systems an operator points it at. Its logs and receipts stay on the operator's machines.

## 8. Where everything is

| Question | Page |
|---|---|
| What has been measured, and how strongly | `docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, `docs/EVIDENCE_LEDGER.md` |
| The rules written before each run | `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md` |
| How to wire it, level by level, and how to confirm it is wired right | `docs/OMNI_COMPASS_MANUAL.md` (chapters 4 and 6), the book `docs/OMNI_COMPASS_MANUAL.pdf` |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Question | Page |
|---|---|
| The license, in plain answers, and third-party components | `LICENSE`, `LICENSING_FAQ.md`, `ENTERPRISE_LICENSE.md`, `NOTICE`, `THIRD_PARTY_NOTICES.md`, `PATENTS.md`, `TRADEMARKS.md`, `SECURITY.md`, `.github/CLA.md` |
| The muscles, what each is for and how it is wired | `docs/MUSCLE_CATALOG.md` |
| Proof the code is the code that ran | `python3 verify.py`, `RELEASE_MANIFEST.json`, `results/SEAL.json` |
| The live benchmark engine: every monitor, its feed and its knobs; the third manual, how to read and operate it | `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`, `docs/engine/monitors.json`, `docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md` and `.pdf` (the Live Benchmark Engine Manual's Appendix L, the page word for word: `docs/engine/PAGE_WORD_FOR_WORD.md`) |
| The whole organism, composed every way it is bought | `results/WHOLE_SYSTEM.md`, `tools/whole_system.py`, `docs/engine/export_whole.py`, `docs/WHOLE_SYSTEM_PREREGISTRATION.md` |
| The Python's runs that the live benchmark engine runs again | `tools/engine_runs.py`, `results/engine_runs/`, `results/heldout_seed_*/` |

## 9. The whole organism, the cloud's back end, and the live benchmark engine

**The composition is a model of measurements.** `results/WHOLE_SYSTEM.md` (`tools/whole_system.py`) composes each layer's own measured ratio, native against Omni-Compass on top of native, into one benefit for a whole stack: the product law $R(g) = \prod_\ell R_\ell(g)$ gauge by gauge, its logarithm a sum with the variances added, each layer's runs A, B and C pooled by the inverse of their variances, and one index over the gauges with its 95% interval under two bounds (the gauges independent; perfectly correlated). No run here has stacked the layers: each factor is one layer's own run with the layers under it native, and the product assumes a layer's ratio does not change when the layers around it also carry Omni-Compass. That assumption is tested where the tables can test it (the governors' four ways), and the stacked run that replaces the composition is preregistered in

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`docs/WHOLE_SYSTEM_PREREGISTRATION.md`. Until it runs, the composition is class S of class P and L measurements, and every page that shows it says so.

**The two places a system runs.** The same law is read in the two places a system runs Omni-Compass (the founder's order of 1 November 2026; `docs/engine/export_whole.py`): the software end carried to bare metal (the software on the owner's own machines and chips), and the software end carried to the cloud (the cloud's build, its bare metal at the back end and the software; and the cloud as two pieces, the back end left to the provider). A piece with no measurement in the composition enters as exactly 1 and is named; neither place's composition is that place's bill. The bills themselves, and the work done inside them, native and Omni-Compass on native (the four arms: A and B on bare metal, C and D through the cloud), are read in `results/TWO_BILLS.md`.

**The cloud's back end is not ours to meter.** Through the cloud, the bare metal belongs to the provider: no tenant can read its power meters or write its chips' limits, and no run here reaches it. The cloud piece's benefit is what the provider itself meters and bills, the machines counted every fifteen seconds and the bill at list price, read native against Omni-Compass on top of the cloud's own autoscaler. The bare-metal piece is gauged on hardware we can meter (a card's own power meter; a processor's energy counter) and is carried to the provider's bare metal only as the same mechanism on the same kind of part, a modelled factor, never as a measurement of the provider's machines.

**The cloud is a wire.** A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center). Omni-Compass sits on all three pieces where a system lets it (the software end in either place, the chip only on bare metal); a system itself runs in one of two places, bare metal or the cloud (the founder's order of 1 November); the wire makes no benefit of its own. Until 31 October every cloud run here was set on the wire alone (the tune-down on the machines only), so it could at most park machines the cloud already made and cut the wire's bill a little. From 31 October (the founder's order of that day: no hard rule) every cloud set runs the whole engine at the max on the cloud's cluster, and what its receipt must show is that the engine holds there (section 10): no judged gauge LOSS against native, the work per dollar read beside the bill. The benefit is shown where it lives, at the software end, and explained through the wire, never claimed for the wire itself. The bare metal's energy and throughput are not under a tenant; they are measured where Omni-Compass sits on them (the card and the machine) and composed with the wire by the law above. A provider running Omni-Compass on its own back end would see that benefit on its own chips and racks; no tenant can meter it, so it is never claimed here as a measurement. No place carries a figure before its runs read it (the founder's order of 2 November: no number on any page that no table carries), and his word of 31 October sets the wire's own benefit at none: the wire carries the benefit, it does not make it.

**What the record shows in each place so far** (the words for each place, and what each benchmark must show, are in section 10): at the software end the benefit, the Omni index over work, speed and machines (`results/OMNI_INDEX.md`; composed layer by layer in `results/WHOLE_SYSTEM.md`), with no bill in it; on bare metal, the A10's own bill and meter on omni-v23 (`results/TWO_BILLS.md`,

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`results/gpu/V23_A10_CAP_FULL_ABC.md`), fully loaded: the bill STEADY, the work done inside it the runs disagree (up in two runs, down in the third, that run's lost work on the next version's list), the chip's energy for each request served GAIN in all three of our runs (the A100's three runs on omni-v21 read work and energy STEADY, `results/gpu/V21_A100_CAP_ABC.md`); through the cloud, Azure's steady set on omni-v18, 97 paired repetitions (`results/live/AKS_STEADY_100_OMNI_V18.md`) reads the bill up 1.2%, inside the noise, power given; the 99th percentile GAIN, down 11.0%; and failed requests up 3.9% by the mean (4.758% to 4.945% of requests), its interval across zero: a LOSS by the 0.5% key, inside the noise, on the next version's list. No gauge moved against native beyond the noise; by the key the set does not yet read that it holds. What the benefit cost the chip, the energy and the wire is listed row by row in section 3b of `results/WHOLE_SYSTEM.md`. In the cloud the yardstick is the work per dollar: the work answered within the line divided by the bill, against native; a bill that rises is power given beside the work it bought, and the work must rise more.

**The four arms** (the founder's order of 1 November): every place is read by its bill and the work done inside it, native against Omni-Compass on native: **A** and **B** on bare metal (the chip), **C** and **D** through the cloud; the seven organisms run both ways in each place, fourteen runs a place, every organism at every size in `results/TWO_BILLS.md`, a place not yet reported naming the runs that will fill it. On omni-v23 so far the A10's compute run at the usual load reads every gauge STEADY (`results/gpu/V23_A10_FAST_ABC.md`); Google Cloud and Amazon Web Services are not yet run (their keys are not in the repository's secrets).

**The organisms.** The seven organisms of the catalog (the spine, the four realms, the four stacked, the tower), the orchestra beside them, the live benchmark engine's own sixty-muscle organism, and the whole organism in its three pieces (the software end, the cloud's wire, the bare metal) composed above: ten organisms. Each is read two ways, native and Omni-Compass on top of native (Omni-Compass is never by itself, the founder's order of 25 October): twenty scenarios, recorded in `results/scenarios/SCENARIOS_20.json` (class S, our own models, three runs), every cell with no run reading "not yet run". None is a product.

**The live benchmark engine.** `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html` is a monitor of the repository's records and of the engine's own mathematics running in the page, and the live tuning bench: a setting other than the factory profile shows a variation of the mechanism, never a result. Every monitor carries its evidence class (P a physical meter, L live software, S our own models) and its state: live (the engine's port, proven equal to the Python and the C++ on every build by `docs/engine/check_live_engine.py`), a replay of a recorded run from its own files, a reading of the repository's tables, or built and awaiting its run, with no invented number and the run that fills it named. The kill switch on the page acts on what runs in the page, and a recorded result changes its colour, never a value, a word or a row. Pointed at an owner's own Prometheus, the page only reads it: it writes to no system.

**The three manuals.** The Operator's Manual wires and runs Omni-Compass, the Owner's holds the whole mechanism and every result, the Live Benchmark Engine Manual reads the page; each carries this notice, the `LICENSE`, the `NOTICE` and this page.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 10. Where Omni-Compass sits, and what each benchmark shows

Omni-Compass is software. It always sits on top of a native controller. A system runs it in one of two places, on bare metal or through the cloud, and in each place there are two arms, native and Omni-Compass on top of native: four arms, each read by its bill and the work done inside it. These are the words for them, used the same way on every page.

| Place | Its benchmark | What it must show | Its readings |
|---|---|---|---|
| The software end | Kubernetes and the data stores on GitHub's machines (kind), class L | the benefit, by the one key, and that it holds | `docs/BENEFIT_SHEET.md`, `results/OMNI_INDEX.md` |
| Bare metal, the chip | a graphics card on Lambda run directly, its rental the bill and its own power meter beside it, class P | that it holds, and the work done inside its bill, A against B: the work per dollar and per joule | `results/TWO_BILLS.md`, `results/gpu/` |
| Through the cloud | Kubernetes through Azure's AKS, the machines it bills, class L | that it holds, and the work done inside its bill, C against D: the work per dollar | `results/TWO_BILLS.md` and each set's `results/live/AKS_*.md` |
| The whole organism | the places composed by the product law, class S of P and L | how the places add up, an explanation and never a measurement | `results/WHOLE_SYSTEM.md` |

- **The software end.** The programs that carry the work: Kubernetes with its pod and cluster autoscalers, its schedulers and the orchestrators on it, the data stores (PostgreSQL, MySQL, Redis, MongoDB, Kafka) and the services they serve. Omni-Compass sits on top of their native controllers. This is where its benefit lives and where it is measured: real software on real computers (class L), on GitHub's own machines in kind clusters. Those machines are rented runners, never called bare metal and never called the chip.
- **Bare metal.** The first of the two places a system runs (the founder's order of 1 November 2026): the owner's own machines, or a chip run directly, with the chip's own controllers under the software. Its bill is the machine's own: the hours it runs, and the energy the chip's own meter reads. The software end goes with Omni-Compass to it, and there Omni-Compass can reach the chip's own knobs too.
- **Through the cloud.** The same software end on a cloud provider's rented machines: Kubernetes through the cloud (Azure's AKS today; Google's GKE and Amazon's EKS wired the same way, not yet run), the machines billed by the hour at list price and added or removed by the cloud's own autoscaler. Its bill is the cloud's own; the cloud's chips and back end are inside its price and out of Omni-Compass's reach: it governs what the cloud gives, through the cloud's own Kubernetes. The cloud is the wire between the software end and the provider's bare metal. The wire makes no benefit of its own, and no tenant can meter the provider's chips.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **The chip.** Bare metal's own part: the processor or graphics card under the software, with its own controllers (the power limit, the clocks, the heat), measured with the card's own power meter (class P) where Omni-Compass sets the card's knobs through the card's published interface. Omni-Compass built into the silicon, or into a battery's own controller, is not something any run here can test, and it is not claimed.
- **The four arms.** Two places, two arms in each: A, native on bare metal; B, Omni-Compass on top of native on bare metal; C, native through the cloud; D, Omni-Compass on top of native through the cloud. Each is read by its bill and the work done inside it, each place on its own, since a system runs in one or the other; an owner who runs work in both may weigh them only by its own spend, through G(total) (`results/TWO_BILLS.md`, `docs/WORK_PER_BILL.md`). The software end's tests are one set read three ways (the Omni index on work, speed and machines; the same on work and speed alone; the layers composed): they never add together, and none of them is either place's bill.
- **Hold.** Omni-Compass on top of native harms nothing in that place: every judged gauge (the work done, the response time, the minutes over the line, the pods waiting, the failures) reads STEADY or GAIN against native by the one key, and a resource that rose (the bill, the machine-hours, the energy) is read beside it as power given. A place holds when its runs read no LOSS.
- **Where the benefit lives.** At the software end, where it is made and measured, with no bill in it. It shows through on the two bills, native against Omni-Compass on native: on bare metal the work done inside the machine's own bill (each dollar, and each joule of the chip's own meter), and through the cloud the work done inside the cloud's own bill (each dollar). The chip and the wire themselves add no gain of their own: they are where Omni-Compass must hold. A benchmark that holds is the proof of no harm in its place; the benefit is shown where it lives and explained for the rest.
- **Work for the bill.** The one number every receipt is read for: the work done for each dollar of the bill, $E = W / B$ (written once in `results/TWO_BILLS.md`), in each place native against Omni-Compass on native, $G = E(\text{Omni-Compass on native}) / E(\text{native}) - 1 = (1 + \Delta W) / (1 + \Delta B) - 1 \approx \Delta W - \Delta B$, $\Delta W$ the change in the work done and $\Delta B$ the change in the bill. A bill that rises is power given; there is more work for the bill when the work rises more than the bill ($\Delta W > \Delta B$, $G > 0$). Both arms pay the same prices, so a place's price level cancels in its G. A system that runs in one place reads that place's G alone; an owner who runs work in both weighs them by its own native spend $S(p)$, $G(\text{total}) = \Sigma_p S(p)(1 + \Delta W(p)) / \Sigma_p S(p)(1 + \Delta B(p)) - 1$ (`docs/WORK_PER_BILL.md`).
- **One mechanism, in one place, felt along the whole path.** Omni-Compass acts in one place, the software end, on top of the controllers a stack already runs; it changes nothing in the chip and nothing in the network or the cloud. Its effect travels the whole path the work travels: a request starts in the software, crosses the wire (the owner's own racks or a cloud's machines), is computed on the chip and comes back. So the effect is checked at every place: the software end shows the gain (more work, faster, the machines it took read beside it, a rise power given), and the wire and the chip are read for whether it holds and carry the gain to their bills. Two places are tested because a system runs in only those two; one mechanism makes the gain, the way a city's traffic signals make more trips an hour on the same roads and the same cars.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Never in the data path.** Omni-Compass never touches the data a system serves: no request, packet, file or byte of the work passes through it, and it never opens, reorders, compresses or caches that data. It reads the system's gauges (one wire in) and writes the settings the native controllers already accept (one wire out). A description that puts it in front of the data, as a pipeline, a packet scheduler or a compressor, is not a description of Omni-Compass. Where it sits among the industry's answers at the chip, the cloud and the software is on `docs/INDUSTRY_LANDSCAPE.md`; no figure there or anywhere is Omni-Compass's unless one of this repository's tables carries it.

# Quick Reference: The Card for the Glove Box

Keep this page where the people who run the system can reach it, the way a driver keeps the card in the glove box. Every line is explained in the section named beside it; nothing on this page replaces those sections.

**What it is.** A supervisory governor on top of the controllers you already run. It never replaces them: it moves only settings they already accept, inside a band you set, and hands every setting back when it stops (sections 1.1 and 5.1).

**The three arms.** *Native* is your system as it runs on its own. *Omni*, Omni-Compass on native, is the same system with Omni-Compass on top. *Omni-Compass direct* is the same system with each knob's native controller switched off and Omni-Compass the only controller moving it; never in a cloud without the owner's word. There is no other arm and no other name (section 1.1).

**The profile and the two restoring controls** (section 4.8). Start on the factory profile; every change makes a custom profile, your own. **Restore factory profile** reapplies the released configuration. **Return to native** is the reset: every setting handed back to the snapshot, read back, the governor stopped. The kill switch is separate, for security only, and instant.

**The modes** (section 1.3).

| Mode | What it does | When |
|---|---|---|
| Off (manual) | nothing; native runs alone | before installing, and whenever you choose |
| **Mode** | **What it does** | **When** |
| Watching | reads every gauge and logs what it would do; writes nothing | the first level of every installation (level 1) |
| **Mode** | **What it does** | **When** |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Mode | What it does | When |
|---|---|---|
| Autopilot | one pedal: push for more, lift off for less, idle (lifted all the way) and reset, by its law | after the watch level has passed |
| Cruise | machines in service up to the pool's own top while a known queue waits | batch work |

**One pedal**, like an electric car's. *Push*: capacity added at once as traffic climbs, never held back. *Lift off*: capacity eased off a step at a time, handed back toward native, never below idle. *Idle*: the pedal lifted all the way, the floor of two machines in service, ready for the next burst. *Reset*: every setting handed back to where native had it, read back, and the record removed.

**The ways it stops, and what each leaves behind** (chapter 9).

| You want to, or it happens that | Do this | What is left |
|---|---|---|
| a run ends | nothing: the reset runs by itself at the end of every run | every setting at the operator's value, read back |
| stop everything at once, for any security reason | `python3 tools/omni_switch.py off` | every governor hands back and exits; none starts again until `python3 tools/omni_switch.py on` |
| stop one muscle | its kill file, or `OMNI_KILL=1` | that muscle handed back; the others run on |
| Omni-Compass dies without handing back (killed, crashed, hung) | nothing: the watchdog's lease hands back for it | every setting back at the operator's value, read back, as soon as the watchdog finds the process gone or its lease run out (chapter 9) |

**The eight levels of an installation** (section 7.3). Take them in order; go to the next only when the pass holds.

| Level | What it touches | Pass |
|---|---|---|
| 0 | nothing: `python3 verify.py` | `VERIFICATION: PASS` |
| 1 | nothing: watch mode, with a read-only identity | zero writes, and readings your operators agree with |
| 2 | the pods, through your autoscalers (**WRITES**) | p95, p99 and failed requests never behind native, over paired runs (chapter 11) |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Level | What it touches | Pass |
|---|---|---|
| 3 | the machines, through the release gate and the verdict (**WRITES**) | fewer machines in service, with no LOSS on any service gauge |
| 4 | the replica decision itself (an option, not a step) | as level 3 |
| 5 | a GPU box, two wires: the clock ceiling and the power limit (**WRITES**) | work per energy up or equal; no request more than 2% slower at the median, p95 or p99 |
| 6 | the processor's clock and power on bare metal (**WRITES**) | energy per unit of work down, no LOSS on any service gauge |
| 7 | site power and the supply-air setpoint (**WRITES**) | the site meter's energy down, every rack inlet inside its band, no LOSS on any service gauge (section 8.10) |

**Before any level that writes:** run the wire check (section 5.4), start in watch, and keep the master switch in reach.

**Warning signs, and the first thing to do.**

| You see | First | Then read |
|---|---|---|
| a service slower or failing more with Omni than without | `python3 tools/omni_switch.py off`: the system is native again | section 5.5, wired right or wired wrong |
| a setting at a value Omni-Compass did not write | nothing: it stops writing that lever by itself (one writer) | section 6.2 |
| a meter that is blind or late | nothing: blind means hold; it moves nothing it cannot see | section 6.3 |
| your results point the other way from the published ones | presume the wiring first, not the law | `DISCLOSURES.md`, section 4 |

**Who to call.** Licensing, pilots and the Omni-Compass Enterprise License: The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com. Security reports: the process in `SECURITY.md`, never a public issue.

**What the letters of the number mean.** Read as one sentence: Omni-Compass takes the system in through a Data Exchange (DEX), runs every setting up to its Yield Limit Line (YLL) and never past it, and closes it all into one loop (1); science and software people read each part on sight. Read in order, DEX-YLL-1 is the way the engine runs:

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Letters | What they mean |
|---|---|
| DEX | **Data Exchange**: the input. In software, data exchange is how one system's readings reach another. Here it is the two-way plug, one wire in and one wire out: the system's own readings come in, Omni-Compass's settings go out. |
| YLL | **Yield Limit Line**: the boundary. In engineering science, the yield limit is the most load a material can carry and still spring back unharmed; past it, the damage is permanent. Omni-Compass runs every muscle up to its own yield limit line, read live, and never past it. |
| 1 | **1**: the output. In code, 1 is true, the finished state; in science, 1 is unity, the whole. Here it is one closed loop: one body, one brain, every result read back and the next step decided from it. |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART ONE

# The Governor, from the Outside

*What Omni-Compass is as an installer meets it, how the muscles of a stack become one body, and how it is tuned.*

CHAPTER 1

# What Omni-Compass Is

A governor on a steam engine does not build the engine and does not turn the shaft. It watches the speed and moves the throttle so the speed stays in a band. Omni-Compass is that, for every machine you run.

It is a process on a host. It reads meters, decides, and writes only the levers it has been given. It is not the chip, not the GPU driver, not Kubernetes, not the building controller. Those keep running exactly as they do today; Omni-Compass sets the values they already accept.

Three facts about that process fix what an installation can and cannot expect of it. First, it is **one process per stack**: a controller that wakes on a fixed period, reads the stack's own gauges through the stack's own interface, decides, writes through the same interface, and reads back what the device took. It has no agent inside the pods, no kernel module and no hook in the request path, so it can add latency to nothing it does not govern. Second, every lever it may touch is **named before it starts**, read once into a snapshot, and handed back to that snapshot when it stops, is killed, or loses a sense; a lever it was not given it cannot reach, because the plug for it does not exist. Third, it has a **watching mode** in which it reads and decides and writes nothing, and every rollout begins there (chapter 7): the first thing an operator learns about Omni-Compass on their own stack is what it would have done, in a log, with the stack untouched.

What follows from those three is the shape of every receipt in Part Five. Because the native controller keeps running underneath, every paired run is the native stack against the same native stack with Omni-Compass on top, and never Omni-Compass against anything else. Because the levers are the stack's own settings, every gain is one the operator could have reached by hand, if they had been able to read every gauge every second and move every knob without ever being wrong; the claim is not a new mechanism but a steady hand on the mechanisms that exist. And because the process is small and separate, its own cost (processor time on the host, reads of the API) is recorded in every run and counted against it where it shows.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

## 1.1 Always on top of a native controller

The first design decision, and the one every other follows from, is that Omni-Compass is **supervisory**. It never replaces the controller a system already has, never bypasses it, and never runs in its place. The Kubernetes Horizontal Pod Autoscaler keeps scaling pods on its own CPU target; Omni-Compass moves that target inside a range the operator allows. The cluster autoscaler keeps adding and deleting machines; Omni-Compass marks machines idle and lets the autoscaler do the deleting. The card's firmware keeps boosting the clock and enforcing its power limit; Omni-Compass sets the ceiling the boost may reach and the limit the firmware enforces. PgBouncer keeps handing connections to clients; Omni-Compass sets its pool size through the pooler's own console. Kafka's consumer group keeps rebalancing partitions on its own; Omni-Compass changes how many consumers are in the group, which is what an operator does. Redis keeps evicting by its own rule; Omni-Compass sets the ceiling that rule works under.

This fixes how an installation is judged: every paired run sets **omni** (Omni-Compass on native: the same system with Omni-Compass on top) against **native** (the system with its own controller, as shipped and as the operator set it), and there is no comparison in which Omni-Compass on native is a rival to that controller. A gain is therefore always "what the governor added on top of what the native controller already did", and a loss is always "what the governor cost on top of it".

Since the founder's order of 23 October one more arm is run beside these two, **Omni-Compass direct** (Omni-Compass by itself), so that a reader can see what Omni-Compass does as the only controller of a knob. The three are always shown together, and each pair is read on its own: omni against native, direct against native, direct against omni.

| Arm | What moves each knob | How an installation runs it |
|---|---|---|
| Native | the native controllers alone, as shipped and as the operator set them | the governor not installed, or OFF (chapter 9) |
| Omni-Compass on native (omni) | the native controllers, with Omni-Compass moving the settings they expose | the governor at the level granted (chapter 7) |
| The arm by itself (called Omni-Compass direct on 23 October) | On Kubernetes, Omni-Compass's own pod rule in place of the autoscaler's arithmetic, and nothing else: Omni-Compass is a brain with no organs there, so every organ of Kubernetes (the API server, the scheduler, the kubelet) and every muscle Omni-Compass cannot be stay, | `--own-replicas namespace/name` on a workload no autoscaler governs (Appendix K); on the kind rig `ARM=solo` (`scripts/kind_bench.sh`); chapter 11 |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Arm | What moves each knob | How an installation runs it |
|---|---|---|
| | native, under it; a knob's native controller is switched off only where Omni-Compass carries that muscle and has proven it can move the knob. The word direct overstates it on Kubernetes (the founder's analysis of 31 October): Omni-Compass can be direct to a knob there, never to a muscle. A mechanism test kept in the record, not a product arm (`docs/SOLO_PREREGISTRATION.md` amendment 2) | |

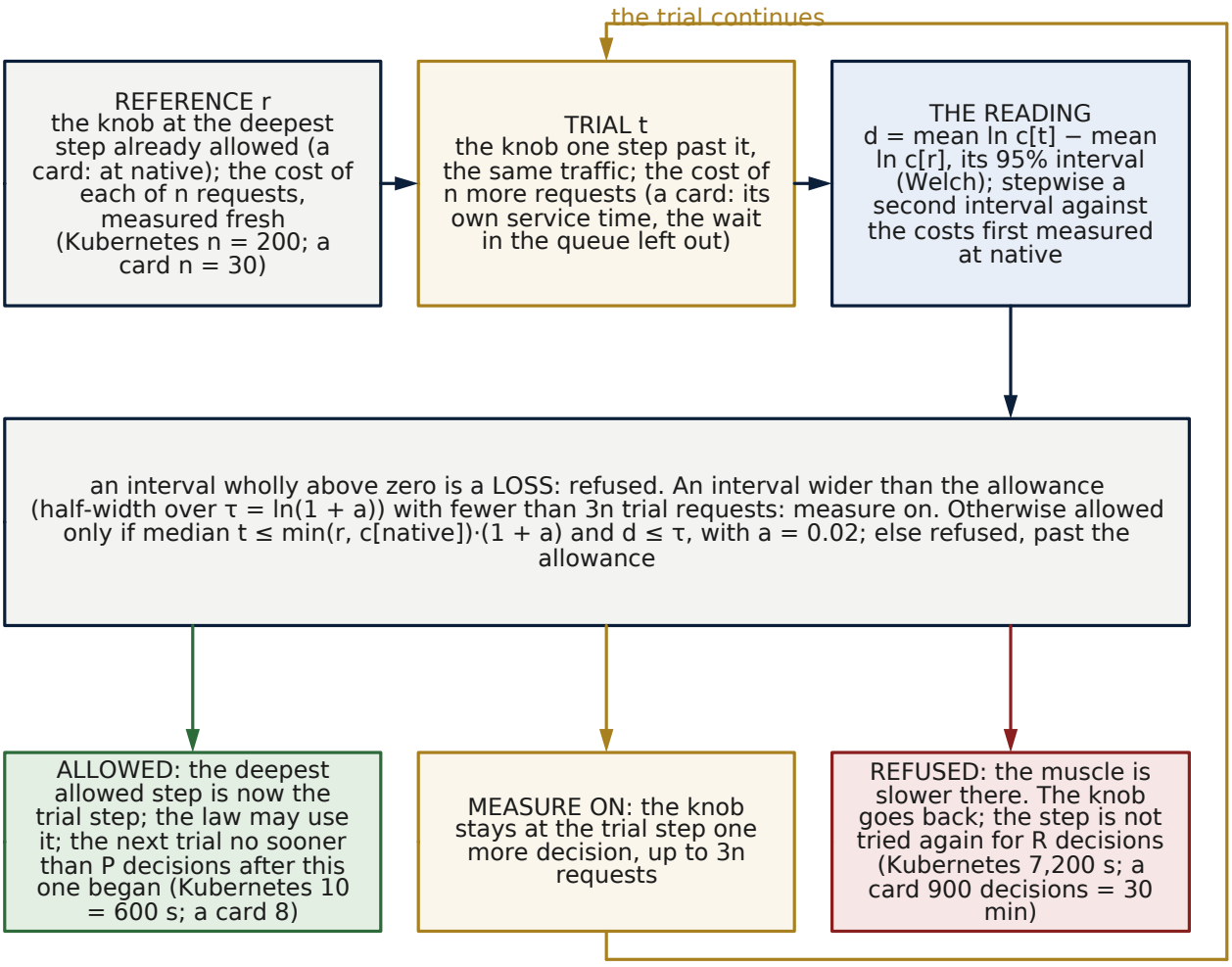
Where native is a setting and not a controller (a cache's ceiling, a buffer pool, a pooler's pool size, a consumer group's size), or where native's own loop is the actuator and cannot be switched off (the card's firmware, a servo, an autopilot, a tap changer), the two Omni-Compass arms are one arm, and a receipt says so in one line. No cloud runs direct: in a cloud Omni-Compass never adds a machine (chapter 20), and the direct arm there waits for the owner's word (`docs/SOLO_PREREGISTRATION.md`).

## 1.2 Do no harm: the paired trial on the muscle

The second design decision is do no harm, and its meaning is the organism's (`DISCLOSURES.md`, section 3): the organism is never left turning out less than its native controllers make it turn out, nor less healthy, and every muscle works only as far as its own limit. The engine that stands keeps it in its strictest form: Omni-Compass must never cause a LOSS on any muscle, judged on the muscle itself, and takes nothing from one muscle to give to another. Before Omni-Compass moves a slow knob (a machine given back, a card's clock ceiling, a robot axis's speed, a drone's cruise override), it runs a **paired trial on the muscle itself**: the muscle's own cost per piece of work is measured at the deepest step already allowed and at one step past it, under the same traffic, with a set number of pieces of work on each side (two hundred response times on Kubernetes; thirty requests on a card). The step is read as a receipt reads a row, by the 95% interval of the change in cost: an interval wholly above zero is a LOSS and the step is refused; an interval wider than the 2% allowance measures on, up to three times the number; otherwise the step is allowed only if the trial's median cost and its mean change are both no more than 2% above the reference and above the cost first measured at native. A refused step is not tried again for a set time (two hours, 7,200 s, on Kubernetes, 720 decisions at the factory's 10 s on site, 120 at its 60 s through a cloud; 900 decisions, thirty minutes, on a card). A trial that sees the service leave calm, or a phase that outlasts its limit (1,200 s on Kubernetes; 30 decisions on a card), is abandoned and the knob goes back, the step neither allowed nor refused. Where no step passes, the trial reads

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**left native**, the knob is never moved, and the receipt for that muscle says "nothing for Omni to move". Gains are not capped; the 2% allowance caps only what may be spent in the trial to look for them.




The verdict stands over every slow knob (the machines, the clock ceiling, a cache's notch). A trial begins only while the service is calm and nothing waits; a phase that sees the service leave calm, or that takes longer than its limit (Kubernetes 1,200 s; a card 30 decisions), is abandoned and the knob goes back, the step neither allowed nor refused. The knob never passes the deepest allowed step, so steps cannot add up past the allowance. On the max, the Kubernetes machine count never gives back a machine the cluster had on its own.

**Figure 1.** Block diagram: the verdict's paired trial around a slow knob, read GAIN, STEADY or LOSS. The reference r is the cost of n requests at the deepest step already allowed (on a card, at native; n = 200 on Kubernetes, 30 on a card); the trial t is the cost of n more one step past it under the same traffic. The reading is the 95% interval of d = mean ln c[t] − mean ln c[r], stepwise also against the costs first measured at native: an interval wholly above zero is a LOSS and the step is refused; an interval wider than τ = ln(1 + a) measures on, up to 3n; otherwise the step is allowed only if median t ≤ min(r, c[native])·(1 + a) and d ≤ τ, with a = 0.02.

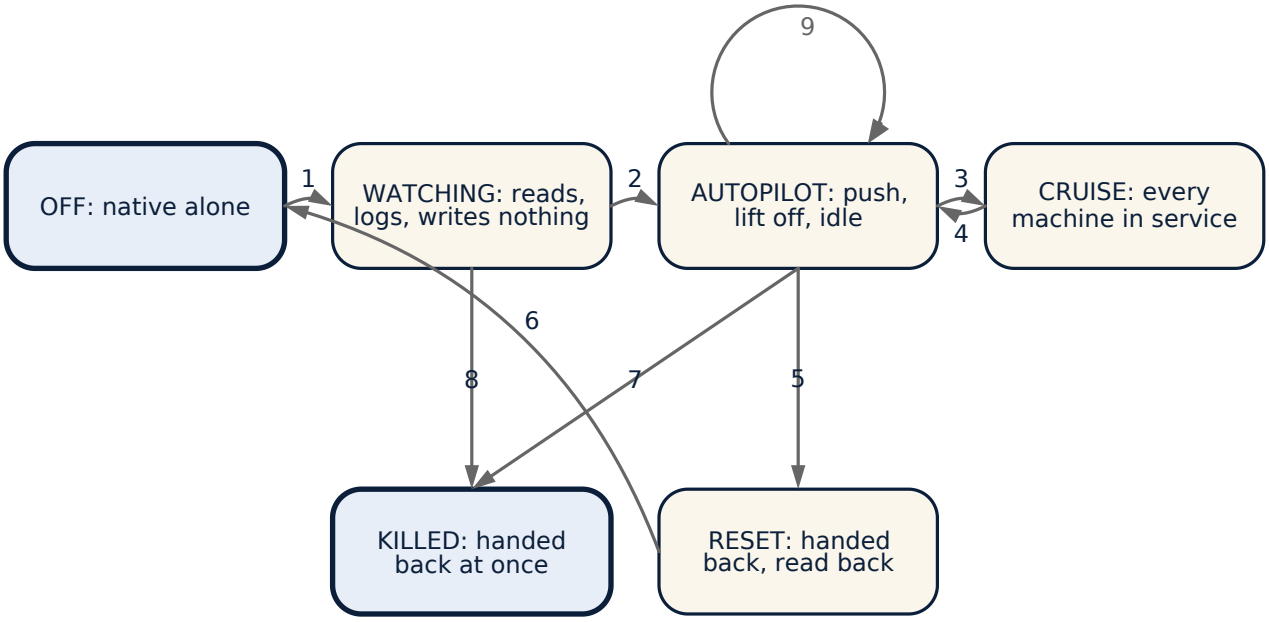
A refused step is not tried again for R (7,200 s on Kubernetes, 480 decisions at the 15 s heartbeat; 900 decisions on a card, 30 min). Steps cannot add up beyond the allowance, because every step is judged against the native cost too.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

For an installation this has two consequences. A knob the operator grants is not a knob Omni-Compass will move: it is a knob Omni-Compass may move once the muscle itself has shown the step costs it nothing. And a muscle that never passes a trial is not a fault: it is the right answer for that muscle on that stack, and it costs nothing.

## 1.3 The pedals, the modes and the two switches

Omni-Compass drives a system the way a self-driving car drives itself: it feels, gauges and adjusts, with nobody in the seat. The vocabulary is used the same way everywhere in this manual:




1 level 1 · 2 the wire check passed; level 2 and up · 3 pods waited 2 decisions · 4 the line empty 2 decisions · 5 the reset file: the end of a run · 6 exit 0 · 7 the kill switch, from any mode · 8 the kill switch · 9 idle at once when demand is at zero

**Figure 2.** State machine: the governor's modes. OFF is native alone; WATCHING reads and logs and writes nothing; AUTOPILOT pushes, lifts off and idles; CRUISE holds every machine in service while pods have waited two decisions; RESET hands every setting back and reads it back at the end of a run; KILLED hands everything back at once, from any mode, on the kill switch. The numbered transitions are read in the key under the diagram.

- **Off (manual).** Omni-Compass is not driving; native runs alone. This is the native arm of every paired run.
- **Watching.** Omni-Compass reads every gauge and logs the decision it would take, and writes nothing. Any difference between a watching arm and native is the cost of its being there (its processor time, its reads) plus noise, never a decision; a watching arm that writes anything fails its run. Watching is the first level of every rollout (chapter 7).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Autopilot.** One pedal: **push** (traffic climbs and capacity is added at once, never held back), **lift off** (traffic falls and capacity is eased off a step at a time, handed back toward native, never below idle), **idle** (the pedal lifted all the way: calm traffic, the engine at its floor of two machines in service, ready for the next burst), **reset** (every setting the governor ever wrote is handed back to where native had it, read back, and the record removed; every paired run's omni arm ends with a reset and checks it).
- **Cruise.** For work that comes as a pile: when pods have waited for a place for two decisions in a row, machines go into service up to the pool's own top (read live: every machine of a fixed pool at once, a growing pool a step a decision) and stay there, without second-guessing, until the line has been empty for two decisions. In cruise the governor also stops its five-second floor reads, because every machine is already in service and the reads would only cost processor time on the machines the queue is using.
- **Idle at once.** When the work is done and demand is at zero (the service at its autoscaler's own floor, wanting no more, at half its target or less), the pedal is lifted all the way in one move, straight to idle, with every safety check still holding.
- **The kill switch.** Separate from all of the above, and for **security only**: one switch in a human hand (`python3 tools/omni_switch.py off`) turns Omni-Compass's governing off across the whole machine at once, for anything rogue, or for anyone trying to drive a system through Omni-Compass. Every governor hands back and exits; none starts while the switch is off. The reset is the end of a run; the kill switch is the end of trust.

The floor of two machines is an operator's number; lifting off never takes a system below it, however far the pedal is lifted. There is no ceiling: every machine the operator has is usable, and the protection is the band inside each machine.

## 1.4 Nothing is hardwired

Omni-Compass carries no table of per-stack special cases, and an installation adds none. The few numbers the governor holds are the same on every muscle and are declared, not chosen per benchmark: the centre of the band, the 5% cushion at each edge, and the 2% do-no-harm allowance. Everything else a muscle needs (how fast it answers, how often it may be moved, how far it may go) is read from the muscle's own declaration in the catalog or from the native controller's own settings at the start of the run, never typed in by hand. Where a stack needs a line, a cover or a step size (the MySQL pool moves in the server's own 128 MB chunks, the Redis ceiling in notches), the number comes from the stack being governed and is written down before the first counted run, so a reader of the receipt can see it was not chosen after the result.

For an installer this means one thing: the settings an installation writes are the operator's own (chapter 4), and nothing else is to be set by hand. A behaviour seen in a receipt is the governor reading the wires and the settings it was given; where it looks wrong, the wires and the settings are the first place to look (section 5.5), because there is no hidden per-stack rule to blame.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

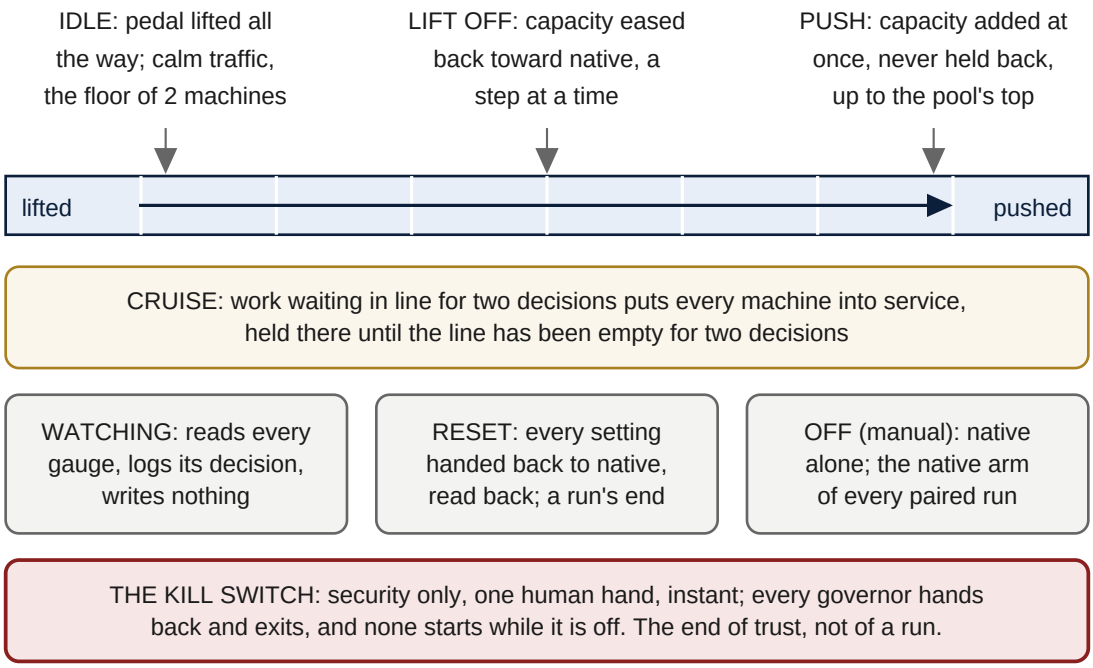



**Figure 3.** One pedal, no brake. Pushing the pedal adds capacity at once; lifting it eases capacity back toward native a step at a time, never below idle, the floor of two machines. Cruise holds every machine in service while a known queue waits. Watching reads and logs but writes nothing; reset hands every setting back to native and reads it back at the end of a run; off is native alone. The kill switch stands apart from all of these: for security only, in a human hand, instant. After a crash the lease walks every setting back to native on its own.

## 1.5 What it sets, muscle by muscle

Every muscle keeps its own control, and Omni-Compass sets what that control already accepts:

| Muscle | Its own control (kept) | What Omni-Compass sets |
|---|---|---|
| Kubernetes service | the HPA | the HPA's CPU target; replica floors |
| Kubernetes node pool | Cluster Autoscaler / Karpenter / MachineSet | how many machines stay in service (park and wake) |
| NVIDIA GPU | the card's firmware (boost, power and thermal limits) | the clock ceiling (up wire) and the power limit (down wire) |
| CPU | the kernel's frequency governor | the frequency ceiling |
| Data hall, building | the chiller and air handler loops | the supply-air setpoint; units in service |
| Battery site | the inverter | the reserve level; the peak ceiling |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Muscle | Its own control (kept) | What Omni-Compass sets |
|---|---|---|
| Robot joint, vehicle axis | the servo loop | speed and effort limits |
| Process loop, feeder | the PI controller | the setpoint inside its band |
| Database behind a pooler | PgBouncer's pool, the DBA's fixed setting | the pool size through the pooler's own console |
| Database's storage engine | MongoDB's WiredTiger cache, the operator's fixed setting | the cache size through the server's own console |
| Message broker | the consumer group at the operator's count | how many consumers are in the group |
| Cache | the operator's memory ceiling and eviction rule | the memory ceiling through the cache's own console |
| Substation | the tap changer's own controller | the tap position, one whole tap per move |
| Drone | the shipped autopilot | the cruise override inside the autopilot's limits |

When Omni-Compass stops, every one of those values goes back to what it was before Omni-Compass acted: not to the last value it wrote, but to the value it read once, before its first write.

CHAPTER 2

# Your Stack as One Body: Muscles and Organisms

## 2.1 The muscles you can wire

A muscle is anything with a meter and a setting: a compute pool and its machine count, a GPU and its clock ceiling, a chiller and its setpoint, a battery and its reserve, a feeder and its tap, a robot joint and its speed limit, a pump and its flow. Each muscle has a native controller of its own, which is what the muscle does when Omni-Compass is not there, and Omni-Compass wires to it through one wire in (the meter) and one wire out (the setting). The catalog (`realms/catalog_v5.csv`) lists **1,107 distinct muscles in 70 families**; Appendix E gives every one with what it is, what Omni-Compass reads from it, what it moves and the factory setting it runs on, and gives every family its wiring recipe (`docs/wiring_recipes.csv`): the native

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

controller, the wire out, the wire in, the restore, and whether the wire has run live in this repository or only on a model.

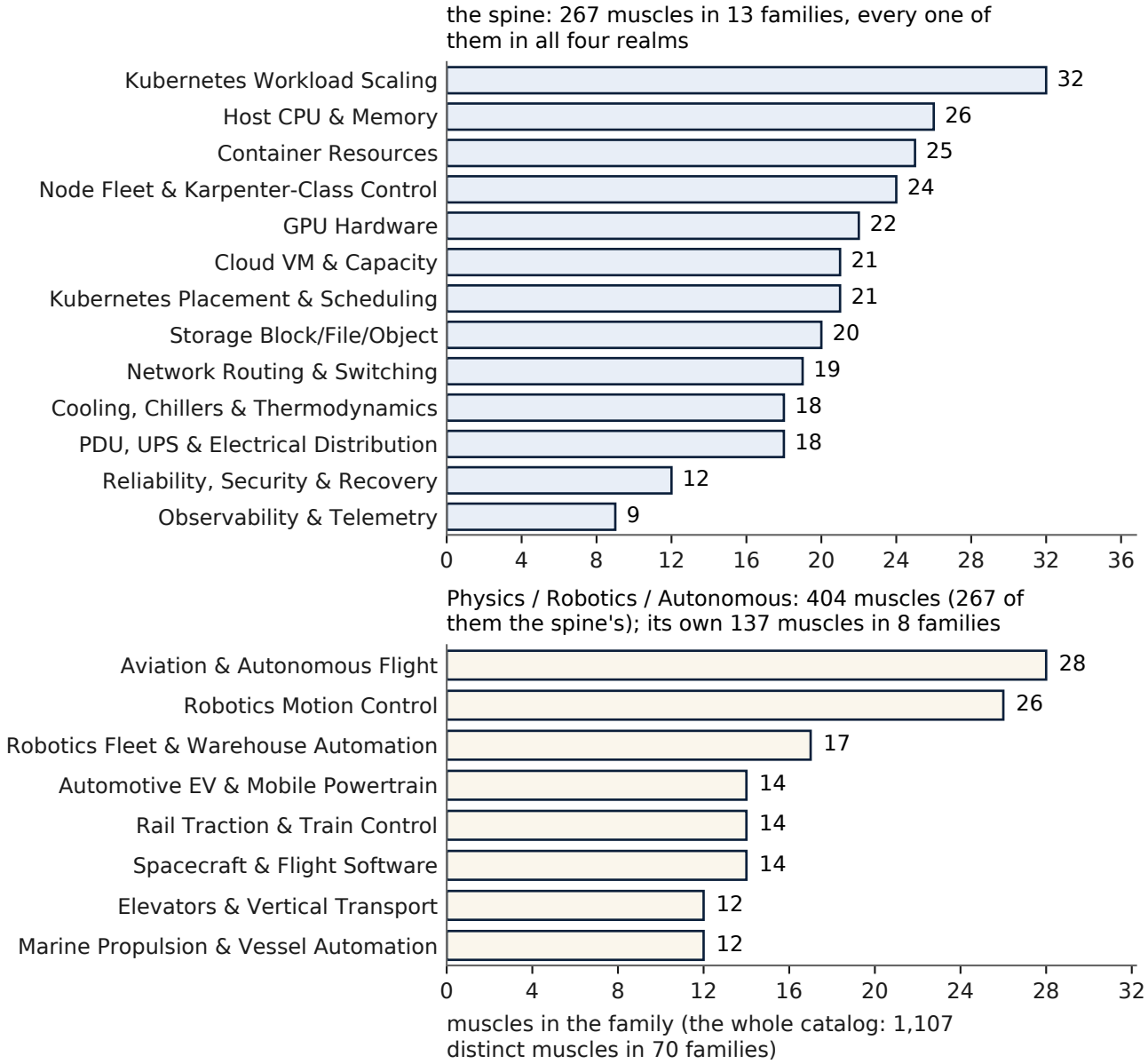



**Figure 4.** Grid: the families of the spine and of the Physics / Robotics / Autonomous realm, from realms/catalog_v5.csv. The spine's 267 muscles, every one of them in all four realms, in their 13 families; and the realm's own muscles, those it adds on the spine, in their families, by count.

The seventy families, by realm, are these. **Compute, AI and cloud** (twenty-seven): AI inference serving, AI training, blockchain and ledger nodes, CI/CD and build farms, cloud virtual machines and capacity, container resources, cross-cluster and edge placement, DPU and programmable IO, distributed cluster managers, game servers and real-time interaction, GPU fabric and RDMA, GPU hardware, HPC and distributed compute, host CPU and memory, Kubernetes dynamic device allocation, Kubernetes placement and scheduling, Kubernetes

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

workload scaling (the largest family, 32 muscles), Linux kernel and operating-system control, media streaming, node fleets and Karpenter-class control, OpenShift and the Machine API, quantum computing control, serverless functions, virtualization and hypervisors, work admission and demand shaping, and the body's gut (data-plane intake and waste) and skin (perimeter and admission). **Physics, robotics and autonomous** (eight): automotive EV and mobile powertrains, aviation and autonomous flight, elevators and vertical transport, marine propulsion and vessel automation, rail traction and train control, robotics fleets and warehouse automation, robotics motion control, and spacecraft and flight software. **Energy, facility and industrial** (seventeen): agriculture and irrigation, building and critical-environment HVAC, cooling and chillers, district heating and cooling, energy storage and microgrids, facility and grid optimisation, grid transmission and distribution, healthcare critical environments, industrial PLCs and process automation, mining and mineral processing, oil and gas pipelines, PDUs, UPS and electrical distribution, pharmaceutical and food manufacturing, power generation and turbine control, renewable generation and inverter control, semiconductor fabs and precision manufacturing, and water and wastewater pumping. **Distribution and specialized** (eighteen): cache and memory services, commerce and payments, content delivery and edge caches, data analytics and ETL, databases and transactions, financial markets and trading systems, medical imaging and clinical systems, messaging and streaming, network routing and switching, observability and telemetry, ports and maritime logistics, reliability, security and recovery, runtimes and applications, search, indexing and vector databases, service meshes and API reliability, block, file and object storage, telecom radio access and edge radio, and workflow, logistics and fulfilment. The open-source power and resource managers many operators already run (Kepler, Climatik, EAR, CruiseKube, Goldilocks, the Kubernetes power manager, PEAKS, adaptive GPU power capping and NVIDIA's platform power budget) are muscles too, each in the family its knob belongs to: Omni-Compass sits on top of them as it sits on any native controller.

## 2.2 One body: the spine, the realms and the organism you build

An installation governs one body. The muscles of a stack, its clusters, databases, caches, cards, cooling and power, are read by one governor and judged together, so that a step on one muscle is refused when it would push another past its cushion (chapter 6). Under every realm's own muscles stands the same **spine** of 267 muscles (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution), because every realm's work runs on computers that must be scheduled, powered and cooled. The spine is an organism in its own right, the base every stack runs on as an operating system is the base of every program, and it is wired first (the chapter on the spine follows this one). The catalog groups the muscles into four realms and builds seven organisms, the spine first because the other six stand on it:

| Organism | Muscles | What it is |
|---|---|---|
| The spine (`spine`) | 267 | the muscles every real stack runs on, every one in all four realms: clusters, containers, placement, node fleets, cloud capacity, hosts, GPUs, network, storage, observability, reliability and security, electrical |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

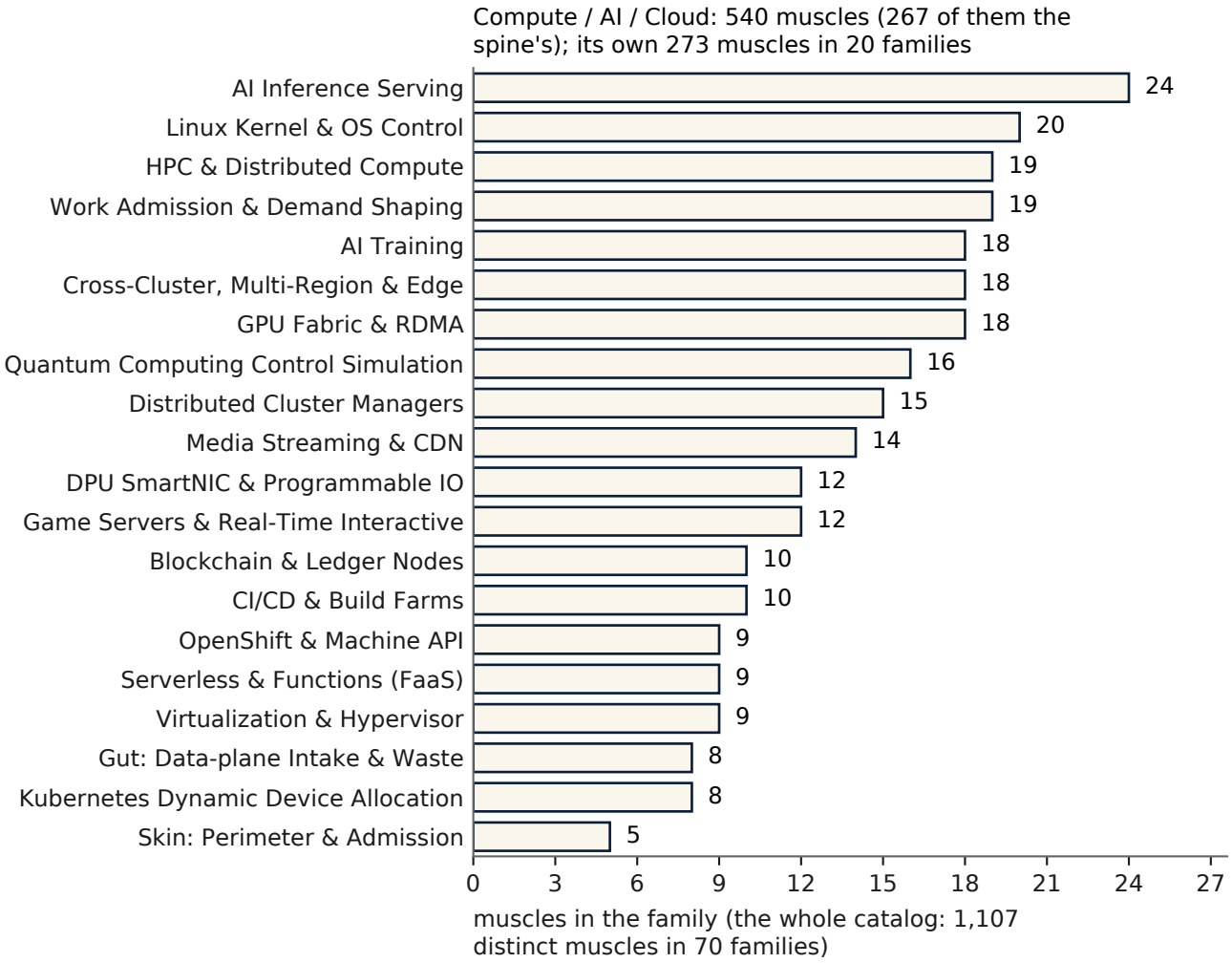



**Figure 5.** Grid: the families of the Compute / AI / Cloud realm, its own muscles on the spine, by count.

| Organism | Muscles | What it is |
|---|---|---|
| | | distribution, cooling; an organism of its own, the base of the other six |
| 1. Compute / AI / Cloud | 540 | clusters, GPUs, AI training and inference, cloud capacity, kernels, functions, media, games, ledgers, build farms |
| 2. Physics / Robotics / Autonomous | 404 | joints, fleets, vehicles, flight and spacecraft axes |
| 3. Energy / Facility / Industrial | 480 | data halls, buildings, batteries, UPS, process loops, feeders, hospitals, farms, pipelines, mines, district heat, power plants, renewables, pharmaceutical and food plants |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Organism | Muscles | What it is |
|---|---|---|
| 4. Distribution / Specialized | 484 | networks, storage, databases, commerce, trading, content delivery, workflows, radio networks, clinical systems, ports |
| 5. The four stacked, every duplicate kept | 1,908 | all four realms on one clock, the shared spine counted in each |
| 6. The whole tower, every muscle once | 1,107 | every distinct muscle on one clock |

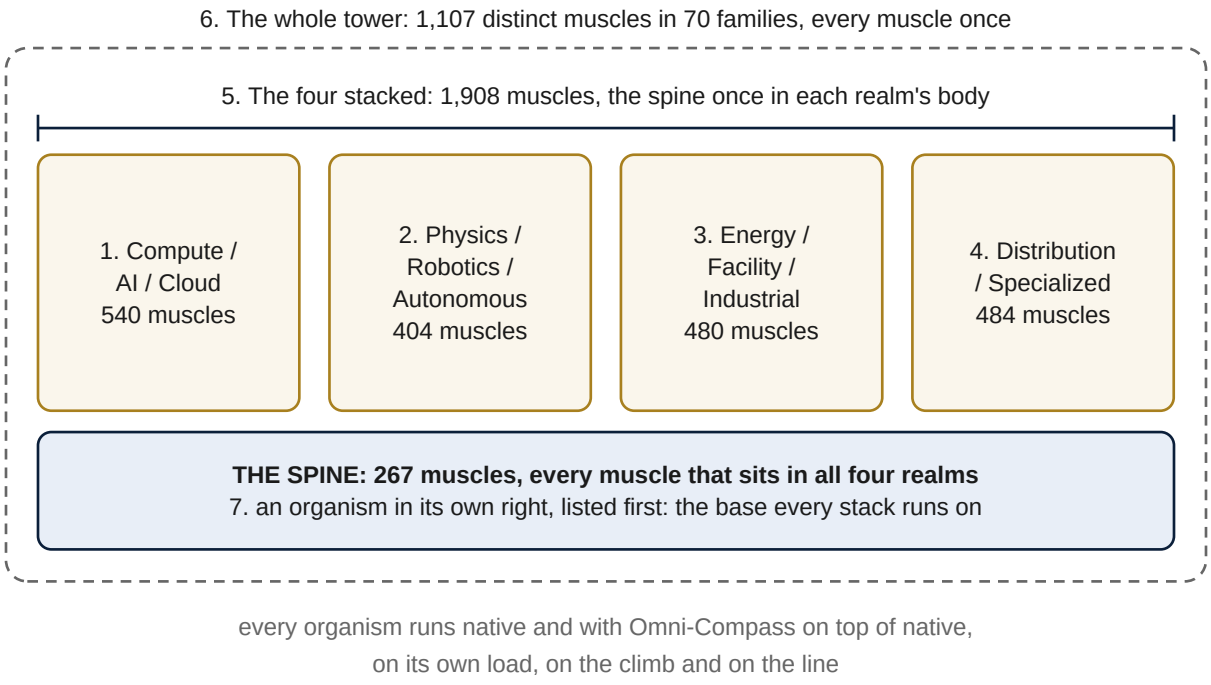



**Figure 6.** The seven organisms. The spine, the 267 muscles that sit in all four realms, is the base every stack runs on and an organism in its own right, listed first. The four realms stand on it; the four stacked count the spine once in each realm's body (1,908 muscles); the whole tower counts every distinct muscle once (1,107 in 70 families). Each organism runs native and with Omni-Compass on top of native.

**Ten organisms, three arms each.** Beside the seven organisms of the catalog the repository reads three more, each a grouping of catalog rows or of real tables and none a product: the **orchestra**, the muscles that stand for the knobs of the governors and orchestrators (the Kubernetes scheduler, Kueue, Volcano, the KAI scheduler, Koordinator, CruiseKube, Crane, KubeRay, Gödel, Armada, Karmada, Open Cluster Management, KubeEdge, Nephio; `tools/run_orchestra.py`); the **live benchmark engine's own organism** of sixty muscles, the one the live benchmark engine page draws (`docs/engine/export_harness_trace.py`); and the **whole organism** in its three places, the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

software end, the cloud's wire and the bare metal, composed from the real tables (`tools/whole_system.py`). Each of the ten is read two ways, native and Omni-Compass on native (Omni-Compass is never by itself, the founder's order of 25 October): twenty scenarios, written by `python3 tools/scenarios_20.py` into `results/scenarios/SCENARIOS_20.md` (our own models, class S). Their readings are in the Owner's Manual; your own organism is the one this chapter builds.

**Building your own organism.** A real installation is rarely a whole realm. It is the muscles one company runs, wired in the order of chapter 7: the cluster first (its pods, then its machines), then the cards and the processors under it, then the databases, brokers and caches beside it, then the site under all of them. Three rules make the result one body rather than a collection of separate governors:

1. **One governor per body.** Every muscle of the body is wired into the same governor, so every step is judged against every other muscle's cushion. Two governors on one body are two brains, and the coupling between them (more pods feeding a database at its connection limit, more cooling pulling a power feed toward its limit) is exactly what neither would see.
2. **Every shared resource is a muscle or a gauge.** Where muscles share a power feed, a cooling loop, a connection limit or a network link, that shared thing is wired in, as a muscle where it has a setting and as a gauge where it does not (a site limit read in watts, a database's connection count), so the governor can see the coupling it must respect.
3. **A muscle with nothing to gain stays wired.** A muscle whose knob never passes a trial stays native, but its gauges still reach the governor, because its state may be what makes another muscle's step unsafe. Watched is not the same as unwired.

CHAPTER 3

# The Spine: the Base Every Stack Runs On

The **spine** is the set of muscles every real stack runs on: the 267 muscles of the 1,107-muscle catalog (`realms/catalog_v5.csv`) whose realm list names all four realms. A robot fleet, a trading system, a hospital's critical environment and an AI training pool differ in every muscle of their own trade, and they are alike in this: their work runs on computers that are scheduled, placed, given processors and memory, connected, stored, watched, kept alive, powered and cooled. Those thirteen families are the spine. Since 16 October (the founder's order) the spine is an **organism in its own right**, the seventh, run native and with Omni-Compass on top of native exactly as the other six are, because the base of a stack deserves its own reading before any trade is put on it. The relation is the one between an operating system and the programs that run on it: no program re-implements scheduling, memory or the disk, it stands on the system that does them; no realm's trade muscles re-implement the cluster, the fleet, the power or the cooling, they stand on the spine.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

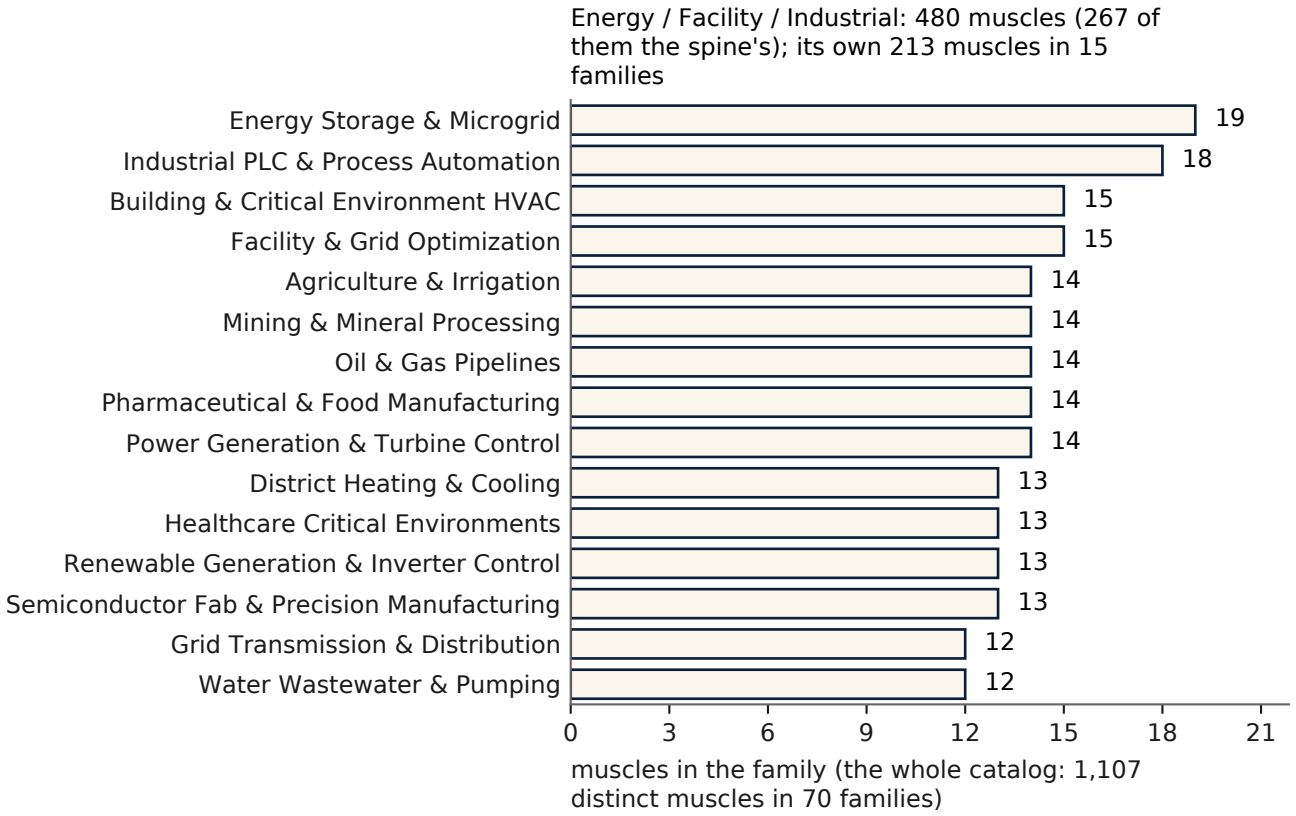



**Figure 7.** Grid: the families of the Energy / Facility / Industrial realm, its own muscles on the spine, by count.

## 3.1 What the spine is made of

| Family | Muscles | Plant model | Native controller class | The knob |
|---|---|---|---|---|
| Kubernetes Workload Scaling | 32 | compute pool (server) | the pod autoscaler, KEDA, the operator's rightsizers | replicas, targets |
| Container Resources | 25 | compute pool (server) | the vertical autoscaler, the scheduler's requests and limits | capacity, admission |
| Kubernetes Placement & Scheduling | 21 | compute pool (server) | the scheduler, descheduler, topology and affinity rules | admission, capacity |
| Node Fleet & Karpenter-Class Control | 24 | compute pool (node) | the cluster autoscaler, Karpenter-class provisioners | machines |
| Cloud VM & Capacity | 21 | compute pool (node) | the cloud's own scaling sets and capacity reservations | machines |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

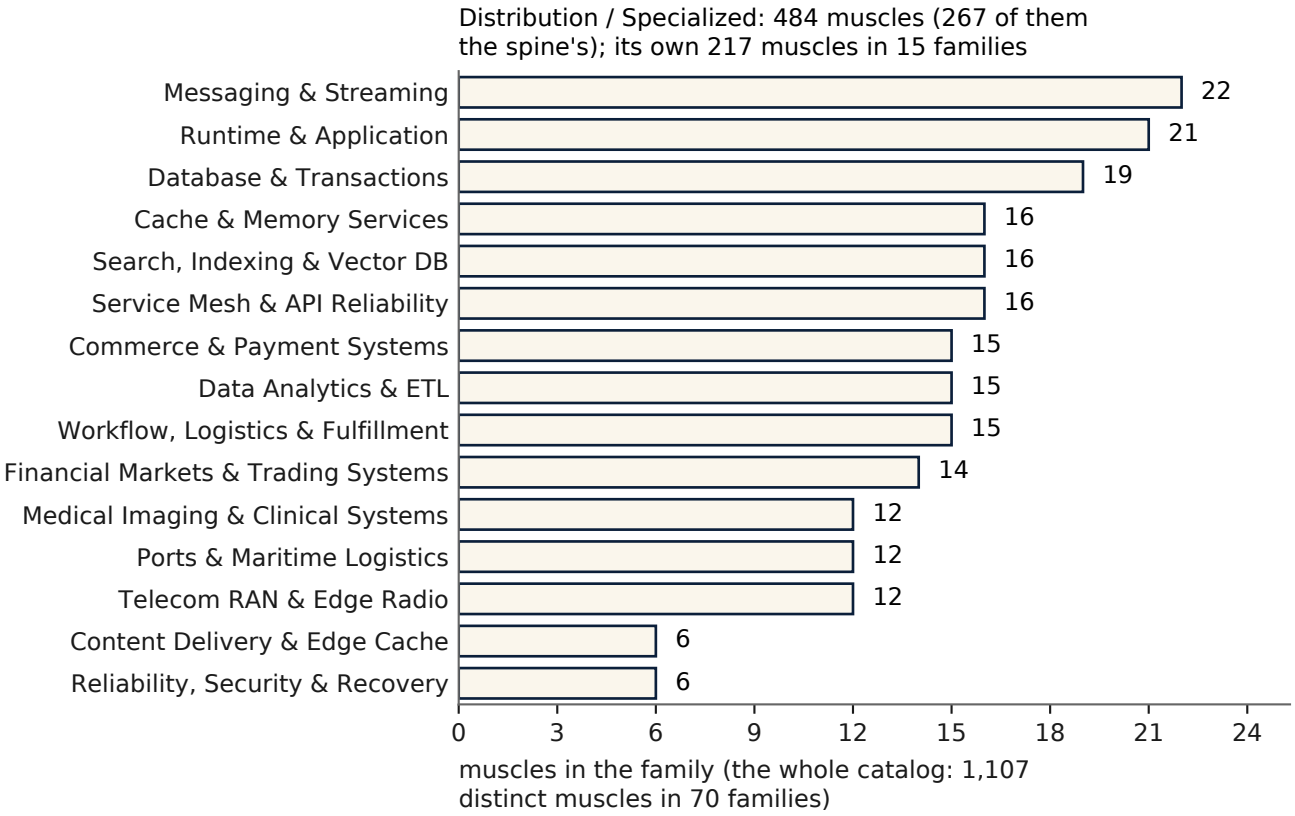



**Figure 8.** Grid: the families of the Distribution / Specialized realm, its own muscles on the spine, by count.

| Family | Muscles | Plant model | Native controller class | The knob |
|---|---|---|---|---|
| Host CPU & Memory | 26 | compute pool (CPU host) | the kernel's frequency governor, memory and cgroup controls | capacity, power |
| GPU Hardware | 22 | compute pool (GPU) | the card's firmware: clocks, power limit, partitioning | clock ceiling, power cap |
| Network Routing & Switching | 19 | compute pool (network) | routing, queueing and buffer controls | capacity, admission |
| Storage Block/File/Object | 20 | compute pool (storage) | tiering, caching and throughput controls | capacity, admission |
| Observability & Telemetry | 9 | compute pool (server) | sampling, retention and scrape controls | admission |
| **Family** | **Muscles** | **Plant model** | **Native controller class** | **The knob** |
| Reliability, Security & Recovery | 12 | compute pool (server) | disruption budgets, recovery and admission controls | admission |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Family | Muscles | Plant model | Native controller class | The knob |
|---|---|---|---|---|
| PDU, UPS & Electrical Distribution | 18 | energy storage (UPS) | the UPS and PDU controllers, their reserves and line limits | power, setpoint |
| Cooling, Chillers & Thermodynamics | 18 | thermal zone (data hall) | the chillers, air handlers and their setpoints | setpoint |

Two hundred and sixty-seven muscles in thirteen families: 156 capacity knobs, 48 admission knobs, 35 power knobs and 28 setpoints. Every one is listed, with its knob, in `docs/REALM_MUSCLES.md` (the spine first), and explained, wired and carded in `docs/MUSCLE_CATALOG.md` (the manuals' muscle appendix, with every family's wiring recipe from `docs/wiring_recipes.csv`: the native controller, the wire out, the wire in, the restore, and whether the wire has run live in this repository or only on a model). The muscles of the spine's Kubernetes families are the ones the live Kubernetes benchmarks run on real clusters (evidence class L); the spine as an organism is modelled (class S), and nothing in it is presented as proof of a real stack.

## 3.2 Why the spine is an organism

An **organism** is a set of muscles stepped on one clock, each from its own plant model with its own native controller, the whole fed one demand from one seed, so that the heat the machines make is load on the rooms, the power swing of the body is load on every site's line, and one brain judges every step against every other muscle's cushion. Run alone, the spine is a data center with nothing of any one trade in it: clusters, fleets, processors, cards, network, storage, power and cooling, and the governor on top of their native controllers. Three readings come from it that the realms alone cannot give:

1. **The base before the trade.** Whether Omni-Compass helps the machines every stack runs on, before any realm's own muscles are put on them. A buyer wiring only a cluster and its site wires the spine and nothing else.
2. **What a trade adds.** Each realm's organism is its own families on the spine. Read beside the spine's reading, a realm's reading shows what that realm's trade muscles add to, or take from, the base they stand on.
3. **The same base under four trades.** The four stacked carry the spine once in each realm's body, the tower carries it once in all. The spine run alone is the control for both: the same muscles, the same native controllers, no trade on top.

The spine is run under the same three loads as every organism (its own load; the climb, 1 2 3 2 3 4 …; work waiting in line), is judged by the same rule (work, speed and misses against native, the energy the max takes shown as power given), and has the same two arms and no other: **native**, every native controller alone, and **omni**, Omni-Compass on top of those same controllers, moving only the settings they expose, within their own ranges, and handing every setting back at the end.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 3.3 How the spine is wired on a real stack

The spine is wired as any organism is: one governor per body, every shared resource a muscle or a gauge, a muscle with nothing to gain kept wired and watched (the Operator's Manual, chapter 2). The order is the order of the levels in the Operator's Manual, chapter 6, which is the order of the spine's own families: the cluster first (its workloads, then their resources and placement, then the machines under them), then the processors and the cards, then network and storage, then the site under all of them (electrical distribution and cooling), with observability and recovery wired from the first level, because the governor reads through them. Each family's wire is one row of `docs/wiring_recipes.csv`: which native controller owns the setting, where the reading is tapped, where the setting is written, and how it is restored. A realm's own muscles are wired after the spine they stand on, never instead of it.

## 3.4 The spine in the runners and the pages

The organisms are groupings of catalog rows made in the runners; no engine file names them, so the spine entered the runners without changing the engine that stands (`python3 tools/omni_version.py` prints the same fingerprint before and after). In every runner the spine is `spine`; in `tools/run_scale.py` and the grid conductor (`scripts/dispatch_grid.py`) it is organism `7`, after the six that were numbered before it, so that every grid run already started reads back under its own number. Every table lists the spine first, because the other six stand on it.

| Where | The spine |
|---|---|
| `tools/run_realms_v5.py` | every muscle alone, then the seven organisms under three loads; the spine's row first in every organisms table (`results/realms_v5/REALMS_V5.md`) |
| `tools/run_scale.py --organisms 7`, workflow `six` | the spine at 1, 10, 100 and, on GitHub's own machines, 1,000 copies, over 1 to 1,000 paired runs |
| `tools/run_kil.py --organism spine`, workflows `six-kube`, `aks-metered`, `gke-metered`, `eks-metered`, `cloud-organism` | the spine with the real Kubernetes cluster inside it as one more muscle |
| `tools/run_hil.py` | the spine with the real card inside it (the GPU Hardware family is a spine family) |
| **Where** | **The spine** |
| `scripts/dispatch_grid.py` | the spine's squares started, read back and never started twice like every other organism's |
| **Where** | **The spine** |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Where | The spine |
|---|---|
| `docs/REALM_MUSCLES.md`, `docs/MUSCLE_CATALOG.md` | the spine's 267 muscles listed first; each muscle's organisms column names the spine |
| `results/realms_v5/`, `results/grid_v5/`, `docs/STATE_OF_PLAY.md` | the spine's readings, native against Omni-Compass on top, each named for the engine it ran on |

CHAPTER 4

# Tuning: The Factory Tune, the Dial and the Settings You Write

## 4.1 The factory tune

Omni-Compass ships tuned. The engine in the repository is presented as The Omni-Compass LLC tuned it and ships it: a self-running engine, set at the factory, the way a vehicle leaves the plant with its engine already mapped. Every muscle in the catalog is wired and runs on the setting found for it: its own card where one was found and kept (section 4.3), the engine's own setting for every other muscle. The whole tune is fixed and fingerprinted by version (`python3 tools/omni_version.py`), so an installation can always tell whether it is running the factory tune or a tune of its own. Every published result is the factory tune of the engine its table names (`python3 tools/omni_version.py`), and is run again on each engine that stands. In the words of `docs/FACTORY_PROFILE.md`, the factory tune is the **factory profile**: the complete, versioned set of Omni-Compass default settings shipped together. The released one is the engine that stands, omni-v25's. The best validated factory profile for the declared operating conditions has not yet been established: it is chosen only after every mechanism is verified and the settings are tuned and validated together (`docs/FACTORY_PROFILE.md`, section 5), and until then the released profile stands and no other is locked as the baseline.

No setting is universal. Muscles differ, and no single setting is right for every muscle, family, sector, domain or realm; none is presented as such. What the factory tune fixes is the starting point an installation can trust: each muscle on the setting that was found for it, every muscle wired, nothing switched off.

**One tuned engine, and a tuning bench.** Every benchmark result is one engine at one tune: the engine that stands at its factory tune. The live benchmark engine page (the Live Benchmark Engine Manual) and the control panel's buttons (Chapter 10) are a tuning bench. On the bench every knob can be turned in real time: the beat, the dial, the pedal, each muscle driven or left native, up or down, in or out, minus or plus. It shows the mechanism of action with native alone, with Omni-Compass on top, or both side by side. Treat

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Omni-Compass as you would any engine. Turning the mixture, the intake, the exhaust, the firing or the pistons changes what a car's engine delivers. In the same way, **turning any knob can change every metric, up or down, in any direction you choose**, which is why one engine can serve many stacks, tuned many ways. What you read on the bench at a setting other than the factory tune shows the range; it is not a benchmark result. A tune you write is your own configuration, its results are your own, and they are read again, native against Omni-Compass, before you rely on them. The factory tune is not set by hand. Of the tunes that harm no gauge against native, it is the one against which no other tried tune reads GAIN on one service gauge and LOSS on none, searched on development seeds and read again on untouched ones. How that search is done is in the Owner's Manual.

## 4.2 The max, and the dial that tunes it down

The factory setting is **the max**: every muscle runs up to its own limit, read live from the system, never past it and never below what native gives. It rides the edge of each muscle's range, not the middle. The energy it takes to do more work is **power given**, shown beside the work, the speed and the wear it bought, never called a loss; on the engine that stands the max takes nothing from one muscle to give to another. A knob that native already runs at the top of its range is "at its limit on native", and the max leaves it there.

The **dial** turns the max down. It has two positions, the same on every live controller:

| Position | Switch | What it does |
|---|---|---|
| **max** (the default) | `--dial max`, or `OMNI_DIAL=max` | the most work, faster: the service held well inside its line so every rush finds its capacity already there; never a machine given back below what the system ran on its own; on a card, the clock ceiling never stepped below the top while the card serves |
| **save** (the tune-down) | `--dial save`, or `OMNI_DIAL=save` | native's work on the least resource: the service held nearer its line, machines and clock steps given back where the paired trial on the muscle shows the response time allows (section 1.2) |

The tune-down never does less work than native and never misses its line more often; it spends less to do the same. An installation chooses its position per governor, writes the choice down before its paired runs, and reads every receipt against the position it ran.

## 4.3 Each muscle's own card

The governor is one, but its settings are each muscle's own. A muscle's **card** holds the numbers the max uses for that muscle alone: for a machine pool, how full it runs on its forecast before it calls for more and how many decisions of history it reads; for a battery, how many wobbles of margin it keeps under its line and, where it is wired by its power, how long it reads its line before it holds and how much of it it keeps clear. The

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

cards were found by a search over every muscle on development cases only, today's engine setting always one of the choices, and a card was kept only where it beat the engine setting beyond the noise on two separate sets of cases and was never behind native on any case under any load (never over its line more often, never less work, never its line cleared later). 364 muscles carry one (`realms/muscle_cards_v5.csv`); every card is written out in plain words with its muscle in Appendix E; the rest run the engine setting. Inside a body, a battery wired by its power keeps the body's own margins on its line, because the line carries the whole body's swing, which the card chosen for the battery alone never saw.

An installation reads a muscle's card before it wires that muscle, so that the receipt can be read against the setting the muscle actually ran on.

## 4.4 The settings an installation writes

Everything else an installation writes is the operator's own, taken from the operator's own system and objectives, and written down before the first counted run:

| Setting | What it is | Where it comes from |
|---|---|---|
| **The line** (`--slo-ms`, or the stack's own line) | the response time, or other limit, the service must meet | the operator's own service objective, never ours |
| **The band** | the reading's range from calm to the line; on a service, from a tenth of the line (the bare service time) to the line | the line, by rule |
| **The cushions** | the last 5% of the band at each edge, 10% in all; a reading in a cushion is treated as already too close to the edge | fixed, the same on every muscle |
| **The centre** (`--compass-center`) | where in the band the governor holds the reading: 0.3 at the max, 0.4 at the tune-down, unless the operator writes another | the dial, or the operator |
| **The cover** | the knob's hard range; every write is clipped to it | read from the system (the autoscaler's range, the card's supported clocks, the pooler's connection limit), or written by the operator inside it |
| **Setting** | **What it is** | **Where it comes from** |
| **The ceiling of a growing lever** (`--replica-ceiling`, the | how far a lever that can keep adding may go | read live by default: what the machines can hold, one under it; |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | What it is | Where it comes from |
|---|---|---|
| machine lever's top) | | an operator may write a lower one, or 0 to keep native's own cap |
| **The floor** | the least a pool may run on | two machines by default; never below 1 for any muscle |
| **The allowance** (`--allow`) | the most a step on trial may cost the muscle's own work | 2%, the same on every muscle |
| **The site's limits** (`--site-limit-w`, `--cooling-min-c`, `--cooling-max-c`, `--cooling-restore-c`) | the facility's power limit and the setpoint band | the facility engineer, inside the equipment's ratings |
| **The heartbeat** (`--interval`) | how often the governor takes its full decision: the factory beat for its place, every 10 s on site and every 60 s through a cloud, read from the cluster's nodes; a dial from 5 to 120 s; the fast checks stay every 5 s | the factory's beat for its place, or the operator's own beat; the live benchmark engine page shows every beat beside native, with what it buys and what it costs (the page's section 1j) |

Two settings are deliberately not on that list. The governor's damping and how fast each muscle answers are the muscle's own declaration, read from the catalog or measured from the system at the start; an installation does not tune them, and a receipt that needed them changed is a receipt to read as wiring first (section 5.5). The heartbeat is on the list since omni-v17: every rule the governor keeps counts seconds, so the beat changes only how often it looks, and an operator may set any beat, a fast one for its reads of the cluster at the cost of the governor's own processor, or a slow one. Each beat may want its own combination of the other settings; a beat other than the factory's for its place (10 s on site, 60 s through a cloud) is the operator's own configuration and its results are its own (section 4.6).

## 4.5 Profiles: service and batch

The governor has two **profiles**, service (the default) and batch, which differ only in what the pedals do with a pile of work: in the service profile the governor paces the slack; in the batch profile, cruise puts every machine in service while work waits for a place, and idle takes them straight to the floor when the queue is done (section 1.3). An installation running a queue of jobs chooses batch for that service and service for everything else, and writes the choice down beside the dial's position.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

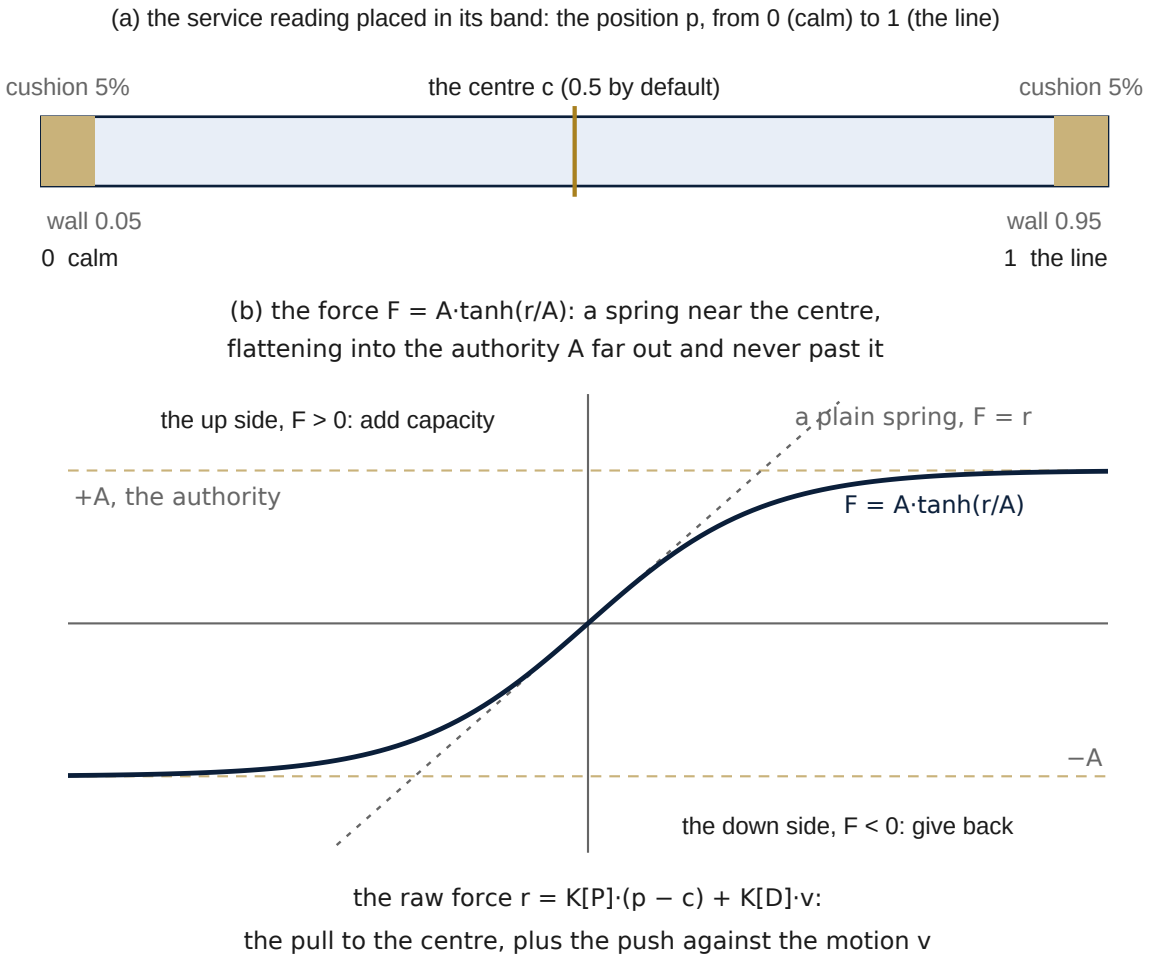



**Figure 9.** The compass band and the smoothing of the force. (a) The service reading placed in its band as a position p from 0, calm, to 1, the operator's line; the walls at 0.05 and 0.95 leave a cushion of 5% at each edge, and the centre c is where the needle comes to rest (0.5 by default; on a service 0.3 at the max and 0.4 at the tune-down).

(b) The force F = A·tanh(r/A) against the raw force r = K[P]·(p − c) + K[D]·v, the pull toward the centre plus the push against the reading's motion: near the centre a spring, far out a curve that flattens into the authority A and never passes it. Past the upper wall the up side goes to A at once, and the down side may not act until the reading is back inside the band.

## 4.6 A changed tune is your own configuration

As with any engine, a licensee may add on, retune, upgrade or tune down any muscle or any combination of muscles: the dial, the cards, the bands and the service lines are all exposed. A changed tune is the licensee's own configuration, and its receipts are its own, never the published ones. The discipline for a changed tune is the discipline for the factory tune: write the change down before the paired runs, run native and omni three

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

times on it, show every row, and read the receipts against the tune that ran. Where a changed tune reads differently from the factory tune, the change is the first thing to look at, and then the wiring.

## 4.7 The output follows the input

Every number Omni-Compass produces is the result of three inputs, and a reader who changes any one of them has changed the result: **the tune** (the factory tune as shipped, or the dial turned down, or a card changed), **the wiring** (which muscles are wired in and which are watched, what each wire reads and writes, how fresh each feed is) and **the load** (the work offered, its shape and its line). The published receipts are the record of one setting of all three, and every receipt names it: the engine's fingerprint, the harness, the load and the window. The same engine wired onto another stack, or tuned to another purpose, gives another result, and that result is its own.

A team that wants a different outcome sets the inputs for it and reads its own receipts against native: the most work at any spend (the max, the default, in a cloud as on site since 31 October); the same work as native for a smaller bill (the tune-down, a tune anyone may set); a muscle held at a limit of the team's own choosing (its card, or its cover); a knob left native (watched, never written). Each of those is a legitimate setting of the inputs, and each has its own result. What no reader may do is read one tune's receipt as another's, or read a receipt without its inputs. The factory tune (section 4.1) is the one setting whose receipts this book publishes; the dial (section 4.2) and the cards (section 4.3) are the inputs a licensee may change, and section 4.6 says what a changed tune's results are. Where a result read elsewhere differs from the published one, the first presumption is an input: the wiring, the settings or the reading (`DISCLOSURES.md`, section 4).

## 4.8 The factory profile, a custom profile, and the two restoring controls

The words of this section are written once, with their exact meanings, in `docs/FACTORY_PROFILE.md`; the mathematics behind them is there and in the Owner's Manual, never here. (The service and batch profiles of section 4.5 are two settings inside a configuration profile, not profiles in this sense.)

**Start from the factory profile.** An installation starts on the released factory profile: the governor started with no setting beyond the ones of section 4.4 that come from your own system (the line, the feed, the floor, the cover), every muscle on its card or on the engine's setting (section 4.3), the dial at the max, the heartbeat at the factory's beat for its place (10 s on site, 60 s through a cloud, read by the governor itself), no muscle left native. Write down, before the first counted run, the engine's fingerprint (`python3 tools/omni_version.py`) and the full command line. A frozen profile fixes the rules and the default tuning, not the knobs: the governor's own state, the settings it commands and the resources it allocates still follow the system as it changes.

**Make a custom profile.** Any setting changed from the factory profile, by a command-line flag (Appendix K), an environment variable, a press on the control panel (section 23.5) or a knob on the live tuning bench, makes a **custom profile**: an identifiable configuration that differs from the released factory profile. Make one as you would make any change to a running system:

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

1. Write the change down: which setting, from what to what, and why, beside the engine's fingerprint.
2. Run the governor in watch (section 1.3) on the custom profile and read its decisions before it writes.
3. Run the paired runs of chapter 11 three times on it, native against it, every row shown.
4. Read its receipts as its own (section 4.6): a custom profile's results are never the published ones, and a published result is never read as a custom profile's.
5. Keep the command line and the control panel's log with the receipts, so the profile that produced them can be named.

On the live tuning bench (the live benchmark engine page; how to read it is the Live Benchmark Engine Manual's) every turned knob makes the **active profile**, the settings governing the run in front of you, a custom one, named by its fingerprint; the sweep's best beat can be tried as a **candidate profile**, a proposed combination under evaluation. Neither becomes a factory configuration without validation and a new release declaration, and no profile is locked as the baseline today.

**The two restoring controls, and the switch that is neither.**

| Control | What it does | On a running installation |
|---|---|---|
| Restore factory profile | reapplies the released Omni-Compass configuration | start the governor again on the factory profile's command line (no setting but your system's own, section 4.4); on the control panel, the dial at max, the heartbeat at 60 s and every muscle on, each press passing through native and starting the governor again (section 23.5) |
| Return to native | hands governance back and restores the recorded native settings, as the adapter supports | the reset: `POST /v1/reset` (role operator) or the governor's reset file; every setting handed back to the snapshot taken before the first write, read back, and the governor stopped until the next mode press (chapter 9) |
| The kill switch (security only) | turns every Omni-Compass governor on the machine off at once | `python3 tools/omni_switch.py off` or `POST /v1/kill` (role security): each governor restores and exits, none starts while the switch is off, and two holders of the security role arm it again (chapter 9) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Return to native is an operator's control and leaves the master switch on; the kill switch is separate, for security only, and instant. On the live benchmark engine page, which runs no real system, both act on what runs in the page.

PART TWO

# The Harness: Plugs, Wires and the Rules They Obey

*The universal plug, the adapters, the wire check that must pass before anything writes, and the rules every wire obeys.*

CHAPTER 5

# The Plug, the Adapters and the Wire Check

Think of a high-end car stereo: one head unit, one standard plug on its back, and an adapter harness for each make of car that matches the car's factory plug. You never cut a factory wire, and when you pull the stereo, the car works as it did. Omni-Compass is wired the same way.

## 5.1 The plug (one design for every muscle)

Each plug has as many channels as its muscle has knobs. Every channel declares its cover (range), its direction (which way is "more"), its units and how fast it may move. Every plug keeps the same five-line contract, and an adapter for a new muscle is written to it:

1. **attach** - read the knob once, before any write: the snapshot. It never moves afterwards.
2. **read** - the service reading, placed in its band.
3. **write** - one value, clipped to the cover, then read back from the device.
4. **one writer** - if the knob is found at a value Omni-Compass did not write, someone else owns it: Omni-Compass stops writing and leaves that value alone.
5. **restore** - on stop, the knob returns to the snapshot (where Omni-Compass found it, not the last value it wrote). For muscles whose native controller moves the knob itself, restore hands control back.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

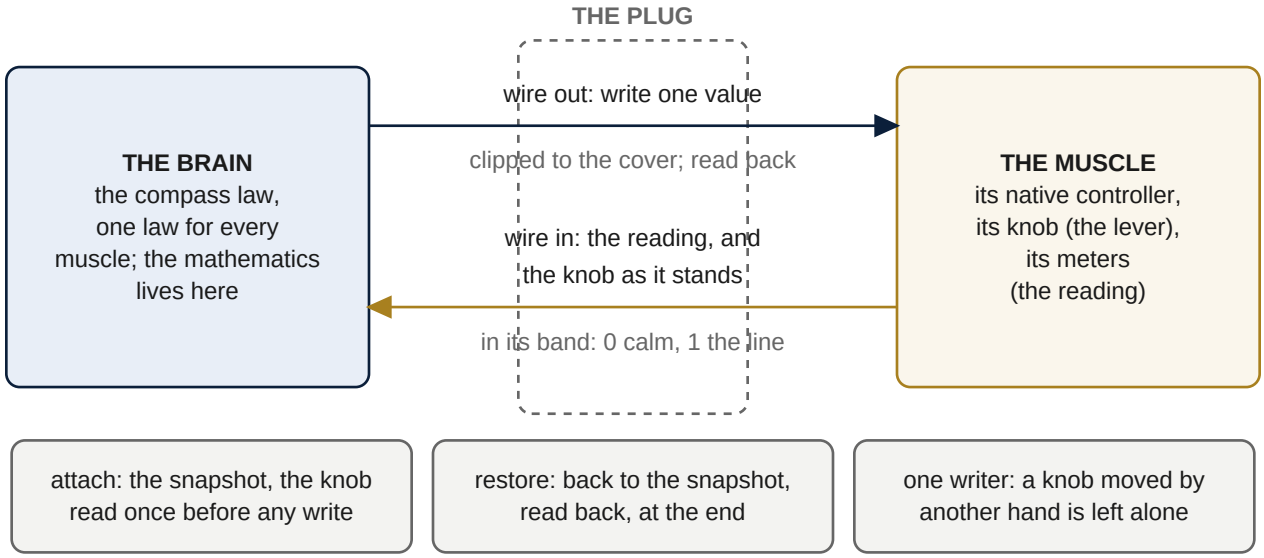



**Figure 10.** The two-way plug. One wire out carries the governor's write, one value per decision, clipped to the knob's cover and read back from the device; one wire in carries the muscle's reading, placed in its band, and the knob as it stands. The snapshot taken at attach, before any write, is the restore point, and a knob found where the governor did not put it belongs to another writer and is left alone. The brain holds the mathematics; the plug holds nothing but the contract.

## 5.2 Adapters (one per standard plug)

Omni-Compass speaks each industry's standard control interface through an adapter. Status is stated plainly:

| Standard | Muscles | Adapter status |
|---|---|---|
| Kubernetes API | HPAs, deployments, pods, nodes | **built** (`python3 -m omni_controller.controller`) |
| NVIDIA NVML / `nvidia-smi` | GPU clock ceiling, power limit | **built** (`python3 -m omni_controller.gpu_compass`, two wires; `omni_controller.gpu_governor`, one wire) |
| Linux cpufreq, RAPL | CPU frequency ceiling, package watts | **built** (`--cpufreq-policy-root`, `--rapl-cmd`) |
| Site meter, building controller by command | site watts, supply-air setpoint | **built** (`--power-cmd`, `--cooling-cmd` command templates) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Standard | Muscles | Adapter status |
|---|---|---|
| Cloud node groups (Karpenter, Cluster Autoscaler, MachineSet) | machines | **built** through `--node-scale-cmd` templates |
| BACnet, Modbus, SNMP | chillers, air handlers, PDUs, UPS | designed; today reached through the command templates above |
| OPC UA, EtherNet/IP, PROFINET, EtherCAT, ROS 2, CAN | PLCs, drives, robots, vehicles | designed; reached today through a command template written to the plug's contract |
| IEC 61850, DNP3, OpenADR, IEEE 2030.5, SunSpec, OCPP | substations, batteries, inverters, chargers | designed; reached today through a command template written to the plug's contract |

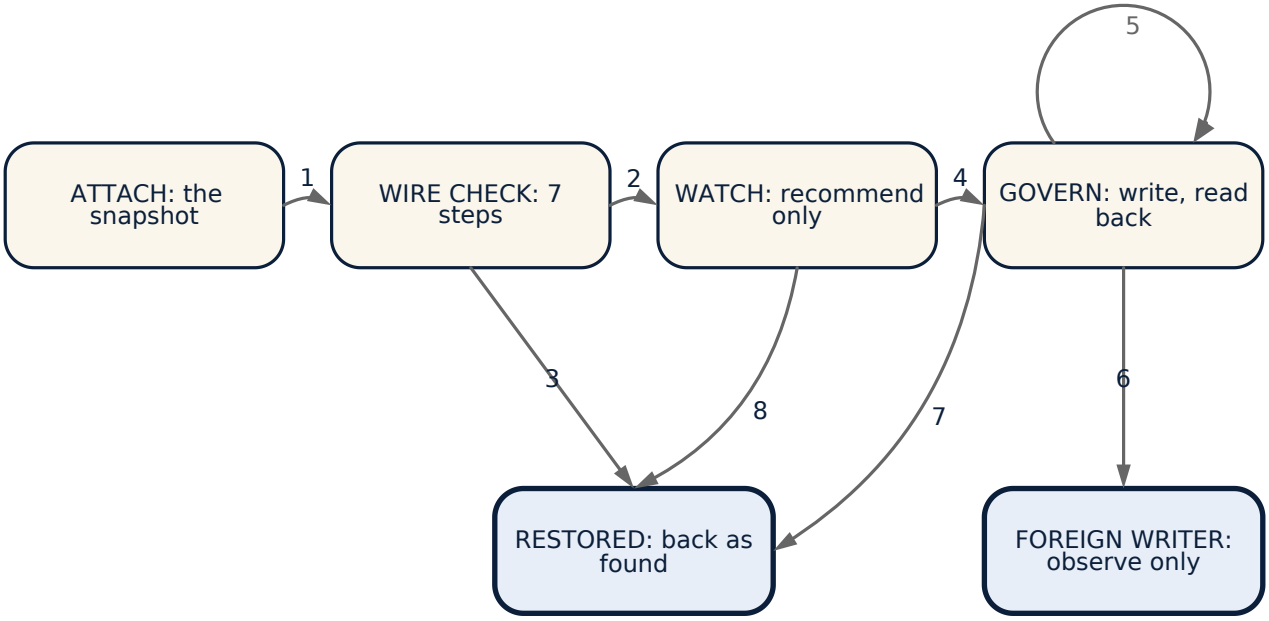
The engineer's job for one muscle is three things: which adapter, the address of the knob, and its safe range.

**The plug contract, line by line, and why each line is there.** *Attach* reads the knob once before any write and keeps that value as the snapshot. It is read once and never again, because a snapshot that moved with the knob would restore to wherever the governor last left it, and the whole point of the restore is to return to where the operator had it. *Read* takes the service reading (a response time, a temperature, a queue, an error) and places it in its band; the plug declares the band, the governor does not guess it. *Write* takes one value, clips it to the cover before it leaves the plug, sends it through the muscle's own interface, and then reads the lever back from the device and records both the sent and the taken value; a device that took something other than what was sent (a clock that rounds to 15 MHz steps, a pool size the pooler capped) is recorded as it is, not as it was asked. *One writer* compares the lever as found at every read with the value last written; a lever found elsewhere has another owner, after which the governor leaves that lever alone for the rest of the run and says so in its audit and its exit code. *Restore* writes the snapshot back on stop and reads it back; a restore that does not read back is an error (exit 3), never a silent success. The same five lines are implemented for Kubernetes objects (annotations hold the snapshot on the object itself, so a watchdog can restore from the object without the governor's memory), for `nvidia-smi`, for PgBouncer's console, for Redis's `CONFIG SET`, for the consumer group and for the simulators' knobs. An adapter written for a new muscle keeps the same five lines, and is proved against a stand-in device of that muscle before it is pointed at a real one.

## 5.3 Power-up order

Read-only first; then take the knobs, the least consequential first; on stop, hand them back in reverse order. In the Kubernetes controller the order is: the latency feed and the cluster's own metrics (read); the HPA target (a setting the HPA accepts and corrects within a minute); the pods' CPU limits (conveyance, in place); the machines (the slowest and most consequential, through the release gate and the verdict). On stop the machines are woken first, then the limits, then the target, so the cluster is never left with fewer machines than its pods need while a setting above them is still being handed back.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*




1 the snapshot taken · 2 WIRED RIGHT · 3 FAIL: nothing written, the knob as found · 4 the paired trial passed · 5 every decision: clamp, gate, write, read back, receipt · 6 the knob found where the governor did not put it · 7 stop: reset, OFF, the lease · 8 stop before the trial passed

**Figure 11.** State machine: the life of one muscle under the plug. Attached with its snapshot, checked by the seven-step wire check, watched in recommend-only mode until the paired trial passes, then governed, one clamp, gate, write, read-back and receipt a decision; a knob found where the governor did not put it moves the muscle to observe only, and every stop restores it to the setting as found.

## 5.4 The wire check (mandatory before any write)

For the GPU it is one command, `sudo python3 tools/gpu_wire_check.py --gpu 0`, and `scripts/gpu_rented_run.sh` runs it first and stops if it fails:

| Step | It proves |
|---|---|
| 1 read | every meter answers: watts, temperature, utilization, limit, clock, top clock |
| 2 snapshot | the start limit is recorded once |
| 3 up wire | with the card busy, a ceiling below its own busy clock pulls the clock down under it; reset, the clock comes back above it |
| 4 down wire | a lower power limit is reported back exactly; the start limit is reported back exactly |
| **Step** | **It proves** |
| 5 restore | clocks reset and the start limit read back |
| **Step** | **It proves** |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Step | It proves |
|---|---|
| 6 stop | the governor, stopped by its signal, hands both wires back |
| 7 other writer | a limit set by someone else while the governor runs is left alone, and the governor exits 5 |

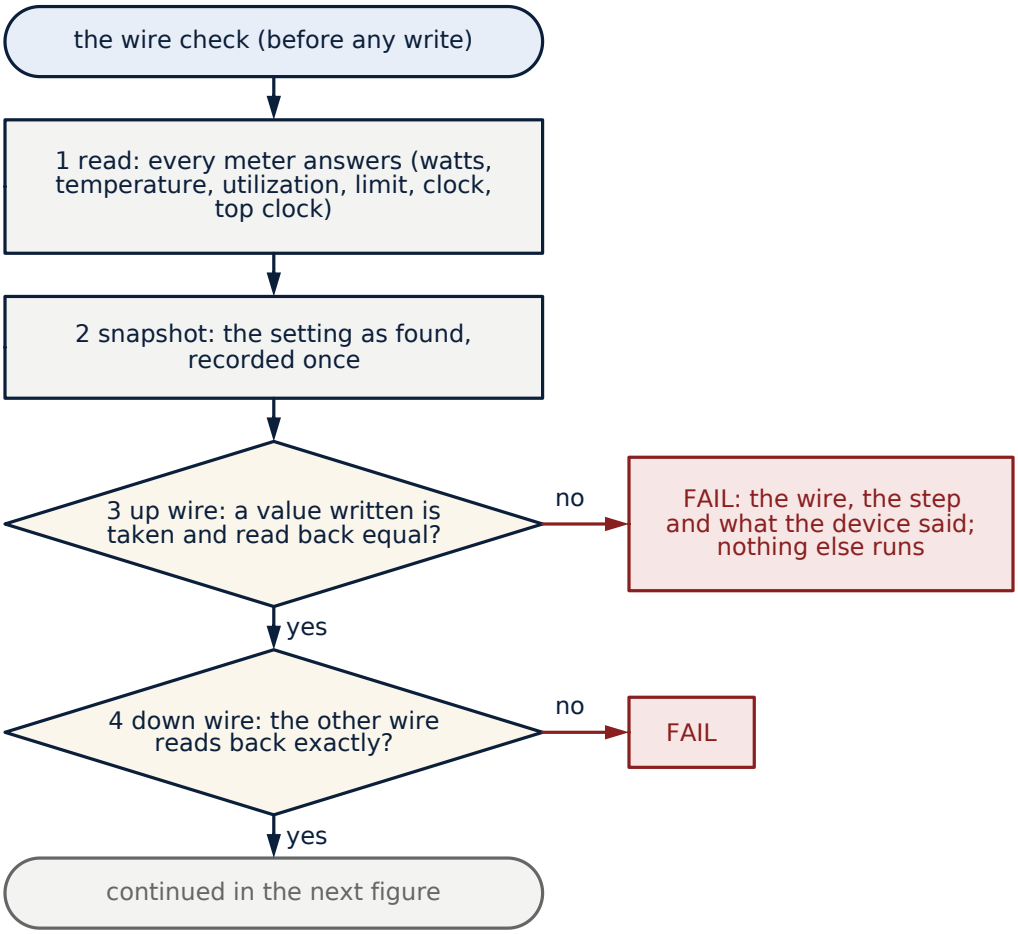



**Figure 12.** Flowchart: the wire check, steps 1 to 4. Before any write: every meter answers; the setting as found is recorded once as the snapshot; a value written on the up wire is taken and read back equal; the down wire reads back exactly. A step that fails stops everything with the wire, the step and what the device said; the chart continues in the next figure.

Any failure prints the wire, the step and what the device said, and nothing else runs. That turns a wiring fault from guesswork into one named line to fix.

## 5.5 Wired right or wired wrong

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

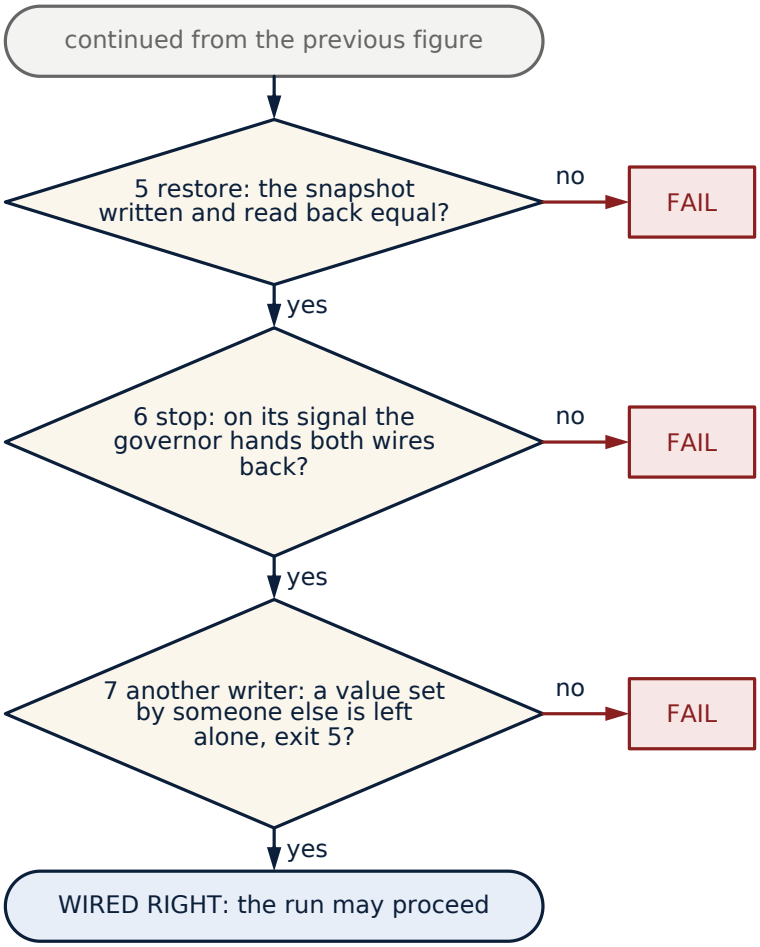



**Figure 13.** Flowchart: the wire check, steps 5 to 7 and the verdict. The snapshot written back reads back equal; on its stop signal the governor hands both wires back; a value set by someone else is left alone (exit 5). Only then is the muscle wired right and the run may proceed.

The wire check proves the wires move. It does not prove the governor reads the right thing. A correct installation leaves signatures in its own receipts; a wrong one leaves others. Check them on your first paired runs, before you believe any number, good or bad.

| Check | Wired right | Wired wrong, and what to look at |
|---|---|---|
| Watch arm against native | equal on every gauge (watch writes nothing) | different: the probe, the load or the machine differs between arms, not Omni-Compass |
| Requests served, failed requests | equal to native | fewer served or more failed: a lever is cutting capacity; check its cover and its sign |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Check | Wired right | Wired wrong, and what to look at |
|---|---|---|
| p95 and p99 response time | at native or faster | slower: the reading or the floors are wrong (rows below) |
| GPU: the card's clock while busy | at or above the clock the card reaches on its own | below it: the speed floor is missing or the lid is under the card's own draw |
| GPU: the lid while busy | at or above the card's own busy draw | at the envelope floor while busy: the lid is sized from a curve, not from the card's own meter |
| GPU: credit per write (`GPU_REPS.md`) | compass decisions pace the slack, race decisions cover the bursts; fail-up is rare | fail-up in most decisions: being busy is being read as a breach, or the service line is set too low |
| Kubernetes: machines given back | through the release gate, one per decision, with no pod waiting | machines given back while pods wait, or none at all while the service is far inside its band: check the latency feed, `--slo-ms` and the release gate's reasons in the audit |
| Kubernetes: the reset | every HPA target, replica range, CPU limit and worker back to native, no record left | anything left: an earlier run or another controller wrote the same objects |
| Every lever after the run | at its snapshot | not at its snapshot: another writer, or a restore that failed (exit 3): restore by hand and investigate |
| The feed itself | fresh samples every few seconds | stale or empty: Omni reads it as blind and fails up; fix the feed, not the governor |

**The rule.** If any row reads "wired wrong", the result of that run says nothing about Omni-Compass: fix the wiring, run the wire check and the watch arm again, and only then compare. The first real card run is the worked example: its receipts showed the card's busy clock under its own (736-768 MHz against 861-889), the lid at the 105 W floor under its own 135 W draw, and fail-up in 46% of decisions. Each of those is a row in this table.

## 5.6 The wire check on the other stacks

The GPU has a one-command wire check because its wires are the hardest to get right. The other stacks carry the same proof in a different place: every Kubernetes arm records `kubectl auth can-i` receipts for what the governor's identity can and cannot do (and fails if it can do anything it must not), checks that the cluster holds nothing but the harness, and ends with the reset check that every HPA target, replica range, CPU limit

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and worker is back at native with no record left; every database, broker and cache arm reads the knob back after every write and hands it back at the end, and the report says "handed back: yes" or the run is invalid; every simulator arm checks that every override was handed back. The watching arm, where it is run, is the wire check of the reading: a watching governor that disagrees with native on any gauge has found a difference in the probe, the load or the machine, not in Omni-Compass.

## 5.7 Coding a new muscle: the muscle SDK

A muscle not in the catalog (Appendix E), or one whose native controller speaks an API no adapter covers, is coded on the muscle SDK (`omnicompass/muscle_sdk.py`), the contract every adapter in this book obeys: **observe, bound, request, execute, read back, receipt, restore**. The SDK adds a muscle; it never adds a brain. The governor proposes a setting; the SDK's runtime earns the authority to write it, verifies the write and writes the receipt. An engineer codes five functions and declares three contracts; everything else (the clamp, the gate, the snapshot, the restore, the receipt) is the runtime's.

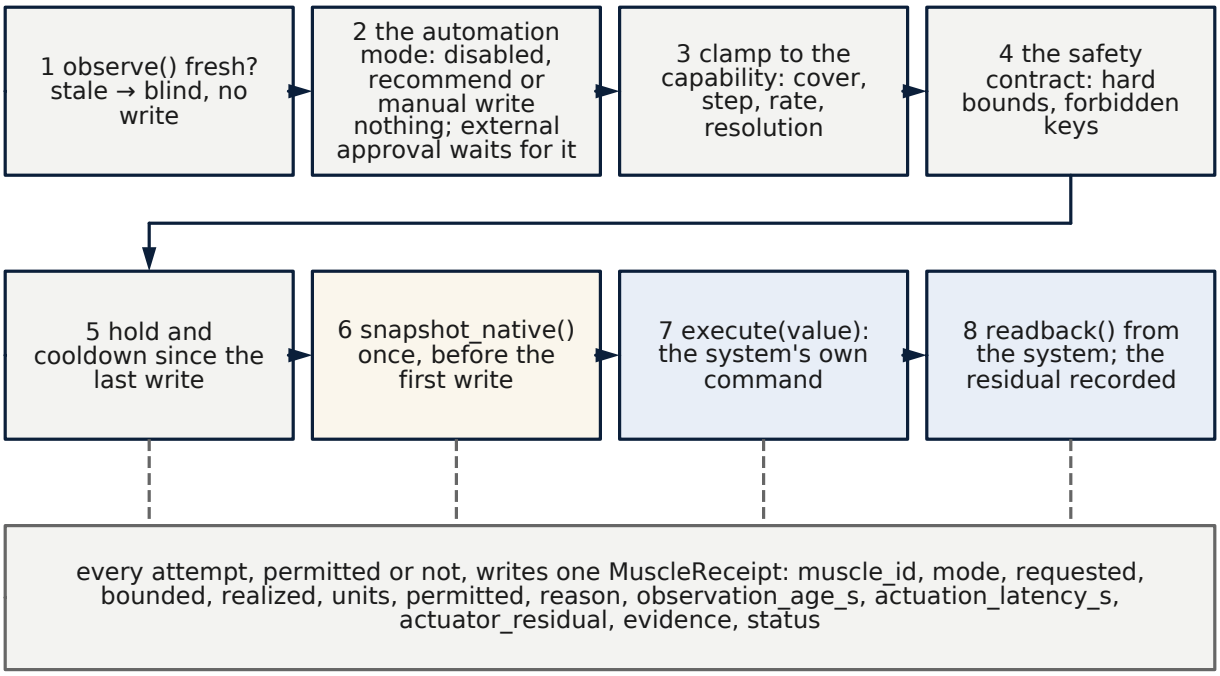
**The five functions** (the `MuscleAdapter` protocol, or `FunctionalMuscle` for a small connector): `observe()` returns the reading with its timestamp and quality (the wire out); `snapshot_native()` records the setting as found, once, before the first write; `execute(value)` writes one setting through the system's own command or API (the wire in; **WRITES**); `readback()` reads the setting back from the system, never from memory; `restore_native(snapshot)` puts the setting back as found. The adapter owns the vendor's API; the brain never sees it.

**The three contracts** (`MuscleSpec`): the **capability** (`CapabilityContract`: the cover's `minimum` and `maximum`, the `resolution`, the most one write may move (`max_step`) or the most a second may move (`max_rate_per_s`), the `units`, whether the knob is `reversible` and whether it reads back); the **timing** (`TimingContract`: the observation and decision periods, the actuation delay, the minimum hold and the cooldown between writes, the timeout, and `stale_after_s`, past which a reading is blind); the **safety** (`SafetyContract`: the `hard_min` and `hard_max` nothing may cross, `fail_closed`, a `safe_default`, and the `forbidden` context keys, such as a security hold, under which no write is permitted). The spec also names the muscle (`muscle_id`, `plant_id`), its class (`resource_type`, one of the standard classes below), its `automation` mode and its evidence class (`Evidence`: D design, S our own models, L live software, P physical meter, C commercial).

**The automation modes**, from least to most authority: `disabled`; `recommend` (every command computed and receipted, nothing written: the watch level of section 7.3); `manual`; `external_approval` (a write only with `approved=True` from your change process); `automatic`. A muscle starts at `recommend` and is promoted exactly as chapter 7's levels promote a stack.

**What the runtime does with each command** (`MuscleRuntime.command`), in order: a stale or invalid observation blocks the write and marks the muscle blind (section 6.3); the mode is checked; the request is clamped to the capability (cover, step, rate, resolution); the safety contract is asked (`outside_hard_bounds`, `forbidden:<key>`); the hold and the cooldown are checked

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.




The five functions an engineer codes: observe, snapshot_native, execute, readback, restore_native. The three contracts declared: capability, timing, safety. The runtime does the rest, in this order, for every command.

**Figure 14.** The muscle SDK's runtime: what happens to one command. The eight steps the runtime takes, in order, for every value the governor proposes: the observation's freshness (stale means blind, no write), the automation mode (recommend writes nothing), the clamp to the capability contract, the safety contract, the hold and the cooldown since the last write, the snapshot taken once before the first write, the execute through the system's own command, and the read-back with its residual. Every attempt, permitted or not, writes one MuscleReceipt with the fields shown.

(`hold_or_cooldown`); only then is the native snapshot taken, once, and the write made, and where the capability says the knob reads back, the setting is read back and the residual (realized minus bounded) recorded. Every attempt, permitted or not, writes one receipt (`MuscleReceipt`): `muscle_id`, `mode`, `requested`, `bounded`, `realized`, `units`, `permitted`, `reason`, `observation_age_s`, `actuation_latency_s`, `actuator_residual`, `evidence`, `status`. `MuscleRegistry` holds every muscle of a body, refuses a duplicate name, answers `catalog()` (every muscle with its cover, mode, evidence and status) and `restore_all()`, which hands every muscle back in the reverse of the order they were registered.

**A worked example.** The power limit of one graphics card, through the card's own command, first in `recommend`:

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
import json, subprocess
from omnicompass.muscle_sdk import (
    AutomationMode, CapabilityContract, Evidence, FunctionalMuscle,
    MuscleRegistry, MuscleRuntime, MuscleSpec, Observation,
    SafetyContract, TimingContract)

def read_limit():
    # the wire out: the setting as the card reports it now
    out = subprocess.run(
        ["nvidia-smi", "-i", "0", "--query-gpu=power.limit",
         "--format=csv,noheader,nounits"],
        capture_output=True, text=True, check=True).stdout
    return float(out.strip())

def write_limit(w):
    # the wire in: one setting, the card's own command (WRITES)
    subprocess.run(["nvidia-smi", "-i", "0", "-pl", str(int(w))],
                   check=True)

spec = MuscleSpec(
    muscle_id="gpu0.power_limit", plant_id="card-0",
    resource_type="gpu.power_limit", adapter_version="1.0",
    capability=CapabilityContract(minimum=100.0, maximum=300.0,
                                  resolution=1.0, max_step=25.0,
                                  units="W"),
    timing=TimingContract(observation_period_s=2.0,
                          decision_period_s=2.0, minimum_hold_s=10.0,
                          stale_after_s=6.0),
    safety=SafetyContract(hard_min=100.0, hard_max=300.0,
                          forbidden=("security_block",)),
    automation=AutomationMode.RECOMMEND,
    evidence=Evidence.LIVE_SOFTWARE,
    native_owner="the card's firmware")
muscle = FunctionalMuscle(
    spec, observe=lambda: Observation(read_limit()),
    execute=write_limit, readback=read_limit,
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    snapshot=read_limit, restore=write_limit)
audit = open("/var/log/omni/muscles.jsonl", "a")
registry = MuscleRegistry()
registry.register(MuscleRuntime(
    muscle, audit=lambda rec: audit.write(json.dumps(rec) + "\n")))

receipt = registry.command("gpu0.power_limit", 250.0,
                           context={"security_block": False})
# recommend: permitted False, reason "recommend", nothing written;
# the receipt is in the audit
registry.restore_all()
# every muscle back as found, in reverse order; each one receipted
```

Promoted to `AutomationMode.AUTOMATIC`, the same code writes: a request of 250 W from the 300 W found is bounded to 250 and read back (residual 0); a request of 200 W ten seconds later is bounded to 225 by `max_step`; a request inside the 10-second hold is refused (`hold_or_cooldown`); a request made while `security_block` is true is refused (`forbidden:security_block`) and nothing moves; `restore_all()` returns the card to the 300 W found at the start and its receipt says `restored`. These outcomes are the SDK's own tests (`tests/test_muscle_sdk.py`).

**The standard classes** (`STANDARD_MUSCLE_CLASSES`) name the knob a new adapter governs, so that the catalog and the receipts read the same across a company's stacks: `kubernetes.replicas`, `.cpu_request`, `.cpu_limit`, `.memory_request`, `.memory_limit`, `.hpa_target`, `.pod_placement`, `.node_pool`; `host.cpufreq`, `.cpu_quota`, `.cpuset`, `.memory`, `.io`; `gpu.power_limit`, `.clock_envelope`, `.mig`, `.admission`; `cloud.instance_count`, `.instance_size`, `.placement`, `.start_stop`, `.capacity_policy`; `storage.volume_size`, `.tier`, `.iops`, `.throughput`, `.placement`; `network.traffic_weight`, `.rate_limit`, `.bandwidth`, `.routing`; `application.concurrency`, `.worker_count`, `.queue_admission`, `.jvm_heap`, `.thread_pool`, `.batch`; `facility.cooling_setpoint`, `.rack_power`, `.site_power`. A class is a contract, not a claim that a connector for it has run live: each adapter earns its own evidence class, and Appendix E's wiring recipes say which have (`docs/wiring_recipes.csv`).

A muscle coded on the SDK is wired into a body as any other (chapter 2): one governor per body, the muscle's reading a gauge of that body, its receipts in the body's audit; the wire check of section 5.4 applies to it unchanged (snapshot, write, read-back, restore, nothing left behind), and it enters service through the levels of chapter 7, watched first.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 6

# The Rules the Wires Obey

Every muscle is wired both ways: a sensory wire in (its meters) and a motor wire out (its knob), with the read-back closing the loop. Between them, the governor decides how much authority each muscle has at each moment, by four rules that an installer must know because they decide what a receipt can show:

- **Expand is always allowed** (except under a security hold, Chapter 10): adding capacity, power, cooling or protection never waits.
- **Contract needs calm and a clean service record:** giving anything back is allowed only while the muscle is calm and the service has been inside its line for the clear time, 180 s (`--slo-clear-s`); a continuous muscle gives back at most its calm share of its surplus per decision, a discrete one one unit at a time, through a release gate (section 6.1).
- **Fail up:** while the service is past the wall (95% of the band) the knob returns at once to full capacity, the operator's own setting or the cover's top, whichever the muscle's direction asks.
- **Blind means hold:** if a sense goes stale or unreadable, nothing is given back until it returns (section 6.3).

The rules are asymmetric on purpose. Adding is cheap to undo and expensive to delay, so it is never gated beyond the security hold. Taking back is cheap to delay and expensive to get wrong, so it is gated three ways: by calm, by the service record (inside the line for the last 180 s, not merely at this decision), and by the release gate, whose reasons are written into the audit every time it holds.

"Two-way" is a statement about wiring, and it is checked, not assumed. Each muscle's plug carries exactly one sensory wire and one motor wire, and the wire check that must pass before any write (section 5.4) proves that the wires move: a value written through the motor wire is taken by the device and read back as the value sent, and the snapshot is restored and read back at the end. No governor writes a knob whose wire check has not passed, because a knob that can be written but not read back cannot be handed back with proof. What the wire check does not prove, section 5.5 says plainly: that the governor is reading the right gauge for the knob; that is what the watching arm is for. The receipts of every run carry, for every write, the value sent and the value the device reported, side by side.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

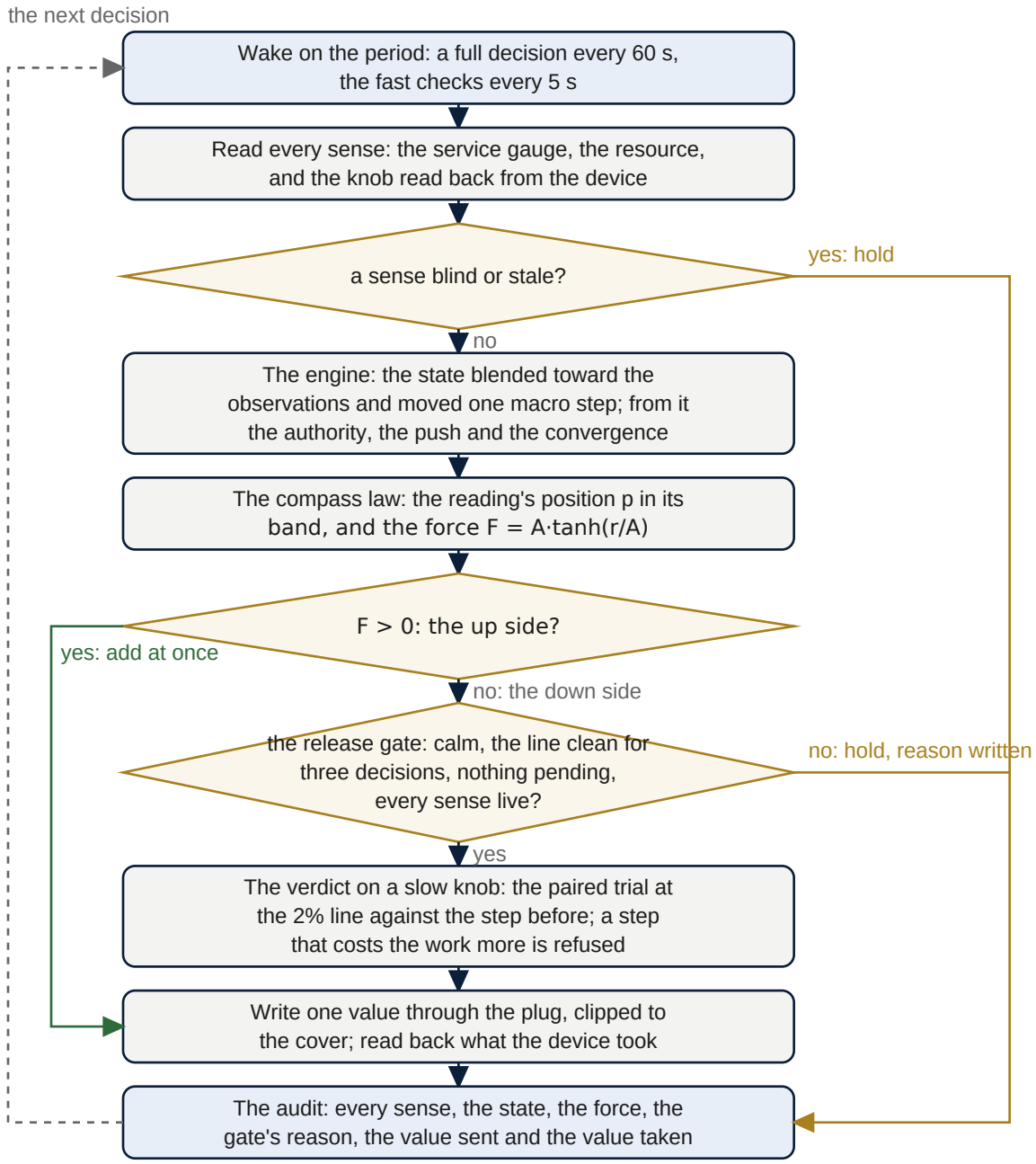



**Figure 15.** One decision of the governor. It wakes on its period, reads every sense, and holds if any sense is blind or stale: nothing is given back, and adding capacity is still allowed. The engine blends its state toward the observations and moves one macro step; the compass law turns the reading's position into a force. A positive force adds capacity at once, gated by nothing but the security hold.

A negative force must pass the release gate (calm, the line clean for three decisions, nothing pending, every sense live) and, on a slow knob, the verdict's paired trial at the 2% line. What survives is one write through the plug, read back, and one line in the audit that records every sense, the state, the force, the gate's reason and the value sent and taken, whether or not anything was written.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

## 6.1 The release gate

On Kubernetes the **release gate** lets one machine be given back only when every one of the following holds, and writes the first reason that fails into the audit, so a reader can see why a machine was kept:

1. more than one machine is in service (the floor of two is never crossed);
2. no pod is waiting for a place;
3. the pods are not scaling up;
4. the service is not past its line now, and its feed is live;
5. the remaining machines would run at or under the utilisation target ρ: the work in use divided by the capacity of the machines that would remain, against the engine's own demand clipped to between 50% and 95%;
6. the machine organ's own engine state grants contraction (its calm at or over 0.7, its line clean, no security hold);
7. every sense is live;
8. the last machine command has landed (no new order goes on top of one the cluster has not yet carried out);
9. a minute has passed since the last release.

The gate is asked only when the compass law itself asks for a machine back (its force past the release threshold, the reading under the centre, the verdict's deepest allowed step covering the count), never on the max and never during a trial; and cruise overrides it (while work waits for a place, every machine stays in service). One machine per decision is the rule in the slow, reversible direction; past the wall, one machine up at once. The same shape holds on every discrete muscle: a consumer, a chunk of a buffer pool, a notch of a cache ceiling, a tap. The gate's records read, in the governor's own words, "release permitted" (with the utilisation the remaining machines would carry), "no contraction authority" (a calm system whose state has not yet earned the right to give back), or a list of reasons at once, such as "latency breached now; a sense is blind". A machine kept in service always has a stated cause.

## 6.2 The snapshot, the read-back and the one-writer rule

The governor knows where its own levers are. The rule begins with the **snapshot**: the governor reads every knob it may move once, at the start, before its first decision, and that reading is the operator's setting for the whole run. Every reset, every hand-back and every kill returns the knob to the snapshot, never to a value computed later, and the receipt records the snapshot beside the final read-back so a reader can check the two are equal. The rule continues with the **read-back** after every write: the plug sends a value, waits for the device to take it, and reads the device's own report of what it holds. Where the device takes time to apply a setting, the plug knows that and does not mistake the transition for a foreign hand: MySQL's buffer pool is resized by the server in chunks over several seconds and reported as it goes, so the plug reads the server's own resize status, treats a pool caught mid-resize as its own write in progress, and waits for the server to report

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

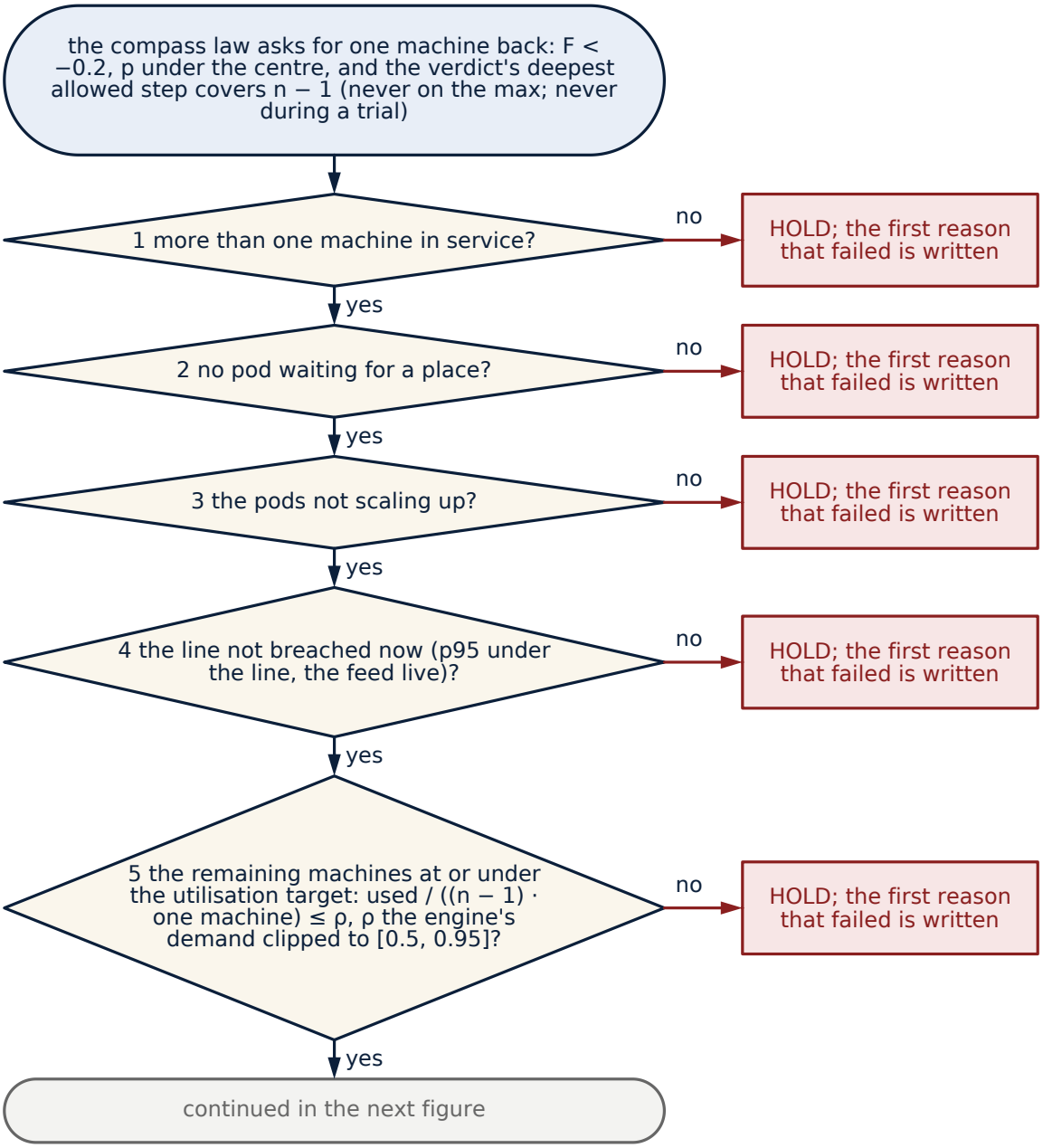



**Figure 16.** Flowchart: the release gate, reasons 1 to 5. The compass law asks for one machine back (its force past the release threshold, the reading under the centre, the verdict's deepest allowed step covering the count; never on the max, never during a trial). The gate then tests, in the governor's own order: more than one machine in service; no pod waiting for a place; the pods not scaling up; the line not breached now, the feed live; the remaining machines at or under the utilisation target ρ (the engine's demand, clipped to between 50% and 95%).

The first reason that fails holds the machine and is written in the audit. Continued in the next figure.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

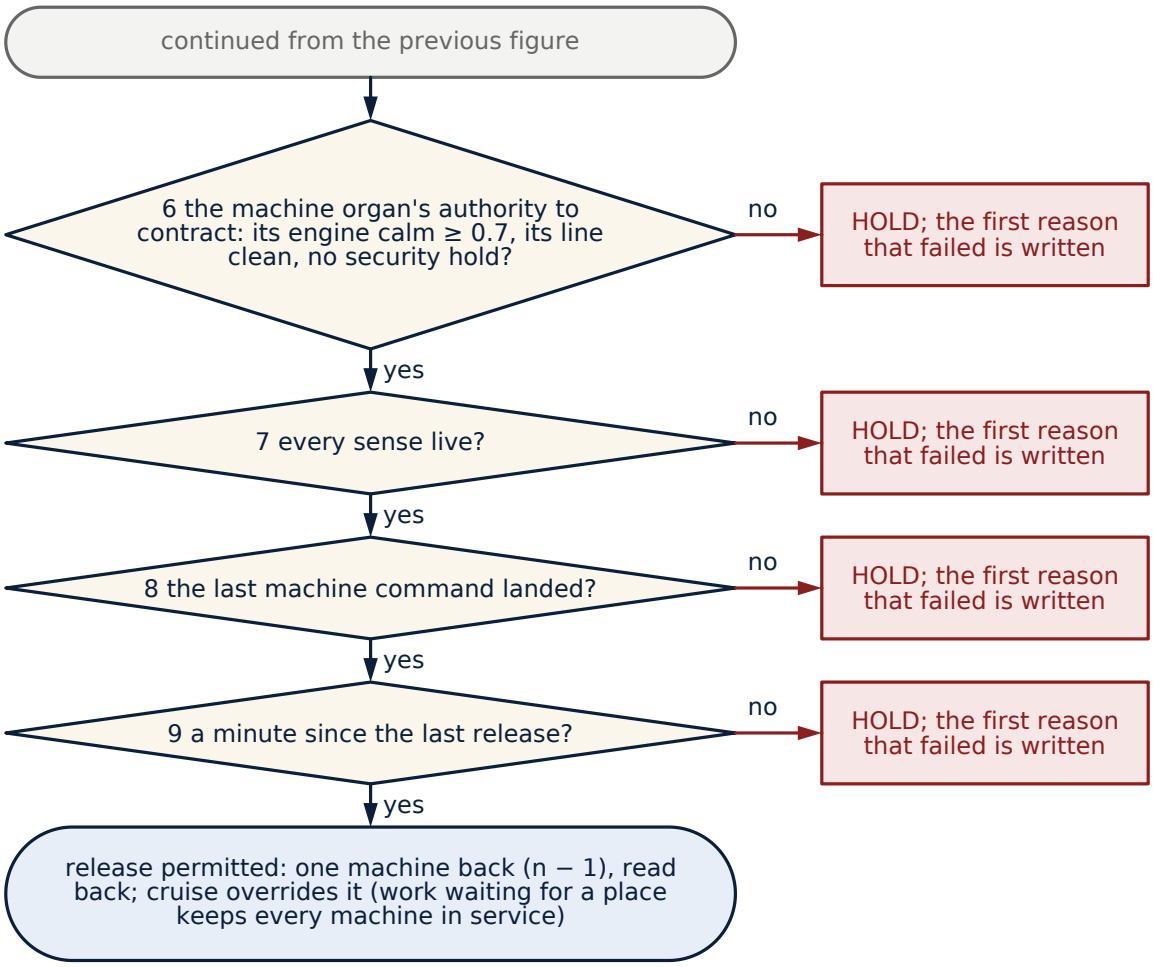



**Figure 17.** Flowchart: the release gate, reasons 6 to 9 and the release. The machine organ's own authority to contract (its engine calm at or over 0.7, its line clean, no security hold), every sense live, the last machine command landed, and a minute since the last release; only then is one machine given back and read back. Cruise overrides the release: while work waits for a place, every machine stays in service.

completion before it judges the write. The same shape holds on the autoscaler's bounds, Redis's memory ceiling and MongoDB's cache size: the value is read from the device's own console, and only a value the governor neither wrote nor is waiting on is read as someone else's.

If a lever is found at a value Omni-Compass did not write, someone else owns it: the governor stops writing that lever and leaves it alone, because restoring over it would fight the new owner. An operator who changed a knob by hand meant to, and a second controller that changed it is a conflict to be reported, not won. The lever is left where it was found, the audit records the value and the time, Omni-Compass goes on supervising the other levers, and its exit code says so. A reader of the receipts who finds such a record knows at once that two hands were on the stack, and the paired run that contains it is read with that in mind. For an installer this rule has one consequence above all: **a knob may have only one writer besides its native controller**. A runbook,

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

a cron job, a building schedule or a second tool that writes the same setting must be stopped or handed the knob before Omni-Compass is granted it.

## 6.3 Blind means hold, and fail up

A sense that goes stale or unreadable is not read as calm. It is read as **blind**, and while any sense is blind nothing is given back; the knob holds, or, if the service was past its line when the sense went blind, returns to full capacity at once. Omni-Compass never acts on a reading it does not have. An API that answers a read with a server error produces a held decision, logged with the reason the API gave, and the next decision is taken on schedule when the API answers again. The same holds for a response-time feed that pauses: the governor holds through the gap and resumes when the feed returns. A stale or missing reading produces a hold, never a move, and the hold is in the record.

For an installer, the consequence is that the freshness of every feed is part of the wiring. A latency file that is written late, a meter that is polled less often than the governor decides, or an exporter that caches its readings will read as blind and keep the governor holding; the audit says so (`gate: a sense is blind`), and the fix is in the feed, never in the governor.

## 6.4 One body, one governor

The alternative to one governor is a controller per layer, each tuned alone: an autoscaler that adds pods while a power cap is trying to hold the machines down, a cooling loop that chases heat the clock governor is about to remove. These controllers fight because none of them knows the others' intent. One governor that sets every knob from one reading of the body cannot fight itself: the step on each lever is decided from the same reading of the same body at the same moment, and a step that would push another part of the body past its cushion is refused. Nothing anywhere is pushed into a LOSS to move one thing to a GAIN; one trial runs at a time inside a body, so every receipt can name the step that produced it.

This is why chapter 2 asks for every muscle of a body, and every resource the muscles share, to be wired into the same governor. Where a shared resource is left out, the governor cannot see the coupling, and no rule in this chapter can respect a limit the wires do not carry.

## 6.5 The rules of the engine that stands, as an operator meets them

The engine that stands in this edition is **omni-v25** (`python3 tools/omni_version.py` prints it; chapter 13); the receipts quoted here ran on omni-v7, whose rules omni-v25 carries (omni-v8, omni-v9 and omni-v10 changed one wire, the CityLearn battery, and the interval table; omni-v11 added one guard to the Kubernetes governor; omni-v12 holds a raise the cluster's admission refused until something changes, and the swarm's wire acts on a neighbour at once; omni-v13 hands the CPU limits it gave back to the operator's on a shared host at its top while another workload there has room of its own, and counts every other pod by its own limit; omni-v14 runs the operator's own commands without a shell unless they hold a pipe, a redirect or a variable, and takes `--native` to leave any muscle where native runs it; omni-v15 asks for the power cap's

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

trim only while the response line has been clean for the governor's own clear time, its `--slo-clear-s`, so the request cannot switch back and forth at one threshold, and holds every wire in its fingerprint; omni-v16 keeps every time the governor counts in seconds, so `--interval` changes only how often it decides, and omni-v17 makes the beat a dial, any beat from 5 to 120 s on the command line (the factory's 60 s from omni-v17 to omni-v23, the beat for its place from omni-v24); omni-v18 carries Omni direct, Omni-Compass run on the muscles with no Kubernetes beneath it (`omni_orchestrator/`), as part of the engine, so its three arms (native, Omni on native, Omni direct) run from one fingerprint; omni-v19 takes `--own-replicas namespace/name,...` (or `OMNI_OWN_REPLICAS`), Omni-Compass by itself on Kubernetes: for a workload no autoscaler governs, Omni-Compass sets its replica count itself through the workload's own scale (the grant: `get`, `patch` and `update` on `deployments/scale` by name, `deploy/rbac-own-pods.yaml`, chapter 15), every `--own-every-s` (15 s), its target and range read from the workload's `omnicompass.io/target-utilization` and `omnicompass.io/replica-range` annotations (else `--own-target`), never below the workload's own count at the max (the tune-down may go to the range's floor), the count recorded on the workload and handed back by the reset; a workload an autoscaler governs is left to it, and `--native replicas` leaves the muscle native; and the conveyance runs every `--convey-every-s` (15 s) whichever check falls due, a quiet deployment is scaled to zero after `--coldstart-idle-s` (180 s), and a step that runs past its slot is skipped, never run in a burst, with `behind_s` in the audit); omni-v20 fixes the five data stores' wires at their causes; omni-v22 brings the five data stores' wires to the max, each spend gated on the store's own account of what a miss costs and the host's room, never below native (`tools/store_brain.py`, `docs/OMNI_V22.md`); and omni-v21 hands a service's autoscaler target back to native while its response time stays over its line under Omni-Compass's target and native's own rule would hold it lower, keeping it where native is not stronger (each change in the audit as `"immune"`, with the replicas native's rule would ask for beside the replicas running); omni-v23 hands a workload's replica count back to native's own autoscaler when the cluster refuses three of Omni-Compass's own writes of its scale in a row (`"own_replicas"`, `"handed_back"` in the audit); and omni-v24 takes its factory beat from the place it runs in, read from the cluster's own nodes at its start: a decision every 10 s on site (kind, your own machines, a node with no providerID) and every 60 s through a cloud (a node whose providerID names a public cloud's machine), 60 s when the nodes cannot be read, a beat you set always kept (the audit's first record, `"factory_beat"`, names the place, the beat and why). The hand-back shows in the audit as `"convey": "handed back"`, with the host's busy share on each write; a muscle left native shows as `"native_muscles"` at the start and `"held"` on each write it withholds. The guard shows in the audit: when the cluster's reading shows no machine in service (every worker cordoned, tainted NoSchedule or unreachable at once), the decision is recorded as `held: the cluster's reading shows no machine in service; nothing decided, nothing written`, and the governor takes up again at the first reading with machines. A held decision is never an error and needs no action from anyone; several in a row point at the cluster's nodes, not at Omni-Compass. Its mechanism is the Owner's Manual's; five of its rules show in the wiring, the settings and the audit, and are stated here so that a line in the audit is read for what it is.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

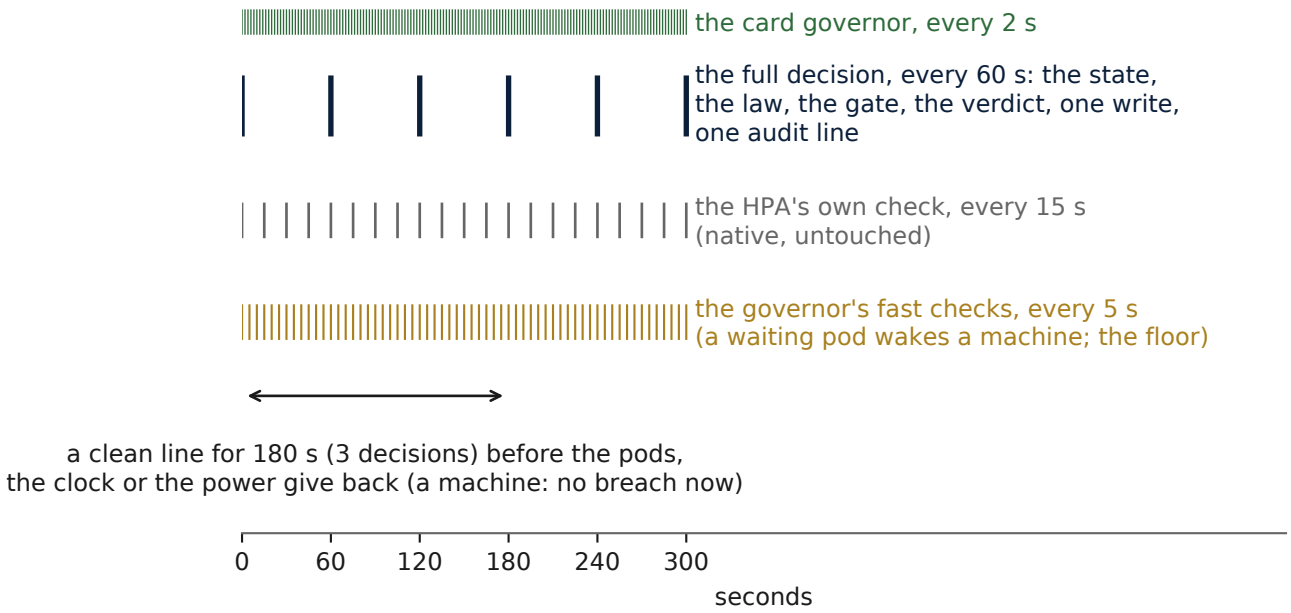



**Figure 18.** Timing diagram: the governor's clocks against the native controllers' own, over five minutes. The card governor decides every 2 s; the Kubernetes governor's fast checks run every 5 s (a waiting pod wakes a machine; the floor); native's own autoscaler keeps its 15-second check underneath, untouched; the full decision (the state, the law, the gate, the verdict, one write, one audit line) runs every 15 s, and nothing is given back until the line has been clean for 180 s, twelve decisions.

**Conveyance obeys the namespace's own quota.** Where the governor conveys a machine's idle processor to the serving pods on it (`--cap-deployments`, level 3), every raise of a pod's CPU limit is bounded by the namespace's ResourceQuota as Kubernetes reads it (hard and used; the stricter of the quota's spec and its status), and a raise the quota would not admit is not written. The governor never raises, lowers or deletes a quota. Where its role cannot read the quotas, the budget is **no budget known**, not zero: the raise proceeds as the plug has always made it, Kubernetes' own admission refuses any write over a quota the governor cannot see, the plug reads the refusal back, and the audit says so once a pass (`quota: no budget known: the raise proceeds, Kubernetes admission is the authority`, with the namespace and the reason). An operator who wants the budget bounded before the write grants the governor's role `get` and `list` on `resourcequotas`; one who does not loses nothing but that line.

**On a shared host, no limit is raised while the machine is at its top.** Where several nodes are containers on one machine (a kind cluster, a laboratory host), the governor raises no pod CPU limit while that machine's measured busy share is at or over 75%, or cannot be read; limits already given stay; lowering is never held. Each hold is one audit record, `convey: held`, with the machines held, the host's busy share and whether the host is shared. On separate machines the guard never binds. It is the same guard the replica room obeys before it adds a replica on a shared host.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

**The clocks are in seconds.** Every rule that waits is stated in seconds and keeps its time at any decision period: the clear time after a breach (`--slo-clear-s`, 180 s), the cruise wait (`--cruise-after-s`, 120 s), the strict window (`--strict-window-s`, 300 s), the verdict's spacing, recheck and trial (`--verdict-every-s` 600 s, `--verdict-recheck-s` 7,200 s, `--verdict-trial-s` 1,200 s) and the reflex window (`--reflex-window-s`, 30 s). Each is converted to a count of decisions at the running period, n = max(1, round(seconds / period)), so a governor deciding every 15 s waits the same minutes as one deciding every 60 s; the cruise wait is never fewer than two decisions, because work that waits decision after decision takes two readings to see (at a beat of 90 or 120 s its 120 s would round to one). The compass moves an autoscaler's target by the same amount a minute at any period, and the node pool gives back at most one machine a minute. The full decision comes at the factory beat for its place, every 10 s on site and every 60 s through a cloud, and the beat is a dial (section 4.4): any value from 5 to 120 s may be set with `--interval`, 15 s among them, the pace of native's own autoscaler; the fast checks run every 5 s at every beat, and native's own autoscaler keeps its own 15-second check untouched.

**The verdict reads a trial in the words of a receipt.** Before a knob moves one step further from native, the step runs as a trial and its cost per piece of work is compared with the cost at the reference, on the logarithm of each piece's cost, with a 95% interval. The interval wholly above zero is a **LOSS** and the step is undone whatever the allowance says; wholly below is a **GAIN**; across zero is **STEADY**; a GAIN or STEADY step is kept only if both the median and the mean of the logarithms are inside the allowance (`--allow`, 2%); while the interval is wider than the allowance the trial measures on, up to three times its sample count, before it decides. Every judgement is in the audit as `reading` (`GAIN`, `STEADY`, `LOSS` or `past the allowance`) and `interval_ln`, so an operator can see why a machine stayed given back or came back (Appendix L, `verdict`).

**The cloud runs the whole engine.** Until 31 October one flag, `--give-back-only`, turned off every pod, processor, power, clock, cooling and batch actuator in a cloud and left the node pool as the only knob (the founder's order of 14 October). His order of 31 October struck it: no hard rule; the cloud install file (`deploy/install/omni-compass-cloud.yaml`) passes the max with every muscle, nothing passes the flag, and it leaves the governor with the next engine version. Chapter 20 is written on the cloud at the max, the bill judged beside the work.

None of these is a setting an operator tunes (Appendix K lists each with its shipped value). They are stated so that the audit lines they write are read as the rules they are, and so that a reading taken on this engine is never set beside one taken on another (chapter 13).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART THREE

# Wiring It onto Your Stack, Step by Step

*The eight levels, from watching to governing the site, and the wiring of each kind of stack.*

CHAPTER 7

# Before You Start, and the Eight Levels

## 7.1 What your system needs

| You run | You need |
|---|---|
| Kubernetes (vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind) | Kubernetes 1.34 or newer (in-place pod resize); metrics-server (`kubectl top nodes` works); an HPA with a CPU target on each governed service; a readiness probe and a short preStop pause on each service |
| A response-time feed (strongly recommended at every level) | a CSV per service, `elapsed_seconds,latency_ms,ok`, written continuously; `scripts/latency_probe.py` writes one against any HTTP endpoint |
| NVIDIA GPUs | a driver with `nvidia-smi`; root (or the capability to run `nvidia-smi -pl` and `-lgc`); on a VM, full GPU passthrough |
| CPU power control | Linux cpufreq with the schedutil governor; RAPL for package watts; usually bare metal |
| Site power and cooling | a command that prints site watts, and a command that sets the supply-air setpoint through your building management system |

**Why each requirement is there.** Kubernetes 1.34 or newer is required because the conveyance muscle resizes a pod's CPU limit in place, which older versions do only by restarting the pod, and a restart is a

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

disruption Omni-Compass never causes. metrics-server is the HPA's own eyes and the governor's: without it neither knows what the pods use. An HPA with a CPU target on each governed service is the native controller Omni-Compass sits on; a service without one has nothing for level 2 to move. A readiness probe and a short preStop pause are what make a drain invisible to clients: the Service stops sending requests to a pod before it stops, and kube-proxy sends them to another ready endpoint. The response-time feed is the one wire a cluster does not already have: Kubernetes knows what its pods use, not what its users wait; the compass needs the second, and `scripts/latency_probe.py` writes it against any HTTP endpoint every five seconds. On the GPU, root is needed only for the two `nvidia-smi` writes, and a VM must pass the whole card through or the driver refuses to lock clocks (the wire check's step 3 is where that shows).

## 7.2 Get the software and check it

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass-LLC
    ↪ .git
cd The-Omni-Compass-LLC
pip install -r requirements.txt
python3 verify.py                       # every check; ends with VERIF
    ↪ ICATION: PASS
```

Build the container image once:

```
docker build -f deploy/Dockerfile -t <your-registry>/omni-compass:<tag
    ↪ > .
docker push <your-registry>/omni-compass:<tag>
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

What `verify.py` checks is worth knowing before it is run, because its last line, `VERIFICATION: PASS` or `FAIL`, is the only thing an installer has to read. It runs every test in the repository (the plugs, the readings rule, every stack's harness against a stand-in server), checks every fingerprinted file of the engine against its registry so that a changed file fails before anything is installed, compares the release's files with its manifest, and checks that every path the documents name exists. It takes a few minutes on a laptop and needs no network beyond the clone. GitHub runs the same script on every push under Python 3.12, and each run is listed under the repository's Actions tab with the commit it checked; install only a commit whose run passed.

To know which engine a checkout carries, ask it:

```
python3 tools/omni_version.py                 # the engine version (f
    ↪ or example omni-v11), or every file that differs
python3 tools/omni_version.py --commit <sha>   # the same question of
    ↪ any pushed commit
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Every receipt names the commit it ran on, and the three-run tools refuse to combine runs from different engine versions (chapter 13). No receipt is ever read across engine versions, and the command above is how an installer checks that for themselves.

## 7.3 The levels

Take them in order. Each has a pass condition; go to the next level only when it holds. The OFF switch works at every level.

The levels exist because a governor earns authority the way a new operator does: by watching first, then by touching the setting that is cheapest to undo, then the next. Level 0 touches nothing and proves the software is the software (`verify.py` checks every sealed file's fingerprint and runs every test). Level 1 watches a live cluster for as long as the operator likes and logs every decision it would have taken, so the operators can read those decisions against what they would have done; nothing passes level 1 until they agree with its readings. Level 2 moves the HPA target, which the HPA itself corrects within a minute if the governor is wrong, and raises replica floors ahead of bursts; it is the first level that can show a gain and the first that can show a loss, and the paired receipts of chapter 11 are its pass condition. Level 3 moves machines, the slowest lever, through the release gate and the verdict; it is where the bill moves and where the most care is spent. Level 4 hands Omni-Compass the replica decision itself, with the HPA kept as the fast reflex upward; it is an option, not a step on the way, and it is not part of the factory tune. Levels 5 to 7 are the GPU, the CPU and the site, each with its own wire check and its own pass. An installation may stop at any level and run there indefinitely; most will stop at level 3.

**Level 0 - Evaluate without touching anything.**

```
python3 verify.py
python3 tools/run_scale.py --runs 10 --scale 1 --out results/scale/eva
    ↪ l     # the seven organisms, modelled
python3 tools/run_gpu_card.py results/sim/gpu_two_wire/eval
    ↪        # the two-wire card, modelled
```

Pass: `VERIFICATION: PASS`. Nothing in your systems is touched. What you should see: `verify.py` ends `VERIFICATION: PASS`, and `python3 tools/omni_version.py` prints `omni-v25 (digest cf56fc0a7b707452, 65 files)`; the modelled run writes a receipt whose organism rows read GAIN on misses with power given beside them, as `results/realms_v5/REALMS_V5.md` does on the same seeds. Anything else means the copy is not the copy: chapter 33.

**Level 1 - Watch (read-only).**

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image
    ↪ line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

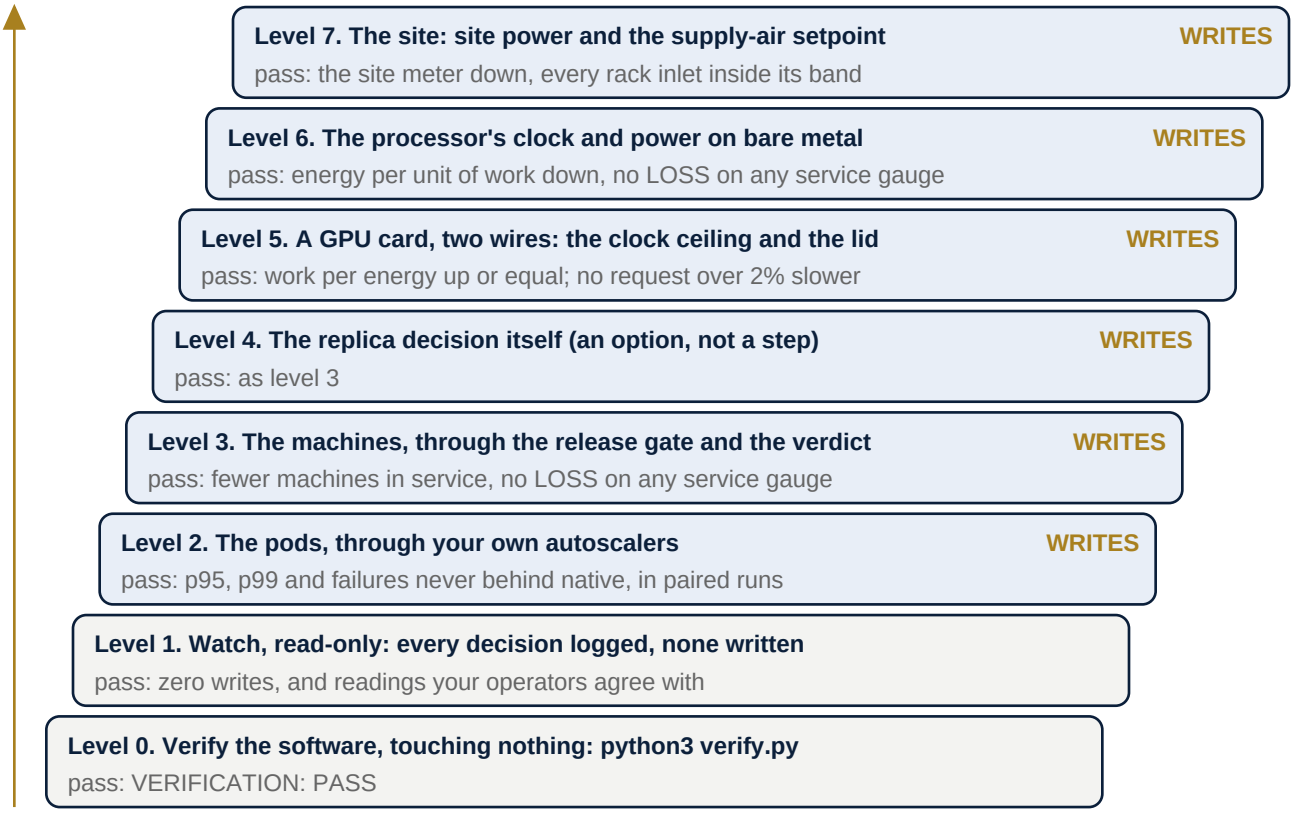



**Figure 19.** The eight levels of an installation, taken in order, each with its pass condition. Levels 0 and 1 write nothing; from level 2 the governor writes, and the wire check must pass before any level that writes. An installation may stop at any level and run there indefinitely.

The install file grants a read-only identity; `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write. Every decision it would take is logged with its reason. Pass: zero writes, and readings your operators agree with. What you should see: an audit that opens with its `factory_beat` record and then a `decision` record every 10 s on site or 60 s through a cloud (the factory beat for its place, or the beat you set) and no `write` record at all (`jq -c 'select(.write)' audit.jsonl` prints nothing); `authority.senses.blind` false for every sense once the feed is wired; `nodes_recommended` at or below `nodes_observed` while the service is calm and above it when it is not. A `write` here is a wiring fault (the identity can write: chapter 10); a sense that stays blind is a feed fault (section 6.3).

**Level 2 - The pods, on top of your autoscalers. WRITES.**

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

| Muscle | Switch | What it does |
|---|---|---|
| convey | `--latency-file` and `--cap-deployments ns/name` | gives each machine's idle CPU to the serving pods on it |
| rightsize | `--rightsize-deployments ns/name` | each pod's CPU request follows its measured use times (1 + headroom), in place |
| coldstart | `--coldstart-deployments ns/name --coldstart-signal ns/configmap` | scales a service to zero while no work waits, wakes it when work arrives |
| batch | `--batch` | admits held Jobs labelled `omnicompass.io/batch=true` when there is load and power headroom |
| batch pace | `--batch-pace` | pauses Jobs labelled `omnicompass.io/pausable=true` under power or heat stress, resumes them after |
| rollout guard | `--rollout-guard ns/name` | pauses a rollout while change is not permitted |
| contain | `--contain-namespaces ns --contain-cpu-m <m>` | holds an agent namespace to a CPU budget |
| security hold | `--security-configmap ns/name` | key `hold: "true"` blocks every expansion |

Your HPAs keep scaling as before; Omni-Compass sets their targets and raises floors ahead of bursts. Pass: p95, p99 and failed requests never behind native, over paired runs (chapter 11). What you should see: in the paired receipt (chapter 11), response time at the 95th percentile and time over the line reading GAIN or STEADY, failed requests STEADY, the autoscaler's replicas STEADY or power given. On our own services of this kind, rightsized with Goldilocks or scaled by KEDA, the 95th percentile read −52% to −75% and alert minutes −93% to −100% in all three of our runs on this engine (chapter 11, the reference bands). A LOSS on any service gauge at this level is read as a wiring fault first: the feed's freshness and its line (chapter 19), the sensed autoscaler's name, the window's length against the interval widths of chapter 11.

**Level 3 - The machines. WRITES.**

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"` and `--node-restore-cmd "<command>"` for the OFF switch. For the compass on the same levers, use `--law compass` (it reads the mean response time of the latency window in the band from a tenth of

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`--slo-ms` to `--slo-ms` and holds it at the centre of the band (section 4.4): 0.3 at the max and 0.4 at the tune-down; p95 at the line, a blind feed or a waiting pod adds capacity at once; the HPA target never goes above the operator's; at the max no machine the cluster had on its own is given back, and at the tune-down machines go back one at a time through the release gate and the paired trial).

| Your platform | The park/wake command |
|---|---|
| any cluster, bare metal, kind | `bash scripts/kind_nodepool.sh {n}` |
| Karpenter / EKS Auto Mode | the NodePool CPU limit at `{n}` times node CPU, parked nodes kept |
| Cluster Autoscaler node group | the group's desired size, scale-down through parking |
| OpenShift / OKD | the worker MachineSet replicas, parked |

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band. Pass: fewer machines in service, with no LOSS on any service gauge. What you should see: `verdict` records reading `trial`, then `step allowed` or `step refused`, each with its `reading`; machines in service and node-hours STEADY or GAIN, the bill STEADY or GAIN; no `refused` record from the scale command; after an OFF, the lease walking every machine back. A machine count that never falls is not a fault: it is the governor finding no machine it can give back without a LOSS, and the audit says which gate held (`node_gate.reason`).

**Level 4 - Omni-Compass decides; Kubernetes is the muscle. WRITES.** Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; the HPA stays as the fast reflex upward. Pass: as level 3. What you should see: as level 3, with the floors the governor set in the audit, and the HPA still raising replicas on a burst before the governor's next decision.

**Level 5 - A GPU box, two wires. WRITES.**

```
sudo python3 tools/gpu_wire_check.py --gpu 0                    # must
    ↪  end: WIRED RIGHT
# watch: computes both wires and writes neither
sudo python3 -m omni_controller.gpu_compass --mode watch --gpus 0 --au
    ↪ dit /var/log/omni/gpu.jsonl \
     --latency-file <feed> --slo-ms <p95 target> --floor-w <your lowes

    ↪ t watts>
# govern: writes the clock ceiling and the power limit
sudo python3 -m omni_controller.gpu_compass --mode cap --gpus 0 --audi
    ↪ t /var/log/omni/gpu.jsonl \
     --latency-file <feed> --slo-ms <p95 target> --floor-w <your lowes
    ↪ t watts>

# OFF: clocks reset, power limit back to the start, read back
sudo touch /tmp/omni-gpu-kill
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Every `--interval` seconds (default 2) it reads the card's own meters and the response times, and:

- **learns the card on its own** first: while the ceiling is at the top and the card is busy, its busy clock and busy draw; until 15 such readings are in, neither wire moves;
- **races while work waits**: at 95% utilization or more, the ceiling to the top and the lid to the start limit;
- **at the max** (`--dial max`, the default): the clock ceiling is never stepped below the top while the card serves, and the lid sits above the card's own busy draw, never over the start limit;
- **at the tune-down** (`--dial save`), it paces the slack: the mean response time of the last 5 s, in the band from a tenth of `--slo-ms` to `--slo-ms`; the ceiling moves in whole 15 MHz steps, never under the card's own busy clock, and the lid never under the card's own busy draw plus 10%, never over the start limit; and before the ceiling may go one step lower, a paired trial measures the card's own time on each request (the workload's `service_ms`) at the top and at that step, and the step is allowed only if it adds at most `--allow` (2%). Where no step passes, the ceiling stays at the top: the card runs as it does alone, and the audit says so (`verdict_state: left native`);
- **fails up** when p95 reaches `--slo-ms`, a request fails, the feed goes blind or the card reports a heat slowdown.

The settings are the card's own (its busy clock and busy draw as learned), the line (`--slo-ms`), the allowance (`--allow`) and the dial. The workload must write `service_ms` (the card's own time per request) in its latency file, as `tools/gpu_workload.py` does; without it no step is ever allowed and the card stays native. Pass: work per energy up or equal; requests served equal; no request more than 2% slower at the median, p95 and p99 at native or within 2%; every row of section 5.5 reads "wired right". What you should see: `tools/gpu_wire_check.py` ending WIRED RIGHT; a `snapshot` record, then a `decision` every 2 s; at the max no `clock_write` below the top while the card serves; at the tune-down a `verdict` for every step and `verdict_state: left native` where no step passed; `restored` with `ok: true` after the kill file (Appendix L.5). The receipt: work per energy GAIN or STEADY, requests served STEADY, the 95th percentile STEADY or inside the allowance.

**Level 6 - CPU clock and power. WRITES.** On bare metal, add to the controller: `--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil --rapl-cmd "<prints CPU package watts>"`. The OFF switch writes every policy's recorded maximum back exactly. Pass: energy per unit of work down, no LOSS on any service gauge. What you should see: one `cpu_pstate_snapshot`, then `cpu_pstate_write` records naming the cap and the envelope; energy per unit of work GAIN; every service gauge STEADY or GAIN; at OFF, `cpu_pstate_restore` putting every policy's ceiling back exactly (Appendix L.4).

**Level 7 - Site power and cooling. WRITES.** `--power-cmd "<prints site watts>" --site-limit-w <limit>` brings site power stress into the engine; `--cooling-cmd "<sets {c}>" --cooling-min-c 18 --cooling-max-c 27 --cooling-restore-c 22` lets it hold the supply-air setpoint in its band. CPU and GPU sharing one power budget, and groups of cards sharing a site

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

budget, run as models today. Batteries are designed as a muscle of the site, not yet wired (section 8.10). What you should see: site power stress entering the decisions (the `compass` record) and the cooling setpoint held inside its band; nothing written outside the band; at OFF, the setpoint back at `--cooling-restore-c`.

## 7.4 Wiring for superiority: watch first, then the paired trial on every knob

Every knob is wired out first and in last, and the wire in is connected only by a measurement the governor makes on the knob itself. The steps, for any stack:

1. **Wire out, everything.** Every muscle's readings reach the governor, one wire out per muscle. Nothing is written. This is watch mode (section 1.3): where every installation starts, and where any knob returns the moment it stops paying.
2. **Run the observation.** With the stack together and under its real or replayed demand, let the governor watch for at least one full cycle of that demand. In watch mode the governor learns the band of every reading, where the native controllers hold them, and which readings leave calm and when. Nothing is decided yet; the audit says what the governor saw.
3. **Let the paired trial decide each knob, one trial at a time.** Switch the knobs the operator is willing to have moved into trial mode. A knob still writes nothing until the governor has run a paired trial on it (section 1.2): the knob is held at the operator's setting for a reference, then one notch away, under the same traffic, and the step is allowed only if the muscle's own cost did not rise under the declared objective. The objective is the operator's one declaration per knob: the resource reading (the service, the resource and the processor time weighed together) or the service reading (work and speed alone). A refused step is not retried for a set time; an allowed step widens the allowance by one notch, and the next trial probes the next. The governor then moves the knob inside the allowance, never outside.
4. **Keep checking.** The trials recur. A knob whose last step stops paying when the demand changes is refused on its recheck and the allowance narrows back; a knob that never passes a trial stays native for good, which is the right answer for that knob and costs nothing. New noise in the system is answered the same way: the governor reacts inside the allowance, and the allowance itself is re-earned by measurement.
5. **Read the trials, muscle by muscle.** The audit carries the state of every knob every second (`verdict_phase`, `verdict_direction`, `allowed_low`, `allowed_high`, the cost sample), and the arm's record sums it. An operator who wants one knob wired both ways and another watched only sets the objective on the first and leaves the second in watch.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

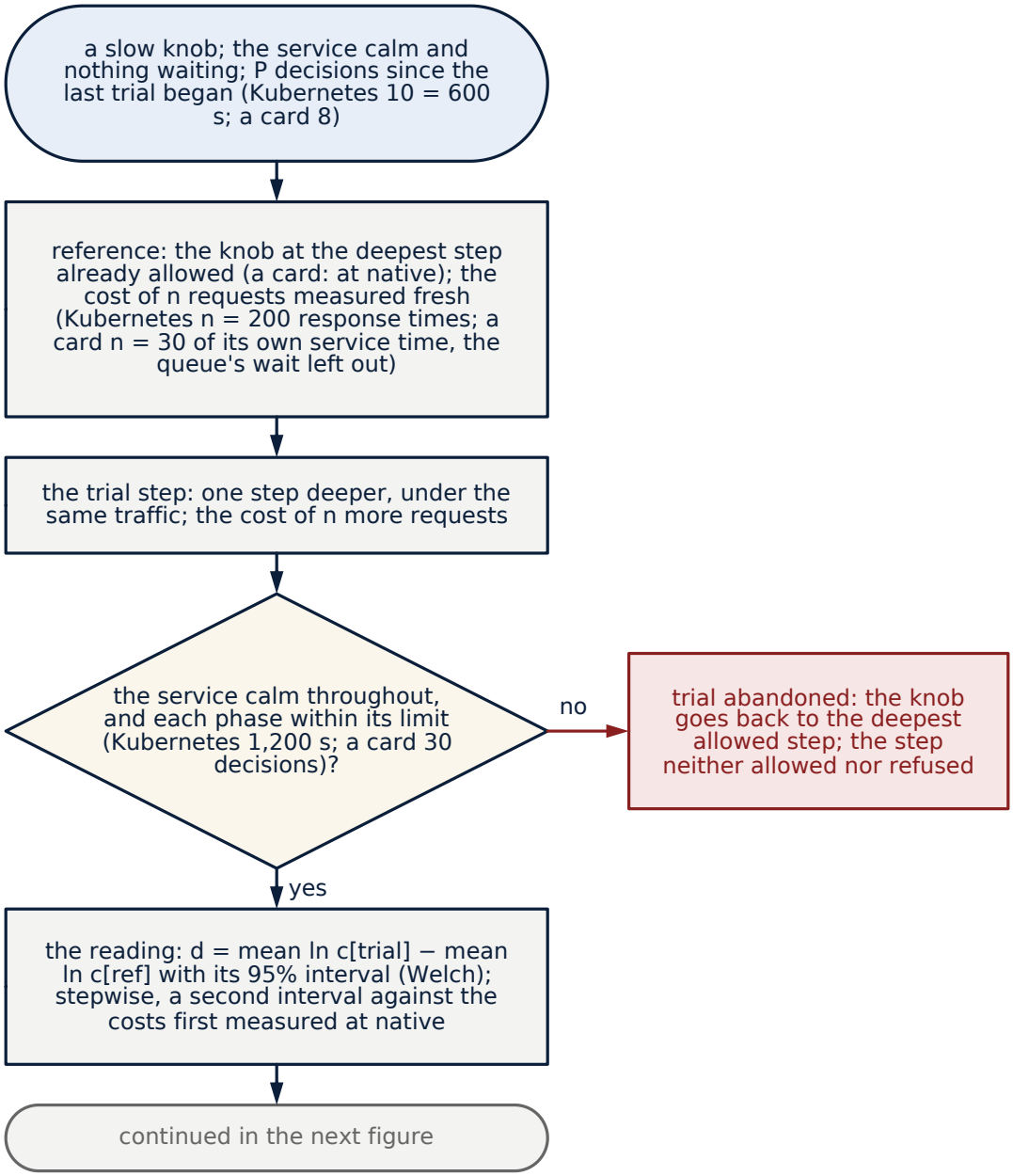



**Figure 20.** Flowchart: the verdict's trial on one step, from the reference to the reading. While the service is calm and nothing waits, and P after the last trial began (600 s on Kubernetes, 10 decisions at a 60 s beat, the factory's through a cloud, 60 at its 10 s on site; 8 decisions on a card), the reference is measured fresh at the deepest step already allowed (on a card, at native): n requests (200 response times on Kubernetes; 30 of a card's own service time).

One step deeper, under the same traffic, n more are the trial. A phase that sees the service leave calm or outlasts its limit (1,200 s on Kubernetes, 20 decisions at a 60 s beat, 120 at 10 s; 30 decisions on a card) is abandoned: the knob goes back, the step neither allowed nor refused. The reading is the 95% interval of the change in log cost. Continued in the next figure.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

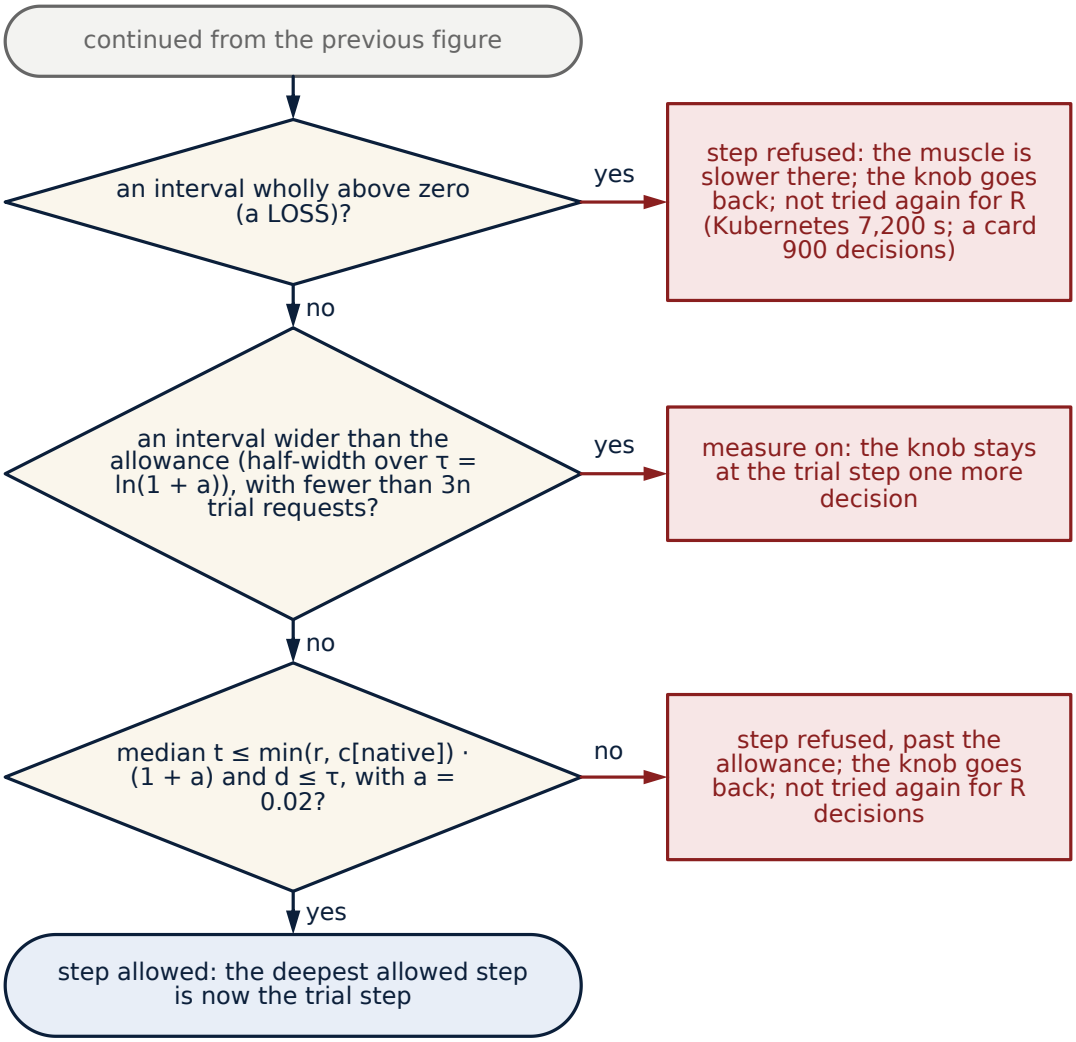



**Figure 21.** Flowchart: the verdict's trial on one step, from the reading to the step allowed or refused. An interval wholly above zero is a LOSS: the step is refused, the knob goes back, and the step is not tried again for R (7,200 s on Kubernetes, 120 decisions at a 60 s beat, 120 at 10 s; 900 decisions on a card). An interval wider than the allowance with fewer than 3n trial requests measures on. Otherwise the step is allowed only if the trial's median is within the allowance of the reference and of native and the mean change is within it too; else it is refused, past the allowance. An allowed step moves the deepest allowed step one deeper.

What this gives: no knob is ever written on faith. Where Omni-Compass can do more than the native controller on the muscle's own gauges, it will have shown it on that muscle before it moved it; where it cannot, the muscle runs by itself and Omni-Compass reads it. The body's result is then the sum of the knobs that earned it, not one turn of a single dial for the whole system. The live stacks of chapter 8 (the pooler, the broker, the cache, the storage engine's cache and the buffer pool) carry this trial on their knob, and on Kubernetes it guards every machine given back (section 1.2).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

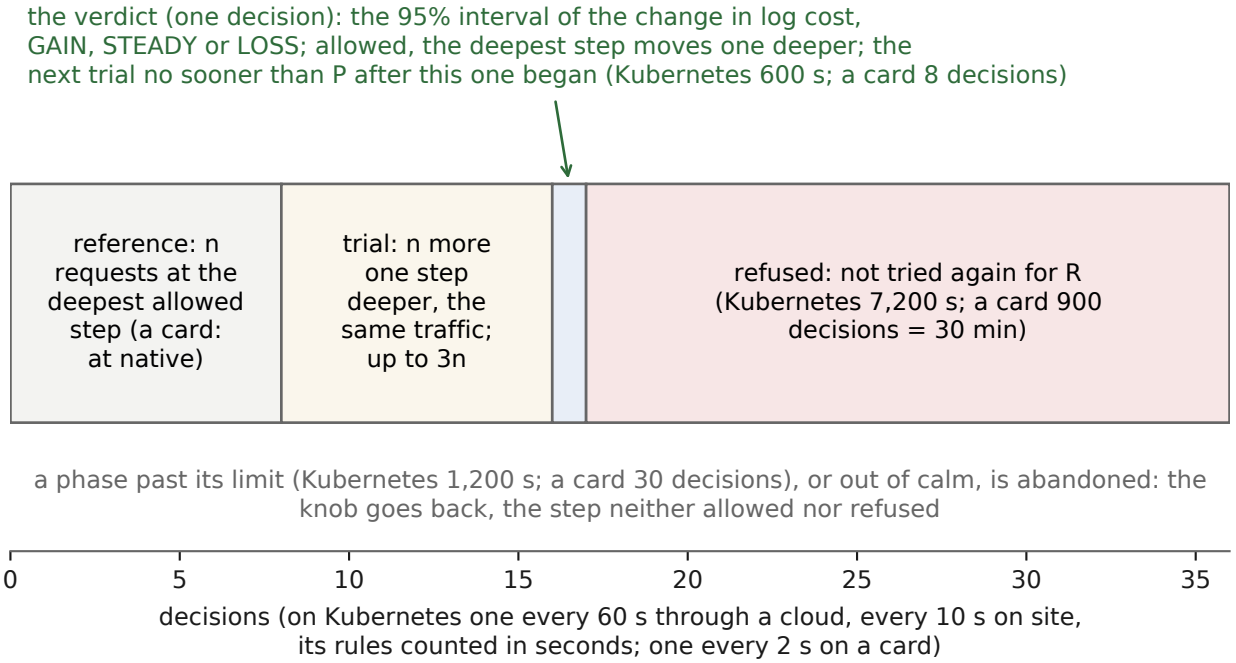



**Figure 22.** Timing diagram: a give-back trial from reference to verdict, in decisions. The reference (n requests at the deepest allowed step; on a card, at native), the trial (n more at one step deeper under the same traffic, measuring on up to 3n while the interval is wider than the allowance), the verdict in one decision, and, on a refusal, the quiet period in which the step is not tried again (7,200 s on Kubernetes, 120 decisions at a 60 s beat, 120 at 10 s; 900 decisions on a card, 30 min).

A phase that outlasts its limit (1,200 s on Kubernetes; 30 decisions on a card), or sees the service leave calm, is abandoned: the knob goes back, the step neither allowed nor refused.

CHAPTER 8

# Stack by Stack

Every wiring below is the same plug (section 5.1): one wire in (the stack's own meter), one wire out (one setting the stack already accepts), a snapshot taken once before the first write, every write read back, everything restored on OFF. Native is always the stack's own controller left as shipped; omni is that controller with Omni-Compass on top.

| Stack | Levels | Notes |
|---|---|---|
| Vanilla Kubernetes, kind, Rancher | 1-7 | as written |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Stack | Levels | Notes |
|---|---|---|
| Amazon EKS | 1-5 | machines via Cluster Autoscaler node group or Karpenter; CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances |
| Google GKE, Azure AKS | 1-5 | the provider's node-pool size as the park/wake command; on AKS the managed cluster autoscaler stays native and deletes the machines Omni-Compass idles (7.1) |
| Red Hat OpenShift / OKD | 1-5 | the same permissions through a Role; machines via the worker MachineSet |
| NVIDIA GPU servers without Kubernetes | 5 | the two-wire GPU governor alone |
| Bare-metal CPU servers | 6 | cpufreq and RAPL through sysfs |
| Slurm / HPC | 5 on the GPU nodes | a job-level Slurm connector is not built |
| Building management | 7 | through your BMS's command line or API, in the command templates |
| PostgreSQL behind PgBouncer (any database behind a pooler with a console) | one knob | the pool size through the pooler's own admin console (7.2) |
| A power grid's substation tap changer (pandapower, SimBench) | one knob | the tap position, one whole tap per move (7.3) |
| Buildings with batteries (CityLearn) | one knob a building | the battery's charge and discharge inside the simulator's own limits (7.3) |
| Robot arms (MuJoCo Menagerie) | one knob an axis | the joint speed where the axis has slack; never on an axis at its takt (7.3) |
| A message broker's consumer group (Apache Kafka) | one knob | the consumer count inside [1, partitions], read from the consumers' own records; the rebalance paid on every move (7.4) |
| A cache's memory ceiling (Redis) | one knob | `maxmemory` through the cache's own console, grown only while the cache is full (7.5) |
| Drone swarms (gym-pybullet-drones; PX4 and ArduPilot next) | one knob a drone | the cruise override inside the autopilot's limits; a separation wall; collisions void the cell (7.6) |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Stack | Levels | Notes |
|---|---|---|
| A database's storage-engine cache (MongoDB under YCSB) | one knob | the WiredTiger cache size through the server's own console, grown only while the cache is full (7.7) |

## 8.1 Managed Kubernetes on Azure: the bill as the gauge

On Azure Kubernetes Service the muscle is the same as on any cluster (chapter 7, levels 1 to 3) and one thing is new: the bill is real. The workflow `aks-metered` builds a fresh cluster for every arm the same way: a system pool of one machine, tainted so no workload lands on it (Azure's add-ons and the load generator live there; it is never measured and never governed), and one or several **work pools** under Azure's own cluster autoscaler, every pool labelled `omni-role=work`, every pool starting full. Native is that autoscaler alone. Omni on top is the same autoscaler with Omni-Compass idling the machines it gives back, which the autoscaler then deletes on its own schedule. The bill counts every work machine that exists, every 15 seconds, priced at Azure's list price for its family; each cluster is deleted when its arm ends, so nothing keeps billing.

**Sizing the fleet to the lever.** The smallest saving a fleet can show is one machine. On 4 workers one machine is 25% of the fleet, so only a saving of about 30% or more can clear the noise; on 15 workers about 10%; on 96 about 3%. A few percent of a 4-worker fleet is less than one machine, so a 4-worker receipt whose machine rows stand inside the noise is the honest reading of a fleet too small for the lever, not a result against it. A new subscription allows 200 vCPUs in a region but 10 per machine family, so one family gives five 2-vCPU workers; the way past it is **several families at once**: eight pools of four to five workers each make a fleet of 39 (`worker_pools` input, `size:max,size:max,...`; `pool_prices` names each family's price). The app's replica ceiling is raised with the fleet (`hpa_max`: nine 200-millicore pods fit one 2-vCPU worker, so 9 × workers lets the fleet fill), the same in both arms; Omni-Compass's replica room, read live from what the machines can hold, lets the omni arm raise that cap while it binds and hand it back once demand holds still (section 3.4). A cloud region can refuse a new cluster for hours at a time; check the subscription's allowances before building anything, so that a refusal costs cents and not a cluster. The account setup, click by click, is in `docs/AZURE_SETUP.md`.

**In the cloud, the work the bill buys.** In a public cloud every machine that exists is billed, whether it carries work or not. The cloud runs the whole engine at the max, as on site (`dial max`, the `dial` input of `aks-metered` and its default; the founder's order of 31 October struck the tune-down on the machines alone of 14 October): the governor moves the service's pod target inside its autoscaler's cover, conveys the processor a machine can spare, and parks a machine the cloud already made only after its own trial shows the response time holds, waking it on a breach; the cloud's own autoscaler, native's machine controller, stays in every arm and alone adds machines. The set must show that the engine holds: no judged gauge LOSS against native, the work per dollar read beside the bill; the benefit is made at the software end and at the chip and comes through the wire. A smaller bill is never the aim; a company that wants a cut turns the dial down (`dial save`, a changed tune, its receipts its own). The bill is shown beside the work it bought, never ruled. In the cloud, as

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

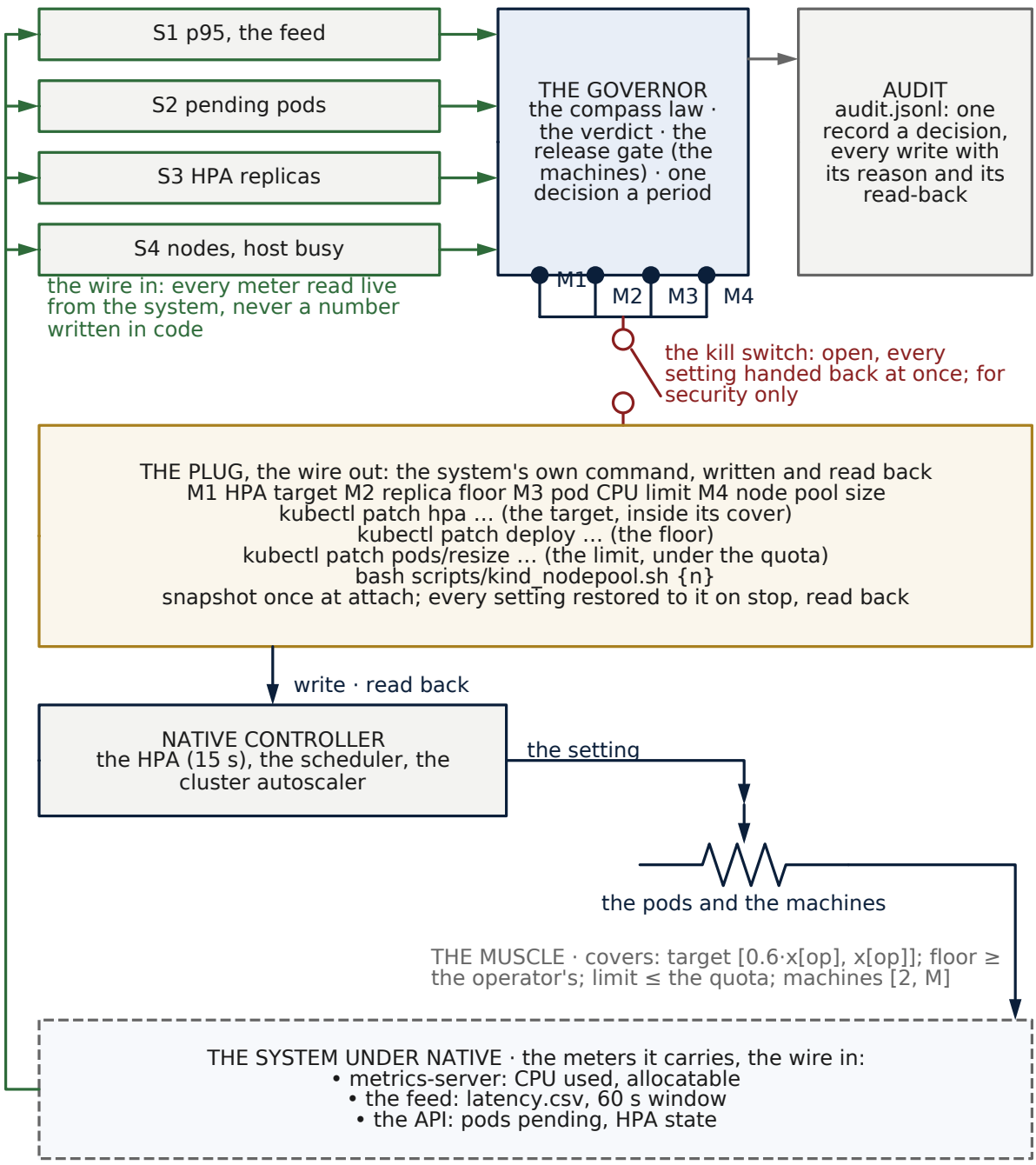



**Figure 23.** Wiring schematic: Kubernetes in the cluster, levels 2 to 4. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The audit stands beside the governor. Here the senses are the response-time feed, the pods waiting for a place, each autoscaler's replicas, and the machines with the host's busy share; the motors are the autoscaler's target, the replica floor, the pod CPU limit and the node pool size, each through kubectl's own patch and resize commands. Level 2: the target and the floor; level 3: the machines through the release gate and the verdict; level 4 (--strict-replicas): the replica decision itself. The HPA keeps its own 15-second check underneath. Nothing is ever deleted or evicted; a parked machine is emptied by the autoscaler's own scale-down.

everywhere, the governor's fast check (a parked machine woken for pods that wait) runs every 5 seconds, three times as fast as the autoscaler's own 15-second check, and its full decision every 60 seconds, the factory beat through a cloud (a dial: 15 seconds, the autoscaler's own pace, is one of its settings; no faster beat kept the bill from rising in the models, docs/OMNI_V24.md); a pod's start and a new machine's join are not shortened by deciding faster, so the governor acts on the reading while it climbs, before the line. On machines already paid for and in a cloud alike, the fast checks also give the running pods the processor their machine already has, which takes seconds and needs no new pod (the founder's order of 31 October struck the cloud's rule that switched this lever off: a pod given more processor may read busier to its autoscaler and ask for more pods, and what that does to the bill is read beside the work it bought, never ruled).

## 8.2 A database behind its connection pooler

PostgreSQL is wired through PgBouncer, the pooler most of its deployments already run. Nothing models the database. The wire in is the pooler's own report (`SHOW STATS`, `SHOW POOLS`), read once a second over one connection held open for the whole run, so that no process is launched for a reading: the larger of two figures, the time a transaction spent in the server plus the time clients waited for one, over the transactions completed, and the share of the pooler's clients queued for a server. The second figure is what makes the wire honest: a pool too small for the offered rate shows at the pooler as clients waiting even while its own service time reads calm. The wire out is one setting through the pooler's own admin console, `SET default_pool_size`, inside the cover [2, 90]: never under two server connections, never within ten of PostgreSQL's connection limit. The reading is held at 40% of a 50 ms line. Where the time was spent decides the direction: while clients waited for a server one percent or more of the last second, servers are added back, one for every percent of waiting, up to the pooler's own setting; a server is taken back only while clients waited under one percent of the second and a transaction was served, and only one the pooler itself shows idle. At 95% of the line the knob is handed back to the pooler's own setting at once. If the knob is found at a value Omni-Compass did not write, it stops writing (one writer). At the end of every omni arm the pool size is restored to the snapshot and read back. The same plug fits any pooler or proxy with a console and a pool-size setting (PgBouncer, Pgpool-II, ProxySQL, the application's own pool); the processor time the plug's own readings take on the host is recorded beside the host's, so the receipt shows what the wiring itself costs.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

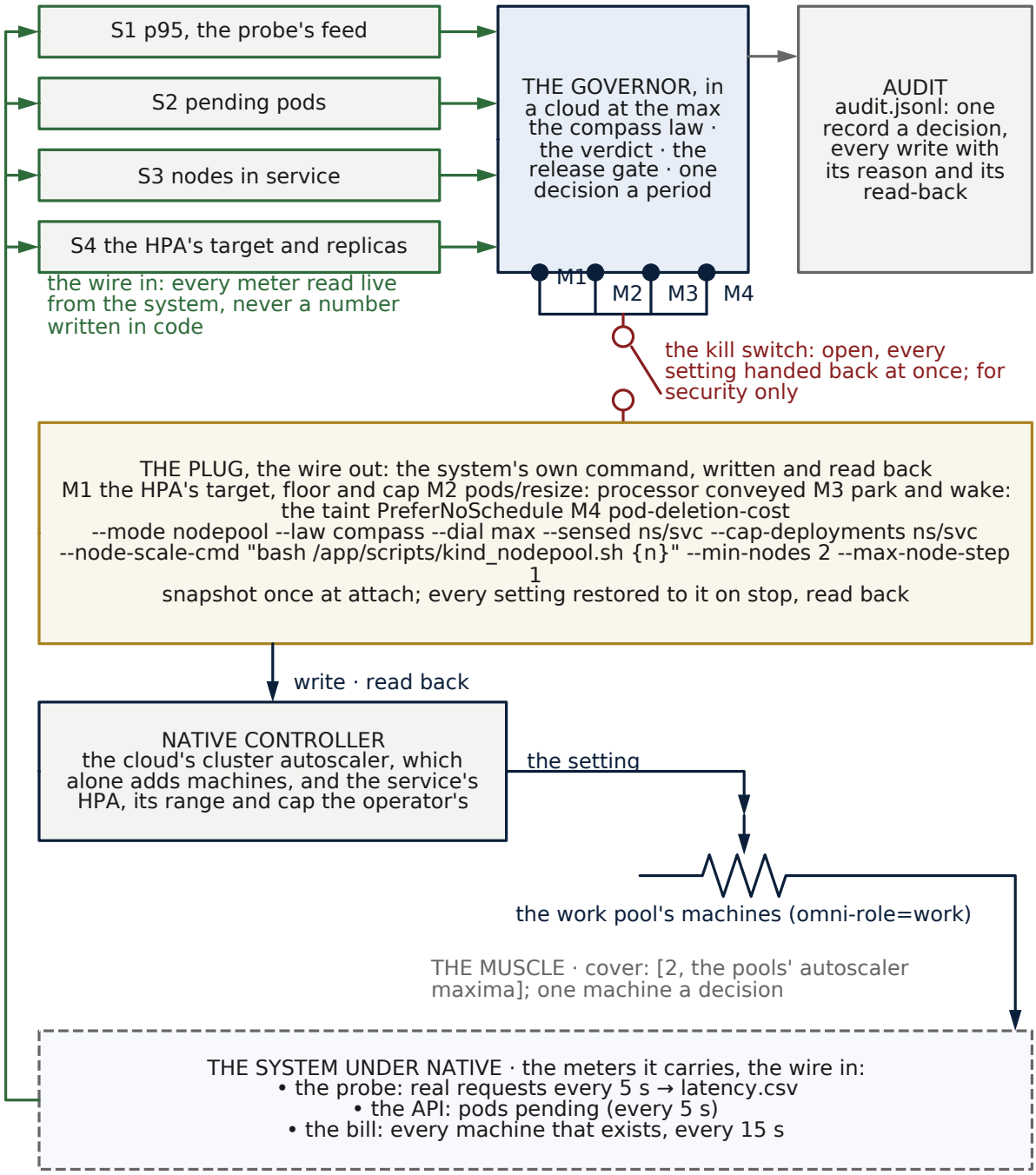



**Figure 24.** Wiring schematic: a managed cloud cluster, the whole engine at the max on top of the cloud's own autoscaler. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The audit stands beside the governor. The senses are the probe's feed, the pods waiting and the machines in service; the only motors park a machine (a PreferNoSchedule taint), wake it (the taint removed) and mark its pods first to go; no autoscaler is moved, no processor conveyed, no power capped. A machine is parked only after its own trial (the response time holds within 2%) and woken on a breach: at the next decision (15 s) for a response time over the line, at the next fast check (5 s) for a waiting pod. No cloud credential: the governor holds a Kubernetes service account that can read, taint and annotate, nothing more.

## 8.3 Independent simulators, each with its own native controller

Three published simulators show how the plug is wired onto three more kinds of native controller, and the simulator's own controller is always native:

- **A building's batteries** (CityLearn): the knob is each building's battery charge and discharge inside the simulator's own limits; native is the simulator's own rule-based control.
- **A substation's tap changer** (pandapower on SimBench grids): the knob is the tap position, one whole tap per move and a week of evidence before stepping down; native holds the tap at 1.00 per unit as the grid ships.
- **A robot arm's joints** (MuJoCo Menagerie): the knob is the joint speed of an axis that has slack; a paired physics trial before the run leaves an axis at its takt native, so an arm whose axes are all at their takt has nothing for Omni-Compass to move.

The same three wirings apply to the real controllers they model: a building management system's battery dispatch, a substation's tap-changer controller, a robot's servo limits. Receipts from a simulator are evidence class S: statements about the simulator's model, never about hardware.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

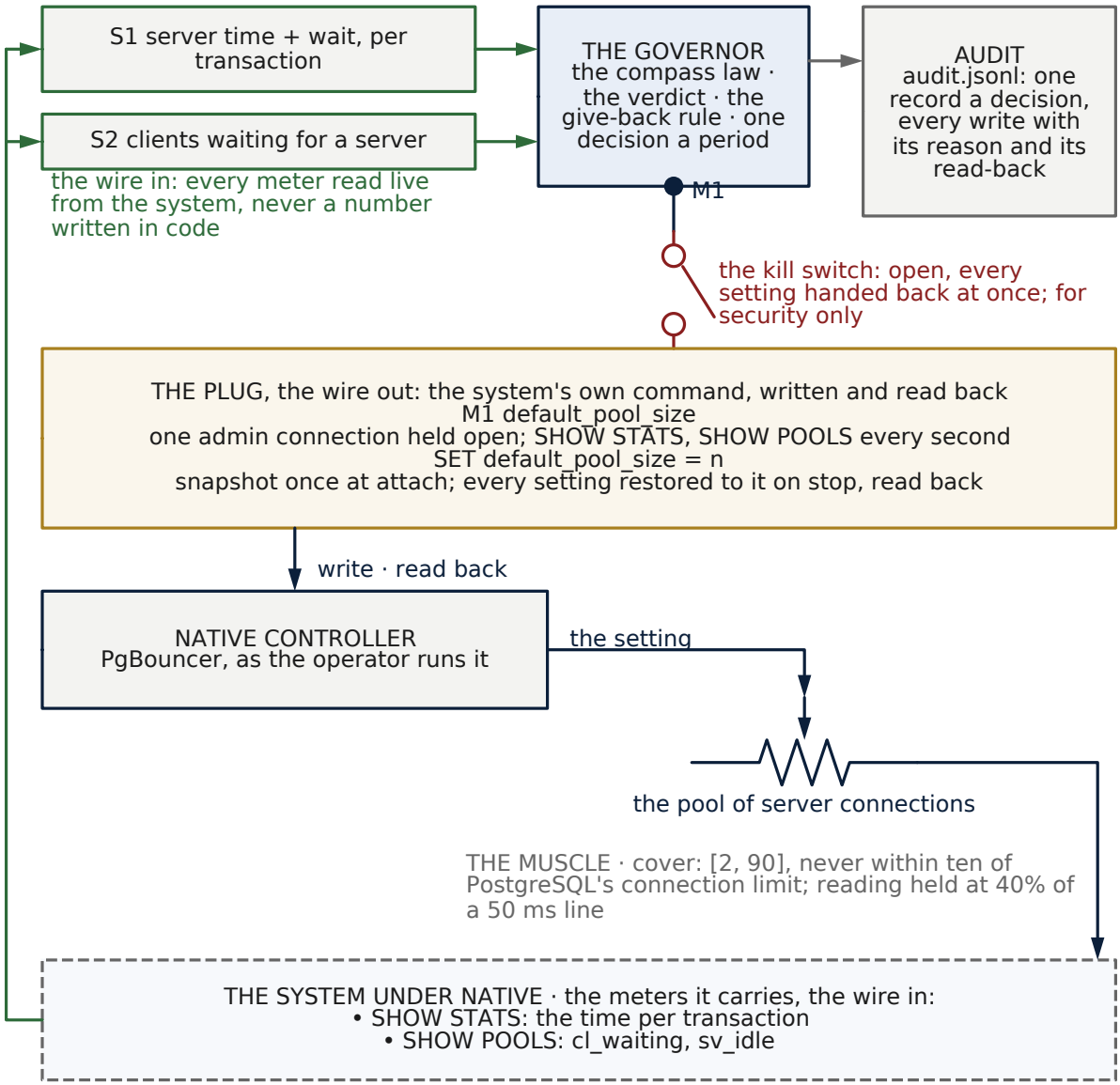
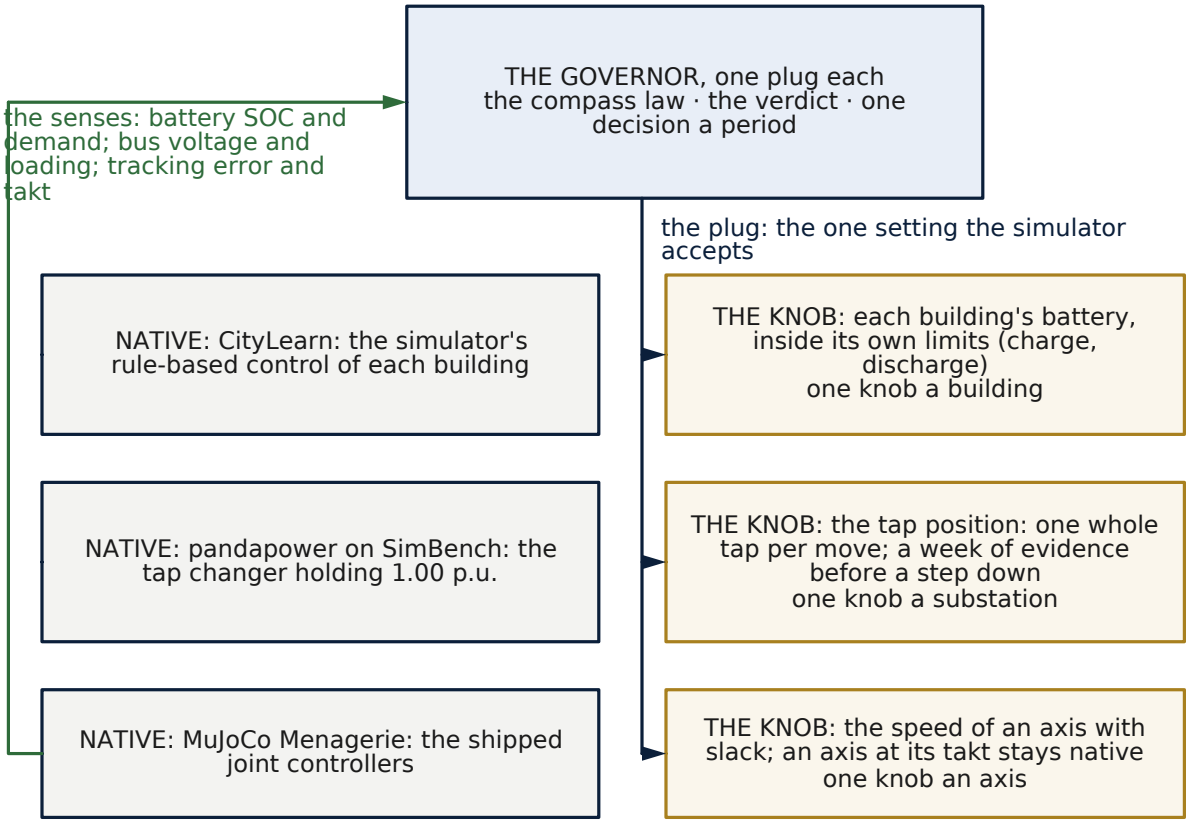



**Figure 25.** Wiring schematic: a database behind its connection pooler. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the server time plus the wait per transaction and the clients waiting for a server; the one motor is default_pool_size, written over one admin connection held open. Up: a server for every percent of clients waiting; down: one idle server, only while clients waited under 1%; at 95% of the line the pool goes back to the pooler's own setting at once. One writer; the restore read back at the end.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.




The simulator's own controller is native in every cell; the plug writes the one setting it accepts, reads it back and restores it at the end. Receipts are class S: statements about the simulator's model, never about hardware.

**Figure 26.** Wiring schematic: three independent simulators, each on its own native controller. One plug for each: CityLearn's rule-based control of each building, pandapower's tap changer on SimBench, and the shipped joint controllers of the MuJoCo Menagerie. The governor writes the one setting each simulator accepts, reads it back and restores it at the end; the senses come back along one bus. Receipts here are class S: statements about the simulator's model, never about hardware.

## 8.4 A message broker's consumer group

Apache Kafka is wired as an operator runs it: one broker as shipped, a topic of eight partitions, a consumer group at the operator's count of two. Nothing models the broker. The wire in is the group's own end-to-end latency: each message carries its production time, each consumer records when its handling ended, and the reading is the mean over the messages consumed in the last second; a second with nothing consumed while messages wait reads as the line (someone is waiting and no one is taking), and nothing consumed with nothing waiting reads as calm. The wire out is the count of consumers in the group, which is what an operator changes, inside the cover [1, 8]: never fewer than one, never more than the topic has partitions. The compass holds the reading at 40% of a 500 ms line with a response time of 3 s, because a consumer added or removed shows only after the group rebalances. Slow service with messages waiting adds consumers in proportion to the force; calm with nothing waiting gives back one idle consumer a second after a five-second dwell; at 95% of

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the line every consumer the topic can use starts at once. The rebalance that every move costs is paid by Omni-Compass and counted in the gauges. At the end of every omni arm the count is handed back to the operator's and read back. The receipt this wiring is built to produce has the queue kept short and no message lost, at the cost of consumers running, which reads power given by rule and stands beside the gain.

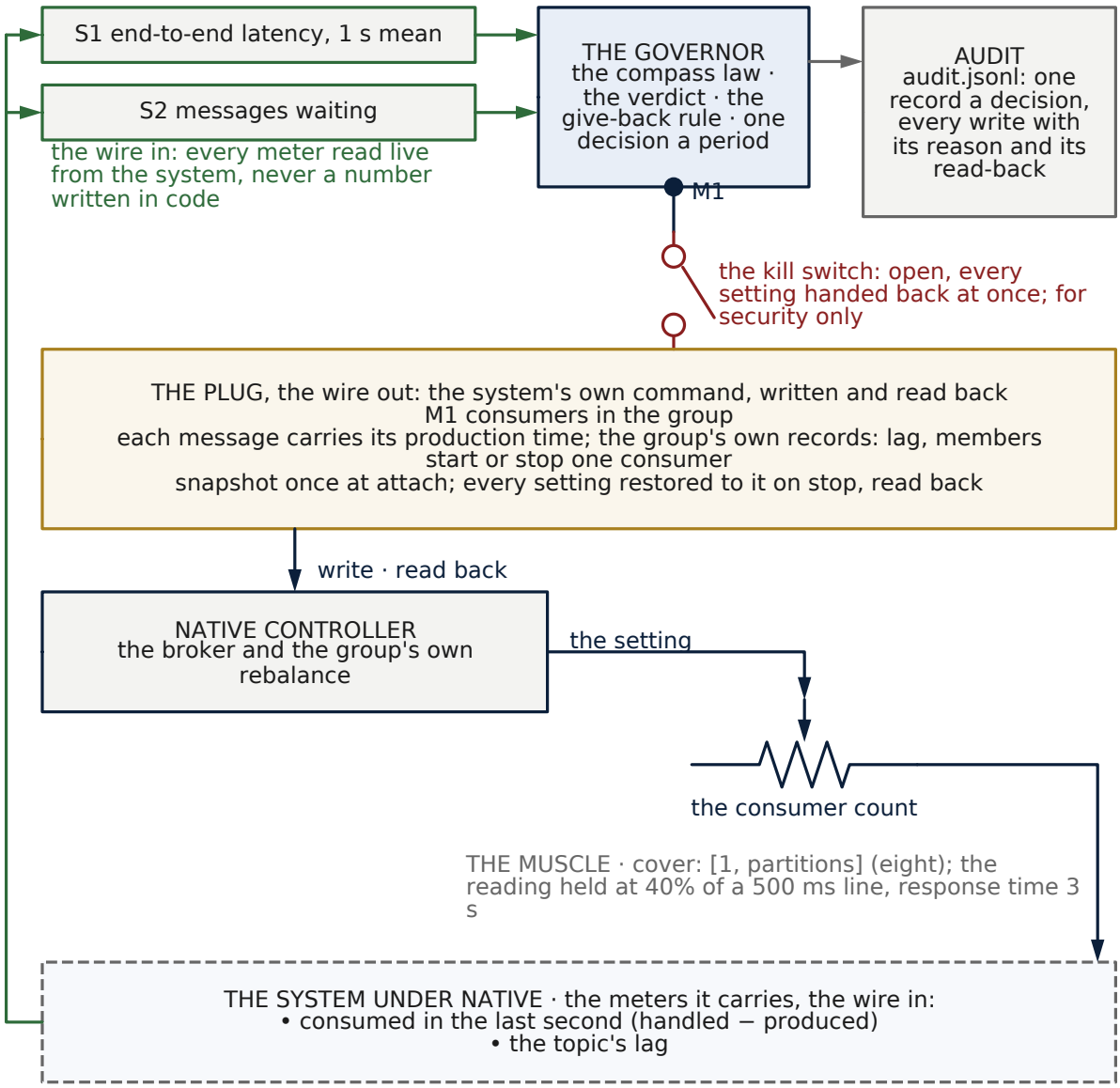



**Figure 27.** Wiring schematic: a message broker's consumer group. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

The audit stands beside the governor. The senses are the end-to-end latency and the messages waiting; the one motor starts or stops one consumer. Slow with messages waiting: consumers added in proportion to the force; calm with nothing waiting: one idle consumer given back a second after a five-second dwell; at 95% of the line every consumer the topic can use. The rebalance every move costs is paid by Omni-Compass and counted in the gauges.

## 8.5 A cache's memory ceiling

Redis is wired as Ubuntu ships it, with the two settings an operator sets for a cache, a memory ceiling of 64 MB and the allkeys-lru eviction rule. The application in front of it is ours and declared: 1,500 requests a second from 32 threads on a Zipf working set that widens and narrows one notch at a time; a miss costs a declared 5 ms trip to the store behind the cache and a write-back. The wire in is the application's own request latency, the mean over the last second (a hit about 0.3 ms, a miss about 5.5 ms). The wire out is one setting through Redis's own console, `CONFIG SET maxmemory`, inside the cover [16, 512] MB. The compass holds the reading at 40% of a 2 ms line. Its direction rule carries the do-no-harm gate in the form this stack needs: misses grow the ceiling by notches of 8 MB **only while the cache is full** (memory used at 90% of the ceiling or more), because a miss in a cache with room to spare is a cold miss that no ceiling can mend; calm with nothing evicted in the last second gives back one notch a second, after a five-second dwell since the last growth; at 95% of the line with the cache full a quarter of the cover is added at once. One writer: a ceiling found at a value Omni-Compass did not write stops it. At the end of every omni arm the ceiling is handed back to the operator's 64 MB and read back. The memory the compass holds for a wide working set is the resource this wiring trades and reads power given by rule; the gain is the hit rate and the work inside the line.

## 8.6 Drone swarms: the autopilot's cruise

In gym-pybullet-drones (University of Toronto, MIT License) every Crazyflie 2.x flies its shipped position controller along the planner's path at the planner's cruise: that is native. The wire in is each drone's tracking error, the distance between the drone and the point its autopilot was told to be at, on a band from 0 to a declared safe error of 0.25 m. The wire out is one knob a drone, the cruise override: the fraction of the planner's cruise the carrot moves at, never under 1.0 (the planner's own) and never over 2.0 (a quarter of the autopilot's own speed limit), moving by at most 0.01 a control tick. With slack the compass spends it as speed; as the error grows it eases; at 95% of the safe error the cruise is the planner's at once. Two guards belong to this stack: a **separation wall** (a drone with another within 0.5 m flies the planner's cruise, whatever the error reads) and the **paired physics trial** before the counted missions, which asks the simulator's own figures whether a faster mission costs less energy at all and keeps the error under the safe error; where it does not, the override is left native for that cell and the cell reads "nothing for Omni to move". Energy is a declared model from the simulator's own motor constants, said to be a model wherever it is printed; a collision voids the cell. At every landing and at the end of the run the override is handed back and read back.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

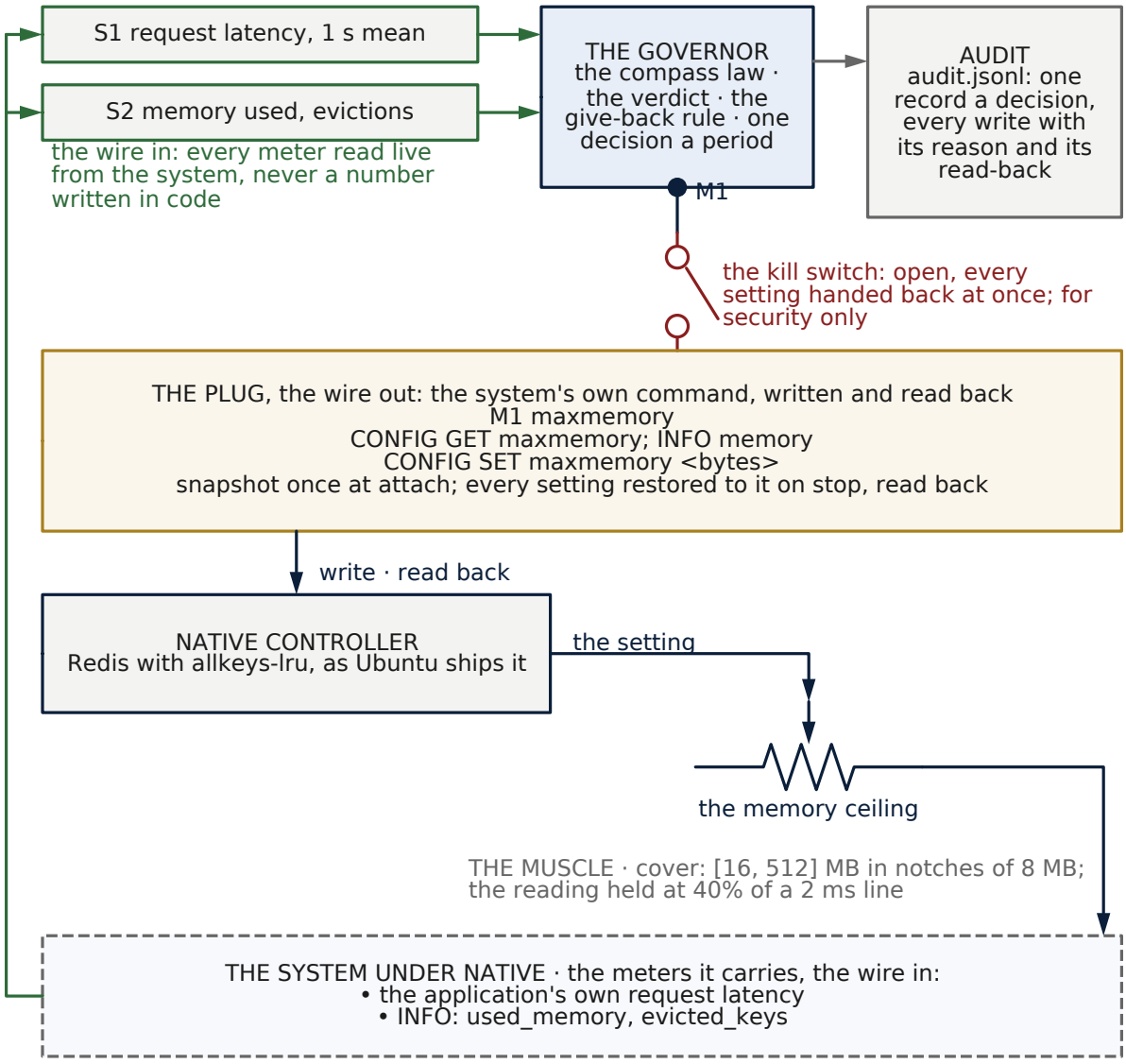



**Figure 28.** Wiring schematic: a cache's memory ceiling. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the request latency and the memory used with the evictions; the one motor is maxmemory, written with CONFIG SET. Misses grow the ceiling only while the cache is full (used ≥ 90% of the ceiling); calm with nothing evicted gives one notch back a second after a five-second dwell; at 95% of the line with the cache full, a quarter of the cover.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

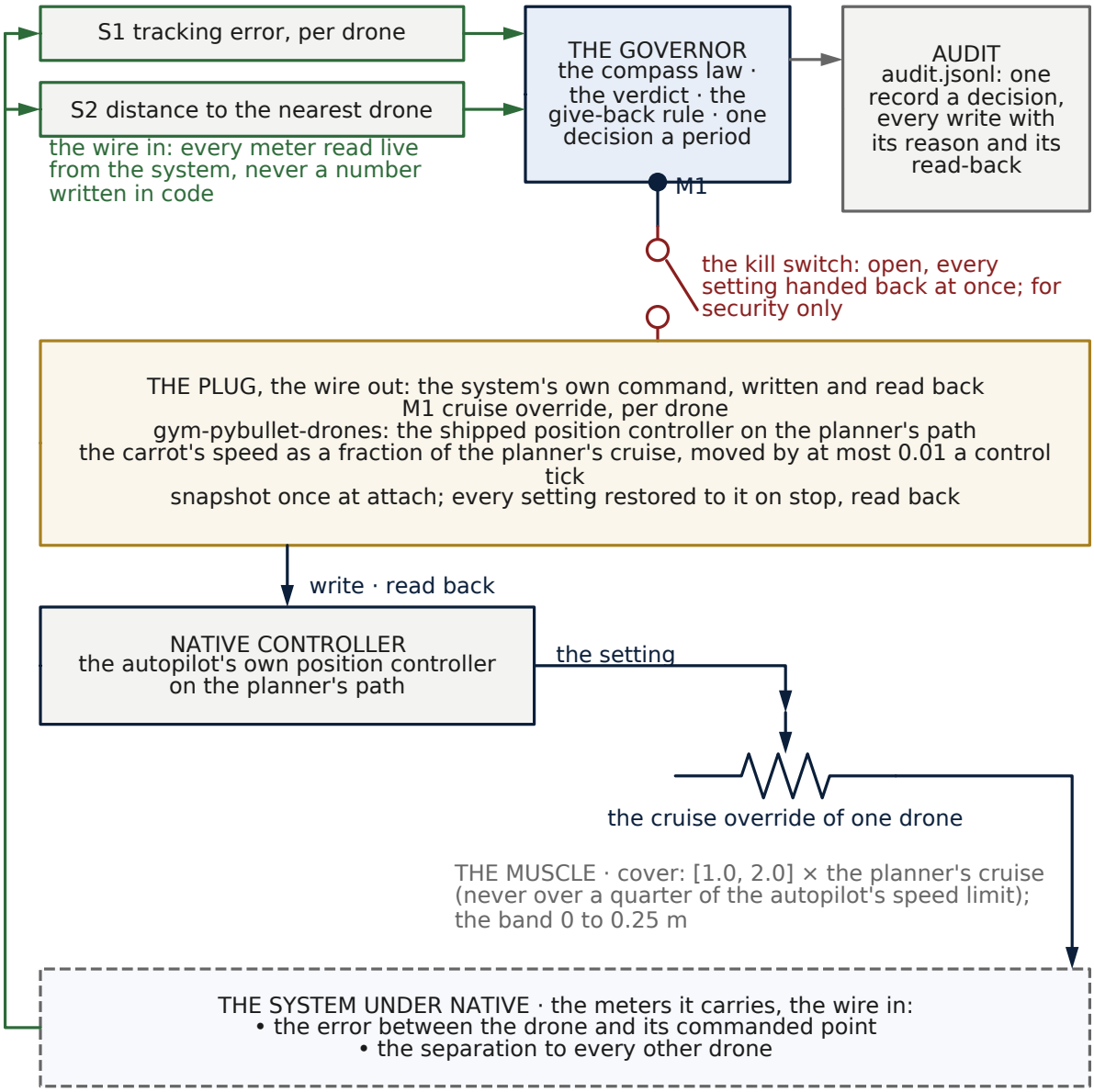



**Figure 29.** Wiring schematic: a drone swarm's cruise override. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are each drone's tracking error and its distance to the nearest drone; the one motor is the cruise override of one drone, a fraction of the planner's cruise moved by at most 0.01 a control tick. With slack the compass spends it as speed; as the error grows it eases; at 95% of the safe error the planner's cruise at once. The separation wall: a drone within 0.5 m of another flies the planner's cruise whatever the error reads. A collision voids the cell.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

## 8.7 A database's storage-engine cache

MongoDB is wired as an operator runs it: the server as its publisher ships it, its configuration as shipped but for the one setting an operator sets for the storage engine, the WiredTiger cache (512 MB in the reference wiring). The wire in is the server's own mean read latency over the last second, from its `serverStatus` operation latencies, differenced, beside the cache's own counters (bytes in the cache, pages requested from it and pages read into it). The wire out is one setting through the server's own console, `setParameter wiredTigerEngineRuntimeConfig cache_size`, inside the cover [256 MB, 2,048 MB]: the server's own floor at one end, a quarter of the machine at the other. The line is set on the stack's own figures so that a hit reads calm and a full cache that misses reads slow (1 ms in the reference wiring, held at 40%). The direction rule is the cache's: slow reads grow the cache by notches of 64 MB only while the cache is full (bytes in it at 90% of its size or more) and missing (pages read into it one percent or more of the pages requested from it in the last second), because slow reads in a cache with room to spare, or in a cache that already holds its working set, are not the cache's to mend; calm gives back one notch a second after a five-second dwell, and only while the cache's miss share is under one percent, because "no page evicted" is true of every cold cache while it fills; at 95% of the line with the cache full a quarter of the cover is added at once. One writer, read-back and the hand-back at the end are the plug's, as everywhere. Where the data files fit in the operating system's own page cache, a storage-engine cache miss is a read from memory and a decompression, not a disk read, so the gain available to this knob is smaller than on a machine whose data does not fit in memory; say so beside the receipt. The same plug fits any store whose engine exposes a cache size at run time (InnoDB's buffer pool, RocksDB's block cache); a store that can only be sized at restart has no knob Omni-Compass moves.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

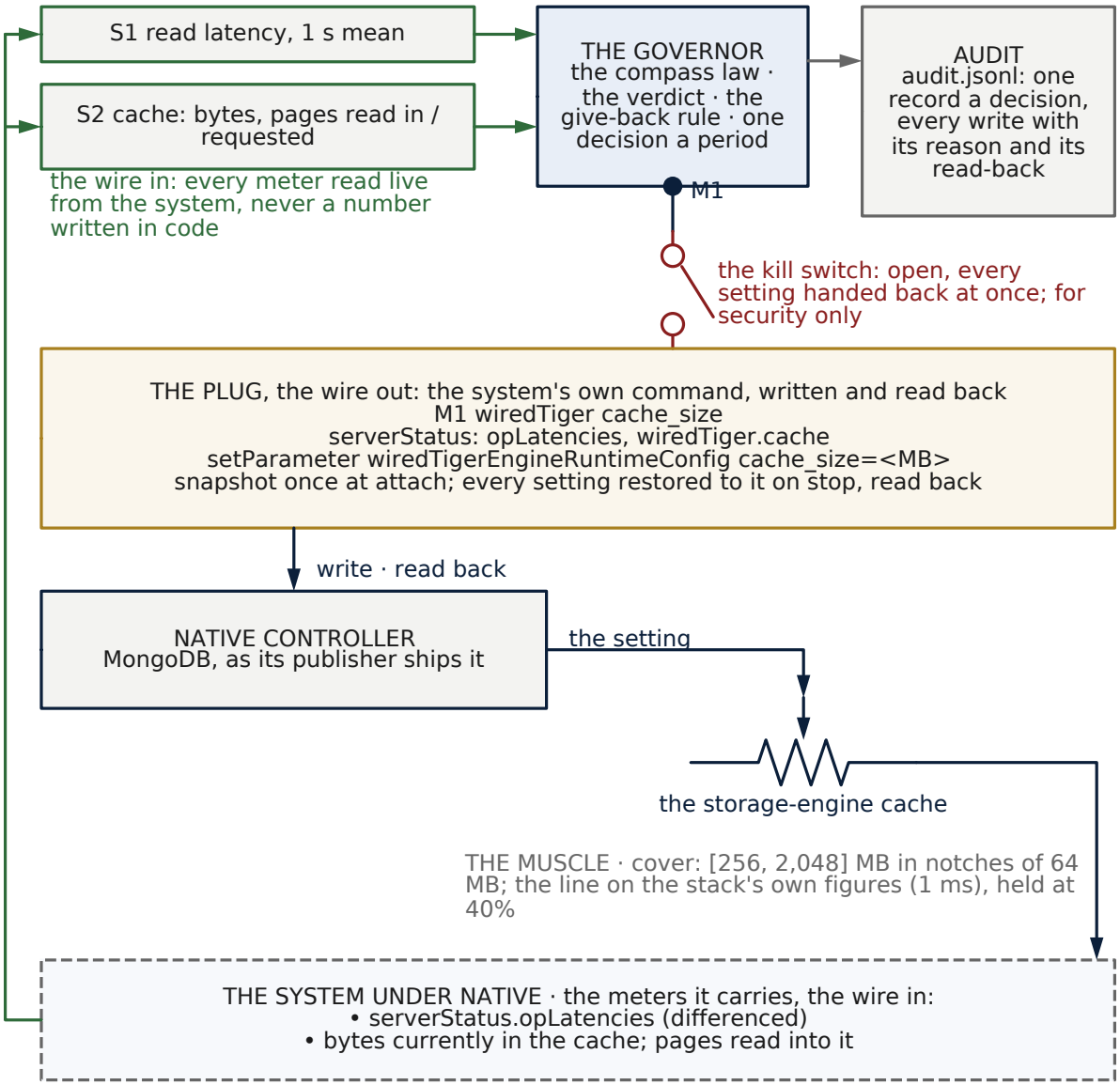



**Figure 30.** Wiring schematic: a database's storage-engine cache. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the read latency and the cache's bytes with its pages read in against pages requested; the one motor is the WiredTiger cache size, written through setParameter. Grow only while the cache is full (≥ 90%) and missing (pages read in ≥ 1% of pages requested); give one notch back after a five-second dwell only while the miss share is under 1%; at 95% of the line with the cache full, a quarter of the cover.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

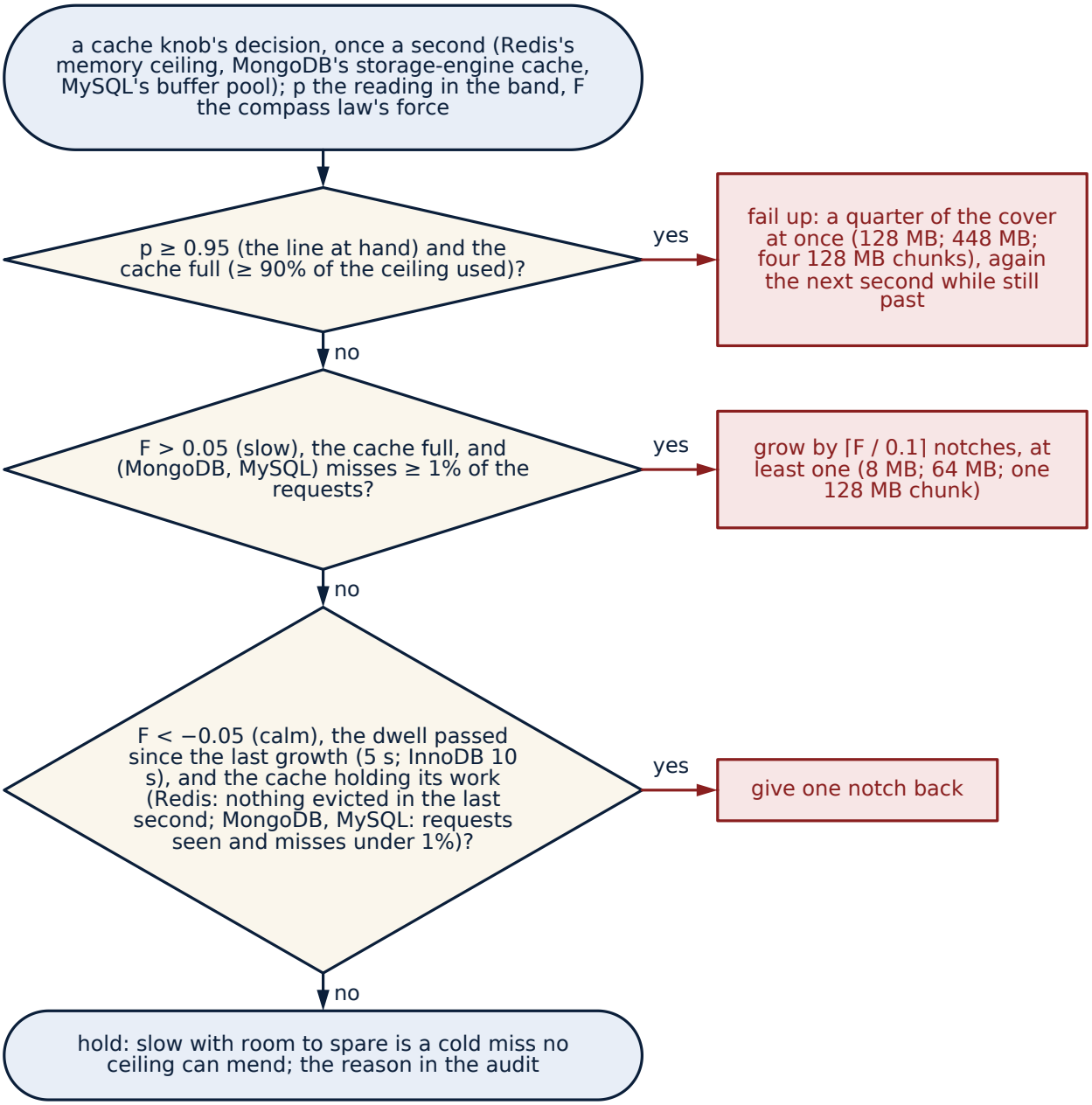



**Figure 31.** Flowchart: the direction rule of a cache knob (Redis, MongoDB, MySQL), once a second, tested in this order. With the reading at 95% of the line or more and the cache full (90% of its ceiling used), a quarter of the cover is added at once (128 MB; 448 MB; four 128 MB chunks), again the next second while still past. Slow (the compass law's force over 0.05) with the cache full, and for MongoDB and MySQL with misses at 1% of the requests or more, the knob grows by ⌈F / 0.1⌉ notches, at least one (8 MB; 64 MB; one 128 MB chunk).

Calm (the force under −0.05), the dwell passed since the last growth (5 s; 10 s for InnoDB) and the cache holding its work (Redis: nothing evicted in the last second; MongoDB and MySQL: requests seen and misses

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

under 1%), one notch is given back. Otherwise the knob holds, because a slow read in a cache with room is a cold miss no ceiling can mend.

## 8.8 A database's buffer pool, resized online in the server's own chunks

MySQL is wired as an operator runs it: the server as its distribution ships it, its configuration as shipped but for the settings an operator sets for InnoDB, the buffer pool (512 MB in the reference wiring), the chunk the server resizes in at its shipped 128 MB, one pool instance, and the performance schema on. The wire in is the server's own mean statement latency over the last second, from the performance schema's statement summary, differenced, beside the pool's own read requests and reads from disk. The wire out is one statement through the server's own console, `SET GLOBAL innodb_buffer_pool_size`, inside the cover [128 MB, 2,048 MB] and always in whole chunks, because that is how the server itself moves the pool. Two things about this knob differ from a cache the server resizes at once, and the plug carries both. The server resizes asynchronously and reports its progress in its own status variable, so the plug waits for the resize to complete before reading back, and a pool found mid-resize is the server still carrying out Omni-Compass's own write, not a foreign hand. And a shrink evicts pages while the resize runs, so the dwell after any move is ten seconds rather than five. The line is set on the stack's own figures, above a hit and under a miss (a 0.6 ms statement line in the reference wiring, held at 40%). Slow statements grow the pool by chunks only while the pool is full and missing (one percent or more of its read requests), at 95% of the line with the pool full four chunks are added at once, and calm gives one chunk back after the dwell only while the pool's misses are under one percent of its read requests and a read request was served, because InnoDB keeps stale pages resident, so neither "pages free" nor "no page read from disk" ever says that a pool holds its working set, and the miss share does. **The hand-back waits for the server.** Under load InnoDB finishes a shrink only when the pages the load has pinned are released, and it ignores a new pool size while a resize is in progress; the plug therefore waits for the resize the server is carrying out, writes the snapshot, writes once more if the server ignored the first write, and records the restore (what was in flight, the seconds waited, the writes, the final read-back) in every omni arm's receipt. A knob the device applies on its own time is handed back on the device's time, and the proof of the hand-back is the read-back, not the write.

## 8.9 The machine itself: the kernel's frequency governor under the processor's meter

The lowest level a governor can sit on is the machine it runs on. Linux already has a controller for the processor's clock, the frequency governor (`schedutil`, or the processor driver's own), choosing the clock every few milliseconds under a ceiling the operator sets once; and the processor has its own energy meter (RAPL) ticking in microjoules. Omni-Compass sits on top of the governor exactly as it sits on the HPA: the governor keeps running, and Omni moves one knob, the frequency ceiling of every CPU, through the kernel's own files.

The reading is the service's own mean latency, held at 40% of a 20 ms line by the compass law; calm gives one notch of the ceiling back (5% of the top clock), a slow service raises it by notches, the wall puts the top on

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

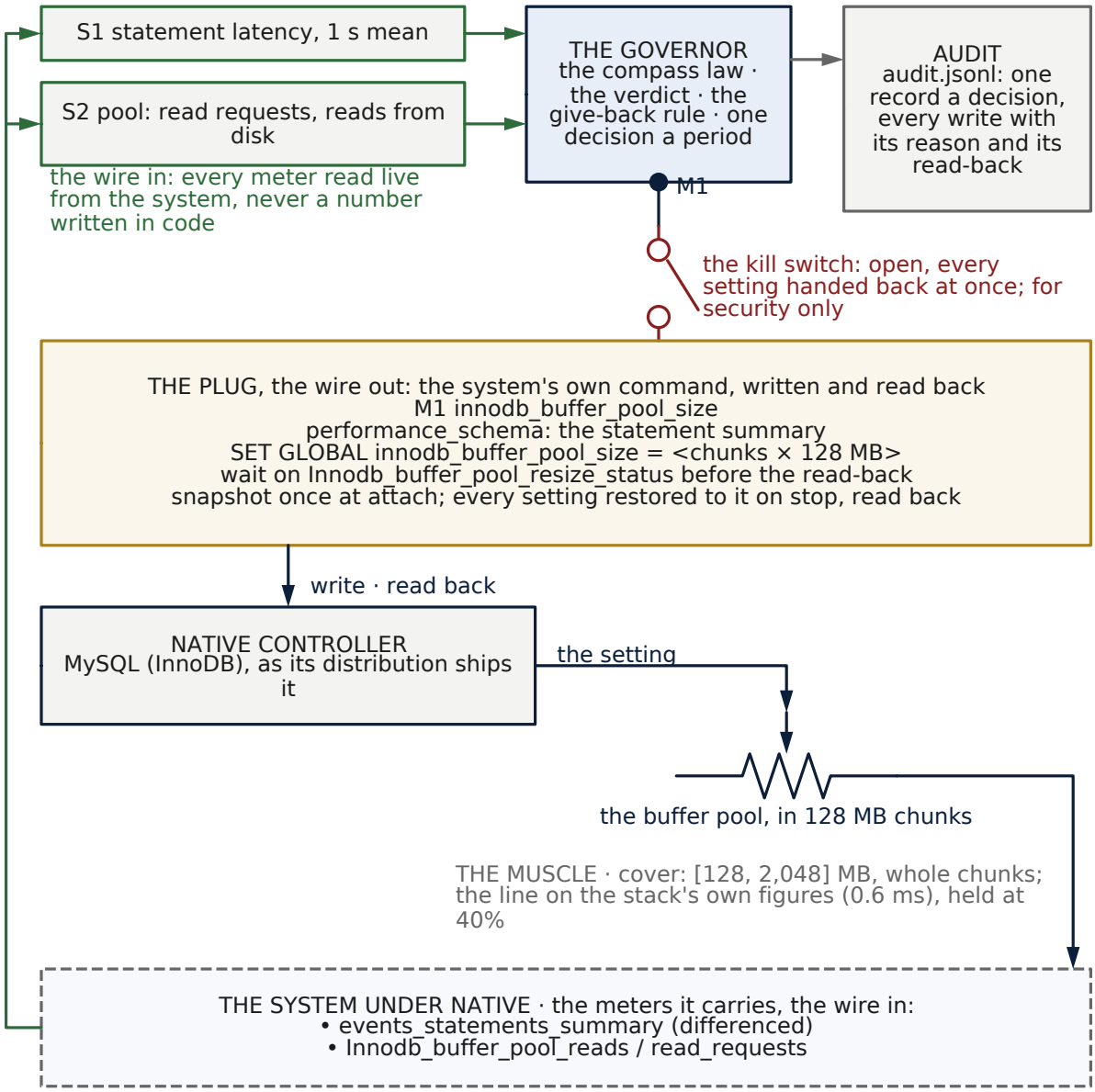



**Figure 32.** Wiring schematic: a buffer pool resized online in the server's own chunks. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the statement latency and the pool's read requests against reads from disk; the one motor is innodb_buffer_pool_size, in whole 128 MB chunks. The server resizes on its own time: the plug waits for the resize before reading back, and the dwell after a move is ten seconds; the hand-back waits for the server too, writes again if ignored, and records what it waited for.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

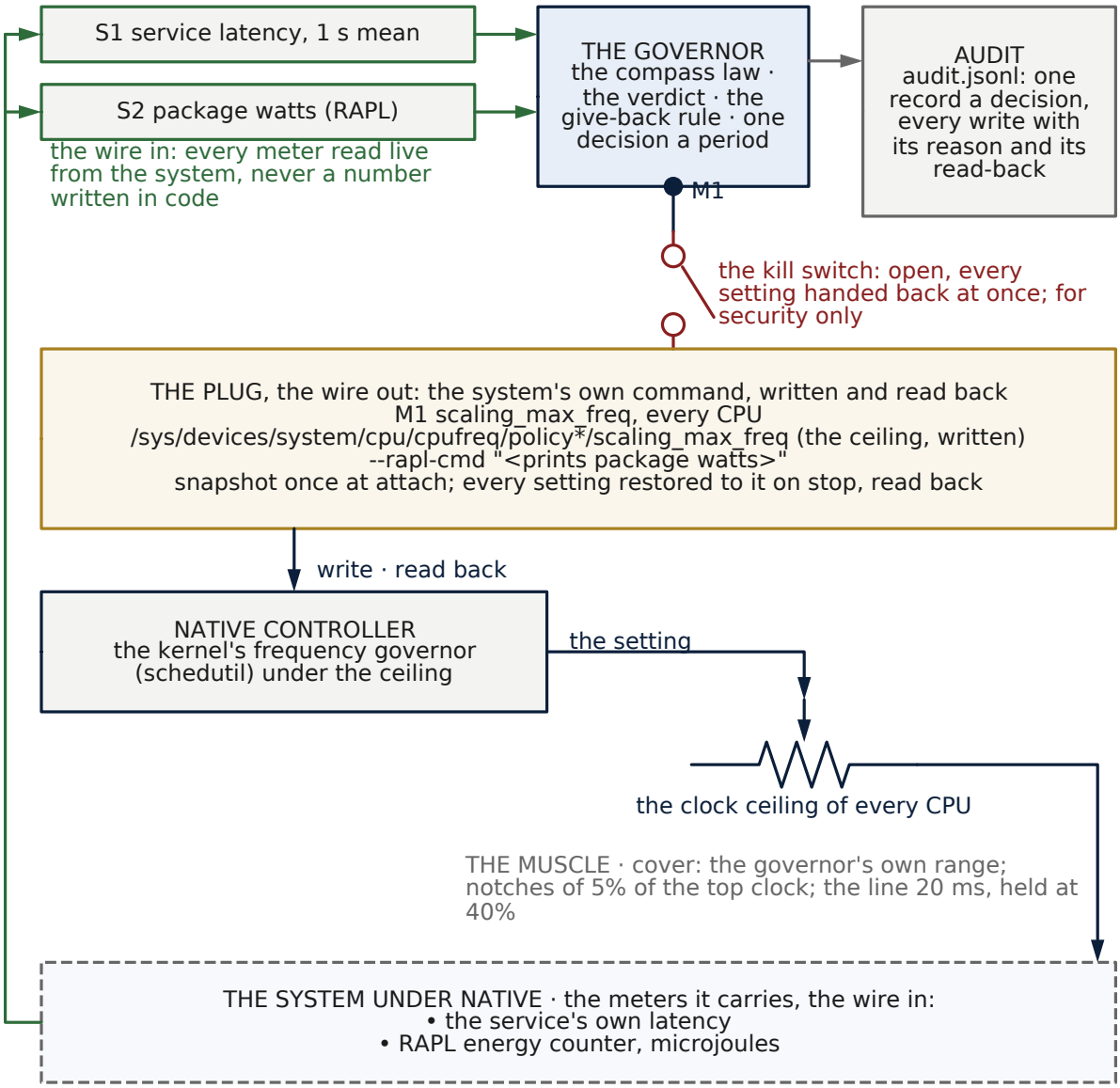



**Figure 33.** Wiring schematic: the machine's frequency governor under the processor's meter. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the service's latency and the package watts from RAPL; the one motor is scaling_max_freq, the ceiling of every CPU's frequency policy. Every notch down is tried on the machine first: eight cost samples at the deepest notch allowed and eight one further, allowed only if package power × latency / work inside the line does not rise within 2%. A snapshot of every policy, restored exactly.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

at once. Every notch down is tried on the machine itself first (section 7.4): the paired trial, eight cost samples at the deepest notch already allowed and eight one notch further, allowed only if the cost does not rise within 2%. The cost is the index's own reading for one machine (package power × latency / work inside the line), so a notch that saves less energy than it costs in speed is refused and the ceiling stays where the operator left it; a second declared objective, energy per request inside the line with the line as the guardrail, is run beside it. The snapshot of every CPU's governor and ceiling is taken once; the ceiling is handed back and read back at the end; a ceiling found at a value Omni-Compass did not write stops it; the master switch and the lease hand back if Omni-Compass dies.

The meter is the processor's own counter, class P with the caveat that it is the processor's calibrated counter and not an external meter; a smart plug on the wall socket (`WALL_METER`) adds the whole machine, read by the harness only. GitHub's own machines are virtual and expose neither the governor nor the meter. The paired run for this muscle runs on a machine on the metal as root, in one command (`sudo bash scripts/cpu_power_run.sh`): a workstation on Linux, a bare-metal server, or a self-hosted runner on either.

## 8.10 The data center under the servers: cooling, power and batteries

Under every cluster in this book stands a building. Its native controllers are older and more conservative than any autoscaler: the building management system holds the supply-air temperature through the cooling units' own loops, the chilled-water plant runs its own sequence, the uninterruptible power supply holds its reserve, the power distribution units meter every rack, and every server's management controller and every card's firmware enforce their own power limits. Omni-Compass sits on top of them exactly as it sits on the HPA. They keep running; it moves only the settings they already accept, and only inside the band the operator writes down.

**What it may set, and what it never sets.**

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| Cooling | the building management system and the cooling units' own loops | the supply-air setpoint, inside the operator's band | `--cooling-cmd "<sets {c}>" --cooling-min-c --cooling-max-c --cooling-restore-c` | built (level 7) |
| Site power | the site's own limit and its breakers | none: site watts are read, and the site limit enters the engine as a stress | `--power-cmd "<prints site watts>" --site-limit-w` | built (level 7) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

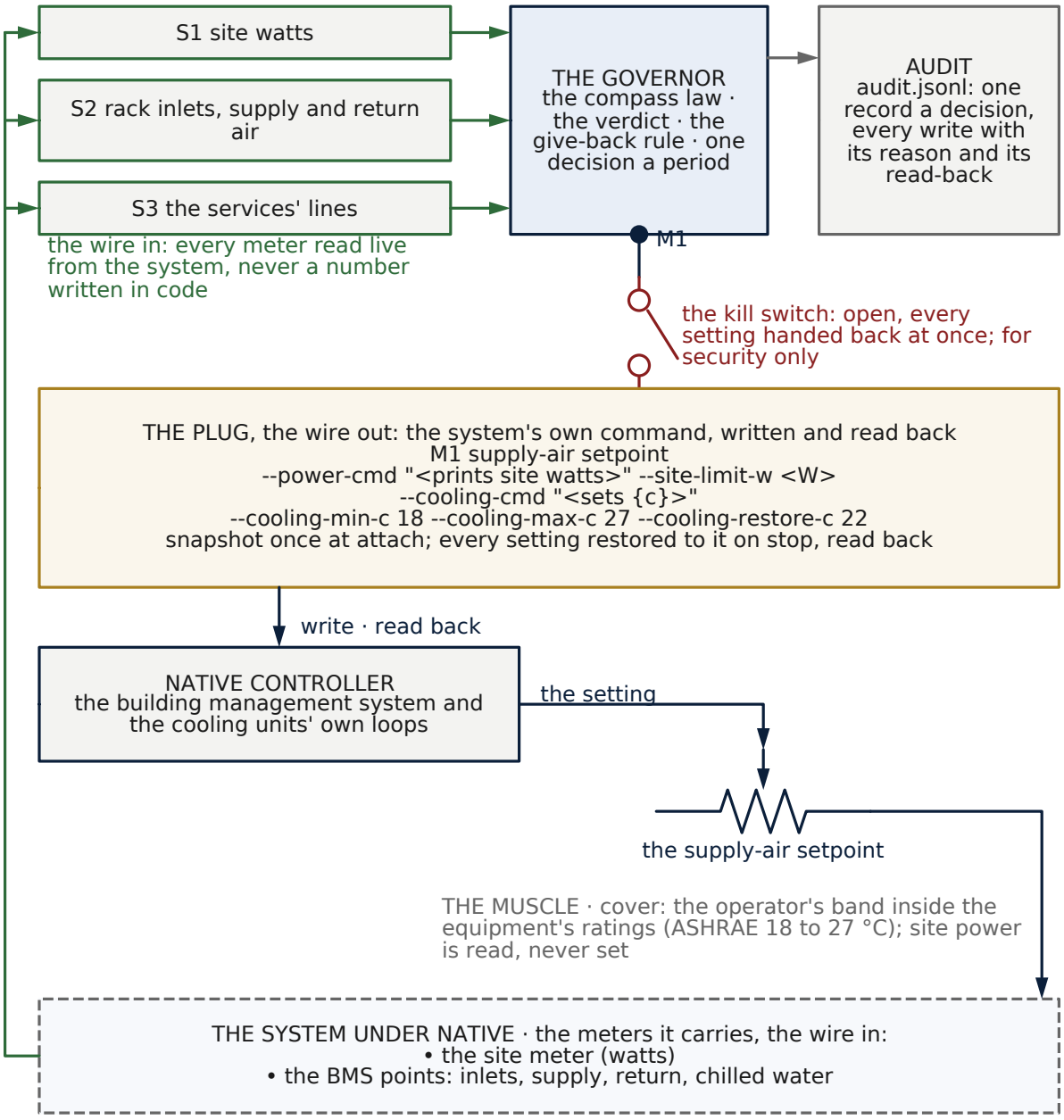



**Figure 34.** Wiring schematic: the data center under the servers. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the site watts, the rack inlets with the supply and return air, and the services' lines; the one motor is the supply-air setpoint, inside the operator's band. Warmer while every reading is cool, colder as heat rises; a step allowed only where the paired trial at the site meter shows the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

site's energy no higher for the same work, every inlet inside its band and no LOSS on any service gauge. A reserve held for an outage (the UPS) is never a lever. Modelled today (class S); no live facility has run yet.

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| | | | `<limit>` | |
| Servers' processors | the kernel's frequency governor and the processor's own limits | the clock ceiling and the power limit | section 8.9 | built (level 6) |
| Cards | the card's firmware | the clock ceiling and the power limit | level 5 | built |
| Uninterruptible power and batteries held as reserve | the UPS's own reserve | **none, ever**: a reserve held for an outage is never a lever (the physics criterion) | | native by design |
| Batteries held for shifting energy | the site's dispatch | the charge window | | designed as an organ, not yet wired |

**How the cooling muscle moves.** The setpoint is moved by the heat state between the operator's minimum and maximum: a warmer setpoint while every reading is cool, which saves the chiller's energy, and a colder one as heat rises. The band must sit inside what the equipment is rated for; for most data-center equipment that is the 18 to 27 °C inlet range of the ASHRAE thermal guidelines, and inside any narrower limit the equipment's own warranty sets. The restore value is the operator's own setpoint, handed back and read back on OFF, at the end of every run, and by the lease if the governor dies.

**The do-no-harm gate at the site meter.** A warmer setpoint saves chiller energy only up to the point where the servers' own fans spend the saving: fan power rises steeply with inlet temperature. That is why the cooling knob is judged at the site meter and not at the chiller: a step is allowed only where the paired trial shows the site's energy for the same work no higher, every rack inlet inside its band, and no LOSS on any service gauge. Where it does not, the setpoint stays the operator's, and the result says so.

**Wiring it, in order.**

1. **Watch.** Give Omni-Compass read access to the building management system's points (supply and return air, every rack inlet, the chilled-water temperatures) and to the site meter, and let it watch at least one full cycle of the site's demand. Nothing is written.
2. **Write the band.** The minimum, the maximum and the restore value of the supply-air setpoint, inside the equipment's ratings, signed off by the facility's engineer; the site limit in watts.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

3. **The wire check** (section 5.4). The cooling command must read back the setpoint it wrote; a building schedule that rewrites the setpoint is another writer, and Omni-Compass stops writing that lever by itself (section 6.2).
4. **One hall first.** A test hall or a single row, paired against its own native weeks, before any other hall.
5. **Keep the switches in reach.** The master switch and the kill file work at the facility level exactly as at every other level.

**What is measured, and what is modelled.** The facility muscles are modelled today (class S): the data-hall cooling family of the catalog, the power distribution family, and the site budget shared by groups of cards, which held out with no minute over its budget. No live facility has run yet. A class P result needs a building management system's point list, read access, a site meter and a test hall; until a facility pilot has produced its own paired receipts, this section is a wiring instruction, not a claim of savings.

PART FOUR

# Operating It

*The OFF switch, the rules the governor obeys, the log it keeps, and the care of a running installation.*

CHAPTER 9

# The OFF Switch, the Rules, and the Log

**The rules Omni-Compass keeps on your system.**

1. **The master switch, for the whole harness, in a human hand:** `python3 tools/omni_switch.py off` turns every Omni-Compass governor on the machine off at once (the Kubernetes controller and the GPU governors). Each puts every setting it ever wrote back to the value it read before its first write, reads it back and exits; the command waits until all have, and says so. While the switch is OFF, no governor will start. `python3 tools/omni_switch.py on` allows them to be started again (nothing restarts by itself); `status` shows the switch and every governor running. Use it the moment anything looks wrong, including a suspected breach. Each governor also keeps its own switch for one muscle at a time: the kill file (or `OMNI_KILL=1`, or SIGTERM to the GPU governor). **If a governor dies without handing back** (killed outright, the machine crashed, it hung), nothing it wrote is left in place: every governor records, before its first write, the exact commands that put every setting back, and renews a lease every decision. Run `python3 tools/omni_switch.py watchdog` as

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

its own service next to the governors: it hands back for any governor whose process is gone or whose lease has run out (a hung one is stopped first). `status` names any governor that died without handing back. The watchdog acts only on records its own user wrote that no other user can change (run it as the governors' user, or name theirs with `--trust-uid`), and never signals a process number the system has given to another process. Put the switch and the registry in a directory only the governors' user can write (`OMNI_MASTER_OFF`, `OMNI_REGISTRY`); `status` warns when they are not.

2. It records before it acts (annotations on Kubernetes objects; the `snapshot` line in the GPU audit).
3. It watches before it writes.
4. It never acts blind.
5. Service first: while response time is over its target, and for `--slo-clear-s` seconds after (180 by default), nothing is given back.
6. Every change is read back; no new order goes on top of one that has not landed.
7. The living band and the cover: no organ is driven outside its range; machines are parked, never switched off by Omni-Compass; it never evicts or moves a pod.
8. One writer: if anyone else changes a knob, Omni-Compass stops writing it and leaves it alone.
9. Everything is logged: every read, decision, write and reason goes to the audit log.

**On autopilot: why nobody babysits it.** Omni-Compass is an autopilot: a closed-loop supervisory controller that runs, steers and corrects itself inside the system it governs, as a self-driving car's controller does inside the car. Once the paired trial on a muscle has passed and Omni-Compass is set to write, nobody turns a knob and nobody has to react. Every change is Omni-Compass's own, and it audits itself: every write is read back from the system, every machine it gives back is kept only after its own paired trial shows the response time holds, every reading, decision and reason is in its audit, and it hands every setting back at the end; nobody has to check its work either. It holds the response time at the centre of its band, under the line, and acts as the reading approaches the line, so the correction lands before the line is crossed, not after a person is paged. What a native stack pays in people for the same service (an engineer watching dashboards, paged while the service is over its line, retuning an autoscaler's target or a node pool by hand) is the **babysitting tax**, and every Kubernetes and cloud receipt counts it in both arms: the **alert minutes** (whole minutes whose 95th percentile, failed requests counted over, was over the line: the minutes an on-call page would be open; judged, the aim lower), the **settings commands the governor wrote on its own**, and the **share of its additions decided while the service was still inside its line** (both shown, not judged). No person touches either arm of a paired run. The aim in the cloud is **no alert minute at all**: a minute over the line is a person woken, so an alert minute in the omni arm is a LOSS of the aim, its cause found and fixed and the test run again. Where native itself is short of machines (over its line on its own), the receipt says so beside the row: the same machines can be made to carry more, but no governor creates compute that the machines do not have.

**The three ways a governor stops, and what each leaves behind.** A referee or an underwriter should know exactly what happens to a governed system in each of the three ways Omni-Compass can stop, because the answer is the same in all three: every setting at native, read back, with no record left.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

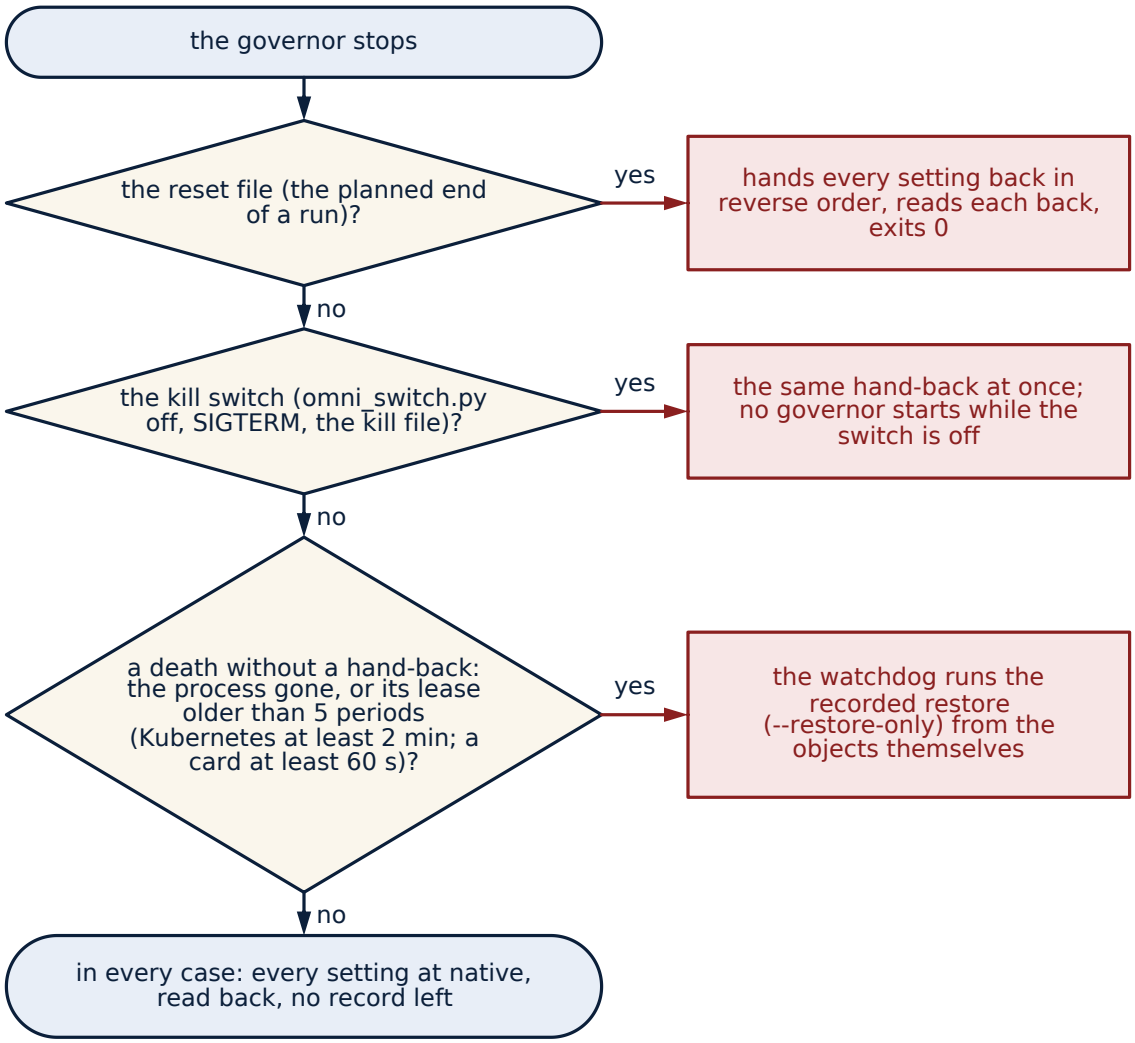



**Figure 35.** Flowchart: the three ways a governor stops. The reset file at the planned end of a run hands every setting back in reverse order, reads each back and exits 0; the kill switch (omni_switch.py off, SIGTERM or the kill file) does the same at once, and no governor starts while the switch is off; a death without a hand-back (the process gone, or its lease older than five periods) is caught by the watchdog, which runs the recorded restore from the objects themselves. In every case every setting ends at native, read back, with no record left.

- **The reset** (every setting handed back to native) is the planned end of a run. The governor sees its reset file, hands every setting back in reverse order of power-up, reads each back, writes its last audit line and exits 0. Every benchmark arm ends with one and then checks the cluster from outside the governor: the HPA target at the operator's, the replica range as set, every pod's CPU limit at its shipped value, every worker back in service, no `omnicompass.io/` annotation left on any object. An arm whose reset check fails is invalid and is left out of the table, never counted.
- **The kill switch** (security) is the unplanned end of trust. `tools/omni_switch.py off` writes the switch file and sends every registered governor SIGTERM; each performs the same hand-back as a reset and exits; the command waits until all have and reports any that have not. No governor starts while the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

switch is off. It is one switch for the whole machine, not one per muscle, because the case it exists for, something driving a system through Omni-Compass, is not a case for picking which muscle to stop.

- **A death without a hand-back** (killed outright, a crash, a hang) is the case the lease covers. Before its first write every governor registers itself with the exact command that puts every setting back (`omni_controller.controller --restore-only`, which restores from the snapshot annotations on the objects themselves, so it needs no memory of the dead process), and renews a lease every decision. The watchdog (`tools/omni_switch.py watchdog`), run as its own service beside Omni-Compass's processes, finds any of them that is gone or whose lease is older than its limit (five decision periods, at least two minutes), stops a hung one, runs its recorded restore command, and records what it did. Drill it before production: kill the governor outright in the middle of a governed paired run on your own system, and record how many seconds pass before every setting is back at the snapshot and what the service saw in that gap.

**Reading the log.**

| Line | Meaning |
|---|---|
| `gate: a sense is blind` | a reading failed; nothing is given back until it returns |
| `gate: pods scaling up` | pods first, machines after |
| `decision failed (n in a row)` | the cluster could not be reached; nothing was written; turn it OFF for native |
| `decided_by: compass` | the compass set the wires this decision |
| `decided_by: fail_up` / `blind_fail_up` | the service crossed the 0.95 wall, or a sense went blind: full capacity at once |
| `decided_by: thermal_hold` | the card reported a heat slowdown; nothing was tightened |
| `foreign_writer` | someone else changed a knob; Omni-Compass now observes only |
| `restored ... ok: true` | the OFF switch put every setting back and read it back |
| `"watchdog": "handed back"` (in the watchdog's own log) | a governor died or hung without handing back; the watchdog ran its recorded restore command; `ok` says whether every command exited 0 |
| `cruise: on` / `cruise: off` | work waited for a place for two decisions: machines in service up to the pool's own top; the line has been empty for two: cruise ends |
| `idle_floor` | the work is done and demand is at zero: the pedal lifted all the way, straight to the floor in one move |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Line | Meaning |
|---|---|
| `verdict_state: left native` | no step of the slow knob passed the paired trial (section 1.2): the knob is never moved on this muscle |
| `shield_interventions` | the shield clipped a write to its bounds or its step limit; the count says how often |
| `overhead` (the last line of the audit) | the governor's own CPU over the window: its process and every command it ran |

**How to read an audit from end to end.** The audit (`audit.jsonl`, one JSON record a line, every line timestamped) is the governor's complete account of itself, and a reader can follow a run through it without the manual: the first lines are the snapshot (every setting as found, written to the objects as annotations so a watchdog can restore from them); then, every decision, the readings (the response time and its age, the pods waiting, the machines seen, whether any sense is blind), the reading's place in its band (`compass`), the authority granted and the release gate's reason (`authority`, `node_gate`), the decision itself (the machines recommended against the machines seen, the HPA target set), every write with the command that made it and the value read back, and the decision's wall time (`decision_ms`); in cruise, the floor step's silence; at the end, the reset's writes and read-backs and the `overhead` record. The paired-run harness prints a decision trail from the audit at the end of every omni arm, one line a decision, so a reviewer reading a run's log sees what the governor saw and did at each minute.

**Reading the command acknowledgements: requested, commanded, acknowledged.** The governor keeps three settings apart for every muscle at every decision, and the audit records each (Appendix L):

- **Requested**: what the decision asked for (`decision`: `nodes_recommended`, and wherever a service is sensed `hpa_target_recommended`).
- **Commanded**: what was sent once every limit was applied, the cover, the muscle's own limits and, at the max, never below native (`write`: the command as it ran, and why; `shield_interventions` counts how often a request was clipped).
- **Acknowledged**: what actually landed, read back by the adapter (the value read back after each write; `authority`: `proprioception`, the drift between the count last commanded and the count now in service, 0 when the command landed).

The system moves with the command that landed, not with the one requested, so read the three together. A request clipped to a limit is **held at the muscle's limit**. A command not yet acknowledged is **still moving** toward its setting, and no new order goes on top of it (rule 6 above). A command the system refused is recorded (`refused`, with its exit code and why) and **held**: it is not written again at every check (a processor raise the cluster's admission refused is held until the pod's limit changes, the raise shrinks or its wait passes). A request that never lands, or a drift that never closes, is a wiring question before it is anything else (section 5.5). The live benchmark engine page shows the same states for each decision on its own model: the settings requested, those that landed as asked, those held at a muscle's limit, and those still moving.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 10

# Maintenance, Upgrades and Security

- Run `python3 verify.py` after every upgrade; it must end `VERIFICATION: PASS`.
- Upgrade in watch mode first; take the levels again from level 1.

**What an upgrade is, and what it is not.** The engine is fingerprinted by version (chapter 13). An upgrade that changes any rule, guard, preset or muscle of the engine is by definition the next engine version, and every receipt an operator made on the old version must be made again on it before the two are read side by side; `tools/omni_version.py` says which version a checkout carries, and a checkout that matches none says which files differ. An upgrade that changes only the harness, the tools or this manual leaves the engine's fingerprint as it was and every receipt standing. An operator can therefore tell, from one command, whether a new checkout is the engine their receipts were made on or something that must prove itself again.

- The container runs non-root, read-only, with no capabilities; the Kubernetes identity has only the permissions of the level you run.
- The GPU governor needs root only for `nvidia-smi -pl` and `-lgc`.
- Report a security issue as `SECURITY.md` describes.

**Least privilege, proven on every run.** The Kubernetes governor runs as its own service account (`deploy/kind/rbac-omni.yaml`) with exactly the verbs the level needs: patch on the governed HPA, nodes, pods and the governed deployment, get on its security ConfigMap, create on pod evictions. It cannot delete nodes, create or delete pods or deployments, touch the load generator or anything in `kube-system`, read any secret, create a namespace, or write its own security hold. Every paired arm records `kubectl auth can-i` for each of these and fails if a "cannot" reads yes or a "can" reads no (`rbac_omni.txt` in every paired run's files). The security hold itself is a ConfigMap another identity owns: when its key reads `hold: "true"` every expansion is blocked, and the governor cannot clear it.

**Credentials, and the notice.** Cloud credentials live only in an encrypted secret store and are used only by the jobs that need them; no credential, key or token belongs in any file, log, receipt or commit. Every source file carries its SPDX license identifier so that license scanners (Black Duck, FOSSA, Snyk) read the evaluation license correctly, and every generated report carries the legal notice at its head and foot.

**Supply chain.** Everything the paired-run harnesses download is pinned and checked: metrics-server by version and SHA-256 (`scripts/kind_bench.sh`), Apache Kafka against the Apache Software Foundation's own SHA-512 file for that release, Redis as the Ubuntu runner's own package, MongoDB from

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

its publisher's own repository signed by the publisher's key and YCSB against the SHA-256 written in its preregistration, gym-pybullet-drones at one named commit, Python packages by `requirements.txt`. A download whose checksum does not match stops the run before anything else happens.

**The upgrade drill, step by step.** An operator who receives a new checkout does five things, in order, and stops at the first that fails. First, `python3 tools/omni_version.py`: if it prints the version the operator's results were made on, the engine is unchanged and the rest of the drill is about the harness; if it prints a list of differing files, the checkout is a new engine and nothing the operator measured before applies to it until measured again. Second, `python3 verify.py`, which must end with its pass line; a checkout that fails it is not installed anywhere. Third, the wire check on the operator's own stack, which proves that every wire the new checkout would use still follows, reads back and returns. Fourth, watch mode for at least one full cycle of the operator's load, reading the log the governor would have written and comparing its would-be decisions against what the operator would have done. Fifth, the levels again from level one, each with its paired receipt. An upgrade that skips a step has not been upgraded; it has been installed.

**The security hold, and who holds it.** The hold is the one control that outranks the governor's own judgment, and it is deliberately not the governor's to hold. It lives in a ConfigMap owned by another identity (the operator's security team, in practice); while its key reads true, every expansion the governor would make is blocked and every contraction is still allowed, so a governor under a hold can only give capacity back, never take more. The governor reads the hold on every decision and records in its audit that it read it; it has no verb that could write it. The kill switch (chapter 9) is the blunter instrument beside it: the hold constrains a running Omni-Compass, the switch stops every Omni-Compass process on the machine and hands everything back. An operator who suspects anything untoward uses the switch first, reads the audit second, and rotates the governor's identity third; the hold is for the slower case of a change window or an incident elsewhere during which no one wants capacity added.

**What has been reviewed, and what has not.** Tested on every paired run: the governor's permissions against the list above, read from the cluster itself; the refusal of every write past a cover, of every give-back while blind, and of every move under a hold; the lease and the watchdog's hand-back on a real cluster; the checksums of every download. Not yet done, and said so: an independent security review of the code by a party outside the company, and a penetration test of the deployed governor on a customer-like cluster. Until those are done, the operator's own review stands in for them, and this chapter tells the operator exactly what to look at.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART FIVE

# Checking It on Your Own System

*Paired runs and receipts on your own system, evidence classes, the engine version and the three-run rule.*

CHAPTER 11

# Paired Runs and Receipts on Your Own System

The method of every published benchmark is the method an installer uses on their own system, and it is deliberately simple: the same system, the same work, native and then omni back to back on the same machines, repeated with the order rotated, each gauge reported with the 95% interval of its paired difference. Pairing is what makes a difference of a few percent visible on a shared machine whose own noise is larger than that: two arms run minutes apart on one runner see the same neighbours and the same hour, so their difference is a difference between arms, while two arms on separate machines differ by about 15% in p95 on their own. The order rotates so that no arm always runs first, warm or cold. The work is held equal by an open-loop load, a fixed rate of requests at fixed moments, so that a lower resource count can never mean that less work was done. The steps below are the whole method.

1. **Paired runs.** Run your service the same way twice, once native and once with Omni-Compass on top, back to back on the same machines, the order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this on Kubernetes; `scripts/gpu_rented_run.sh` does it on a GPU box, in one command: the wire check, a smoke run, then the counted run.
2. **The switch drill** after every run: turn Omni-Compass OFF; check every setting is back at its recorded original; turn it ON.
3. **The receipt.** Each gauge: native, Omni-Compass, the change, the 95% interval of the difference, and whether the interval excludes zero. A change under 0.5% reads STEADY; a change of 0.5% or more reads its word by the gauge's aim, GAIN, LOSS or power given, and the receipt says whether its interval clears zero (beyond the noise) or not.
4. **The label, by rule written before the run:** SUPERIOR WITHIN GUARDRAILS / ENERGY IMPROVEMENT WITH SERVICE TRADEOFF / NONINFERIOR / INCONCLUSIVE / NOT

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

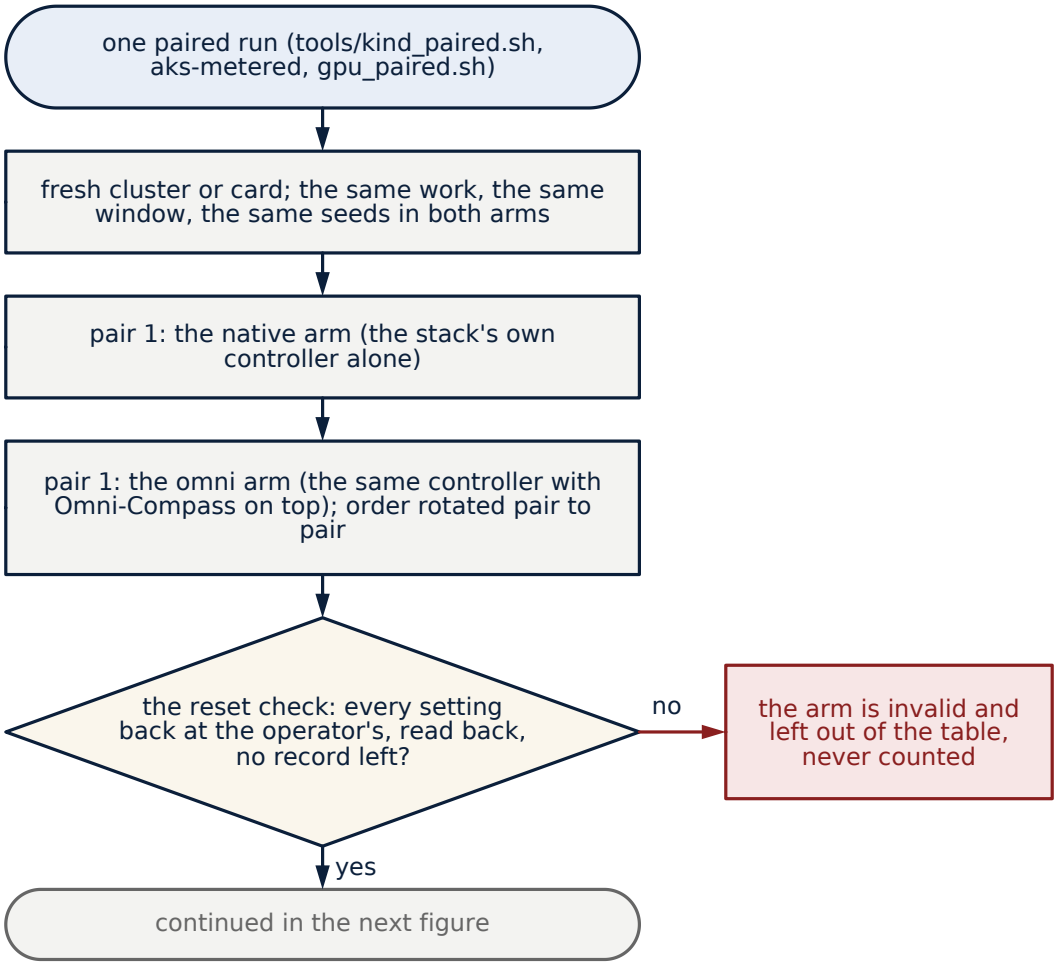



**Figure 36.** Flowchart: one paired run, the pairs. A fresh cluster or card; the same work, the same window and the same seeds in both arms; each pair runs the native arm (the stack's own controller alone) and the omni arm (the same controller with Omni-Compass on top), the order rotated pair to pair; the reset check after each arm (every setting back at the operator's, read back, no record left) invalidates an arm that fails it, and that arm is never counted. Continued in the next figure.

ESTABLISHED / AWAY FROM ITS AIM / NOT ATTRIBUTABLE / INVALID. Guardrails: work not lower by more than 1%; the share of time outside the service line not higher by more than 1 percentage point. The band-first rule is stricter: no win is claimed while the time outside the service line is above native's.

5. **Write it down, then run.** The arms, the knob, the cover, the gauges and the readings are written down before the first counted run; the rules are fixed on one tuning case and then applied unchanged to the cases that are counted. A change made after a result has been seen is written down with its time and reason, and the receipt says so.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

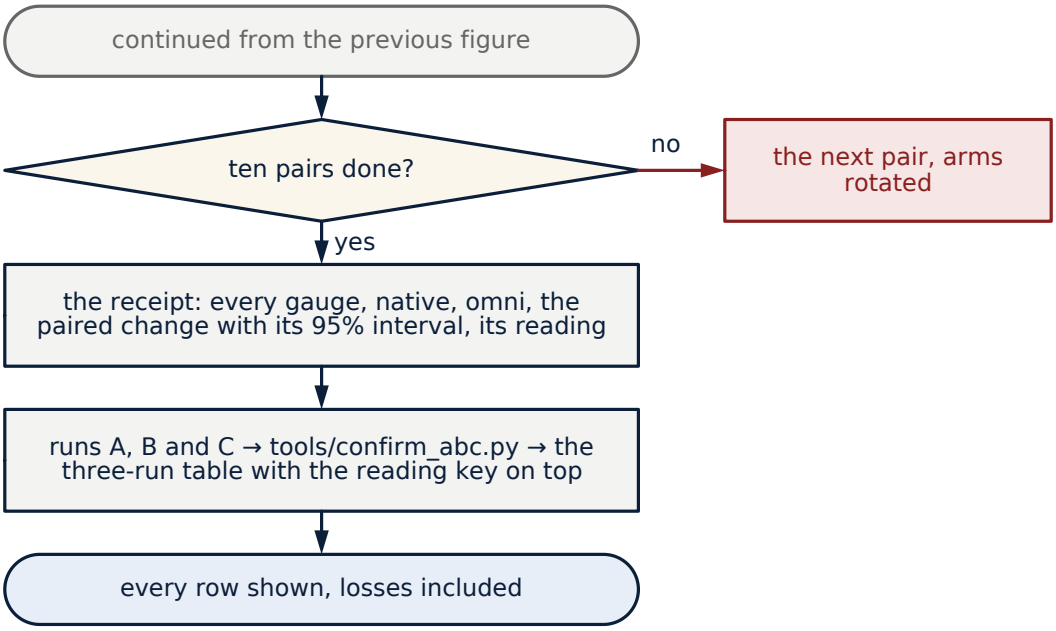



**Figure 37.** Flowchart: one paired run, from the pairs to the three-run table. After ten pairs the receipt is written: every gauge, native, omni, the paired change with its 95% interval and its reading; runs A, B and C go through tools/confirm_abc.py into the three-run table with the reading key on top; every row is shown, losses included.

6. **Every row, losses included.** A row that went against Omni-Compass is reported with the others. If Omni-Compass loses, the first suspect is the wiring, then native's setup, then the scoring; that is fixed and the run repeated, and the loss stays in the record.
7. **Keep the raw files.** Every run's files (capture, response times, audit log, controller log, one SHA-256 manifest per repetition) are kept with the receipt, and every receipt names the runs and the commit it was made from, so that anyone holding the files can rebuild it.

**The three arms on your own system.** Each arm is run on the same machines with the same work, and each pair is read by the same key.

| What you run | Native | Omni-Compass on native | Omni-Compass direct |
|---|---|---|---|
| Kubernetes on the kind rig (`scripts/kind_bench.sh`) | `ARM=native` | `ARM=omni`, or `ARM=compass` with the compass law | `ARM=solo`: the pod autoscaler removed after the deploy, the workload at its manifest's count, the governor's own pod muscle on it |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| What you run | Native | Omni-Compass on native | Omni-Compass direct |
|---|---|---|---|
| Paired repetitions on GitHub (`benchmark-reps`) | `native` among the arms | `compass` among the arms | `solo` among the arms; `tools/live_reps.py` reads Omni-Compass by itself against native and against Omni-Compass and native |
| Omni-Compass's own orchestrator, the three arms in one showing (`python3 tools/orchestrator_show.py --out DIR [--quick]`, run as root; on GitHub `orchestrator.yml`) | Kubernetes' published autoscaling rule, written as the yardstick | Omni-Compass on top of that rule | Omni-Compass alone on the muscles |
| Our own models, the ten organisms (`python3 tools/scenarios_20.py`) | every plant under its own native controller | the max on top of every native controller | not run: Omni-Compass is never by itself (the founder's order of 25 October) |

A cloud has no direct arm without the owner's word (chapter 20), and a muscle whose native is a setting has one Omni-Compass arm, not two (section 1.1).

**How many repetitions, and why.** The width of a paired interval shrinks with the square root of the number of pairs, $w \propto 1/\sqrt{n}$, so going from five pairs to ten narrows it by about a third ($1 - 1/\sqrt{2} \approx 0.29$), and from ten to forty by half again. On shared cloud runners, ten pairs of a 900 s window typically put the interval on a cluster's work inside the line at about ±10 points of native and on its 95th-percentile response time at ±15 to ±30 points: wide intervals, which clear zero only when the effect is tens of percent. A gauge computed from a declared formula over counts (an energy model over machines in service) has a much narrower interval, and a difference inside it is read as no difference, never as a small gain. A gain of a few percent on a cluster needs forty pairs or more to clear zero at these widths. On a stack with a finer knob (connections, consumers, megabytes) the intervals are narrower at three pairs than a cluster's at ten, because the knob moves in hundreds of small steps rather than in whole machines. Run five pairs first, look at the interval widths on the gauges you care about, and set the count for the counted run from those widths, never from the result.

**Why the load is open-loop.** A closed-loop load generator waits for each answer before sending the next request, so a slower system is offered less work and its throughput "improves" its own latency; two arms under closed-loop load are not doing the same work. An open-loop generator sends requests at fixed moments whatever the system does, so both arms are offered the same work to the request, a queue forms when the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

system falls behind, and the time over the line and the failed requests record the cost. Every paired-run harness (the cluster's load generator, pgbench's rate limit, Kafka's producer at a fixed rate, Redis and YCSB at a fixed target rate) offers its work open-loop, and a lower resource count in the omni arm can therefore never mean less work was asked. A burst is a schedule of open-loop rates written before the run and applied identically in both arms; only the load generator reads it, never the governor.

**What a receipt looks like.** One row per gauge: the gauge's name and direction; native's value and omni's; the paired change as a percentage of native with its 95% interval; the reading. A head naming the system, the knob, the cover, the window, the number of pairs, the order rotation, the engine's fingerprint and the commit; a foot saying whether every arm was handed back. `tools/live_reps.py` prints exactly this for a Kubernetes run and `tools/confirm_abc.py` prints it for three runs side by side; a receipt from any other tool should be laid out the same way so that a reader of this manual can read it without learning a new shape.

**The readings to expect, and what it means when you do not see them.** The published tables are the reference an installer reads their own receipt against: not their numbers, which belong to our systems and our loads and are printed only in the Owner's Manual (chapter 17) and `results/live/`, but the words each kind of service has read in all three of our runs on real Kubernetes:

| The service | Reads GAIN | Reads STEADY | Power given |
|---|---|---|---|
| Rightsized with Goldilocks and the VPA's recommender | response time (mean, 95th and 99th percentile); time over the line | machines, failed requests, replicas, pods started, pod start wait, alert minutes | parked-worker energy |
| Scaled by KEDA's HTTP add-on | response time (mean and 95th percentile); alert minutes; replicas; pods started | machines, failed requests, pod start wait | parked-worker energy |
| Scaled by KEDA on CPU and requests | response time (mean, 95th and 99th percentile); time over the line | machines, failed requests, replicas, alert minutes | parked-worker energy |
| Governed by CruiseKube's own recommendations | nothing: no setting written | every gauge | none |
| Held at a fixed CPU cap, four caps | nothing: the cap blocks the one lever | completed work and work inside the line at every cap | none |

Read your receipt against this table in three steps. **First the words.** A service gauge that reads LOSS on your receipt, where the table reads GAIN or STEADY for the same kind of service, is read as a wiring fault before anything else (`DISCLOSURES.md`, section 4): the feed's freshness and its line (chapter 19), the sensed autoscaler's name, the identity's permissions (chapter 15), the window's length against the widths of your receipt's own intervals. **Second the direction.** Where the table reads GAIN, your receipt should read GAIN or STEADY; a smaller gain is your stack's, not a fault, because the gain is bounded by how far native stood from

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

its own best. **Third the power given.** A resource that rose on your receipt is read beside the work and the speed it bought; a rise with nothing bought is a question for Appendix L: what did the governor write, and why. Two readings are not faults: STEADY on every gauge where native already runs at the top of its range (the knob is at its limit on native, and the governor leaves it there by design, section 6.5), and a machine count that does not fall where no machine can be given back without a LOSS (the audit's `node_gate.reason` names the gate that held). What the table cannot give is your number: the same engine on another stack is another result, and it is yours (section 4.7).

CHAPTER 12

# Evidence Classes and How to Read a Receipt

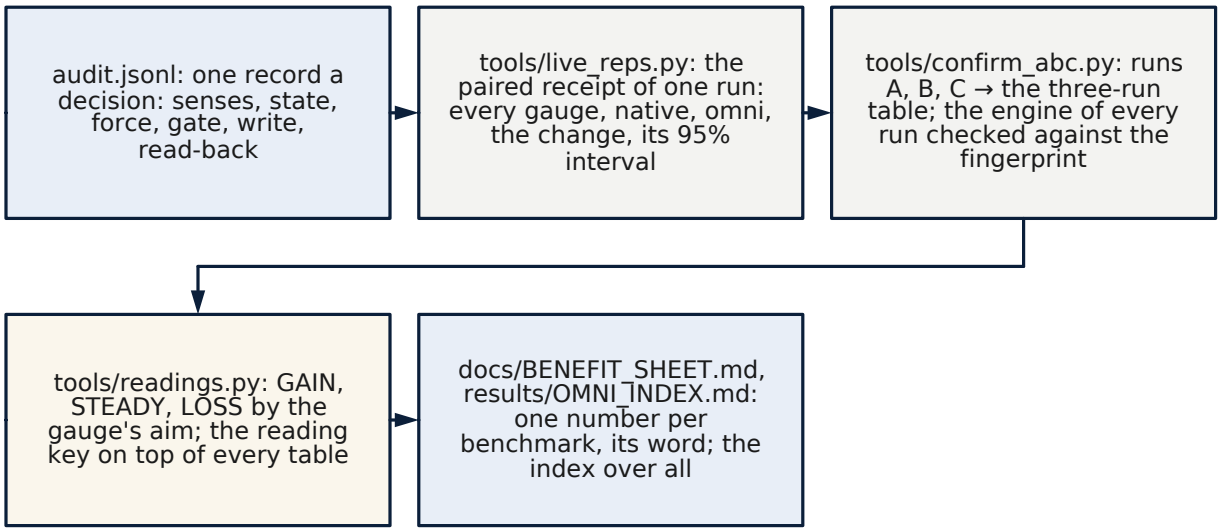
**One number, one word.** In every receipt a change keeps its raw sign: a response time that fell reads "-65%", a cache that grew reads "+300%", and the Reading column beside it gives its word, GAIN, STEADY or LOSS, decided by what the gauge is for and never by its sign (the key at the front of this manual): a response time cut 65% is GAIN, a cache grown 300% is **power given**, a fall in work is LOSS. Wherever a change is written in a sentence, it is written in words, never as a bare sign: "p95 cut 65% (GAIN)", so it cannot be misread against a receipt that shows the same change as "-65%". Read the Reading column, never the sign alone.

| Class | Rung | What it is | What it can show |
|---|---|---|---|
| T / V | E1 | deterministic tests of the software | the software does what its documentation says |
| S | E2 | a run on a modelled plant | what the governor does to the model, and where it breaks |
| L | E3 | real software, no hardware meter | real decisions on real software; energy there is a declared model |
| P | E4 | a physical meter (a card's own power reading, the processor's energy counters, a site meter) | the hardware's own answer |

Read every number with its class beside it. A modelled number is never quoted as a hardware result. When a receipt's energy line is modelled, the receipt says so.

**What each class can and cannot carry.** A class **T / V** result says that the software computes what its documentation says it computes; it says nothing about whether the governor helps any machine. A class **S**

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.




Every table is made by a tool, never by hand; raw records keep their recorded values, every page that shows them uses the words. A reader can follow one run from its audit line to its row in the benefit sheet.

**Figure 38.** Flowchart: from the audit to the receipt, the table and the one number. The audit (one record a decision) is read by tools/live_reps.py into the paired receipt of one run; runs A, B and C go through tools/confirm_abc.py into the three-run table, each run's engine checked against the fingerprint; tools/readings.py sets GAIN, STEADY or LOSS by the gauge's aim and puts the reading key on top of every table; the benefit sheet and the Omni index carry one number per benchmark. Every table is made by a tool, never by hand.

result says what the governor does to a model of a plant under that model's own native controller; it carries exactly the model's fidelity and no more, and every S line that reports energy says that the energy is a declared model. A class **L** result says what the governor did to real software making real decisions on real requests; where the machines are containers on one host, they are counted in machine-hours in service and their energy, where it is reported, is a declared formula, which the receipt says; the processor seconds of the host are measured. A cloud bill is an L result with a real bill: the machines billed are the provider's own count and the price its list price, and the energy is still not a meter. A class **P** result says what a physical meter read, and only a P result speaks for hardware energy.

**How to read a row.** Every row in every three-run table has the same shape: the gauge and its direction; native's value in run A and omni's value in run A, so the size of the numbers is visible; the paired change in each of the three runs as a percentage of native with its 95% interval; and the reading. Read the reading first, then the intervals, then the sizes. A reading of "GAIN in all three of our runs" with narrow intervals, each clear of zero, is a small, real effect; a reading of "GAIN by the mean of our three runs, beyond the noise in 0 of 3" with intervals of −30% to +25% is a change the runs point to that the test could not tell from its noise, which is a statement about the test's power, and the receipt should say so (a 4-worker fleet cannot show a saving smaller than one machine, section 8.1). A "LOSS in all three runs" row stands in the receipt.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

**Three questions to ask of any class L result.** First, what was native, exactly: which controller, at which published version, with which settings, and were those settings an operator's or ours? (Write them down before the run; a native set anywhere other than the operator's own setting must say so.) Second, what did the governor spend: machines, memory, connections, CPU, its own cost? (Every receipt has the resource rows; a result with a gain and no resource row is not a receipt.) Third, who else could have produced the gain: the order of the arms, the warm-up, the runner's neighbour, the clock? (Rotation of the arm order, the warm-up before every window, the paired design on one machine and the clock rule are the answers; whatever residue they do not remove is written beside the receipt.)

CHAPTER 13

# The Engine Version and the Three-Run Rule

**Every engine version has a fingerprint.** The engine is fingerprinted file by file: one SHA-256 per file and one digest over all of them, held in the version's registry (`OMNI_V5.json` for the engine that stands). Any change to a rule, guard, preset or muscle makes **the next version**, and every receipt is made again on it; no receipt is ever read across versions. When the engine is declared final, the engine that stands then is published as Omni-Compass 1.0.

```
python3 tools/omni_version.py                  # this checkout: its e
    ↪ ngine version, or every file that differs
python3 tools/omni_version.py --commit <sha>   # the engine at the co
    ↪ mmit any receipt ran on
```

A receipt names the commit it ran on, and every table made by rule checks each run's commit against the fingerprint and heads itself with a warning if any run is not the engine it claims or if the three are not separate runs. An operator's own tune (section 4.6) does not change the engine's fingerprint, and must therefore be written down beside the receipt.

**The three-run rule** (the founder's order of 24 October 2026: any change of 0.5% or more is a change; `tools/readings.py`, `three_run_reading`). A result is run as **A, B and C**, three separate runs on the same engine version and profile. The change is the mean of the three runs' changes, each in percent of native, and all three runs are shown beside the reading:

- **GAIN**, **LOSS** or **power given in all three of our runs**: the mean moved 0.5% or more, and every run moved the same way with its 95% interval clear of zero. The word comes from what the gauge is for, never from the sign; a resource spent that rose reads power given, never a loss.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

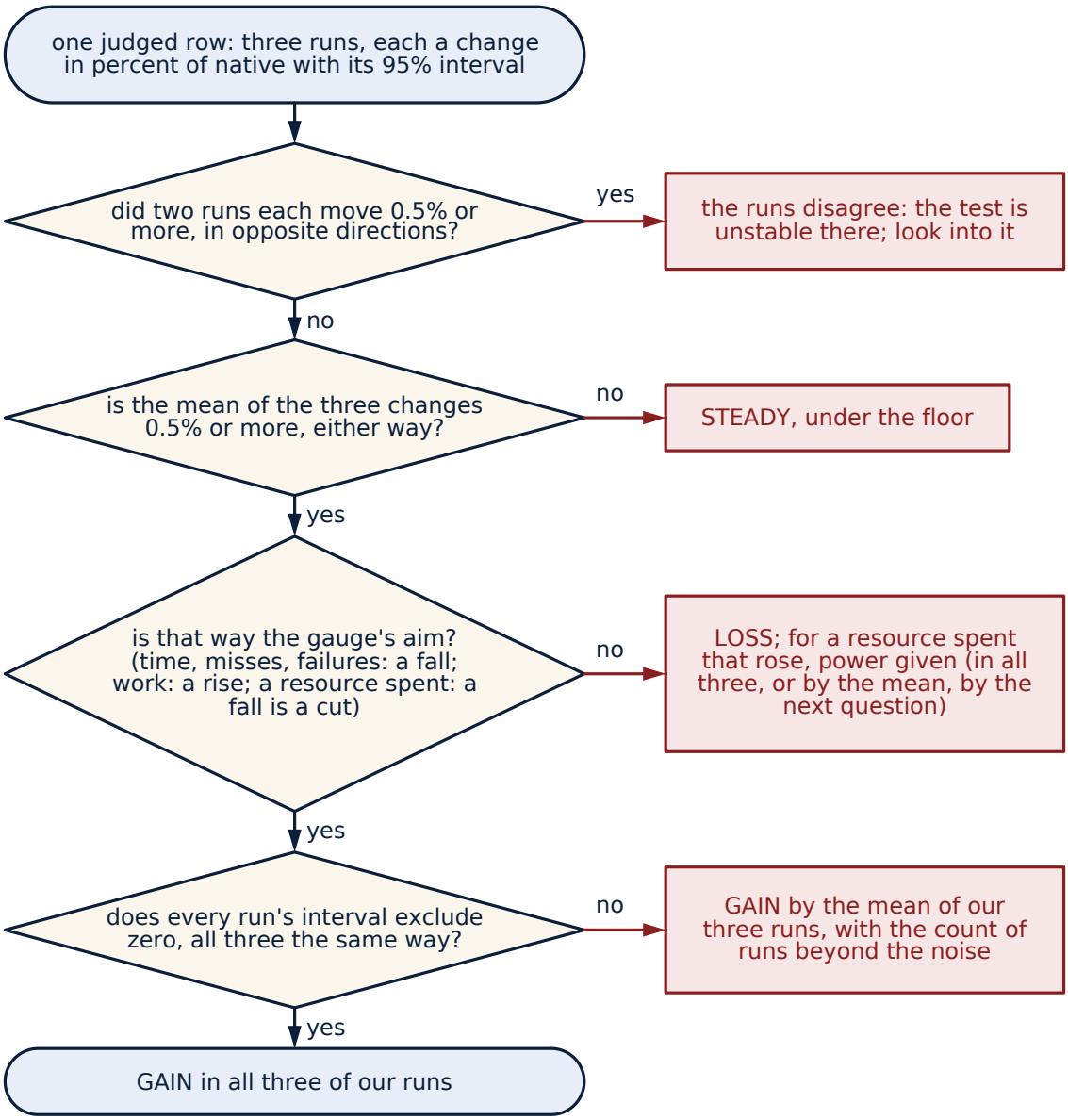



**Figure 39.** Flowchart: the three-run reading rule, as a procedure, as drawn before the rule of 24 October. For one judged row: if any of the three intervals includes zero the row reads STEADY, with the count of runs it holds in; if the three clear intervals do not point the same way the runs disagree and the test is unstable there; if they point the gauge's way the row reads GAIN in all three of our runs, and the other way LOSS, except for a resource spent that rose, which is power given.

Since 24 October the 0.5% floor on the mean of the three runs comes first, and an interval across zero no longer makes a row STEADY: it reads its word by the mean of our three runs, with the count beyond the noise (the text below).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **GAIN**, **LOSS** or **power given by the mean of our three runs**: the mean moved 0.5% or more, but not every run cleared zero the same way; the count of runs beyond the noise stands beside the word ("beyond the noise in 1 of 3").
- **STEADY, under the floor**: the mean moved less than 0.5%. **STEADY, the same**: nothing moved.
- **The runs disagree**: runs that moved 0.5% or more pointed in opposite directions. The test itself is then unstable on that measure, and it is looked into before anything is claimed.
- A count that must never rise (failed requests, lost messages) has no floor: any rise in any run reads LOSS.

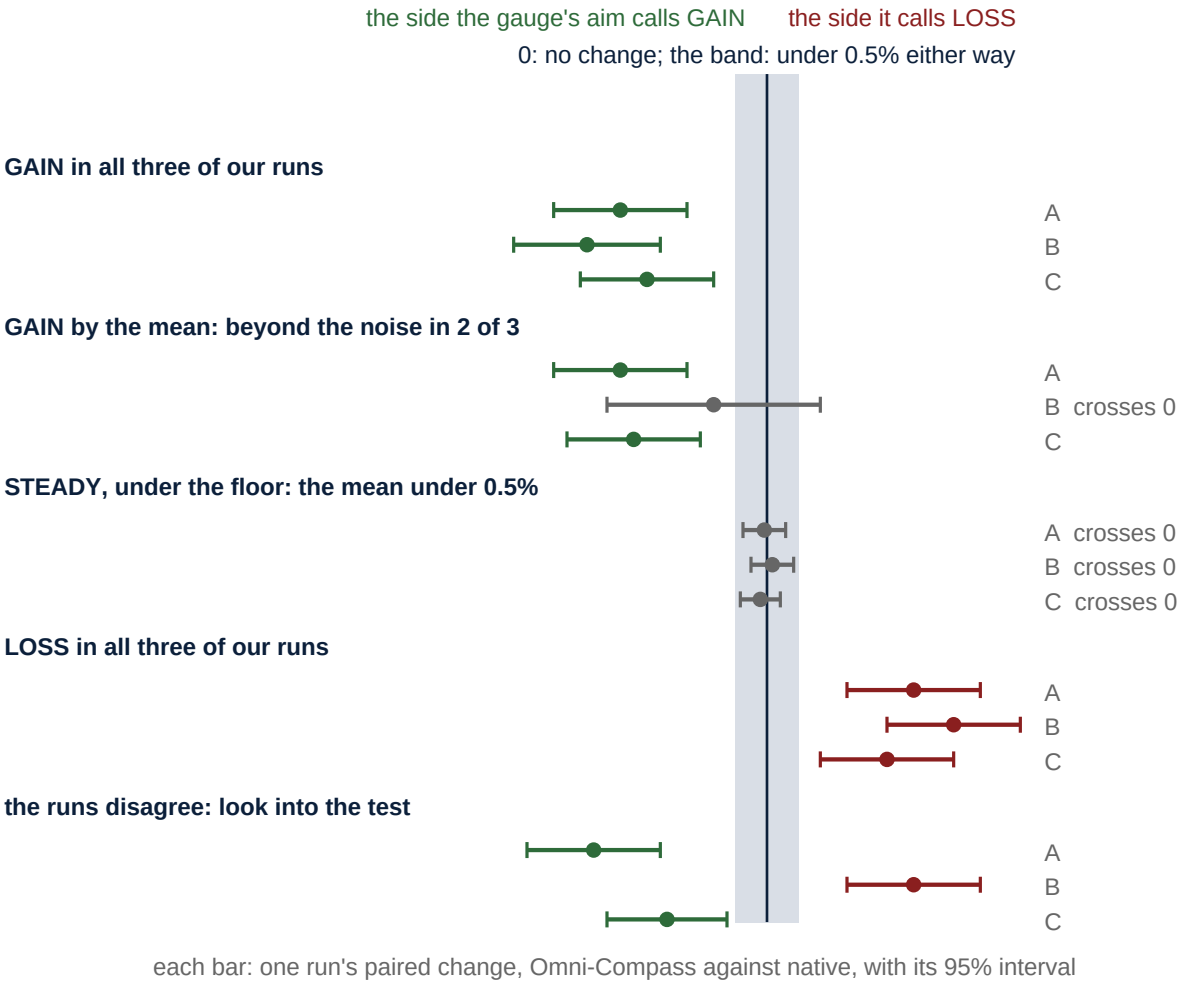



**Figure 40.** The three-run rule, read from the intervals, as drawn before the rule of 24 October (since then a mean change under 0.5% is STEADY and a change of 0.5% or more whose intervals do not all clear zero reads its word by the mean of our three runs). Each row is one gauge; each bar is one run's paired change of Omni-Compass against native with its 95% interval.

GAIN or LOSS in all three of our runs: the same side in all three, every interval clear of zero. STEADY, as drawn: at least one interval includes zero. The runs disagree: clear intervals on both sides, an unstable test to

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

be looked into. Which side reads GAIN is set by what the gauge is for: for time waited, misses and failures a fall; for work done a rise; for a resource spent a fall is a cut and a rise is power given, never a loss.

The three-run table is made by a tool, never by hand: `tools/confirm_abc.py` for the paired Kubernetes and cloud runs, with one sibling tool per kind of raw record for the other stacks (`pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `ycsb_abc.py`, `sysbench_abc.py`), each checking the engine of every run it reads. A deterministic simulator's three runs must reproduce each other to the digit, or to the declared tolerance where the simulator is not bit-reproducible across machines.

**Why three separate runs, and why the readings are what they are.** One run of ten paired repetitions gives one interval, and an interval from one run can be narrow by luck: the machine was quiet that hour, the order of arms happened to favour one side. A second run on another day and another machine is a replication; a third makes a coincidence expensive. The three runs are separate runs, on separate machines, at separate hours, with separate installations of every dependency, so what agrees across them is not an artefact of one machine. The readings are designed so that no reading can be mistaken for a weaker form of "yes": a row reads "in all three of our runs" only when every run moved the same way with its interval clear of zero; a change of 0.5% or more that some run could not tell from the noise keeps its word, "by the mean of our three runs", with the count of runs beyond the noise beside it; a mean under 0.5% is STEADY; and runs of 0.5% or more that point in opposite directions are reported as a disagreement, which is an instability in the test to be found and fixed, not averaged away.

PART SIX

# The Implementer's Guide: Omni-Compass on a Cloud

*For the engineer who installs it on Azure, Google Cloud or Amazon Web Services and keeps it in service: the path and its gates, the permissions, each cloud step by step, the response-time feed, the cloud itself, day two, the graphics processor, observability, change and risk, planning a cloud run, own racks and private clouds, hardening, a fleet, the failure drills, a complete installation, and the questions implementers ask.*

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 14

# The Implementer's Path: From an Empty Account to a Governed Cluster

This part is written for the **implementer**: the platform, site-reliability or infrastructure engineer who is handed Omni-Compass and a cloud account and is responsible for installing it, proving it on the company's own cluster, putting it into service and keeping it there. It assumes fluency with Kubernetes objects, role-based access control (RBAC), the cloud provider's command-line tool and a container registry, and nothing about how the governor is built. Every step names the object it creates, the permission it needs, the evidence it leaves and the check that must pass before the next step begins. Where the three clouds differ, the step is written once per cloud (chapters 16 to 18); where they do not, it is written once here or in chapters 15, 19, 20 and 21.

**The cloud is a wire.** A cloud is the wire between the software end (the owner's programs, Kubernetes and the open-source orchestrators, managers and autoscalers that carry the work) and the hardware end (the bare metal: the GPU chips, the Linux racks, the data center, on site or off site). Omni-Compass can be wired in at all three places, and in each it acts in one place, the software end, on top of the controllers already there: on the software end (the cluster and its services, chapters 7 and 8), on the bare metal (the card and the machine, chapter 22 and section 8.9) and on the wire itself, the cloud of this part. The wire makes no benefit of its own (chapter 20): on a managed cluster the governor runs the whole engine on the cluster and its services, the software end, so a cloud receipt shows that the engine holds there as on site and how much of that end's gain comes through the wire as work per dollar. It cannot show the energy and the throughput gained at the bare metal, because the provider's chips are not under it. An installation that wants the whole benefit installs Omni-Compass at each place it owns, receipts each one where it sits, and composes them into one organism and one bill (`DISCLOSURES.md`, section 9; the bill as the sum of the costs inside a declared boundary, `docs/FACTORY_PROFILE.md`, section 8). The numbers measured so far are in the Owner's Manual; they are not your receipt.

## 14.1 What the implementer delivers

At the end of this part an implementer has delivered five things, each with its own evidence:

| Deliverable | Evidence that it is done |
|---|---|
| A verified copy of the software, at a known engine version | `python3 verify.py` ends `VERIFICATION: PASS`; `python3 tools/omni_version.py` names the version (chapter 33) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Deliverable | Evidence that it is done |
|---|---|
| The governor installed in watch mode, writing nothing | its audit shows a `decision` record every period (the machines and the target it recommends) and no `write` record; every object it reads is listed in chapter 15 |
| A response-time feed for the service, and the service's line written down | the feed file advancing at least every 5 seconds; the line taken from the service's own objective (chapter 19) |
| A paired receipt on the company's own cluster: native against native with Omni-Compass on top | the receipt's rows, each read GAIN, STEADY or LOSS, and the run's commit and engine version (chapters 11 to 13) |
| The governor in service, with its OFF switch, its watchdog and its alarms wired | the switch tested, the watchdog's hand-back drilled, the alert route proven by a test page (chapter 21) |

None of the five is skipped, and they are delivered in that order. The order is not bureaucracy: each one is the evidence the next one stands on. A governor that has not watched cannot be judged before it writes; a receipt without a written line cannot be read; a governor in service without a drilled OFF switch is not in service, it is at large.

## 14.2 What Omni-Compass needs from the cluster, and what it never touches

**What it needs.** A Kubernetes cluster on a supported minor version of the provider's managed service; the Kubernetes Metrics API (`metrics.k8s.io`, served by metrics-server: built into Azure Kubernetes Service and Google Kubernetes Engine, installed by the implementer on Amazon Elastic Kubernetes Service); a HorizontalPodAutoscaler (HPA) on each service it governs, scaling on CPU utilisation; a cluster autoscaler on the work node pools (the provider's own, or Karpenter on Amazon); a namespace and a service account for the governor; and a response-time feed for each service whose line it holds (chapter 19). In-place pod resize (the `pods/resize` subresource) is needed where the governor conveys processor to running pods, which it does in a cloud as on site (chapter 20).

**What it never touches, in any mode.** The governor never creates, deletes, evicts or restarts a pod; never changes a container image, a command, an environment variable or a request; never changes a node's machine type or a pool's configuration in the provider; never holds a cloud credential (it acts through the Kubernetes API alone); and never writes a setting it has not first recorded on the object it writes, so that every setting can be put back from the object itself (chapter 9). In a cloud it moves the same muscles as on site, on top of the cloud's own autoscaler, which stays in every arm and alone adds machines (chapter 20).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 14.3 The eight phases and their gates

| Phase | What happens | Gate to the next phase | Typical time |
|---|---|---|---|
| 0. Verify | the copy is checked against its manifest and engine version | `VERIFICATION: PASS`, the version named | under an hour |
| 1. Account and cluster | the cloud account, quotas, the cluster with a system pool and work pools | the cluster autoscaler scales a work pool up and down on its own | half a day |
| 2. Identity | namespace, service account, the read-only role | `kubectl auth can-i` answers exactly the table of chapter 15 | an hour |
| 3. Watch | the governor installed in watch mode | an audit with decisions and no write, for at least one full day of the service's own load | one to seven days |
| 4. Feed and line | the response-time feed and the written line | the feed's age under 15 seconds at every decision; the line signed off by the service's owner | a day |
| 5. Paired run | native against native with Omni-Compass on top, on the company's own cluster | a receipt read row by row, every LOSS explained or fixed and rerun | two to five days |
| 6. Govern | the role that lets it write granted, Omni-Compass switched from watching to writing, the OFF switch and the watchdog wired | the OFF switch drilled: every setting back and read back within 60 seconds | a day |
| 7. Day two | upgrades, rollbacks, audits, alarms, fleet roll-out | quarterly drill of the OFF switch and the watchdog | ongoing |

Phases 3 and 5 are the long ones, and they are long on purpose: a governor is judged on the service's own load, which has its own daily and weekly shape. A watch phase shorter than one full cycle of the service's own load (a day for most services, a week for a service with a weekly peak) has not seen the service.

## 14.4 Roles, approvals and the separation of duties

| Role | Holds | Signs off |
|---|---|---|
| Implementer | the namespace, the service account, the install, the feed | phases 2 to 5 |
| Service owner | the service's line (its response-time objective) and its HPA | the line (phase 4) and the receipt (phase 5) |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

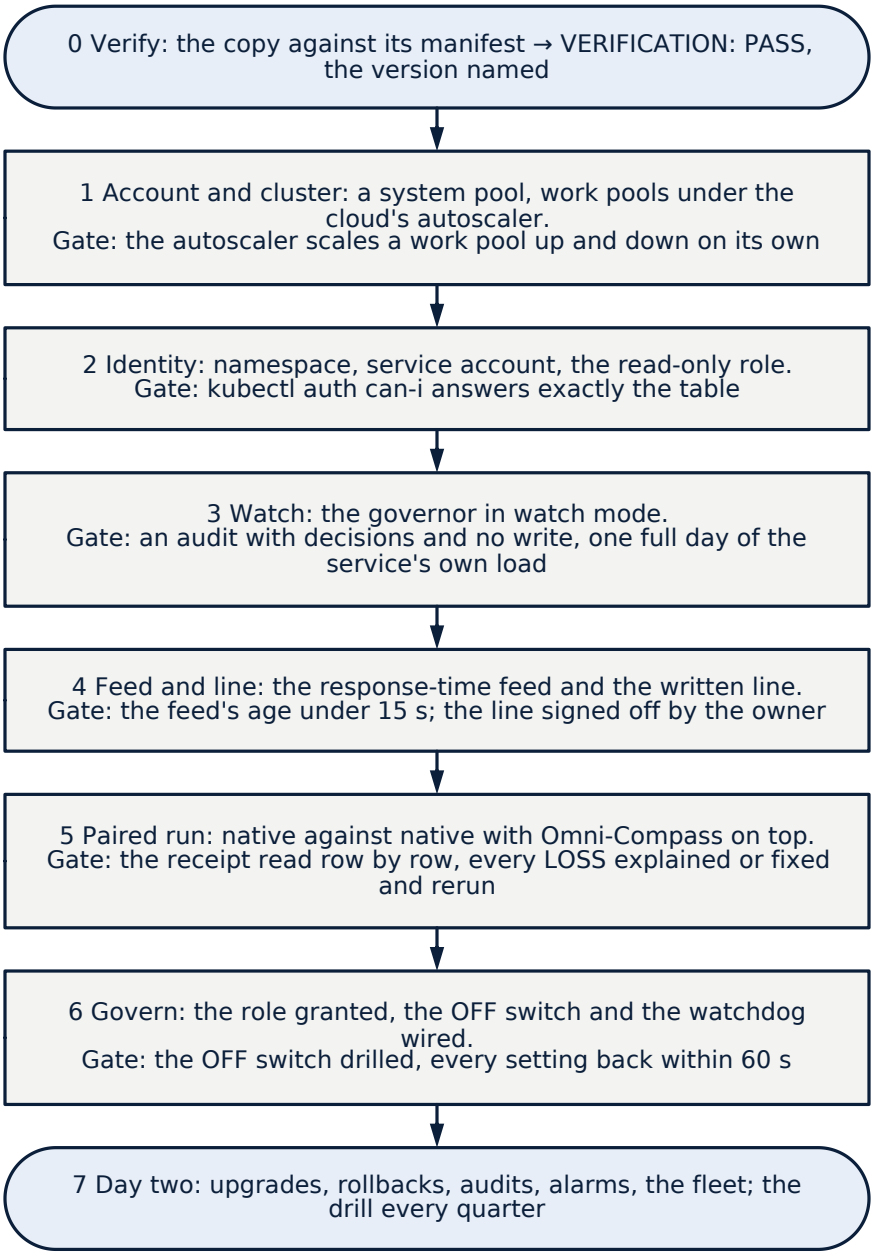



**Figure 41.** Flowchart: the implementer's eight phases and their gates. From the verified copy through the account and the cluster, identity, the watch, the feed and the line, the paired run and the governing grant to day two; each phase has a gate that must pass before the next begins, and the OFF switch is drilled before the governor writes.

| Role | Holds | Signs off |
|---|---|---|
| Platform owner | the cluster, its pools and its autoscaler profile | the governing role (phase 6) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Role | Holds | Signs off |
|---|---|---|
| Security | the RBAC review and the OFF switch | the role table (phase 2) and the OFF switch drill (phase 6) |
| Finance (cloud cost) | the bill's gauge and its list prices | the bill rows of the receipt (phase 5) |

The OFF switch (`python3 tools/omni_switch.py off`, chapter 9) is held by someone who is not the implementer. It is the one control in this part whose purpose is to stop the governor when the people who installed it are not available or not trusted, and it is only worth what that separation makes it worth.

## 14.5 What it costs to implement

The governor's own cost is small and is measured in every receipt (the row "Omni's own CPU (cores), mean"): one process, a request of 50 millicores and 128 MiB, a limit of 500 millicores and 256 MiB in the shipped manifest. The cost of an implementation is the cost of the evidence: a cluster for the paired run, sized like the service's own, for the hours the run takes. On the managed services in this part, a two-vCPU worker costs between about USD 0.07 and USD 0.10 an hour at list price (Google's e2-standard-2, Azure's Standard_D2s_v4 and Amazon's m5.large in their eastern United States regions; check each provider's price list on the day), a control plane costs nothing or about USD 0.10 an hour depending on the provider and tier, and a paired run of five repetitions on four workers is a few tens of dollars. The run builds a fresh cluster for every arm and deletes it when the arm ends, so nothing keeps billing.

## 14.6 How to read the rest of this part

Chapter 15 sets the permissions, identically on every cloud. Chapters 16, 17 and 18 build the cluster and install the governor on Azure, Google Cloud and Amazon Web Services, click by click and command by command. Chapter 19 sets up the response-time feed and the line, on any cloud. Chapter 20 is the cloud itself: the whole engine on a managed cluster, what a cloud run can show (the engine holding as on site, and work per dollar), and how to read the bill and the alarm minutes. Chapter 21 is day two: running the governor as a service, upgrades, rollbacks, the watchdog, alarms and a fleet of clusters. Chapter 21 is the graphics processor, written for the implementer who will put the governor on a card, rented or owned. Chapter 23 connects the governor to the company's own monitoring in both directions, and chapter 24 maps the installation onto change management, risk and compliance. Chapter 25 bounds, before any run, the saving a cloud pool can show and where in the day it lives. Chapter 26 is for clusters on the company's own racks and in private clouds; chapter 27 hardens the installation and its supply chain; chapter 28 rolls it out across a fleet and under continuous delivery; chapter 29 lists every failure to expect, with the drills that rehearse them; and chapter 30 is the whole path at once, on one cluster, start to finish. Chapter 31 is the implementer's questions, answered. The accounts and the paired run from GitHub are in Appendices F to H, the install manifests in Appendix I, and the checklists in Appendix J.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 15

# Identity, Permissions and Least Privilege on a Managed Cluster

The governor acts on a cluster only through the Kubernetes API, as one service account, with the verbs its level needs and no more. This chapter gives those verbs exactly, level by level, the same on every cloud; the cloud-specific steps (registry, cluster credentials, the provider's own identity) are in chapters 16 to 18.

## 15.1 One namespace, one service account, one process

The governor runs as one Deployment of one replica in its own namespace, `omni-compass`, under the service account `omni-compass`. One replica is a rule, not a default: the governor is the one writer of every setting it moves (section 5.2), and two copies would be two writers. A second replica for availability is not needed and must not be added; if the one process dies, the watchdog of chapter 21 puts every setting back from the records on the objects themselves, and the native controllers carry the service as they did before the governor was installed.

The pod runs as a non-root user (UID 10001) with a read-only root filesystem, no Linux capabilities, no privilege escalation and the runtime's default seccomp profile. It writes only two places: its audit log (an `emptyDir` volume mounted at `/var/log/omni`, or a persistent volume where the audit must outlive the pod, section 21.4) and `/tmp` (an `emptyDir` for its kill file). It needs no secret, no host path, no host network and no service exposed to anything.

## 15.2 The verbs, level by level

The table is the complete list. A verb not in it is not needed at that level and must not be granted.

| Level | API group | Resources | Verbs | Why |
|---|---|---|---|---|
| Watch (every level) | core | `nodes`, `pods` | `get`, `list` | read the machines, their allocatable processor and the pods on them |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Level | API group | Resources | Verbs | Why |
|---|---|---|---|---|
| Watch | `metrics.k8s.io` | `nodes`, `pods` | `get`, `list` | read processor in use |
| Watch | `autoscaling` | `horizontalpodautoscalers` | `get`, `list` | read each autoscaler's target, range and current replicas |
| Watch | `apps` | `deployments`, `replicasets`, `daemonsets` | `get`, `list` | read what each autoscaler scales, and tell workload pods from per-node agents |
| Watch | core | `configmaps` (the one security-hold ConfigMap, by name) | `get` | read the security hold, where one is wired |
| Machines | core | `nodes` | `patch` | park a machine (a `PreferNoSchedule` taint) and wake it (remove the taint); a machine someone else cordoned is never uncordoned |
| Machines | core | `pods` | `patch` | mark the pods on a parked machine first to go (the `pod-deletion-cost` annotation), so the autoscaler's own scale-down removes them first |
| Pod target | `autoscaling` | `horizontalpodautoscalers` (each governed one, by name) | `patch`, `update` | move the autoscaler's CPU target inside its cover, and record the operator's own target on the object |
| Processor conveyance | core | `pods/resize` | `get`, `patch` | resize a running pod's processor limit in place; no pod is restarted |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Level | API group | Resources | Verbs | Why |
|---|---|---|---|---|
| Pod muscle of its own (`--own-replicas`, the arm by itself; never in a cloud without the founder's word) | `apps` | `deployments/scale` (each named workload) | `get`, `patch`, `update` | set the replica count of a workload no autoscaler governs through its own scale subresource, never below the operator's count (`deploy/rbac-own-pods.yaml`); under RBAC `deployments/scale` is a resource of its own, which a grant on `deployments` does not carry |
| Batch admission (where wired) | `batch` | `jobs` | `patch` | admit held work |

Two verbs are absent from every level and are refused by review if they appear: `delete` on anything, and `create` on `pods/eviction`. The governor does not delete or evict; a parked machine is emptied by the cluster autoscaler's own scale-down, as it would be without the governor, and only once its pods have gone by the autoscaler's own rules.

## 15.3 The cloud's grant, complete

On a managed cluster the governor runs the whole engine at the max (chapter 20; the founder's order of 31 October struck the machines-alone grant of 14 October), so the grant is the watch role plus the writes the kind rig grants (`deploy/kind/rbac-omni.yaml`), in one file (`deploy/install/rbac-cloud.yaml`): cluster-wide, the machines and the pods' deletion-cost mark; in the service's own namespace, by name, its autoscaler, its Deployment's record of the original limit, and the running pods' processor. Set `<namespace>` and `<deployment>` to the service that `--sensed` and `--cap-deployments` name, one Role and RoleBinding per governed service, and apply it in place of the level-2 and level-3 files of chapter 7:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: {name: omni-compass-cloud-machines}
rules:
  - apiGroups: [""]
    resources: [nodes]
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    verbs: [patch]
  - apiGroups: [""]
    resources: [pods]
    verbs: [patch]
---
apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRoleBinding
metadata: {name: omni-compass-cloud-machines}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-cloud-machines}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: omni-compass-cloud-service, namespace: "<namespace>"}
rules:
  - apiGroups: [autoscaling]

    resources: [horizontalpodautoscalers]
    resourceNames: ["<deployment>"]
    verbs: [get, patch, update]
  - apiGroups: [apps]
    resources: [deployments]
    resourceNames: ["<deployment>"]

    verbs: [get, patch, update]
  - apiGroups: [""]
    resources: [pods/resize]
    verbs: [get, patch]
---
apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding
metadata: {name: omni-compass-cloud-service, namespace: "<namespace>"}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: Role, name: omni-
    ↪ compass-cloud-service}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The watch role is in the install manifest (`deploy/install/omni-compass.yaml`, printed in Appendix I); it grants only `get` and `list`. Together the two files are everything the governor can do in a cloud, and every write it makes is in its audit with the command and the reason.

## 15.4 Proving the grant before the governor runs

Before the governor is started at any level, the implementer proves the grant from outside it, as the service account, and keeps the output with the installation record:

```
SA=system:serviceaccount:omni-compass:omni-compass
for v in "get nodes" "list pods" "get horizontalpodautoscalers.autosca
    ↪ ling" "list deployments.apps" \
         "patch nodes" "patch pods" "patch horizontalpodautoscalers.au
    ↪ toscaling" "patch pods/resize" "patch deployments/scale" \
         "delete pods" "create pods/eviction" "delete nodes" "create p
    ↪ ods"; do
  printf '%-48s %s\n' "$v" "$(kubectl auth can-i $v --as=$SA -A)"
done
```

In a cloud the answers across all namespaces are `yes` for the four reads and for `patch nodes` and `patch pods`, and `no` for everything else, including `patch deployments/scale`, `delete pods`, `create pods/eviction`, `delete nodes` and `create pods`; the service's own writes are granted in its namespace by name, so prove them there, each answering `yes`:

```
NS=<namespace>; D=<deployment>
kubectl auth can-i patch horizontalpodautoscalers.autoscaling/$D -n $N
    ↪ S --as=$SA
kubectl auth can-i patch deployments.apps/$D -n $NS --as=$SA
kubectl auth can-i patch pods --subresource=resize -n $NS --as=$SA
```

Any other answer is a grant to correct before the governor starts, not after. Where `--own-replicas` names a workload, prove its scale write by name (`kubectl auth can-i patch deployment/NAME --subresource=scale -n NAMESPACE --as=$SA`): a grant on `deployments` alone answers `yes` to `patch deployment/NAME` and `no` here, and the governor's every pod write would be refused.

## 15.5 No cloud credential

In a cloud the governor holds no credential of the cloud provider: no Azure service principal, no Google service account key, no Amazon access key, no workload-identity binding. It parks and wakes machines, moves the autoscalers' targets and resizes the pods' processor through the Kubernetes API; the provider's own cluster autoscaler, with the provider's own identity, makes and deletes machines. This is a security property

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

worth stating to a reviewer in one sentence: an attacker who took the governor's pod would hold a Kubernetes service account that can read the cluster's shape, taint nodes, annotate pods, move an autoscaler's target and resize a pod's processor, and nothing that can create or delete anything, spend money in the provider directly, open a network path or read a secret.

The paired runs from GitHub (Appendices F to H) are different: there the workflow itself builds and deletes clusters, and it holds a provider credential as a GitHub secret (`AZURE_CREDENTIALS`, `GCP_CREDENTIALS`, `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`). That credential belongs to the test harness, never to the governor, and it is removed when the evaluation ends.

## 15.6 Network and data

The governor talks to one endpoint: the cluster's API server, through a single local proxy it starts itself (one `kubectl proxy` for the life of the process; `--no-api-proxy` falls back to one `kubectl` call per read, at a higher cost of its own). A NetworkPolicy may restrict its egress to the API server and, where the response-time feed is read over the network, to the feed; it accepts no ingress. It sends nothing off the cluster. The audit holds object names, replica counts, machine counts, response-time statistics and the governor's reasons; it holds no request payload and no personal data, because the governor never sees either.

## 15.7 The record an installation keeps

| Item | Where |
|---|---|
| The engine version and the copy's verification receipt | `python3 tools/omni_version.py`, `results/VERIFY_RECEIPT.txt` (chapter 33) |
| The grant as applied, and the `kubectl auth can-i` output of section 15.4 | the installation record |
| The governor's command line (its arguments in the Deployment) | the Deployment object, kept with the installation record |
| The service's line, and who signed it | the installation record (chapter 19) |
| The audit of every decision and every write | the audit volume (chapter 21) |
| Every OFF-switch drill, with the seconds to hand back | the installation record (chapter 21) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

CHAPTER 16

# Azure Kubernetes Service, Step by Step

This chapter takes an implementer from an Azure subscription to Omni-Compass watching an Azure Kubernetes Service (AKS) cluster, ready for the governing switch of chapter 20. Every command runs in Azure Cloud Shell (Bash) or on a workstation with the Azure CLI (`az`), `kubectl` and Docker; nothing else is needed. Replace the names in angle brackets.

## 16.1 The subscription and its allowances

1. **Sign in and choose the subscription.** `az login`, then `az account set --subscription <subscription-id>`.
2. **Register the provider once:** `az provider register --namespace Microsoft.ContainerService --wait`.
3. **Read the allowances before building anything.** Azure counts vCPUs per region and, separately, per machine family in that region; a cluster fails on the family's allowance, not the region's. Read both:

``bash az vm list-usage -l <region> -o table | grep -Ei "Total Regional vCPUs|Standard DSv4|Standard DSv5|Standard Dv4" ``

A new pay-as-you-go subscription commonly has 10 vCPUs a family and a few hundred a region. A work pool of four two-vCPU machines plus a one-machine system pool needs 10 vCPUs of the families used. Raise an allowance in the portal (**Quotas**, then **Compute**, the family and region, **Request quota increase**) before the cluster is built, not after a half-built cluster fails.

4. **Pick the machine family the region offers.** `az vm list-skus -l <region> --size Standard_D2 -o table` lists what the subscription may create there. A size marked "NotAvailableForSubscription" is refused at build time.

## 16.2 The cluster: a system pool, a work pool, the autoscaler

The layout is the one every receipt in this manual is made on, and it is the right layout for production too: the cluster's own add-ons and the governor on a small **system pool** that no workload may land on, and the service on one or more **work pools** under Azure's cluster autoscaler, each labelled `omni-role=work`.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
RG=<resource-group>; NAME=<cluster>; LOC=<region>; SIZE=Standard_D2s_v
    ↪ 4
az group create -n $RG -l $LOC -o none
az aks create -g $RG -n $NAME -l $LOC --tier standard --node-count 1 -
    ↪ -node-vm-size $SIZE \
  --nodepool-name system --nodepool-taints CriticalAddonsOnly=true:NoS
    ↪ chedule --nodepool-labels omni-role=system \
  --cluster-autoscaler-profile scale-down-unneeded-time=10m scale-down
    ↪ -delay-after-add=10m scan-interval=10s \
  --generate-ssh-keys -o none
az aks nodepool add -g $RG --cluster-name $NAME -n work0 --mode User -
    ↪ -node-vm-size $SIZE --labels omni-role=work \
  --node-count 2 --enable-cluster-autoscaler --min-count 2 --max-count
    ↪  <pool top> -o none
az aks get-credentials -g $RG -n $NAME
kubectl get nodes -L omni-role,agentpool
```

Four choices in those lines matter, and each is the operator's, not the governor's:

- **The tier.** `standard` (about USD 0.10 an hour a cluster, with Azure's uptime agreement) or `free`. The control plane is not in the bill the receipts count, which is worker machines only. In some regions at busy hours Azure refuses new free-tier clusters ("creating a new cluster is unavailable at this time in region"); the standard tier is the way past.
- **The autoscaler profile.** `scale-down-unneeded-time` is how long a machine must be unneeded before Azure deletes it, and so how long a machine the governor has parked keeps billing once its pods are gone. Azure's default is 10 minutes; the paired receipts in this manual used 2 minutes in both arms. Choose it for the service, write it down, and keep it the same in both arms of any paired run. The governor never changes it.
- **The pool's floor.** `--min-count` is the autoscaler's own floor. The governor's floor of two machines in service (`MIN_NODES`, section 4.4) is applied on top of it; set the pool's floor to at least two for a service that must survive one machine's loss.
- **The pool's top.** `--max-count` is the most machines Azure will make. The governor never asks Azure for a machine; it parks and wakes the ones Azure made (chapter 20), so the pool's top is the operator's cost ceiling, as it is without the governor.

Several machine families can be pooled, each as its own work pool with its own top, all labelled `omni-role=work`; that is how a fleet gets past a ten-vCPU allowance a family, and the governor treats all labelled pools as one set of machines.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Prove the autoscaler before anything else.** Scale a test Deployment past the pool's capacity and back, and watch Azure add and delete a machine (`kubectl get nodes -w`). A cluster whose autoscaler has not been seen to work both ways is not ready for a governor that relies on it to delete the machines it parks.

## 16.3 metrics-server and the service's autoscaler

AKS runs metrics-server in every cluster; check it with `kubectl top nodes`. The service must have a HorizontalPodAutoscaler on CPU utilisation, with requests set on its containers (an HPA cannot compute utilisation without them):

```
kubectl autoscale deployment <service> -n <namespace> --cpu-percent=<t
    ↪ arget> --min=<min> --max=<max>
kubectl get hpa -n <namespace>
```

The target, the minimum and the maximum are the service owner's, written down before the governor is installed. The governor moves the target inside that cover, raises the floor ahead of a climb (the pod reflex) and the cap within its replica room only while the cap binds and the line is threatened (Appendix K, `--replica-ceiling`), hands each back once the demand holds still, records the owner's own settings on the object so the reset puts them back, and the receipt's native arm runs on the owner's settings untouched.

## 16.4 The registry and the governor's image

```
ACR=<registry-name>
az acr create -g $RG -n $ACR --sku Basic -o none
az aks update -g $RG -n $NAME --attach-acr $ACR -o none
az acr build -r $ACR -t omni-compass:<version> -f deploy/Dockerfile .
az acr build -r $ACR -t omni-compass-cloud:<version> -f deploy/Dockerf
    ↪ ile.cloud \

  --build-arg BASE=$ACR.azurecr.io/omni-compass:<version> .
```

The first image is the shipped one (`deploy/Dockerfile`): the governor, non-root, read-only, with `kubectl`. The second adds what a cloud needs to move machines and to feed itself, and is written once for every cloud in section 19.4. Tag images with the engine version printed by `python3 tools/omni_version.py` and the commit, never `latest`, so the audit of any decision can be tied to the exact code that made it.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 16.5 Watch first

```
kubectl apply -f deploy/install/omni-compass.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ ACR.azurecr.io/omni-compass:<version>
kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 5 /var/log
    ↪ /omni/audit.jsonl
```

The install manifest (Appendix I) creates the namespace, the service account, the read-only role and one Deployment running `--mode observe`. In watch mode the governor computes every decision it would make and writes none: each `decision` record carries the machines and the pod target it recommends (`nodes_recommended`, `hpa_target_recommended`), and no `write` record appears. Leave it watching for at least one full cycle of the service's own load (section 13.3). To keep the audit past a pod restart, mount a persistent volume at `/var/log/omni` (section 21.4); on AKS the `managed-csi` storage class provides one.

## 16.6 The paired run on Azure

The receipt that decides whether the governor goes into service is a paired run on the company's own service: native against native with Omni-Compass on top, the same load in both arms, a fresh cluster per arm, repeated (chapter 11). The repository's own workflow `aks-metered` runs exactly that from GitHub, on clusters it builds and deletes itself, and counts Azure's billed machines every 15 seconds at list price; Appendix F connects it to a subscription, click by click. To run it on a company's own service instead of the reference service, the service's manifests replace `deploy/kind/demo.yaml` in a fork, and its line replaces the reference line of 500 ms (chapter 19).

## 16.7 Tearing down

`az group delete -n $RG --yes --no-wait` deletes the cluster, its machines and its registry. Every paired run of the workflow deletes its own resource group at the end of every repetition, whatever happened, so nothing keeps billing; a run cancelled by hand runs the same deletion as its last step.

## 16.8 What goes wrong on Azure, and what to do

| Symptom | Cause | Fix |
|---|---|---|
| `QuotaExceeded` or `OperationNotAllowed` on `az aks create` or `nodepool add` | the family's vCPU allowance in the region | request the family's increase; or add a second work pool in another family |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

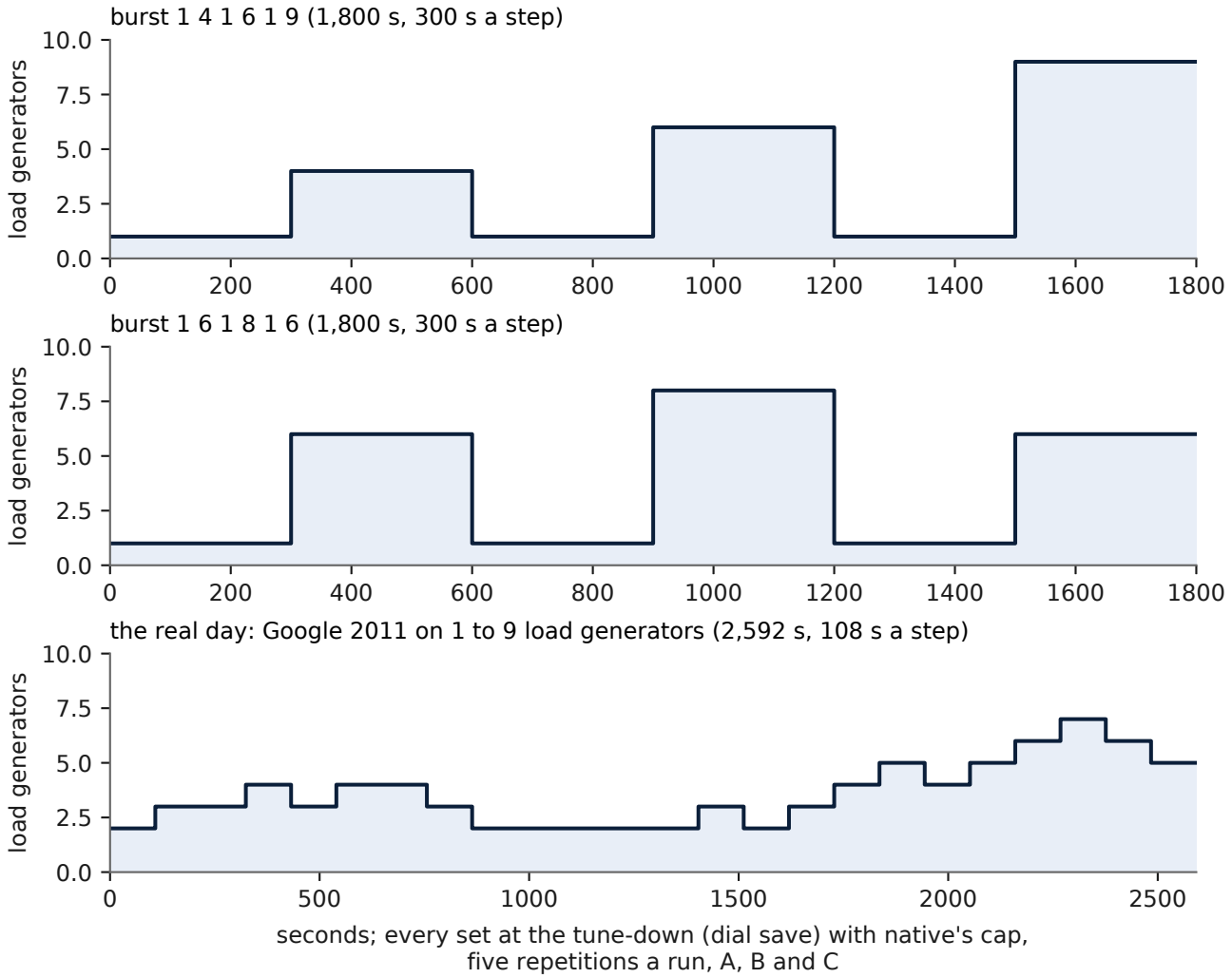



**Figure 42.** The load schedules of the cloud sets: the two bursts (1 4 1 6 1 9 and 1 6 1 8 1 6, 300 s a step) and the real day (the Google 2011 trace on one to nine load generators, 108 s a step). Every set runs at the tune-down with native's cap, five repetitions a run, as runs A, B and C; the load generator alone reads the schedule, never the governor.

| Symptom | Cause | Fix |
|---|---|---|
| "creating a new cluster is unavailable at this time in region" | Azure's free-tier capacity in the region | `--tier standard`, or another region |
| A size "is not available for this subscription" | the family is not offered there | `az vm list-skus` and choose an offered size (the receipts used Standard_D2s_v4 and Standard_D2_v4 in eastus) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Symptom | Cause | Fix |
|---|---|---|
| Pods `Pending`, no machine added | the pool is at its `--max-count`, or the pods do not fit any machine size in the pool | raise the top, or add a pool of a larger size |
| A parked machine keeps billing for minutes | `scale-down-unneeded-time` (10 minutes by default) | expected; it is native's own setting, kept the same in both arms |
| The governor's image will not pull | the registry is not attached | `az aks update --attach-acr` |

CHAPTER 17

# Google Kubernetes Engine, Step by Step

This chapter takes an implementer from a Google Cloud project to Omni-Compass watching a Google Kubernetes Engine (GKE) cluster, ready for the governing switch of chapter 20. Every command runs in Cloud Shell or on a workstation with the Google Cloud CLI (`gcloud`), `kubectl` and Docker. The layout and the order are those of chapter 16; only the provider's commands differ.

## 17.1 The project and its allowances

1. **Sign in and choose the project.** `gcloud auth login`, then `gcloud config set project <project-id>`.
2. **Enable the two services once:** `gcloud services enable container.googleapis.com compute.googleapis.com artifactregistry.googleapis.com`.
3. **Read the allowances.** Compute Engine counts CPUs per region (the `CPUS` quota) and, for some families, per family (for example `N2_CPUS`); a new project's regional CPU quota is often 24 or less. Read them with `gcloud compute regions describe <region> --format="table(quotas.metric,quotas.limit,quotas.usage)"` and raise them in the console (**IAM and Admin**, **Quotas**) before the cluster is built.
4. **Pick the zone and machine type.** The receipts' reference is `e2-standard-2` (two vCPUs) in `us-central1-a`.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 17.2 The cluster: a system pool, a work pool, the autoscaler

```
NAME=<cluster>; ZONE=<zone>; MACHINE=e2-standard-2
gcloud container clusters create $NAME --zone $ZONE --num-nodes 1 --ma
    ↪ chine-type $MACHINE \
  --node-taints CriticalAddonsOnly=true:NoSchedule --node-labels omni-
    ↪ role=system \
  --autoscaling-profile balanced --release-channel regular

gcloud container node-pools create work0 --cluster $NAME --zone $ZONE
    ↪ --machine-type $MACHINE \
  --num-nodes 2 --enable-autoscaling --min-nodes 2 --max-nodes <pool t
    ↪ op> --node-labels omni-role=work
gcloud container clusters get-credentials $NAME --zone $ZONE
kubectl get nodes -L omni-role
```

The default node pool created with the cluster becomes the system pool: one machine, tainted so that no workload lands on it, labelled `omni-role=system`. The work pool carries the label `omni-role=work`, which is how the governor knows which machines it may park.

**The autoscaling profile.** GKE offers two: `balanced` (Google's default) and `optimize-utilization`, which removes underused machines sooner and so lets a parked, emptied machine leave the bill sooner. The receipts used `optimize-utilization` in both arms. As on Azure, the profile is native's own setting: choose it for the service, write it down, keep it the same in both arms of any paired run. The governor never changes it.

**Prove the autoscaler.** As in section 16.2: push a test Deployment past the pool and back, and watch a machine come and go. GKE's autoscaler may take several minutes to remove a machine even under `optimize-utilization`; that time is the time a parked machine keeps billing after its pods have gone, and it is the same in both arms.

## 17.3 metrics-server and the service's autoscaler

GKE runs metrics-server; check with `kubectl top nodes`. The service needs a CPU HorizontalPodAutoscaler and container requests, exactly as in section 16.3. GKE Autopilot clusters are a different product: their nodes are managed by Google and not visible as a pool Omni-Compass can park, so its machine lever has nothing to move there. Use a Standard cluster.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 17.4 The registry and the governor's image

```
REPO=<region>-docker.pkg.dev/<project-id>/omni-compass
gcloud artifacts repositories create omni-compass --repository-format=
    ↪ docker --location=<region>
gcloud auth configure-docker <region>-docker.pkg.dev
docker build -t $REPO/omni-compass:<version> -f deploy/Dockerfile . &&
    ↪  docker push $REPO/omni-compass:<version>

docker build -t $REPO/omni-compass-cloud:<version> -f deploy/Dockerfil
    ↪ e.cloud \
  --build-arg BASE=$REPO/omni-compass:<version> . && docker push $REPO
    ↪ /omni-compass-cloud:<version>
```

The cluster's nodes pull from an Artifact Registry repository in the same project with their default service account; a repository in another project needs `roles/artifactregistry.reader` granted to the nodes' service account there.

## 17.5 Watch first

```
kubectl apply -f deploy/install/omni-compass.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ REPO/omni-compass:<version>
kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 5 /var/log
    ↪ /omni/audit.jsonl
```

The governor watches and writes nothing (section 16.5). For an audit that outlives the pod, the `standard-rwo` storage class provides a persistent volume (section 21.4).

## 17.6 The paired run on Google Cloud

The workflow `gke-metered` is the same design as Azure's, on clusters it builds and deletes itself, with Google's billed machines counted every 15 seconds at the list price of the machine type; Appendix G connects it to a project, click by click. It runs the whole engine at the max, as every cloud set does from 31 October (chapter 20), and its run title carries the load and the dial.

## 17.7 Tearing down

`gcloud container clusters delete $NAME --zone $ZONE` deletes the cluster and its machines; `gcloud artifacts repositories delete omni-compass`

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`--location=<region>` deletes the images. Every paired run deletes every cluster it made, whatever happened, as its last step.

## 17.8 What goes wrong on Google Cloud, and what to do

| Symptom | Cause | Fix |
|---|---|---|
| `Quota 'CPUS' exceeded` | the region's CPU quota | request an increase; or a smaller machine type |
| `ZONE_RESOURCE_POOL_EXHAUSTED` | the zone has no capacity of that type now | another zone in the region, or another machine type |
| The work pool never shrinks | the autoscaler sees a pod it may not move (local storage, a pod without a controller, a PodDisruptionBudget at zero) | find the pod in `kubectl describe` of the autoscaler's status ConfigMap; it holds native too |
| The governor's image will not pull | the nodes' service account cannot read the repository | grant `roles/artifactregistry.reader` |
| No pool for the governor to park on an Autopilot cluster | Autopilot hides the nodes | use a Standard cluster |

CHAPTER 18

# Amazon Elastic Kubernetes Service, Step by Step

This chapter takes an implementer from an Amazon Web Services account to Omni-Compass watching an Amazon Elastic Kubernetes Service (EKS) cluster, ready for the governing switch of chapter 20. Every command runs in AWS CloudShell or on a workstation with the AWS CLI (`aws`), `eksctl`, `helm`, `kubectl` and Docker. EKS differs from the other two in one respect that matters: it ships neither a cluster autoscaler nor metrics-server, and the implementer installs both.

## 18.1 The account and its allowances

1. **Sign in.** `aws configure` with an identity allowed to create EKS clusters, EC2 instances, IAM roles and CloudFormation stacks (eksctl builds the cluster through CloudFormation).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. **Read the allowances.** EC2 counts vCPUs of on-demand standard instances per region (the quota "Running On-Demand Standard (A, C, D, H, I, M, R, T, Z) instances"). Read it with `aws service-quotas get-service-quota --service-code ec2 --quota-code L-1216C47A --region <region>` and raise it in the Service Quotas console before building.
3. **Pick the region and instance type.** The receipts' reference is `m5.large` (two vCPUs) in `us-east-1`.

## 18.2 The cluster: a system node group, a work node group, the autoscaler

eksctl reads the whole cluster from one file. The system group carries the add-ons and the governor; the work group is tagged for the cluster autoscaler's auto-discovery and labelled `omni-role=work`:

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata: {name: <cluster>, region: <region>}
iam:
  withOIDC: true
  serviceAccounts:
    - metadata: {name: cluster-autoscaler, namespace: kube-system}
      wellKnownPolicies: {autoScaler: true}
managedNodeGroups:
  - name: system
    instanceType: m5.large
    minSize: 1
    maxSize: 1
    labels: {omni-role: system}
    taints: [{key: CriticalAddonsOnly, value: "true", effect: NoSchedu
    ↪ le}]
  - name: work0
    instanceType: m5.large
    minSize: 2
    maxSize: <pool top>
    labels: {omni-role: work}
    tags: {"k8s.io/cluster-autoscaler/enabled": "true", "k8s.io/cluste
    ↪ r-autoscaler/<cluster>": "owned"}
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
eksctl create cluster -f cluster.yaml
helm repo add autoscaler https://kubernetes.github.io/autoscaler && he
    ↪ lm repo update
helm upgrade --install cluster-autoscaler autoscaler/cluster-autoscale
    ↪ r -n kube-system \
  --set image.tag=v1.<minor>.0 --set autoDiscovery.clusterName=<cluste
    ↪ r> --set awsRegion=<region> \
  --set rbac.serviceAccount.create=false --set rbac.serviceAccount.nam
    ↪ e=cluster-autoscaler \
  --set extraArgs.scale-down-unneeded-time=10m --set extraArgs.scale-d
    ↪ own-delay-after-add=10m \
  --set "tolerations[0].key=CriticalAddonsOnly" --set "tolerations[0].
    ↪ operator=Exists" \
  --set "nodeSelector.omni-role=system" --wait
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/rel
    ↪ eases/download/v0.7.2/components.yaml
```

- **The autoscaler's version** must match the cluster's Kubernetes minor version (`kubectl version` prints it); the autoscaler publishes one release a minor.
- **The autoscaler's identity** is the one place a cloud credential enters the cluster, and it belongs to the autoscaler, never to the governor: eksctl creates an IAM role for the `cluster-autoscaler` service account with the autoscaling policy and nothing else.
- **The scale-down timing** (`scale-down-unneeded-time`, 10 minutes by default) is native's own, kept the same in both arms of any paired run, never changed by the governor; the receipts used 2 minutes in both arms.
- **metrics-server** goes on the system machine: patch its Deployment with the same toleration and node selector, or it will take a work machine's room.

**Karpenter.** Where the cluster runs Karpenter instead of the cluster autoscaler, the governor's machine lever works the same way: it parks a machine with a `PreferNoSchedule` taint and marks its pods first to go, and Karpenter's consolidation removes the machine once it is empty. Karpenter's NodePool must carry the label `omni-role=work` on the machines it makes, and its disruption budget and consolidation policy are native's own settings, kept the same in both arms.

## 18.3 The service's autoscaler

As in section 16.3: a CPU HorizontalPodAutoscaler and container requests, the target and range written down by the service owner. Check that metrics-server answers with `kubectl top nodes` before the governor is installed; without it the governor reads the service's processor as blind and holds (section 6.3).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 18.4 The registry and the governor's image

```
ACCT=$(aws sts get-caller-identity --query Account --output text); REG
    ↪ =$ACCT.dkr.ecr.<region>.amazonaws.com
aws ecr create-repository --repository-name omni-compass --region <reg
    ↪ ion>
aws ecr create-repository --repository-name omni-compass-cloud --regio
    ↪ n <region>

aws ecr get-login-password --region <region> | docker login --username
    ↪  AWS --password-stdin $REG
docker build -t $REG/omni-compass:<version> -f deploy/Dockerfile . &&
    ↪ docker push $REG/omni-compass:<version>
docker build -t $REG/omni-compass-cloud:<version> -f deploy/Dockerfile
    ↪ .cloud \

  --build-arg BASE=$REG/omni-compass:<version> . && docker push $REG/o
    ↪ mni-compass-cloud:<version>
```

eksctl's managed node groups carry `AmazonEC2ContainerRegistryReadOnly` on their instance role, so the nodes pull from ECR in the same account without further grants.

## 18.5 Watch first

```
kubectl apply -f deploy/install/omni-compass.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ REG/omni-compass:<version>
kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 5 /var/log
    ↪ /omni/audit.jsonl
```

The governor watches and writes nothing (section 16.5). For an audit that outlives the pod, the EBS CSI driver's `gp3` storage class provides a persistent volume once the driver add-on is installed (section 21.4).

## 18.6 The paired run on Amazon

The workflow `eks-metered` is the same design as Azure's and Google's, on clusters it builds with eksctl and deletes itself, Amazon's billed instances counted every 15 seconds at list price; Appendix H connects it to an account, click by click. It runs the whole engine at the max, as every cloud set does from 31 October (chapter 20).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 18.7 Tearing down

`eksctl delete cluster --name <cluster> --region <region> --wait` deletes the cluster, its node groups and the CloudFormation stacks; `aws ecr delete-repository --repository-name omni-compass --force` deletes the images. Every paired run deletes every cluster it made as its last step, whatever happened.

## 18.8 What goes wrong on Amazon, and what to do

| Symptom | Cause | Fix |
|---|---|---|
| `VcpuLimitExceeded` | the region's on-demand vCPU quota | request an increase |
| eksctl stops with a CloudFormation `ROLLBACK_COMPLETE` | a quota, a subnet or an IAM permission | read the stack's events in the CloudFormation console; delete the stack before retrying |
| The work group never grows or shrinks | the autoscaler's version does not match the cluster, or the group is not tagged for discovery | match the image tag to the cluster's minor; check the two tags |
| `kubectl top` fails | metrics-server is not installed or cannot reach the kubelets | install the pinned release; check its logs |
| A parked machine is never removed | Karpenter's or the autoscaler's disruption rules hold it (a PodDisruptionBudget, a do-not-disrupt annotation) | the same rule holds native; it is the operator's |

CHAPTER 19

# The Response-Time Feed and the Line

The governor holds a service at its line by reading the service's response time. The **feed** is how that reading reaches it; the **line** is the response time the service must meet, written down by the service's owner. Both are the implementer's to provide, on any cloud, and both are decided before Omni-Compass watches, because a supervisor watching a wrong feed against a wrong line produces an audit that teaches nothing.

## 19.1 The line

The line is a response-time objective in milliseconds, passed as `--slo-ms`. It is the service owner's, never the implementer's and never the governor's, and it is written down with its basis before the first counted run:

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **The percentile it binds.** The governor reads the line against the 95th percentile of recent responses (a breach is the 95th percentile at or over the line) and holds the mean of recent responses inside its band below the line. An objective written on another percentile is translated by the owner, not by the implementer.
- **What counts as a response.** In the receipt a failed request (a timeout, a connection refused, a 5xx where the owner says so) counts over the line, whatever its timing. The governor builds its own reading from the served responses of its window; a window in which no request was served reads as past the line, and every machine is held.
- **Where it is measured.** At the edge the customer sees (the load balancer or ingress), or behind it; the line and the feed must be measured at the same place.

A line set far above the service's real response times leaves the governor nothing to hold and reads STEADY, or a word not beyond the noise, on every row; a line set below what the service can meet on native reads as a breach in every decision and the governor holds every machine. Both are wiring faults, and Appendix D lists their symptoms.

## 19.2 The feed's format

The feed is a text file of comma-separated values that the governor reads with `--latency-file`. Its first line is the header and every later line is one response:

```
elapsed_seconds,latency_ms,ok
0.0,212.4,1
0.3,198.0,1
0.6,5003.1,0
```

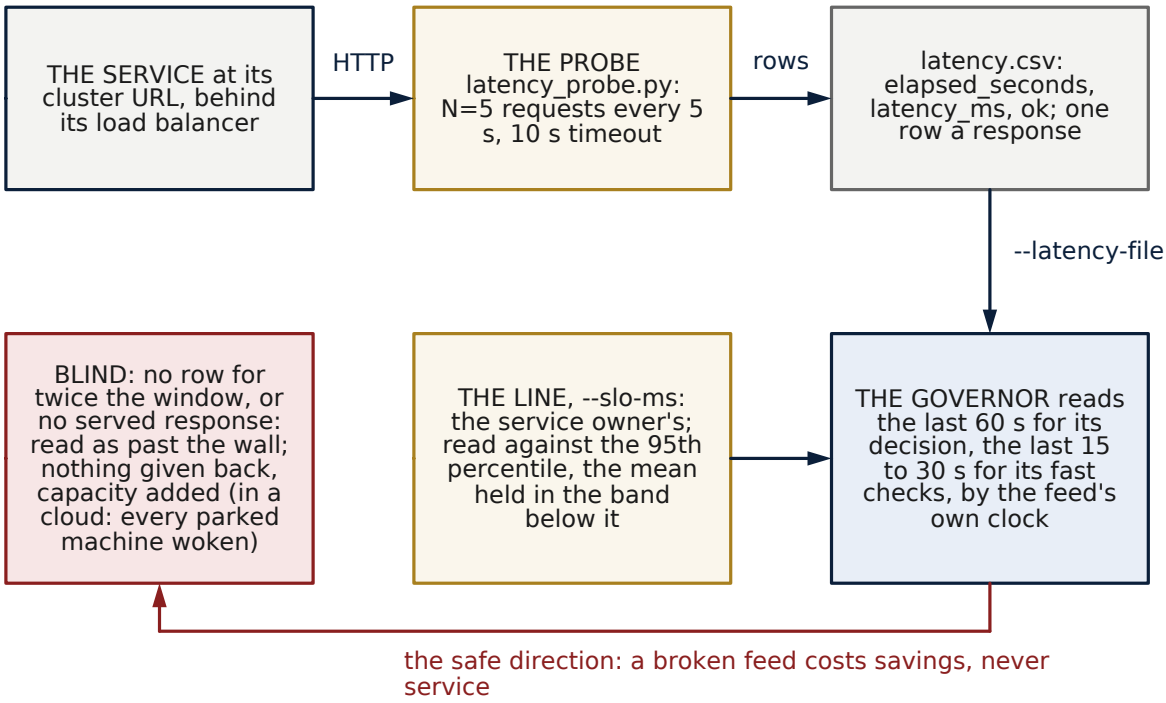
- `elapsed_seconds`: seconds since the feed began, increasing; the governor reads only rows newer than the last it read.
- `latency_ms`: the response's time, in milliseconds.
- `ok`: 1 for a response the owner counts as served, 0 for a failure.

The governor reads the window of recent rows (the last 60 seconds for its decision, the last 15 to 30 seconds for its fast reflexes) by the feed's own clock. **A feed whose file has not changed for twice the window, or whose window holds no served response, reads as blind**, and a blind sense makes the governor hold every machine in service and give nothing back until the feed returns (section 6.3). That is the safe direction, and it is why a broken feed costs savings, never service.

## 19.3 Feed one: the probe

The repository's probe (`scripts/latency_probe.py`) times real HTTP requests to one URL and writes exactly this file:

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.




Feed two: the service's own timings (an ingress, a mesh, the application's histogram) written in the same three columns, measured at the same place as the line, never pre-smoothed. One feed for one governor.

**Figure 43.** Wiring schematic: the response-time feed and the line. The probe times real requests to the service and writes one row a response to latency.csv (elapsed_seconds, latency_ms, ok); the governor reads the file by its own clock, the last 60 s for its decision and the last 15 to 30 s for its fast checks, against the line the service owner wrote (--slo-ms).

A feed that has not advanced for twice the window, or holds no served response, reads as blind, and a blind sense is read as past the wall: nothing is given back and capacity is added (in the cloud mode every parked machine is woken), the safe direction. A second feed, the service's own timings, may be written in the same three columns, measured at the same place as the line and never pre-smoothed.

```
INTERVAL=5 N=5 DURATION=31536000 python3 scripts/latency_probe.py http
    ↪ ://<service>.<namespace>.svc.cluster.local/ /feed/latency.csv
```

It sends `N` requests back to back, pauses `INTERVAL` seconds, and repeats for `DURATION` seconds; each request has a 10-second timeout, and a timeout or an error is written as `ok=0`. In a cloud it runs as a second container in the governor's own pod (section 20.4), writing to a volume both containers share. A probe measures a synthetic request; it is the right feed where the service has one representative endpoint, and the wrong one where the customer's requests differ widely in cost.

**A probe restart.** The probe's clock starts again at zero when its container restarts. The governor's reading follows the newest rows by their own clock and carries on, but a give-back trial counts only rows newer than

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the last it counted, so no trial completes, and no machine is given back, until the new clock passes the old one. That is the safe direction. Restart the governor's pod so both clocks start together (the governor hands every setting back on its way out; section 21.3), and alert on a restarted probe container (section 21.5).

## 19.4 Feed two: the service's own timings

Where the service already measures its responses (an ingress controller, a service mesh, the application's own histogram), an adapter can write those timings in the same three columns. The adapter is a small program the implementer owns: it appends one row per response, or one row per short interval carrying that interval's 95th percentile with `ok=0` rows for the interval's failures. Two rules keep it honest:

1. **The same place as the line.** Timings from the ingress for a line written at the ingress; timings from the application for a line written at the application.
2. **No smoothing the governor does not know about.** A row is a response or an interval's own statistic, never a moving average of earlier rows; the governor builds its own windows, and a pre-smoothed feed makes it late.

## 19.5 Checking the feed before the governor watches

| Check | How | Pass |
|---|---|---|
| The file advances | `tail -f /feed/latency.csv` for a minute | a new row at least every 5 seconds |
| The clock is monotonic | the first column never decreases | yes, across a full day |
| The failures are counted | stop the service's endpoint for 30 seconds in a test window | rows with `ok=0` appear, and the governor's audit reads the latency sense as blind or breached: either way past the line, every machine held |
| The line is plausible | the feed's 95th percentile on a normal day against `--slo-ms` | the 95th percentile sits below the line on a normal day, and reaches it only under real pressure |
| Blind is safe | stop the probe for two minutes in a test window | the audit shows `blind` and no machine given back until the feed returns |

## 19.6 One feed for one governor

A governor reads one feed (`--latency-file`), and a cluster has one governor on its machines (the one-writer rule, section 6.2). Where several services share the governed machines, the feed must speak for all of them: either the feed of the service most sensitive to a lost machine, or an adapter (section 19.4) that writes,

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

for each short interval, the worst of the services' own 95th percentiles against their own lines, scaled to one line. `--sensed` names the services whose autoscalers the feed speaks for, and only those are moved, in a cloud as on site; the feed is also the guard on the machines the governor parks: a machine is parked only while the line holds, and a breach wakes it at the next decision.

CHAPTER 20

# Running in the Cloud: the Whole Engine, the Bill and the Alarm Minutes

## 20.1 What the governor moves in a cloud, and how the bill is read

In a public cloud every machine that exists is billed, whether it carries work or not, so the bill is a gauge of its own there: a service gauge, its aim lower, read by the one key beside the work, the response time, the failures and the alert minutes. The governor has four levers in a cloud: the pod autoscaler's target, the processor of running pods, their power model and the machines. Until 31 October it pulled the machines alone, by rule (the founder's order of 14 October: a lower pod target asks for more pods, more pods ask the cloud for machines, and the bill was kept from rising by moving nothing else). The founder's order of 31 October struck that rule: no hard rule; the whole engine runs at the max in a cloud as on site, and what the bill does is judged, never ruled. So in a cloud the governor moves every muscle, and the machines so:

- **The pod autoscaler's target** is moved inside the autoscaler's own cover, as on site (chapter 7): the compass moves the target, native's autoscaler keeps its range and cap, and a target that would ask for pods the machines cannot place is handed back to native's own (omni-v21's immune rule). The replica room is read live, as on kind.
- **The processor** is conveyed: a serving pod gets the processor its machine can spare and gives it back where a neighbour wants it (chapter 8); **the power model** is read, and a cap written only where a meter is wired.
- **The machines.** The governor parks a machine the cloud's autoscaler already made, only after its own trial shows the service's response time holds without it (within 2% of the response time with it, through the release gate, one machine at a time), and wakes it on a breach: at the next full decision (within 60 seconds) for a response time over the line, and at the next fast check (within 5 seconds) for a pod that cannot be placed. A pod never waits for either: a parked machine's taint is `PreferNoSchedule`, so a pod that finds the open machines full lands on a parked one at once. A parked machine whose pods have gone is deleted by the cloud's own autoscaler, on the cloud's own schedule.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

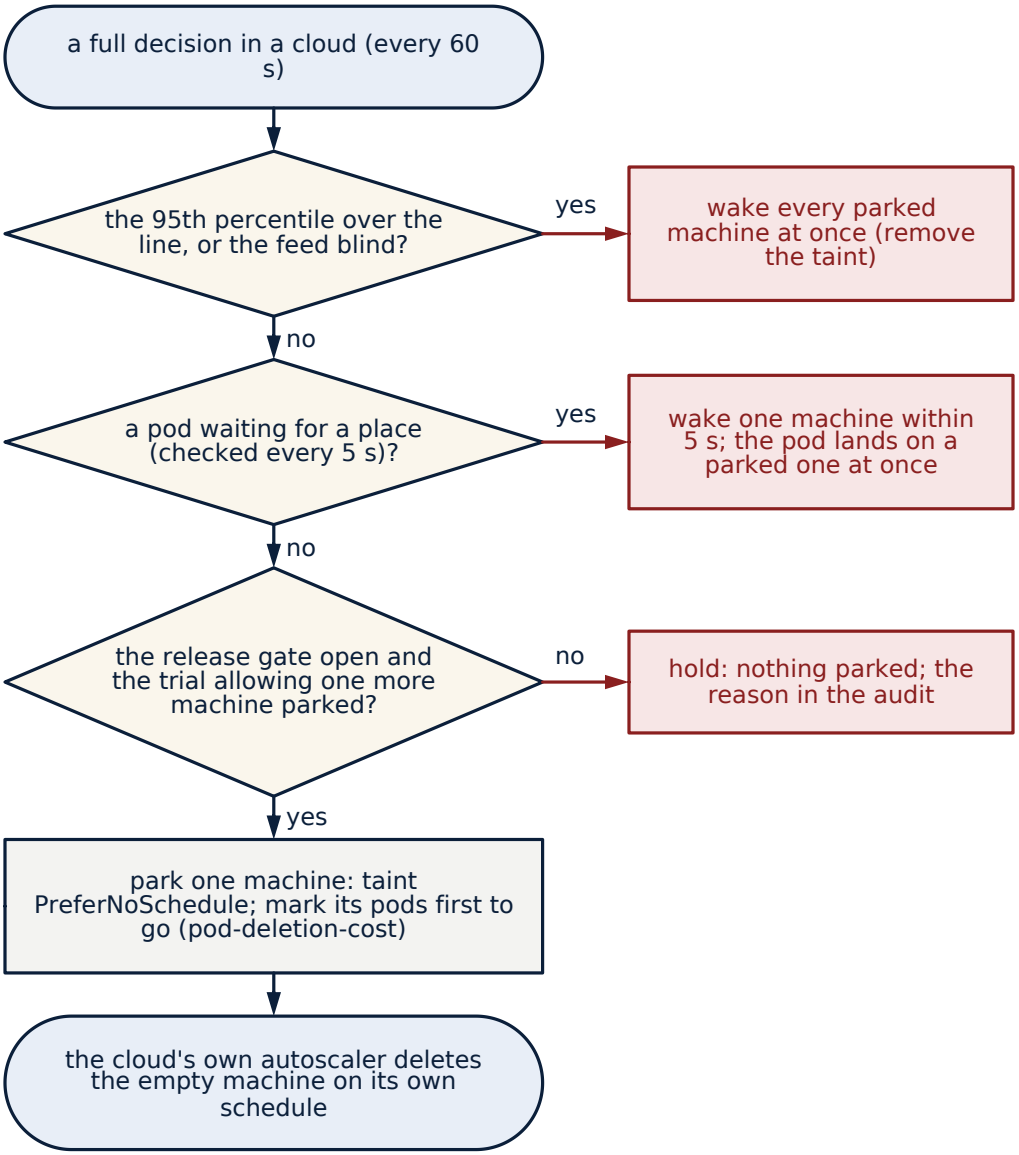



**Figure 44.** Flowchart: parking and waking a machine in the cloud. At every full decision: a 95th percentile over the line, or a blind feed, wakes every parked machine at once; a pod waiting for a place, checked every 5 s, wakes one machine within 5 s while the pod lands on a parked one at once; otherwise, only with the release gate open and the trial allowing it, one machine is parked (a PreferNoSchedule taint, its pods marked first to go), and the cloud's own autoscaler deletes it once empty, on its own schedule.

The governor asks the cloud for no machine itself: a pool the cloud's autoscaler owns refuses a count set by hand, so the autoscaler, native's machine controller, stays in every arm and alone adds machines, for the pods the governor's target asks for (its machine lever, `scripts/kind_nodepool.sh`, never asks for more machines than the cluster has). **The bill is judged, never ruled.** It falls where native holds machines the service does not need: the quiet stretches between bursts, the hours of a day's trough, the weekend. It may rise where the target asks for pods ahead of the load and the autoscaler adds a machine for them; a rise reads power

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

given beside the work and the speed it bought, and the cloud's judged number is the work per dollar: a tune whose work per dollar falls against native is not admissible as the factory tune (`docs/THE_TUNE.md`): it is a fault of the tune, root-caused in the model first. Where native is already short of machines the governor has nothing to give back, and the receipt reads STEADY on the bill.

**What a cloud receipt can show, and what it cannot.** The cloud is a wire (Part Six's opening): it makes no benefit of its own. A cloud receipt shows two things: that the engine holds on the cloud's machines as it does on site (every gauge read against native by the one key, no LOSS), and how much of the software end's gain comes through the wire, read as work per dollar beside the bill. The bare metal's energy is the provider's and is not on its receipt; it is measured where Omni-Compass sits on the chip. Where Omni-Compass sits on neither end, the cloud's bill shows nothing of Omni-Compass. The arm by itself (section 1.1) is never run in a cloud without the founder's word: by itself Omni-Compass would have to make machines with the cloud's autoscaler off.

## 20.2 The dial in the cloud

The cloud runs at the max (`--dial max`), the factory tune, as on site: the most healthy work from what each muscle can carry, the bill shown beside it (the founder's order of 31 October). The tune-down (`--dial save`, native's work on the least resource) is a tune any company may set where it wants a cut rather than more work; it is a changed tune, and its receipts are its own. Until 31 October every cloud set ran at the tune-down on the machines alone (the order of 14 October), and those sets stand as their own record, read in no table with a set at the max.

## 20.3 The settings a cloud runs

| Setting | Value in a cloud | Why |
|---|---|---|
| `--mode` | `nodepool` | the machine lever |
| `--law` | `compass` | the compass holds the response time in its band and gives machines back through the trial |
| `--dial` | `max` | the factory tune, as on site |
| `--sensed` | `<namespace>/<deployment>` | the service whose autoscaler's target is moved inside its cover; its range and cap stay the operator's |
| `--cap-deployments` | `<namespace>/<deployment>` | the pods whose processor is conveyed in place, and handed back where a neighbour wants it |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Value in a cloud | Why |
|---|---|---|
| `--replica-ceiling` | absent: read live | the replica room: how far the cap may rise while it binds and the line is threatened (what the machines can hold, one pod under), handed back once the demand holds still |
| `--power-cmd` | absent | a rented machine has no meter the tenant can read; no power is capped |
| `--latency-file`, `--slo-ms` | the feed and the line (chapter 19) | the guard on every machine parked |
| `--node-scale-cmd` | `bash /app/scripts/kind_nodepool.sh {n}` | parks and wakes machines that exist; never asks for one |
| `--node-restore-cmd` | `bash /app/scripts/kind_nodepool.sh 100000` | on a reset, wakes every parked machine (the script caps the number at the machines that exist) |
| `--active-nodes-only` | on | counts only machines in service |
| `--min-nodes` and `MIN_NODES` | 2, or the operator's floor | never fewer machines in service |
| `--max-node-step` | 1 | one machine a decision |
| `--max-nodes` | the work pools' top (the sum of their autoscaler maxima) | the most machines the lever counts on; it never asks for more than exist |
| `--interval`, `--floor-interval` | unset (the factory beat for its place) and 5 | the full decision every 10 seconds on site and every 60 seconds through a cloud, read from the cluster's own nodes at the start (a dial from 5 to 120 seconds, section 4.4), the fast checks every 5 seconds |
| `--closure` | `/app/law/closure.json` | shipped in the image |
| `--audit`, `--kill-file` | `/var/log/omni/audit.jsonl`, `/tmp/omni.kill` | the record and the switch |
| `WORKER_SEL` (environment) | `omni-role=work` | which machines may be parked |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 20.4 The cloud image and the governing Deployment

The cloud image adds five things to the shipped one: `jq` (the machine lever reads the cluster's JSON with it), the machine lever's script, the probe, the Prometheus feed adapter (section 23.8), and the switch tool that the in-pod watchdog runs (section 21.2). It is `deploy/Dockerfile.cloud`:

```
ARG BASE=omni-compass:local
FROM ${BASE}
USER root
RUN apt-get update && apt-get install -y --no-install-recommends jq &&
    ↪  rm -rf /var/lib/apt/lists/*
COPY scripts/kind_nodepool.sh /app/scripts/kind_nodepool.sh

COPY scripts/latency_probe.py /app/scripts/latency_probe.py
COPY tools/omni_switch.py /app/tools/omni_switch.py
COPY tools/prom_feed.py /app/tools/prom_feed.py
USER 10001
```

Build the shipped image first and pass it as the base (`--build-arg BASE=<registry>/omni-compass:<version>`). The governing Deployment is `deploy/install/omni-compass-cloud.yaml` (printed in Appendix I). Apply the cloud's grant of section 15.3 (`deploy/install/rbac-cloud.yaml`) first, set the image, the service, the service's URL and the line, and replace the watch Deployment:

```
kubectl apply -f deploy/install/rbac-cloud.yaml
kubectl apply -f deploy/install/omni-compass-cloud.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=<
    ↪ registry>/omni-compass-cloud:<version> \
  probe=<registry>/omni-compass-cloud:<version>
kubectl -n omni-compass set env deployment/omni-compass -c probe TARGE

    ↪ T_URL=http://<service>.<namespace>.svc.cluster.local/
kubectl -n omni-compass rollout status deployment/omni-compass
```

**Dry run first.** The manifest ships with `--dry-run` on the governor's command line: it decides as it will, writes nothing, and records every command it would have run as a `write` record marked `"dry_run": true`. Read a day of its audit (section 23.7 traces one decision) before removing `--dry-run`. A dry run checks the commands, not their effect: because no machine is parked, a trial during a dry run compares the service with itself, and its `step allowed` is no evidence that the service holds without the machine. That evidence is the paired run's (phase 5). The first write after `--dry-run` is removed is the first machine the governor parks, in its audit with the trial that allowed it.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

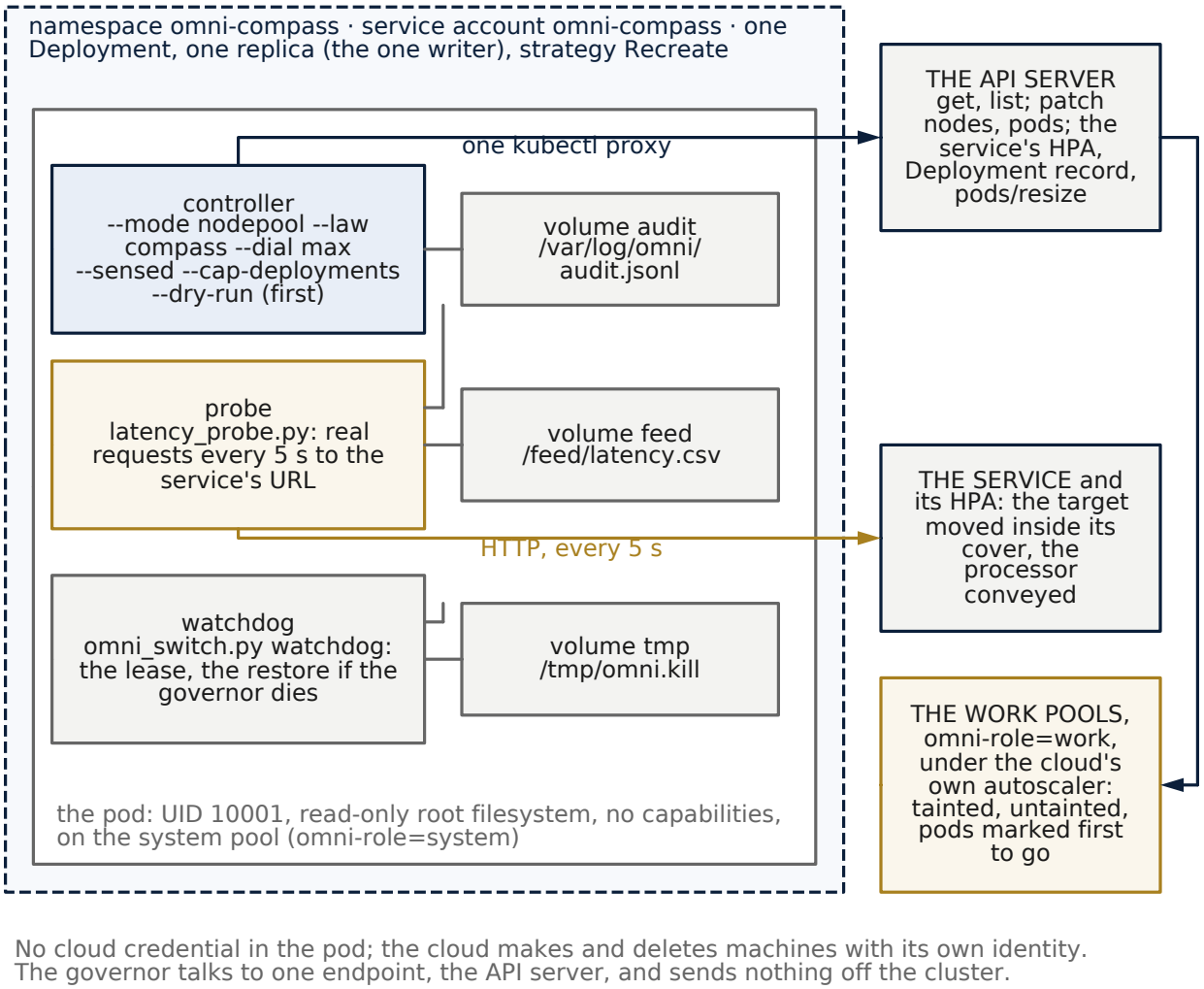



**Figure 45.** Deployment diagram: the governor on a managed cluster. One namespace, one service account, one Deployment of one replica (the one writer): the controller container with its command line, the probe beside it writing the feed to a shared volume, and the watchdog holding the lease; the audit, the feed and the kill file each on their own volume.

Outside the pod: the API server, reached through one kubectl proxy and granted get, list, patch nodes and patch pods, and in the service's namespace by name its autoscaler, its Deployment's records and its pods' processor, nothing else; the service and its autoscaler, whose target the governor moves inside its cover; and the work pools under the cloud's own autoscaler, where machines are tainted, untainted and their pods marked first to go. No cloud credential is in the pod.

## 20.5 What to expect, hour by hour

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Hour of a typical day | Native | With Omni-Compass on top, the whole engine |
|---|---|---|
| The peak | every machine the pods need | the same machines; nothing parked while the line is at risk |
| The descent after a peak | machines held until the autoscaler's own scale-down finds them unneeded | machines parked one at a time as each trial passes; their pods leave first on the autoscaler's next scale-down |
| The trough (night, weekend) | the pool held at what the pods' requests need | parked down to the floor where the trial allows; each empty parked machine deleted by the cloud |
| A new burst | the autoscaler adds machines when pods cannot be placed | pods that find the open machines full land on a parked machine at once (its taint only prefers against it); the governor wakes it within 5 seconds if pods wait, or at the next decision on a breach; the cloud adds machines past those, as for native |

The work per dollar comes from three places: the processor conveyed to the running pods, which answer faster on the machines already billed; the pod target moved ahead of the load, so the service is ready before the line; and the machines the governor parks before native's autoscaler would find them unneeded, whose empty machines the cloud deletes sooner because their pods were marked first to go. None of them delays a pod: a parked machine carries a `PreferNoSchedule` taint, so a pod that finds the open machines full lands on a parked one at once. Where the target asks for pods ahead of the load, the cloud's autoscaler may add a machine for them: the bill it adds is read beside the work it bought.

## 20.6 Reading the bill and the alarm minutes

Every paired receipt on a cloud carries these rows, and they are read by the rules of chapter 12:

| Row | Its aim | How to read it |
|---|---|---|
| completed work per dollar of the bill | higher | the cloud's one number: the work answered within the line for each dollar of the bill; a rise is GAIN (more work for the same money), a fall is power given (money spent for speed, never a LOSS); read only where the load lets the work grow and the answers are counted |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

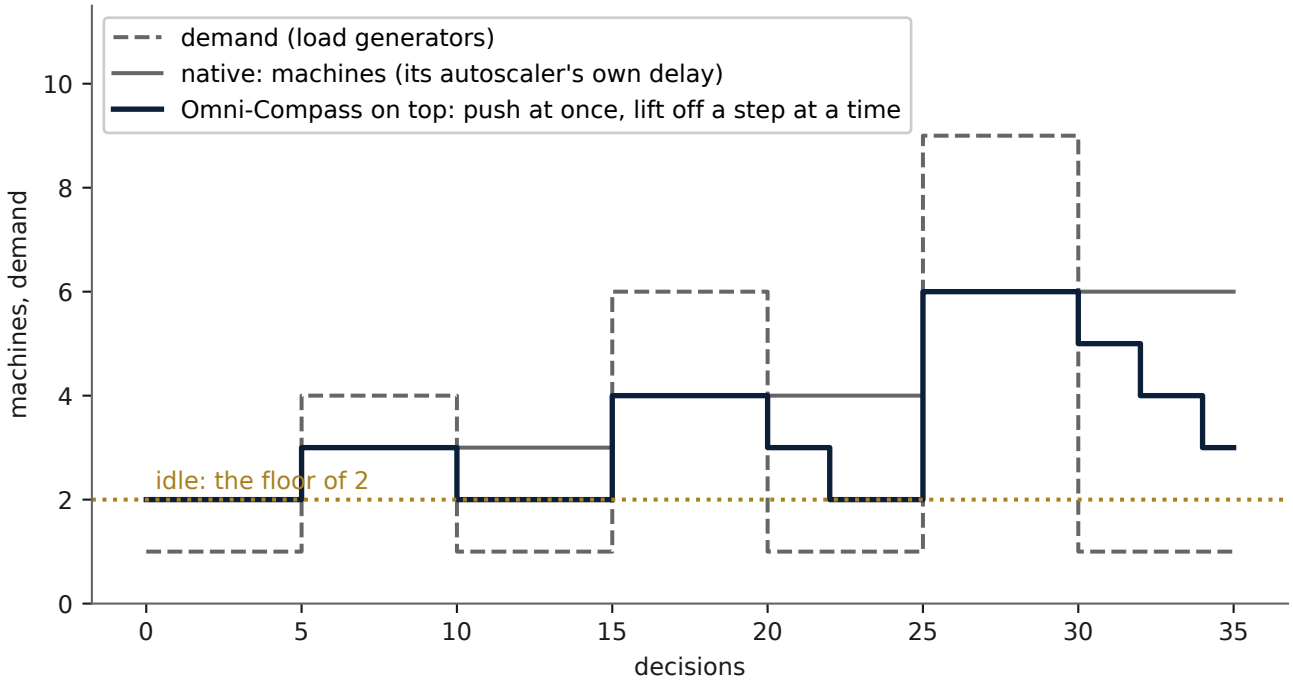



**Figure 46.** Modelled trace: one pedal over a burst, machines against demand (class S: the rules of the pedal drawn over the burst 1 4 1 6 1 9, five decisions a step). Native's autoscaler adds a machine when pods cannot be placed and removes one after its own delay; with Omni-Compass on top the push adds capacity at once at the wall and the lift off gives it back one machine at a time through the gate and the trial, never below the floor of 2.

| Row | Its aim | How to read it |
|---|---|---|
| compute bill at list price ($) | lower | shown beside the work it bought; a cut is GAIN, a rise is power given; a smaller bill is never the aim, and a tune whose bill rises against native for the same work is not admissible as the factory tune (`docs/THE_TUNE.md`) |
| machines billed, machine-hours | lower | as the bill |
| response time, 95th and 99th percentile | lower | read by the one key; a LOSS is a wiring fault to find first (section 5.5) |
| failed requests | lower | must read STEADY or GAIN |
| alert minutes | lower | the whole minutes whose 95th percentile, failed requests counted over, was over the line: the minutes an on-call page would be open. The aim in a cloud is none at all; an alert minute on the omni side is found and fixed |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Row | Its aim | How to read it |
|---|---|---|
| settings commands the governor wrote on its own | shown | the machines it parked and woke, each in its audit with its reason; no person touched either arm |
| HPA replicas | shown | the pods each arm ran; with the target moved they may differ from native's, and the machines they asked for are on the bill row |

**A run of one day of real traffic says more than any burst.** The machines given back live in the quiet hours; a paired run of a steady load at the service's peak can read STEADY on the bill, honestly, because at the peak there is nothing to give back. The day-shaped load of the repository's own cloud sets (the Google 2011 trace on one to nine load generators) is the test that shows the trough.

## 20.7 What the governor never does in a cloud

It never moves a pod autoscaler's floor or cap but within its replica room, handed back once the demand holds still (it moves the target inside them); never caps power where no meter is wired; never asks the cloud for a machine; never deletes a machine (the cloud's autoscaler does, on its own schedule, once a machine is empty); never evicts or deletes a pod; and never holds a cloud credential. Each of those is checked from outside the governor at the end of every paired arm: every HPA back at the operator's, every pod's limit back at its shipped value, every machine back in service, no `omnicompass.io/` record left on any object.

CHAPTER 21

# Day Two: Running Omni-Compass in Production

An installation is in production when the governor writes, the service depends on the result, and nobody watches it by hand. This chapter is what keeps that state safe: the watchdog and the lease, the OFF switch drill, the audit as a record, the alarms, upgrades and rollbacks, a fleet of clusters, and the runbooks for the incidents an implementer should expect.

## 21.1 The governor's lifecycle on a cluster

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Event | What the governor does | What the implementer checks |
|---|---|---|
| Start | records every setting it may move, on the object itself (an `omnicompass.io/` annotation), before its first write; registers its lease | the first audit lines: the snapshot |
| Every decision | renews its lease, reads, decides, writes (or holds), reads back every write | the decision trail (Chapter 9) |
| Reset (planned stop) | hands every setting back in reverse order of power-up, reads each back, writes its last audit line, exits 0 | no `omnicompass.io/` annotation left; every machine in service |
| OFF switch (security) | the same hand-back at once, from a signal; no governor starts while the switch is off | `python3 tools/omni_switch.py status` |
| Death without hand-back (killed, crashed, hung) | nothing; the watchdog acts for it | the watchdog's log: `handed back` with `ok` |

## 21.2 The watchdog and the lease, in a cluster

A governor that dies without handing back leaves its settings in place: in a cloud, machines parked, a moved target and conveyed limits. The lease covers that case. Before its first write the governor registers the exact command that puts every setting back (`python -m omni_controller.controller --restore-only`, which restores from the annotations on the objects themselves, so it needs no memory of the dead process) and renews a lease every decision. The watchdog (`python3 tools/omni_switch.py watchdog`) finds a governor whose process is gone or whose lease is older than its limit (five decision periods, at least two minutes), stops a hung one, runs its recorded restore and records what it did.

In a cluster, the watchdog runs as a second container in the governor's own pod, with the pod's process namespace shared (`shareProcessNamespace: true`), as the cloud manifest ships it (Appendix I). It must: the watchdog judges a governor dead by its process, and a process is visible only inside its own process namespace, so a watchdog in a separate pod would find every governor dead and hand back over a governor that is running. In the pod, the watchdog sees the governor's process and its lease (on the pod's shared `/tmp`); when the governor's container dies or hangs, the watchdog stops a hung one and runs its recorded restore, which wakes every parked machine, and the kubelet restarts the governor's container, which begins again from the cluster as it finds it. When the whole pod is lost (its machine fails, or it is evicted), the Deployment starts a new pod; until it does, the parked machines stay parked, which never blocks a pod (a `PreferNoSchedule`

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

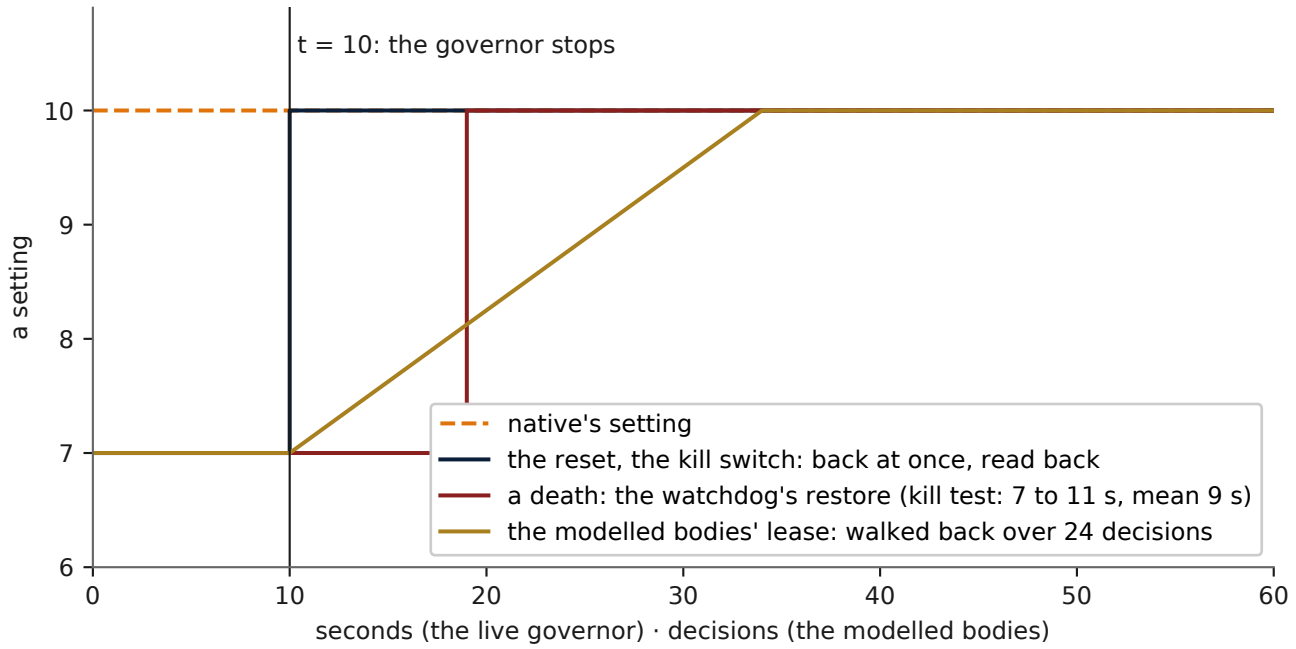



**Figure 47.** Timing diagram: the hand-back after a reset, a kill and a death. At t = 10 the governor stops. A reset or the kill switch writes every setting back at once and reads it back; a death without a hand-back is caught by the watchdog, which restored within 7 to 11 s (mean 9 s) in the kill test; in the modelled bodies the lease walks the setting back to native over 24 decisions.

taint), and the cloud's autoscaler adds machines for any pod that cannot be placed. No failure of the governor or the watchdog can leave a pod unscheduled.

## 21.3 The OFF switch drill

In a cluster the OFF switch has three forms, and the person who holds it (section 14.4) knows all three:

| Form | Command | What happens | How long it holds |
|---|---|---|---|
| Scale to zero (the OFF switch of record) | `kubectl -n omni-compass scale deployment/omni-compass --replicas=0` | the pod receives `SIGTERM`; the governor hands every setting back at once (every parked machine woken, its marks on pods removed), reads each back, writes its last audit line and exits | until the holder scales it back to 1 |
| The kill file | `kubectl -n omni-compass exec deploy/omni-compas` | at its next full decision (within 60 seconds) the governor hands every | until the file is removed; the file lives in the pod's own `/tmp`, so a replaced |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Form | Command | What happens | How long it holds |
|---|---|---|---|
| | `s -c controller -- touch /tmp/omni.kill` | setting back, then writes nothing; each decision's audit reads `killed` | pod starts ON |
| Revoke the grant | `kubectl delete clusterrolebinding omni-compass-machines` | every write the governor attempts is refused by the API server and recorded | until the binding is applied again |

Revoke the grant only after one of the first two forms has handed everything back: a governor without its grant cannot wake a machine it parked. (A parked machine never blocks a pod, so even then nothing waits; but its taint stays until the grant returns or a person removes it.)

The OFF switch is drilled before production and every quarter after, by its holder, on the production cluster, in a quiet hour:

1. Note the time and the machines parked: `kubectl get nodes -l omni-role=work -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints[*].key`.
2. Scale to zero.
3. Time it: no `omnicompass.io/idle` taint left on any machine, no `controller.kubernetes.io/pod-deletion-cost` mark left on any pod of the governed pools (unless another controller of the company's own sets that mark), the governor's pod gone. The limit is 60 seconds.
4. Scale back to 1 (`--replicas=1`) and record the drill: the date, the holder, the seconds, anything left behind.

A drill that leaves anything behind, or takes more than 60 seconds, stops the installation until the cause is found.

## 21.4 The audit as a record

The audit (`audit.jsonl`, one JSON record a line, each timestamped) is the governor's complete account of itself: every reading, decision, write, read-back and reason. In production it is kept like any other operational record:

- **Storage.** Mount a persistent volume at `/var/log/omni` (Azure `managed-csi`, Google `standard-rwo`, Amazon `gp3` through the EBS CSI driver). An `emptyDir` loses the audit with the pod.
- **Shipping.** A log collector on the node reads the file and ships it to the company's log store; every line is already JSON. Keep the audit at least as long as the change record the company keeps for any production change.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Size.** One decision a minute writes a few records; a month is tens of megabytes. Rotate by size or day.
- **What to search for.** `"write"` (every change), `"refused"` (a write the cluster refused), `"error"` (a failed read or decision), `"proprioception"` in the `authority` record written with each decision (the drift between what the governor last commanded and what the cluster shows: a drift that does not close means a command did not land or another writer moved the setting), `"verdict"` (each give-back trial and its outcome), `"blind"` (a sense lost).

## 21.5 Alarms an implementer wires

| Alarm | Source | Why |
|---|---|---|
| Alert minutes on the service | the company's own monitoring of the line | the line is the service's; an alert minute is a person woken, whether or not Omni-Compass is installed |
| The governor's pod not running, or restarting | the cluster's own pod status | the watchdog hands back; someone must know why |
| The probe container restarted | the cluster's own pod status | no machine is given back until the governor's pod restarts (section 19.3) |
| `"error"` records in the audit, more than a few in an hour | the log store | the API is unreachable or a grant is missing; the governor holds |
| A `proprioception` drift that does not close over several decisions | the log store | a command did not land, or another controller or a person moves the same setting; find it (the one-writer rule) |
| A machine parked longer than a day with pods on it | `kubectl get nodes` and the taints | expected only in a long quiet stretch; worth a look otherwise |

## 21.6 Upgrades and rollbacks

An upgrade is a new image of the governor at a new engine version (chapter 13):

1. Verify the new copy (chapter 33) and read what changed in its version notes.
2. Run the new version in watch mode beside nothing: replace the Deployment with the new image and `--mode observe` for at least a day; read its decisions against the old version's.
3. Run a paired check on the new version where the receipts matter (chapter 11): receipts are never read across versions.
4. Switch it to govern. The old image stays in the registry, tagged, for the rollback.

A rollback is the same in reverse, and safe at any moment: stop the governor (its reset hands every setting back, or the watchdog does), set the old image, start it. The governor carries no state between processes except

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the annotations it reads back at start, so an old version starting after a new one begins from the cluster as it finds it.

## 21.7 A fleet of clusters

One Omni-Compass per cluster, never one for several clusters: each cluster has its own autoscaler, its own machines and its own line. A fleet is rolled out cluster by cluster, each through the phases of section 14.3, with a paired receipt on at least one cluster of each kind (each provider, each machine family, each service class) before the rest of that kind go to govern. Omni-Compass's settings are the same text in every cluster of a kind; keep them in the company's configuration repository and deploy them like any other workload, and keep one audit store for the fleet.

## 21.8 Runbooks

**A breach while machines are parked.** Expected and handled: a breach wakes a parked machine at the next full decision (within 60 seconds), and a pod that cannot be placed wakes one at the next fast check (within 5 seconds). If alert minutes persist with machines parked, turn the governor OFF (section 21.3), which wakes every machine, and read the audit for the decision that parked the last one: its trial, its readings and the feed's age.

**The governor parks nothing for days.** In a cloud at a steady peak that is the right answer: there is nothing to give back. Otherwise read `"verdict"` records: trials abandoned for too little work mean the feed is too thin (section 18.3); trials refused mean the service is slower without the machine, and the governor is right to keep it.

**The cloud's bill did not fall.** Read the receipt's machine-hours beside the autoscaler's own profile: a long `scale-down-unneeded-time` keeps a parked, empty machine billing; a PodDisruptionBudget or a pod with local storage can keep the autoscaler from removing one. Both hold native too, and both are the operator's settings.

**Someone else moves a setting.** The `proprioception` drift in each decision's `authority` record does not close: the machines in service are not the machines the governor commanded. Find the other writer (a GitOps controller resetting node taints, a second autoscaler or descheduler, a person running `kubectl uncordon`); there must be one writer for each setting, and until there is, turn the governor OFF.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 22

# The Graphics Processor: Putting the Governor on a Card

A graphics processor (a GPU card) is the other end of the cloud from the bill: the card is paid for by the hour or owned outright, and the benefit there is **more work inside the card at native's own heat, never hotter**. This chapter is written for the implementer who will put the governor on one card or on every card of a server, rented or owned. It is complete and ready to run; it is not run until the processor side is finished, and the first run is given one exact commit.

## 22.1 What the governor moves on a card

Two wires, both the card's own settings, both written through the vendor's own management interface (`nvidia-smi`, the NVIDIA System Management Interface) and read back after every write:

| Wire | What it is | Its range |
|---|---|---|
| The clock ceiling (up wire) | the highest graphics clock the card may run (locked clocks) | the card's own supported clocks, read from the card |
| The power limit (down wire) | the card's own power cap, in watts | the card's own minimum and default limits, read from the card |

It moves nothing else: not the application, not the model, not the batch, not the card's fan curve, not its memory clock, not its persistence mode, not another process's settings. Native is the card as it ships: the vendor's own firmware and driver deciding its clock under the default power limit.

## 22.2 What the card needs

- **Bare metal or full passthrough.** A virtual machine that hides clock locking or the power limit from the guest cannot be governed; the wire check (section 22.4) fails on it and says which wire.
- **The vendor's driver**, recent enough to support locked clocks and power limits on the card; `nvidia-smi -q -d CLOCK,POWER` lists the supported clocks and the power-limit range.
- **Root**, for Omni-Compass's writes on the card; it is the only process allowed to set the card's clocks and limit while it runs, and the machine check refuses to start if another management agent holds them.
- **A response-time feed** for the work on the card (chapter 19): the serving engine's own per-request timings, or the probe against the serving endpoint.
- **The heat sensor** the card reports (`nvidia-smi --query-gpu=temperature.gpu,clocks_throttle_reasons.active`), read by the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

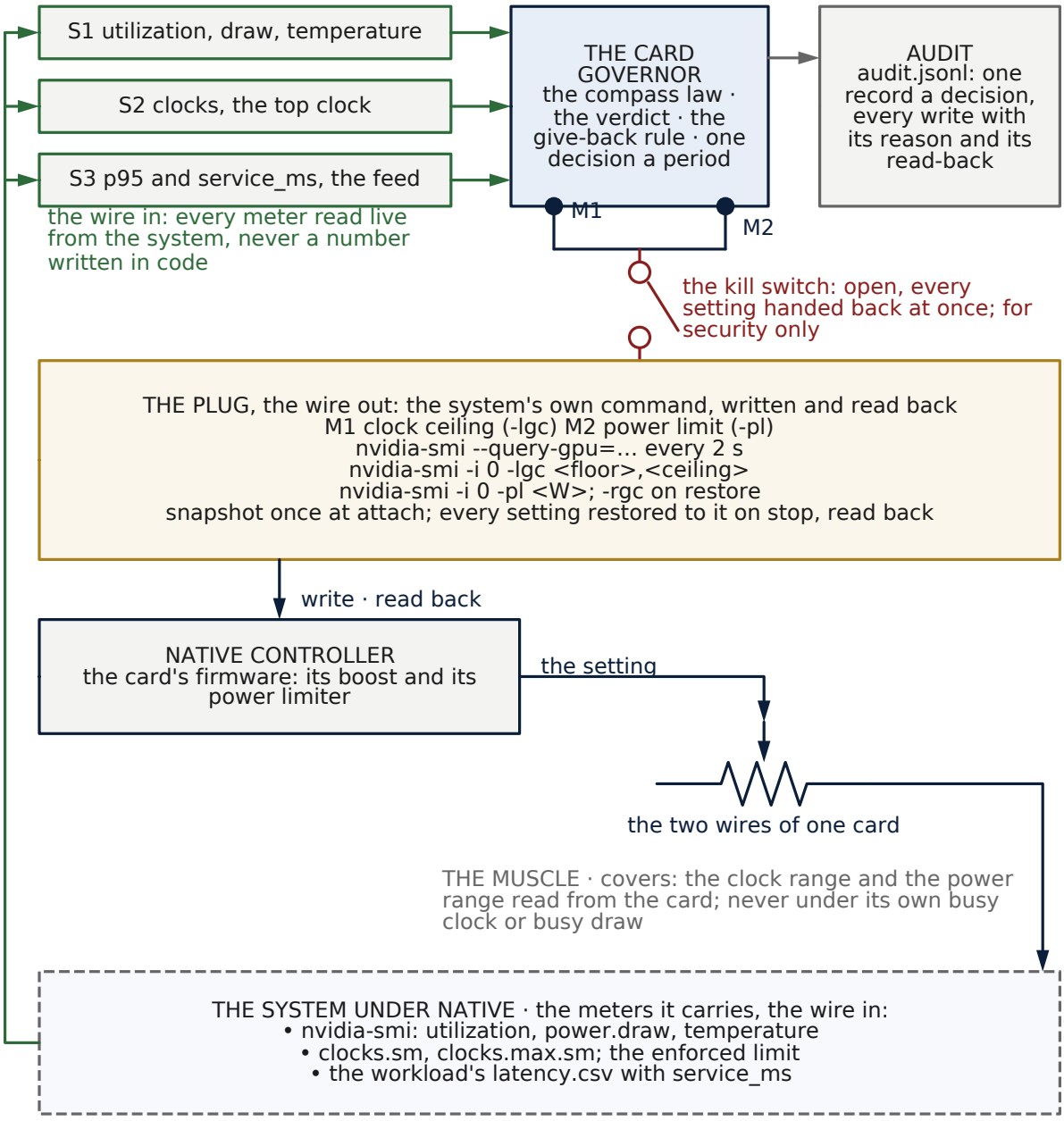



**Figure 48.** Wiring schematic: a graphics card, two wires. The schematic is read top to bottom as an electrical drawing: the governor (the control unit) with its sense terminals S1, S2, … on the left and its motor terminals M1, M2, … on its lower edge; the motor bus through the kill switch into the plug, the system's own command written and read back; the plug into the native controller; the native controller setting the muscle, drawn as a potentiometer with its cover; the system under native carrying the meters; and the meters back along one bus, the wire in, to the sense terminals.

The audit stands beside the governor. The senses are the card's utilization, draw and temperature, its clocks and top clock, and the workload's feed; the two motors are the clock ceiling (-lgc) and the power limit (-pl), both through nvidia-smi and both read back. Learns the card on its own first (15 busy readings); races while

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

work waits (≥ 95% utilization); at the max never below the top while the card serves; at the tune-down paces the slack in 15 MHz steps, each step through the paired trial.

> governor every decision; a thermal slowdown reported by the card holds every tightening (Chapter 9, `thermal_hold`).

## 22.3 Rented or owned

| | A rented card (for example a Lambda Cloud instance) | An owned card |
|---|---|---|
| Who pays for idle | the renter, by the hour | the owner, in power and heat |
| The gain | more work in the hours rented: fewer card-hours for the same work | more work from the same card at native's heat, and the same work at less power where the card is not the bottleneck |
| The meter | the card's own power reading (`power.draw`), class P | the card's own reading and, where fitted, a wall meter on the server (`tools/wall_meter.py`), class P |
| Access | root on the instance; full passthrough is standard on bare-metal rentals | root on the host |

## 22.4 The order on a card, step by step

1. **Verify the copy** (chapter 33) and record the commit; a card run is given one exact commit and run on it alone.
2. **The machine check.** One copy of the harness only, nothing else on the card, the card's default limit and full clock range restored. The harness's own scripts do this and refuse to start otherwise.
3. **Declare the envelope** once, before any trial: `python3 tools/declare_envelope.py envelope.json --gpu 0` writes the power range the governor may set, read from the card itself: from the larger of the card's own minimum and 70% of the limit read at declaration, up to that limit. The file is kept with the run.
4. **The wire check:** `sudo python3 tools/gpu_wire_check.py --gpu 0` must end `WIRED RIGHT`. It moves each wire and reads the card back, proving the up wire raises and lowers the clock and the down wire moves the power. Its failures are listed in Appendix D with their fixes.
5. **Watch:** `sudo python3 -m omni_controller.gpu_compass --mode watch --gpus 0 --latency-file <feed> --slo-ms <line>` computes both wires every decision and writes neither.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

6. **The paired run** on the card: native against native with Omni-Compass on top, the same work, repeated, the card's own power reading and the response times counted in both arms (the harness's `scripts/gpu_paired.sh` on an owned card, `scripts/gpu_rented_run.sh` for the whole rented sequence in one command, `scripts/gpu_8card.sh` for every card of an eight-card server at once).
7. **Govern:** `--mode cap` in place of `--mode watch`, with the OFF switch drilled (section 22.6).

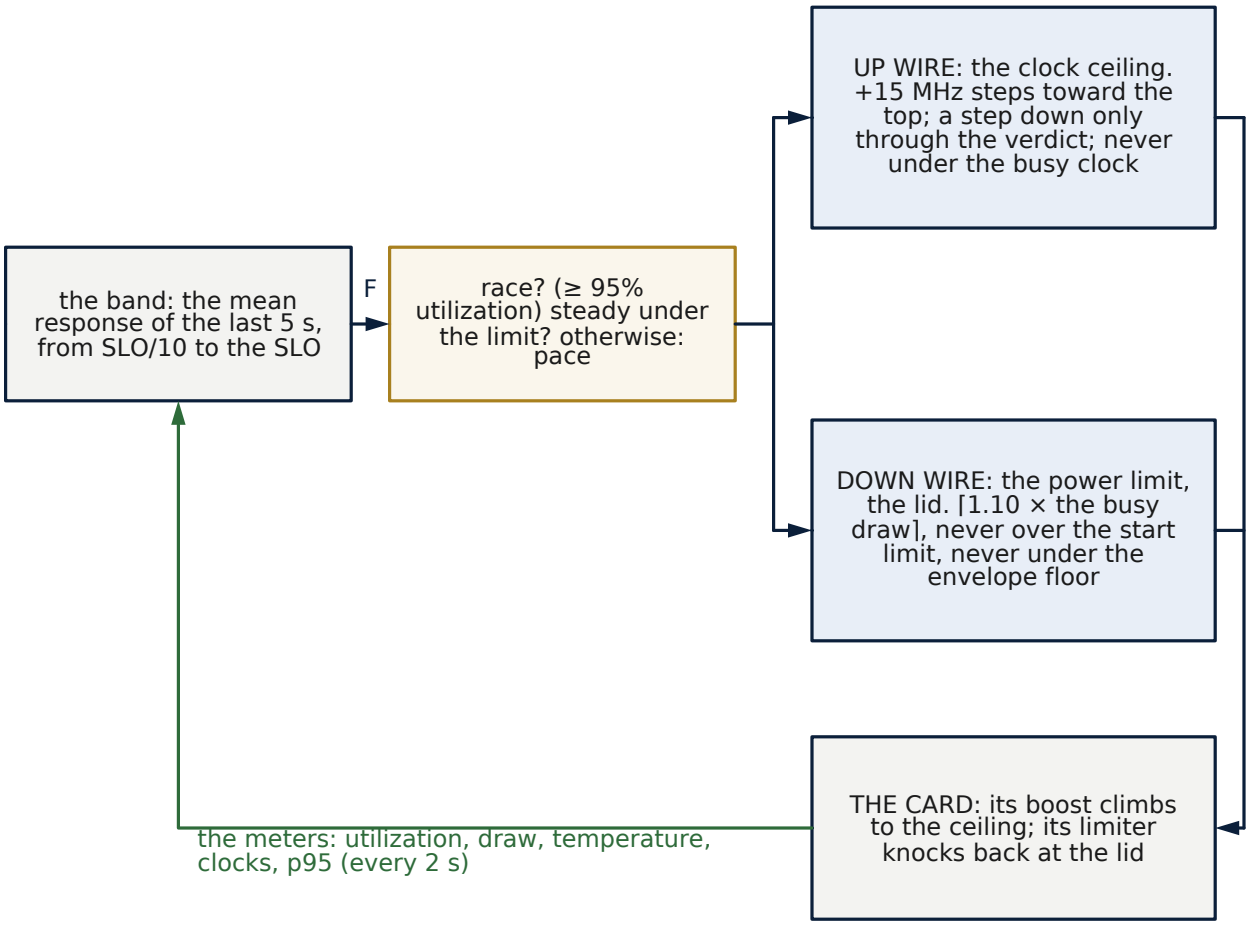



**Figure 49.** Block diagram: the two wires of a card as an antagonist pair. The band (the mean response of the last 5 s, from a tenth of the line to the line) yields the force F; the mode decides between racing (both wires to full while work waits, utilization ≥ 95%), steady under the limit (the card's own busy clock as the ceiling) and pacing.

The up wire is the clock ceiling, raised in 15 MHz steps toward the top and lowered only through the verdict; the down wire is the power limit, the lid at ⌈1.10 × the busy draw⌉, never over the start limit and never under the envelope floor. The card's own boost climbs to the ceiling and its own limiter knocks back at the lid; the meters close the loop every 2 s.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

## 22.5 The heat line

The aim on a card is more work at native's own heat. The governor reads the card's temperature and its own reported slowdown every decision and never tightens while the card reports a thermal slowdown; the card's own firmware protects the silicon underneath whatever the governor writes. Each receipt on a card reports the card's temperature in both arms beside the work and the power, so an implementer sees directly whether the omni arm ran hotter than native. A receipt in which it did is read as a wiring fault first: the clock ceiling raised where the card's cooling could not carry it.

## 22.6 OFF on a card

`sudo touch /tmp/omni-gpu-kill` (Omni-Compass on one card), `SIGTERM` to its process, or the master switch (`sudo python3 tools/omni_switch.py off`, every governor on the machine) each restore the card at once: clocks reset to the card's own range (`nvidia-smi -rgc`), the power limit back to the value read at start, both read back. A restore that does not read back exits with code 3 and must be completed by hand before anything runs again (Appendix D). The fault drill of the rented sequence kills the governor outright mid-run and times the watchdog's hand-back; an implementer repeats it on every new card model.

## 22.7 Cards inside Kubernetes

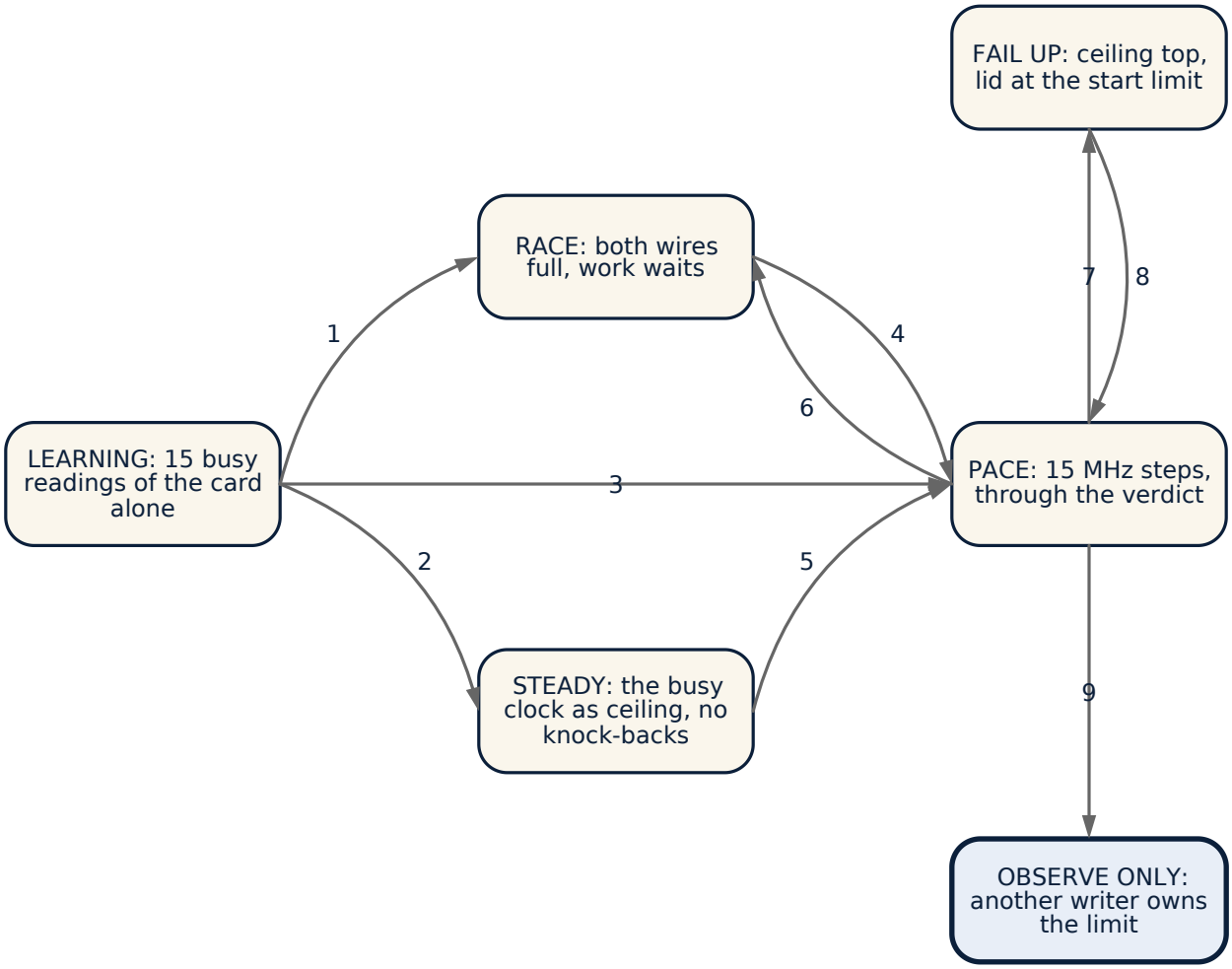
Where the cards are nodes of a Kubernetes cluster (the vendor's device plugin exposing `nvidia.com/gpu`), the card governor runs as a privileged agent on each GPU node (it writes the card's own settings, which needs the host's management interface), one per node, with its feed taken from the serving pods on that node. It never schedules or moves a pod; the cluster's own scheduler and autoscaler place the work, and Omni-Compass on the cluster (chapter 20) moves the machines, if the cards are in a cloud. The two governors are different writers of different settings and never touch the same one.

CHAPTER 23

# Observability: Feeding the Governor from Your Monitoring, and Watching It

A company that runs a cloud cluster already measures it: an ingress or a service mesh times every request, a metrics system (Prometheus, Azure Monitor, Google Cloud Monitoring, Amazon CloudWatch, Datadog) stores the series, and a log store keeps the clusters' logs. This chapter connects the governor to that apparatus in both directions: the company's own response times as the governor's feed (section 19.4), and the governor's own audit as series and alarms the company's engineers already read.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.




1 learned; utilization ≥ 95% · 2 learned; saturated at an operator's cap (draw ≥ 97% of it) · 3 learned; slack · 4 slack (the tune-down) · 5 off the cap: slack · 6 work waits again · 7 from any state: p95 at the line, a failed request, a blind feed, a heat slowdown · 8 clear for --slo-clear · 9 from any state: a limit not written by the governor

**Figure 50.** State machine: the two-wire card governor. It learns the card on its own first (15 busy readings), then races while work waits (utilization at 95% or more), holds steady at the card's own busy clock while saturated under an operator's cap, or paces the slack in 15 MHz steps through the verdict; from any state it fails up on a breach, a failed request, a blind feed or a heat slowdown, and it observes only, for the rest of the run, once a limit is found that it did not write.

## 23.1 The feed from the company's own response times

The governor's feed is a file of rows `elapsed_seconds,latency_ms,ok` (section 19.2). An adapter that writes it from the company's own measurements has four duties: read the measurements at the point where the line is written; write rows whose own distribution is the served requests' distribution; keep the feed as dense as the probe's (the probe writes five rows every five seconds; a feed thinner than twenty rows a minute leaves the governor's give-back trials short of samples, and the audit shows them as `trial abandoned`

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`(too little work)`, section 21.8); and never smooth. A metrics system stores counters and histograms, not single requests, so the adapter works in windows. For each window of length Δ ending at time t:

1. Read the count of requests served in the window, n[ok], and the count failed, n[fail], from the request counter's increase over exactly that window (not a rate over a longer one).
2. If n[ok] > 0, read the served requests' quantiles q[0.05], q[0.15], …, q[0.95] (the midpoints of ten equal shares of the requests) from the response-time histogram's bucket increases over the same window, and write one row with `ok` = 1 for each. Each row then stands for a tenth of the window's served requests, so the rows' own mean and percentiles are the window's, to the precision of the histogram's buckets.
3. Write one row with `ok` = 0 for each failure, up to a cap (twenty a window is enough for the governor to see a failing service; the receipt counts failures from its own probe or the service's own logs, not from the adapter).

Δ must be at least twice the metrics system's scrape interval, or the counter's increase over it is undefined (two samples are needed); with a 15-second scrape, a 30-second window read every 15 seconds gives forty rows a minute.

With Prometheus and the NGINX ingress controller, for example, n[ok] and n[fail] are the increases of `nginx_ingress_controller_requests` (status below 500, and 500 and above) for the service's ingress over the window, and the quantile q is `histogram_quantile(q, sum by (le) (increase(nginx_ingress_controller_request_duration_seconds_bucket[30s])))`. A histogram's buckets bound the precision of every quantile: a service whose line is 500 ms needs buckets around 250, 400, 500 and 750 ms, or the quantiles jump between bucket edges and the governor reads steps that are not there. A service mesh (Istio, Linkerd) exports the same two series under its own names; the application's own histogram is the closest measurement of all when the line is written at the application. Section 23.8 gives an adapter that does all of this.

**What the adapter must not do.** It must not average across windows, carry a value forward when a window is empty (an empty window is no row; a feed with no new row for twice the governor's window reads as blind, which is the correct reading of a service that answered nothing), or rewrite the file from the start (the governor reads rows newer than the last it read, by the feed's own clock; a restarted adapter continues the clock where it stopped, or the governor's pod is restarted with it, section 19.3).

## 23.2 The governor's own series

The audit is JSON, one record a line (section 21.4). A log store that parses JSON (Loki, Elasticsearch or OpenSearch, Azure Log Analytics, Google Cloud Logging, CloudWatch Logs, Datadog Logs) turns it into series without any code in the governor:

| Series | From the audit | What it shows |
|---|---|---|
| machines in service, machines seen | each `decision`: `nodes_observed`, `nodes_recommended` | the governor's view of the pool, decision by decision |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Series | From the audit | What it shows |
|---|---|---|
| machines parked and woken | each `write` whose `why` is `node pool size` (a decision) or `scheduling floor (pending pods)` (a fast check) | every move of the machine lever |
| trials and their outcomes | each `verdict`: `trial`, `step allowed`, `step refused`, `trial abandoned` | the give-back trials; abandoned trials point to a thin feed |
| the response time Omni-Compass read | each `decision`: `latency_p95_ms` | its own reading, beside the company's own |
| senses | the `senses` of the `authority` record written with each decision: `blind`, `latency_age_s` | a feed lost, or late |
| proprioception | the `proprioception` of the same `authority` record | a command that did not land, or another writer |
| decision time | each `decision_ms` | the governor keeping its period |
| own cost | the last `overhead` record | the governor's processor over its life |

Without a log store, `tools/omni_exporter.py` serves the same series to Prometheus. It follows the audit file the governor writes, answers `GET /metrics` on one port (9464 unless `--port` says otherwise, bound to 127.0.0.1 unless `--bind` says otherwise) and writes nothing anywhere: it has no button, so a monitoring system that scrapes it can see everything and change nothing. Run it beside the governor, pointed at the same audit:

```
python3 tools/omni_exporter.py serve --audit /var/log/omni/audit.jsonl
    ↪  --port 9464 --bind 0.0.0.0
python3 tools/omni_exporter.py once --audit audit.jsonl      # a finis
    ↪ hed run's audit, read once
```

| Gauge | What it shows |
|---|---|
| `omni_last_decision_timestamp_seconds` | the heartbeat: alert when `time()` less it passes three beats |
| `omni_heartbeat_seconds` | the beat the governor runs at, as measured: the median spacing of its last eleven decisions |
| `omni_master_switch` | 1 while governing, 0 once the kill switch has handed every setting back |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Gauge | What it shows |
|---|---|
| `omni_verdict_state{state}` | where the trial of a step stands: left native, trial, kept, given back |
| `omni_sense_blind{sense}` | 1 while a feed is blind; the governor then holds |
| `omni_decisions_total`, `omni_decision_seconds_sum`, `_count`, `_last` | its pace, and the time each decision took |
| `omni_writes_total{why}` | every settings command it wrote on its own, by the head of its reason |
| `omni_recommended{setting}`, `omni_recommended_moves_total`, `omni_recommended_reversals_total` | what its last decision recommends, how often that moved and how often it turned back; a recommendation is not a write |
| `omni_nodes_observed` | the machines it saw in service |
| `omni_own_cpu_cores` | its own processor, from its last overhead record |
| `omni_compass_decisions_total{zone}` | decisions by where the service stood against its line: `slack` (under the lower cushion), `band`, `wall` (at or past the line); the share in `band` is the time-in-band figure |
| `omni_wall_recovery_seconds_sum`, `_count`, `omni_wall_recovery_seconds_max` | from the first decision at the line to the first back inside the band: the settling time after each excursion |
| `omni_recommended_travel_total{setting}` | the summed size of every change of each recommendation (actuator travel) |
| `omni_errors_total{kind}` | the errors the governor recorded, by the head of the error (a refused write, an unreadable sense) |

The zone, recovery and travel gauges describe the governor's own steering and are not judged: native has no governor to read them from. The judged comparison of time over the line is the paired receipt's alert-minutes row (section 20.6). A test whose load holds the service near its line in both arms shows a large `wall` share in the governor's gauges while its alert minutes, the comparison with native, read GAIN.

`deploy/grafana/omni-compass.json` is a ready Grafana screen of these gauges: import it and choose the Prometheus data source. Its top row is the heartbeat, the switch, the verdict and the blind senses; below them the decisions, the writes by reason, the recommendations and their reversals, the machines and the governor's own processor, and at the foot the zones, the recovery time, the errors and the travel. Its links include the control panel's console (section 23.5).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 23.3 A dashboard an on-call engineer can read in ten seconds

Four panels, on the service's own dashboard, beside the service's own response time:

1. **The line and the response time.** The service's 95th percentile against its line, from the company's own measurement, with the governor's own reading (`latency_p95_ms`) as a second line. The two should agree; where they do not, the feed is measuring something else (section 19.4).
2. **Machines.** Machines in the pool (from the cluster), machines in service and machines parked (from the audit). A parked machine is never a problem in itself; a parked machine during a breach is impossible by design and is a fault to report.
3. **Trials.** Allowed, refused and abandoned trials an hour. Many abandoned trials: the feed is too thin. Many refused: the service is slower without the machine, and the governor is right to keep it.
4. **Health.** Blind decisions, errors, decision time and the governor's own processor.

## 23.4 Alarms, routed where the company's alarms already go

The alarms of section 21.5 are written once in the company's alerting system (Prometheus Alertmanager, Azure Monitor alerts, Google Cloud alerting policies, CloudWatch alarms, Datadog monitors) against the series above, and routed to the same on-call rotation as the service. For Prometheus they are written already: `deploy/prometheus/omni-compass-rules.yaml` (load it with `rule_files:`, or wrap its `groups:` in a PrometheusRule for the Prometheus Operator) carries ten rules, each with its severity, its meaning and the action to take: the governor stale (no decision for three intervals while writing; critical), an Omni-Compass process orphaned (died or hung without handing back; critical), writes hammering (more than 300 settings commands an hour), write errors, a recommendation oscillating (more than 20 reversals an hour), a sense blind for five minutes, the service at its line in more than 10% of decisions, the master switch off (information), the audit unreadable, and the governor's own processor above half a core. The staleness bound is three of the governor's own beats as the exporter measures them (30 s at the factory's 10 s on site, 180 s at its 60 s through a cloud, 360 s at 120 s), so it needs no edit when the dial is turned; the other rules count in minutes and hours, not in decisions. The alert that matters most is not the governor's: it is the service's own line. An installation in a cloud is judged first by whether that alert fires more often with the governor than without it, and the paired receipt's alert-minutes row (section 20.6) is the same measure taken under control.

## 23.5 The control panel: the buttons, behind sign-in, every press logged

`tools/omni_control.py` runs the governor as its child and serves its buttons over an authenticated HTTP API, with a console page for a browser. It writes no setting itself. The governor takes its mode and its dial when it starts, and when its own kill file appears it hands every setting it wrote back to the snapshot it took at its start, reads each one back and exits. So every press that changes the mode or the dial is carried out in four steps: the kill file is set, the governor's exit is awaited, the file is removed, and the governor is started again with the new arguments. Every change passes through native, and nothing is changed under a running governor.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
python3 tools/omni_control.py token --name alice --roles operator --to
    ↪ kens /etc/omni/tokens.json     # printed once
python3 tools/omni_control.py token --name sam --roles security --toke
    ↪ ns /etc/omni/tokens.json
python3 tools/omni_control.py serve --tokens /etc/omni/tokens.json --a
    ↪ udit /var/log/omni/audit.jsonl \

    --log /var/log/omni/control.jsonl --state /var/lib/omni/control.js
    ↪ on \
    --governor "python3 -m omni_controller.controller --mode nodepool
    ↪ --audit /var/log/omni/audit.jsonl --interval 60"
```

| Button | Request | Role | What happens |
|---|---|---|---|
| Autopilot | `POST /v1/mode {"mode": "autopilot"}` | operator | the governor runs and writes |
| Watch | `POST /v1/mode {"mode": "watch"}` | operator | the governor runs, decides and audits every decision, and writes nothing (its `--dry-run`) |
| Off | `POST /v1/mode {"mode": "off"}` | operator | every setting handed back; the governor stops |
| Max, tune-down | `POST /v1/dial {"dial": "max" or "save"}` | operator | the governor handed back and started again on the new dial |
| One muscle on, or left native | `POST /v1/muscle {"muscle": NAME, "on": true or false}` | operator | the governor handed back and started again with `--native` naming every muscle left native; a muscle left native is never written (omni-v14; the names are `machines`, `replicas`, `cpu`, `clock`, `gpu`, `rollout`, `batch`, `rightsize`, `coldstart`, `contain`, `cooling`) |
| Heartbeat | `POST /v1/beat {"seconds": 5 to 120}` | operator | the governor handed back and started again with `--interval` set to the beat: its full decision every so many seconds (the factory's 10 on site, 60 through a cloud; the console's buttons 5, 15, 30, 60, 75, 90 and 120), the fast checks every 5 s at every beat (omni-v17) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Button | Request | Role | What happens |
|---|---|---|---|
| Reset | `POST /v1/reset {"reason": ...}` | operator | every setting handed back to native; mode off until the next press |
| Kill switch | `POST /v1/kill {"reason": ...}` | security | the master switch OFF for every Omni-Compass governor on the machine; each restores and exits; none starts while it is OFF |
| Arm | `POST /v1/arm {"reason": ...}` | security | the master switch ON again once two different holders of the security role press it within ten minutes |

The reads need the viewer role, which every token holds: `GET /v1/status` (the switch and its reason, the mode and dial the panel holds, the governor's process and its exit code, the registry with each lease's age, the governors orphaned, the age of the last decision), `GET /v1/presses` (the last presses, refused ones included) and `GET /metrics` (the panel's counters: presses by button and outcome, the mode, whether the governor is up, the master switch, the orphans). `GET /` is the console and `GET /v1/openapi.json` the OpenAPI 3 document; neither carries data, and every call the console makes carries the token typed into it. Reset, kill and arm need a reason, and every press, refused or done, is one JSON line in the control log: the time, the holder, the button, the reason, the outcome.

Security: the panel binds to 127.0.0.1 unless `--bind` says otherwise, and refuses a bind beyond the loopback without `--cert` and `--key` (HTTPS, TLS 1.2 or later). Tokens are stored as SHA-256 digests only, in a file only its owner can read; a request without a valid token is answered 401, one whose holder lacks the role 403. A governor that exits on its own is not restarted by the panel: its lease hands back what it wrote (`tools/omni_switch.py watchdog`), and a new start would take a new snapshot over settings not yet handed back; the status shows it as exited and the next press starts it. The kill switch stays on the command line and as a file as well (chapter 9), so the panel being down never stops anyone from switching Omni-Compass off.

## 23.6 The bill, from the provider's own records

The receipt's bill is the machines the cluster held, counted every 15 seconds, at list price (chapter 12). In service, the company reads the same thing from the provider's own billing records, and should, because list price is not what every company pays:

| Provider | Where the machine cost lives | How to separate the governed pools |
|---|---|---|
| Azure | Cost Management (cost analysis), or the cost export to a storage account | the node resource group of the cluster, and a tag on each work pool (`az aks nodepool update --tags`) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Provider | Where the machine cost lives | How to separate the governed pools |
|---|---|---|
| Google Cloud | the Cloud Billing export to BigQuery (detailed usage cost) | GKE cost allocation, enabled on the cluster, by namespace and node pool label |
| Amazon | the Cost and Usage Report, or Cost Explorer | a cost allocation tag on each node group's instances, activated in the Billing console |

The comparison that means something is the same service, the same weeks of the year, before and after the governor went into service, in machine-hours of the governed pools, with the load (requests served) beside it; a bill that fell because traffic fell is not the governor's saving, and the paired receipt, run on the same day for both arms, exists to remove exactly that doubt.

## 23.7 Tracing a single decision end to end

An auditor's or an engineer's question about one moment ("why was this machine parked at 03:12?") is answered from the audit alone, in this order: the `verdict` record that allowed the step (the trial's response times with and without the machine); the `authority` and `decision` records of that minute (the senses, the release gate's reason, the machines seen and recommended); the `write` record (the command, its reason, whether it was a dry run); and the next decision's `authority` record, whose `proprioception` shows whether the command landed. If any of the four is missing, the governor did not park the machine, and the cause is elsewhere (the cluster autoscaler's own scale-down, a person, another controller).

## 23.8 A complete adapter: Prometheus to the feed

The repository's `tools/prom_feed.py` is the adapter of section 23.1 for any metrics system that answers Prometheus's HTTP query API (Prometheus itself, Thanos, Cortex, Grafana Mimir, and the managed services of the three clouds that expose that API). It reads three queries the implementer writes for the service, every `--interval` seconds over the last `--window`, and appends the rows of section 23.1 to the feed. It continues the feed's clock across its own restarts and never truncates the file; it writes nothing for an empty window or when the metrics system does not answer, so the governor reads a silent adapter as blind. It is in the cloud image (`deploy/Dockerfile.cloud`) and runs in place of the probe container:

```
      - name: feed                    # in place of the probe: the se
  ↪ rvice's own response times, from Prometheus
        image: <registry>/omni-compass-cloud:<version>
        command: ["python", "/app/tools/prom_feed.py"]
        args:
          - "--prometheus=http://prometheus-operated.monitoring.svc:
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ 9090"
            - "--served=sum(increase(nginx_ingress_controller_requests
    ↪ {ingress=\"checkout\",status!~\"5..\"}[{w}]))"
            - "--failed=sum(increase(nginx_ingress_controller_requests
    ↪ {ingress=\"checkout\",status=~\"5..\"}[{w}]))"
            - "--quantile=histogram_quantile({q}, sum by (le) (increas
    ↪ e(nginx_ingress_controller_request_duration_seconds_bucket{ingre
    ↪ ss=\"checkout\"}[{w}])))"
            - "--unit-ms=1000"
            - "--interval=15"
            - "--window=30s"
            - "--out=/feed/latency.csv"
          resources: {requests: {cpu: 10m, memory: 32Mi}, limits: {cpu
    ↪ : 100m, memory: 64Mi}}
          securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
          volumeMounts: [{name: feed, mountPath: /feed}]
```

In the queries, `{w}` is replaced by the window and `{q}` by each quantile; every other brace is PromQL's own. The network policy of section 27.5 then allows the metrics system's address in place of the service's. Before the governor relies on the adapter, check its feed as section 19.5 checks any feed, and run it beside the probe for a day: the two feeds' 95th percentiles should agree within the histogram's precision, and the dashboard's two lines (section 23.3) should coincide.

| Setting | Default | What it is |
|---|---|---|
| `--prometheus` | (required) | the metrics system's base URL |
| `--served`, `--failed`, `--quantile` | (required; `--failed` may be empty) | the three PromQL templates |
| `--quantiles` | 0.05, 0.15, ..., 0.95 | the quantiles read, one row each |
| `--unit-ms` | 1000 | milliseconds per unit of the histogram (1000 for seconds) |
| `--interval` | 15 s | seconds between reads |
| `--window` | `30s` | the range of each read: at least twice the scrape interval |
| `--fail-cap` | 20 | the most failure rows a window |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Default | What it is |
|---|---|---|
| `--out` | (required) | the feed the governor reads (`--latency-file`) |
| `--once`, `--duration` | off, 0 | read one window and exit; run for a number of seconds |

CHAPTER 24

# Change Management, Risk and Compliance

A governor that writes to production is a change to production, and companies with a change-management process will treat its installation as one. This chapter maps the phases of section 14.3 onto that process, lists the risks a risk owner will ask about with the control that answers each, and names the evidence a compliance review will want.

## 24.1 The installation as changes, phase by phase

| Phase | Change class it usually is | Why |
|---|---|---|
| 1. The cluster | the company's own class for new infrastructure | the governor is not involved yet |
| 2. Identity | standard (pre-approved) | a namespace, a service account and a read-only role |
| 3. Watch | standard | the governor writes nothing; its role cannot write |
| 4. Feed and line | standard, or normal where an adapter is new code | the line is a decision of the service owner, recorded |
| 5. Paired run | none in production | the paired run is on its own clusters, built and deleted by the run |
| 6. Govern | normal (reviewed and approved) | the first grant that can write, and the first write |
| 7. Upgrades | normal, with the same gates as phase 6 | a new engine version is a new governor (chapter 13) |

The normal change of phase 6 carries, as its evidence, the paired receipt of phase 5 read row by row, the grant and its `kubectl auth can-i` proof (section 15.4), the OFF switch drill (section 21.3), and the rollback

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

plan, which is short: turn the governor OFF, and every setting it ever wrote is back at the operator's, read back, within 60 seconds.

## 24.2 The risks and the controls that answer them

| Risk | Control | Evidence |
|---|---|---|
| The governor makes a decision that harms the service | it gives back a machine only after its own trial shows the response time holds; a breach wakes it at the next decision; nothing is given back while a sense is blind | the `verdict` and `decision` records; the paired receipt |
| The governor raises the bill | in a cloud the bill is judged by the one key beside the work it bought: a tune whose work per dollar falls against native is not admissible as the factory tune (chapter 20, `docs/THE_TUNE.md`); the governor asks the cloud for no machine itself | the receipt's bill row; the machine lever's grant (patch nodes, patch pods; no cloud credential) |
| The governor dies holding machines parked | the lease and the watchdog hand everything back from the records on the objects (section 21.2); a parked machine never blocks a pod | the watchdog drill |
| Someone else moves the same setting | the one-writer rule; the drift is visible in every decision | the `proprioception` field of each `authority` record |
| A compromised governor is used against the cluster | least privilege: read, patch nodes and patch pods; no delete, no eviction, no secrets, no cloud credential (chapter 15) | the role and its proof |
| A tampered copy is installed | the copy is verified against its manifest and seal before it is installed (chapter 33) | the verification receipt |
| The feed lies | the feed is checked before the governor watches (section 19.5); a stopped feed reads as blind and the governor holds | the feed checks |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Risk | Control | Evidence |
|---|---|---|
| The audit is lost | the audit on a persistent volume, shipped to the log store (section 21.4) | the log store's retention |

## 24.3 What a compliance review will ask for

Frameworks differ in their words, not in the questions. A review against the company's change-management control (for example SOC 2's change-management criterion, CC8.1, or ISO/IEC 27001:2022 Annex A control 8.32), its logging control (Annex A control 8.15) and its privileged-access control (Annex A control 8.2) asks the same five things of the installation, and each has its answer in the installation record (section 15.7):

1. **Who approved the change?** The phase-6 change record, with the service owner's and the platform owner's sign-off.
2. **What exactly can the system do?** The role, the `kubectl auth can-i` output, and the statement that it holds no cloud credential.
3. **How is every action recorded?** The audit: every decision, write and reason, on a persistent volume, shipped to the log store, kept for the retention period.
4. **How is it stopped?** The OFF switch, its holder, and the drill records.
5. **How do you know it works?** The paired receipt, read row by row on the company's own cluster, on the engine version in service.

## 24.4 Separation of duties, restated for auditors

The implementer installs; the service owner writes the line and signs the receipt; the platform owner grants the governing role; security holds the OFF switch; finance signs the bill rows (section 14.4). No one person can both grant the governor its writes and be the only person able to stop it.

## 24.5 Records an installation keeps, and for how long

| Record | Keep for |
|---|---|
| The verification receipt and engine version of every version installed | the life of the installation |
| The grant as applied, with its proof | the life of the installation, and every change to it |
| The line, its basis, and who signed it | the life of the line |
| **Record** | **Keep for** |
| Every paired receipt that decided a phase | the life of the engine version it was made on |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Record | Keep for |
|---|---|
| The audit | the company's retention period for production change records |
| Every OFF-switch and watchdog drill | the company's retention period for control tests |

## 24.6 Communicating the installation

The people who will notice the governor are the service's on-call engineers, and they are told before it governs: what it moves (the service's autoscaler target inside its cover, the pods' processor and which machines are in service), what it never moves, how a parked machine looks in `kubectl get nodes` (a `PreferNoSchedule` taint, `omnicompass.io/idle`), where its audit is, how to turn it OFF, and who holds the switch. An engineer who finds a tainted node at three in the morning and does not know why is the failure this section exists to prevent.

CHAPTER 25

# Planning a Cloud Run: Capacity, Cost and the Shape of Load

A paired run costs days of the implementer's time and the price of two clusters. Before it is spent, the implementer can bound, from the company's own monitoring and with arithmetic alone, how many machine-hours the machine lever could give back on the pools it will govern, and where in the day they live. This chapter gives that bound, the settings that move it, and the pricing terms that decide whether a machine-hour given back is a dollar. It is one part of a cloud run's reading: the work per dollar the software end buys (the processor conveyed, the target moved ahead of the load) is measured only by the paired run, and a smaller bill is never the aim (chapter 20). Nothing in it predicts what the governor will do: the bound is a ceiling that no machine lever on top of the same pods can pass, and the paired run (phase 5) is the only measurement of how much of it the governor takes while the line holds.

## 25.1 Where the bill of a governed pool comes from

In every provider's on-demand pricing, a work pool's compute bill over a period T is the sum, over the machines that existed in it, of each machine's price per hour times the hours it existed:

$B = \Sigma_m \, p_m \, h_m$, $h_m = \int_0^T 1[\text{m exists at t}] \, dt$ .

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

For a pool of one machine family at list price p, this is $B = p \int_0^T N(t)\, dt$, where N(t) is the number of machines in the pool at time t. The machine lever lowers B in one way only: it makes machines exist for fewer hours. It parks a machine (a `PreferNoSchedule` taint, chapter 20), marks that machine's pods first to go (`controller.kubernetes.io/pod-deletion-cost`), so that the next scale-down of the service's own HPA removes them first, and leaves the machine to empty; the cloud's own cluster autoscaler then deletes the empty machine on its own schedule. Every term of B that the governor reduces is a machine the cluster autoscaler deleted. Two consequences follow, and an implementer plans with both:

1. **The saving is realised by the cluster autoscaler.** A parked machine still exists, and is still billed, until the autoscaler deletes it. The autoscaler's own profile (section 25.4) sets how long that takes.
2. **The saving needs the pods to leave.** A parked machine whose pods stay keeps serving and keeps billing. Pods leave when the service's HPA scales down (the ReplicaSet controller removes the pods with the lowest deletion cost first) or when the cluster autoscaler drains an underused machine on its own rules. A service whose replica count never falls gives a parked machine no way to empty.

## 25.2 The room: an upper bound on what any machine lever can save

Define, for each sampling instant t of the governed pools:

- N(t): the machines in the pools;
- $Q_{[c]}(t)$ and $Q_m(t)$: the processor and memory **requests** of the pods running on them, DaemonSet pods excluded (a DaemonSet runs one pod on every machine whatever the count, so it is a per-machine overhead, not work);
- $C_{[c]}$ and $C_m$: one machine's allocatable processor and memory (`kubectl get node <name> -o jsonpath='{.status.allocatable}'`) less the requests of the DaemonSet pods on it;
- N[f]: the floor of machines in service (`--min-nodes`, never fewer than 2).

The fewest machines that could hold the pods' requests at instant t, packing perfectly, is

$$N_{[req]}(t) = \max\left(N_{[f]}, \left\lceil \max\left(\frac{Q_{[c]}(t)}{C_{[c]}}, \frac{Q_m(t)}{C_m}\right)\right\rceil\right),$$

and the **room** over the period is the machine-hours above it:

$$R = \int_0^T \left(N(t) - N_{[req]}(t)\right)^+ dt \approx \sum_k \Delta t \left(N(t_k) - N_{[req]}(t_k)\right)^+,$$

where $(x)^+ = \max(x, 0)$ and $\Delta t$ is the sampling interval. R is an upper bound for three reasons, each of which a real cluster exhibits: perfect packing is not achievable (pods have affinity rules, topology spread constraints and sizes that do not tile a machine); the scheduler places pods by its own scoring, not by packing; and a machine that is parked must also empty before the cloud deletes it. The machine lever never asks the cloud for a machine, so the governed pool can never fall below $N_{[req]}$ by its action, and it does not try to: every machine it parks first passes its own trial against the line. (A pod target moved ahead of the load may ask native's autoscaler for more pods, and the cloud's autoscaler for a machine to hold them; that machine is on the bill row beside the work it bought.)

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**The billable room.** Where the company holds a commitment that pays for K machines of the family whether they exist or not (section 25.5), only machine-hours above K reach the bill:

$R[\$] = p \, \Sigma_k \, \Delta t \, (N(t_k) - \max(K, N[req]\,(t_k)))^+ .$

**Omni-Compass's own floor.** It never parks a machine that would leave the processor requests of the pods on the machines in service, with the spare of its `--headroom` setting (half again, by default; Appendix K), or their processor in use, without a place. With $Q'[c]\,(t)$ the requests of every pod on the machines in service (DaemonSet pods included), $U[c]\,(t)$ the processor in use, and $C'[c]$ one machine's allocatable processor, it keeps at least

$N[gov]\,(t) = \max(N[f], \lceil((1+h)\,Q'[c]\,(t))/(C'[c])\rceil, \lceil(U[c]\,(t))/(C'[c])\rceil), \; h = 0.5,$

so the room the machine lever can take is at most $R[gov] = \int_0^T (N(t) - N[gov]\,(t))^+ \, dt \le R$. A pool whose pods' requests, with half again to spare, already fill the machines the cluster autoscaler keeps has $R[gov] = 0$: there is nothing to park, and a paired run on it reads STEADY on the bill, correctly.

**An arithmetic illustration (not a measurement).** A pool holds 12 machines from 08:00 to 20:00 and 8 machines overnight; the pods' requests need 9 machines at the day's level and 3 overnight; the floor is 2 and there is no commitment. Then $R = 12\text{ h} \times (12 - 9) + 12\text{ h} \times (8 - 3) = 36 + 60 = 96$ machine-hours a day. The overnight trough holds almost two thirds of the room; a paired run made only at the day's level could show at most 36 of the 96, and a run at a steady peak where $N = N[req]$ could show none. The illustration's numbers are invented to show the arithmetic; the company's own samples replace them.

## 25.3 Measuring the room on the company's own cluster

The samples need nothing the cluster does not already have. The following loop, run from any workstation or as a CronJob with the watch role of chapter 15, writes one row every five minutes for the pools labelled `omni-role=work`:

```
#!/usr/bin/env bash
# One row every 5 minutes: time, machines in the governed pools, the r
    ↪ equests of the pods on them (DaemonSet pods
# excluded: processor in millicores, memory in MiB), and the processor
    ↪  requests of every pod on them (for N_gov).
SEL="omni-role=work"; OUT="room.csv"

[ -s "$OUT" ] || echo "epoch,machines,cpu_req_m,mem_req_mib,cpu_req_al
    ↪ l_m" > "$OUT"
while true; do
  nodes=$(kubectl get nodes -l "$SEL" -o json | jq -c '[.items[].metad
    ↪ ata.name]')
  kubectl get pods -A --field-selector status.phase=Running -o json |
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ jq -r --argjson nodes "$nodes" '
    def mcpu: if endswith("m") then .[:-1] | tonumber else tonumber *
    ↪ 1000 end;
    def mib:  if endswith("Ki") then .[:-2] | tonumber / 1024 elif end
    ↪ swith("Mi") then .[:-2] | tonumber
              elif endswith("Gi") then .[:-2] | tonumber * 1024 elif e
    ↪ ndswith("k") then .[:-1] | tonumber * 1e3 / 1048576
              elif endswith("M") then .[:-1] | tonumber * 1e6 / 104857
    ↪ 6 elif endswith("G") then .[:-1] | tonumber * 1e9 / 1048576
              else tonumber / 1048576 end;
    [.items[] | select(.spec.nodeName | IN($nodes[]))] as $on
    | [$on[] | .spec.containers[].resources.requests // {}] as $all
    | [$on[] | select(all(.metadata.ownerReferences[]?; .kind != "Daem
    ↪ onSet"))
             | .spec.containers[].resources.requests // {}] as $r
    | [now | floor, ($nodes | length),
       ($r | map(.cpu // "0" | mcpu) | add // 0 | floor), ($r | map(.m
    ↪ emory // "0" | mib) | add // 0 | floor),
       ($all | map(.cpu // "0" | mcpu) | add // 0 | floor)]
    | @csv' >> "$OUT"
  sleep 300
done
```

Two weeks of rows cover two weekly cycles. The room then follows from the formulas of section 25.2:

```
import csv, math
C_CPU, C_MEM = 3860, 12800      # one machine's allocatable less its D
    ↪ aemonSet requests (millicores, MiB): read them
FLOOR, DT_H, K = 2, 300 / 3600, 0  # the floor, the sampling interval
    ↪ in hours, the committed machines
rows = list(csv.DictReader(open("room.csv")))

need = [max(FLOOR, math.ceil(max(int(r["cpu_req_m"]) / C_CPU, int(r["m
    ↪ em_req_mib"]) / C_MEM))) for r in rows]
room = sum(DT_H * max(0, int(r["machines"]) - n) for r, n in zip(rows,
    ↪  need))
billable = sum(DT_H * max(0, int(r["machines"]) - max(K, n)) for r, n
    ↪ in zip(rows, need))
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
C_ALLOC, H = 3860, 0.5            # one machine's allocatable process
    ↪ or (millicores), the governor's headroom
gov = [max(FLOOR, math.ceil((1 + H) * int(r["cpu_req_all_m"]) / C_ALLO
    ↪ C)) for r in rows]
room_gov = sum(DT_H * max(0, int(r["machines"]) - g) for r, g in zip(r
    ↪ ows, gov))

print(f"{len(rows) * DT_H:.0f} h sampled: room {room:.0f} machine-hour
    ↪ s, billable room {billable:.0f}, "
      f"room under the governor's own floor {room_gov:.0f}")
```

The constants `C_CPU`, `C_MEM` and `C_ALLOC` above are placeholders for a machine with four virtual processors; each installation reads its own. The use term of N[gov] is left out of the script, because on a pool that holds its requests it rarely binds; it can be added from `kubectl top nodes`. The same rows, plotted by hour of the week, show where the room lives, and so which load the paired run must carry (section 25.6).

## 25.4 The settings that bound the room, and whose they are

None of the settings below is the governor's, and none is changed by it. They are the operator's, they shape native and omni alike, and a paired run holds each of them identical in both arms.

| Setting | Where | What it does to the room |
|---|---|---|
| The floor (`--min-nodes`, `MIN_NODES`) | the governor's command line | the least the governor keeps in service; set it to at least the number of zones the service's availability design needs serving at every moment, never fewer than 2 |
| The pools labelled `omni-role=work` | the node pools' labels | only these machines may be parked; a pool left unlabelled is never touched |
| The HPA's minimum replicas | the service's HPA | a replica floor holds pods, and their machines, at every hour |
| The HPA's scale-down behaviour (`behavior.scaleDown`, stabilization window 300 s by default) | the service's HPA | a long window keeps pods after a peak, so parked machines empty later |
| The cluster autoscaler's scale-down profile | the provider (table below) | how long an empty or underused machine is kept before it is deleted |
| PodDisruptionBudgets | the service's manifests | a budget of zero disruptions keeps the cluster autoscaler from draining a machine; native is held the same way |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Where | What it does to the room |
|---|---|---|
| Pods the cluster autoscaler will not move (`cluster-autoscaler.kubernetes.io/safe-to-evict: "false"`, local storage on some providers, bare pods with no controller) | the workloads | such a pod pins its machine until it ends |
| Spot or preemptible pools | the node pools | billed at the spot price; parking one saves that price; the provider may reclaim it on its own |

The cluster autoscaler's scale-down profile is set per provider:

| Provider | How the profile is set | The settings that matter |
|---|---|---|
| Azure (AKS) | `az aks update -g <rg> -n <cluster> --cluster-autoscaler-profile scale-down-unneeded-time=10m scale-down-delay-after-add=10m scale-down-utilization-threshold=0.5 scan-interval=10s` (the values shown are AKS's defaults) | how long a machine must be unneeded before deletion; how long after a scale-up no scale-down happens; the utilisation under which a machine counts as unneeded |
| Google (GKE) | `gcloud container clusters update <cluster> --autoscaling-profile balanced` or `optimize-utilization` | the profile as a whole; `optimize-utilization` removes underused machines sooner |
| Amazon (EKS), Cluster Autoscaler | the flags on the autoscaler's own Deployment: `--scale-down-unneeded-time`, `--scale-down-delay-after-add`, `--scale-down-utilization-threshold`, `--scan-interval` | as on Azure |
| Amazon (EKS), Karpenter | the NodePool's `spec.disruption`: `consolidationPolicy` (`WhenEmpty` or `WhenEmptyOrUnderutilized`) and `consolidateAfter` | how soon an empty or underused machine is consolidated away |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

A shorter scale-down profile also lowers native's bill. The comparison the governor is judged on is always at the same profile in both arms; a saving that came from a profile changed for one arm only is not the governor's, and a paired run that changes the profile between arms is not a paired run.

## 25.5 Pricing terms, and whether a machine-hour saved is a dollar saved

| Pricing term | What a deleted machine-hour saves | Planning rule |
|---|---|---|
| On-demand, billed by the second or the minute (the common case on all three providers; read the provider's own page for the machine family) | its full price | the billable room is the room |
| A reservation, committed-use discount or savings plan covering K machines of the family | nothing below K; the full price above K | size commitments to the governed pool's floor and trough, not its peak; R[$] of section 25.2 |
| Spot or preemptible | the spot price | govern the on-demand pools first; a spot pool's machines can vanish on the provider's notice, governor or not |
| Software licensed per machine or per core (a commercial operating system, an agent billed per node) | that licence too, where it is billed by the hour the machine exists | add it to p |
| The cluster's own fee (the control plane), storage, network egress, load balancers | nothing | never in the bill rows of a receipt; the governor moves none of them |

The bill rows of a receipt (section 20.6) price machine-hours at the provider's list price for the family, because list price is the one price both arms share and anyone can check. The company's own discounts change the dollars, never the machine-hours; finance converts the machine-hour row to the company's own price after the run (section 23.5).

## 25.6 Choosing the load for the paired run from the shape of the room

| Shape of the company's load | Where the room lives | The paired run that can show it |
|---|---|---|
| Steady at a level the pool was sized for | nowhere: N ≈ N[req] | none; a steady run reads STEADY on the bill, honestly, and is still worth running for the response-time |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Shape of the company's load | Where the room lives | The paired run that can show it |
|---|---|---|
| | | and alert rows |
| A daily cycle (business hours, consumer evenings) | the trough and the descent after the peak | a replay of the company's own day, compressed only as far as the autoscaler's profile allows (a 10-minute unneeded time cannot be compressed below 10 minutes), or the repository's own day-shaped load (the Google 2011 trace on one to nine load generators) |
| Bursts separated by quiet stretches | between bursts, as native's autoscaler holds machines through its scale-down delay | the burst steps of the service's own traffic; in the repository, the burst sets of section 20.6 |
| A weekly cycle with a quiet weekend | the weekend | a run long enough to include a quiet stretch as long as the profile's scale-down time several times over |
| Batch windows | after each window | the batch itself, with the service's line on the jobs' own completion time where that is the objective |

**The run must be long enough for the autoscaler to act.** A quiet stretch shorter than the cluster autoscaler's unneeded time plus its scan interval gives neither arm a chance to delete a machine, and the bill reads STEADY for a reason that has nothing to do with the governor. Plan each quiet stretch of the paired run to last at least three times the unneeded time.

## 25.7 What the plan delivers to the paired run

Before phase 5 starts, the implementer writes, and the service owner and finance sign:

1. the room and the billable room from two weeks of samples, with the constants used;
2. the hours of the week that hold most of the room;
3. the load the paired run will carry, chosen from section 25.6, with its quiet stretches sized to the profile;
4. the settings of section 25.4 as they stand, identical in both arms;
5. the price used for the bill rows (list price) and the company's own price for the conversion afterwards.

A receipt that then reads GAIN on the bill, with response time, failed requests and alert minutes reading STEADY or GAIN, has measured a share of a room that was known beforehand; a receipt that reads STEADY

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

on the bill where the plan found room is a wiring question to answer first (section 5.5): the feed, the line, the autoscaler's profile, a pod that pins its machine.

CHAPTER 26

# On Your Own Racks and in Private Clouds

Many companies that run a public cloud also run Kubernetes on machines they own: bare-metal clusters in their own data centers, Red Hat OpenShift, VMware's Kubernetes on vSphere, Rancher's distributions, clusters at the edge, and clusters with no route to the internet at all. This chapter is for the implementer who installs Omni-Compass there. The plug, the grant, the feed, the audit, the OFF switch and the phases are the same as in the cloud; what changes is what a machine costs, and so which way the dial stands.

## 26.1 What changes when the machines are already paid for

In a public cloud every machine that exists is billed by the hour, so the benefit is the bill and the dial stands at the tune-down (chapter 20). On machines the company owns, the machine is paid for whether it works or idles; an idle machine saves only the difference between its idle power and its working power, and nothing at all of its purchase price. The benefit there is **more work from the same machines**: the dial stands at the max (section 4.2), every muscle run at its own limit and never past it. The max never gives back a machine the cluster had on its own; it works the pods, the processor and, where wired, the power, each at its limit.

| | Public cloud | Own racks or a private cloud |
|---|---|---|
| What an idle machine costs | its full hourly price | its idle power, and its share of cooling |
| The dial | the tune-down (`--dial save`) | the max (`--dial max`), the default |
| The levers | the same levers, each moved toward the least resource (the v5 edge) | the pods, the processor, the power and the machines, level by level (section 7.3) |
| What the receipt is judged on | the bill, at native's work, speed and failures | work, speed, misses and failures; the power it took shown beside them as power given |
| The meter | the provider's bill at list price | the machine's own meter, a wall meter, or the facility's (evidence class P) |

An owner who wants the power an idle machine draws back, rather than more work, turns the dial down to the tune-down on that cluster; the machines then go back one at a time through the trial, exactly as in the cloud, and the receipt reads the machines and the power instead of the bill.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 26.2 Bare-metal Kubernetes

A cluster built on the company's own machines (with kubeadm, RKE2, k3s, Talos or a vendor's distribution) usually has no cluster autoscaler: the machines exist all the time. The wiring follows section 7.3 level by level, with three points specific to bare metal.

**No machine is created or deleted by the governor.** The shipped machine script (`scripts/kind_nodepool.sh`) parks and wakes machines that exist, with a `PreferNoSchedule` taint; a parked machine stays powered, joined and Ready, and a pod that needs it lands on it at once. On bare metal, parking packs the work onto the machines left in service, and a parked machine whose pods have gone idles at its idle power.

**Powering a parked machine down is the operator's own command, and its price is the boot.** The governor's machine lever runs whatever command the operator writes in `--node-scale-cmd`, with `{n}` replaced by the number of machines to keep in service. An operator may write a command that, in addition to parking, powers an empty parked machine down through its baseboard management controller (Redfish or IPMI, or a provisioning system such as Metal3 or MAAS) and powers it up before waking it. The command's contract is the shipped script's:

| The command must | Why |
|---|---|
| keep exactly `{n}` machines in service, never fewer than the floor and never more than exist | the governor reads the count back as proprioception (section 21.4) |
| be idempotent: the same `{n}` twice changes nothing the second time | the governor issues the count again at its next decision while the machines in service differ from it |
| exit 0 when the count stands, and non-zero when it cannot be met | a non-zero exit is recorded as `refused`, and the governor learns the pool's top from it |
| never move, evict or delete a pod, and never power down a machine carrying a pod | parking never disturbs work |
| never touch a machine someone else cordoned | a cordon is a person's or an upgrade's, never the governor's (section 29.1) |
| wake a powered-down machine and return only when it is Ready | a woken machine must take pods |

A powered-down machine takes minutes to boot and join; the shipped script's parked machine takes no time at all. A wake that takes minutes turns the governor's answer to a breach from seconds into minutes. An installation that powers machines down therefore runs its paired run (phase 5) with the power-down command in place, so that the boot time is in the receipt's response-time and alert-minute rows, and keeps powered-down machines only beyond a number of parked, powered machines large enough to carry the service's largest burst.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**The meter.** On owned machines the power the governor gives or takes is measured, not priced. The repository's wall meter reader (`tools/wall_meter.py`) reads a networked power meter or a power distribution unit; Kepler (the Kubernetes efficiency exporter) and the processor's own energy counters (RAPL on x86 servers) read per-machine estimates. A receipt that reports power names its meter and its evidence class (chapter 12).

## 26.3 Red Hat OpenShift and OKD

OpenShift adds three things an implementer must account for.

**Security context constraints.** OpenShift admits pods through security context constraints (SCCs). The default for a service account, `restricted-v2`, assigns each pod a user ID from the namespace's own range and refuses a pod that asks for a fixed one, so the governing Deployment, which runs as user 10001 (chapter 20), is refused under it. Grant the governor's service account the `nonroot-v2` constraint, which admits any non-root user ID and keeps every other restriction of `restricted-v2`:

```
oc new-project omni-compass
oc apply -f deploy/install/omni-compass.yaml
oc adm policy add-scc-to-user nonroot-v2 -z omni-compass -n omni-compa
    ↪ ss
oc -n omni-compass get pod -l app=omni-compass -o jsonpath='{.items[0]
    ↪ .metadata.annotations.openshift\.io/scc}'

# prints: nonroot-v2
```

No other constraint is needed for the governor on a cluster: it is not privileged, mounts no host path and adds no capability. (The card governor of section 22.7 is a privileged agent on each graphics node and is a separate decision for the cluster's security owner.)

**Machine sets and the cluster autoscaler.** On an installer-provisioned OpenShift cluster the machines are made by the Machine API from MachineSets, and the cluster autoscaler is configured by a `ClusterAutoscaler` resource and one `MachineAutoscaler` per MachineSet. Native is that autoscaler. The governed machines are those of the MachineSets whose node template carries the label `omni-role: work`:

```
# in the MachineSet: the label every node it makes will carry
spec:
  template:
    spec:
      metadata:
        labels:

          omni-role: work
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

The governor itself runs on the infrastructure or control-plane machines, never on a machine it may park: set its `nodeSelector` to the infrastructure label the cluster uses (for example `node-role.kubernetes.io/infra: ""`) and add the matching toleration, in place of the manifest's `omni-role: system`.

**`oc` and `kubectl`.** The governor's image carries `kubectl`, which works against OpenShift's API unchanged. Commands in this manual written with `kubectl` work with `oc` in their place.

## 26.4 VMware vSphere, Rancher and Cluster API

Where the cluster's machines are virtual machines on a private cloud and a cluster autoscaler makes and deletes them (VMware's Kubernetes on vSphere through Cluster API, Rancher's RKE2 with its node-pool autoscaling, OpenStack's Magnum, or any Cluster API provider with the cluster autoscaler's Cluster API integration), the cloud mode of chapter 20 applies as written: the work pools are labelled `omni-role=work`, the system pool carries the governor, and native is the cluster autoscaler. What changes is the price. A private cloud's bill is the company's own internal charge (chargeback or showback) per machine-hour, or, where there is none, the power and the licences per host; the receipt's machine-hour row is the measurement, and finance prices it.

Where the private cloud licenses its hypervisor per host and hosts are not released when virtual machines are deleted, deleting a virtual machine saves nothing until the platform consolidates hosts. Read the platform's own consolidation behaviour (for vSphere, Distributed Power Management) before claiming a saving from machine-hours alone.

## 26.5 Clusters with no route to the internet

An air-gapped cluster pulls nothing from outside, so every artifact the installation needs is carried in:

| Artifact | How it is carried in |
|---|---|
| The verified copy | the repository archive, its manifest and its seal, verified on a connected machine (`python3 verify.py`) and again inside (`python3 verify.py` and `python3 tools/seal.py --check` need no network) |
| The base images | `python:3.12-slim` and `registry.k8s.io/kubectl` (the shipped Dockerfile's two bases), pulled on a connected machine, pinned by digest, saved (`docker save`), carried in and pushed to the internal registry |
| The Python packages | the wheels for `requirements.txt` downloaded on a connected machine (`pip download -r requirements.txt -d wheels`) and installed inside from that directory (`pip install --no-index --find-links wheels -r requirements.txt`) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Artifact | How it is carried in |
|---|---|
| `jq` for the cloud image | the distribution's package carried in with the base image, or a static `jq` binary copied into the image |
| The built images | built inside from the carried-in bases, with the `FROM` lines pointed at the internal registry |

The governor needs no network beyond the cluster's own API server and, for the probe, the service it measures. It sends nothing anywhere: no telemetry, no licence check, no update check.

## 26.6 Clusters at the edge

A cluster of a few machines at a factory, a store or a cell tower has a floor problem: the governor never keeps fewer than two machines in service, and a three-machine cluster can park at most one. The gain at the edge is usually not machines but work and power: the max on the pods and the processor, and, where a site has a power limit, the site's watts read into the engine (`--power-cmd`, `--site-limit-w`, level 7). The OFF switch, the watchdog and the audit are the same; the audit should be shipped off the site, because an edge site's disks are the first thing lost.

## 26.7 Own machines: what to check before claiming a gain

| Claim | What backs it |
|---|---|
| More work from the same machines | the paired receipt's work rows (requests served, jobs completed) reading GAIN, with response time and failures STEADY or GAIN, on the same machines in both arms |
| A claim of the same work with fewer machines in service | the machine rows reading GAIN at the tune-down, with every service row STEADY or GAIN |
| Power | a meter of class P named in the receipt; the power the max took shown as power given beside the work it bought, never called a loss |
| A dollar figure | the company's own price per kilowatt-hour or per machine-hour, applied by finance to the measured rows, never by the governor's receipt |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 27

# Hardening and the Supply Chain

A governor that can write to production is a privileged workload, however narrow its grant. This chapter takes the installation of chapter 20 from correct to hardened: the image built, described, scanned and signed; only that image admitted; the governor's writes narrowed below what role-based access control alone can express; its network reduced to the two endpoints it needs; and its every write cross-checked against the API server's own record. Each step is optional in the sense that the governor works without it, and expected in the sense that a security review will ask for it.

## 27.1 What the governor's grant allows, read as an attacker would

Least privilege (chapter 15) limits the governor to reading the cluster and patching nodes and pods. A security reviewer reads that grant by asking what a stolen copy of the governor's service-account token could do, not what the governor does with it:

| With the cloud-mode grant, a stolen token could | Consequence | The control that closes it |
|---|---|---|
| read nodes, pods, deployments, HPAs and config maps cluster-wide | the cluster's shape is disclosed | the token is short-lived and projected; the namespace admits only the signed image (section 27.3); no person or other workload uses the service account |
| taint a node `NoSchedule`, or cordon it | new pods cannot land there; enough of them and new pods wait | the admission policy of section 27.4 admits only the governor's own `omnicompass.io/idle` taint with the `PreferNoSchedule` effect, and no cordon |
| change a node's labels or annotations (for example, disable the cluster autoscaler's scale-down on it) | scheduling or the bill is distorted | the admission policy of section 27.4 |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| With the cloud-mode grant, a stolen token could | Consequence | The control that closes it |
|---|---|---|
| change a pod's labels | the pod leaves its Service's selector and stops receiving traffic | the admission policy of section 27.4 admits, on a pod, only the `controller.kubernetes.io/pod-deletion-cost` annotation |
| change a mutable field of a pod's spec (its container image, its active deadline, its tolerations) | code the company did not ship runs, or pods end | the admission policy of section 27.4 admits no change to a pod's spec |

Role-based access control cannot express the right-hand column: its verbs are whole-object (`patch` on `nodes` is every field of every node). The Kubernetes admission policy language can, and section 27.4 uses it. With it, the governor's token is worth, to an attacker, what it is worth to the governor: the ability to mark a machine "prefer not" and to mark a pod "first to go".

## 27.2 The image: built from the verified copy, described, scanned, signed

1. **Build from the verified copy** (chapter 33), with the base images pinned by digest rather than by tag:

```bash python3 verify.py # VERIFICATION: PASS, on the commit being built docker build -t <registry>/omni-compass:<version> -f deploy/Dockerfile . docker build -t <registry>/omni-compass-cloud:<version> -f deploy/Dockerfile.cloud \

```
  --build-arg BASE=<registry>/omni-compass:<version> .
```

```

Pin `python:3.12-slim` and `registry.k8s.io/kubectl` by digest in a copy of `deploy/Dockerfile` kept in the company's own repository (`FROM python:3.12-slim@sha256:<digest>`), so that a rebuild of the same commit yields the same layers. Tag every image with the engine version and the commit; never deploy `latest`.

2. **Describe it** with a software bill of materials, in SPDX or CycloneDX: `syft <registry>/omni-compass-cloud:<version> -o spdx-json > sbom.spdx.json`. Keep the bill with the installation record.
3. **Scan it** for known vulnerabilities before it is pushed and on a schedule after: `trivy image <registry>/omni-compass-cloud:<version>` or `grype <registry>/omni-compass-cloud:<version>`. The image holds a Python runtime, its packages, `kubectl`, `bash` and `jq`; a finding in any of them is handled under the company's own vulnerability policy, by rebuilding on a patched base, never by editing the engine.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

4. **Sign it** in the registry, so that admission can prove where it came from: with Sigstore's `cosign` (`cosign sign --key <key> <registry>/omni-compass-cloud@sha256:<digest>`, or keyless signing bound to the company's build identity), or with Notation and the cloud's key service (Azure Key Vault, Google Cloud KMS, AWS Signer). Sign the digest, not the tag.
5. **Push to the company's private registry** (Azure Container Registry, Google Artifact Registry, Amazon ECR), with tag immutability turned on, and pull from it only.

## 27.3 Admitting only the signed image, and only the restricted profile

**Only the signed image.** Each provider offers an admission control that refuses an unsigned or wrongly signed image:

| Provider | Admission control for image signatures |
|---|---|
| Azure (AKS) | Image Integrity (Ratify with Azure Policy), verifying Notation signatures |
| Google (GKE) | Binary Authorization, with an attestor bound to the company's signing key |
| Amazon (EKS) | Kyverno's `verifyImages` rule or the Sigstore policy controller, verifying cosign or Notation signatures |
| Any cluster | Kyverno `verifyImages`, the Sigstore policy controller, or Gatekeeper with Ratify |

Scope the rule to the `omni-compass` namespace at least; the governor's pod then cannot be replaced by a pod running an image the company did not build and sign.

**The restricted Pod Security Standard.** The governing Deployment meets the `restricted` profile of the Kubernetes Pod Security Standards as shipped: it runs as a non-root user, forbids privilege escalation, drops every Linux capability, uses the runtime's default seccomp profile, mounts only `emptyDir` volumes (or a persistent volume claim, section 21.4) and has a read-only root filesystem. Enforce the profile on its namespace, so that no later edit can loosen it:

```
kubectl label namespace omni-compass \
  pod-security.kubernetes.io/enforce=restricted pod-security.kubernete
    ↪ s.io/enforce-version=latest
```

On OpenShift the security context constraints of section 26.3 play this part.

## 27.4 Narrowing the writes below the role: an admission policy

A `ValidatingAdmissionPolicy` (Kubernetes 1.30 and later, which every current managed Kubernetes version satisfies) evaluates rules written in the Common Expression Language (CEL) against every write a chosen identity makes, and refuses a write that breaks them, after role-based access control has allowed it. The four policies below bind the governor's service account, and only it, to the writes it makes in a cloud: on a node, its own idle taint and nothing else; on a pod, its own deletion-cost mark and nothing else (a pod's

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

processor is resized through the `pods/resize` subresource, which the pod policy does not match and the grant names apart); on the service's autoscaler, its target, floor and cap and its own records, never what it scales or how; on the service's Deployment, its own records and nothing else.

```
# The governor's writes on machines: its own idle taint (PreferNoSched
    ↪ ule), nothing else.
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: {name: omni-compass-nodes}
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - {apiGroups: [""], apiVersions: ["v1"], operations: ["UPDATE"],
    ↪  resources: ["nodes"]}
  matchConditions:
    - name: the-governor
      expression: "request.userInfo.username == 'system:serviceaccount
    ↪ :omni-compass:omni-compass'"
  validations:
    - expression: >-
        has(object.metadata.labels) == has(oldObject.metadata.labels)
    ↪ &&
        (!has(object.metadata.labels) || object.metadata.labels == old
    ↪ Object.metadata.labels)
      message: "the governor changes no label of a machine"
    - expression: >-
        has(object.metadata.annotations) == has(oldObject.metadata.ann
    ↪ otations) &&
        (!has(object.metadata.annotations) || object.metadata.annotati
    ↪ ons == oldObject.metadata.annotations)
      message: "the governor changes no annotation of a machine"
    - expression: >-
        (has(object.spec.unschedulable) && object.spec.unschedulable)
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ==
        (has(oldObject.spec.unschedulable) && oldObject.spec.unschedul
    ↪ able)
      message: "the governor never cordons or uncordons a machine"
    - expression: >-
        (!has(object.spec.taints) || object.spec.taints.all(t,

           (t.key == 'omnicompass.io/idle' && t.effect == 'PreferNoSch
    ↪ edule') ||
           (has(oldObject.spec.taints) && oldObject.spec.taints.exists
    ↪ (o, o.key == t.key && o.effect == t.effect)))) &&
        (!has(oldObject.spec.taints) || oldObject.spec.taints.all(o, o
    ↪ .key == 'omnicompass.io/idle' ||

           (has(object.spec.taints) && object.spec.taints.exists(t, t.
    ↪ key == o.key && t.effect == o.effect))))
      message: "the governor adds or removes no taint but its own omni
    ↪ compass.io/idle:PreferNoSchedule"
---
apiVersion: admissionregistration.k8s.io/v1

kind: ValidatingAdmissionPolicyBinding
metadata: {name: omni-compass-nodes}
spec: {policyName: omni-compass-nodes, validationActions: [Deny]}
---
# The governor's writes on pods: its own deletion-cost mark, nothing e
    ↪ lse.

apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: {name: omni-compass-pods}
spec:
  failurePolicy: Fail
  matchConstraints:

    resourceRules:
      - {apiGroups: [""], apiVersions: ["v1"], operations: ["UPDATE"],
    ↪  resources: ["pods"]}
  matchConditions:
    - name: the-governor
      expression: "request.userInfo.username == 'system:serviceaccount
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ :omni-compass:omni-compass'"
  validations:
    - expression: "object.spec == oldObject.spec"
      message: "the governor changes nothing in a pod's spec"
    - expression: >-
        has(object.metadata.labels) == has(oldObject.metadata.labels)
    ↪ &&
        (!has(object.metadata.labels) || object.metadata.labels == old
    ↪ Object.metadata.labels)
      message: "the governor changes no label of a pod"
    - expression: >-
        (!has(object.metadata.annotations) || object.metadata.annotati
    ↪ ons.all(k,
            k == 'controller.kubernetes.io/pod-deletion-cost' ||
            (has(oldObject.metadata.annotations) && k in oldObject.meta
    ↪ data.annotations &&
             oldObject.metadata.annotations[k] == object.metadata.annot
    ↪ ations[k]))) &&
        (!has(oldObject.metadata.annotations) || oldObject.metadata.an
    ↪ notations.all(k,
            k == 'controller.kubernetes.io/pod-deletion-cost' ||
            (has(object.metadata.annotations) && k in object.metadata.a
    ↪ nnotations)))
      message: "the governor changes no annotation of a pod but contro
    ↪ ller.kubernetes.io/pod-deletion-cost"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata: {name: omni-compass-pods}
spec: {policyName: omni-compass-pods, validationActions: [Deny]}
---
# The governor's writes on the service's autoscaler: its target, floor
    ↪  and cap, and its own records, nothing else.
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: {name: omni-compass-autoscalers}
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - {apiGroups: ["autoscaling"], apiVersions: ["v2"], operations:
    ↪ ["UPDATE"], resources: ["horizontalpodautoscalers"]}

  matchConditions:
    - name: the-governor
      expression: "request.userInfo.username == 'system:serviceaccount
    ↪ :omni-compass:omni-compass'"
  validations:
    - expression: "object.spec.scaleTargetRef == oldObject.spec.scaleT

    ↪ argetRef"
      message: "the governor never changes what an autoscaler scales"
    - expression: >-
        has(object.spec.behavior) == has(oldObject.spec.behavior) &&
        (!has(object.spec.behavior) || object.spec.behavior == oldObje
    ↪ ct.spec.behavior)

      message: "the governor never changes how an autoscaler scales"
    - expression: >-
        has(object.spec.metrics) == has(oldObject.spec.metrics) &&
        (!has(object.spec.metrics) || object.spec.metrics.size() == ol
    ↪ dObject.spec.metrics.size())
      message: "the governor adds or removes no metric (it moves a tar

    ↪ get)"
    - expression: >-
        has(object.metadata.labels) == has(oldObject.metadata.labels)
    ↪ &&
        (!has(object.metadata.labels) || object.metadata.labels == old
    ↪ Object.metadata.labels)

      message: "the governor changes no label of an autoscaler"
    - expression: >-
        (!has(object.metadata.annotations) || object.metadata.annotati
    ↪ ons.all(k, k.startsWith('omnicompass.io/') ||
            (has(oldObject.metadata.annotations) && k in oldObject.meta
    ↪ data.annotations &&
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
            oldObject.metadata.annotations[k] == object.metadata.annot
    ↪ ations[k]))) &&
        (!has(oldObject.metadata.annotations) || oldObject.metadata.an
    ↪ notations.all(k, k.startsWith('omnicompass.io/') ||
            (has(object.metadata.annotations) && k in object.metadata.a
    ↪ nnotations)))

      message: "the governor changes no annotation of an autoscaler bu
    ↪ t its own omnicompass.io/ records"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata: {name: omni-compass-autoscalers}

spec: {policyName: omni-compass-autoscalers, validationActions: [Deny]
    ↪ }
---
# The governor's writes on the service's Deployment: its own records (
    ↪ the pods' original processor limit), nothing else.
apiVersion: admissionregistration.k8s.io/v1

kind: ValidatingAdmissionPolicy
metadata: {name: omni-compass-deployments}
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:

      - {apiGroups: ["apps"], apiVersions: ["v1"], operations: ["UPDAT
    ↪ E"], resources: ["deployments"]}
  matchConditions:
    - name: the-governor
      expression: "request.userInfo.username == 'system:serviceaccount
    ↪ :omni-compass:omni-compass'"

  validations:
    - expression: "object.spec == oldObject.spec"
      message: "the governor changes nothing in a Deployment's spec"
    - expression: >-
        has(object.metadata.labels) == has(oldObject.metadata.labels)
    ↪ &&
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        (!has(object.metadata.labels) || object.metadata.labels == old
    ↪ Object.metadata.labels)
      message: "the governor changes no label of a Deployment"
    - expression: >-
        (!has(object.metadata.annotations) || object.metadata.annotati
    ↪ ons.all(k, k.startsWith('omnicompass.io/') ||

           (has(oldObject.metadata.annotations) && k in oldObject.meta
    ↪ data.annotations &&
            oldObject.metadata.annotations[k] == object.metadata.annot
    ↪ ations[k]))) &&
        (!has(oldObject.metadata.annotations) || oldObject.metadata.an
    ↪ notations.all(k, k.startsWith('omnicompass.io/') ||

           (has(object.metadata.annotations) && k in object.metadata.a
    ↪ nnotations)))
      message: "the governor changes no annotation of a Deployment but
    ↪  its own omnicompass.io/ records"
---
apiVersion: admissionregistration.k8s.io/v1

kind: ValidatingAdmissionPolicyBinding
metadata: {name: omni-compass-deployments}
spec: {policyName: omni-compass-deployments, validationActions: [Deny]
    ↪ }
```

**Prove the policies before the governor relies on them.** A policy is proved the way the grant is (section 15.4): by asking the API server, as the governor, to do what the policy forbids and what it allows, and reading the answers. The implementer's own account needs the right to impersonate the service account, which a cluster administrator has:

```
SA=system:serviceaccount:omni-compass:omni-compass; N=<a work machine>
    ↪ ; P=<a pod of the service>; NS=<its namespace>
kubectl --as=$SA label node $N omni-test=1
    ↪        # refused: changes no label of a machine
kubectl --as=$SA cordon $N
    ↪        # refused: never cordons
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
kubectl --as=$SA taint node $N omni-test=1:NoSchedule
    ↪        # refused: no taint but its own
kubectl --as=$SA taint node $N omnicompass.io/idle=true:PreferNoSchedu
    ↪ le     # admitted
kubectl --as=$SA taint node $N omnicompass.io/idle:PreferNoSchedule-
    ↪        # admitted

kubectl --as=$SA label pod $P -n $NS omni-test=1
    ↪        # refused: changes no label of a pod
kubectl --as=$SA annotate pod $P -n $NS controller.kubernetes.io/pod-d
    ↪ eletion-cost=-1000 --overwrite   # admitted
kubectl --as=$SA annotate pod $P -n $NS controller.kubernetes.io/pod-d
    ↪ eletion-cost-                    # admitted

D=<the service's Deployment, its autoscaler of the same name>
kubectl --as=$SA patch hpa $D -n $NS --type=merge -p '{"spec":{"scaleT
    ↪ argetRef":{"name":"omni-test"}}}' # refused: never what it scale
    ↪ s
kubectl --as=$SA annotate hpa $D -n $NS omnicompass.io/drill=1
    ↪         # admitted

kubectl --as=$SA annotate hpa $D -n $NS omnicompass.io/drill-
    ↪         # admitted
kubectl --as=$SA label deployment $D -n $NS omni-test=1
    ↪         # refused: changes no label of a Deployment
kubectl --as=$SA annotate deployment $D -n $NS omnicompass.io/drill=1
    ↪         # admitted

kubectl --as=$SA annotate deployment $D -n $NS omnicompass.io/drill-
    ↪         # admitted
```

Each refusal names the policy and its message. Keep the fourteen answers with the installation record beside the grant's proof. Run the drill in a quiet hour: the two admitted taint commands park and wake the machine `$N` for a moment, the two admitted pod annotations mark and unmark one pod, and the four admitted record annotations come and go on the service's autoscaler and Deployment.

**What `failurePolicy: Fail` means here.** If the policy itself cannot be evaluated, the governor's write is refused, never admitted. A refused write is recorded as `refused` in the governor's audit (section 21.4); a refused park leaves a machine in service, and a refused wake leaves a machine parked, which never blocks a pod. The policy can therefore stop the governor from saving, never make it harm.

A cloud needs nothing the policies forbid: the shipped machine script parks and wakes machines with the idle taint alone, marks pods with the deletion-cost annotation alone, and never cordons or uncordons a machine

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

(section 29.1); the governor moves an autoscaler's target, floor and cap and records the owner's own settings in `omnicompass.io/` annotations on the autoscaler and the Deployment, and resizes a pod's processor through `pods/resize` alone.

## 27.5 The network: two endpoints

The governor's pod talks to two endpoints: the cluster's API server (through `kubectl`, from the controller and the watchdog) and the service it measures (from the probe). It receives no connection at all. A `NetworkPolicy` can hold it to exactly that, where the cluster's network plugin enforces network policy (Azure Network Policy Manager or Cilium on AKS, GKE Dataplane V2 or Calico on GKE, the Amazon VPC CNI's network policy agent or Calico on EKS):

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: omni-compass, namespace: omni-compass}
spec:
  podSelector: {matchLabels: {app: omni-compass}}
  policyTypes: [Ingress, Egress]

  ingress: []                                    # nothing connects to
    ↪ the governor
  egress:
    - to:                                        # the cluster's DNS
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.nam
    ↪ e: kube-system}}

          podSelector: {matchLabels: {k8s-app: kube-dns}}
      ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
    - to:                                        # the API server: its
    ↪ address from `kubectl get endpoints kubernetes`
        - ipBlock: {cidr: <api-server-address>/32}
      ports: [{protocol: TCP, port: 443}]

    - to:                                        # the measured service
    ↪ , for the probe
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.nam
    ↪ e: <namespace>}}
          podSelector: {matchLabels: {<the service's selector>}}
      ports: [{protocol: TCP, port: <the service's target port>}]
```

Three details decide whether this policy works on a given cluster: the API server's address, which on a managed cluster is outside the pod network and is read from `kubectl get endpoints kubernetes`

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

`-n default` (and some providers translate it, so test from the pod); the DNS pods' label, which differs by provider (`k8s-app: kube-dns` on all three as of this writing); and the measured service's pods' labels and target port. Prove the policy as the grant is proved: from the governor's pod, the API server and the service answer, and nothing else does (`kubectl -n omni-compass exec deploy/omni-compass -c probe -- python -c "import urllib.request; urllib.request.urlopen('https://example.com', timeout=5)"` must fail).

## 27.6 The API server's own record of every write

The governor's audit records every write it makes; the API server's audit log records every write it receives. Read together, they prove that the governor wrote nothing it did not record, and that nothing wrote as the governor that was not the governor. Turn the control plane's audit log on, and filter it by the service account `system:serviceaccount:omni-compass:omni-compass`:

| Provider | The API server's audit log |
|---|---|
| Azure (AKS) | diagnostic settings on the cluster, category `kube-audit` (or `kube-audit-admin` for writes only), to a Log Analytics workspace |
| Google (GKE) | Cloud Audit Logs, Admin Activity (always on) for writes, in Cloud Logging |
| Amazon (EKS) | control plane logging, log type `audit`, to CloudWatch Logs |

Every `patch` on a node or a pod by the service account must match a `write` record in the governor's audit within the same few seconds, with the same object; every `write` record that is not marked `"dry_run": true` must match a patch. A patch by the service account with no `write` record behind it is a security incident: the token was used by something other than the governor.

## 27.7 The token, and no other secret

The governor holds no secret of any kind: no cloud credential (section 15.5), no password, no key; its role reads no `Secret` object. Its only credential is the Kubernetes service-account token that the kubelet projects into its pod, short-lived and rotated by the kubelet, bound to the pod's lifetime. No person and no other workload uses the `omni-compass` service account; a request by it from anywhere but the governor's pod is visible in the API server's audit log (section 27.6) by its source address and its user agent.

## 27.8 The hardening record

| Item | Evidence kept |
|---|---|
| The image | its digest, its software bill of materials, its last scan, its signature |
| Admission | the signature rule and the namespace's Pod Security label |
| The writes | the two admission policies, and the eight answers of their proof |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Item | Evidence kept |
|---|---|
| The network | the network policy and the proof that only the two endpoints answer |
| The record | the API server's audit log turned on, and one quarter's cross-check against the governor's audit |

CHAPTER 28

# Rolling Out Across a Fleet: Many Clusters, Many Services, GitOps

Section 21.7 sets the rule for a fleet: one governor per cluster, rolled out cluster by cluster through the phases of section 14.3, with a paired receipt on at least one cluster of each kind. This chapter is the practice behind the rule: how the governor's configuration is kept as code, what differs from cluster to cluster and what must not, how a continuous-delivery controller interacts with the OFF switch, and how a fleet of audits is read as one.

## 28.1 The unit of rollout

| Unit | Rule |
|---|---|
| The cluster | one Omni-Compass, one feed, one line, one set of pools it moves; never one Omni-Compass for several clusters, never two on one cluster |
| The kind | clusters on the same provider, machine families, autoscaler profile and service class; one paired receipt per kind decides whether the kind may govern |
| The engine version | one engine version across the fleet at a time, apart from the wave that is upgrading; a receipt is read only against the version it was made on (chapter 13) |
| The wave | an ordered group of clusters that moves from one phase to the next together, after the wave before it has run a full cycle of its load without a fault |

A conventional order of waves is: one non-production cluster of the kind; one production cluster with the smallest blast radius; a tenth of the kind; the rest. Each wave starts at phase 3 (watch), never at phase 6, even when the kind already has a receipt: watching on a new cluster shows that its feed, its line and its pools are what the receipt assumed.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 28.2 What differs from cluster to cluster, and what must not

| Setting | Per cluster | Why |
|---|---|---|
| `TARGET_URL` (the probe), or the adapter's source | yes | the service's address in that cluster |
| `--slo-ms` | per service, usually the same across clusters | the service owner's line; a cluster with a different line is a different kind |
| `--max-nodes` | yes | the sum of that cluster's work pools' autoscaler maxima |
| `--min-nodes` and `MIN_NODES` | per kind | the floor, at least the zones the service needs serving |
| `WORKER_SEL` | per kind | the label of the governed pools, `omni-role=work` by convention |
| The image digest | **never** | one engine version, one build, one signature |
| `--mode`, `--law`, `--dial`, `--sensed`, `--cap-deployments`, `--interval`, `--floor-interval`, `--max-node-step`, the node commands | **never** | the cloud's settings as section 20.3 writes them; a cluster that differs runs a changed tune and its receipt is its own |

## 28.3 The configuration as code

The repository's manifests are the starting point, not the company's deployable unit. Two of them cannot be applied together as written: `deploy/install/omni-compass.yaml` carries the identity (the namespace, the service account, the watch role and its binding) and the watch Deployment, and `deploy/install/omni-compass-cloud.yaml` carries the governing Deployment under the same name. In the company's own configuration repository, split the first file into its identity half and its watch half, and keep a base for each phase:

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
omni-compass/
  base/identity/        namespace, service account, watch role and bin
    ↪ ding (from deploy/install/omni-compass.yaml)
  base/watch/           the watch Deployment (from deploy/install/omni
    ↪ -compass.yaml)
  base/govern/          rbac-cloud.yaml, omni-compass-cloud.yaml, the

    ↪ network policy and admission policies (chapter 27)
  overlays/<cluster>/   kustomization.yaml: identity + watch, or ident
    ↪ ity + govern, and this cluster's settings
```

An overlay for one cluster in phase 6:

```
# overlays/aks-eastus-prod-1/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [../../base/identity, ../../base/govern]
images:
  - name: omni-compass-cloud             # the image name in the mani

    ↪ fest, without its tag
    newName: <registry>/omni-compass-cloud
    digest: sha256:<the signed digest>
patches:
  - target: {kind: Deployment, name: omni-compass}
    patch: |-

      - op: replace
        path: /spec/template/spec/containers/1/env/0/value
        value: http://checkout.shop.svc.cluster.local/
      - op: replace
        path: /spec/template/spec/containers/0/args
        value: ["--mode", "nodepool", "--law", "compass", "--dial", "s

    ↪ ave", "--sensed", "none/none",
                "--active-nodes-only", "--interval", "15", "--floor-in
    ↪ terval", "5",
                "--min-nodes", "3", "--max-nodes", "24", "--max-node-s
    ↪ tep", "1",
                "--node-scale-cmd", "bash /app/scripts/kind_nodepool.s
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ h {n}",
                "--node-restore-cmd", "bash /app/scripts/kind_nodepool
    ↪ .sh 100000",
                "--latency-file", "/feed/latency.csv", "--slo-ms", "40
    ↪ 0",
                "--closure", "/app/law/closure.json",
                "--audit", "/var/log/omni/audit.jsonl", "--kill-file",
    ↪  "/tmp/omni.kill"]
```

The patch replaces the governor's whole command line rather than single arguments, so that the line in Git is the line that runs, readable in one place; its absence of `--dry-run` is the change that the phase-6 change record approves (section 24.1). The containers are addressed by position (0 the governor, 1 the probe, 2 the watchdog), as the manifest orders them; `MIN_NODES` in the governor's and the watchdog's environment must be patched to the same floor as `--min-nodes`.

## 28.4 Continuous delivery and the OFF switch

A continuous-delivery controller that keeps the cluster equal to Git (Argo CD with automated sync and self-heal, Flux with its default reconciliation) undoes a change made directly in the cluster. Two of the OFF switch's forms (section 20.3) are such changes, and the holder must know what the controller will do with them:

| OFF form | What a self-healing controller does | What the installation does about it |
|---|---|---|
| Scale to zero (`kubectl scale ... --replicas=0`) | puts the replica count back to Git's 1 within its reconciliation interval, and the governor starts again | either the OFF switch of record is a commit to Git (`replicas: 0`), or the controller is told to leave the replica count alone: in Argo CD an `ignoreDifferences` entry on the Application for `/spec/replicas` of the `omni-compass` Deployment; in Flux, `flux suspend kustomization <name>` as the first step of the drill |
| The kill file | nothing: the file is in the pod's `/tmp`, which Git does not describe | none needed; but a pod replaced for any reason, including a sync that changes the pod template, starts ON |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| OFF form | What a self-healing controller does | What the installation does about it |
|---|---|---|
| Revoke the grant (`kubectl delete clusterrolebinding omni-compass-machines`) | applies the binding again from Git | revoke in Git, or suspend the controller for the binding first |

Write the chosen form into the runbook and drill it as it is written (section 21.3). A drill on a GitOps cluster that scales to zero and then watches the controller start the governor again has found the fault this section exists to prevent.

**Other reconcilers of the same objects.** The governor writes two things: a taint on a node and an annotation on a pod. A tool that reconciles node taints to a declared list (some node-management controllers, some provider settings applied to a pool) removes the governor's `omnicompass.io/idle` taint, and a tool that reconciles pod annotations would remove the deletion-cost mark; either is a second writer of the governor's setting (section 21.8) and shows as a `proprioception` drift that does not close. Exclude the key from that tool's reconciliation, or leave that pool ungoverned. Pods are created by their controllers, not by Git, so a continuous-delivery controller never sees the deletion-cost mark.

## 28.5 Many services on one cluster

Omni-Compass's machine lever moves machines, and machines are shared. Where several services run on the pools it moves, its single feed must speak for all of them (section 19.6): the feed of the service most sensitive to a lost machine, or an adapter that writes the worst of the services' own percentiles against their own lines. Three practical rules follow:

1. **Govern a pool, not a service.** Label `omni-role=work` on the pools whose machines the feed speaks for, and leave pools that serve other services unlabelled. A pool dedicated to one service (a node pool with a taint the service tolerates) is the simplest case and the cleanest receipt.
2. **One line per pool.** If two services on the same pool have lines that cannot be scaled to one, they are not supervised together: split the pool, or hold it to the stricter line alone.
3. **Batch and stateful work.** Pods that the cluster autoscaler will not move (section 25.4) pin their machines; the governor may park such a machine, but it empties only when those pods end. A pool that carries long jobs and serving pods together gives the governor little to give back, honestly; the receipt shows it as STEADY on the bill.

## 28.6 Regions, zones and failover

Each region's clusters are separate units. A governor in one region never reads another region's feed, and a regional failover (traffic moved from one region's clusters to another's) is, to each governor, a change in its own service's load: the receiving region's governor sees its line approach and wakes what it parked; the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

sending region's governor sees the load fall and, after its trials, parks what the trough allows. Two settings deserve attention in a region that must be able to absorb another's traffic at once:

- **The floor.** A region that must carry a failover without waiting for the cloud to create machines sets its floor (`--min-nodes`) to the machines the failover needs at its first minute, not to the trough's need; the bill's room then starts above that floor (section 25.2).
- **The zones.** The governor does not read zones, and the machine script chooses which machines to park by the work on them. A service that needs a minimum number of serving machines in every zone keeps that minimum through its own pods' topology spread constraints and a floor of at least that number times the zones; the `PreferNoSchedule` taint never prevents a pod from landing on a parked machine in the zone it needs.

## 28.7 Reading a fleet's audits as one

Every governor writes its own audit (section 21.4); a fleet's audits become one record when the log collector adds the cluster's name to every line on its way to the log store (Fluent Bit's Kubernetes filter, the Azure Monitor agent, Google Cloud's logging agent and the CloudWatch agent each attach the cluster's identity, or can be configured to). Four fleet views answer most questions:

| View | From | Shows |
|---|---|---|
| Machines parked, by cluster and hour | `write` records with `why` = `node pool size` | where and when the fleet gives machines back |
| Trials refused or abandoned, by cluster | `verdict` records | clusters whose feed is thin or whose service is short of machines |
| Blind and error decisions, by cluster | `authority` records' `senses`, and `error` records | clusters with a feed or API problem |
| Drift that does not close, by cluster | `authority` records' `proprioception` | clusters with a second writer |

The fleet's bill is read from the provider's own billing records by cluster and pool (section 23.5), never summed from receipts: a receipt is a controlled comparison on a day, and a bill is the fleet's actual cost over a period.

## 28.8 Upgrading a fleet

An engine upgrade moves through the fleet in the same waves as the first installation (section 21.6 for one cluster): each wave watches on the new version for a full cycle of its load, a paired receipt on the new version decides each kind, and only then does the wave govern on it. During the upgrade the fleet runs two versions, and every audit line, dashboard and receipt names the version it came from (`python3 tools/omni_version.py` on the build, the image tag in the cluster). The rollback of a wave is the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

rollback of one cluster repeated (section 21.6), safe at any moment because the governor carries no state between processes beyond the marks it reads back from the cluster.

CHAPTER 29

# Failure Modes and the Drill Book

Every component of an installation fails eventually: the governor's process, its pod, the machine under it, its feed, the API server it talks to, the service it measures, and the other controllers that share the cluster with it. This chapter lists each failure an implementer should expect, what the governor does when it happens, what the audit and the cluster show, and what the operator does. Section 29.2 turns the most important ones into drills, each with the command that causes the failure on purpose, so that the first time the on-call engineer sees it is not the night it happens.

The design rule behind every row is the same, and it is worth stating once: **a failure of the governor never leaves a pod without a place to run.** A parked machine carries a `PreferNoSchedule` taint, which the scheduler treats as a preference and never as a refusal, so the worst a dead or blind governor can leave behind is machines that are still in the cluster, still Ready and still usable, with a mark asking new pods to go elsewhere first.

## 29.1 What fails, what the governor does, and what you do

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| The governor's process is killed (out of memory, a signal) | nothing; the watchdog in the same pod finds the process gone within 10 seconds and runs the governor's recorded restore, waking every parked machine and removing its marks; the kubelet restarts the governor's container | the watchdog's `handed back` record; the container's restart count rises; the restarted governor's first decisions | find why it died (`kubectl describe pod`, the container's last state); an out-of-memory kill means the limit is too low for the cluster's size |
| The governor hangs | its lease stops being renewed; after five decision periods (300 seconds at a 60 s beat, 50 | the watchdog's record with `hung: true` | as above |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| | at the factory's 10 s on site) the watchdog kills it and runs the restore | | |
| The governor's pod is deleted or evicted gracefully (a drain of the system pool, an upgrade, `kubectl delete pod`) | on `SIGTERM` it hands every setting back, reads each back and exits; the Deployment starts a new pod, which starts ON | the audit record `master switch: OFF; every setting handed back; exiting`, then a new pod's first decisions | nothing, unless the new pod cannot be placed |
| The machine under the governor fails abruptly | nothing can act for it at that moment; the parked machines stay parked, which blocks no pod; the Deployment starts a new pod on another system machine, which begins from the cluster as it finds it | the pod rescheduled; if the system pool has one machine, the pod waits for the provider's repair | run the system pool on at least two machines in production, so the governor's pod always has somewhere to go |
| The feed stops (the probe's container dies, the adapter stops writing) | after twice its window with no new row the governor reads the feed as blind: it gives nothing back, treats the reading as past its wall, and wakes a parked machine each decision while the sense stays blind | `authority` records whose `senses.blind` names `latency`, with a growing `latency_age_s`; the probe container's restarts | restore the feed (section 19.3 on a probe restart); the governor resumes on its own |
| The service stops answering (every request fails) | no served row means no reading: blind, as above | as above, and the service's own alert | the service's own incident; the governor has already stopped giving back and is waking what it parked |
| The feed lies (a reading that does not track the service) | the governor cannot know; it acts on the reading it is given | the dashboard's two response-time lines disagree (section 23.3) | turn the governor OFF until the feed is fixed; this is why the feed is |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| | | | checked before the governor watches (section 19.5) |
| The API server is unreachable or throttling | each decision that cannot read the cluster writes an `error` record (with the count of consecutive failures) and changes nothing; the fast checks record their own errors | `error` records; the provider's own control-plane status | nothing on the governor's side: it resumes when the API answers; parked machines stay usable meanwhile |
| A write is refused (the role, an admission policy, a conflict) | the command's failure is recorded as `refused` with its exit code; with `--max-nodes` set, as the cloud manifest sets it, the governor's top is unchanged, and it issues the count again at its next decision while the machines in service differ from it | `refused` records | read the reason; a policy refusal of a write the governor needs means the policy is wrong (chapter 27); a role refusal means the grant is missing (chapter 15) |
| A person cordons a governed machine (maintenance) | the machine script leaves a machine someone else cordoned alone: it never uncordons it, never counts it in service or parked, and removes its own idle mark from it, if any | the machine `SchedulingDisabled`; a `proprioception` drift while the cordoned machine still carries work | none needed; for a long maintenance, also remove the machine's `omni-role=work` label so the governor no longer sees it at all, and put the label back afterwards |
| The provider upgrades the cluster or a node pool (cordon, drain, surge machines) | as for a cordon; surge machines join open and are counted | the upgrade's own events | scale the governor to zero before the upgrade starts (its hand-back wakes everything) and back to one when the upgrade completes, so the upgrade runs exactly as it |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| | | | does without the governor |
| The cluster autoscaler deletes a parked, empty machine | nothing; that is the saving (section 25.1) | the machine gone; the next decision counts one fewer | nothing |
| The cluster autoscaler adds machines | they join open; the governor counts them at its next read | new machines | nothing |
| A spot or preemptible machine is reclaimed | as a machine failure; parked or open, it is gone | the provider's eviction notice | nothing on the governor's side |
| A descheduler evicts pods from parked machines | nothing; the descheduler is a second actor on the same machines | pods moved off machines with the `omnicompass.io/idle` taint | exclude the taint: the descheduler's `RemovePodsViolatingNodeTaints` plugin acts on `PreferNoSchedule` taints only when told to, and takes an `excludedTaints` list |
| Someone else removes the governor's taint, or uncordons a parked machine | the machine counts as open at the next decision; the governor may park it again | a `proprioception` drift; machines parked and unparked repeatedly | find the other writer (section 21.8); until there is one writer, turn the governor OFF |

## 29.2 The drills

Each drill is run first on a non-production cluster, then in production in a quiet hour, by the people who will handle the failure for real, with the result recorded as the OFF switch drill is (section 21.3). The pass condition of every drill is the same: no pod waited for a place, no alert minute was caused, every setting was back at the operator's within the stated time, and the audit and the cluster show what this chapter says they show.

**Drill 1: the governor's process killed outright.** The watchdog's hand-back is the lease (section 21.2), and this is its test. From the watchdog's container, which shares the governor's process namespace, send `SIGKILL` to every governor the lease registry lists:

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
kubectl -n omni-compass exec deploy/omni-compass -c watchdog -- python
    ↪  -c "
import glob, json, os, signal
for f in glob.glob('/tmp/omni-compass/running/*.json'):
    os.kill(json.load(open(f))['pid'], signal.SIGKILL)"
kubectl -n omni-compass logs deploy/omni-compass -c watchdog --tail=5
    ↪      # the 'handed back' record, ok: true
kubectl get nodes -l omni-role=work -o custom-columns=NAME:.metadata.n
    ↪ ame,TAINTS:.spec.taints[*].key
```

Pass: within 30 seconds of the kill, no `omnicompass.io/idle` taint remains, the watchdog's record reads `ok: true`, and the governor's container has restarted and is deciding again.

**Drill 2: the pod deleted.** `kubectl -n omni-compass delete pod -l app=omni-compass`. Pass: the old pod's audit ends with the hand-back; every parked machine is woken before it exits (within the 60-second grace period); the new pod is deciding within two minutes.

**Drill 3: the feed stopped.** Pause the probe without touching the service or replacing the pod: from the watchdog's container, which shares the pod's process namespace, stop the probe's process, and later let it continue:

```
PAUSE='import os, signal, sys
for p in os.listdir("/proc"):
    if p.isdigit() and b"latency_probe.py" in open("/proc/%s/cmdline"
    ↪ % p, "rb").read():
        os.kill(int(p), getattr(signal, sys.argv[1]))'
kubectl -n omni-compass exec deploy/omni-compass -c watchdog -- python
    ↪  -c "$PAUSE" SIGSTOP    # the feed stops
# ... three decision periods later ...
kubectl -n omni-compass exec deploy/omni-compass -c watchdog -- python
    ↪  -c "$PAUSE" SIGCONT    # the feed resumes
```

Pass: within three decision periods every `authority` record shows the feed blind; no machine is parked while it is blind; parked machines are woken one per decision; after `SIGCONT` the feed's rows resume on the same clock and the governor reads it again.

**Drill 4: a write refused.** With the admission policies of section 27.4 in place, refuse the governor's next write by binding a policy that denies every node update from its service account for the drill's duration (a copy of `omni-compass-nodes` whose only validation is `false`). Pass: `refused` records appear; no other record changes; the service is untouched; when the drill policy is removed, the next decision's write lands.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Drill 5: a maintenance cordon.** Cordon one parked machine and one open machine (`kubectl cordon <name>`). Pass: the governor's next decisions neither uncordon nor count either of them; the parked one loses its `omnicompass.io/idle` taint; the OFF switch drill run now leaves both cordoned. Uncordon them afterwards.

**Drill 6: the OFF switch under continuous delivery.** On a cluster whose governor is deployed by Argo CD or Flux, run the OFF switch drill exactly as the runbook writes it (section 28.4). Pass: the governor stays off until the holder turns it on, and the delivery controller does not start it again.

## 29.3 What the drill book is for

The drills prove three claims the installation makes to the people who run the cluster: that the governor can always be stopped, that its failure never strands a pod, and that every failure is visible in the audit with its cause. The records of the drills, kept with the installation record (section 15.7), are the evidence a reviewer asks for under "how do you know it fails safe?" (section 24.3), and the quarterly repetition of drills 1, 2 and 6 is part of day two (Appendix J).

CHAPTER 30

# A Complete Installation, Start to Finish

The chapters of this part each take one step in depth. This chapter is the whole path at once, on Azure Kubernetes Service, as one engineer would run it: every command in order, what each should print, and the gate each must pass. The same sequence runs on Google Kubernetes Engine and Amazon EKS with the provider commands of chapters 17 and 18 in place of the Azure ones; every Kubernetes command is the same on all three. The example service is a web shop's checkout, `checkout` in the namespace `shop`, with a line of 400 ms at the 95th percentile, written down by its owner. Every value in angle brackets is the installation's own.

## 30.1 Day 0: the copy, the cluster and the service

```
# The copy (phase 0, chapter 33)
git clone <the licensed repository> omni-compass && cd omni-compass
python3 verify.py                        # ends: VERIFICATION: PASS
python3 tools/omni_version.py            # prints the engine version: r
    ↪ ecord it with the commit
git rev-parse HEAD
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# The cluster (phase 1, section 16.2): a system pool of two machines,
    ↪ one work pool under the autoscaler
RG=shop-prod-rg; NAME=shop-prod; LOC=eastus; SIZE=Standard_D4s_v5
az group create -n $RG -l $LOC -o none
az aks create -g $RG -n $NAME -l $LOC --tier standard --node-count 2 -
    ↪ -node-vm-size Standard_D2s_v5 \
  --nodepool-name system --nodepool-taints CriticalAddonsOnly=true:NoS
    ↪ chedule --nodepool-labels omni-role=system \
  --cluster-autoscaler-profile scale-down-unneeded-time=10m scale-down
    ↪ -delay-after-add=10m scan-interval=10s \
  --generate-ssh-keys -o none
az aks nodepool add -g $RG --cluster-name $NAME -n work0 --mode User -
    ↪ -node-vm-size $SIZE --labels omni-role=work \
  --node-count 3 --enable-cluster-autoscaler --min-count 3 --max-count
    ↪  12 -o none
az aks get-credentials -g $RG -n $NAME
kubectl get nodes -L omni-role                # 2 system, 3 work
```

The system pool has two machines (section 29.1: the governor's pod must always have somewhere to go). The work pool's floor is 3 because the checkout spreads across three zones; its top is 12, the operator's cost ceiling. **Gate:** a test Deployment scaled past the pool and back shows Azure adding and deleting a machine (section 16.2).

The service's own autoscaler is the operator's and stays so: `kubectl -n shop get hpa checkout` shows its target, minimum and maximum, which are written into the installation record (section 15.7).

## 30.2 Day 1: identity, image and watch

```
# Identity (phase 2, chapter 15): the namespace, the service account,
    ↪ the watch role; the restricted profile
kubectl apply -f deploy/install/omni-compass.yaml
kubectl label namespace omni-compass pod-security.kubernetes.io/enforc
    ↪ e=restricted
for verb in get list; do for r in nodes pods horizontalpodautoscalers
    ↪ deployments; do
  echo "$verb $r: $(kubectl auth can-i $verb $r --as=system:serviceacc
    ↪ ount:omni-compass:omni-compass -A)"; done; done
kubectl auth can-i patch nodes --as=system:serviceaccount:omni-compass
    ↪ :omni-compass       # no (not yet granted)
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# The images (section 16.4), built from the verified copy and tagged w
    ↪ ith the version and the commit
ACR=shopprodacr; V=<version>-<commit>
az acr create -g $RG -n $ACR --sku Standard -o none && az aks update -
    ↪ g $RG -n $NAME --attach-acr $ACR -o none
az acr build -r $ACR -t omni-compass:$V -f deploy/Dockerfile .

az acr build -r $ACR -t omni-compass-cloud:$V -f deploy/Dockerfile.clo
    ↪ ud --build-arg BASE=$ACR.azurecr.io/omni-compass:$V .

# Watch (phase 3, section 16.5)
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ ACR.azurecr.io/omni-compass:$V

kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 3 /var/log
    ↪ /omni/audit.jsonl
```

The last command prints `decision` records, one a minute, each with `nodes_observed` and `nodes_recommended`, and no `write` record. **Gate:** at least one full cycle of the checkout's load watched (a week, because the shop's peak is weekly), and the platform owner agrees with what the governor would have done.

## 30.3 Day 2: the feed and the line

The checkout has one representative endpoint, so the probe is the feed (section 19.3). Its URL and the line go into the installation record, signed by the service owner. Before governing, the feed is checked on its own (section 19.5): the probe is run beside the watch Deployment for an hour, and its rows advance every five seconds, its 95th percentile tracks the shop's own monitoring of the same endpoint, and a stopped endpoint shows as failed rows.

At the same time the room is measured (chapter 25): two weeks of the sampling loop of section 25.3, from which the plan records where the room lives (here: the nights and the hours after the weekday peak), the billable room (the shop holds no reservation on this machine family) and the load the paired run will carry: a replay of one recorded day of checkout traffic, compressed no further than the autoscaler's ten-minute profile allows.

## 30.4 Days 3 to 7: the paired run

The paired run (phase 5, section 16.6) builds two identical clusters for each repetition, one native and one with Omni-Compass on top at the max, drives the same recorded day through both, and deletes both. Its receipt is read row by row (section 20.6), and before the installation proceeds it must show: the bill and the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

machine-hours reading GAIN or STEADY; response time, failed requests and alert minutes reading STEADY or GAIN; HPA replicas equal to native's within the noise. A LOSS row stops the installation until its cause is found in the wiring and the run is repeated (section 5.5). The receipt is signed by the service owner and by finance.

## 30.5 Day 8: hardening

```
# The admission policies (section 27.4) and their proof: eight answers
    ↪ , kept with the record
kubectl apply -f omni-compass-policies.yaml
SA=system:serviceaccount:omni-compass:omni-compass
kubectl --as=$SA label node <work machine> omni-test=1          # refu
    ↪ sed by omni-compass-nodes

# ... the remaining seven commands of section 27.4 ...

# The network policy (section 27.5), with this cluster's API server ad
    ↪ dress and the checkout's labels and port
kubectl get endpoints kubernetes -n default -o jsonpath='{.subsets[0].
    ↪ addresses[0].ip}'

kubectl apply -f omni-compass-netpol.yaml

# The API server's own audit log to the workspace the platform team re
    ↪ ads (section 27.6)
az monitor diagnostic-settings create --name aks-audit --resource $(az
    ↪  aks show -g $RG -n $NAME --query id -o tsv) \

  --workspace <log-analytics-workspace-id> --logs '[{"category":"kube-
    ↪ audit-admin","enabled":true}]'
```

The image's signature rule (section 27.3) is applied by the platform team's existing admission control; the cloud image's digest is the one it allows in the `omni-compass` namespace.

## 30.6 Day 9: govern, with the dry run first

The governing manifest is the repository's, with this cluster's settings, a persistent volume for the audit, and the probe pointed at the checkout. Kept in the company's configuration repository as an overlay (section 28.3), its patch for this cluster is:

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# The audit on a persistent volume (section 21.4) instead of the manif
    ↪ est's emptyDir
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: omni-audit, namespace: omni-compass}
spec: {accessModes: [ReadWriteOnce], storageClassName: managed-csi, re
    ↪ sources: {requests: {storage: 5Gi}}}
---
# patch on Deployment omni-compass (JSON 6902)
- {op: replace, path: /spec/template/spec/volumes/0, value: {name: aud
    ↪ it, persistentVolumeClaim: {claimName: omni-audit}}}
- {op: replace, path: /spec/template/spec/containers/1/env/0/value, va
    ↪ lue: "http://checkout.shop.svc.cluster.local/"}
- {op: replace, path: /spec/template/spec/containers/0/env/1/value, va
    ↪ lue: "3"}     # MIN_NODES, the governor
- {op: replace, path: /spec/template/spec/containers/2/env/1/value, va
    ↪ lue: "3"}     # MIN_NODES, the watchdog
```

and the governor's command line, written whole (section 28.3), carries `--min-nodes 3 --max-nodes 12 --slo-ms 400` and, for the first day, `--dry-run`. Then:

```
kubectl apply -f deploy/install/rbac-cloud.yaml                # the
    ↪ cloud's grant (section 15.3), approved in the change record
kubectl -n omni-compass delete deployment omni-compass          # the
    ↪  watch Deployment goes; the governing one replaces it
kubectl apply -k overlays/shop-prod                             # the
    ↪  governing Deployment, with --dry-run

kubectl -n omni-compass get pod -l app=omni-compass             # REA
    ↪ DY 3/3: governor, probe, watchdog
kubectl auth can-i patch nodes --as=system:serviceaccount:omni-compass
    ↪ :omni-compass      # yes
```

A day of the dry run's audit is read: every `write` record is marked `"dry_run": true` and asks for a count between 3 and the machines that exist, one from the last (Appendix J, phase 6). Then the change record's approved commit removes `--dry-run` from the overlay, the delivery controller applies it, and the first real write is read when it comes: a `verdict` record with `step allowed`, then a `write` record with `why: node pool size`, then the next decision's `authority` record, whose `proprioception` shows that the command landed. In the cluster, one work machine now carries the `omnicompass.io/idle:PreferNoSchedule` taint.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Gate:** the OFF switch drilled by its holder in the form the runbook names (section 21.3, section 28.4 for a GitOps cluster): every parked machine woken and every mark removed within 60 seconds; the drill recorded.

## 30.7 Day 10 onward: the first month

| When | What is read | Expected |
|---|---|---|
| Every morning, first week | the dashboard of section 23.3 | machines parked overnight and woken before the morning's rise; the two response-time lines agree; no blind decisions |
| Each weekly peak | alert minutes on the checkout's line | none caused by the governor; any alert minute is traced with section 23.7 to its cause |
| End of week one | the drills of section 29.2, numbers 1, 2, 3 and 5 | each passes as written |
| End of month one | the provider's billing records for the work pool (section 23.5) against the same weeks before installation, with requests served beside them | machine-hours lower in the hours the plan found room (chapter 25); higher only where traffic was higher |
| End of month one | the API server's audit against the governor's (section 27.6) | every patch by the service account matched by a `write` record |
| Every quarter | the OFF switch and drills 1, 2 and 6; the engine version against the receipts kept | as Appendix J, phase 7 |

## 30.8 What the installation record holds at the end

The record of section 15.7, complete: the verification receipt and the engine version and commit; the image digests, the software bill of materials, the scan and the signature; the grant as applied and its proof; the admission policies and their eight answers; the network policy and its proof; the line and its signature; the room and the plan; the paired receipt, signed; the governing command line as it runs; the change records of phases 2 and 6; and the records of every OFF switch drill and every failure drill. An installation whose record is complete can be audited by a person who was not there, which is the standard this manual holds every installation to.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 31

# Questions an Implementer Asks

The questions below are the ones platform teams ask in the first week, answered from the wiring as it stands. Each answer points to the section that holds the detail.

**Does Omni-Compass replace our cluster autoscaler or our HPA?** No. It sits on top of both and never replaces either (section 1.1). In a cloud the HPA keeps the operator's range and cap (the governor moves its target inside them, and its floor and cap within the replica room, handing each back), and the cluster autoscaler keeps making and deleting machines on its own schedule; the governor parks and wakes machines between them (chapter 20).

**Can it make our bill bigger?** It can, and the receipt shows it. At the max the governor may move a service's target ahead of the load, and the cloud's autoscaler may add a machine for the pods it asks for; the bill row shows that spend as power given beside the work and the speed it bought, and the cloud's own number is the work per dollar (section 20.6). A smaller bill is never the aim: the factory tune aims at more work for each dollar, and a tune whose work per dollar falls against native is not admissible as the factory tune (`docs/THE_TUNE.md`). A company that wants a cut instead turns the dial down (`--dial save`); how it spends the gain is its own choice.

**Can it wake our on-call engineer?** It is built so that it should not: it acts on the reading as it climbs toward the line, a breach wakes a parked machine at the next decision, and nothing is given back while the line is at risk. The receipt counts alert minutes in both arms (section 20.6), and the aim in a cloud is none on the omni side; an alert minute there is a fault to find, never a cost to accept.

**Does it take the 3 a.m. call and the knob turning off our engineers?** That is what it is built for, and the receipt counts both in each arm. The 3 a.m. call: the alert minutes, every minute of the window whose 95th percentile was over the line, a minute a page would be open (section 20.6). The knob turning: the settings commands the governor wrote on its own, each read back and logged with its reason; native writes none, and no person touches either arm. What a person keeps is the switch: off hands every setting back to native at once (chapter 9). How the minutes become a line of the bill, at your team's own share of what the stack costs, and what the record reads, are in the Owner's Manual and `results/OVERSIGHT.md`; on your own stack, count the pages and the hands-on minutes in both arms of a paired pilot (`docs/PILOT_PROTOCOL.md`).

**How fast does it act?** Its fast checks run every 5 seconds (a machine for pods that wait, a parked machine woken), three times as fast as the HPA's own 15-second check; its full decision runs at the factory beat for its

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

place, every 10 seconds on site and every 60 seconds through a cloud, read by the governor from the cluster's nodes, and a beat set by hand always wins (section 4.4). Native's own checks run underneath, untouched, whatever the governor's period.

**What happens if the governor crashes?** Nothing harmful. The watchdog puts every setting back from the records on the objects themselves (section 21.2); a parked machine never blocks a pod even before that, because its taint is `PreferNoSchedule`; and the native controllers carry the service as they did before the governor was installed.

**Does it need cloud credentials?** No (section 15.5). It acts through the Kubernetes API with a service account that can read the cluster, patch nodes and pods, and in the service's namespace by name move its autoscaler, record on its Deployment and resize its pods' processor, and nothing else.

**Does it work with Karpenter, KEDA, the Vertical Pod Autoscaler or a descheduler?** Karpenter: yes, in place of the cluster autoscaler; consolidation removes a parked machine once it is empty (section 18.2). KEDA: yes; KEDA drives the HPA, and the governor moves that HPA's target inside its cover as it moves any other. The Vertical Pod Autoscaler: yes, if it does not also act on the nodes; it changes requests, which the governor reads. A descheduler that uncordons or rebalances nodes is a second writer of the setting the governor moves, and must be configured to leave tainted nodes alone (section 21.8).

**Can two governors run on one cluster for high availability?** No. One writer per setting is a rule (section 6.2), and the watchdog, not a second governor, is the availability story (section 21.2).

**How much does the governor itself cost?** One pod: a request of 50 millicores and 128 MiB, a limit of 500 millicores and 256 MiB, plus a probe container of 10 millicores and 32 MiB. Every receipt reports the governor's own processor use (the row "Omni's own CPU (cores), mean").

**What does it log, and is any of it sensitive?** Object names, counts, response-time statistics, its decisions and its reasons (section 21.4). No request payload and no personal data, which it never sees.

**Can we tune it?** The factory tune is the engine as shipped (section 4.1); an installation writes the settings of section 3.4 (the line, the floor, the dial, the ceiling of a growing lever) from its own system. A changed tune is the installation's own configuration, and its receipts are its own (section 4.6).

**How do we know it is working?** The audit shows every decision and every write with its reason; the receipt of a paired run on the company's own cluster shows what it did against native, row by row, read GAIN, STEADY or LOSS (chapters 11 and 11). An installation whose receipt it has not seen is not yet an installation.

**What if our receipt shows a LOSS?** Suspect the wiring first (section 5.5): the feed, the line, the grant, the autoscaler profile, a second writer. Fix it and run the receipt again. A LOSS is never accepted and never cut from the receipt.

**What do we need to keep for an audit by our own risk team?** The installation record of section 15.7: the engine version and verification receipt, the grant and its proof, the governor's command line, the signed line, the audit, and every drill of the OFF switch.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Can we run it on our own machines, not a cloud?** Yes: chapter 8 wires every stack the governor knows, from a cluster on the company's own racks to a data center's cooling and power. There, the machines are already paid for, and the max (the most work from the same machines) is the default.

PART SEVEN

# License and Your Copy

*The terms the software and this manual are licensed under, and how to verify a copy before it is installed.*

CHAPTER 32

# License and Commercial Terms

The software and this manual are licensed under the Omni-Compass Evaluation License (`LICENSE`): evaluation and simulation use only. Everything else - commercial use, production use, operating any system beyond evaluation, redistribution, a hosted or managed service, incorporation into a product or service, or using the software or its results to build a competing product - requires a written Omni-Compass Enterprise License signed by The Omni-Compass LLC and paid for. All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by Dubra Software Inc. / The Omni-Compass LLC. No patent or trademark license is granted for any other use. Contributions are accepted only on the terms in `.github/CONTRIBUTING.md`, which assign their rights to The Omni-Compass LLC.

The copyright in Omni-Compass belongs to The Omni-Compass LLC. The source is visible under the Evaluation License, which is not an open-source license; seeing the source grants no right beyond it. The owner of a copyright may grant it under more than one license at once (the Evaluation License to everyone, a paid Enterprise License to a customer), may set the terms of each new version, and may assign the copyright; copies already received keep the terms they were received under. The license texts (`LICENSE`, `NOTICE`, `DISCLOSURES.md`) govern.

Every paper a buyer's counsel asks for is bound into this book in full, as it stands in the repository: the Evaluation License (Appendix O), the licensing questions and answers (Appendix P), the third-party notices (Appendix Q), the standard terms of the Enterprise License (Appendix R: the grant, the term, the fees' basis, the warranties and their limits, the indemnities, the audit right, the source escrow and the governing law, as a company wiring Omni-Compass onto its own stack will sign them), the patents and trademarks statements

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

(Appendix S) and the security policy (Appendix T); the disclosures, declarations and disclaimers stand in the front matter. Where a paper and this chapter differ, the paper governs.

CHAPTER 33

# Verifying Your Copy

Before a copy is installed anywhere, and again after every upgrade, it is checked in three ways, and an installation proceeds only when all three pass:

1. **The whole check.** `python3 verify.py` runs every test and every fingerprint check and ends with `VERIFICATION: PASS` or `FAIL`. A copy that fails it is not installed.
2. **The engine version.** `python3 tools/omni_version.py` prints the engine version the copy carries, or every file that differs from it. Receipts made on one version are never read against another (chapter 13).
3. **The seal and the manifest.** `python3 tools/seal.py --check` confirms that every sealed file is the file that was sealed, and the release manifest (`RELEASE_MANIFEST.json`) names the commit, the engine and the verification receipt the copy was released with. A copy whose files do not match its manifest is not the release it claims to be.

A reader holding any one of the three can tell whether the files in front of them are the files a receipt was made on. Every copy, export or printout carries the legal notice at its head and foot, with `LICENSE`, `NOTICE` and `DISCLOSURES.md` beside it.

# Back Matter

BACK MATTER

# Glossary

| Term | Meaning |
|---|---|
| Authority | how far the governor lets a muscle move this decision; adding is always allowed, giving back needs calm and a clean service record (chapter 6) |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Band | a reading's or a knob's safe range |
| Card | a muscle's own settings, found for it and kept only where it was never behind native; the factory tune for that muscle (section 4.3) |
| Dial | max (the default) or save (the tune-down), the same switch on every governor (section 4.2) |
| Factory tune | the engine as The Omni-Compass LLC tuned and ships it: every muscle on its own card or the engine's setting (section 4.1) |
| Line | the limit the service must meet, from the operator's own objectives (`--slo-ms`) |
| Engine version | the engine as fingerprinted file by file; a receipt belongs to the version its commit carries (chapter 13) |
| Compass | the band seen as a position from 0 (calm) to 1 (the line), with a 5% cushion at each edge and the centre where the reading is held (section 4.4) |
| STEADY | Equalized, Verified baseline, Equilibrium, Neutral: the mean change under 0.5% (under the floor), or nothing moved (the same), or a knob at its limit on native; the key, front matter |
| GAIN | Growth, Advantage, Improvement, Net-positive: a gauge moved the way its declared aim asks, against native; a resource cut is a GAIN; the key, front matter |
| GAIN in all three of our runs | the mean change 0.5% or more, every run (A, B, C) moved the same way with its 95% interval clear of zero, and that way a GAIN for the gauge's aim |
| GAIN by the mean of our three runs | the mean change 0.5% or more toward the gauge's aim, not every run clear of zero; the count of runs beyond the noise stands beside it (chapter 13) |
| LOSS | Leakage, Off-track, Setback, Subpar: a gauge moved against its declared aim (less work, a longer wait, more misses); it should never appear, and where it does it is shown, its cause found and fixed; the key, front matter |
| Power given | a resource spent or held that rose: what Omni put in to buy the work and speed beside it; shown, never called a loss |
| Cover | a knob's hard range; every write is clipped to it |
| Cushion | the 5% at each edge of the compass, 10% in all |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| DEX-YLL-1 | the company's number, 1-888-DEX-YLL-1, read in order as the engine runs: DEX, Data Exchange (the input, the two-way plug); YLL, Yield Limit Line (the boundary: every muscle up to its own yield limit, never past it); 1, one closed loop (the output; in code 1 is true, in science 1 is unity) |
| Fail up | the return to full capacity the moment service crosses the wall or a sense goes blind |
| Governor | the process that reads, decides and writes; Omni-Compass |
| Muscle | any machine, service or controller Omni-Compass reads and sets |
| Native | the system as it runs without Omni-Compass: every muscle under its own native controller, as its vendor ships it (the native baseline) |
| Omni-Compass on native | the second arm: the native system with Omni-Compass on top of it ("omni") |
| Omni-Compass direct | the third arm, Omni-Compass by itself: each knob's native controller switched off and Omni-Compass the only controller moving it (section 1.1) |
| Factory profile | the complete, versioned set of Omni-Compass default settings shipped together (section 4.8; `docs/FACTORY_PROFILE.md`) |
| Custom profile | an identifiable configuration that differs from the released factory profile; its results are its own |
| Active profile | the settings actually governing the current run |
| Candidate profile | a proposed combination of settings under evaluation |
| Restore factory profile | the control that reapplies the released configuration (section 4.8) |
| Return to native | the control that hands governance back and restores the recorded native settings: the reset (section 4.8, chapter 9) |
| Requested, commanded, acknowledged | what a decision asked for, what was sent once every limit was applied, and what landed and was read back (chapter 9) |
| Beyond the noise | a run whose 95% interval excludes zero; a three-run reading carries the count of its runs beyond the noise (chapter 13) |
| Plug | the two-way connection to one muscle: read, write, read back, restore |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Pool (Azure) | one machine family under Azure's autoscaler with its own ceiling; several pools make the fleet |
| Profile (service or batch) | service (the default) or batch: what the pedals do with a pile of work, a setting inside a configuration profile (section 4.5) |
| Race | full speed at once while work waits, so a burst is never served slowly |
| Yield limit line | a muscle's own limit, read live: Omni-Compass takes it right up to the line and never past it, as an engineer loads a material up to its yield limit, the most it carries and still springs back unharmed |
| Speed floor | the clock a card reaches on its own while busy, learned from its own meter; the governor never sets the ceiling under it |
| Receipt | the paired record of native against Omni-Compass for one run |
| Snapshot | a knob's value read once before the first write; the restore point |
| The runs disagree | runs clear of the noise point different ways; the test is unstable on that measure and is looked into |
| Wire check | the test that proves every wire follows, reads back and returns before anything runs |
| Work per energy | work done divided by energy used |
| Work inside the line | the requests, transactions or messages a second answered within the response line; the product number on every real stack |
| One pedal | push for more, lift off for less, idle (the pedal lifted all the way), reset (everything back to native): how the governor moves capacity under autopilot |
| Cruise | every machine in service while work waits for a place, held without second-guessing until the line has been empty for two decisions |
| Idle at once | the work done and demand at zero: the pedal lifted all the way in one move, every safety check still holding |
| Kill switch | security only: one switch in a human hand that turns every governor on the machine off at once; never the reset |
| Lease | the record every governor renews each decision; a lease older than its limit, or a dead process, makes the watchdog hand back for it |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Watchdog | the service beside Omni-Compass's processes that hands back for any of them that died or hung without doing it itself |
| Verdict, paired trial | the trial on the muscle itself that allows a slow knob to move only where the cost is within 2%; "left native" where no step passes (section 1.2) |
| Tuning case | the one workload or case the rules were fixed on; shown in every receipt and never counted |
| Untouched case | a workload or case the rules were applied to unchanged; the judged rows |
| Amendment | a change to the written rules of a paired run after they were written, with its time and reason; one made after a result was seen says so |
| Dwell | the seconds after a knob moves during which the law will not move it back |
| Notch | the smallest step a knob moves in one decision (one consumer, 8 MB of cache, one tap) |
| Evidence class P, L, S | a physical meter; live software with no meter; a model (chapter 12) |
| Handed back | the knob read back at the operator's value at the end of an arm; every receipt says whether every arm was |
| Working set | the records a workload actually touches over a window; a cache benchmark measures nothing unless the working set reaches the cache |
| Full-cache gate | the MongoDB and Redis rule: slow reads grow the cache only while the cache is full (nine tenths used), because a miss in a cache with room to spare is not the cache's to mend |
| Buffer pool | InnoDB's cache of table and index pages, the one memory setting every MySQL operator sizes; resized online by the server in whole chunks |
| Chunk | the unit in which MySQL resizes its buffer pool (128 MB as shipped); the knob's notch on that stack |
| **Term** | **Meaning** |
| Page cache | the operating system's own cache of file contents; where the data fits in it, a storage-engine miss is a read from memory and a |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| | decompression, not a disk read, and the gain available to the cache knob is smaller |
| Cloud mode | the governor on top of a public cloud's cluster: since 31 October the whole engine at the max, the bill judged beside the work by the one key; until then, by the order of 14 October, the tune-down on the machines alone (chapter 20) |
| Parked machine | a machine the governor has closed to new work with the taint `omnicompass.io/idle:PreferNoSchedule`; it stays Ready and usable, its pods are marked first to go, and the cloud's autoscaler deletes it once it is empty |
| Taint, `PreferNoSchedule` | a mark on a machine that the scheduler treats as a preference against placing new pods there, never as a refusal; the only taint the governor writes |
| Cordon | a machine marked unschedulable (`kubectl cordon`), by a person, an upgrade or a repair; the governor never cordons or uncordons a machine, and leaves a cordoned machine alone |
| Pod deletion cost | the annotation `controller.kubernetes.io/pod-deletion-cost`, by which a ReplicaSet removes the lowest-cost pods first when it scales down; the governor marks a parked machine's pods with it so that they leave first |
| Cluster autoscaler | the cloud's own controller that adds a machine when pods cannot be placed and deletes a machine that has been unneeded for its configured time; native, never changed by the governor (Karpenter plays the same part on Amazon) |
| HorizontalPodAutoscaler (HPA) | the Kubernetes controller that sets a service's replica count from its pods' measured use against a target; native, its target moved by the governor inside its cover and its floor and cap within the replica room, each handed back |
| Room | the machine-hours a pool holds above what its pods' requests need; the upper bound on any machine lever's saving, measured before any run (section 25.2) |
| Billable room | the room above the machines a commitment already pays for: the part of the room that can reach the bill (section 25.2) |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Feed | the file of rows `elapsed_seconds,latency_ms,ok` the governor reads its response times from: the probe's, or an adapter's (chapters 19 and 23) |
| Blind | a sense with no fresh reading (the feed silent for twice its window, or no served request in it); the governor gives nothing back while any sense is blind |
| Lease | the record each governor renews every decision; a lease out of date, or a process gone, makes the watchdog hand back for it (section 21.2) |
| Watchdog | the process that hands every setting back for a governor that died or hung; in a cluster, a container in the governor's own pod (section 21.2) |
| Proprioception | the governor's reading of whether its last command landed: the drift between the machines it commanded and the machines in service, in each `authority` record |
| Dry run | `--dry-run`: the governor decides and records every command it would run, marked `"dry_run": true`, and writes none |
| Admission policy | a rule the API server applies to every write after the role allows it; section 27.4 binds the governor's writes to its own taint and its own pod mark |
| System pool | the pool of machines that carries the cluster's add-ons and the governor, tainted so no workload lands there, never governed |
| Work pool | a pool labelled `omni-role=work`, under the cloud's autoscaler, whose machines the governor may park |
| OFF switch of record | the form of the OFF switch the runbook names for a cluster: scaling the governor to zero, or a commit to Git where continuous delivery reconciles it (sections 21.3 and 28.4) |
| Data path | the route the work's own data takes (requests, packets, files, bytes); Omni-Compass is never in it: it reads the gauges (one wire in) and writes the settings native controllers already accept (one wire out) |
| The seat | where Omni-Compass sits: in one place, the software end, on top of every controller the stack runs, as one body; it changes nothing in the silicon or the cloud's back end, and its effect is carried along the whole path the work travels |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| The three layers | the chip, the cloud and the software, where the industry attacks the cost of computing; each layer's own answer moves its own terms of the one equation, and Omni-Compass's gain is read on top of it (`docs/INDUSTRY_LANDSCAPE.md`) |
| Work for the bill, E | the work done over the bill, E = W / B; the gain G = E(Omni-Compass on native) / E(native) − 1 = (1 + ΔW) / (1 + ΔB) − 1 (`results/TWO_BILLS.md`) |
| Four arms | A and B, native and Omni-Compass on native on bare metal; C and D, the same through the cloud; each read by its bill, the work done inside it and the work per dollar |
| Alert minutes | whole minutes whose 95th percentile, failed requests counted over, was over the response line: the minutes an on-call page would be open, native beside Omni-Compass; the oversight the record measures, never people |

APPENDIX

# Appendix A. Command Reference

| Task | Command | | | |
|---|---|---|---|---|
| Verify the copy | `python3 verify.py` (ends `VERIFICATION: PASS`) | | | |
| Which engine version a checkout or a commit carries | `python3 tools/omni_version.py [--commit <sha>]` | | | |
| Seal check | `python3 tools/seal.py --check` | | | |
| Evaluate without touching anything: the organisms, modelled | `python3 tools/run_scale.py --runs N --scale K --out <dir>` | | | |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Command | | | |
|---|---|---|---|---|
| Evaluate without touching anything: the two-wire card, modelled | `python3 tools/run_gpu_card.py <dir>` | | | |
| Kubernetes, watch (read-only) | `kubectl apply -f deploy/install/omni-compass.yaml` (it runs `--mode observe`) | | | |
| Kubernetes, govern | `python3 -m omni_controller.controller --mode target | nodepool --latency-file <feed> --slo-ms <line> [--law compass] [--dial max | save ]` | |
| A response-time feed against any HTTP endpoint | `python3 scripts/latency_probe.py <url> <out.csv>` | | | |
| GPU wire check | `sudo python3 tools/gpu_wire_check.py --gpu 0` (must end `WIRED RIGHT`) | | | |
| GPU governor, two wires | `sudo python3 -m omni_controller.gpu_compass --mode watch | cap --gpus 0 --latency-file <feed> --slo-ms <line> [--dial max | save ]` | |
| GPU, the whole paired run in one command | `sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &` | | | |
| The processor's clock under its meter, the paired run | `sudo bash scripts/cpu_power_run.sh` | | | |
| Kubernetes paired runs, then the receipt | `scripts/kind_paired.sh`, then `python3 tools/live_reps.py <dir>` | | | |
| The three-run table (Kubernetes, cloud) | `python3 tools/confirm_abc.py "<title>" <run A dir> <run B dir> <run C dir> --out <table.md>` | | | |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Command | | | |
|---|---|---|---|---|
| The three-run table (pooler, broker, cache, storage engine, buffer pool) | `tools/pgbench_abc.py`, `tools/kafka_abc.py`, `tools/redis_abc.py`, `tools/ycsb_abc.py`, `tools/sysbench_abc.py`, the same arguments | | | |
| OFF, one muscle (Kubernetes) | `touch /tmp/omni.kill` | | | |
| OFF, one card | `sudo touch /tmp/omni-gpu-kill` | | | |
| The whole harness OFF, ON, status, watchdog | `python3 tools/omni_switch.py off | on | status | watchdog` |
| Cloud: the cloud's grant, then the governing Deployment (dry run first) | `kubectl apply -f deploy/install/rbac-cloud.yaml`, then `kubectl apply -f deploy/install/omni-compass-cloud.yaml` (chapter 20) | | | |
| Cloud mode: the cloud image | `docker build -f deploy/Dockerfile.cloud --build-arg BASE=<registry>/omni-compass:<version> -t <registry>/omni-compass-cloud:<version> .` | | | |
| Cloud mode: OFF, the switch of record | `kubectl -n omni-compass scale deployment/omni-compass --replicas=0` (section 21.3) | | | |
| Cloud mode: OFF from inside the pod | `kubectl -n omni-compass exec deploy/omni-compass -c controller -- touch /tmp/omni.kill` | | | |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Command | | | |
|---|---|---|---|---|
| Cloud mode: the grant, as the governor sees it | `kubectl auth can-i <verb> <resource> --as=system:serviceaccount:omni-compass:omni-compass -A` (section 15.4) | | | |
| Cloud mode: machines parked now | `kubectl get nodes -l omni-role=work -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints[*].key` | | | |
| Cloud mode: the last decisions | `kubectl -n omni-compass exec deploy/omni-compass -c controller -- tail -n 20 /var/log/omni/audit.jsonl` | | | |
| Planning: the room on a cluster | the sampling loop and the arithmetic of section 25.3 | | | |
| A feed from the company's own Prometheus | `python3 tools/prom_feed.py --prometheus <url> --served <q> --failed <q> --quantile <q> --out <feed>` (section 23.8) | | | |
| The paired run on a cloud, from GitHub | the `aks-metered`, `gke-metered` or `eks-metered` workflow (Appendix N) | | | |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

# Appendix B. The Files an Installation Touches

| Path | What it is for |
|---|---|
| `verify.py`, `tools/omni_version.py`, `tools/seal.py`, `RELEASE_MANIFEST.json` | checking the copy before it is installed (chapter 33) |
| `deploy/Dockerfile` | the container image: non-root, read-only root filesystem, no Linux capabilities |
| `deploy/install/omni-compass.yaml` | the watch-mode install with a read-only identity (level 1) |
| `deploy/install/rbac-cloud.yaml` | the cloud's grant: patch nodes and pods, and in the service's namespace by name its autoscaler, its Deployment's records and its pods' processor, nothing else (section 15.3) |
| `deploy/install/omni-compass-cloud.yaml` | the cloud's Deployment: Omni-Compass, its probe and its watchdog in one pod, shipped with `--dry-run` (chapter 20, Appendix I) |
| `deploy/Dockerfile.cloud` | the cloud image: the shipped image with `jq`, the machine script, the probe, the Prometheus feed adapter and the switch tool |
| `scripts/kind_nodepool.sh` | the machine script: keeps `{n}` machines in service by the idle taint alone; never creates, deletes, cordons or uncordons a machine, and never moves a pod |
| `tools/prom_feed.py` | the response-time feed from the company's own Prometheus (section 23.8) |
| `deploy/rbac-target.yaml`, `deploy/kind/rbac-omni.yaml` | the permissions of levels 2 and 3, exactly the verbs each level needs |
| `scripts/latency_probe.py` | the response-time feed against any HTTP endpoint |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Path | What it is for |
|---|---|
| `scripts/kind_paired.sh`, `tools/live_reps.py`, `tools/confirm_abc.py` | the paired runs, the receipt and the three-run table on Kubernetes |
| `tools/gpu_wire_check.py`, `scripts/gpu_rented_run.sh`, `tools/gpu_workload.py` | the card's wire check, its paired run, and a workload that writes the card's own time per request |
| `scripts/cpu_power_run.sh` | the processor's paired run on bare metal |
| `tools/omni_switch.py` | the master switch, the status and the watchdog (chapter 9) |
| `realms/catalog_v5.csv`, `realms/muscle_cards_v5.csv`, `docs/MUSCLE_CATALOG.md` | every muscle, its wiring and its own card (Appendix E) |
| `docs/AZURE_SETUP.md`, `docs/GOOGLE_SETUP.md`, `docs/AMAZON_SETUP.md` | each cloud's account and its secrets, click by click (Appendices F to H) |
| `.github/workflows/aks-metered.yml`, `gke-metered.yml`, `eks-metered.yml`, `aks-metered-full.yml` | the paired runs on each cloud, from GitHub (Appendix N) |
| `LICENSE`, `NOTICE`, `DISCLOSURES.md`, `SECURITY.md` | the license, the notices, every declaration and disclaimer, and how to report a security issue |

APPENDIX

# Appendix C. The Gauges

Every gauge in a receipt, where it comes from, and whether it is measured or modelled:

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| work inside the response line (requests, transactions or messages a second answered within the line) | the probe's own records, pgbench's log, the consumers' records, the application's records | measured | higher; the product number |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| response time, mean, median, 95th and 99th percentile | the same | measured | lower |
| time over the response line (share of samples) | the same, against the written line | measured | lower |
| failed requests, lost messages | the same | measured | lower; any increase is marked |
| alert minutes: whole minutes whose 95th percentile, failed requests counted over, was over the line | the probe's own records, minute by minute | measured | lower: the minutes an on-call page would be open |
| settings commands Omni-Compass wrote on its own | its own audit | measured | shown, not judged (no person touched either arm) |
| the governor's additions decided while the service was still inside its line (% of its additions) | the governor's audit | measured | shown, not judged |
| worker machines in service, mean; node-hours | the cluster's own node records every 15 s | measured (machine-hours on a local cluster; the provider's own count in a cloud) | lower; a rise is power given |
| compute bill at list price | the provider's own machine count × list price per family | measured (a real bill) | lower; a rise is power given |
| energy (Wh), the standby model (in the receipts, "parked-worker energy": a parked machine still drawing its idle power) | a declared formula over the machines in service | **modelled**, said so on every line | lower; a rise is power given |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| energy, the machine's own meter | the processor's RAPL counters, a card's own power reading, a site meter | measured (class P) | lower; a rise is power given |
| host CPU busy; host CPU-seconds; CPU-seconds per 1,000 units inside the line | `/proc/stat` of the real machine under the run | measured | lower |
| Omni's own CPU (cores), mean | the governor's audit (its process and every command it ran) | measured | shown beside the receipt |
| connections held open, consumers running, memory ceiling held, cache size held | the pooler's, the group's, the cache's and the server's own reports | measured | lower; a rise is power given |
| cache hit rate; consumer lag | Redis's INFO; the consumers' records | measured | higher; lower |
| pages read into a cache, the cache's miss share | the server's own counters | measured | shown, not judged |
| HPA replicas, pods started, pod start wait | the API server's own records | measured | shown, not judged except pod start wait (lower) |
| pods waiting for a place (pending), mean | the API server's own pod records | measured | lower: work with no machine to run on |
| CPU limits conveyed; conveyance holds (`convey: held`); quota lines (`quota`) | the governor's audit | measured | shown, not judged: what the processor lever did and what held it (section 6.5) |
| seconds from a kill to every setting back at the operator's | the harness polling the system once a second from the kill | measured | lower; past 60 s marked; never, INVALID |
| handed back (every arm) | the knob read back at the end of the arm | measured | yes in every arm, or the run is invalid |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Three rules for reading any gauge in this appendix.** A measured gauge is one the system under test, or the host's own kernel, reported about itself; a modelled gauge is one computed from a declared formula over measured inputs, and every receipt says which it is on the line itself. A resource gauge (machines, connections, consumers, memory, cache) is always read against an aim of lower, even where the governor's purpose was to add the resource to buy service, so that a gain bought with a resource is shown as power given in the same receipt. And a gauge marked "shown, not judged" never enters a reading; it is there so that a reader can see what the governor did (how often it moved, how many pages it read, how many pods it started), not to be counted for or against it.

APPENDIX

# Appendix D. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| Wire check `FAIL 3 up wire (lock)` | the driver or VM refuses clock locking | use bare metal or full passthrough; or run the one-wire governor |
| Wire check `FAIL 3 up wire (follows down)` | the card ignores the ceiling | update the driver; check for another management agent holding clocks |
| Wire check `FAIL 4 down wire` | the power limit is refused or capped by the board | check `nvidia-smi -q -d POWER`; run as root |
| `another copy of this test is already running` | a second copy was started | wait for the first or reboot; start once |
| `something else is using the GPU` | another process holds the card | stop it; the paired run must run alone |
| Governor exit 5 | another writer changed a knob | find the other controller; Omni-Compass left its value alone (section 6.2) |
| Governor exit 3 | a restore did not read back | restore by hand (`nvidia-smi -rgc`, `-pl <start>`); investigate before running again |
| `decision failed (n in a row)` | the cluster API is unreachable | turn it OFF; native runs on |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Symptom | Cause | Fix |
|---|---|---|
| Energy saved but p95 slower than native | the card served bursts below its own clock, or the lid sat under its own draw | read section 5.5; in the audit compare `telemetry.clock_mhz` while busy with `native_busy_clock_mhz`, and `want_w` with `native_busy_draw_w` |
| Fail-up in most decisions | `--slo-ms` set too low for the workload, or a stale feed read as blind | set `--slo-ms` from the workload's own target; check the feed's age (section 6.3) |
| No saving at all, service unchanged | the card is saturated almost all the time (it races), or the dial is at the max | expected: the max spends where it buys work; the tune-down (section 4.2) saves in the quiet stretches |
| Kubernetes: no machine ever given back | the dial is at the max (no machine below the cluster's own is given back), or the release gate refuses (its reason is in the audit) | expected at the max; at the tune-down read `node_gate.reason` in the audit and check `--slo-ms` and the latency feed |
| A pooler's clock reads calm while users wait | the pooler cannot see a client waiting behind its own schedule | wire the queue share beside the service time and read the larger (section 8.2) |
| Broker: consumers climb to the cover and stay | messages wait at every step (the producer is past the group's capacity) | check the offered rate against the group's measured capacity in the run's log |
| Cache: the ceiling grows while little memory is used | cold misses read as pressure | the full-cache gate is the rule (used ≥ 90% of the ceiling); check `used_mb` beside `maxmemory_mb` in the run's log |
| Cache: the ceiling never grows | the working set fits the operator's ceiling | expected: a cache that fits has nothing for the ceiling to mend, and the row reads STEADY |
| Storage-engine cache: the cache never fills, whatever the load, and omni reads as a free memory saving | the requests concentrate on a few hot records, or the run asks for records the load did not write, so the reads are served from the index | a result that flatters the governor is the first thing to suspect: check the bytes in the cache against its size; draw requests over the whole key space and use the same key order |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Symptom | Cause | Fix |
|---|---|---|
| | | in load and run; a knob with nothing to do is not a saving |
| Storage-engine cache or buffer pool: the governor only ever reads calm | the line is set far above the stack's own latencies | set the line on the tuning case's own figures so that a hit is calm and a full-cache miss is slow, and write the figures down before any counted run |
| Buffer pool: not handed back at the end of an arm | the server was still carrying out a shrink and ignored the restore | the plug waits for the resize and writes again (section 8.8); check the restore record in the arm's receipt |
| A three-run table heads itself with a warning | a run's commit is not the engine version it claims, or the three are not separate runs | run `tools/omni_version.py --commit <sha>` on each; make the table only from runs on one engine version |

| **Cloud: the governor's pod shows `2/3` or restarts its watchdog** | **the pod was applied without `shareProcessNamespace: true`, or the watchdog lacks the shared `/tmp`** | **apply the manifest as shipped (Appendix I); a watchdog that cannot see the governor's process judges it dead and hands back over it (section 21.2)** |
|---|---|---|
| Cloud: parked machines are woken every few seconds, then parked again | the watchdog runs in a separate pod, or a second Omni-Compass runs on the cluster | one Omni-Compass, its watchdog in its own pod (section 21.2); `kubectl get pods -A -l app=omni-compass` must list exactly one pod |
| Cloud: the pod is refused at admission with a Pod Security message | the namespace enforces a profile the pod does not meet, or a later edit loosened the pod | the shipped pod meets `restricted` (section 27.3); compare the applied pod with Appendix I |
| OpenShift: the pod is refused with `unable to validate against any security context constraint` | `restricted-v2` refuses the fixed user 10001 | grant `nonroot-v2` to the service account (section 26.3) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| **Cloud: the governor's pod shows `2/3` or restarts its watchdog** | **the pod was applied without `shareProcessNamespace: true`, or the watchdog lacks the shared `/tmp`** | **apply the manifest as shipped (Appendix I); a watchdog that cannot see the governor's process judges it dead and hands back over it (section 21.2)** |
|---|---|---|
| Cloud: every decision shows the feed blind | the probe cannot reach the service (its URL, a network policy, the service's port), or the adapter's queries return nothing | `kubectl -n omni-compass logs deploy/omni-compass -c probe` (or `-c feed`); test the URL from the pod; check the network policy (section 27.5) and, for the adapter, run each query in the metrics system's own console |
| Cloud: `trial abandoned (too little work)` in most `verdict` records | the feed is thinner than twenty rows a minute, or the service is idle | densify the feed (section 23.1); at an idle service, nothing is given back on trial, by design |
| Cloud: `step refused: the muscle is slower there` in most `verdict` records | the service is slower without the machine | the governor is right to keep it; the pool has no room at this load (section 25.2) |
| Cloud: the governor parks nothing for days and its trials are never begun | the pool already holds only what the pods' requests need with the governor's spare, or the floor is the pool | read N[gov] from the room sampler (section 25.3); STEADY on the bill is then the honest receipt |
| Cloud: `refused` records with code 1 on every park | the cloud's grant is not bound, or an admission policy refuses the taint | `kubectl auth can-i patch nodes --as=system:serviceaccount:omni-compass:omni-compass`; the policy's message names the rule (section 27.4) |
| Cloud: a parked machine is never deleted though its pods are gone | the cluster autoscaler's unneeded time is long, a DaemonSet or a pod it will not move remains, or a PodDisruptionBudget allows no disruption | read the autoscaler's own status (its config map or events, `kubectl -n kube-system describe configmap cluster-autoscaler-status`) for the machine's reason (section 25.4) |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Cloud: the governor's pod shows `2/3` or restarts its watchdog | the pod was applied without `shareProcessNamespace: true`, or the watchdog lacks the shared `/tmp` | apply the manifest as shipped (Appendix I); a watchdog that cannot see the governor's process judges it dead and hands back over it (section 21.2) |
|---|---|---|
| Cloud: the audit is empty after a restart | the audit was on the pod's `emptyDir` | mount a persistent volume at `/var/log/omni` and ship the file (section 21.4) |
| Cloud: the governor starts again minutes after it was scaled to zero | a continuous-delivery controller restored Git's replica count | make the OFF switch a Git change, or tell the controller to ignore the replica count (section 28.4) |
| Cloud: the bill rises against native for the same work | the target moved ahead of the load asked the cloud's autoscaler for machines the work did not need | read the work per dollar row and the trial records; a factory tune whose bill rises for the same work is a fault of the tune, written to the next version's list with its cause (`docs/THE_TUNE.md`) |
| Cloud: the cloud image build fails at `FROM` | the base image argument was not given | pass `--build-arg BASE=<registry>/omni-compass:<version>` (section 20.4) |

APPENDIX

# Appendix E. The Muscles: What Each Is For, and How It Is Wired

A **muscle** is one setting on one machine that already has its own control: a replica target, a node pool's size, a GPU's clock ceiling, a chiller's setpoint, a battery's reserve, a joint's effort. Omni-Compass does not replace that control. It reads the machine's meters and, at its default, the max, moves the setting the machine already accepts toward the machine's own limit for the most work, never past it and never below native, through a plug that reads the setting once before the first write, reads back every write, steps aside if another controller moves it, and walks it back to native at the end. A knob native already runs at the top of its range is at its limit on native and is left there.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

This chapter lists all 1107 muscles of the v5 catalog (`realms/catalog_v5.csv`); each run records its engine separately. For each one it says what kind of machine it is, which of the four kinds of knob it is, what Omni-Compass reads and does with it, how such a muscle is reached in a real stack, and which organisms it belongs to. The plants behind the benchmark numbers are models of these machines (evidence class **S**); a muscle in this list is wired on a real system only through the levels and the checks of the manual (chapters 4 and 6). See `DISCLOSURES.md`.

## E.1 How to read an entry

**The four kinds of knob.**

- **capacity**: how much of the machine is in service (replicas, instances, units, speed). At the max Omni-Compass pushes it to the machine's own limit for the most work, never past it and never below native.
- **admission**: what is let in (queues, rate limits, admission control). Native already lets everything in: the knob is at its limit on native, and Omni-Compass reads it and leaves it native.
- **power**: the power share or limit the machine may draw. Native already runs it at full: at its limit on native, read and left native.
- **setpoint**: the target the machine's own loop holds (a temperature, a reserve, a process value). At the max a room is cooled ahead of the heat and a battery's charge is held for its line, inside the safe band, never behind native on its line.

**The five kinds of plant.** Every muscle is modelled on one of five plants, each with its own service reading and lever:

| Plant | What it is | What Omni-Compass reads | What Omni-Compass moves |
|---|---|---|---|
| `compute_pool` | a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs | how far its queue and its load sit toward the service line (queue or lateness, and load above half) | the capacity it may use: its replica or instance count (through its autoscaler's target or floor), its power share, or one unit given back through the release gate |
| `thermal_zone` | a data hall or building zone cooled by chillers, air handlers or fans | its temperature against its limit | its supply setpoint (colder is more cooling), its power share, or one cooling unit given back when the room has margin |
| `energy_storage` | a battery, UPS or microgrid store with a load and a grid connection | its draw on the grid connection and its reserve | its reserve setpoint (how much charge it holds back); its power limit is left native |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Plant | What it is | What Omni-Compass reads | What Omni-Compass moves |
|---|---|---|---|
| `motion_axis` | a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor | its tracking error and its load | its speed and effort share, or its power share |
| `process_loop` | a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint | how far the process sits from its band | its setpoint within its safe band, its actuator range, or its power share |

**Every muscle its own card.** The brain is one, its settings each muscle's own. A muscle's card holds the settings the max uses for that muscle (a machine pool: how full it runs on its forecast before it calls for more, and how many decisions of history it reads; a battery: how many wobbles of margin it keeps under its line, and, wired by its power, how long it reads its line before it holds and how much of it it keeps clear). The cards were found by an automatic search over every muscle on the development seeds, today's global setting always one of the choices, and kept only where the card beat the global setting beyond the noise on two separate sets of seeds and on no seed under any load fell behind native: never over its line more often, never less work, never its line cleared later (`tools/tune_cards.py`, `realms/muscle_cards_v5.csv`, `results/cards_v5/`). Inside a body, a battery wired by its power keeps the body's own margins on its line. 364 muscles carry a card of their own; every other muscle runs the global setting (machines started at 95% full, the last 20 decisions read, 3.09 wobbles of margin on a battery's line, 5% of it kept clear). Each muscle's card is in the last column of its realm's table.

**The organisms.** The 1107 muscles build seven organisms: the spine (267 muscles, every muscle that sits in all four realms: the base every stack runs on, as an operating system is the base of every program), each of the four realms (every muscle whose realm list includes it, the spine among them), the four stacked with every duplicate kept (1,908), and the whole tower with every muscle once (1107). Each is run native and with Omni-Compass on top of native (`docs/THE_SPINE.md`, `docs/REALMS.md`).

| Organism | Muscles |
|---|---|
| The spine, the muscles every stack runs on | 267 |
| Compute / AI / Cloud | 540 |
| Physics / Robotics / Autonomous | 404 |
| Energy / Facility / Industrial | 480 |
| Distribution / Specialized | 484 |
| The four stacked, every duplicate kept | 1,908 |
| The whole tower, every muscle once | 1107 |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**How a muscle is reached in a real stack.** The class of machine decides the wire:

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `batch` | batch and queued jobs | the job queue (Kubernetes Jobs, Slurm, Spark) |
| `batch_reactor` | a GMP batch reactor, fermenter or food process | the batch control system (ISA-88, OPC UA) |
| `building` | a building zone | the building management system |
| `chamber` | a controlled chamber (clean room, kiln, reactor) | the PLC or DCS |
| `clinical` | clinical systems: imaging archives, records, interface engines, monitoring gateways | the application's own scaling and admission settings (Kubernetes, the vendor console) |
| `commerce` | a customer-facing service with bursts | the Kubernetes API |
| `cpu_host` | a CPU host whose clock and power can be set | Linux cpufreq and RAPL |
| `data_hall` | a data hall cooling plant | the building management system (BACnet, Modbus) or its command line |
| `database` | a database cluster | the database operator (replicas, connection pools) |
| `district_heat` | a district heating or cooling network | the network control system and substation controllers (Modbus, M-Bus) |
| `elevator_hoist` | an elevator or escalator drive and its group control | the lift controller and destination dispatch (modelled only) |
| `ev_traction` | an electric vehicle traction motor | the vehicle's motor controller (CAN; modelled only) |
| `fabric` | a high-speed fabric (RDMA, DPU, SmartNIC) | the fabric manager's API |
| `facility` | a facility battery behind the meter | the energy management system |
| `feeder_voltage` | a distribution feeder's voltage | the distribution management system (IEC 61850, DNP3; modelled only) |
| `flight_axis` | a flight-control axis | the flight controller (modelled only) |
| `gpu` | GPU serving with a response-time target | nvidia-smi (clock ceiling and power limit) or the serving autoscaler |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `gpu_batch` | GPU training and batch work | nvidia-smi, the job scheduler's pause and resume |
| `hospital` | a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites) | the hospital's building management system (BACnet) |
| `inverter` | a plant of grid-support inverters or wind turbines | the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547) |
| `irrigation` | pumps, pressure and climate on a farm or in a greenhouse | the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud) |
| `marine_propulsion` | a ship's propulsion shaft and power plant | the vessel automation and power management system (modelled only) |
| `microgrid` | a microgrid battery and loads | the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR) |
| `mill` | a grinding, flotation or materials-handling circuit | the plant DCS or PLC (OPC UA) |
| `network` | network routing and switching | the network controller (SDN, routing API) |
| `node` | a fleet of machines or VMs that boot in minutes | the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG) |
| `pipeline` | a pipeline segment with its compressors, pumps and valves | pipeline SCADA (DNP3, Modbus, OPC UA) |
| `port` | a container terminal's cranes and vehicles taking moves | the terminal operating system and fleet controllers |
| `process` | an industrial process loop | the PLC or DCS (OPC UA, EtherNet/IP, PROFINET) |
| `qpu` | a quantum or specialised accelerator queue | the accelerator's job queue |
| `rail_traction` | a train's traction and auxiliary systems | the train control and management system and ATO (modelled only) |
| `ran` | a radio access network cell or site | the RAN controller (O-RAN interfaces) |
| `reaction_wheel` | a spacecraft reaction wheel | the attitude controller (modelled only) |
| `robot_fleet` | a fleet of robots or vehicles taking tasks | the fleet manager's dispatch API |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `robot_joint` | a robot joint servo | the robot controller (ROS 2, EtherCAT, the drive's fieldbus) |
| `server` | an application service on servers or pods | the Kubernetes API (HPA target and floor, pod resize) |
| `storage` | block, file or object storage | the storage system's QoS and tiering API |
| `turbine` | a generating unit's governor, boiler or excitation loop | the plant DCS and governor (OPC UA, IEC 61850; modelled only) |
| `ups` | an uninterruptible power supply | the UPS and PDU management interface (SNMP, Modbus) |
| `water` | a water or pumping process | the SCADA system (Modbus, DNP3) |
| `workflow` | a workflow or pipeline engine | the workflow engine's concurrency settings |

Wires marked *modelled only* exist as plants in the benchmark and are not built for live use.

**How to wire any muscle on your own stack (the plug, chapter 4).** Every muscle below is wired the same way: Omni-Compass never replaces the native controller; it reads the controller's own meters (the wire out) and writes the one setting the controller already accepts (the wire in), through a plug that (1) reads the setting once before the first write and records it, (2) reads back every write and stops writing if another writer moved the setting, (3) renews a lease every decision so a dead governor's settings walk back to native by themselves, and (4) restores the recorded setting at the end. Each family's entry says which native controller owns its muscles, where the reading is tapped, where the setting is written, how it is restored, and whether that wire has run live in this repository or only on a model (`docs/wiring_recipes.csv`). The muscle's name is the setting's own name on that controller. Where the entry says *modelled only*, the plant exists in the benchmark and the wire is written against the controller's documented interface, not yet run on the machine.

## E.2 The 1,107 muscles, heaviest hitters first, in the four realms they live in

**Heaviest hitters first** (the founder's order of 1 November): every list of the muscles is ordered by what Omni-Compass did for each muscle against native in the record of every muscle alone (`results/realms_v5/REALMS_V5.json`; our own models, evidence class S; omni-v8, run 2026-10-17 15:39 UTC, 10 seeds, both loads). First the muscles that take decisions back under their line, the largest share of all their decisions first (a decision over the line is a muscle past its own limit, the first harm do no harm names); then the muscles that clear the work waiting in line sooner, the most first; then the muscles that only add work; then the muscles the same as native; last the muscles at their limit on native, where Omni-Compass

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

writes nothing; equal hits by the catalog's number. The four lists are the four realms, each muscle under its home realm, every muscle once (`tools/heavy_hitters.py`).

Each realm's table lists its home muscles (Compute / AI / Cloud 444, Physics / Robotics / Autonomous 137, Energy / Facility / Industrial 249, Distribution / Specialized 277; 1,107 in all), heaviest hitter first and down to its last. A muscle of the spine sits in all four realms: it is part of every realm's organism, counted once in the tower and four times in the stack, and listed once here, under its home realm. Its hit is the record's own reading of that muscle alone (omni-v8, class S): a model's reading of what the engine did there, never a benchmark result. How each family is wired on a real stack (its native controller, where the reading is tapped, where the setting is written, how it is restored) follows the four tables, the family with the heaviest hitter first.

### *E.2.1 Compute / AI / Cloud: 444 muscles, heaviest hitter first*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 1 | 108 | architecture selection | Cloud VM & Capacity | capacity | GAIN, 3.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 2 | AUDIT-0002 | dra claim capacity | Kubernetes Dynamic Device Allocation | capacity | GAIN, 3.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what | Compute | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | it no longer needs at once | | |
| 3 | 619 | global failover | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 3.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 40 decisions |
| 4 | 1065 | lora adapter load | AI Inference Serving | capacity | GAIN, 3.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 5 | 109 | accelerator selection | Cloud VM & Capacity | capacity | GAIN, 2.97% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 6 | 1117 | sync boost | GPU Hardware | capacity | GAIN, 2.95% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 7 | 1121 | instance refresh | Cloud VM & Capacity | capacity | GAIN, 2.95% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 8 | 166 | continuous batching | AI Inference Serving | capacity | GAIN, 2.92% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 9 | 75 | nodepool weight | Node Fleet & Karpenter-Class Control | setpoint | GAIN, 2.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 10 | V5-5032 | vm overcommit density | Virtualization & Hypervisor | capacity | GAIN, 2.78% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 11 | V5-5128 | edge site node pool | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.58% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 12 | 622 | edge offload | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.58% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 13 | AUDIT-0006 | dra binding readiness | Kubernetes Dynamic Device Allocation | capacity | GAIN, 2.56% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 85% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 14 | AUDIT-0007 | dra binding failure response | Kubernetes Dynamic Device Allocation | capacity | GAIN, 2.53% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 15 | 611 | zone dispatch | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.51% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 16 | 161 | model replicas | AI Inference Serving | capacity | GAIN, 2.51% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 17 | 165 | model instance count | AI Inference Serving | capacity | GAIN, 2.50% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 18 | 167 | max batch size | AI Inference Serving | capacity | GAIN, 2.49% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 40 decisions |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 19 | 99 | vm start stop | Cloud VM & Capacity | capacity | GAIN, 2.47% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 20 | V5-5088 | agones fleet max replicas | Game Servers & Real-Time Interactive | capacity | GAIN, 2.47% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 21 | 73 | node shutdown | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.45% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 75% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 22 | 1037 | max pods per node | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.43% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 23 | 68 | node provision | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.42% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 60 decisions |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 24 | 71 | node consolidate | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.40% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 25 | 66 | node pool min | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.38% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 10 decisions |
| 26 | V5-5117 | warm runner pool | CI/CD & Build Farms | capacity | GAIN, 2.38% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 75% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 27 | 91 | node config rollout | OpenShift & Machine API | capacity | GAIN, 2.36% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 28 | 613 | cloud capacity class | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.34% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 29 | 614 | workload migrate region | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.33% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 30 | V5-5109 | runner scale set max runners | CI/CD & Build Farms | capacity | GAIN, 2.33% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 60 decisions |
| 31 | 107 | region selection | Cloud VM & Capacity | capacity | GAIN, 2.30% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 32 | 610 | region dispatch | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.29% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 40 decisions |
| 33 | 111 | placement group | Cloud VM & Capacity | capacity | GAIN, 2.28% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 34 | 100 | vm migrate | Cloud VM & Capacity | capacity | GAIN, 2.27% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 35 | 1063 | tensor parallel degree | AI Inference Serving | capacity | GAIN, 2.26% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 80% full; reads the last 40 decisions |
| 36 | 67 | node pool max | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.25% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 40 decisions |
| 37 | 612 | edge dispatch | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.25% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 38 | 103 | asg max | Cloud VM & Capacity | capacity | GAIN, 2.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 39 | 106 | zone selection | Cloud VM & Capacity | capacity | GAIN, 2.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 40 | 609 | multi cluster dispatch | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 60 decisions |
| 41 | V5-5095 | game node pool autoscaler | Game Servers & Real-Time Interactive | capacity | GAIN, 2.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 42 | 168 | batch queue delay | AI Inference Serving | capacity | GAIN, 2.21% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 43 | 133 | gpu persistence mode | GPU Hardware | capacity | GAIN, 2.20% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 44 | 135 | mig mode | GPU Hardware | capacity | GAIN, 2.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 45 | 162 | model route weight | AI Inference Serving | setpoint | GAIN, 2.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 46 | V5-5036 | vgpu partition | Virtualization & Hypervisor | capacity | GAIN, 2.16% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 47 | 1033 | expander | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.15% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 80% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 48 | 85 | machine remediation | OpenShift & Machine API | capacity | GAIN, 2.13% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 49 | 621 | cross cluster replication | Cross-Cluster, Multi-Region & Edge | capacity | GAIN, 2.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 50 | AUDIT-0001 | dra device class selection | Kubernetes Dynamic Device Allocation | capacity | GAIN, 2.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 85% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 51 | 112 | interruption response | Cloud VM & Capacity | capacity | GAIN, 2.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 52 | 80 | capacity class | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 53 | 98 | vm size | Cloud VM & Capacity | capacity | GAIN, 2.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 75% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 54 | 1038 | target tracking policy | Cloud VM & Capacity | setpoint | GAIN, 2.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 55 | 78 | consolidate after | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 90% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 56 | 1031 | scale down unneeded time | Node Fleet & Karpenter-Class Control | capacity | GAIN, 2.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 57 | 104 | spot mix | Cloud VM & Capacity | capacity | GAIN, 2.05% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 58 | 600 | pulse phase | Quantum Computing Control Simulation | capacity | GAIN, 2.02% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 59 | 90 | mcp max unavailable | OpenShift & Machine API | capacity | GAIN, 2.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 60 | 1062 | chunked prefill | AI Inference Serving | capacity | GAIN, 2.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 80% full; reads the last 20 decisions |
| 61 | AUDIT-0008 | dra device configuration | Kubernetes Dynamic Device Allocation | setpoint | GAIN, 2.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 62 | 72 | node replace | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.99% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 63 | 1061 | kv cache swap space | AI Inference Serving | capacity | GAIN, 1.99% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 64 | 65 | node desired | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.97% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 65 | 604 | measurement scheduling | Quantum Computing Control Simulation | capacity | GAIN, 1.93% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 66 | 1036 | kubelet reserved resources | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.93% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 67 | 101 | instance family | Cloud VM & Capacity | capacity | GAIN, 1.92% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 68 | 77 | consolidation policy | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.91% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 20 decisions |
| 69 | 93 | cluster version pacing | OpenShift & Machine API | capacity | GAIN, 1.89% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 70 | 110 | boot disk class | Cloud VM & Capacity | capacity | GAIN, 1.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 71 | 1039 | scaling cooldown warmup | Cloud VM & Capacity | capacity | GAIN, 1.85% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 72 | 138 | gpu mps | GPU Hardware | capacity | GAIN, 1.84% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 73 | 74 | node power on | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.83% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 74 | 137 | gpu timeslice | GPU Hardware | capacity | GAIN, 1.83% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 75% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 75 | 1034 | node pool limits | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.81% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 76 | AUDIT-0003 | dra claim sharing | Kubernetes Dynamic Device Allocation | capacity | GAIN, 1.80% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 20 decisions |
| 77 | 170 | inference max tokens | AI Inference Serving | capacity | GAIN, 1.78% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 85% full; reads the last 60 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 78 | 129 | gpu allocate | GPU Hardware | capacity | GAIN, 1.77% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 79 | 136 | mig geometry | GPU Hardware | capacity | GAIN, 1.74% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 85% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 80 | 142 | gpu ecc response | GPU Hardware | capacity | GAIN, 1.74% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 81 | 79 | expire after | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.70% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 82 | 97 | vm count | Cloud VM & Capacity | capacity | GAIN, 1.70% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 83 | 134 | gpu compute mode | GPU Hardware | capacity | GAIN, 1.67% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 84 | AUDIT-0004 | dra device taint | Kubernetes Dynamic Device Allocation | capacity | GAIN, 1.66% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 85 | 1064 | disaggregated prefill decode | AI Inference Serving | capacity | GAIN, 1.65% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 86 | 163 | model load | AI Inference Serving | capacity | GAIN, 1.62% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 87 | 1120 | predictive scaling | Cloud VM & Capacity | capacity | GAIN, 1.56% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 88 | 1045 | gpu temperature target | GPU Hardware | setpoint | GAIN, 1.53% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 89 | V5-5038 | hypervisor cpu pinning | Virtualization & Hypervisor | capacity | GAIN, 1.49% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 90 | 1032 | scale down delay after add | Node Fleet & Karpenter-Class Control | capacity | GAIN, 1.49% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 75% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 91 | 102 | asg min | Cloud VM & Capacity | capacity | GAIN, 1.48% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 92 | 95 | machine failure domain | OpenShift & Machine API | capacity | GAIN, 1.42% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 93 | 164 | model unload | AI Inference Serving | capacity | GAIN, 1.40% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 94 | V5-5028 | fn snapstart restore | Serverless & Functions (FaaS) | capacity | GAIN, 1.38% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 95 | 64 | deschedule | Kubernetes Placement & Scheduling | capacity | GAIN, 1.35% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 96 | 1029 | cpu manager | Kubernetes Placement & Scheduling | capacity | GAIN, 1.35% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 97 | 1009 | vpa minmax allowed resources | Kubernetes Workload Scaling | capacity | GAIN, 1.33% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 98 | 18 | hpa cpu target | Kubernetes Workload Scaling | setpoint | GAIN, 1.32% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 75% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 99 | 140 | gpu reset | GPU Hardware | capacity | GAIN, 1.31% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 100 | 2179 | peaks power aware placement | Kubernetes Placement & Scheduling | capacity | GAIN, 1.28% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 101 | 1059 | scale to zero panic window | AI Inference Serving | setpoint | GAIN, 1.24% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 102 | 1020 | default limits | Container Resources | capacity | GAIN, 1.24% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 103 | V5-5018 | irqbalance affinity | Linux Kernel & OS Control | capacity | GAIN, 1.23% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 60 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 104 | 24 | rollout rate | Kubernetes Workload Scaling | capacity | GAIN, 1.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 105 | 1026 | scheduler scoring strategy | Kubernetes Placement & Scheduling | capacity | GAIN, 1.21% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 106 | 52 | topology spread | Kubernetes Placement & Scheduling | capacity | GAIN, 1.20% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 60 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 107 | 20 | hpa custom target | Kubernetes Workload Scaling | setpoint | GAIN, 1.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 108 | 1003 | hpa scale down stabilisation window | Kubernetes Workload Scaling | setpoint | GAIN, 1.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 109 | 1024 | io latency target | Container Resources | setpoint | GAIN, 1.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 110 | 105 | reservation mix | Cloud VM & Capacity | capacity | GAIN, 1.18% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 40 decisions |
| 111 | 25 | max surge | Kubernetes Workload Scaling | capacity | GAIN, 1.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 112 | 47 | cpuset | Container Resources | capacity | GAIN, 1.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 113 | 41 | memory high | Container Resources | capacity | GAIN, 1.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 114 | 45 | cpu weight | Container Resources | setpoint | GAIN, 1.16% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 115 | 1043 | swappiness reclaim | Host CPU & Memory | capacity | GAIN, 1.15% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 116 | V5-5020 | hugepage pool reserve | Linux Kernel & OS Control | capacity | GAIN, 1.15% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 20 decisions |
| 117 | 21 | vpa apply | Kubernetes Workload Scaling | capacity | GAIN, 1.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 118 | 40 | pids limit | Container Resources | capacity | GAIN, 1.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 119 | 36 | memory limit | Container Resources | capacity | GAIN, 1.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 120 | 51 | pod anti affinity | Kubernetes Placement & Scheduling | capacity | GAIN, 1.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 121 | 1022 | memory hard limit | Container Resources | capacity | GAIN, 1.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 122 | 2177 | goldilocks vpa request recommendation | Container Resources | capacity | GAIN, 1.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 75% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 123 | V5-5022 | fn provisioned concurrency | Serverless & Functions (FaaS) | capacity | GAIN, 1.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |
| 124 | 48 | runtime class | Container Resources | capacity | GAIN, 1.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 125 | 17 | replicas | Kubernetes Workload Scaling | capacity | GAIN, 1.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 126 | 57 | taint toleration | Kubernetes Placement & Scheduling | setpoint | GAIN, 1.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 127 | 60 | device claim | Kubernetes Placement & Scheduling | capacity | GAIN, 1.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 128 | V5-5025 | fn prewarm pool | Serverless & Functions (FaaS) | capacity | GAIN, 1.08% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |
| 129 | V5-5026 | fn reserved concurrency | Serverless & Functions (FaaS) | capacity | GAIN, 1.08% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |
| 130 | 1010 | keda polling interval | Kubernetes Workload Scaling | setpoint | GAIN, 1.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 131 | 1015 | graceful termination | Kubernetes Workload Scaling | capacity | GAIN, 1.04% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 132 | V5-5082 | drm license server pool | Media Streaming & CDN | capacity | GAIN, 1.04% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 85% full; reads the last 40 decisions |
| 133 | 1112 | core packing | Host CPU & Memory | capacity | GAIN, 1.02% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 134 | 1017 | in place resize | Container Resources | capacity | GAIN, 1.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 135 | V5-5001 | cgroup cpu weight | Linux Kernel & OS Control | capacity | GAIN, 1.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 136 | 121 | irq affinity | Host CPU & Memory | capacity | GAIN, 1.00% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 137 | 22 | scale to zero | Kubernetes Workload Scaling | capacity | GAIN, 0.99% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 80% full; reads the last 40 decisions |
| 138 | 2176 | cruisekube request rightsizing | Container Resources | capacity | GAIN, 0.98% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 139 | 119 | memory bandwidth | Host CPU & Memory | capacity | GAIN, 0.97% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 140 | 1027 | scheduler plugin weights | Kubernetes Placement & Scheduling | setpoint | GAIN, 0.97% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 40 decisions |
| 141 | 53 | numa placement | Kubernetes Placement & Scheduling | capacity | GAIN, 0.97% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 142 | V5-5030 | vcpu allocation | Virtualization & Hypervisor | capacity | GAIN, 0.97% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |
| 143 | 122 | llc allocation | Host CPU & Memory | capacity | GAIN, 0.96% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 144 | 1004 | hpa scale up stabilisation window | Kubernetes Workload Scaling | setpoint | GAIN, 0.95% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 145 | V5-5024 | fn memory size | Serverless & Functions (FaaS) | capacity | GAIN, 0.94% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 146 | 44 | cgroup io max | Container Resources | capacity | GAIN, 0.93% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 147 | 56 | network locality | Kubernetes Placement & Scheduling | capacity | GAIN, 0.93% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 148 | 1012 | rollout readiness delay | Kubernetes Workload Scaling | capacity | GAIN, 0.93% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 149 | 19 | hpa memory target | Kubernetes Workload Scaling | setpoint | GAIN, 0.92% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 150 | 124 | page reclaim rate | Host CPU & Memory | capacity | GAIN, 0.92% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 151 | V5-5077 | packager workers | Media Streaming & CDN | capacity | GAIN, 0.92% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 85% full; reads the last 60 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 152 | 34 | cpu limit | Container Resources | capacity | GAIN, 0.91% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 40 decisions |
| 153 | 1023 | memory protection | Container Resources | capacity | GAIN, 0.90% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 154 | 1016 | probes | Kubernetes Workload Scaling | capacity | GAIN, 0.89% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 155 | 43 | swap limit | Container Resources | capacity | GAIN, 0.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 156 | 39 | io weight | Container Resources | setpoint | GAIN, 0.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 157 | 1005 | hpa scaling rate policies | Kubernetes Workload Scaling | capacity | GAIN, 0.87% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 158 | V5-5094 | voice chat mixer pool | Game Servers & Real-Time Interactive | capacity | GAIN, 0.85% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |
| 159 | 33 | cpu request | Container Resources | capacity | GAIN, 0.85% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 160 | 37 | ephemeral storage | Container Resources | capacity | GAIN, 0.85% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 161 | 1002 | hpa max replicas | Kubernetes Workload Scaling | capacity | GAIN, 0.85% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 162 | V5-5086 | agones fleet buffer size | Game Servers & Real-Time Interactive | capacity | GAIN, 0.85% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 163 | 603 | reset scheduling | Quantum Computing Control Simulation | capacity | GAIN, 0.84% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 164 | 30 | pdb policy | Kubernetes Workload Scaling | capacity | GAIN, 0.83% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 165 | V5-5087 | agones fleet min replicas | Game Servers & Real-Time Interactive | capacity | GAIN, 0.83% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 60 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 166 | V5-5031 | memory ballooning | Virtualization & Hypervisor | capacity | GAIN, 0.83% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |
| 167 | 597 | gate scheduling | Quantum Computing Control Simulation | capacity | GAIN, 0.82% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 168 | 38 | hugepages | Container Resources | capacity | GAIN, 0.81% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 169 | 54 | gpu topology | Kubernetes Placement & Scheduling | capacity | GAIN, 0.81% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 20 decisions |
| 170 | 1001 | hpa min replicas | Kubernetes Workload Scaling | capacity | GAIN, 0.81% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 171 | 2172 | kepler energy target autoscaling | Kubernetes Workload Scaling | setpoint | GAIN, 0.81% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 172 | 1041 | turbo boost | Host CPU & Memory | capacity | GAIN, 0.80% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 173 | V5-5002 | cgroup cpu quota | Linux Kernel & OS Control | capacity | GAIN, 0.80% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 174 | V5-5003 | cgroup memory high | Linux Kernel & OS Control | capacity | GAIN, 0.79% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 10 decisions |
| 175 | 55 | storage locality | Kubernetes Placement & Scheduling | capacity | GAIN, 0.78% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 176 | 49 | node selector | Kubernetes Placement & Scheduling | capacity | GAIN, 0.78% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 85% full; reads the last 10 decisions |
| 177 | 1006 | hpa policy selection | Kubernetes Workload Scaling | capacity | GAIN, 0.77% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 178 | 42 | memory reclaim | Container Resources | capacity | GAIN, 0.76% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 179 | V5-5101 | script validation threads | Blockchain & Ledger Nodes | capacity | GAIN, 0.76% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 10 decisions |
| 180 | 1007 | hpa tolerance | Kubernetes Workload Scaling | capacity | GAIN, 0.75% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 181 | 599 | pulse duration | Quantum Computing Control Simulation | setpoint | GAIN, 0.74% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 182 | 61 | failure domain spread | Kubernetes Placement & Scheduling | capacity | GAIN, 0.74% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 183 | 125 | transparent hugepages | Host CPU & Memory | capacity | GAIN, 0.72% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 184 | 35 | memory request | Container Resources | capacity | GAIN, 0.72% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 185 | 120 | numa balance | Host CPU & Memory | capacity | GAIN, 0.71% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 186 | V5-5021 | fn concurrency limit | Serverless & Functions (FaaS) | capacity | GAIN, 0.65% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 187 | 126 | core online offline | Host CPU & Memory | capacity | GAIN, 0.65% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 188 | 593 | qubit mapping | Quantum Computing Control Simulation | capacity | GAIN, 0.65% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 189 | 1008 | hpa sync period | Kubernetes Workload Scaling | capacity | GAIN, 0.63% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 190 | 26 | max unavailable | Kubernetes Workload Scaling | capacity | GAIN, 0.61% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 191 | V5-5078 | origin shield cache size | Media Streaming & CDN | capacity | GAIN, 0.61% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 192 | V5-5056 | tls session cache | Skin: Perimeter & Admission | capacity | GAIN, 0.59% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 193 | 605 | dynamical decoupling | Quantum Computing Control Simulation | capacity | GAIN, 0.55% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 194 | 595 | shot allocation | Quantum Computing Control Simulation | capacity | GAIN, 0.54% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 195 | V5-5011 | tcp buffer autotune | Linux Kernel & OS Control | capacity | GAIN, 0.53% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 60 decisions |
| 196 | 606 | noise aware routing | Quantum Computing Control Simulation | capacity | GAIN, 0.51% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 197 | 50 | node affinity | Kubernetes Placement & Scheduling | capacity | GAIN, 0.50% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 198 | V5-5093 | relay server pool | Game Servers & Real-Time Interactive | capacity | GAIN, 0.43% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 199 | V5-5007 | io scheduler depth | Linux Kernel & OS Control | capacity | GAIN, 0.38% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 200 | V5-5012 | block nr requests | Linux Kernel & OS Control | capacity | GAIN, 0.34% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 201 | 602 | coupling control | Quantum Computing Control Simulation | capacity | GAIN, 0.33% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 202 | V5-5014 | net rx coalesce | Linux Kernel & OS Control | capacity | GAIN, 0.32% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 203 | V5-5106 | archive node storage tier | Blockchain & Ledger Nodes | capacity | GAIN, 0.31% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 204 | V5-5098 | node db cache size | Blockchain & Ledger Nodes | capacity | GAIN, 0.31% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 205 | 148 | rdma bandwidth | GPU Fabric & RDMA | capacity | GAIN, 0.26% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 75% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 206 | AUDIT-0023 | dpu sf tx rate | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.26% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 207 | 150 | nic affinity | GPU Fabric & RDMA | capacity | GAIN, 0.25% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 208 | AUDIT-0027 | smartnic flow steering | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.25% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 80% full; reads the last 10 decisions |
| 209 | AUDIT-0031 | dpu service placement | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.24% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 210 | V5-5100 | max peer connections | Blockchain & Ledger Nodes | capacity | GAIN, 0.24% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 211 | 598 | pulse amplitude | Quantum Computing Control Simulation | capacity | GAIN, 0.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 212 | V5-5035 | sriov vf allocation | Virtualization & Hypervisor | capacity | GAIN, 0.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 213 | 156 | congestion response | GPU Fabric & RDMA | capacity | GAIN, 0.22% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 214 | V5-5111 | build cache size | CI/CD & Build Farms | capacity | GAIN, 0.21% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 215 | 145 | nvlink placement | GPU Fabric & RDMA | capacity | GAIN, 0.20% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 216 | AUDIT-0026 | sriov vf count | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 217 | AUDIT-0028 | smartnic offload enable | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 218 | 146 | nvswitch route | GPU Fabric & RDMA | capacity | GAIN, 0.18% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 219 | 157 | rail selection | GPU Fabric & RDMA | capacity | GAIN, 0.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 220 | 1068 | collective protocol | GPU Fabric & RDMA | capacity | GAIN, 0.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 221 | 154 | rank placement | GPU Fabric & RDMA | capacity | GAIN, 0.16% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 222 | 155 | gpudirect policy | GPU Fabric & RDMA | capacity | GAIN, 0.15% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 223 | AUDIT-0021 | dpu pf tx rate | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.13% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 224 | 1069 | nic hca selection | GPU Fabric & RDMA | capacity | GAIN, 0.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 225 | AUDIT-0022 | dpu vf tx rate | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 226 | AUDIT-0024 | dpu bandwidth share | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 227 | AUDIT-0025 | dpu qos group | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 228 | 152 | collective algorithm | GPU Fabric & RDMA | capacity | GAIN, 0.11% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 229 | 158 | fabric failover | GPU Fabric & RDMA | capacity | GAIN, 0.08% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 230 | 149 | rdma route | GPU Fabric & RDMA | capacity | GAIN, 0.07% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 231 | AUDIT-0032 | dpu failover | DPU SmartNIC & Programmable IO | capacity | GAIN, 0.07% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 232 | 153 | collective chunk size | GPU Fabric & RDMA | capacity | GAIN, 0.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 233 | V5-5074 | transcoder worker pool | Media Streaming & CDN | capacity | GAIN, the line cleared 56.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 234 | 186 | checkpoint trigger | AI Training | capacity | GAIN, the line cleared 54.5% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 235 | 190 | straggler mitigation | AI Training | capacity | GAIN, the line cleared 53.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 20 decisions |
| 236 | 195 | rank mapping | HPC & Distributed Compute | capacity | GAIN, the line cleared 53.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 237 | 217 | worker allocation | Distributed Cluster Managers | capacity | GAIN, the line cleared 52.9% sooner | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 238 | 208 | backfill policy | HPC & Distributed Compute | capacity | GAIN, the line cleared 52.0% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 239 | 177 | training workers | AI Training | capacity | GAIN, the line cleared 51.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 240 | 181 | data parallelism | AI Training | capacity | GAIN, the line cleared 51.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 60 decisions |
| 241 | 193 | job slots | HPC & Distributed Compute | capacity | GAIN, the line cleared 50.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 242 | 180 | gradient accumulation | AI Training | capacity | GAIN, the line cleared 50.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 243 | 211 | resource reservation | Distributed Cluster Managers | capacity | GAIN, the line cleared 50.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 80% full; reads the last 10 decisions |
| 244 | 197 | job walltime | HPC & Distributed Compute | capacity | GAIN, the line cleared 50.2% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 245 | 200 | checkpoint restart | HPC & Distributed Compute | capacity | GAIN, the line cleared 49.9% sooner | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 80% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 246 | 185 | checkpoint interval | AI Training | setpoint | GAIN, the line cleared 49.8% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 10 decisions |
| 247 | 194 | mpi ranks | HPC & Distributed Compute | capacity | GAIN, the line cleared 49.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 248 | V5-5083 | thumbnail sprite batch | Media Streaming & CDN | capacity | GAIN, the line cleared 48.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 80% full; reads the last 10 decisions |
| 249 | 196 | node allocation | HPC & Distributed Compute | capacity | GAIN, the line cleared 48.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 250 | V5-5105 | indexer batch workers | Blockchain & Ledger Nodes | capacity | GAIN, the line cleared 48.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 251 | 203 | accelerator share | HPC & Distributed Compute | capacity | GAIN, the line cleared 47.9% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 20 decisions |
| 252 | 183 | pipeline parallelism | AI Training | capacity | GAIN, the line cleared 47.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 10 decisions |
| 253 | 219 | oversubscription | Distributed Cluster Managers | capacity | GAIN, the line cleared 47.7% sooner | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 75% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 254 | 184 | expert parallelism | AI Training | capacity | GAIN, the line cleared 47.5% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 85% full; reads the last 20 decisions |
| 255 | 182 | tensor parallelism | AI Training | capacity | GAIN, the line cleared 47.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 75% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 256 | 178 | global batch size | AI Training | capacity | GAIN, the line cleared 47.2% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 85% full; reads the last 10 decisions |
| 257 | 220 | resource reclaim | Distributed Cluster Managers | capacity | GAIN, the line cleared 46.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 85% full; reads the last 20 decisions |
| 258 | V5-5112 | test shard parallelism | CI/CD & Build Farms | capacity | GAIN, the line cleared 46.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 259 | V5-5108 | runner scale set min runners | CI/CD & Build Farms | capacity | GAIN, the line cleared 46.2% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 260 | 204 | cpu gpu ratio | HPC & Distributed Compute | setpoint | GAIN, the line cleared 45.9% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 261 | 216 | gang schedule | Distributed Cluster Managers | capacity | GAIN, the line cleared 45.8% sooner | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 90% full; reads the last 60 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 262 | V5-5116 | remote execution workers | CI/CD & Build Farms | capacity | GAIN, the line cleared 45.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |
| 263 | 1072 | partition limits | HPC & Distributed Compute | capacity | GAIN, the line cleared 45.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 264 | 214 | binpack pressure | Distributed Cluster Managers | capacity | GAIN, the line cleared 45.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 85% full; reads the last 10 decisions |
| 265 | 189 | elastic worker count | AI Training | capacity | GAIN, the line cleared 45.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 90% full; reads the last 10 decisions |
| 266 | 215 | spread pressure | Distributed Cluster Managers | capacity | GAIN, the line cleared 45.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | machines started at 80% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 267 | 218 | maintenance evacuation | Distributed Cluster Managers | capacity | GAIN, the line cleared 45.2% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 90% full; reads the last 10 decisions |
| 268 | V5-5096 | replay render batch | Game Servers & Real-Time Interactive | capacity | GAIN, the line cleared 44.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 269 | 179 | microbatch size | AI Training | capacity | GAIN, the line cleared 44.2% sooner | starts the machines the forecast needs before the rush lands and clears | Compute | machines started at 90% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 270 | 1067 | dataloader workers | AI Training | capacity | GAIN, the line cleared 44.0% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 70% full; reads the last 40 decisions |
| 271 | 188 | training gang size | AI Training | capacity | GAIN, the line cleared 43.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 80% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 272 | 192 | compute comm overlap | AI Training | capacity | GAIN, the line cleared 42.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | machines started at 80% full; reads the last 40 decisions |
| 273 | 1066 | activation checkpointing | AI Training | capacity | GAIN, the line cleared 41.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute | global |
| 274 | 205 | memory per rank | HPC & Distributed Compute | capacity | GAIN, the line cleared 41.2% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native runs; hands back what it no longer needs at once | | |
| 275 | V5-5004 | cfs scheduler latency | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 276 | V5-5006 | transparent hugepages | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 277 | V5-5008 | numa auto balancing | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 278 | V5-5009 | dirty ratio writeback | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 279 | V5-5010 | vm swappiness | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 280 | V5-5016 | zswap pool fraction | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 281 | V5-5017 | page cache reclaim | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 282 | V5-5019 | net qdisc codel target | Linux Kernel & OS Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 283 | V5-5029 | fn timeout budget | Serverless & Functions (FaaS) | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 284 | V5-5034 | numa vcpu pinning | Virtualization & Hypervisor | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 285 | V5-5046 | dead letter retry cap | Gut: Data-plane Intake & Waste | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 286 | V5-5048 | ttl eviction waste | Gut: Data-plane Intake & Waste | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 287 | V5-5051 | retry budget window | Gut: Data-plane Intake & Waste | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 288 | V5-5072 | abr bitrate ladder cap | Media Streaming & CDN | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 289 | V5-5073 | player buffer target | Media Streaming & CDN | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 290 | V5-5079 | edge cache ttl | Media Streaming & CDN | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 291 | V5-5080 | segment duration target | Media Streaming & CDN | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 292 | V5-5085 | encoder lookahead frames | Media Streaming & CDN | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 293 | V5-5089 | game server tick rate | Game Servers & Real-Time Interactive | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 294 | V5-5092 | matchmaking wait window | Game Servers & Real-Time Interactive | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 295 | V5-5099 | mempool max size | Blockchain & Ledger Nodes | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 296 | V5-5104 | utxo cache flush interval | Blockchain & Ledger Nodes | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 297 | V5-5114 | artifact retention ttl | CI/CD & Build Farms | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 298 | V5-5130 | edge sync interval | Cross-Cluster, Multi-Region & Edge | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Compute | global |
| 299 | 1 | api concurrency | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 300 | 2 | queue concurrency | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 301 | 3 | queue backpressure | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 302 | 4 | job admission | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 303 | 5 | batch admission | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 304 | 6 | inference admission | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 305 | 7 | load shed | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 306 | 8 | priority gate | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 307 | 9 | tenant admission | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 308 | 10 | burst limit | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 309 | 11 | deadline admission | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 310 | 12 | work budget | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 311 | 13 | request queue limit | Work Admission & Demand | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | Shaping | | | left native | | |
| 312 | 14 | retry admission | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 313 | 15 | background work gate | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 314 | 16 | maintenance work gate | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 315 | 23 | keda threshold | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 316 | 27 | deployment pause resume | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 317 | 28 | rollout abort | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 318 | 29 | pod eviction | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 319 | 31 | scheduler queue priority | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 320 | 32 | api priority fairness | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 321 | 46 | cpu quota | Container Resources | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 322 | 58 | priority class | Kubernetes Placement & Scheduling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 323 | 59 | preemption policy | Kubernetes Placement & Scheduling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 324 | 62 | scheduler backoff | Kubernetes Placement & Schedul | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | ing | | | left native | | |
| 325 | 63 | gang admission | Kubernetes Placement & Scheduling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 326 | 69 | node cordon | Node Fleet & Karpenter-Class Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 327 | 70 | node drain | Node Fleet & Karpenter-Class Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 328 | 76 | disruption budget | Node Fleet & Karpenter-Class Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 329 | 88 | machine health gate | OpenShift & Machine API | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 330 | 89 | mcp pause | OpenShift & Machine API | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 331 | 92 | operator reconcile budget | OpenShift & Machine API | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 332 | 94 | infra machine admission | OpenShift & Machine API | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 333 | 113 | cpufreq min | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 334 | 114 | cpufreq max | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 335 | 115 | rapl package power | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 336 | 116 | uncore frequency | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 337 | 117 | energy perf preference | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 338 | 118 | cpu idle policy | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 339 | 123 | memory pressure gate | Host CPU & Memory | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 340 | 127 | thermal throttle policy | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 341 | 128 | host power profile | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 342 | 130 | gpu power limit | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 343 | 131 | gpu sm clock | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 344 | 132 | gpu memory clock | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 345 | 139 | gpu quarantine | GPU Hardware | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 346 | 141 | gpu thermal limit | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 347 | 143 | gpu job power budget | GPU Hardware | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 348 | 147 | gpu fabric quarantine | GPU Fabric & RDMA | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 349 | 151 | collective concurrency | GPU Fabric & RDMA | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 350 | 159 | communication priority | GPU Fabric & RDMA | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 351 | 160 | fabric isolation | GPU Fabric & RDMA | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 352 | 169 | inference concurrency | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 353 | 171 | kv cache budget | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | left native | | |
| 354 | 172 | prefix cache budget | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 355 | 173 | speculative decode budget | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 356 | 174 | model precision | AI Inference Serving | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 357 | 175 | inference priority | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 358 | 176 | inference slo gate | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 359 | 187 | training preempt | AI Training | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 360 | 191 | training precision | AI Training | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 361 | 198 | job priority | HPC & Distributed Compute | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 362 | 199 | job preemption | HPC & Distributed Compute | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 363 | 201 | parallel io budget | HPC & Distributed Compute | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 364 | 202 | collective budget | HPC & Distributed Compute | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 365 | 206 | scratch budget | HPC & Distributed Compute | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 366 | 207 | scheduler fair share | HPC & Distributed Compute | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 367 | 209 | task admission | Distributed Cluster Managers | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | left native | | |
| 368 | 210 | task priority | Distributed Cluster Managers | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 369 | 212 | cluster quota | Distributed Cluster Managers | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 370 | 213 | task preemption | Distributed Cluster Managers | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 371 | 221 | queue fairness | Distributed Cluster Managers | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 372 | 222 | deadline pressure | Distributed Cluster Managers | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 373 | 223 | scheduler retry | Distributed Cluster Managers | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 374 | 594 | circuit admission | Quantum Computing Control Simulation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 375 | 596 | circuit scheduling | Quantum Computing Control Simulation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 376 | 601 | pulse frequency | Quantum Computing Control Simulation | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 377 | 607 | error mitigation budget | Quantum Computing Control Simulation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 378 | 608 | quantum queue priority | Quantum Computing Control Simulation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 379 | 615 | data residency gate | Cross-Cluster, Multi-Region & Edge | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 380 | 616 | latency region gate | Cross-Cluster, Multi-Region & Edge | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 381 | 617 | cost region gate | Cross-Cluster, Multi-Region & Edge | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 382 | 618 | carbon region gate | Cross-Cluster, Multi-Region & Edge | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 383 | 620 | federation quota | Cross-Cluster, Multi-Region & Edge | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 384 | 623 | global admission | Cross-Cluster, Multi-Region & Edge | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 385 | 1011 | rollout progress deadline | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 386 | 1014 | unhealthy pod eviction policy | Kubernetes Workload Scaling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 387 | 1019 | namespace quota | Container Resources | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | left native | | |
| 388 | 1025 | freeze | Container Resources | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 389 | 1028 | scheduling gates | Kubernetes Placement & Scheduling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 390 | 1030 | scale down utilisation threshold | Node Fleet & Karpenter-Class Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 391 | 1035 | kubelet eviction thresholds | Node Fleet & Karpenter-Class Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 392 | 1040 | frequency governor | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 393 | 1042 | dram power limit | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 394 | 1044 | application clocks | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 395 | 1058 | autoscaling target concurrency qps | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 396 | 1060 | request priority levels | AI Inference Serving | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 397 | 1070 | node power saving | HPC & Distributed Compute | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 398 | 1071 | cpu frequency per job | HPC & Distributed Compute | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 399 | 1074 | request concurrency limit | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 400 | 1114 | vm cpu cap by priority | Cloud VM & Capacity | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 401 | 1115 | per pod cpu frequency range and c state access | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 402 | 1116 | uncore frequency per node | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 403 | 1118 | target clocks | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 404 | 1123 | adaptive concurrency limit | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 405 | 1124 | priority request queue under limit | Work Admission & Demand Shaping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 406 | 2173 | climatik inference power cap | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 407 | 2174 | ear cpu frequency policy | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 408 | 2175 | ear global energy budget | Host CPU & Memory | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 409 | 2178 | k8s power manager core profile | Host CPU & Memory | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 410 | 2180 | adaptive gpu power cap | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 411 | 2181 | nvidia platform power budget | GPU Hardware | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 412 | AUDIT-0005 | dra device eviction | Kubernetes Dynamic Device Allocation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 413 | AUDIT-0029 | dpu cpu budget | DPU SmartNIC & Programmable IO | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 414 | AUDIT-0030 | dpu memory budget | DPU SmartNIC & Programmable IO | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 415 | V5-5005 | cpufreq dvfs governor | Linux Kernel & OS Control | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 416 | V5-5013 | ksm memory dedup | Linux Kernel & OS Control | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Compute | global |
| 417 | V5-5015 | cpu cstate governor | Linux Kernel & OS Control | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 418 | V5-5023 | fn scale to zero idle | Serverless & Functions (FaaS) | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 419 | V5-5027 | fn burst admission | Serverless & Functions (FaaS) | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 420 | V5-5033 | live migration pacing | Virtualization & Hypervisor | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 421 | V5-5037 | vm memory dedup ksm | Virtualization & Hypervisor | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Compute | global |
| 422 | V5-5045 | ingress intake pacing | Gut: Data-plane Intake & Waste | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 423 | V5-5047 | gc compaction pacing | Gut: Data-plane Intake & Waste | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Compute | global |
| 424 | V5-5049 | backpressure threshold | Gut: Data-plane Intake & Waste | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 425 | V5-5050 | queue drain rate | Gut: Data-plane Intake & Waste | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Compute | global |
| 426 | V5-5052 | compaction debt drain | Gut: Data-plane Intake & Waste | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Compute | global |
| 427 | V5-5053 | rate limit token bucket | Skin: Perimeter & Admission | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 428 | V5-5054 | connection admission | Skin: Perimeter & Admission | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 429 | V5-5055 | waf inspection sampling | Skin: Perimeter & Admission | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 430 | V5-5057 | ddos shield admission | Skin: Perimeter & Admission | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 431 | V5-5075 | live transcode gpu clocks | Media Streaming & CDN | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 432 | V5-5076 | vod transcode batch power | Media Streaming & CDN | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 433 | V5-5081 | cdn request coalescing | Media Streaming & CDN | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 434 | V5-5084 | media ingest admission | Media Streaming & CDN | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 435 | V5-5090 | players per server cap | Game Servers & Real-Time Interactive | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 436 | V5-5091 | matchmaking admission rate | Game Servers & Real-Time Interactive | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 437 | V5-5097 | cloud gaming gpu clocks | Game Servers & Real-Time Interactive | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 438 | V5-5102 | block relay admission | Blockchain & Ledger Nodes | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 439 | V5-5103 | rpc work queue depth | Blockchain & Ledger Nodes | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 440 | V5-5107 | validator host power profile | Blockchain & Ledger Nodes | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |
| 441 | V5-5110 | build concurrency limit | CI/CD & Build Farms | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 442 | V5-5113 | job queue admission | CI/CD & Build Farms | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Compute | global |
| 443 | V5-5115 | build node cpu frequency | CI/CD & Build Farms | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 444 | V5-5129 | edge inference gpu clocks | Cross-Cluster, Multi-Region & Edge | power | at its limit on native | at its limit on native (the full clock): read and left native | Compute | global |

### *E.2.2 Physics / Robotics / Autonomous: 137 muscles, heaviest hitter first*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 1 | 1078 | controller update rate | Robotics Motion Control | capacity | GAIN, 12.97% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 2 | V5-5139 | humanoid gait speed | Robotics Motion Control | capacity | GAIN, 12.65% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 3 | 440 | jerk limit | Robotics Motion Control | capacity | GAIN, 12.10% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 4 | AUDIT-0051 | vehicle speed envelope | Automotive EV & Mobile Powertrain | capacity | GAIN, 10.86% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 5 | 1077 | velocity acceleration scaling | Robotics Motion Control | capacity | GAIN, 9.33% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 6 | 434 | joint velocity | Robotics Motion Control | capacity | GAIN, 7.81% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 7 | 433 | joint position | Robotics Motion Control | capacity | GAIN, 7.71% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 8 | 447 | balance correction | Robotics Motion Control | capacity | GAIN, 5.00% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 9 | V5-5149 | drone battery failsafe reserve | Aviation & Autonomous Flight | setpoint | GAIN, 3.74% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Physics | 1.5 wobbles of margin on its line |
| 10 | V5-5141 | humanoid step height | Robotics Motion Control | capacity | GAIN, 3.65% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 11 | V5-5151 | amr fleet charge threshold | Robotics Fleet & Warehouse Automation | setpoint | GAIN, 3.53% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Physics | 1.5 wobbles of margin on its line |
| 12 | 444 | locomotion speed | Robotics Motion Control | capacity | GAIN, 3.44% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | (alone; inside a body, its everyday speed) | | |
| 13 | 1076 | trajectory tolerances | Robotics Motion Control | capacity | GAIN, 3.12% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 14 | 435 | joint acceleration | Robotics Motion Control | capacity | GAIN, 3.04% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 15 | V5-5142 | manipulator payload speed scaling | Robotics Motion Control | capacity | GAIN, 2.08% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 16 | V5-5138 | humanoid balance margin | Robotics Motion Control | capacity | GAIN, 1.82% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 17 | V5-5150 | swarm formation spacing | Aviation & Autonomous Flight | capacity | GAIN, 1.51% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 18 | 441 | collision margin | Robotics Motion Control | capacity | GAIN, 1.40% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 19 | 2068 | shaft speed target | Marine Propulsion & Vessel Automation | capacity | GAIN, 1.40% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 20 | V5-5135 | ros2 executor threads | Robotics Motion Control | capacity | GAIN, 1.33% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | machines started at 70% full; reads the last 40 decisions |
| 21 | V5-5147 | ardupilot wpnav speed | Aviation & Autonomous Flight | capacity | GAIN, 1.15% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 22 | 437 | cartesian velocity | Robotics Motion Control | capacity | GAIN, 1.04% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 23 | 438 | trajectory speed | Robotics Motion Control | capacity | GAIN, 1.00% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 24 | 439 | trajectory acceleration | Robotics Motion Control | capacity | GAIN, 0.89% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 25 | V5-5143 | px4 mpc xy vel max | Aviation & Autonomous Flight | capacity | GAIN, 0.74% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 26 | 445 | steering angle | Robotics Motion Control | capacity | GAIN, 0.57% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 27 | 2073 | engine load sharing setpoint | Marine Propulsion & Vessel Automation | capacity | GAIN, 0.46% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 28 | 2078 | trim target | Marine Propulsion & Vessel Automation | capacity | GAIN, 0.40% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 29 | 2075 | auxiliary engine staging | Marine Propulsion & Vessel Automation | capacity | GAIN, 0.28% of decisions back under the line | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 30 | 458 | agv speed | Robotics Fleet & Warehouse Automation | capacity | GAIN, 0.18% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 31 | 457 | conveyor speed | Robotics Fleet & Warehouse Automation | capacity | GAIN, 0.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |
| 32 | 450 | task assignment | Robotics Fleet & Warehouse Automation | capacity | GAIN, 0.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |
| 33 | 451 | traffic reservation | Robotics Fleet & Warehouse Automation | capacity | GAIN, 0.03% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 34 | 455 | elevator request | Robotics Fleet & Warehouse Automation | capacity | GAIN, 0.03% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |
| 35 | V5-5152 | amr fleet active robots | Robotics Fleet & Warehouse Automation | capacity | GAIN, 0.03% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 36 | 452 | robot route | Robotics Fleet & Warehouse Automation | capacity | GAIN, 0.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |
| 37 | V5-5145 | px4 mpc acc hor | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 10.5% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 38 | 485 | altitude target | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 10.5% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 39 | 486 | waypoint authority | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 10.3% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 40 | V5-5146 | px4 mc rollrate max | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 10.0% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 41 | 494 | actuator saturation envelope | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 9.9% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 42 | V5-5144 | px4 mpc z vel max up | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 9.9% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 43 | 484 | velocity target | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 9.8% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 44 | 483 | attitude rate target | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 9.8% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 45 | 482 | attitude target | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 9.6% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 46 | 1079 | attitude rate gains | Aviation & Autonomous Flight | capacity | GAIN, the line cleared 9.5% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 47 | 454 | battery reserve | Robotics Fleet & Warehouse Automation | capacity | GAIN, the line cleared 7.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 48 | 449 | robot dispatch | Robotics Fleet & Warehouse Automation | capacity | GAIN, the line cleared 7.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |
| 49 | 2074 | slow steaming speed target | Marine Propulsion & Vessel Automation | capacity | GAIN, the line cleared 6.0% sooner | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics | global |
| 50 | 462 | human safe stop | Robotics Fleet & Warehouse Automation | capacity | GAIN, the line cleared 4.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | back what it no longer needs at once | | |
| 51 | 456 | door request | Robotics Fleet & Warehouse Automation | capacity | GAIN, the line cleared 4.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |
| 52 | 453 | charging dispatch | Robotics Fleet & Warehouse Automation | capacity | GAIN, the line cleared 3.9% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics | global |
| 53 | 461 | fleet failover | Robotics Fleet & Warehouse Automation | capacity | GAIN, the line cleared 3.1% sooner | starts the machines the forecast needs before the rush | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 54 | V5-5136 | ros2 qos history depth | Robotics Motion Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Physics | global |
| 55 | V5-5137 | ros2 control loop rate | Robotics Motion Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Physics | global |
| 56 | 436 | joint effort | Robotics Motion Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 57 | 442 | force limit | Robotics Motion Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 58 | 443 | gripper force | Robotics Motion Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 59 | 446 | braking force | Robotics Motion Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 60 | 459 | warehouse zone admission | Robotics Fleet & Warehouse Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 61 | 460 | robot quarantine | Robotics Fleet & Warehouse Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 62 | 463 | fleet concurrency | Robotics Fleet & Warehouse Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 63 | 481 | throttle envelope | Aviation & Autonomous Flight | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 64 | 487 | flight hold | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 65 | 488 | return to home | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | left native | | |
| 66 | 489 | land action | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 67 | 490 | mission admission | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 68 | 491 | geofence response | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 69 | 492 | failsafe selection | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 70 | 493 | battery reserve threshold | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 71 | 495 | flight mode transition | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 72 | 496 | flight termination safe state | Aviation & Autonomous Flight | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 73 | 497 | space command admission | Spacecraft & Flight Software | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 74 | 499 | flight task schedule | Spacecraft & Flight Software | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 75 | 500 | space mode transition | Spacecraft & Flight Software | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 76 | 501 | payload duty cycle | Spacecraft & Flight Software | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 77 | 502 | communication allocation | Spacecraft & Flight Software | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 78 | 503 | space power budget | Spacecraft & Flight Software | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 79 | 504 | space thermal command | Spacecraft & Flight Software | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 80 | 505 | attitude command envelope | Spacecraft & Flight Software | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 81 | 506 | reaction wheel allocation | Spacecraft & Flight Software | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 82 | 507 | rcs authority | Spacecraft & Flight Software | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 83 | 508 | safe mode transition | Spacecraft & Flight Software | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 84 | 509 | watchdog recovery | Spacecraft & Flight Software | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 85 | 510 | instrument activation | Spacecraft & Flight Software | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 86 | 511 | fault isolation | Spacecraft & Flight Software | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 87 | 1080 | horizontal speed limit | Aviation & Autonomous Flight | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 88 | 1081 | vertical speed limits | Aviation & Autonomous Flight | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 89 | 1082 | acceleration jerk limits | Aviation & Autonomous Flight | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 90 | 1083 | state of charge limits | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 91 | 1084 | smart charging schedule | Automotive EV & Mobile Powertrain | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 92 | 2054 | traction effort limit | Rail Traction & Train Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 93 | 2055 | regenerative braking share | Rail Traction & Train Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 94 | 2056 | coasting speed target | Rail Traction & Train Control | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 95 | 2057 | dwell time schedule | Rail Traction & Train Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 96 | 2058 | hvac duty cycle | Rail Traction & Train Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 97 | 2059 | train auxiliary power budget | Rail Traction & Train Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 98 | 2060 | platform approach speed | Rail Traction & Train Control | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 99 | 2061 | acceleration rate setpoint | Rail Traction & Train Control | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 100 | 2062 | wheel slip protection mode | Rail Traction & Train Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 101 | 2063 | traction motor thermal derate | Rail Traction & Train Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 102 | 2064 | catenary voltage limit | Rail Traction & Train Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 103 | 2065 | timetable recovery margin | Rail Traction & Train Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 104 | 2066 | headway target | Rail Traction & Train Control | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 105 | 2067 | door release hold | Rail Traction & Train Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 106 | 2069 | propeller pitch limit | Marine Propulsion & Vessel Automation | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 107 | 2070 | shaft power limit | Marine Propulsion & Vessel Automation | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 108 | 2071 | bow thruster duty | Marine Propulsion & Vessel Automation | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 109 | 2072 | ballast pump mode | Marine Propulsion & Vessel Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 110 | 2076 | shore power mode | Marine Propulsion & Vessel Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | left native | | |
| 111 | 2077 | hotel load budget | Marine Propulsion & Vessel Automation | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 112 | 2079 | rudder rate limit | Marine Propulsion & Vessel Automation | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 113 | 2146 | hoist speed target | Elevators & Vertical Transport | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 114 | 2147 | acceleration limit | Elevators & Vertical Transport | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 115 | 2148 | regen drive mode | Elevators & Vertical Transport | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 116 | 2149 | standby power mode | Elevators & Vertical Transport | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 117 | 2150 | destination dispatch schedule | Elevators & Vertical Transport | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 118 | 2151 | car parking mode | Elevators & Vertical Transport | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 119 | 2152 | door dwell hold | Elevators & Vertical Transport | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 120 | 2153 | escalator speed target | Elevators & Vertical Transport | capacity | at its limit on native | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics | global |
| 121 | 2154 | group capacity mode | Elevators & Vertical Transport | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 122 | 2155 | motor thermal derate | Elevators & Vertical Transport | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 123 | 2156 | brake test schedule | Elevators & Vertical Transport | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 124 | 2157 | fire recall mode | Elevators & Vertical Transport | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 125 | AUDIT-0045 | traction torque limit | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 126 | AUDIT-0046 | regen braking level | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 127 | AUDIT-0047 | battery charge limit | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 128 | AUDIT-0048 | battery discharge limit | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 129 | AUDIT-0049 | battery thermal target | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 130 | AUDIT-0050 | motor thermal limit | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 131 | AUDIT-0052 | energy recovery target | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 132 | AUDIT-0053 | auxiliary power budget | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 133 | AUDIT-0054 | fast charge current | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 134 | AUDIT-0055 | fast charge voltage | Automotive EV & Mobile Powertrain | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |
| 135 | AUDIT-0056 | vehicle safe state | Automotive EV & Mobile Powertrain | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Physics | global |
| 136 | V5-5140 | humanoid joint torque limit | Robotics Motion Control | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 137 | V5-5148 | ardupilotatc accel r max | Aviation & Autonomous Flight | power | at its limit on native | at its limit on native (full torque): read and left native | Physics | global |

### *E.2.3 Energy / Facility / Industrial: 249 muscles, heaviest hitter first*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 1 | 568 | site battery dispatch | Facility & Grid Optimization | capacity | GAIN, 9.97% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0 wobbles of margin on its line |
| 2 | 566 | renewable dispatch | Facility & Grid Optimization | capacity | GAIN, 9.21% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0 wobbles of margin on its line |
| 3 | 567 | generator start stop | Facility & Grid Optimization | capacity | GAIN, 8.83% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its | Energy | 0 wobbles of margin on its line |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | outage floor | | |
| 4 | 573 | facility peak guard | Facility & Grid Optimization | capacity | GAIN, 8.67% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0 wobbles of margin on its line |
| 5 | 1085 | volt var | Energy Storage & Microgrid | capacity | GAIN, 8.64% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0 wobbles of margin on its line |
| 6 | 527 | storage dispatch | Energy Storage & Microgrid | capacity | GAIN, 7.62% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0 wobbles of margin on its line |
| 7 | 1086 | volt watt | Energy Storage & Microgrid | capacity | GAIN, 7.36% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0 wobbles of margin on its line |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 8 | 523 | peak shaving | Energy Storage & Microgrid | capacity | GAIN, 7.28% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0 wobbles of margin on its line |
| 9 | 569 | pue target | Facility & Grid Optimization | setpoint | GAIN, 4.10% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 1 wobbles of margin on its line |
| 10 | 528 | microgrid emergency reserve | Energy Storage & Microgrid | setpoint | GAIN, 3.90% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 1 wobbles of margin on its line |
| 11 | 524 | time of use schedule | Energy Storage & Microgrid | setpoint | GAIN, 3.69% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 1 wobbles of margin on its line |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 12 | 515 | battery soc reserve | Energy Storage & Microgrid | setpoint | GAIN, 3.67% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy | 0.75 wobbles of margin on its line |
| 13 | 521 | electrolyzer power | Energy Storage & Microgrid | power | GAIN, 2.85% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 5 decisions before it holds; keeps 0% of its line clear |
| 14 | 520 | heat pump power | Energy Storage & Microgrid | power | GAIN, 2.14% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 15 | 518 | pv curtailment | Energy Storage & Microgrid | power | GAIN, 2.05% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 16 | 514 | battery discharge power | Energy Storage & Microgrid | power | GAIN, 1.74% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | | line clear |
| 17 | 532 | pdu outlet control | PDU, UPS & Electrical Distribution | capacity | GAIN, 1.71% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four: the spine | 0 wobbles of margin on its line |
| 18 | 513 | battery charge power | Energy Storage & Microgrid | power | GAIN, 1.61% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 10 decisions before it holds; keeps 0% of its line clear |
| 19 | 537 | phase balance | PDU, UPS & Electrical Distribution | capacity | GAIN, 1.53% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four: the spine | 0 wobbles of margin on its line |
| 20 | 541 | generator dispatch | PDU, UPS & Electrical Distribution | capacity | GAIN, 1.49% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four: the spine | 0 wobbles of margin on its line |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 21 | 570 | cooling power budget | Facility & Grid Optimization | power | GAIN, 1.19% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 22 | 1057 | ups shutdown battery test | PDU, UPS & Electrical Distribution | capacity | GAIN, 1.12% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four: the spine | 0 wobbles of margin on its line |
| 23 | 534 | ups operating mode | PDU, UPS & Electrical Distribution | capacity | GAIN, 1.10% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four: the spine | 0 wobbles of margin on its line |
| 24 | 539 | reactive power target | PDU, UPS & Electrical Distribution | setpoint | GAIN, 1.06% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four: the spine | 0 wobbles of margin on its line |
| 25 | 517 | grid export limit | Energy Storage & Microgrid | power | GAIN, 0.95% of decisions back under | at its limit on native (full battery power): read and left | Energy | 0 wobbles of margin on its line; reads its line 3 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | the line | native | | before it holds; keeps 5% of its line clear |
| 26 | 540 | voltage target | PDU, UPS & Electrical Distribution | setpoint | GAIN, 0.90% of decisions back under the line | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four: the spine | 0.25 wobbles of margin on its line |
| 27 | 519 | ev charge power | Energy Storage & Microgrid | power | GAIN, 0.74% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | global |
| 28 | 516 | grid import limit | Energy Storage & Microgrid | power | GAIN, 0.51% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | global |
| 29 | 1087 | constant power factor reactive power | Energy Storage & Microgrid | power | GAIN, 0.51% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | global |
| 30 | 561 | facility power budget | Facility & Grid Optimization | power | GAIN, 0.48% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 31 | 562 | utility demand limit | Facility & Grid Optimization | power | GAIN, 0.38% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 32 | 571 | it power budget | Facility & Grid Optimization | power | GAIN, 0.35% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 33 | 522 | microgrid demand limit | Energy Storage & Microgrid | power | GAIN, 0.26% of decisions back under the line | at its limit on native (full battery power): read and left native | Energy | global |
| 34 | 467 | valve position | Industrial PLC & Process Automation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 35 | 471 | furnace setpoint | Industrial PLC & Process Automation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 36 | 472 | pressure setpoint | Industrial PLC & Process Automation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 37 | 473 | temperature setpoint | Industrial PLC & Process Automation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 38 | 474 | mass flow setpoint | Industrial PLC & Process Automation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 39 | 475 | tank level target | Industrial PLC & Process Automation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 40 | 529 | server power cap | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left native | all four: the spine | global |
| 41 | 530 | rack power cap | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left native | all four: the spine | global |
| 42 | 531 | pdu branch power limit | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left native | all four: the spine | global |
| 43 | 535 | ups charge rate | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | native | | |
| 44 | 536 | ups discharge rate | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left native | all four: the spine | global |
| 45 | 545 | supply air temperature | Cooling, Chillers & Thermodynamics | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four: the spine | global |
| 46 | 546 | return air target | Cooling, Chillers & Thermodynamics | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four: the spine | global |
| 47 | 547 | coolant supply temperature | Cooling, Chillers & Thermodynamics | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four: the spine | global |
| 48 | 548 | coolant flow | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | nothing to win) | | |
| 49 | 549 | pump speed | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | all four: the spine | global |
| 50 | 550 | fan speed | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | all four: the spine | global |
| 51 | 551 | chiller setpoint | Cooling, Chillers & Thermodynamics | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four: the spine | global |
| 52 | 553 | cooling tower fan | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | all four: the spine | global |
| 53 | 554 | cooling capacity | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | all four: the spine | global |
| 54 | 556 | gpu thermal envelope | Cooling, Chillers & Thermodynamics | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | flat out; never warmer than native | | |
| 55 | 557 | cpu thermal envelope | Cooling, Chillers & Thermodynamics | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four: the spine | global |
| 56 | 572 | rack power allocation | Facility & Grid Optimization | power | STEADY | at its limit on native (full battery power): read and left native | Energy | global |
| 57 | 1055 | server fan speed | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | all four: the spine | global |
| 58 | 1056 | economiser | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | all four: the spine | global |
| 59 | 1094 | setpoint reset trim and respond | Building & Critical Environment HVAC | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 60 | 1095 | duct static pressure setpoint | Building & Critical Environment HVAC | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 61 | 1096 | optimal start stop | Building & Critical Environment HVAC | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 62 | 1110 | hierarchical power budget row pdu switchboard controllers | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left native | all four: the spine | global |
| 63 | 1111 | priority aware capping throttle low priority first | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left native | all four: the spine | global |
| 64 | 1113 | power oversubscription with prediction | PDU, UPS & Electrical Distribution | power | STEADY | at its limit on native (full battery power): read and left native | all four: the spine | global |
| 65 | 1122 | chiller plant supervisory control rl with safety layer | Cooling, Chillers & Thermodynamics | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 66 | 2001 | operating room air change setpoint | Healthcare Critical Environments | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 67 | 2002 | isolation room pressure target | Healthcare Critical Environments | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 68 | 2003 | patient room temperature setpoint | Healthcare Critical Environments | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 69 | 2004 | surgical suite humidity target | Healthcare Critical Environments | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 70 | 2005 | ahu supply air temperature | Healthcare Critical Environments | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 71 | 2006 | pharmacy cold room setpoint | Healthcare Critical Environments | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 72 | 2007 | sterile storage humidity setpoint | Healthcare Critical Environments | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 73 | 2008 | imaging suite cooling capacity | Healthcare Critical Environments | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 74 | 2009 | chiller plant staging | Healthcare Critical Environments | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 75 | 2010 | exhaust fan capacity | Healthcare Critical Environments | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 76 | 2027 | mainline pressure setpoint | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 77 | 2028 | soil moisture target | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 78 | 2029 | pivot speed setpoint | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 79 | 2031 | greenhouse temperature setpoint | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 80 | 2032 | greenhouse co2 target | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 81 | 2033 | vent position setpoint | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | (nothing to win) | | |
| 82 | 2034 | grain dryer temperature setpoint | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 83 | 2035 | cold storage temperature setpoint | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 84 | 2037 | milking vacuum level setpoint | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 85 | 2038 | well drawdown level target | Agriculture & Irrigation | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 86 | 2040 | compressor discharge pressure setpoint | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 87 | 2041 | pump station suction pressure target | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 88 | 2042 | line pack target | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 89 | 2043 | pipeline flow setpoint | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 90 | 2046 | terminal tank level target | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 91 | 2049 | heater outlet temperature setpoint | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 92 | 2051 | valve position target | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 93 | 2052 | cathodic protection voltage | Oil & Gas Pipelines | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 94 | 2092 | sag mill load setpoint | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | (nothing to win) | | |
| 95 | 2093 | mill speed target | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 96 | 2094 | crusher gap setpoint | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 97 | 2096 | cyclone feed pressure target | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 98 | 2097 | thickener underflow density target | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 99 | 2099 | conveyor speed setpoint | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 100 | 2100 | dewatering pump level target | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 101 | 2102 | stockpile level target | Mining & Mineral Processing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 102 | 2106 | supply temperature setpoint | District Heating & Cooling | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 103 | 2107 | return temperature target | District Heating & Cooling | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 104 | 2108 | differential pressure setpoint | District Heating & Cooling | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 105 | 2112 | thermal storage level target | District Heating & Cooling | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 106 | 2116 | outdoor reset setpoint | District Heating & Cooling | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 107 | 2117 | cooling network supply temperature | District Heating & Cooling | setpoint | STEADY | left native: native already holds it inside its band | Energy | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | (nothing to win) | | |
| 108 | 2118 | cooling storage level target | District Heating & Cooling | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 109 | 2119 | turbine speed droop setpoint | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 110 | 2120 | unit load setpoint | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 111 | 2121 | boiler steam pressure setpoint | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 112 | 2122 | feedwater level target | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 113 | 2124 | excitation voltage setpoint | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 114 | 2125 | hydro gate position target | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 115 | 2126 | combustion air ratio setpoint | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 116 | 2127 | cooling water flow setpoint | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 117 | 2128 | inlet guide vane position | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 118 | 2132 | nuclear rod position target | Power Generation & Turbine Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 119 | 2133 | inverter volt var setpoint | Renewable Generation & Inverter Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 120 | 2134 | inverter volt watt setpoint | Renewable Generation & Inverter Control | setpoint | STEADY | left native: native already holds it inside its band | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | (nothing to win) | | |
| 121 | 2136 | plant reactive power target | Renewable Generation & Inverter Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 122 | 2139 | rotor speed setpoint | Renewable Generation & Inverter Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 123 | 2140 | inverter frequency droop setpoint | Renewable Generation & Inverter Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 124 | 2143 | string mppt voltage setpoint | Renewable Generation & Inverter Control | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 125 | 2158 | reactor temperature setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 126 | 2159 | agitator speed setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 127 | 2160 | fermenter dissolved oxygen target | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 128 | 2161 | ph setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 129 | 2162 | pasteurizer holding temperature setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 130 | 2163 | freezer tunnel temperature setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 131 | 2165 | cleanroom pressure cascade setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 132 | 2166 | lyophilizer shelf temperature setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 133 | 2167 | chromatography flow setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | (nothing to win) | | |
| 134 | 2169 | oven zone temperature setpoint | Pharmaceutical & Food Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 135 | AU DIT-0034 | voltage regulator tap | Grid Transmission & Distribution | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 136 | AU DIT-0035 | transformer tap | Grid Transmission & Distribution | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 137 | AU DIT-0038 | feeder voltage target | Grid Transmission & Distribution | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 138 | AU DIT-0042 | frequency droop setpoint | Grid Transmission & Distribution | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 139 | AU DIT-0060 | chamber temperature | Semiconductor Fab & Precision Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 140 | AU DIT-0061 | chamber pressure | Semiconductor Fab & Precision Manufacturing | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 141 | AU DIT-0070 | valve position water | Water Wastewater & Pumping | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 142 | AU DIT-0071 | reservoir level target | Water Wastewater & Pumping | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 143 | AU DIT-0072 | line pressure target | Water Wastewater & Pumping | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 144 | AU DIT-0073 | flow target water | Water Wastewater & Pumping | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Energy | global |
| 145 | AU DIT-0081 | zone temperature target | Building & Critical Environment HVAC | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 146 | AUDIT-0082 | zone airflow | Building & Critical Environment HVAC | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 147 | AUDIT-0083 | ahu fan speed | Building & Critical Environment HVAC | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 148 | AUDIT-0084 | damper position | Building & Critical Environment HVAC | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 149 | AUDIT-0085 | economizer position | Building & Critical Environment HVAC | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 150 | AUDIT-0086 | boiler setpoint | Building & Critical Environment HVAC | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy | global |
| 151 | AUDIT-0088 | humidity target | Building & Critical Environment HVAC | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | warmer than native | | |
| 152 | AUDIT-0091 | thermal storage dispatch | Building & Critical Environment HVAC | capacity | STEADY | left to its own staging (native already has the cooling it needs: nothing to win) | Energy | global |
| 153 | 465 | plc cycle authority | Industrial PLC & Process Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 154 | 466 | machine cell admission | Industrial PLC & Process Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 155 | 468 | pump flow | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 156 | 469 | compressor speed | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 157 | 470 | heater power | Industrial PLC & Process Automation | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 158 | 476 | conveyor rate | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 159 | 477 | feed rate | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 160 | 478 | purge vent action | Industrial PLC & Process Automation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 161 | 525 | energy load shed | Energy Storage & Microgrid | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 162 | 526 | flex load admission | Energy Storage & Microgrid | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 163 | 538 | load transfer | PDU, UPS & Electrical Distribution | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 164 | 542 | electrical isolation | PDU, UPS & Electrical Distribution | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 165 | 543 | breaker trip gate | PDU, UPS & Electrical Distribution | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 166 | 552 | compressor authority | Cooling, Chillers & Thermodynamics | power | at its limit on native | at its limit on native (the site envelope at full): read and left native | all four: the spine | global |
| 167 | 555 | rack thermal budget | Cooling, Chillers & Thermodynamics | power | at its limit on native | at its limit on native (the site envelope at full): read and left native | all four: the spine | global |
| 168 | 558 | thermal workload migrate | Cooling, Chillers & Thermodynamics | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 169 | 559 | thermal load shed | Cooling, Chillers & Thermodynamics | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 170 | 563 | demand response | Facility & Grid Optimization | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 171 | 564 | electricity price gate | Facility & Grid Optimization | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 172 | 565 | carbon intensity gate | Facility & Grid Optimization | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | left native | | |
| 173 | 574 | grid frequency response | Facility & Grid Optimization | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 174 | 575 | facility islanding | Facility & Grid Optimization | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 175 | 1088 | controller mode | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 176 | 1089 | alarm limits | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 177 | 1090 | safety interlock trip | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 178 | 1091 | opc ua writes | Industrial PLC & Process Automation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 179 | 1092 | run to run control | Semiconductor Fab & Precision Manufacturing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 180 | 1093 | idle sleep mode | Semiconductor Fab & Precision Manufacturing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 181 | 2011 | ward night setback mode | Healthcare Critical Environments | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 182 | 2012 | medical gas plant demand limit | Healthcare Critical Environments | power | at its limit on native | at its limit on native (the site envelope at full): read and left native | Energy | global |
| 183 | 2013 | emergency power load shed | Healthcare Critical Environments | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 184 | 2026 | irrigation pump speed | Agriculture & Irrigation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 185 | 2030 | fertigation dose | Agriculture & Irrigation | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 186 | 2036 | barn ventilation rate | Agriculture & Irrigation | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 187 | 2039 | drip zone dispatch | Agriculture & Irrigation | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 188 | 2044 | compressor unit staging | Oil & Gas Pipelines | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 189 | 2045 | vfd pump speed | Oil & Gas Pipelines | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 190 | 2047 | leak detection shutdown | Oil & Gas Pipelines | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 191 | 2048 | batch interface dispatch | Oil & Gas Pipelines | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 192 | 2050 | drag reducing agent dose | Oil & Gas Pipelines | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 193 | 2053 | pressure protection priority | Oil & Gas Pipelines | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 194 | 2095 | flotation aeration rate | Mining & Mineral Processing | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 195 | 2098 | reagent dose | Mining & Mineral Processing | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 196 | 2101 | ventilation on demand airflow | Mining & Mineral Processing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 197 | 2103 | slurry pump speed | Mining & Mineral Processing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 198 | 2104 | tailings discharge shutdown | Mining & Mineral Processing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 199 | 2105 | ore blend dispatch | Mining & Mineral Processing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 200 | 2109 | network pump speed | District Heating & Cooling | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 201 | 2110 | heat pump staging | District Heating & Cooling | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 202 | 2111 | chp dispatch | District Heating & Cooling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 203 | 2113 | peak boiler heater output | District Heating & Cooling | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 204 | 2114 | substation flow limit | District Heating & Cooling | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 205 | 2115 | peak demand shed | District Heating & Cooling | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 206 | 2123 | agc participation limit | Power Generation & Turbine Control | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 207 | 2129 | reserve dispatch priority | Power Generation & Turbine Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 208 | 2130 | turbine ramp rate limit | Power Generation & Turbine Control | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 209 | 2131 | emissions shutdown | Power Generation & Turbine Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 210 | 2135 | active power curtailment | Renewable Generation & Inverter Control | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 211 | 2137 | wind turbine yaw offset | Renewable Generation & Inverter Control | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 212 | 2138 | pitch angle limit | Renewable Generation & Inverter Control | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 213 | 2141 | ramp rate limit | Renewable Generation & Inverter Control | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 214 | 2142 | tracker stow mode | Renewable Generation & Inverter Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 215 | 2144 | noise mode schedule | Renewable Generation & Inverter Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 216 | 2145 | ice detection shutdown | Renewable Generation & Inverter Control | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 217 | 2164 | cip cycle dispatch | Pharmaceutical & Food Manufacturing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 218 | 2168 | steam sterilizer cycle priority | Pharmaceutical & Food Manufacturing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 219 | 2170 | refrigerant compressor authority | Pharmaceutical & Food Manufacturing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 220 | 2171 | batch hold quarantine | Pharmaceutical & Food Manufacturing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 221 | AUDIT-0033 | capacitor bank switch | Grid Transmission & Distribution | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 222 | AUDIT-0036 | inverter real power | Grid Transmission & Distribution | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 223 | AU DIT-0037 | inverter reactive power | Grid Transmission & Distribution | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 224 | AU DIT-0039 | feeder load transfer | Grid Transmission & Distribution | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 225 | AU DIT-0040 | distribution storage dispatch | Grid Transmission & Distribution | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 226 | AU DIT-0041 | demand response dispatch | Grid Transmission & Distribution | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 227 | AU DIT-0043 | grid protection mode | Grid Transmission & Distribution | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 228 | AU DIT-0044 | grid restoration sequence | Grid Transmission & Distribution | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 229 | AU DIT-0057 | tool job dispatch | Semiconductor Fab & Precision Manufacturing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 230 | AUDIT-0058 | wafer route | Semiconductor Fab & Precision Manufacturing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 231 | AUDIT-0059 | chamber recipe selection | Semiconductor Fab & Precision Manufacturing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 232 | AUDIT-0062 | gas flow | Semiconductor Fab & Precision Manufacturing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 233 | AUDIT-0063 | rf power | Semiconductor Fab & Precision Manufacturing | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 234 | AUDIT-0064 | vacuum pump speed | Semiconductor Fab & Precision Manufacturing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 235 | AUDIT-0065 | robot transfer rate | Semiconductor Fab & Precision Manufacturing | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 236 | AUDIT-0066 | lot priority | Semiconductor Fab & Precision Manufactur | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | ing | | | left native | | |
| 237 | AU DIT-0068 | tool quarantine | Semiconductor Fab & Precision Manufacturing | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 238 | AU DIT-0069 | pump speed water | Water Wastewater & Pumping | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 239 | AU DIT-0074 | aeration rate | Water Wastewater & Pumping | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 240 | AU DIT-0075 | chemical dose rate | Water Wastewater & Pumping | power | at its limit on native | at its limit on native (the full actuator): read and left native | Energy | global |
| 241 | AU DIT-0076 | filtration backwash | Water Wastewater & Pumping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 242 | AU DIT-0077 | lift station dispatch | Water Wastewater & Pumping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 243 | AU DIT-0078 | leak isolation | Water Wastewater & Pumping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 244 | AU DIT-0079 | water demand shed | Water Wastewater & Pumping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 245 | AU DIT-0080 | water emergency shutdown | Water Wastewater & Pumping | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 246 | AU DIT-0087 | heat pump mode | Building & Critical Environment HVAC | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 247 | AU DIT-0089 | occupancy ventilation | Building & Critical Environment HVAC | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |
| 248 | AU DIT-0090 | building demand limit | Building & Critical Environment HVAC | power | at its limit on native | at its limit on native (the site envelope at full): read and left native | Energy | global |
| 249 | AU DIT-0092 | hvac emergency mode | Building & Critical Environment HVAC | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Energy | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *E.2.4 Distribution / Specialized: 277 muscles, heaviest hitter first*

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 1 | 1103 | io capacity | Database & Transactions | capacity | GAIN, 6.81% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 2 | 287 | vacuum compaction rate | Database & Transactions | capacity | GAIN, 6.24% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 3 | 278 | read route | Database & Transactions | capacity | GAIN, 6.07% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in | Distribution | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 4 | 280 | db failover | Database & Transactions | capacity | GAIN, 5.90% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 5 | 276 | db cache | Database & Transactions | capacity | GAIN, 5.82% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 90% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 6 | 415 | payment failover | Commerce & Payment Systems | capacity | GAIN, 5.53% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 60 decisions |
| 7 | 283 | db pool resize | Database & Transactions | capacity | GAIN, 5.28% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 75% full; reads the last 40 decisions |
| 8 | 279 | shard placement | Database & Transactions | capacity | GAIN, 5.21% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 75% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 9 | 273 | db replicas | Database & Transactions | capacity | GAIN, 5.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 60 decisions |
| 10 | 275 | db memory | Database & Transactions | capacity | GAIN, 5.11% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 11 | 286 | checkpoint rate | Database & Transactions | capacity | GAIN, 5.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 12 | 414 | settlement batch | Commerce & Payment Systems | capacity | GAIN, 4.74% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 13 | 410 | order reservation | Commerce & Payment Systems | capacity | GAIN, 4.71% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 14 | 406 | authorization route | Commerce & Payment Systems | capacity | GAIN, 4.62% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 15 | 407 | processor route weight | Commerce & Payment Systems | setpoint | GAIN, 4.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 16 | 1102 | buffer pool | Database & Transactions | capacity | GAIN, 3.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 75% full; reads the last 40 decisions |
| 17 | 1100 | background writer | Database & Transactions | capacity | GAIN, 3.83% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 18 | 1101 | synchronous replication | Database & Transactions | capacity | GAIN, 3.82% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 19 | 1099 | parallel workers | Database & Transactions | capacity | GAIN, 3.75% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 60 decisions |
| 20 | 409 | idempotency window | Commerce & Payment Systems | setpoint | GAIN, 3.45% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 21 | 413 | refund queue rate | Commerce & Payment Systems | capacity | GAIN, 3.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 22 | 2016 | ehr application replicas | Medical Imaging & Clinical Systems | capacity | GAIN, 2.65% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 23 | V5-5070 | backtest grid nodes | Financial Markets & Trading Systems | capacity | GAIN, 2.54% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 24 | 2014 | pacs archive tier target | Medical Imaging & Clinical Systems | setpoint | GAIN, 2.52% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 25 | V5-5127 | broker cluster nodes | Messaging & Streaming | capacity | GAIN, 2.31% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 26 | 2019 | reconstruction gpu workers | Medical Imaging & Clinical Systems | capacity | GAIN, 2.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 27 | 255 | service failover | Service Mesh & API Reliability | capacity | GAIN, 1.96% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 28 | 364 | background workers | Runtime & Application | capacity | GAIN, 1.57% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 29 | 359 | runtime memory | Runtime & Application | capacity | GAIN, 1.46% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 60 decisions |
| 30 | 354 | thread pool | Runtime & Application | capacity | GAIN, 1.44% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 31 | 298 | fetch batch size | Messaging & Streaming | capacity | GAIN, 1.42% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 32 | 373 | collector memory limit | Observability & Telemetry | capacity | GAIN, 1.35% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 33 | 389 | degraded mode | Reliability, Security & Recovery | capacity | GAIN, 1.33% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 34 | 1108 | go runtime | Runtime & Application | capacity | GAIN, 1.32% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 60 decisions |
| 35 | 305 | cache size | Cache & Memory Services | capacity | GAIN, 1.30% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 75% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 36 | 361 | event loop workers | Runtime & Application | capacity | GAIN, 1.28% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 80% full; reads the last 20 decisions |
| 37 | 321 | index workers | Search, Indexing & Vector DB | capacity | GAIN, 1.24% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 75% full; reads the last 20 decisions |
| 38 | 318 | cache failover | Cache & Memory Services | capacity | GAIN, 1.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 60 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 39 | 1119 | per pod energy attribution | Observability & Telemetry | capacity | GAIN, 1.19% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 60 decisions |
| 40 | 248 | canary weight | Service Mesh & API Reliability | setpoint | GAIN, 1.18% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 41 | 251 | dns traffic weight | Service Mesh & API Reliability | setpoint | GAIN, 1.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 42 | 309 | cache sharding | Cache & Memory Services | capacity | GAIN, 1.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 43 | 331 | embedding workers | Search, Indexing & Vector DB | capacity | GAIN, 1.16% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 44 | AUDIT-0014 | ran scheduler weight | Telecom RAN & Edge Radio | setpoint | GAIN, 1.15% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 45 | 308 | cache replicas | Cache & Memory Services | capacity | GAIN, 1.13% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 46 | 397 | backup trigger | Reliability, Security & Recovery | capacity | GAIN, 1.12% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 47 | 299 | rebalance rate | Messaging & Streaming | capacity | GAIN, 1.11% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 48 | 323 | segment merge rate | Search, Indexing & Vector DB | capacity | GAIN, 1.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 49 | 1097 | load balancing policy | Service Mesh & API Reliability | capacity | GAIN, 1.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 50 | 304 | broker failover | Messaging & Streaming | capacity | GAIN, 1.06% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 51 | 311 | cache writeback rate | Cache & Memory Services | capacity | GAIN, 1.05% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 52 | 355 | jvm heap | Runtime & Application | capacity | GAIN, 1.05% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 53 | 377 | retention window | Observability & Telemetry | setpoint | GAIN, 1.04% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | machines started at 85% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 54 | 302 | dead letter divert | Messaging & Streaming | capacity | GAIN, 1.03% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 55 | 362 | process count | Runtime & Application | capacity | GAIN, 1.03% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 56 | 1109 | keepalive connection reuse | Runtime & Application | capacity | GAIN, 1.03% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 60 decisions |
| 57 | 306 | cache ttl | Cache & Memory Services | setpoint | GAIN, 1.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 58 | 358 | application cache size | Runtime & Application | capacity | GAIN, 1.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 59 | V5-5121 | waf inspection workers | Reliability, Security & Recovery | capacity | GAIN, 1.01% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 60 | 330 | vector batch size | Search, Indexing & Vector DB | capacity | GAIN, 1.00% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 61 | 1106 | memory size threads | Cache & Memory Services | capacity | GAIN, 1.00% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 62 | 247 | traffic mirror | Service Mesh & API Reliability | capacity | GAIN, 0.99% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 63 | 326 | shard count | Search, Indexing & Vector DB | capacity | GAIN, 0.99% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 64 | 327 | shard replication | Search, Indexing & Vector DB | capacity | GAIN, 0.98% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 65 | 310 | cache prefetch | Cache & Memory Services | capacity | GAIN, 0.97% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 66 | 252 | session affinity | Service Mesh & API Reliability | capacity | GAIN, 0.95% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 67 | V5-5059 | order gateway sessions | Financial Markets & Trading Systems | capacity | GAIN, 0.94% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 68 | 246 | traffic divert | Service Mesh & API Reliability | capacity | GAIN, 0.94% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 69 | 300 | replication factor | Messaging & Streaming | capacity | GAIN, 0.94% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 75% full; reads the last 10 decisions |
| 70 | 388 | traffic divert recovery | Reliability, Security & Recovery | capacity | GAIN, 0.94% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 71 | 289 | partition count | Messaging & Streaming | capacity | GAIN, 0.92% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 72 | 333 | query route | Search, Indexing & Vector DB | capacity | GAIN, 0.92% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 60 decisions |
| 73 | 386 | rollback | Reliability, Security & Recovery | capacity | GAIN, 0.91% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 74 | 297 | producer batch size | Messaging & Streaming | capacity | GAIN, 0.90% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 75 | 328 | shard rebalance | Search, Indexing & Vector DB | capacity | GAIN, 0.90% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 76 | AU DIT-0010 | ran ue handover | Telecom RAN & Edge Radio | capacity | GAIN, 0.89% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 77 | AU DIT-0016 | ran antenna tilt | Telecom RAN & Edge Radio | capacity | GAIN, 0.89% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 78 | 294 | message retention | Messaging & Streaming | capacity | GAIN, 0.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 79 | 329 | vector search k | Search, Indexing & Vector DB | capacity | GAIN, 0.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 80 | V5-5061 | pre trade risk check workers | Financial Markets & Trading Systems | capacity | GAIN, 0.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 81 | V5-5058 | matching engine threads | Financial Markets & Trading Systems | capacity | GAIN, 0.88% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 82 | 378 | remote write queue | Observability & Telemetry | capacity | GAIN, 0.86% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 83 | 394 | policy enforcement | Reliability, Security & Recovery | capacity | GAIN, 0.86% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 84 | AU DIT-0017 | ran carrier enable | Telecom RAN & Edge Radio | capacity | GAIN, 0.85% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 85 | AU DIT-0018 | ran cell sleep | Telecom RAN & Edge Radio | capacity | GAIN, 0.84% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 86 | 315 | cache memory limit | Cache & Memory Services | capacity | GAIN, 0.83% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 87 | 322 | index refresh rate | Search, Indexing & Vector DB | capacity | GAIN, 0.82% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 88 | 253 | request hedging | Service Mesh & API Reliability | capacity | GAIN, 0.81% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 89 | 353 | worker count | Runtime & Application | capacity | GAIN, 0.79% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 75% full; reads the last 10 decisions |
| 90 | 1053 | metric scrape interval | Observability & Telemetry | setpoint | GAIN, 0.79% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 91 | AUDIT-0013 | ran prb allocation | Telecom RAN & Edge Radio | capacity | GAIN, 0.77% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 92 | 290 | partition placement | Messaging & Streaming | capacity | GAIN, 0.76% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 93 | 367 | runtime restart | Runtime & Application | capacity | GAIN, 0.74% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 40 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 94 | 316 | cache compression | Cache & Memory Services | capacity | GAIN, 0.72% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 95 | 357 | connection pool runtime | Runtime & Application | capacity | GAIN, 0.71% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 96 | 317 | cache warmup | Cache & Memory Services | capacity | GAIN, 0.69% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 97 | 1052 | lifecycle tiering | Storage Block/File/Object | capacity | GAIN, 0.69% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 98 | 319 | cache flush rate | Cache & Memory Services | capacity | GAIN, 0.69% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 99 | 270 | erasure code profile | Storage Block/File/Object | capacity | GAIN, 0.67% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 100 | AUDIT-0019 | ran du cu placement | Telecom RAN & Edge Radio | capacity | GAIN, 0.65% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 101 | 385 | restart | Reliability, Security & Recovery | capacity | GAIN, 0.63% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 10 decisions |
| 102 | 1054 | batching | Observability & Telemetry | capacity | GAIN, 0.61% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 80% full; reads the last 10 decisions |
| 103 | 1104 | prefetch | Messaging & Streaming | capacity | GAIN, 0.59% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 104 | 259 | throughput limit | Storage Block/File/Object | capacity | GAIN, 0.58% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 105 | AUDIT-0011 | ran cell traffic steering | Telecom RAN & Edge Radio | capacity | GAIN, 0.58% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 106 | 1049 | scrub schedule | Storage Block/File/Object | capacity | GAIN, 0.54% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 70% full; reads the last 20 decisions |
| 107 | 1105 | memory disk alarm | Messaging & Streaming | capacity | GAIN, 0.53% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 108 | 228 | bandwidth limit | Network Routing & Switching | capacity | GAIN, 0.51% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 109 | 262 | volume placement | Storage Block/File/Object | capacity | GAIN, 0.51% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 110 | V5-5124 | mqtt broker max connections | Messaging & Streaming | capacity | GAIN, 0.50% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 111 | 257 | volume size | Storage Block/File/Object | capacity | GAIN, 0.49% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 112 | 1046 | nic interrupt coalescing | Network Routing & Switching | capacity | GAIN, 0.47% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 113 | 233 | nic queue count | Network Routing & Switching | capacity | GAIN, 0.46% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 114 | V5-5041 | origin shield concurrency | Content Delivery & Edge Cache | capacity | GAIN, 0.46% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 115 | 226 | route weight | Network Routing & Switching | setpoint | GAIN, 0.45% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 116 | 268 | cache allocation | Storage Block/File/Object | capacity | GAIN, 0.45% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 10 decisions |
| 117 | 229 | qos class | Network Routing & Switching | capacity | GAIN, 0.44% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 118 | 269 | object replication | Storage Block/File/Object | capacity | GAIN, 0.44% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 119 | 261 | storage tier | Storage Block/File/Object | capacity | GAIN, 0.43% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 120 | 266 | backfill rate | Storage Block/File/Object | capacity | GAIN, 0.43% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 121 | 1047 | nic ring size | Network Routing & Switching | capacity | GAIN, 0.42% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 80% full; reads the last 10 decisions |
| 122 | V5-5040 | edge cache size | Content Delivery & Edge Cache | capacity | GAIN, 0.41% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 123 | V5-5131 | dns resolver cache size | Runtime & Application | capacity | GAIN, 0.41% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 124 | V5-5062 | market data fanout threads | Financial Markets & Trading Systems | capacity | GAIN, 0.39% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 125 | 225 | lb weight | Network Routing & Switching | setpoint | GAIN, 0.38% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 126 | 234 | queue discipline | Network Routing & Switching | capacity | GAIN, 0.38% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 127 | V5-5118 | ids capture threads | Reliability, Security & Recovery | capacity | GAIN, 0.38% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 128 | 260 | replica count | Storage Block/File/Object | capacity | GAIN, 0.37% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 129 | 265 | recovery rate | Storage Block/File/Object | capacity | GAIN, 0.36% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 130 | 245 | connection pool | Service Mesh & API Reliability | capacity | GAIN, 0.35% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 131 | 264 | rebalance | Storage Block/File/Object | capacity | GAIN, 0.35% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | machines started at 75% full; reads the last 10 decisions |
| 132 | 1050 | io scheduler | Storage Block/File/Object | capacity | GAIN, 0.34% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 133 | 263 | snapshot trigger | Storage Block/File/Object | capacity | GAIN, 0.33% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | machines started at 75% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 134 | 267 | compaction pressure | Storage Block/File/Object | capacity | GAIN, 0.32% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 135 | 235 | congestion control | Network Routing & Switching | capacity | GAIN, 0.31% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 136 | 239 | path selection | Network Routing & Switching | capacity | GAIN, 0.28% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 137 | 231 | failover route | Network Routing & Switching | capacity | GAIN, 0.26% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |
| 138 | 238 | ecmp weight | Network Routing & Switching | setpoint | GAIN, 0.26% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 139 | V5-5133 | mail queue concurrency | Runtime & Application | capacity | GAIN, 0.18% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 140 | 258 | iops limit | Storage Block/File/Object | capacity | GAIN, 0.17% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 141 | 429 | route replan | Workflow, Logistics & Fulfillment | capacity | GAIN, 0.10% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 142 | 431 | compensation action | Workflow, Logistics & Fulfillment | capacity | GAIN, 0.09% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 40 decisions |
| 143 | 428 | shipment batch | Workflow, Logistics & Fulfillment | capacity | GAIN, 0.04% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 144 | V5-5071 | surveillance alert queue workers | Financial Markets & Trading Systems | capacity | GAIN, 0.04% of decisions back under the line | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 145 | 344 | stage parallelism | Data Analytics & ETL | capacity | GAIN, the line cleared 54.0% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 146 | 338 | executor size | Data Analytics & ETL | capacity | GAIN, the line cleared 52.5% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 147 | 342 | shuffle bandwidth | Data Analytics & ETL | capacity | GAIN, the line cleared 51.8% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 148 | V5-5122 | siem ingest pipeline workers | Reliability, Security & Recovery | capacity | GAIN, the line cleared 51.8% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 80% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 149 | 347 | cache fraction | Data Analytics & ETL | setpoint | GAIN, the line cleared 51.5% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 80% full; reads the last 20 decisions |
| 150 | 341 | shuffle partitions | Data Analytics & ETL | capacity | GAIN, the line cleared 50.9% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 151 | 340 | dynamic allocation max | Data Analytics & ETL | capacity | GAIN, the line cleared 50.2% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 20 decisions |
| 152 | 351 | speculation policy | Data Analytics & ETL | capacity | GAIN, the line cleared 50.0% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 75% full; reads the last 10 decisions |
| 153 | 348 | batch interval | Data Analytics & ETL | setpoint | GAIN, the line cleared 49.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 80% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 154 | 425 | warehouse queue | Workflow, Logistics & Fulfillment | capacity | GAIN, the line cleared 46.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 155 | 339 | dynamic allocation min | Data Analytics & ETL | capacity | GAIN, the line cleared 46.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 156 | 345 | query slots | Data Analytics & ETL | capacity | GAIN, the line cleared 43.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 85% full; reads the last 10 decisions |
| 157 | V5-5067 | post trade settlement workers | Financial Markets & Trading Systems | capacity | GAIN, the line cleared 42.9% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 80% full; reads the last 20 decisions |
| 158 | 350 | data locality wait | Data Analytics & ETL | capacity | GAIN, the line cleared 42.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | machines started at 80% full; reads the last 20 decisions |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 159 | 418 | workflow worker rate | Workflow, Logistics & Fulfillment | capacity | GAIN, the line cleared 38.0% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | machines started at 70% full; reads the last 10 decisions |
| 160 | 423 | inventory allocation | Workflow, Logistics & Fulfillment | capacity | GAIN, the line cleared 35.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 161 | 424 | fulfillment route | Workflow, Logistics & Fulfillment | capacity | GAIN, the line cleared 35.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 162 | 426 | carrier selection | Workflow, Logistics & Fulfillment | capacity | GAIN, the line cleared 31.8% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 163 | 419 | task queue rate | Workflow, Logistics & Fulfillment | capacity | GAIN, the line cleared 29.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 164 | 430 | sla escalation | Workflow, Logistics & Fulfillment | capacity | GAIN, the line cleared 24.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 165 | 2087 | rail mounted gantry dispatch | Ports & Maritime Logistics | capacity | GAIN, the line cleared 13.5% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 166 | 2080 | quay crane allocation | Ports & Maritime Logistics | capacity | GAIN, the line cleared 11.6% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 167 | 2090 | stacking height target | Ports & Maritime Logistics | setpoint | GAIN, the line cleared 10.7% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 168 | 2084 | horizontal transport fleet size | Ports & Maritime Logistics | capacity | GAIN, the line cleared 8.3% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 169 | 2081 | yard crane fleet size | Ports & Maritime Logistics | capacity | GAIN, the line cleared 8.1% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |
| 170 | 2086 | shore power connection capacity | Ports & Maritime Logistics | capacity | GAIN, the line cleared 6.4% sooner | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 171 | V5-5039 | cdn cache ttl | Content Delivery & Edge Cache | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 172 | V5-5042 | prefetch aggressiveness | Content Delivery & Edge Cache | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 173 | V5-5063 | market data conflation interval | Financial Markets & Trading Systems | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 174 | V5-5064 | fix session heartbeat interval | Financial Markets & Trading Systems | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 175 | V5-5066 | tick store write batch | Financial Markets & Trading Systems | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 176 | V5-5069 | colocation rack cooling setpoint | Financial Markets & Trading Systems | setpoint | STEADY | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Distribution | global |
| 177 | V5-5119 | ids ring buffer size | Reliability, Security & Recovery | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 178 | V5-5125 | mqtt max inflight messages | Messaging & Streaming | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 179 | V5-5126 | mqtt session expiry | Messaging & Streaming | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 180 | V5-5132 | dns negative cache ttl | Runtime & Application | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 181 | V5-5134 | mail retry interval | Runtime & Application | setpoint | STEADY | left native: native already holds it inside its band (nothing to win) | Distribution | global |
| 182 | 227 | rate limit | Network Routing & Switching | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 183 | 230 | connection limit | Network Routing & Switching | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 184 | 232 | nic rate limit | Network Routing & Switching | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 185 | 236 | egress budget | Network Routing & Switching | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 186 | 237 | ingress budget | Network Routing & Switching | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 187 | 240 | network isolation | Network Routing & Switching | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 188 | 241 | circuit breaker | Service Mesh & API Reliability | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 189 | 242 | retry budget | Service Mesh & API Reliability | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 190 | 243 | service timeout | Service Mesh & API Reliability | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 191 | 244 | service concurrency | Service Mesh & API Reliability | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 192 | 249 | outlier ejection | Service Mesh & API Reliability | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 193 | 250 | health threshold | Service Mesh & API Reliability | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 194 | 254 | fault injection gate | Service Mesh & API Reliability | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 195 | 271 | storage admission | Storage Block/File/Object | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 196 | 272 | degraded storage gate | Storage Block/File/Object | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 197 | 277 | query concurrency | Database & Transactions | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 198 | 281 | replication lag gate | Database & Transactions | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 199 | 282 | db write throttle | Database & Transactions | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 200 | 284 | transaction concurrency | Database & Transactions | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 201 | 285 | lock timeout | Database & Transactions | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 202 | 291 | producer quota | Messaging & Streaming | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 203 | 292 | consumer quota | Messaging & Streaming | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 204 | 293 | broker io quota | Messaging & Streaming | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 205 | 295 | queue depth limit | Messaging & Streaming | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 206 | 296 | consumer concurrency | Messaging & Streaming | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 207 | 301 | retry backoff | Messaging & Streaming | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 208 | 303 | stream priority | Messaging & Streaming | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 209 | 307 | cache eviction policy | Cache & Memory Services | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 210 | 312 | cache admission | Cache & Memory Services | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 211 | 313 | hot key isolation | Cache & Memory Services | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 212 | 314 | cache connection limit | Cache & Memory Services | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 213 | 324 | search concurrency | Search, Indexing & Vector DB | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 214 | 325 | search timeout | Search, Indexing & Vector DB | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 215 | 332 | index memory budget | Search, Indexing & Vector DB | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 216 | 334 | hot shard isolation | Search, Indexing & Vector DB | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 217 | 335 | search admission | Search, Indexing & Vector DB | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 218 | 343 | etl concurrency | Data Analytics & ETL | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 219 | 346 | spill threshold | Data Analytics & ETL | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 220 | 349 | stream backpressure | Data Analytics & ETL | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 221 | 352 | analytics admission | Data Analytics & ETL | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 222 | 356 | gc budget | Runtime & Application | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 223 | 360 | async concurrency | Runtime & Application | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 224 | 363 | request timeout | Runtime & Application | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 225 | 365 | runtime cpu budget | Runtime & Application | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 226 | 366 | runtime io budget | Runtime & Application | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 227 | 374 | export concurrency | Observability & Telemetry | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 228 | 376 | cardinality budget | Observability & Telemetry | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 229 | 379 | telemetry shed | Observability & Telemetry | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 230 | 390 | actuator freeze | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 231 | 391 | workload isolate | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 232 | 393 | credential rotation gate | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 233 | 395 | rate abuse gate | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 234 | 396 | fault domain isolate | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 235 | 400 | kill switch | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | all four: the spine | global |
| 236 | 401 | payment admission | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 237 | 402 | payment concurrency | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 238 | 403 | payment retry | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 239 | 404 | payment timeout | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 240 | 405 | fraud review gate | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 241 | 408 | transaction queue limit | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 242 | 411 | inventory hold | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 243 | 412 | checkout load shed | Commerce & Payment Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 244 | 417 | workflow admission | Workflow, Logistics & Fulfillment | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 245 | 420 | workflow retry | Workflow, Logistics & Fulfillment | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 246 | 421 | workflow backoff | Workflow, Logistics & Fulfillment | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 247 | 422 | workflow timeout | Workflow, Logistics & Fulfillment | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 248 | 427 | dispatch priority | Workflow, Logistics & Fulfillment | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 249 | 1048 | port link power | Network Routing & Switching | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 250 | 1051 | disk power spin down | Storage Block/File/Object | power | at its limit on native | at its limit on native (the full clock): read and left native | all four: the spine | global |
| 251 | 1107 | circuit breakers | Search, Indexing & Vector DB | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| | | | | | | left native | | |
| 252 | 2015 | dicom router concurrency | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 253 | 2017 | hl7 interface queue limit | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 254 | 2018 | fhir api rate limit | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 255 | 2020 | modality worklist timeout | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 256 | 2021 | patient monitoring gateway capacity | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 257 | 2022 | lab analyzer batch window | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 258 | 2023 | telehealth session admission | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 259 | 2024 | clinical backup window | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 260 | 2025 | imaging prefetch priority | Medical Imaging & Clinical Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 261 | 2082 | berth window admission | Ports & Maritime Logistics | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 262 | 2083 | truck gate rate limit | Ports & Maritime Logistics | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 263 | 2085 | reefer plug power budget | Ports & Maritime Logistics | power | at its limit on native | at its limit on native (the full clock): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 264 | 2088 | container dwell priority | Ports & Maritime Logistics | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 265 | 2089 | vessel arrival pacing | Ports & Maritime Logistics | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 266 | 2091 | equipment charging window | Ports & Maritime Logistics | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 267 | AU DIT-0009 | ran connection admission | Telecom RAN & Edge Radio | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 268 | AU DIT-0012 | ran slice resource budget | Telecom RAN & Edge Radio | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 269 | AU DIT-0015 | ran tx power | Telecom RAN & Edge Radio | power | at its limit on native | at its limit on native (the full clock): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 270 | AUDIT-0020 | ran fronthaul budget | Telecom RAN & Edge Radio | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 271 | V5-5043 | tiered cache promotion | Content Delivery & Edge Cache | capacity | at its limit on native | at its limit on native (the full actuator): read and left native | Distribution | global |
| 272 | V5-5044 | edge pop admission | Content Delivery & Edge Cache | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 273 | V5-5060 | order admission rate limit | Financial Markets & Trading Systems | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |
| 274 | V5-5065 | kernel bypass poll cores | Financial Markets & Trading Systems | power | at its limit on native | at its limit on native (the full clock): read and left native | Distribution | global |
| 275 | V5-5068 | end of day risk batch power | Financial Markets & Trading Systems | power | at its limit on native | at its limit on native (the full clock): read and left native | Distribution | global |
| 276 | V5-5120 | ids rule group admission | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Rank | # | Muscle (the setting's own name) | Family | Knob | Its hit | What Omni-Compass does with it | Realms | Its own card |
|---|---|---|---|---|---|---|---|---|
| 277 | V5-5123 | endpoint scan scheduling | Reliability, Security & Recovery | admission | at its limit on native | at its limit on native (nothing paced or turned away): read and left native | Distribution | global |

## E.3 How each family is wired, the family with the heaviest hitter first

Every family's muscles are wired the same way on a real stack; its muscles are named after its wire, heaviest hitter first, with each one's rank in its realm's table above.

### *E.3.1 Robotics Motion Control (26 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `robot_joint` (a robot joint servo), reached through the robot controller (ROS 2, EtherCAT, the drive's fieldbus). Omni-Compass reads its tracking error and its load.

- **Native controller:** the robot's own controller: ros2_control and MoveIt parameters, the drive's limits over EtherCAT or the vendor fieldbus.
- **Wire out (read):** joint tracking error, effort, temperature, cycle time (joint_states, the drive's registers).
- **Wire in (write):** velocity and acceleration scaling, trajectory tolerances and controller rates through ROS 2 parameters (ros2 param set) or the drive's limit registers; never past the manufacturer's limits.
- **Restore:** every parameter read once (ros2 param get) and restored (ros2 param set).
- **Run live?** modelled: MuJoCo (results/live/V3_MUJOCO.md); the ROS 2 path is written, not yet run on a physical arm.
- **Its muscles, heaviest hitter first:** controller update rate (Physics 1), humanoid gait speed (Physics 2), jerk limit (Physics 3), velocity acceleration scaling (Physics 5), joint velocity (Physics 6), joint position (Physics 7), balance correction (Physics 8), humanoid step height (Physics 10), locomotion speed (Physics 12), trajectory tolerances (Physics 13), joint acceleration (Physics 14), manipulator payload speed scaling (Physics 15), humanoid balance margin (Physics 16), collision margin (Physics 18), ros2 executor threads (Physics 20), cartesian velocity (Physics 22), trajectory speed (Physics 23), trajectory acceleration (Physics 24), steering angle (Physics 26), ros2 qos history depth (Physics 54), ros2 control loop rate (Physics 55), joint effort (Physics 56), force limit (Physics 57), gripper force (Physics 58), braking force (Physics 59), humanoid joint torque limit (Physics 136).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *E.3.2 Automotive EV & Mobile Powertrain (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `ev_traction` (an electric vehicle traction motor), reached through the vehicle's motor controller (CAN; modelled only). Omni-Compass reads its tracking error and its load.

- **Native controller:** the vehicle's motor and battery controllers (CAN, the OEM's diagnostic interface).
- **Wire out (read):** torque, speed, state of charge, temperatures (CAN signals).
- **Wire in (write):** torque, regeneration and charge limits within the OEM's envelope.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** vehicle speed envelope (Physics 4), state of charge limits (Physics 90), smart charging schedule (Physics 91), traction torque limit (Physics 125), regen braking level (Physics 126), battery charge limit (Physics 127), battery discharge limit (Physics 128), battery thermal target (Physics 129), motor thermal limit (Physics 130), energy recovery target (Physics 131), auxiliary power budget (Physics 132), fast charge current (Physics 133), fast charge voltage (Physics 134), vehicle safe state (Physics 135).

### *E.3.3 Facility & Grid Optimization (15 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `facility` (a facility battery behind the meter), reached through the energy management system. Omni-Compass reads its draw on the grid connection and its reserve.

- **Native controller:** the facility's energy management system and demand-response gateway (OpenADR).
- **Wire out (read):** site power, utility signals, PUE, carbon intensity (the EMS and the utility's feed).
- **Wire in (write):** the site power budget, demand limit and battery dispatch through the EMS.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** site battery dispatch (Energy 1), renewable dispatch (Energy 2), generator start stop (Energy 3), facility peak guard (Energy 4), pue target (Energy 9), cooling power budget (Energy 21), facility power budget (Energy 30), utility demand limit (Energy 31), it power budget (Energy 32), rack power allocation (Energy 56), demand response (Energy 170), electricity price gate (Energy 171), carbon intensity gate (Energy 172), grid frequency response (Energy 173), facility islanding (Energy 174).

### *E.3.4 Energy Storage & Microgrid (19 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `microgrid` (a microgrid battery and loads), reached through the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR). Omni-Compass reads its draw on the grid connection and its reserve.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Native controller:** the battery's or site's energy management system and inverter (SunSpec, IEEE 2030.5, OpenADR, Modbus).
- **Wire out (read):** state of charge, grid import and export, PV output, load (the EMS or inverter registers).
- **Wire in (write):** the reserve setpoint, charge and discharge limits, import limit and curtailment through SunSpec or Modbus registers.
- **Restore:** registers read once and restored.
- **Run live?** modelled: CityLearn (results/live/V3_CITYLEARN.md) for the districts; the register path is written.
- **Its muscles, heaviest hitter first:** volt var (Energy 5), storage dispatch (Energy 6), volt watt (Energy 7), peak shaving (Energy 8), microgrid emergency reserve (Energy 10), time of use schedule (Energy 11), battery soc reserve (Energy 12), electrolyzer power (Energy 13), heat pump power (Energy 14), pv curtailment (Energy 15), battery discharge power (Energy 16), battery charge power (Energy 18), grid export limit (Energy 25), ev charge power (Energy 27), grid import limit (Energy 28), constant power factor reactive power (Energy 29), microgrid demand limit (Energy 33), energy load shed (Energy 161), flex load admission (Energy 162).

### *E.3.5 Database & Transactions (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `database` (a database cluster), reached through the database operator (replicas, connection pools). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the database's own pooler and server settings: PgBouncer's admin console, MySQL SET GLOBAL, MongoDB setParameter, the database operator.
- **Wire out (read):** pool waits and active connections (SHOW POOLS), buffer-pool size and hit rate, cache size and evictions, p95 per statement.
- **Wire in (write):** PgBouncer default_pool_size through its console; MySQL innodb_buffer_pool_size by SET GLOBAL; MongoDB wiredTigerEngineRuntimeConfig cache_size by setParameter; replicas through the operator.
- **Restore:** the setting read once before the first write and restored; a failed read-back stops the writer.
- **Run live?** live: tools/run_pgbench.py, run_sysbench.py, run_ycsb.py, three-run tables V3_PGBENCH, V3_SYSBENCH, V3_YCSB.
- **Its muscles, heaviest hitter first:** io capacity (Distribution 1), vacuum compaction rate (Distribution 2), read route (Distribution 3), db failover (Distribution 4), db cache (Distribution 5), db pool resize (Distribution 7), shard placement (Distribution 8), db replicas (Distribution 9), db memory (Distribution 10), checkpoint rate (Distribution 11), buffer pool (Distribution 16), background writer (Distribution 17), synchronous replication (Distribution 18), parallel workers (Distribution 19), query concurrency

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

(Distribution 197), replication lag gate (Distribution 198), db write throttle (Distribution 199), transaction concurrency (Distribution 200), lock timeout (Distribution 201).

### *E.3.6 Commerce & Payment Systems (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `commerce` (a customer-facing service with bursts), reached through the Kubernetes API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the payment platform's gateway and its routing rules.
- **Wire out (read):** authorisation latency, decline and retry rates, queue depth (the platform's metrics).
- **Wire in (write):** concurrency, timeout and route weights through the platform's admin API.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** payment failover (Distribution 6), settlement batch (Distribution 12), order reservation (Distribution 13), authorization route (Distribution 14), processor route weight (Distribution 15), idempotency window (Distribution 20), refund queue rate (Distribution 21), payment admission (Distribution 236), payment concurrency (Distribution 237), payment retry (Distribution 238), payment timeout (Distribution 239), fraud review gate (Distribution 240), transaction queue limit (Distribution 241), inventory hold (Distribution 242), checkout load shed (Distribution 243).

### *E.3.7 Aviation & Autonomous Flight (28 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `microgrid` (a microgrid battery and loads), reached through the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR). Omni-Compass reads its draw on the grid connection and its reserve.

- **Native controller:** the flight controller's own parameters: PX4 (MPC_XY_VEL_MAX, MPC_Z_VEL_MAX_UP, MPC_ACC_HOR, MC_ROLLRATE_MAX) and ArduPilot (WPNAV_SPEED, ATC_ACCEL_R_MAX), the battery failsafe.
- **Wire out (read):** mission progress, battery state, tracking error and near-miss distance over MAVLink telemetry.
- **Wire in (write):** the speed, acceleration and rate parameters over MAVLink PARAM_SET within the autopilot's own limits; failsafes and geofence never loosened.
- **Restore:** every parameter read (PARAM_REQUEST_READ) once and written back at the end.
- **Run live?** modelled: the swarm cells (results/live/V3_SWARM.md); the MAVLink path is written, not yet flown.
- **Its muscles, heaviest hitter first:** drone battery failsafe reserve (Physics 9), swarm formation spacing (Physics 17), ardupilot wpnav speed (Physics 21), px4 mpc xy vel max (Physics 25), px4 mpc acc hor (Physics 37), altitude target (Physics 38), waypoint authority (Physics 39), px4 mc rollrate max (Physics 40), actuator saturation envelope (Physics 41), px4 mpc z vel max up (Physics 42), velocity target

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

(Physics 43), attitude rate target (Physics 44), attitude target (Physics 45), attitude rate gains (Physics 46), throttle envelope (Physics 63), flight hold (Physics 64), return to home (Physics 65), land action (Physics 66), mission admission (Physics 67), geofence response (Physics 68), failsafe selection (Physics 69), battery reserve threshold (Physics 70), flight mode transition (Physics 71), flight termination safe state (Physics 72), horizontal speed limit (Physics 87), vertical speed limits (Physics 88), acceleration jerk limits (Physics 89), ardupilot atc accel r max (Physics 137).

### *E.3.8 Robotics Fleet & Warehouse Automation (17 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `microgrid` (a microgrid battery and loads), reached through the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR). Omni-Compass reads its draw on the grid connection and its reserve.

- **Native controller:** the fleet manager's dispatch and charging rules (Open-RMF, the vendor fleet manager).
- **Wire out (read):** tasks waiting, robots active and charging, battery states (the fleet API).
- **Wire in (write):** active robot count, charge thresholds and zone admission through the fleet API.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** amr fleet charge threshold (Physics 11), agv speed (Physics 30), conveyor speed (Physics 31), task assignment (Physics 32), traffic reservation (Physics 33), elevator request (Physics 34), amr fleet active robots (Physics 35), robot route (Physics 36), battery reserve (Physics 47), robot dispatch (Physics 48), human safe stop (Physics 50), door request (Physics 51), charging dispatch (Physics 52), fleet failover (Physics 53), warehouse zone admission (Physics 60), robot quarantine (Physics 61), fleet concurrency (Physics 62).

### *E.3.9 Cloud VM & Capacity (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the cloud's autoscaling group or scale set: AWS Auto Scaling, Azure VMSS, GCP MIG.
- **Wire out (read):** group size, instance states, the target-tracking metric (the cloud's API and monitoring).
- **Wire in (write):** desired capacity, min and max, the target-tracking policy's target, through the cloud CLI or SDK.
- **Restore:** the group's desired, min, max and policy recorded once; restored from it.
- **Run live?** live on AKS through the node pool (family 5); the cloud SDK path itself is written, not yet run live.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Its muscles, heaviest hitter first:** architecture selection (Compute 1), accelerator selection (Compute 5), instance refresh (Compute 7), vm start stop (Compute 19), region selection (Compute 31), placement group (Compute 33), vm migrate (Compute 34), asg max (Compute 38), zone selection (Compute 39), interruption response (Compute 51), vm size (Compute 53), target tracking policy (Compute 54), spot mix (Compute 57), instance family (Compute 67), boot disk class (Compute 70), scaling cooldown warmup (Compute 71), vm count (Compute 82), predictive scaling (Compute 87), asg min (Compute 91), reservation mix (Compute 110), vm cpu cap by priority (Compute 400).

### *E.3.10 Kubernetes Dynamic Device Allocation (8 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** Kubernetes DRA: DeviceClass, ResourceClaim, ResourceSlice.
- **Wire out (read):** claims pending, devices allocated and tainted (the Kubernetes API).
- **Wire in (write):** DeviceClass selectors and claim capacity by kubectl apply; devices tainted, never removed.
- **Restore:** the objects recorded; restored.
- **Run live?** modelled; the API path is kubectl.
- **Its muscles, heaviest hitter first:** dra claim capacity (Compute 2), dra binding readiness (Compute 13), dra binding failure response (Compute 14), dra device class selection (Compute 50), dra device configuration (Compute 61), dra claim sharing (Compute 76), dra device taint (Compute 84), dra device eviction (Compute 412).

### *E.3.11 Cross-Cluster, Multi-Region & Edge (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the global load balancer and the fleet manager (Karmada, Fleet, the cloud's global LB).
- **Wire out (read):** per-region latency, capacity and cost, pending work (the global LB's and clusters' metrics).
- **Wire in (write):** region and cluster weights and quotas through the global controller's API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Its muscles, heaviest hitter first:** global failover (Compute 3), edge site node pool (Compute 11), edge offload (Compute 12), zone dispatch (Compute 15), cloud capacity class (Compute 28), workload migrate region (Compute 29), region dispatch (Compute 32), edge dispatch (Compute 37), multi cluster dispatch (Compute 40), cross cluster replication (Compute 49), edge sync interval (Compute 298), data residency gate (Compute 379), latency region gate (Compute 380), cost region gate (Compute 381), carbon region gate (Compute 382), federation quota (Compute 383), global admission (Compute 384), edge inference gpu clocks (Compute 444).

### *E.3.12 AI Inference Serving (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the serving engine's own autoscaler and batcher (vLLM, Triton, KServe, Ray Serve): replicas, batch size, concurrency, KV cache.
- **Wire out (read):** queue depth, tokens per second, p95 latency, KV-cache use from the server's /metrics.
- **Wire in (write):** the server's replica count or KServe/Knative autoscaling target, max_num_seqs and KV-cache budget through its API or its ConfigMap.
- **Restore:** the serving spec recorded once; restored to it.
- **Run live?** live in part: GPU serving under the card governor (results/gpu); the vLLM run is ready and waits on Lambda.
- **Its muscles, heaviest hitter first:** lora adapter load (Compute 4), continuous batching (Compute 8), model replicas (Compute 16), model instance count (Compute 17), max batch size (Compute 18), tensor parallel degree (Compute 35), batch queue delay (Compute 42), model route weight (Compute 45), chunked prefill (Compute 60), kv cache swap space (Compute 63), inference max tokens (Compute 77), disaggregated prefill decode (Compute 85), model load (Compute 86), model unload (Compute 93), scale to zero panic window (Compute 101), inference concurrency (Compute 352), kv cache budget (Compute 353), prefix cache budget (Compute 354), speculative decode budget (Compute 355), model precision (Compute 356), inference priority (Compute 357), inference slo gate (Compute 358), autoscaling target concurrency qps (Compute 395), request priority levels (Compute 396).

### *E.3.13 GPU Hardware (22 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the card's own firmware limits through NVML and nvidia-smi (power limit, application clocks, MIG, compute mode).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

- **Wire out (read):** nvidia-smi --query-gpu (clocks, power draw, temperature, utilisation, memory), the serving response time from the probe.
- **Wire in (write):** nvidia-smi -pl (power limit), -lgc/-rgc (clock ceiling and reset), -ac (application clocks), -mig and -c (modes), per card.
- **Restore:** every limit read once at start (the snapshot line in the GPU audit) and restored by the same commands at the end or on the lease.
- **Run live?** live: omni_controller/gpu_governor.py and scripts/gpu_paired.sh on a rented A10 (results/gpu); the new card runs wait on Lambda by the founder's order.
- **Its muscles, heaviest hitter first:** sync boost (Compute 6), gpu persistence mode (Compute 43), mig mode (Compute 44), gpu mps (Compute 72), gpu timeslice (Compute 74), gpu allocate (Compute 78), mig geometry (Compute 79), gpu ecc response (Compute 80), gpu compute mode (Compute 83), gpu temperature target (Compute 88), gpu reset (Compute 99), gpu power limit (Compute 342), gpu sm clock (Compute 343), gpu memory clock (Compute 344), gpu quarantine (Compute 345), gpu thermal limit (Compute 346), gpu job power budget (Compute 347), application clocks (Compute 394), target clocks (Compute 403), climatik inference power cap (Compute 406), adaptive gpu power cap (Compute 410), nvidia platform power budget (Compute 411).

### *E.3.14 Node Fleet & Karpenter-Class Control (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the cluster's own node autoscaler: Karpenter (NodePool), the Cluster Autoscaler, a MachineSet, a cloud node pool.
- **Wire out (read):** ready and schedulable workers, pending pods, allocatable and used CPU (Kubernetes API), the autoscaler's status ConfigMap.
- **Wire in (write):** the pool's size through the operator's own command (--node-scale-cmd: az aks nodepool scale, gcloud container clusters resize, eksctl scale nodegroup, kubectl scale machineset, a Karpenter NodePool limit); a machine parked by taint and drain, never deleted by Omni-Compass.
- **Restore:** the pool's starting count recorded once; every parked machine untainted and the pool restored to it at the end or on the lease.
- **Run live?** live: omni_controller nodepool mode on kind and on AKS (results/live/V1_AKS_*, the capacity and steady tables).
- **Its muscles, heaviest hitter first:** nodepool weight (Compute 9), node shutdown (Compute 21), max pods per node (Compute 22), node provision (Compute 23), node consolidate (Compute 24), node pool min (Compute 25), node pool max (Compute 36), expander (Compute 47), capacity class (Compute 52), consolidate after (Compute 55), scale down unneeded time (Compute 56), node replace (Compute 62),

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

node desired (Compute 64), kubelet reserved resources (Compute 66), consolidation policy (Compute 68), node power on (Compute 73), node pool limits (Compute 75), expire after (Compute 81), scale down delay after add (Compute 90), node cordon (Compute 326), node drain (Compute 327), disruption budget (Compute 328), scale down utilisation threshold (Compute 390), kubelet eviction thresholds (Compute 391).

### *E.3.15 Virtualization & Hypervisor (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the hypervisor's management API (libvirt, vSphere, Hyper-V).
- **Wire out (read):** vCPU steal, balloon state, migration load (the hypervisor's metrics).
- **Wire in (write):** vCPU allocation, ballooning and migration pacing through the management API.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** vm overcommit density (Compute 10), vgpu partition (Compute 46), hypervisor cpu pinning (Compute 89), vcpu allocation (Compute 142), memory ballooning (Compute 166), sriov vf allocation (Compute 212), numa vcpu pinning (Compute 284), live migration pacing (Compute 420), vm memory dedup ksm (Compute 421).

### *E.3.16 Medical Imaging & Clinical Systems (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `clinical` (clinical systems: imaging archives, records, interface engines, monitoring gateways), reached through the application's own scaling and admission settings (Kubernetes, the vendor console). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the clinical application's own scaling and admission settings (Kubernetes, the vendor console).
- **Wire out (read):** queue depths, interface backlog, reconstruction wait (the application's metrics).
- **Wire in (write):** replicas, concurrency and rate limits through the Kubernetes API or the vendor console.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** ehr application replicas (Distribution 22), pacs archive tier target (Distribution 24), reconstruction gpu workers (Distribution 26), dicom router concurrency (Distribution 252), hl7 interface queue limit (Distribution 253), fhir api rate limit (Distribution 254), modality worklist timeout (Distribution 255), patient monitoring gateway capacity (Distribution 256), lab analyzer batch

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

window (Distribution 257), telehealth session admission (Distribution 258), clinical backup window (Distribution 259), imaging prefetch priority (Distribution 260).

### *E.3.17 Financial Markets & Trading Systems (14 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the trading platform's gateway, matching engine and risk settings, the colocation BMS.
- **Wire out (read):** order latency, session counts, risk-check backlog, market-data fanout (the platform's metrics).
- **Wire in (write):** thread and session pools, conflation intervals and batch sizes through the platform's configuration; risk checks never relaxed.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** backtest grid nodes (Distribution 23), order gateway sessions (Distribution 67), pre trade risk check workers (Distribution 80), matching engine threads (Distribution 81), market data fanout threads (Distribution 124), surveillance alert queue workers (Distribution 144), post trade settlement workers (Distribution 157), market data conflation interval (Distribution 173), fix session heartbeat interval (Distribution 174), tick store write batch (Distribution 175), colocation rack cooling setpoint (Distribution 176), order admission rate limit (Distribution 273), kernel bypass poll cores (Distribution 274), end of day risk batch power (Distribution 275).

### *E.3.18 Game Servers & Real-Time Interactive (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** Agones Fleets and the matchmaker, the game node pool.
- **Wire out (read):** allocated and ready servers, matchmaking waits, tick rate (Agones and the matchmaker's metrics).
- **Wire in (write):** Fleet replicas and buffer sizes by kubectl patch; matchmaking admission through its API.
- **Restore:** the Fleet spec recorded; restored.
- **Run live?** modelled; the API path is kubectl patch.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Its muscles, heaviest hitter first:** agones fleet max replicas (Compute 20), game node pool autoscaler (Compute 41), voice chat mixer pool (Compute 158), agones fleet buffer size (Compute 162), agones fleet min replicas (Compute 165), relay server pool (Compute 198), replay render batch (Compute 268), game server tick rate (Compute 293), matchmaking wait window (Compute 294), players per server cap (Compute 435), matchmaking admission rate (Compute 436), cloud gaming gpu clocks (Compute 437).

### *E.3.19 CI/CD & Build Farms (10 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the runner scale set (actions-runner-controller, GitLab runners) and the build cache.
- **Wire out (read):** jobs queued, runners busy, cache hit rate (the controller's metrics).
- **Wire in (write):** min and max runners, build concurrency and cache size by kubectl patch of the scale set.
- **Restore:** the scale set spec recorded; restored.
- **Run live?** modelled; the API path is kubectl patch.
- **Its muscles, heaviest hitter first:** warm runner pool (Compute 26), runner scale set max runners (Compute 30), build cache size (Compute 214), test shard parallelism (Compute 258), runner scale set min runners (Compute 259), remote execution workers (Compute 262), artifact retention ttl (Compute 297), build concurrency limit (Compute 441), job queue admission (Compute 442), build node cpu frequency (Compute 443).

### *E.3.20 OpenShift & Machine API (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** OpenShift's Machine API operators: MachineHealthCheck, MachineConfigPool, the Cluster Version Operator.
- **Wire out (read):** machine phases, MCP status and pause, degraded counts (the OpenShift API).
- **Wire in (write):** MachineHealthCheck maxUnhealthy, MachineConfigPool paused and maxUnavailable by oc patch.
- **Restore:** the fields recorded before the first write, restored from the record.
- **Run live?** modelled; written against the documented OpenShift API, not yet run live.
- **Its muscles, heaviest hitter first:** node config rollout (Compute 27), machine remediation (Compute 48), mcp max unavailable (Compute 59), cluster version pacing (Compute 69), machine failure domain

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

(Compute 92), machine health gate (Compute 329), mcp pause (Compute 330), operator reconcile budget (Compute 331), infra machine admission (Compute 332).

### *E.3.21 Messaging & Streaming (22 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the broker's own controls: Kafka's admin API and consumer group, RabbitMQ policies, MQTT broker settings.
- **Wire out (read):** consumer lag, messages per second, p95 end to end, broker CPU (the broker's metrics, the consumer group's offsets).
- **Wire in (write):** the consumer group's size (consumers running), quotas and partition counts through the admin API.
- **Restore:** the group's size recorded; restored at the end.
- **Run live?** live: tools/run_kafka.py, V3_KAFKA.
- **Its muscles, heaviest hitter first:** broker cluster nodes (Distribution 25), fetch batch size (Distribution 31), rebalance rate (Distribution 47), broker failover (Distribution 50), dead letter divert (Distribution 54), replication factor (Distribution 69), partition count (Distribution 71), producer batch size (Distribution 74), message retention (Distribution 78), partition placement (Distribution 92), prefetch (Distribution 103), memory disk alarm (Distribution 107), mqtt broker max connections (Distribution 110), mqtt max inflight messages (Distribution 178), mqtt session expiry (Distribution 179), producer quota (Distribution 202), consumer quota (Distribution 203), broker io quota (Distribution 204), queue depth limit (Distribution 205), consumer concurrency (Distribution 206), retry backoff (Distribution 207), stream priority (Distribution 208).

### *E.3.22 Quantum Computing Control Simulation (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `qpu` (a quantum or specialised accelerator queue), reached through the accelerator's job queue. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the quantum control stack's job queue and pulse schedules (Qiskit Runtime, the vendor's control software).
- **Wire out (read):** queue depth, calibration drift, error rates (the stack's metrics).
- **Wire in (write):** shot allocation, circuit admission and scheduling through the stack's API; pulse parameters never written live.
- **Restore:** recorded; restored.
- **Run live?** modelled only.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Its muscles, heaviest hitter first:** pulse phase (Compute 58), measurement scheduling (Compute 65), reset scheduling (Compute 163), gate scheduling (Compute 167), pulse duration (Compute 181), qubit mapping (Compute 188), dynamical decoupling (Compute 193), shot allocation (Compute 194), noise aware routing (Compute 196), coupling control (Compute 201), pulse amplitude (Compute 211), circuit admission (Compute 374), circuit scheduling (Compute 375), pulse frequency (Compute 376), error mitigation budget (Compute 377), quantum queue priority (Compute 378).

### *E.3.23 Service Mesh & API Reliability (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the mesh's own policies: Istio DestinationRule and VirtualService, Linkerd ServiceProfile, Envoy filters.
- **Wire out (read):** request rate, error rate, p95 and outlier ejections from the mesh's telemetry.
- **Wire in (write):** connection pool, outlier detection, retry budget and traffic weights by kubectl patch of the policy objects.
- **Restore:** the policy objects recorded; restored.
- **Run live?** modelled; the API path is kubectl patch.
- **Its muscles, heaviest hitter first:** service failover (Distribution 27), canary weight (Distribution 40), dns traffic weight (Distribution 41), load balancing policy (Distribution 49), traffic mirror (Distribution 62), session affinity (Distribution 66), traffic divert (Distribution 68), request hedging (Distribution 88), connection pool (Distribution 130), circuit breaker (Distribution 188), retry budget (Distribution 189), service timeout (Distribution 190), service concurrency (Distribution 191), outlier ejection (Distribution 192), health threshold (Distribution 193), fault injection gate (Distribution 194).

### *E.3.24 PDU, UPS & Electrical Distribution (18 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `ups` (an uninterruptible power supply), reached through the UPS and PDU management interface (SNMP, Modbus). Omni-Compass reads its draw on the grid connection and its reserve.

- **Native controller:** the UPS and PDU management interfaces (SNMP, Modbus) and the servers' BMC power caps (IPMI DCMI, Redfish).
- **Wire out (read):** load per branch, UPS mode and charge, power per server (SNMP, Redfish).
- **Wire in (write):** server and rack power caps through Redfish or DCMI; UPS charge rate and mode through SNMP; breakers never written.
- **Restore:** every cap and mode read once and restored.
- **Run live?** modelled.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Its muscles, heaviest hitter first:** pdu outlet control (Energy 17), phase balance (Energy 19), generator dispatch (Energy 20), ups shutdown battery test (Energy 22), ups operating mode (Energy 23), reactive power target (Energy 24), voltage target (Energy 26), server power cap (Energy 40), rack power cap (Energy 41), pdu branch power limit (Energy 42), ups charge rate (Energy 43), ups discharge rate (Energy 44), hierarchical power budget row pdu switchboard controllers (Energy 62), priority aware capping throttle low priority first (Energy 63), power oversubscription with prediction (Energy 64), load transfer (Energy 163), electrical isolation (Energy 164), breaker trip gate (Energy 165).

### *E.3.25 Runtime & Application (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the application runtime's own settings: thread pools, JVM heap and GC, worker counts, connection pools.
- **Wire out (read):** queue waits, GC pauses, worker busy share, p95 from the application's metrics.
- **Wire in (write):** worker and pool sizes through the application's admin endpoint, environment or config reload; JVM flags at restart only.
- **Restore:** the settings recorded; restored (a restart-bound setting is left native).
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** background workers (Distribution 28), runtime memory (Distribution 29), thread pool (Distribution 30), go runtime (Distribution 34), event loop workers (Distribution 36), jvm heap (Distribution 52), process count (Distribution 55), keepalive connection reuse (Distribution 56), application cache size (Distribution 58), worker count (Distribution 89), runtime restart (Distribution 93), connection pool runtime (Distribution 95), dns resolver cache size (Distribution 123), mail queue concurrency (Distribution 139), dns negative cache ttl (Distribution 180), mail retry interval (Distribution 181), gc budget (Distribution 222), async concurrency (Distribution 223), request timeout (Distribution 224), runtime cpu budget (Distribution 225), runtime io budget (Distribution 226).

### *E.3.26 Marine Propulsion & Vessel Automation (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `marine_propulsion` (a ship's propulsion shaft and power plant), reached through the vessel automation and power management system (modelled only). Omni-Compass reads its tracking error and its load.

- **Native controller:** the vessel's automation and power management system.
- **Wire out (read):** shaft speed and power, fuel rate, engine load (the automation system).
- **Wire in (write):** shaft speed and power limits, engine staging within the manufacturer's envelope.
- **Restore:** recorded; restored.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** shaft speed target (Physics 19), engine load sharing setpoint (Physics 27), trim target (Physics 28), auxiliary engine staging (Physics 29), slow steaming speed target (Physics 49), propeller pitch limit (Physics 106), shaft power limit (Physics 107), bow thruster duty (Physics 108), ballast pump mode (Physics 109), shore power mode (Physics 110), hotel load budget (Physics 111), rudder rate limit (Physics 112).

### *E.3.27 Serverless & Functions (FaaS) (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the function platform's concurrency settings (Knative autoscaling annotations, AWS Lambda concurrency, OpenFaaS).
- **Wire out (read):** concurrency, cold starts, queue waits (the platform's metrics).
- **Wire in (write):** provisioned and reserved concurrency, scale-to-zero windows through the platform's API or annotations.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** fn snapstart restore (Compute 94), fn provisioned concurrency (Compute 123), fn prewarm pool (Compute 128), fn reserved concurrency (Compute 129), fn memory size (Compute 145), fn concurrency limit (Compute 186), fn timeout budget (Compute 283), fn scale to zero idle (Compute 418), fn burst admission (Compute 419).

### *E.3.28 Kubernetes Placement & Scheduling (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** kube-scheduler and its profiles, the descheduler, PriorityClasses, scheduling gates.
- **Wire out (read):** pods pending and their reasons, node fill, topology spread (Kubernetes API).
- **Wire in (write):** a pod's or deployment's placement fields (nodeSelector, affinity, topologySpreadConstraints, priorityClassName, schedulingGates) by kubectl patch, or the scheduler's profile.
- **Restore:** the fields as they stood before the first write, restored field by field.
- **Run live?** modelled; the API path is the same kubectl patch the live controller uses for the HPA.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Its muscles, heaviest hitter first:** deschedule (Compute 95), cpu manager (Compute 96), peaks power aware placement (Compute 100), scheduler scoring strategy (Compute 105), topology spread (Compute 106), pod anti affinity (Compute 120), taint toleration (Compute 126), device claim (Compute 127), scheduler plugin weights (Compute 140), numa placement (Compute 141), network locality (Compute 147), gpu topology (Compute 169), storage locality (Compute 175), node selector (Compute 176), failure domain spread (Compute 182), node affinity (Compute 197), priority class (Compute 322), preemption policy (Compute 323), scheduler backoff (Compute 324), gang admission (Compute 325), scheduling gates (Compute 389).

### *E.3.29 Observability & Telemetry (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the collector's own configuration: the OpenTelemetry Collector, Prometheus scrape and remote-write settings.
- **Wire out (read):** dropped samples, queue depth, memory, cardinality (the collector's self-metrics).
- **Wire in (write):** the collector's batch, queue and memory limits and scrape intervals through its config reload.
- **Restore:** the config recorded; reloaded from it.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** collector memory limit (Distribution 32), per pod energy attribution (Distribution 39), retention window (Distribution 53), remote write queue (Distribution 82), metric scrape interval (Distribution 90), batching (Distribution 102), export concurrency (Distribution 227), cardinality budget (Distribution 228), telemetry shed (Distribution 229).

### *E.3.30 Reliability, Security & Recovery (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the platform's own recovery and security controls: rollout and rollback, PodDisruptionBudgets, the security hold, a kill switch.
- **Wire out (read):** error rate, rollout status, the security hold (a ConfigMap key), the kill file.
- **Wire in (write):** rollout pause and undo by kubectl; the security hold read, never written; the kill switch (tools/omni_switch.py) is a human's.
- **Restore:** every write recorded and undone on the lease.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Run live?** live in part: the security hold and kill switch run in every Kubernetes benchmark (omnicompass/master.py).
- **Its muscles, heaviest hitter first:** degraded mode (Distribution 33), backup trigger (Distribution 46), waf inspection workers (Distribution 59), traffic divert recovery (Distribution 70), rollback (Distribution 73), policy enforcement (Distribution 83), restart (Distribution 101), ids capture threads (Distribution 127), siem ingest pipeline workers (Distribution 148), ids ring buffer size (Distribution 177), actuator freeze (Distribution 230), workload isolate (Distribution 231), credential rotation gate (Distribution 232), rate abuse gate (Distribution 233), fault domain isolate (Distribution 234), kill switch (Distribution 235), ids rule group admission (Distribution 276), endpoint scan scheduling (Distribution 277).

### *E.3.31 Kubernetes Workload Scaling (32 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the HorizontalPodAutoscaler, KEDA ScaledObject or VPA that owns the workload's scale.
- **Wire out (read):** the HPA's current and desired replicas, the Deployment's ready replicas (Kubernetes API, metrics-server), the service's response time from the probe.
- **Wire in (write):** the HPA's target (spec.metrics averageUtilization) and floor (minReplicas) by kubectl patch; KEDA through its ScaledObject, so the add-on keeps its own loop.
- **Restore:** the operator's target and range recorded in an annotation before the first write, restored from it, annotation removed.
- **Run live?** live: scripts/kind_bench.sh and omni_controller (HPA target, KEDA, Goldilocks, CruiseKube), three-run tables in results/live.
- **Its muscles, heaviest hitter first:** vpa minmax allowed resources (Compute 97), hpa cpu target (Compute 98), rollout rate (Compute 104), hpa custom target (Compute 107), hpa scale down stabilisation window (Compute 108), max surge (Compute 111), vpa apply (Compute 117), replicas (Compute 125), keda polling interval (Compute 130), graceful termination (Compute 131), scale to zero (Compute 137), hpa scale up stabilisation window (Compute 144), rollout readiness delay (Compute 148), hpa memory target (Compute 149), probes (Compute 154), hpa scaling rate policies (Compute 157), hpa max replicas (Compute 161), pdb policy (Compute 164), hpa min replicas (Compute 170), kepler energy target autoscaling (Compute 171), hpa policy selection (Compute 177), hpa tolerance (Compute 180), hpa sync period (Compute 189), max unavailable (Compute 190), keda threshold (Compute 315), deployment pause resume (Compute 316), rollout abort (Compute 317), pod eviction (Compute 318), scheduler queue priority (Compute 319), api priority fairness (Compute 320), rollout progress deadline (Compute 385), unhealthy pod eviction policy (Compute 386).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *E.3.32 Cache & Memory Services (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the cache's own configuration: Redis CONFIG SET, Memcached runtime settings.
- **Wire out (read):** hit rate, evictions, used memory, p95 (INFO, the cache's stats).
- **Wire in (write):** maxmemory and eviction policy by CONFIG SET; replicas and shards through the operator.
- **Restore:** the setting read once (CONFIG GET) before the first write and restored (CONFIG SET).
- **Run live?** live: tools/run_redis.py, V3_REDIS.
- **Its muscles, heaviest hitter first:** cache size (Distribution 35), cache failover (Distribution 38), cache sharding (Distribution 42), cache replicas (Distribution 45), cache writeback rate (Distribution 51), cache ttl (Distribution 57), memory size threads (Distribution 61), cache prefetch (Distribution 65), cache memory limit (Distribution 86), cache compression (Distribution 94), cache warmup (Distribution 96), cache flush rate (Distribution 98), cache eviction policy (Distribution 209), cache admission (Distribution 210), hot key isolation (Distribution 211), cache connection limit (Distribution 212).

### *E.3.33 Search, Indexing & Vector DB (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the search engine's settings API: OpenSearch/Elasticsearch cluster and index settings, the vector database's config.
- **Wire out (read):** indexing and search latency, queue rejections, merge pressure (the engine's stats API).
- **Wire in (write):** index refresh interval, merge throttling, shard counts and search thread pools through the settings API.
- **Restore:** settings recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** index workers (Distribution 37), embedding workers (Distribution 43), segment merge rate (Distribution 48), vector batch size (Distribution 60), shard count (Distribution 63), shard replication (Distribution 64), query route (Distribution 72), shard rebalance (Distribution 75), vector search k (Distribution 79), index refresh rate (Distribution 87), search concurrency (Distribution 213), search timeout (Distribution 214), index memory budget (Distribution 215), hot shard isolation (Distribution 216), search admission (Distribution 217), circuit breakers (Distribution 251).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *E.3.34 Container Resources (25 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the pod's own resource requests and limits (the Deployment template) and, where it runs, the VPA, Goldilocks or CruiseKube recommender.
- **Wire out (read):** each pod's CPU use (kubectl top), its request and limit, the node's allocatable CPU (Kubernetes API).
- **Wire in (write):** the running pod's CPU limit in place through pods/resize (no restart); the template is never changed, so no rollout.
- **Restore:** the operator's limit recorded in an annotation before the first write; every pod returned to it.
- **Run live?** live: omni_controller/muscles.py convey() (conveyance); KEDA and Goldilocks tables.
- **Its muscles, heaviest hitter first:** default limits (Compute 102), io latency target (Compute 109), cpuset (Compute 112), memory high (Compute 113), cpu weight (Compute 114), pids limit (Compute 118), memory limit (Compute 119), memory hard limit (Compute 121), goldilocks vpa request recommendation (Compute 122), runtime class (Compute 124), in place resize (Compute 134), cruisekube request rightsizing (Compute 138), cgroup io max (Compute 146), cpu limit (Compute 152), memory protection (Compute 153), swap limit (Compute 155), io weight (Compute 156), cpu request (Compute 159), ephemeral storage (Compute 160), hugepages (Compute 168), memory reclaim (Compute 178), memory request (Compute 184), cpu quota (Compute 321), namespace quota (Compute 387), freeze (Compute 388).

### *E.3.35 Linux Kernel & OS Control (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the Linux kernel's own tunables: cgroup v2 files, /proc/sys/vm and net, the cpufreq and cpuidle governors, the I/O scheduler.
- **Wire out (read):** /sys/fs/cgroup, /proc/pressure, /proc/vmstat, /sys/block queue stats, the frequency files.
- **Wire in (write):** cgroup cpu.weight, cpu.max and memory.high; sysctl vm and net keys; scaling_max_freq, through sysfs and sysctl (root).
- **Restore:** every file snapshotted once and restored byte for byte.
- **Run live?** the CPU frequency path is written (tools/run_cpu_power.py); the other knobs are registered and queued, they need a real Linux machine.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Its muscles, heaviest hitter first:** irqbalance affinity (Compute 103), hugepage pool reserve (Compute 116), cgroup cpu weight (Compute 135), cgroup cpu quota (Compute 173), cgroup memory high (Compute 174), tcp buffer autotune (Compute 195), io scheduler depth (Compute 199), block nr requests (Compute 200), net rx coalesce (Compute 202), cfs scheduler latency (Compute 275), transparent hugepages (Compute 276), numa auto balancing (Compute 277), dirty ratio writeback (Compute 278), vm swappiness (Compute 279), zswap pool fraction (Compute 280), page cache reclaim (Compute 281), net qdisc codel target (Compute 282), cpufreq dvfs governor (Compute 415), ksm memory dedup (Compute 416), cpu cstate governor (Compute 417).

### *E.3.36 Telecom RAN & Edge Radio (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `ran` (a radio access network cell or site), reached through the RAN controller (O-RAN interfaces). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the RAN's near-real-time RIC and O-RAN interfaces (E2, O1).
- **Wire out (read):** PRB use, connected UEs, cell load, handover rates (the RIC's KPIs).
- **Wire in (write):** scheduler weights, slice budgets and traffic steering through the RIC's xApps; transmit power within the licence.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** ran scheduler weight (Distribution 44), ran ue handover (Distribution 76), ran antenna tilt (Distribution 77), ran carrier enable (Distribution 84), ran cell sleep (Distribution 85), ran prb allocation (Distribution 91), ran du cu placement (Distribution 100), ran cell traffic steering (Distribution 105), ran connection admission (Distribution 267), ran slice resource budget (Distribution 268), ran tx power (Distribution 269), ran fronthaul budget (Distribution 270).

### *E.3.37 Host CPU & Memory (26 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the Linux kernel's own governors: cpufreq (schedutil or the processor driver), RAPL, cpuidle, the memory manager.
- **Wire out (read):** /sys/devices/system/cpu/cpufreq (frequencies and governor), /sys/class/powercap intel-rapl energy counters, /proc/stat busy share, /proc/pressure.
- **Wire in (write):** scaling_max_freq per policy and the RAPL package power limit through sysfs (root); on Kubernetes through the Kubernetes Power Manager's PowerProfile or EAR's policy.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Restore:** the governor's files snapshotted once and restored byte for byte (tools/run_cpu_power.py --restore).
- **Run live?** written and tested on the modelled machine (tools/run_cpu_power.py, cpu-power.yml); GitHub's machines are virtual and cannot write the governor: the live run needs a real Linux machine.
- **Its muscles, heaviest hitter first:** swappiness reclaim (Compute 115), core packing (Compute 133), irq affinity (Compute 136), memory bandwidth (Compute 139), llc allocation (Compute 143), page reclaim rate (Compute 150), turbo boost (Compute 172), transparent hugepages (Compute 183), numa balance (Compute 185), core online offline (Compute 187), cpufreq min (Compute 333), cpufreq max (Compute 334), rapl package power (Compute 335), uncore frequency (Compute 336), energy perf preference (Compute 337), cpu idle policy (Compute 338), memory pressure gate (Compute 339), thermal throttle policy (Compute 340), host power profile (Compute 341), frequency governor (Compute 392), dram power limit (Compute 393), per pod cpu frequency range and c state access (Compute 401), uncore frequency per node (Compute 402), ear cpu frequency policy (Compute 407), ear global energy budget (Compute 408), k8s power manager core profile (Compute 409).

### *E.3.38 Media Streaming & CDN (14 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the streaming stack's transcoder pools, packager and CDN settings.
- **Wire out (read):** transcode queue, GPU clocks, buffer health, origin load (the stack's metrics).
- **Wire in (write):** worker pools, bitrate-ladder cap and cache TTLs through the stack's API; GPU clocks through nvidia-smi (family 9).
- **Restore:** recorded; restored.
- **Run live?** modelled; the GPU path is live (family 9).
- **Its muscles, heaviest hitter first:** drm license server pool (Compute 132), packager workers (Compute 151), origin shield cache size (Compute 191), transcoder worker pool (Compute 233), thumbnail sprite batch (Compute 248), abr bitrate ladder cap (Compute 288), player buffer target (Compute 289), edge cache ttl (Compute 290), segment duration target (Compute 291), encoder lookahead frames (Compute 292), live transcode gpu clocks (Compute 431), vod transcode batch power (Compute 432), cdn request coalescing (Compute 433), media ingest admission (Compute 434).

### *E.3.39 Blockchain & Ledger Nodes (10 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Native controller:** the node's configuration (cache sizes, peer limits, mempool) and its host's power profile.
- **Wire out (read):** block relay latency, mempool size, peer counts, validation backlog (the node's RPC).
- **Wire in (write):** cache and pool sizes through the node's config and RPC; the host's power profile through family 8.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** script validation threads (Compute 179), archive node storage tier (Compute 203), node db cache size (Compute 204), max peer connections (Compute 210), indexer batch workers (Compute 250), mempool max size (Compute 295), utxo cache flush interval (Compute 296), block relay admission (Compute 438), rpc work queue depth (Compute 439), validator host power profile (Compute 440).

### *E.3.40 Storage Block/File/Object (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the storage system's QoS and tiering: Ceph (ceph osd pool set, QoS), a cloud volume's IOPS class, S3 lifecycle.
- **Wire out (read):** IOPS, throughput, latency and queue depth from the storage system's metrics.
- **Wire in (write):** volume IOPS and throughput limits, replica counts and tiering through the storage API or CSI VolumeAttributesClass.
- **Restore:** the volume's class and limits recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** lifecycle tiering (Distribution 97), erasure code profile (Distribution 99), throughput limit (Distribution 104), scrub schedule (Distribution 106), volume placement (Distribution 109), volume size (Distribution 111), cache allocation (Distribution 116), object replication (Distribution 118), storage tier (Distribution 119), backfill rate (Distribution 120), replica count (Distribution 128), recovery rate (Distribution 129), rebalance (Distribution 131), io scheduler (Distribution 132), snapshot trigger (Distribution 133), compaction pressure (Distribution 134), iops limit (Distribution 140), storage admission (Distribution 195), degraded storage gate (Distribution 196), disk power spin down (Distribution 250).

### *E.3.41 Skin: Perimeter & Admission (5 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the perimeter's own controls: the WAF, the DDoS shield, the TLS terminator.
- **Wire out (read):** request admission, inspection sampling, session cache hit rate (the perimeter's metrics).
- **Wire in (write):** token-bucket rates, sampling and session cache size through the perimeter's API; blocking rules never loosened.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** tls session cache (Compute 192), rate limit token bucket (Compute 427), connection admission (Compute 428), waf inspection sampling (Compute 429), ddos shield admission (Compute 430).

### *E.3.42 Network Routing & Switching (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the network controller: the load balancer's weights, the SDN controller, the NIC's ethtool and tc settings.
- **Wire out (read):** link utilisation, drops, queue latency (SNMP, the controller's API, ethtool -S).
- **Wire in (write):** LB and route weights through the controller's API; rate limits and queue disciplines through tc; NIC queues through ethtool.
- **Restore:** every weight and setting recorded; restored with the same commands.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** bandwidth limit (Distribution 108), nic interrupt coalescing (Distribution 112), nic queue count (Distribution 113), route weight (Distribution 115), qos class (Distribution 117), nic ring size (Distribution 121), lb weight (Distribution 125), queue discipline (Distribution 126), congestion control (Distribution 135), path selection (Distribution 136), failover route (Distribution 137), ecmp weight (Distribution 138), rate limit (Distribution 182), connection limit (Distribution 183), nic rate limit (Distribution 184), egress budget (Distribution 185), ingress budget (Distribution 186), network isolation (Distribution 187), port link power (Distribution 249).

### *E.3.43 Content Delivery & Edge Cache (6 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the CDN's configuration API and the edge cache's settings.
- **Wire out (read):** hit ratio, origin load, TTLs (the CDN's metrics).
- **Wire in (write):** cache TTLs, origin-shield concurrency and prefetch through the CDN API.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** origin shield concurrency (Distribution 114), edge cache size (Distribution 122), cdn cache ttl (Distribution 171), prefetch aggressiveness (Distribution 172), tiered cache promotion (Distribution 271), edge pop admission (Distribution 272).

### *E.3.44 GPU Fabric & RDMA (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the fabric manager and the collective library (NVIDIA Fabric Manager, NCCL settings, RDMA CM).
- **Wire out (read):** link and NVSwitch counters (nvidia-smi nvlink, DCGM), collective time per step from the training job.
- **Wire in (write):** NCCL environment and the job's rank placement at launch; RDMA QoS through the fabric manager's API.
- **Restore:** the launch settings recorded; the next job launched with the originals.
- **Run live?** modelled; written against NCCL and Fabric Manager documentation.
- **Its muscles, heaviest hitter first:** rdma bandwidth (Compute 205), nic affinity (Compute 207), congestion response (Compute 213), nvlink placement (Compute 215), nvswitch route (Compute 218), rail selection (Compute 219), collective protocol (Compute 220), rank placement (Compute 221), gpudirect policy (Compute 222), nic hca selection (Compute 224), collective algorithm (Compute 228), fabric failover (Compute 229), rdma route (Compute 230), collective chunk size (Compute 232), gpu fabric quarantine (Compute 348), collective concurrency (Compute 349), communication priority (Compute 350), fabric isolation (Compute 351).

### *E.3.45 DPU SmartNIC & Programmable IO (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the DPU's own management (NVIDIA DOCA, the SmartNIC's SDK) and SR-IOV.
- **Wire out (read):** per-function rates, flow-steering hits, DPU CPU and memory (the DPU's counters).
- **Wire in (write):** rate limits, QoS groups and VF counts through the DPU management API.
- **Restore:** recorded; restored.
- **Run live?** modelled.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Its muscles, heaviest hitter first:** dpu sf tx rate (Compute 206), smartnic flow steering (Compute 208), dpu service placement (Compute 209), sriov vf count (Compute 216), smartnic offload enable (Compute 217), dpu pf tx rate (Compute 223), dpu vf tx rate (Compute 225), dpu bandwidth share (Compute 226), dpu qos group (Compute 227), dpu failover (Compute 231), dpu cpu budget (Compute 413), dpu memory budget (Compute 414).

### *E.3.46 Workflow, Logistics & Fulfillment (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `workflow` (a workflow or pipeline engine), reached through the workflow engine's concurrency settings. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the workflow engine (Temporal, Airflow, Camunda) and the fulfilment system's dispatch rules.
- **Wire out (read):** task queue depth, worker utilisation, SLA misses (the engine's metrics).
- **Wire in (write):** worker rates, concurrency and retry policies through the engine's API.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** route replan (Distribution 141), compensation action (Distribution 142), shipment batch (Distribution 143), warehouse queue (Distribution 154), workflow worker rate (Distribution 159), inventory allocation (Distribution 160), fulfillment route (Distribution 161), carrier selection (Distribution 162), task queue rate (Distribution 163), sla escalation (Distribution 164), workflow admission (Distribution 244), workflow retry (Distribution 245), workflow backoff (Distribution 246), workflow timeout (Distribution 247), dispatch priority (Distribution 248).

### *E.3.47 AI Training (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu_batch` (GPU training and batch work), reached through nvidia-smi, the job scheduler's pause and resume. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the training job's launcher and scheduler (Kubeflow, Ray Train, Slurm, torchrun).
- **Wire out (read):** step time, GPU utilisation, checkpoint cadence, stragglers (the job's metrics).
- **Wire in (write):** the job's worker count (elastic training), batch sizes and checkpoint interval through the launcher's API or the job spec.
- **Restore:** the job spec recorded; the next run launched from it.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** checkpoint trigger (Compute 234), straggler mitigation (Compute 235), training workers (Compute 239), data parallelism (Compute 240), gradient accumulation (Compute 242),

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

checkpoint interval (Compute 246), pipeline parallelism (Compute 252), expert parallelism (Compute 254), tensor parallelism (Compute 255), global batch size (Compute 256), elastic worker count (Compute 265), microbatch size (Compute 269), dataloader workers (Compute 270), training gang size (Compute 271), compute comm overlap (Compute 272), activation checkpointing (Compute 273), training preempt (Compute 359), training precision (Compute 360).

### *E.3.48 Data Analytics & ETL (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** Spark's dynamic allocation and the SQL engine's slots (spark.dynamicAllocation, Flink's parallelism, warehouse slots).
- **Wire out (read):** executor count, task backlog, shuffle spill, stage times (the engine's metrics).
- **Wire in (write):** dynamic allocation min and max, parallelism and slots through the engine's config at submit or its REST API.
- **Restore:** the submit settings recorded; the next job launched from them.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** stage parallelism (Distribution 145), executor size (Distribution 146), shuffle bandwidth (Distribution 147), cache fraction (Distribution 149), shuffle partitions (Distribution 150), dynamic allocation max (Distribution 151), speculation policy (Distribution 152), batch interval (Distribution 153), dynamic allocation min (Distribution 155), query slots (Distribution 156), data locality wait (Distribution 158), etl concurrency (Distribution 218), spill threshold (Distribution 219), stream backpressure (Distribution 220), analytics admission (Distribution 221).

### *E.3.49 HPC & Distributed Compute (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the batch scheduler: Slurm (scontrol, sacctmgr), PBS, LSF.
- **Wire out (read):** queue depth, node states, job wait times, power per node (sacct, sinfo, the power plugin).
- **Wire in (write):** partition limits, job priority and fair-share factors, node power-saving settings through scontrol and the scheduler config.
- **Restore:** the scheduler's settings recorded; restored with the same commands.
- **Run live?** modelled; EAR (family 8) is the live energy path on HPC.
- **Its muscles, heaviest hitter first:** rank mapping (Compute 236), backfill policy (Compute 238), job slots (Compute 241), job walltime (Compute 244), checkpoint restart (Compute 245), mpi ranks (Compute

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

247), node allocation (Compute 249), accelerator share (Compute 251), cpu gpu ratio (Compute 260), partition limits (Compute 263), memory per rank (Compute 274), job priority (Compute 361), job preemption (Compute 362), parallel io budget (Compute 363), collective budget (Compute 364), scratch budget (Compute 365), scheduler fair share (Compute 366), node power saving (Compute 397), cpu frequency per job (Compute 398).

### *E.3.50 Distributed Cluster Managers (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the cluster manager's scheduler: YARN, Mesos, Nomad, Ray.
- **Wire out (read):** queue lengths, resource reservations, preemptions (the manager's API).
- **Wire in (write):** quotas, reservations and preemption settings through the manager's API.
- **Restore:** recorded once, restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** worker allocation (Compute 237), resource reservation (Compute 243), oversubscription (Compute 253), resource reclaim (Compute 257), gang schedule (Compute 261), binpack pressure (Compute 264), spread pressure (Compute 266), maintenance evacuation (Compute 267), task admission (Compute 367), task priority (Compute 368), cluster quota (Compute 369), task preemption (Compute 370), queue fairness (Compute 371), deadline pressure (Compute 372), scheduler retry (Compute 373).

### *E.3.51 Ports & Maritime Logistics (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `port` (a container terminal's cranes and vehicles taking moves), reached through the terminal operating system and fleet controllers. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the terminal operating system and the equipment fleet controllers.
- **Wire out (read):** moves waiting, cranes active, gate queue, equipment charge (the TOS).
- **Wire in (write):** crane and vehicle allocation, gate rate and charging windows through the TOS.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** rail mounted gantry dispatch (Distribution 165), quay crane allocation (Distribution 166), stacking height target (Distribution 167), horizontal transport fleet size (Distribution 168), yard crane fleet size (Distribution 169), shore power connection capacity (Distribution 170), berth window admission (Distribution 261), truck gate rate limit (Distribution 262), reefer plug power budget

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

(Distribution 263), container dwell priority (Distribution 264), vessel arrival pacing (Distribution 265), equipment charging window (Distribution 266).

### *E.3.52 Industrial PLC & Process Automation (18 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the PLC or DCS loop (OPC UA, EtherNet/IP, PROFINET): the setpoint and the mode of each loop.
- **Wire out (read):** the process value, setpoint, output and alarm state of each loop (OPC UA reads).
- **Wire in (write):** the loop's setpoint within its safe band and its actuator range by OPC UA write; safety interlocks are never written.
- **Restore:** every setpoint read once and restored; a value outside the band is refused by the plug.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** valve position (Energy 34), furnace setpoint (Energy 35), pressure setpoint (Energy 36), temperature setpoint (Energy 37), mass flow setpoint (Energy 38), tank level target (Energy 39), plc cycle authority (Energy 153), machine cell admission (Energy 154), pump flow (Energy 155), compressor speed (Energy 156), heater power (Energy 157), conveyor rate (Energy 158), feed rate (Energy 159), purge vent action (Energy 160), controller mode (Energy 175), alarm limits (Energy 176), safety interlock trip (Energy 177), opc ua writes (Energy 178).

### *E.3.53 Cooling, Chillers & Thermodynamics (18 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `data_hall` (a data hall cooling plant), reached through the building management system (BACnet, Modbus) or its command line. Omni-Compass reads its temperature against its limit.

- **Native controller:** the building management system and the chiller plant's own supervisory controller (BACnet, Modbus).
- **Wire out (read):** supply and return temperatures, chiller load, fan and pump speeds, rack inlet temperatures (BACnet reads).
- **Wire in (write):** supply-air and chilled-water setpoints, fan and pump speeds within their safe bands by BACnet write; the plant's own staging keeps control.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled (the data-hall plant); the BACnet path is written.
- **Its muscles, heaviest hitter first:** supply air temperature (Energy 45), return air target (Energy 46), coolant supply temperature (Energy 47), coolant flow (Energy 48), pump speed (Energy 49), fan speed (Energy 50), chiller setpoint (Energy 51), cooling tower fan (Energy 52), cooling capacity (Energy 53), gpu thermal envelope (Energy 54), cpu thermal envelope (Energy 55), server fan speed (Energy 57),

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

economiser (Energy 58), chiller plant supervisory control rl with safety layer (Energy 65), compressor authority (Energy 166), rack thermal budget (Energy 167), thermal workload migrate (Energy 168), thermal load shed (Energy 169).

### *E.3.54 Building & Critical Environment HVAC (15 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `building` (a building zone), reached through the building management system. Omni-Compass reads its temperature against its limit.

- **Native controller:** the building management system (BACnet) and the AHU and zone controllers.
- **Wire out (read):** zone temperatures, airflows, duct static pressure, occupancy (BACnet reads).
- **Wire in (write):** zone setpoints, fan speeds and damper positions within their bands by BACnet write; the BMS's own sequences keep control.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled: CityLearn districts (results/live/V3_CITYLEARN.md); the BACnet path is written.
- **Its muscles, heaviest hitter first:** setpoint reset trim and respond (Energy 59), duct static pressure setpoint (Energy 60), optimal start stop (Energy 61), zone temperature target (Energy 145), zone airflow (Energy 146), ahu fan speed (Energy 147), damper position (Energy 148), economizer position (Energy 149), boiler setpoint (Energy 150), humidity target (Energy 151), thermal storage dispatch (Energy 152), heat pump mode (Energy 246), occupancy ventilation (Energy 247), building demand limit (Energy 248), hvac emergency mode (Energy 249).

### *E.3.55 Healthcare Critical Environments (13 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `hospital` (a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites)), reached through the hospital's building management system (BACnet). Omni-Compass reads its temperature against its limit.

- **Native controller:** the hospital's building management system (BACnet) with its clinical limits.
- **Wire out (read):** room pressures, air changes, temperatures and humidity (BACnet reads).
- **Wire in (write):** setpoints within the clinical envelope by BACnet write; pressure cascades and alarms never loosened.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** operating room air change setpoint (Energy 66), isolation room pressure target (Energy 67), patient room temperature setpoint (Energy 68), surgical suite humidity target (Energy 69), ahu supply air temperature (Energy 70), pharmacy cold room setpoint (Energy 71), sterile storage humidity setpoint (Energy 72), imaging suite cooling capacity (Energy 73), chiller plant staging (Energy 74), exhaust fan capacity (Energy 75), ward night setback mode (Energy 181), medical gas plant demand limit (Energy 182), emergency power load shed (Energy 183).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *E.3.56 Agriculture & Irrigation (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `irrigation` (pumps, pressure and climate on a farm or in a greenhouse), reached through the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the pump station controller, pivot panel or climate computer (Modbus, the vendor cloud).
- **Wire out (read):** pressure, soil moisture, temperature, flow (the controller's registers).
- **Wire in (write):** pump speed and climate setpoints within their bands by Modbus write.
- **Restore:** registers read once and restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** mainline pressure setpoint (Energy 76), soil moisture target (Energy 77), pivot speed setpoint (Energy 78), greenhouse temperature setpoint (Energy 79), greenhouse co2 target (Energy 80), vent position setpoint (Energy 81), grain dryer temperature setpoint (Energy 82), cold storage temperature setpoint (Energy 83), milking vacuum level setpoint (Energy 84), well drawdown level target (Energy 85), irrigation pump speed (Energy 184), fertigation dose (Energy 185), barn ventilation rate (Energy 186), drip zone dispatch (Energy 187).

### *E.3.57 Oil & Gas Pipelines (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `pipeline` (a pipeline segment with its compressors, pumps and valves), reached through pipeline SCADA (DNP3, Modbus, OPC UA). Omni-Compass reads how far the process sits from its band.

- **Native controller:** pipeline SCADA (DNP3, Modbus, OPC UA).
- **Wire out (read):** pressures, flows, line pack, compressor states (SCADA reads).
- **Wire in (write):** pressure and flow setpoints and unit staging within the safe envelope; protection never written.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** compressor discharge pressure setpoint (Energy 86), pump station suction pressure target (Energy 87), line pack target (Energy 88), pipeline flow setpoint (Energy 89), terminal tank level target (Energy 90), heater outlet temperature setpoint (Energy 91), valve position target (Energy 92), cathodic protection voltage (Energy 93), compressor unit staging (Energy 188), vfd pump speed (Energy 189), leak detection shutdown (Energy 190), batch interface dispatch (Energy 191), drag reducing agent dose (Energy 192), pressure protection priority (Energy 193).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *E.3.58 Mining & Mineral Processing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `mill` (a grinding, flotation or materials-handling circuit), reached through the plant DCS or PLC (OPC UA). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the plant DCS or PLC (OPC UA).
- **Wire out (read):** mill load, speed, pressures, densities, power (OPC UA reads).
- **Wire in (write):** loop setpoints within their bands by OPC UA write; shutdowns never written.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** sag mill load setpoint (Energy 94), mill speed target (Energy 95), crusher gap setpoint (Energy 96), cyclone feed pressure target (Energy 97), thickener underflow density target (Energy 98), conveyor speed setpoint (Energy 99), dewatering pump level target (Energy 100), stockpile level target (Energy 101), flotation aeration rate (Energy 194), reagent dose (Energy 195), ventilation on demand airflow (Energy 196), slurry pump speed (Energy 197), tailings discharge shutdown (Energy 198), ore blend dispatch (Energy 199).

### *E.3.59 District Heating & Cooling (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `district_heat` (a district heating or cooling network), reached through the network control system and substation controllers (Modbus, M-Bus). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the network control system and substation controllers (Modbus, M-Bus).
- **Wire out (read):** supply and return temperatures, differential pressure, storage level (the controllers).
- **Wire in (write):** supply temperature and pressure setpoints, pump speed and staging within their bands.
- **Restore:** registers read once and restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** supply temperature setpoint (Energy 102), return temperature target (Energy 103), differential pressure setpoint (Energy 104), thermal storage level target (Energy 105), outdoor reset setpoint (Energy 106), cooling network supply temperature (Energy 107), cooling storage level target (Energy 108), network pump speed (Energy 200), heat pump staging (Energy 201), chp dispatch (Energy 202), peak boiler heater output (Energy 203), substation flow limit (Energy 204), peak demand shed (Energy 205).

### *E.3.60 Power Generation & Turbine Control (14 muscles)*

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `turbine` (a generating unit's governor, boiler or excitation loop), reached through the plant DCS and governor (OPC UA, IEC 61850; modelled only). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the plant DCS and the unit's governor (OPC UA, IEC 61850).
- **Wire out (read):** unit load, frequency, pressures, temperatures (the DCS).
- **Wire in (write):** load setpoints and ramp limits within the dispatch envelope; protection and rod positions never written.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** turbine speed droop setpoint (Energy 109), unit load setpoint (Energy 110), boiler steam pressure setpoint (Energy 111), feedwater level target (Energy 112), excitation voltage setpoint (Energy 113), hydro gate position target (Energy 114), combustion air ratio setpoint (Energy 115), cooling water flow setpoint (Energy 116), inlet guide vane position (Energy 117), nuclear rod position target (Energy 118), agc participation limit (Energy 206), reserve dispatch priority (Energy 207), turbine ramp rate limit (Energy 208), emissions shutdown (Energy 209).

### *E.3.61 Renewable Generation & Inverter Control (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `inverter` (a plant of grid-support inverters or wind turbines), reached through the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547).
- **Wire out (read):** active and reactive power, voltage, curtailment state (SunSpec registers).
- **Wire in (write):** volt-var and volt-watt setpoints, curtailment and ramp limits by SunSpec write within the interconnection agreement.
- **Restore:** registers read once and restored.
- **Run live?** modelled: SimBench grids solved by pandapower (results/live/V3_PANDAPOWER.md); the register path is written.
- **Its muscles, heaviest hitter first:** inverter volt var setpoint (Energy 119), inverter volt watt setpoint (Energy 120), plant reactive power target (Energy 121), rotor speed setpoint (Energy 122), inverter frequency droop setpoint (Energy 123), string mppt voltage setpoint (Energy 124), active power curtailment (Energy 210), wind turbine yaw offset (Energy 211), pitch angle limit (Energy 212), ramp rate limit (Energy 213), tracker stow mode (Energy 214), noise mode schedule (Energy 215), ice detection shutdown (Energy 216).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *E.3.62 Pharmaceutical & Food Manufacturing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `batch_reactor` (a GMP batch reactor, fermenter or food process), reached through the batch control system (ISA-88, OPC UA). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the batch control system (ISA-88, OPC UA) with the validated recipe limits.
- **Wire out (read):** process values and phase states (OPC UA reads).
- **Wire in (write):** setpoints within the validated recipe band only; a value outside it is refused by the plug.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** reactor temperature setpoint (Energy 125), agitator speed setpoint (Energy 126), fermenter dissolved oxygen target (Energy 127), ph setpoint (Energy 128), pasteurizer holding temperature setpoint (Energy 129), freezer tunnel temperature setpoint (Energy 130), cleanroom pressure cascade setpoint (Energy 131), lyophilizer shelf temperature setpoint (Energy 132), chromatography flow setpoint (Energy 133), oven zone temperature setpoint (Energy 134), cip cycle dispatch (Energy 217), steam sterilizer cycle priority (Energy 218), refrigerant compressor authority (Energy 219), batch hold quarantine (Energy 220).

### *E.3.63 Grid Transmission & Distribution (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `feeder_voltage` (a distribution feeder's voltage), reached through the distribution management system (IEC 61850, DNP3; modelled only). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the distribution management system and the devices' own controllers (IEC 61850, DNP3): regulators, capacitor banks, inverters.
- **Wire out (read):** feeder voltages, loads, tap positions, inverter output (the DMS).
- **Wire in (write):** regulator and transformer taps, capacitor switching and inverter reactive power within the planning envelope; protection never written.
- **Restore:** every setting read once and restored.
- **Run live?** modelled: SimBench grids (results/live/V3_PANDAPOWER.md).
- **Its muscles, heaviest hitter first:** voltage regulator tap (Energy 135), transformer tap (Energy 136), feeder voltage target (Energy 137), frequency droop setpoint (Energy 138), capacitor bank switch (Energy 221), inverter real power (Energy 222), inverter reactive power (Energy 223), feeder load transfer (Energy 224), distribution storage dispatch (Energy 225), demand response dispatch (Energy 226), grid protection mode (Energy 227), grid restoration sequence (Energy 228).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *E.3.64 Semiconductor Fab & Precision Manufacturing (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `chamber` (a controlled chamber (clean room, kiln, reactor)), reached through the PLC or DCS. Omni-Compass reads how far the process sits from its band.

- **Native controller:** the fab's MES and each tool's controller (SECS/GEM).
- **Wire out (read):** tool queues, chamber states, process values (MES and GEM).
- **Wire in (write):** lot dispatch and tool admission through the MES; chamber recipes and parameters never written outside the qualified window.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** chamber temperature (Energy 139), chamber pressure (Energy 140), run to run control (Energy 179), idle sleep mode (Energy 180), tool job dispatch (Energy 229), wafer route (Energy 230), chamber recipe selection (Energy 231), gas flow (Energy 232), rf power (Energy 233), vacuum pump speed (Energy 234), robot transfer rate (Energy 235), lot priority (Energy 236), tool quarantine (Energy 237).

### *E.3.65 Water Wastewater & Pumping (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `water` (a water or pumping process), reached through the SCADA system (Modbus, DNP3). Omni-Compass reads how far the process sits from its band.

- **Native controller:** the utility's SCADA (Modbus, DNP3) and the pump station controllers.
- **Wire out (read):** levels, pressures, flows, aeration and dosing (SCADA reads).
- **Wire in (write):** pump speeds, level and pressure targets, aeration and dosing within their bands by SCADA write; emergency shutdowns never written.
- **Restore:** every setpoint read once and restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** valve position water (Energy 141), reservoir level target (Energy 142), line pressure target (Energy 143), flow target water (Energy 144), pump speed water (Energy 238), aeration rate (Energy 239), chemical dose rate (Energy 240), filtration backwash (Energy 241), lift station dispatch (Energy 242), leak isolation (Energy 243), water demand shed (Energy 244), water emergency shutdown (Energy 245).

### *E.3.66 Gut: Data-plane Intake & Waste (8 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Native controller:** the data plane's own intake pacing and compaction settings (the ingress, the queue, the store's compactor).
- **Wire out (read):** intake rate, backpressure, compaction debt, dead-letter depth (the components' metrics).
- **Wire in (write):** pacing, drain rates and compaction budgets through each component's settings API.
- **Restore:** recorded; restored.
- **Run live?** modelled.
- **Its muscles, heaviest hitter first:** dead letter retry cap (Compute 285), ttl eviction waste (Compute 286), retry budget window (Compute 287), ingress intake pacing (Compute 422), gc compaction pacing (Compute 423), backpressure threshold (Compute 424), queue drain rate (Compute 425), compaction debt drain (Compute 426).

### *E.3.67 Work Admission & Demand Shaping (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half).

- **Native controller:** the service's own admission layer: its API gateway, queue or admission controller (Envoy, NGINX, Kong, a job queue's concurrency setting).
- **Wire out (read):** queue depth, requests in flight and the response-time line from the gateway's or queue's metrics (Prometheus, the admin API).
- **Wire in (write):** the gateway's or queue's concurrency and rate settings (the limit named by the muscle) through its admin API or its config object.
- **Restore:** snapshot of each limit once before the first write; every limit restored from it.
- **Run live?** at its limit on native: read and left native in every run (nothing is paced or turned away); the wire is read-only.
- **Its muscles, heaviest hitter first:** api concurrency (Compute 299), queue concurrency (Compute 300), queue backpressure (Compute 301), job admission (Compute 302), batch admission (Compute 303), inference admission (Compute 304), load shed (Compute 305), priority gate (Compute 306), tenant admission (Compute 307), burst limit (Compute 308), deadline admission (Compute 309), work budget (Compute 310), request queue limit (Compute 311), retry admission (Compute 312), background work gate (Compute 313), maintenance work gate (Compute 314), request concurrency limit (Compute 399), adaptive concurrency limit (Compute 404), priority request queue under limit (Compute 405).

### *E.3.68 Spacecraft & Flight Software (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `reaction_wheel` (a spacecraft reaction wheel), reached through the attitude controller (modelled only). Omni-Compass reads its tracking error and its load.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **Native controller:** the flight software's command and mode tables (cFS, F Prime).
- **Wire out (read):** attitude error, wheel speeds, power and thermal state from telemetry.
- **Wire in (write):** payload duty cycle, communication allocation and wheel allocation through commands; safe modes never commanded.
- **Restore:** the command table recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** space command admission (Physics 73), flight task schedule (Physics 74), space mode transition (Physics 75), payload duty cycle (Physics 76), communication allocation (Physics 77), space power budget (Physics 78), space thermal command (Physics 79), attitude command envelope (Physics 80), reaction wheel allocation (Physics 81), rcs authority (Physics 82), safe mode transition (Physics 83), watchdog recovery (Physics 84), instrument activation (Physics 85), fault isolation (Physics 86).

### *E.3.69 Rail Traction & Train Control (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `rail_traction` (a train's traction and auxiliary systems), reached through the train control and management system and ATO (modelled only). Omni-Compass reads its tracking error and its load.

- **Native controller:** the train control and management system and ATO.
- **Wire out (read):** speed, traction effort, timetable margin, auxiliary power (TCMS telemetry).
- **Wire in (write):** traction and coasting targets within the ATO's envelope.
- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** traction effort limit (Physics 92), regenerative braking share (Physics 93), coasting speed target (Physics 94), dwell time schedule (Physics 95), hvac duty cycle (Physics 96), train auxiliary power budget (Physics 97), platform approach speed (Physics 98), acceleration rate setpoint (Physics 99), wheel slip protection mode (Physics 100), traction motor thermal derate (Physics 101), catenary voltage limit (Physics 102), timetable recovery margin (Physics 103), headway target (Physics 104), door release hold (Physics 105).

### *E.3.70 Elevators & Vertical Transport (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `elevator_hoist` (an elevator or escalator drive and its group control), reached through the lift controller and destination dispatch (modelled only). Omni-Compass reads its tracking error and its load.

- **Native controller:** the lift controller and destination dispatch.
- **Wire out (read):** calls waiting, car positions, drive temperature (the controller).
- **Wire in (write):** speed and dispatch modes within the manufacturer's limits; safety modes never written.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Restore:** recorded; restored.
- **Run live?** modelled only.
- **Its muscles, heaviest hitter first:** hoist speed target (Physics 113), acceleration limit (Physics 114), regen drive mode (Physics 115), standby power mode (Physics 116), destination dispatch schedule (Physics 117), car parking mode (Physics 118), door dwell hold (Physics 119), escalator speed target (Physics 120), group capacity mode (Physics 121), motor thermal derate (Physics 122), brake test schedule (Physics 123), fire recall mode (Physics 124).

APPENDIX

# Appendix F. The Azure Account, Click by Click

> Evaluation and Simulation use only; any other use requires a signed, paid Omni-Compass Enterprise License. See `LICENSE`.

This connects GitHub to an Azure account so the workflow `aks-metered` can build a real Azure Kubernetes Service cluster, run native Kubernetes (with Azure's own cluster autoscaler) against the same with Omni-Compass on top, count the machines Azure bills every 15 seconds, and delete everything when it is done. The design is written before the run in `docs/K8S_COMPASS_PREREGISTRATION.md`, "The bill on a real cloud". It costs about USD 10 to 30 of Azure time; a new account's free credit usually covers it. About ten minutes, once.

## F.1 An Azure account

1. Open **azure.microsoft.com/free** and click **Start free** (or **Try Azure for free**).
2. Sign in with a Microsoft account, or make one.
3. Fill in the form. Azure asks for a card to confirm who you are; the free credit is used first.
4. When it says the account is ready, go to **portal.azure.com**.

## F.2 Azure's own terminal (nothing to install)

1. In the Azure portal, at the top of the page, click the >_ icon (Cloud Shell), to the right of the search bar.
2. If it asks, choose **Bash** (not PowerShell).
3. If it asks about storage, choose **No storage account required** (or accept the default and **Create**).
4. A black terminal opens at the bottom of the page.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## F.3 One line: a login for GitHub

Paste this line into that terminal and press **Enter**:

```
az ad sp create-for-rbac --name omni-compass-github --role contributor
    ↪  --scopes /subscriptions/$(az account show --query id -o tsv) --
    ↪ sdk-auth
```

It prints a block of text that starts with `{` and ends with `}`. Select all of it, from `{` to `}`, and copy it. That block is a password for this one purpose; never paste it anywhere but the GitHub secret below.

## F.4 Put it in GitHub

1. Open the repository on GitHub.
2. Click **Settings** (the tab at the top right of the repository, not your account settings).
3. On the left: **Secrets and variables**, then **Actions**.
4. Click **New repository secret**.
5. **Name:** `AZURE_CREDENTIALS`
6. **Secret:** paste the whole block from step 3.
7. Click **Add secret**.

## F.5 Run it

**Actions** tab, then **aks-metered** on the left, then **Run workflow**, then the green **Run workflow** button. The defaults are the preregistered design (5 repetitions, native and native with Omni-Compass on top, 900 measured seconds, Standard_D2s_v4 in eastus, the whole engine at the max on top of Azure's own autoscaler, the bill judged beside the work: the founder's order of 31 October). About 6 hours on GitHub. Each cluster is deleted when its arm ends, and the resource group at the end of every repetition whatever happens, so nothing keeps billing.

If Azure answers **quota exceeded**: in the portal, search **Subscriptions**, open yours, and upgrade it to **Pay-As-You-Go** (the credit stays), or ask for a quota of 10 vCPUs for the DSv4 family in the region.

## F.6 Afterwards

The result is the job `aggregate`'s receipt: billed machine-hours and the bill at list price, native against native with Omni-Compass on top, beside response time, failures and CPU, each with its 95% interval. To remove GitHub's access when you are done: in the Azure portal search **App registrations**, open **omni-compass-github**, **Delete**; and delete the `AZURE_CREDENTIALS` secret in GitHub.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## F.7 When Azure's check says the key is not whole

Every Azure set, and the conductor's own check at no cost (the run **cloud-check azure** on the **Actions** tab), reads the shape of the login before Azure's login and says what is wrong without printing a character of it: that a box is empty, that it holds an ID where a password goes, that it was cut short, or which field is missing.

### *F.7.1 The four boxes, one value each (the simplest way)*

Since 24 October the login can be stored one value to a box, exactly as the portal shows each value, with nothing to type around it. The check builds the block from them; the boxes are read only when `AZURE_CREDENTIALS` does not hold the whole block. All four values come from the same app.

1. In the Azure portal, search **App registrations**, click **All applications**, and open the app (or **+ New registration**, name `omni-compass-github-2`, **Register**). Its **Overview** shows three numbers; skip **Object ID**.
2. On GitHub, open **https://github.com/The-Omni-Compass-LLC/The-Omni-Compass-LLC/settings/secrets/actions**. For each box below, click its pencil (or **New repository secret** if it is not listed), paste the one value, and save:
   - `AZURE_CLIENT_ID`: the app's **Application (client) ID**.
   - `AZURE_TENANT_ID`: the app's **Directory (tenant) ID**.
   - `AZURE_SUBSCRIPTION_ID`: search **Subscriptions** in the portal, open yours, copy its **Subscription ID**.
   - `AZURE_CLIENT_SECRET`: on the app, **Certificates & secrets**, **+ New client secret**, **Add**, then copy the **Value** column at once (about 40 characters; Azure shows it once). Never the **Secret ID** beside it: that is another 36-character number, not a password.
3. Let the app build machines: **Subscriptions**, yours, **Access control (IAM)**, **+ Add**, **Add role assignment**, **Privileged administrator roles**, **Contributor**, **Next**, **+ Select members**, the app's name, **Select**, **Review + assign**.
4. Leave `AZURE_CREDENTIALS` as it is or empty it; while it is not the whole block, the four boxes are used.

### *F.7.2 The one box, the whole block*

A set already running keeps the key it started with: on 24 October the steady set started on 19 October was still signing in at every repetition while new runs read 88 characters in the box (the size of a storage account's access key, not the block). So make the new block under a new name, which leaves the login a running set still uses untouched; running the line of section F.3 with its old name again can replace that login's password, and a repetition that cannot sign in at its end cannot delete what it built.

1. In the Azure portal, click >_ (Cloud Shell) and choose **Bash**, as in section F.2.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. Paste this line and press **Enter** (the line of section F.3 with the name `omni-compass-github-2`). It prints a new block that starts with `{` and ends with `}`:

``bash az ad sp create-for-rbac --name omni-compass-github-2 --role contributor --scopes /subscriptions/$(az account show --query id -o tsv) --sdk-auth ``

3. Select all of it, from `{` to `}`, and copy it.
4. On GitHub, open the repository, then **Settings**, then **Secrets and variables**, then **Actions**.
5. Beside `AZURE_CREDENTIALS`, click the pencil (**Update**).
6. Paste the whole block as the new value and click **Update secret**.
7. Once no Azure set is running (the **Actions** tab shows no run named `aks-metered` in progress or queued), the older login may be retired: in the Azure portal search **App registrations**, open **omni-compass-github**, and click **Delete**. Nothing in the repository names it.

The conductor runs Azure's check again (an hour after a failure, the wait doubling with every failure in a row up to a day; start it by hand to have it read at once: **Actions**, **cloud-check**, **Run workflow**, `azure`), and the sets start on their own once it passes. Nothing is built and nothing is billed while it fails.

## F.8 The second way in: the sign-in without a password (OIDC)

Since 23 October, evening, the repository also holds a sign-in with no password at all: three IDs (`AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`) and a rule in Azure that trusts this repository's own runs. The inventory (`azure-inventory.yml`) uses it. On 24 October it was refused: Azure answered that no trust rule matched the name GitHub presented (`AADSTS700213`), because GitHub now presents this repository by its ID numbers, and the rule in Azure names it the older way. One rule with the exact name makes it work:

1. In the Azure portal, search **App registrations** and open the app whose **Application (client) ID** is the one stored as `AZURE_CLIENT_ID`.
2. Click **Certificates & secrets**, then the tab **Federated credentials**, then **+ Add credential**.
3. **Federated credential scenario**: choose **Other issuer**.
4. **Issuer**: `https://token.actions.githubusercontent.com`
5. **Subject identifier** (type it exactly, or copy it from this page): `repo:The-Omni-Compass-LLC@334396871/The-Omni-Compass@1390458092:ref:refs/heads/main`
6. **Name**: `github-main`. Leave **Audience** at `api://AzureADTokenExchange`. Click **Add**.
7. The app needs a role on the subscription: **Subscriptions**, the subscription, **Access control (IAM)**, **Add role assignment**, **Reader** for the inventory.

The benchmark sets keep the key of section F.3. A sign-in without a password holds for about an hour and cannot renew itself inside a job, while one Azure repetition runs about 70 minutes and must still be signed in at its end to delete what it built; a lapsed sign-in there could leave machines running and billing. So the sets,

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

and Azure's $0 check that clears them, sign in with a password: `AZURE_CREDENTIALS` whole, or the four boxes of section F.7.1 (the three IDs here and `AZURE_CLIENT_SECRET`), and nothing is started while neither is whole.

APPENDIX

# Appendix G. The Google Cloud Account, Click by Click

**Status (24 October): not yet run, its queue switched off.** The key has not been stored whole. Everything here stays ready, wired as Azure's sets run (the same harness and 60 s beat, 100 paired repetitions a set under the stopping rule, the $0 check first); the founder switches it on once the key is stored whole (section G.4), and nothing is built or billed before.

This connects GitHub to a Google Cloud account so two workflows can run the same tests Azure runs:

- `gke-metered`: a real Google Kubernetes Engine cluster, native Kubernetes (with Google's own cluster autoscaler) against the same with Omni-Compass on top, the machines Google bills counted every 15 seconds, the organisms with the cluster inside at 1, 10 and 100 copies (`organism`, `organism_scale`), everything deleted at the end; every set with the whole engine at the max on top of the cloud's own autoscaler, the bill judged beside the work (31 October). The same design as Azure's `aks-metered` (`docs/K8S_COMPASS_PREREGISTRATION.md`).
- `cloud-organism` with `cloud` = `gcp`: the big organisms (the whole tower and the four stacked, the real cluster inside, at 100 copies) on one rented 8-processor machine, the same test Azure's `big-organism-detached` runs. With `test` = `direct` the same button runs Omni direct's three arms instead (native, Omni on native and Omni direct, no Kubernetes; `docs/OMNI_V18.md`), `reps` runs on the one machine; it starts only on the founder's word.

About fifteen minutes, once. New accounts get **$300 of free credit for 90 days**; Google does not charge the card during the free trial.

## G.1 A Google Cloud account with the free credit

1. On the laptop, open **cloud.google.com/free** and click **Get started for free**.
2. Sign in with a Google account (a Gmail address works), or make one.
3. Choose the country, tick the terms, click **Continue**.
4. Enter a card. Google uses it to confirm who you are; the $300 credit is used first.
5. Click **Start my free trial**. The Google Cloud console opens (**console.cloud.google.com**).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

6. **Turn on the full account** (the credit stays and is still used first; the free trial otherwise limits how many machines run at once and refuses GPUs): at the top of the console click **Activate** (or open the menu ☰, **Billing**, then **Upgrade** / **Activate full account**) and confirm.

## G.2 A project for Omni-Compass

1. At the top left of the console, click the project picker (next to "Google Cloud").
2. Click **New project**. Name: `omni-compass`. Click **Create**.
3. When it is made, pick it in the project picker. Under the name you see its **Project ID** (for example `omni-compass-123456`). Write it down: it is the second secret below.

## G.3 Google's own terminal (nothing to install)

1. At the top right of the console, click the >_ icon (**Activate Cloud Shell**).
2. A terminal opens at the bottom of the page. If it asks to authorize, click **Authorize**.

## G.4 One paste: a login for GitHub

Paste this whole block into that terminal and press **Enter** (it takes about a minute). It is safe to paste again: a login already made is kept, and a fresh key is made each time.

```
P=$(gcloud config get-value project)
gcloud services enable container.googleapis.com compute.googleapis.com
    ↪  iam.googleapis.com
gcloud iam service-accounts create omni-compass-github --display-name
    ↪ "Omni-Compass GitHub"
for r in roles/container.admin roles/compute.admin roles/iam.serviceAc
    ↪ countUser; do
  gcloud projects add-iam-policy-binding "$P" --member "serviceAccount
    ↪ :omni-compass-github@$P.iam.gserviceaccount.com" --role "$r" --q
    ↪ uiet >/dev/null
done
gcloud iam service-accounts keys create key.json --iam-account "omni-c
    ↪ ompass-github@$P.iam.gserviceaccount.com"
cloudshell download key.json
```

The browser asks to download **key.json**: click **Download**. Never copy the key from the terminal's screen: a block that long is cut short when it is selected there (on 18 October the stored key ended part of the way through, and every Google set failed at the login). Then:

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

1. On the laptop, open the **Downloads** folder, right-click **key.json**, **Open with**, **Notepad** (on a Mac: **TextEdit**).
2. Press **Ctrl+A** (select all), then **Ctrl+C** (copy). The whole file is about 2,300 characters, from `{` to `}`.
3. Back in Cloud Shell, type `rm key.json` and press **Enter**; on the laptop, delete **key.json** from **Downloads** once the secret below is saved.

That file is a password for this one purpose: never paste it anywhere but the GitHub secret below. (If the project picker showed no project, run `gcloud config set project YOUR-PROJECT-ID` first.)

**A new key file by clicks alone** (once the paste above has run, the login exists and only its key is needed again):

1. In the console menu ☰, **IAM & Admin**, then **Service Accounts**.
2. Click **omni-compass-github@…** (the address ends `.iam.gserviceaccount.com`).
3. Open the **Keys** tab, click **Add key**, then **Create new key**, choose **JSON**, click **Create**. The browser downloads the key file (its name is the Project ID and some letters, ending `.json`).
4. Open that file in Notepad as in step 1 above, **Ctrl+A**, **Ctrl+C**, and store it as below. On the same **Keys** tab, an older key can be deleted with its bin icon.

## G.5 Put the two secrets in GitHub

1. Open the repository on GitHub, click **Settings** (the repository's tab, at the top right), then on the left **Secrets and variables**, then **Actions**.
2. **New repository secret** (or, when it is already there, the pencil beside it: **Update**). Name: `GCP_CREDENTIALS`. Secret: click in the box, **Ctrl+V** (the whole file from step 4). **Add secret** (or **Update secret**).
3. **New repository secret**. Name: `GCP_PROJECT`. Secret: the Project ID from step 2. **Add secret**.

That is all. To have it read at once, start it by hand: **Actions**, **cloud-check**, **Run workflow**, choose `google`, **Run workflow**. Otherwise the conductor runs it on its own: after a failed check it waits an hour, and the wait doubles with every failure in a row (two, four, eight, sixteen hours) up to a day, so a key not yet stored is read once a day, not every hour. The check costs nothing (**Actions**, **cloud-check**, the run titled `cloud-check google`) and reads the key's shape (only its length and missing parts are ever printed, never a character of it), the login, the machine type and every quota a set needs. When it passes, the sets start on their own on the engine that stands (branch `main`, the working branch kept level with it), and no result goes up before it has been read. When it fails, it says which of these to fix, and nothing is built or billed.

## G.6 Google's limits (quotas)

A set needs, at once, 5 machines of 2 processors (10 processors), their 5 boot disks of 100 GB (500 GB of solid-state disk) and 6 external addresses in `us-central1`. The check reads each of Google's limits and names any that is short:

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **CPUS_ALL_REGIONS 8**: the free trial (8 processors at once). Activate the full account: at the top of the console click **Activate** (or ☰, **Billing**, **Activate full account**) and confirm. The $300 credit stays and pays first.
- **Any other limit short** (`CPUS`, `SSD_TOTAL_GB`, `IN_USE_ADDRESSES`): ☰, **IAM & Admin**, **Quotas & System Limits**; search the name the check printed, tick it (region `us-central1` where it asks), **Edit**, ask for what the check said is needed or more, **Submit**. Google usually answers within minutes to a day. The set is never shrunk to fit.

## G.7 What it costs

Google's list prices (us-central1, on demand): a worker of the Kubernetes test (`e2-standard-2`) about $0.067 an hour, up to 4 workers an arm, about $5 to $15 for a full five-repetition set; the big-organism machine (`e2-standard-8`) about $0.27 an hour, about $10 for its 39-hour ceiling. All of it comes out of the $300 credit first. Every cluster and machine is deleted when its run ends, whatever happens.

## G.8 Afterwards

To remove GitHub's access: in the console menu ☰ **IAM & Admin**, **Service Accounts**, tick **omni-compass-github**, **Delete**; and delete the two secrets in GitHub.

APPENDIX

# Appendix H. The Amazon Web Services Account, Click by Click

**Status (24 October): not yet run, its queue switched off.** The account is on Amazon's Free plan, which Amazon will not raise before the $100 credit is spent or the account is six months old. Everything here stays ready, wired as Azure's sets run (the same harness and 60 s beat, 100 paired repetitions a set under the stopping rule, the $0 check first); the founder switches it on once the account is on the Paid plan (section H.2), and nothing is built or billed before.

This connects GitHub to an Amazon Web Services (AWS) account so two workflows can run the same tests Azure runs:

- `eks-metered`: a real Amazon Elastic Kubernetes Service cluster, native Kubernetes (with its own cluster autoscaler) against the same with Omni-Compass on top, the machines Amazon bills counted every 15 seconds, the organisms with the cluster inside at 1, 10 and 100 copies (`organism`, `organism_scale`), everything deleted at the end; every set with the whole engine at the max on top of

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

the cloud's own autoscaler, the bill judged beside the work (31 October). The same design as Azure's `aks-metered` (`docs/K8S_COMPASS_PREREGISTRATION.md`).

- `cloud-organism` with `cloud` = `aws`: the big organisms (the whole tower and the four stacked, the real cluster inside, at 100 copies) on one rented 8-processor machine, the same test Azure's `big-organism-detached` runs. With `test` = `direct` the same button runs Omni direct's three arms instead (native, Omni on native and Omni direct, no Kubernetes; `docs/OMNI_V18.md`), `reps` runs on the one machine; it starts only on the founder's word.

About fifteen minutes, once. New accounts get **$100 of credit at sign-up and up to $100 more** for trying services, used before the card.

## H.1 An AWS account

1. On the laptop, open **aws.amazon.com/free** and click **Create a Free Account** (or **Create an AWS account**).
2. Enter an e-mail address and an account name (`Omni-Compass`), click **Verify email address**, and type the code Amazon e-mails you.
3. Choose a password for the root user.
4. Contact information: **Business** (The Omni-Compass LLC) or **Personal**, the address and phone.
5. **Choose the Paid plan** if Amazon asks you to pick a plan. Your credits still pay first; the Free plan blocks services these tests need (the Kubernetes service among them).
6. Enter a card, then confirm the phone number with the code Amazon sends.
7. Support plan: **Basic support - Free**. Click **Complete sign up**, then **Go to the AWS Management Console** and sign in as the root user.

## H.2 An account already on the Free plan: move it to the Paid plan

An account that chose the Free plan cannot run these tests: Amazon puts a policy above every login of the account that refuses every machine read and every machine (on 18 October each Amazon set stopped there, at no cost, with "explicit deny in a service control policy"). Moving to the Paid plan removes it; the credits stay and still pay first.

1. Sign in to the console as the root user (the e-mail and password of step 1).
2. At the top of the console click **Upgrade plan** (beside the Free plan's credit and days left), or search **Billing and Cost Management**, open it, and click **Upgrade plan** on its home page.
3. Read the page, and click **Upgrade to paid plan** (or **Upgrade plan**) to confirm.

Nothing else changes: the login made for GitHub (step 2) and the two secrets (step 3) stay as they are. The conductor's check reads the account again (start it by hand to have it read at once: **Actions**, **cloud-check**, **Run workflow**, `amazon`) and the sets start on their own.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## H.3 A login for GitHub (an IAM user)

1. In the search bar at the top of the console type **IAM** and open it.
2. On the left, **Users**, then **Create user**. User name: `omni-compass-github`. Do **not** tick console access. **Next**.
3. **Attach policies directly**. Tick **AdministratorAccess** (the tests build whole Kubernetes clusters, which needs EKS, EC2, CloudFormation and roles; this user is used only by GitHub, and you delete it when the tests are done). **Next**, **Create user**.
4. Click the new user **omni-compass-github**, open the **Security credentials** tab, and under **Access keys** click **Create access key**.
5. Choose **Application running outside AWS** (or **Other**), **Next**, **Create access key**.
6. The page shows an **Access key** (starts with `AKIA`) and a **Secret access key**. Keep the page open for the next step; the secret is shown only once. These two are passwords for this one purpose: never paste them anywhere but GitHub.

## H.4 Put the two secrets in GitHub

1. Open the repository on GitHub, click **Settings** (the repository's tab, at the top right), then on the left **Secrets and variables**, then **Actions**.
2. **New repository secret**. Name: `AWS_ACCESS_KEY_ID`. Secret: the Access key. **Add secret**.
3. **New repository secret**. Name: `AWS_SECRET_ACCESS_KEY`. Secret: the Secret access key. **Add secret**.

That is all. To have it read at once, start it by hand: **Actions**, **cloud-check**, **Run workflow**, choose `amazon`, **Run workflow**. Otherwise the conductor runs it on its own: after a failed check it waits an hour, and the wait doubles with every failure in a row (two, four, eight, sixteen hours) up to a day, so a key not yet stored is read once a day, not every hour. The check costs nothing (**Actions**, **cloud-check**, the run titled `cloud-check amazon`) and reads whether the keys log in, the account can read and rent machines (not the Free plan), the instance type is offered and the processor quota holds a set. When it passes, the sets start on their own on the engine that stands (branch `main`, the working branch kept level with it), and no result goes up before it has been read. When it fails, it says which of these to fix, and nothing is built or billed.

## H.5 What it costs

Amazon's list prices (us-east-1, on demand): a worker of the Kubernetes test (`m5.large`) about $0.096 an hour, up to 4 workers an arm, plus about $0.10 an hour for each cluster's control plane, about $10 to $25 for a full five-repetition set; the big-organism machine (`m6i.2xlarge`) about $0.38 an hour, about $15 for its 39-hour ceiling. The credits pay first. Every cluster and machine is deleted when its run ends, whatever happens.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

A set needs 10 processors at once (5 machines of 2). New accounts can start with less. When the check finds the quota short it asks Amazon for the increase itself (once; it waits for Amazon's answer before asking again), and Amazon usually answers within a day. To ask by hand: in the console search **Service Quotas**, open **Amazon Elastic Compute Cloud (Amazon EC2)**, **Running On-Demand Standard (A, C, D, H, I, M, R, T, Z) instances**, **Request increase at account level**, ask for 64, **Request**. The set is never shrunk to fit.

## H.6 Afterwards

To remove GitHub's access: in IAM, **Users**, tick **omni-compass-github**, **Delete**; and delete the two secrets in GitHub.

APPENDIX

# Appendix I. The Install Manifests

Every file an installation applies or builds, printed in full from the repository: the watch install (namespace, service account, the read-only role and the watch Deployment), the cloud's grant and its governing Deployment, the pod-target grant of level 2 (in a cloud it is part of the cloud's grant), and the two container images. Each file carries the legal notice at its head; the `<...>` values are the installation's own.

## I.1 `deploy/install/omni-compass.yaml`

*60 lines.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 Dubra Software Inc. / The Omni-Compass LLC. All r
    ↪ ights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# Omni-Compass: one-file install. Stage 1 (shadow): read-only identity
    ↪ , observe mode. It cannot write to the cluster.
# Image: build with `docker build -f deploy/Dockerfile -t <registry>/o
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ mni-compass:<tag> .` and set it below.
# Reset, any time: kubectl -n omni-compass exec deploy/omni-compass --
    ↪  touch /tmp/omni.kill
# Promote to stage 2/3 only by the steps in docs/WIRING_GUIDE.md (each
    ↪  adds exactly the permissions that stage needs).
apiVersion: v1

kind: Namespace
metadata: {name: omni-compass}
---
apiVersion: v1
kind: ServiceAccount
metadata: {name: omni-compass, namespace: omni-compass}

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: {name: omni-compass-observe}
rules:
  - apiGroups: [""]

    resources: [nodes, pods, configmaps]
    verbs: [get, list]
  - apiGroups: [metrics.k8s.io]
    resources: [nodes, pods]
    verbs: [get, list]
  - apiGroups: [autoscaling]

    resources: [horizontalpodautoscalers]
    verbs: [get, list]
  - apiGroups: [apps]
    resources: [deployments]
    verbs: [get, list]
---

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata: {name: omni-compass-observe}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-observe}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ compass}]
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: omni-compass, namespace: omni-compass, labels: {app:
    ↪ omni-compass}}
spec:
  replicas: 1
  selector: {matchLabels: {app: omni-compass}}
  template:
    metadata: {labels: {app: omni-compass}}
    spec:
      serviceAccountName: omni-compass
      securityContext: {runAsNonRoot: true, runAsUser: 10001, seccompP
    ↪ rofile: {type: RuntimeDefault}}
      containers:
        - name: controller
          image: omni-compass:local            # <- your registry imag
    ↪ e
          args: ["--mode", "observe", "--closure", "/app/law/closure.j
    ↪ son",
                 "--audit", "/var/log/omni/audit.jsonl", "--kill-file"
    ↪ , "/tmp/omni.kill"]
          resources: {requests: {cpu: 50m, memory: 128Mi}, limits: {cp
    ↪ u: 500m, memory: 256Mi}}
          securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
          volumeMounts: [{name: audit, mountPath: /var/log/omni}, {nam
    ↪ e: tmp, mountPath: /tmp}]
      volumes: [{name: audit, emptyDir: {}}, {name: tmp, emptyDir: {}}
    ↪ ]
```

## I.2 `deploy/install/rbac-cloud.yaml`

*54 lines.*

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 Dubra Software Inc. / The Omni-Compass LLC. All r
    ↪ ights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# The cloud's grant (the Operator's Manual, section 14.3): the watch r
    ↪ ole of omni-compass.yaml plus the writes the whole engine
# makes on a managed cluster at the max, the same writes the kind rig
    ↪ grants (deploy/kind/rbac-omni.yaml; the founder's order of
# 31 October: no hard rule; the machines-alone grant of 14 October is
    ↪ struck). Cluster-wide: park a machine (a PreferNoSchedule
# taint) and wake it, patch nodes; mark a parked machine's pods first
    ↪ to go (the pod-deletion-cost annotation), patch pods. In
# the service's own namespace, by name: move its autoscaler's target,
    ↪ its floor and its cap inside the replica room and record
# the operator's own on the object, get, patch and update its Horizont
    ↪ alPodAutoscaler; record its pods' original processor
# limit on its Deployment, get, patch and update the Deployment; give
    ↪ its running pods the processor their machine can spare,
# in place, and hand it back, get and patch pods/resize. Nothing else:
    ↪  no delete, no eviction, no create, no cloud credential
# (the cloud's own autoscaler, with the provider's identity, alone add
    ↪ s machines). Set <namespace> and <deployment> (the
# service, as --sensed and --cap-deployments name it in omni-compass-c
    ↪ loud.yaml) before applying; one Role and RoleBinding
# per governed service.
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: {name: omni-compass-cloud-machines}
rules:
  - apiGroups: [""]
    resources: [nodes]
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    verbs: [patch]
  - apiGroups: [""]
    resources: [pods]
    verbs: [patch]
---
apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRoleBinding
metadata: {name: omni-compass-cloud-machines}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-cloud-machines}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: omni-compass-cloud-service, namespace: "<namespace>"}
rules:
  - apiGroups: [autoscaling]

    resources: [horizontalpodautoscalers]
    resourceNames: ["<deployment>"]
    verbs: [get, patch, update]
  - apiGroups: [apps]
    resources: [deployments]
    resourceNames: ["<deployment>"]

    verbs: [get, patch, update]
  - apiGroups: [""]
    resources: [pods/resize]
    verbs: [get, patch]
---
apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding
metadata: {name: omni-compass-cloud-service, namespace: "<namespace>"}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: Role, name: omni-
    ↪ compass-cloud-service}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## I.3 deploy/install/omni-compass-cloud.yaml

*73 lines.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 Dubra Software Inc. / The Omni-Compass LLC. All r
    ↪ ights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# The cloud (the Operator's Manual, chapter 19): Omni-Compass on top o
    ↪ f a managed cluster at the max, the whole engine, every
# muscle (the founder's order of 31 October: no hard rule; the tune-do
    ↪ wn on the machines alone of 14 October is struck). It
# senses the service and moves its pod target inside the autoscaler's
    ↪ cover, conveys the processor a machine can spare, and
# parks machines the cloud's own autoscaler made, after its own trial,
    ↪  waking them on a breach; it asks the cloud for no
# machine itself (the autoscaler, native's machine controller, stays a
    ↪ nd adds them). The bill is judged beside the work,
# never ruled. Apply after omni-compass.yaml (the namespace, the servi
    ↪ ce account and the watch role) and rbac-cloud.yaml
# (the writes the whole engine makes). It ships with --dry-run: it dec
    ↪ ides and writes nothing until that line is removed.
# Set the image (<registry>/omni-compass-cloud:<version>), --sensed an
    ↪ d --cap-deployments (the service's
# namespace/deployment: its autoscaler's target moved, its pods' proce
    ↪ ssor conveyed), TARGET_URL (the service's URL),
# --slo-ms (the service's line) and --max-nodes (the work pools' top).
    ↪  Three containers in one pod: the governor, the probe
# (its response-time feed) and the watchdog (its lease), sharing one p
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ rocess namespace.
apiVersion: apps/v1
kind: Deployment
metadata: {name: omni-compass, namespace: omni-compass, labels: {app:
    ↪ omni-compass}}
spec:
  replicas: 1                          # one writer: never more than o
    ↪ ne
  strategy: {type: Recreate}           # never two governors at once,
    ↪ even during a rollout
  selector: {matchLabels: {app: omni-compass}}
  template:
    metadata: {labels: {app: omni-compass}}
    spec:
      serviceAccountName: omni-compass
      shareProcessNamespace: true          # the watchdog sees the gov
    ↪ ernor's process (the lease, chapter 20.2)
      terminationGracePeriodSeconds: 60    # room for the hand-back on
    ↪  SIGTERM: every parked machine woken, read back
      securityContext: {runAsNonRoot: true, runAsUser: 10001, seccompP
    ↪ rofile: {type: RuntimeDefault}}
      tolerations: [{key: CriticalAddonsOnly, operator: Exists}]     #
    ↪  runs on the system pool, never on a machine it parks
      nodeSelector: {omni-role: system}
      containers:
        - name: controller
          image: omni-compass-cloud:local
          args: ["--mode", "nodepool", "--law", "compass", "--dial", "
    ↪ max", "--sensed", "<namespace>/<deployment>",
                 "--cap-deployments", "<namespace>/<deployment>",
                 "--active-nodes-only", "--floor-interval", "5",
                 "--min-nodes", "2", "--max-nodes", "4", "--max-node-s
    ↪ tep", "1",
                 "--node-scale-cmd", "bash /app/scripts/kind_nodepool.
    ↪ sh {n}",
                 "--node-restore-cmd", "bash /app/scripts/kind_nodepoo
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ l.sh 100000",
                "--latency-file", "/feed/latency.csv", "--slo-ms", "5
    ↪ 00",
                "--closure", "/app/law/closure.json",
                "--audit", "/var/log/omni/audit.jsonl", "--kill-file"
    ↪ , "/tmp/omni.kill",

                "--dry-run"]
          env:
            - {name: WORKER_SEL, value: "omni-role=work"}      # the m
    ↪ achines it may park
            - {name: MIN_NODES, value: "2"}                    # never
    ↪  fewer in service

          resources: {requests: {cpu: 50m, memory: 128Mi}, limits: {cp
    ↪ u: 500m, memory: 256Mi}}
          securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
          volumeMounts: [{name: audit, mountPath: /var/log/omni}, {nam
    ↪ e: tmp, mountPath: /tmp}, {name: feed, mountPath: /feed}]

        - name: probe                  # the response-time feed (chapt
    ↪ er 18): real requests to the service, every 5 s
          image: omni-compass-cloud:local
          command: ["python", "/app/scripts/latency_probe.py"]
          args: ["$(TARGET_URL)", "/feed/latency.csv"]
          env:

            - {name: TARGET_URL, value: "http://<service>.<namespace>.
    ↪ svc.cluster.local/"}
            - {name: INTERVAL, value: "5"}
            - {name: N, value: "5"}
            - {name: DURATION, value: "31536000"}
          resources: {requests: {cpu: 10m, memory: 32Mi}, limits: {cpu

    ↪ : 100m, memory: 64Mi}}
          securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
          volumeMounts: [{name: feed, mountPath: /feed}]
        - name: watchdog               # the lease (chapter 20.2): if
    ↪ the governor dies or hangs, it runs the governor's own
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        image: omni-compass-cloud:local    # recorded restore (--res
    ↪ tore-only), which wakes every parked machine
        command: ["python", "/app/tools/omni_switch.py", "watchdog",
    ↪  "--every", "10"]
        env:                          # the restore is the governor's
    ↪  own command line: it needs the governor's environment
          - {name: WORKER_SEL, value: "omni-role=work"}
          - {name: MIN_NODES, value: "2"}
        resources: {requests: {cpu: 10m, memory: 64Mi}, limits: {cpu
    ↪ : 500m, memory: 256Mi}}
        securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
        volumeMounts: [{name: audit, mountPath: /var/log/omni}, {nam
    ↪ e: tmp, mountPath: /tmp}]
      volumes: [{name: audit, emptyDir: {}}, {name: tmp, emptyDir: {}}
    ↪ , {name: feed, emptyDir: {}}]
```

## I.4 deploy/rbac-target.yaml

*19 lines.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 Dubra Software Inc. / The Omni-Compass LLC. All r
    ↪ ights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# Additional write access for --mode target: patch and annotate HPAs o
    ↪ nly. Apply only after an observe period.
apiVersion: rbac.authorization.k8s.io/v1
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
kind: ClusterRole
metadata: {name: omni-compass-target}
rules:
  - apiGroups: [autoscaling]
    resources: [horizontalpodautoscalers]
    verbs: [patch, update]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata: {name: omni-compass-target}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-target}

subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]
```

## I.5 deploy/Dockerfile

*19 lines.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 Dubra Software Inc. / The Omni-Compass LLC. All r
    ↪ ights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES

    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# Omni-Compass controller image. kubectl comes from the official Kuber
    ↪ netes image (no package manager, no curl).
FROM registry.k8s.io/kubectl:v1.32.2 AS kubectl

FROM python:3.12-slim
COPY --from=kubectl /bin/kubectl /usr/local/bin/kubectl
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY omnicompass omnicompass
```

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
COPY omni_controller omni_controller
COPY hardware hardware
# the frozen, benchmarked closure-law setting (--closure /app/law/clos
    ↪ ure.json)
COPY tuning/GLOBAL_LEAGUE_PREREGISTRATION.json law/closure.json
USER 10001

ENTRYPOINT ["python", "-m", "omni_controller.controller"]
```

## I.6 deploy/Dockerfile.cloud

*17 lines.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 Dubra Software Inc. / The Omni-Compass LLC. All r
    ↪ ights reserved.
# Evaluation and Simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES

    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# The cloud mode's image (the Operator's Manual, chapter 19): the ship
    ↪ ped image (deploy/Dockerfile) plus what moving the
# machines alone needs: jq (the machine lever reads the cluster's JSON

    ↪  with it), the machine lever's script, the
# response-time probe or the Prometheus feed adapter, and the switch t
    ↪ ool the in-pod watchdog runs (the lease). Build the shipped imag
    ↪ e first and pass it: --build-arg BASE=<registry>/omni-compass:<v
    ↪ ersion>.
ARG BASE=omni-compass:local

FROM ${BASE}
USER root
RUN apt-get update && apt-get install -y --no-install-recommends jq &&
    ↪  rm -rf /var/lib/apt/lists/*
COPY scripts/kind_nodepool.sh /app/scripts/kind_nodepool.sh
COPY scripts/latency_probe.py /app/scripts/latency_probe.py
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
COPY tools/omni_switch.py /app/tools/omni_switch.py
COPY tools/prom_feed.py /app/tools/prom_feed.py
USER 10001
```

APPENDIX

# Appendix J. The Implementer's Checklists

One checklist a phase (section 14.3). Each line is checked and initialled before the next phase begins, and the sheets are kept with the installation record (section 15.7).

## J.1 Phase 0: the copy

- [ ] `python3 verify.py` ends `VERIFICATION: PASS`; the receipt kept.
- [ ] `python3 tools/omni_version.py` names the engine version; the commit recorded.
- [ ] `python3 tools/seal.py --check` passes.
- [ ] `LICENSE`, `NOTICE` and `DISCLOSURES.md` read by counsel; the license in force for this installation named.

## J.2 Phase 1: the account and the cluster

- [ ] The provider's allowances read for the region and the machine families used; increases granted.
- [ ] A system pool of one machine, tainted `CriticalAddonsOnly=true:NoSchedule`, labelled `omni-role=system`.
- [ ] One or more work pools labelled `omni-role=work`, under the provider's cluster autoscaler, floor at least 2.
- [ ] The autoscaler's profile written down (scale-down timing, scan interval); the same in both arms of every paired run.
- [ ] The autoscaler seen to add a machine and delete one on its own.
- [ ] metrics-server answering (`kubectl top nodes`).
- [ ] The service's HPA on CPU, with container requests; its target, minimum and maximum written down by the service owner.

## J.3 Phase 2: identity

- [ ] Namespace `omni-compass` and service account `omni-compass` created.

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

- [ ] The watch role applied (`deploy/install/omni-compass.yaml`).
- [ ] In a cloud, the cloud's grant applied (`deploy/install/rbac-cloud.yaml`, its namespace and Deployment set) and nothing else.
- [ ] `kubectl auth can-i` output of section 15.4 matches the table exactly; kept.
- [ ] No cloud credential given to the governor.

## J.4 Phase 3: watch

- [ ] The shipped image built and pushed, tagged with the engine version and commit.
- [ ] The governor running `--mode observe`; its audit shows `decision` records (`nodes_recommended`) and no `write` record.
- [ ] The audit on a persistent volume, shipped to the log store.
- [ ] At least one full cycle of the service's own load watched (a day; a week for a weekly peak).

## J.5 Phase 4: the feed and the line

- [ ] The line written down by the service owner: milliseconds, the percentile, what counts as a failure, where measured.
- [ ] The feed in place (the probe or an adapter), writing `elapsed_seconds,latency_ms,ok`.
- [ ] The feed advancing every 5 seconds or faster; its clock monotonic over a day.
- [ ] A stopped endpoint seen in the audit as blind or breached, every machine held.
- [ ] A stopped feed seen in the audit as blind, nothing given back.

## J.6 Phase 5: the paired run

- [ ] The run's design written before it starts: arms (native, and native with Omni-Compass on top), load, duration, repetitions, the line, the machine family, the autoscaler profile.
- [ ] The run made on one engine version; its commit recorded.
- [ ] The receipt read row by row: every row GAIN, STEADY or LOSS by the rules of chapter 12.
- [ ] Every LOSS explained as wiring and fixed, and the run repeated; none cut.
- [ ] In a cloud: every gauge read against native by the one key; the work per dollar and the bill read side by side.
- [ ] The receipt signed by the service owner and by finance (the bill rows).

## J.7 Phase 6: govern

- [ ] The cloud image built (`deploy/Dockerfile.cloud`) and pushed.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- [ ] `deploy/install/omni-compass-cloud.yaml` applied with the image, the service's URL, the line and the pools' top.
- [ ] A day of `--dry-run`: every `write` record marked `"dry_run": true` read; each command asks for a count of machines in service between the floor and the work machines that exist, one machine from the last.
- [ ] `--dry-run` removed; the first parked machine read in the audit with the trial that allowed it.
- [ ] The watchdog running in the governor's pod (the manifest's third container): the pod reads `3/3` ready.
- [ ] The OFF switch drilled by its holder: every machine woken, every record gone, within 60 seconds; the drill recorded.
- [ ] The alarms of section 21.5 wired and each proven by a test.

## J.8 Phase 7: day two, every quarter

- [ ] The OFF switch drilled and recorded.
- [ ] The watchdog's hand-back drilled (the governor's process killed outright in a quiet hour, section 29.2) and recorded.
- [ ] The audit searched for `refused`, `error`, `blind` and an unclosed `proprioception` drift; each explained.
- [ ] The engine version checked against the receipts kept; no receipt read across versions.
- [ ] The installation record complete.

APPENDIX

# Appendix K. The Kubernetes Governor's Settings

Every setting of the Kubernetes governor (`python -m omni_controller.controller`), grouped by who writes it. A setting not given on the command line takes its default. Settings in the last group are the factory tune (section 4.1): they are listed so that a command line can be read in full, and an installation leaves them as shipped; a command line that changes one runs a changed tune, whose receipts are its own (section 4.6).

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## K.1 The settings an installation writes

| Setting | Default | The cloud's value | What it is |
|---|---|---|---|
| `--slo-ms` | 0 (no line) | the service's line | the 95th-percentile response time the service must meet, in milliseconds (section 19.1) |
| `--latency-file` | none | `/feed/latency.csv` | the feed: rows `elapsed_seconds,latency_ms,ok` (section 19.2) |
| `--min-nodes` (and `MIN_NODES` in the environment of the governor and the watchdog) | 2 | the operator's floor, at least 2 | machines always kept in service |
| `--max-nodes` | 0 (read live) | the work pools' top | the most machines the machine lever counts on; it never asks for more than exist |
| `--node-scale-cmd` | none | `bash /app/scripts/kind_nodepool.sh {n}` | the command that keeps `{n}` machines in service (section 26.2 for its contract) |
| `--node-restore-cmd` | none | `bash /app/scripts/kind_nodepool.sh 100000` | the command a reset runs: every parked machine woken |
| `WORKER_SEL` (environment) | every machine that is not the control plane | `omni-role=work` | the label selector of the machines that may be parked |
| `NODE_CUSHION` (environment) | 0 | 0 | whole machines the operator holds back from the lever |
| `--sensed` | every HPA | the service, `<namespace>/<deployment>` | the services whose autoscalers the feed speaks for, and only those are moved; `none/none` names none |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Default | The cloud's value | What it is |
|---|---|---|---|
| `--dial` | `max` | `max` | the max, the factory tune in a cloud as on site, or the tune-down (section 4.2) |
| `--replica-ceiling` | -1 (read live) | -1 (read live) | how far a sensed autoscaler's cap may be raised while it binds and the line is threatened (what the machines can hold, one pod under), handed back once the demand holds still; 0 keeps the operator's cap |
| `--audit` | `omni_audit.jsonl` | `/var/log/omni/audit.jsonl` | the audit file (section 21.4) |
| `--kill-file` | `/tmp/omni.kill` | `/tmp/omni.kill` | the file whose presence turns the governor OFF (section 21.3) |
| `--dry-run` | off | on for the first day, then removed | decide and record every command, write none |
| `--native` (and `OMNI_NATIVE`) | none | none, or the muscles the site keeps on its own controller | the muscles left where native runs them, never written: `machines`, `replicas`, `cpu`, `clock`, `gpu`, `rollout`, `batch`, `rightsize`, `coldstart`, `contain`, `cooling`; a name that is not one of them stops the start (omni-v14; the control panel's muscle button sets it) |
| `--own-replicas` (and `OMNI_OWN_REPLICAS`) | none | absent (no cloud runs by itself without the owner's word) | omni-v18: the workloads (`namespace/name`, comma-separated) whose pods Omni-Compass runs by itself, with no autoscaler on them: their target from |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Default | The cloud's value | What it is |
|---|---|---|---|
| | | | the workload's `omnicompass.io/target-utilization` annotation (else `--own-target`), their range from `omnicompass.io/replica-range` (`min,max`); a workload an autoscaler governs keeps its autoscaler and nothing is written; the count the workload shipped with is recorded on it and handed back by the reset |
| `--own-every-s` | 15 | 15 | seconds between two reconciliations of those workloads' pods on the fast checks, the pod autoscaler's own sync |
| `--kubectl` | `kubectl` | `kubectl` | the client the governor reads and writes through |
| `--site-limit-w`, `--power-cmd` | none | absent | a site's power limit and the command that prints its watts (level 7; a rented machine has no meter the tenant reads); the command runs without a shell unless it holds a pipe, a redirect or a variable |
| `--cooling-cmd`, `--cooling-min-c`, `--cooling-max-c`, `--cooling-restore-c` | none; 18, 27, 22 °C | absent | the facility's cooling setpoint command and band (section 8.10) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## K.2 The settings a cloud fixes

| Setting | Default | The cloud's value | What it is |
|---|---|---|---|
| `--mode` | `observe` | `nodepool` | `observe` writes nothing; `target` moves autoscaler targets; `nodepool` also moves machines |
| `--law` | `governor` | `compass` | the law Omni-Compass holds the service with |
| `--interval` | unset: the factory beat for its place | 10 s on site, 60 s through a cloud | the full decision period, read from the cluster's own nodes at the start (a dial from 5 to 120 s; section 4.4) |
| `--floor-interval` | 5 s | 5 s | the fast checks' period, between decisions, in `nodepool` mode |
| `--max-node-step` | 2 | 1 | the most machines one decision moves |
| `--active-nodes-only` | off | on | count only machines in service |
| `--closure` | none | `/app/law/closure.json` | the shipped setting file the image carries |
| `--iterations` | 0 (run until stopped) | 0 | decisions before exiting; the paired-run harness sets it |
| `--restore-only` | off | used only by the watchdog | put every setting back from the cluster's own records and exit |
| `--no-api-proxy` | off | off | read through a new client process every time instead of one local API proxy (costs more processor) |

## K.3 The switches of the other muscles

These turn on muscles of levels 2 to 7 (section 7.3). In a cloud the factory tune turns on the conveyance (`--cap-deployments`, section 20.3); the others are wired where a site's own stack carries their muscles.

| Setting | What it turns on |
|---|---|
| `--cap-deployments ns/name[,...]` | conveyance: the idle processor of each machine to the serving pods on it |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | What it turns on |
|---|---|
| `--rightsize-deployments ns/name[,...]` | each pod's processor request follows its measured use |
| `--coldstart-deployments ns/name[,...]`, `--coldstart-signal ns/configmap` | a service scaled to zero while no work waits and woken when work arrives |
| `--batch` | held Jobs labelled `omnicompass.io/batch=true` admitted when there is room |
| `--batch-pace` | Jobs labelled `omnicompass.io/pausable=true` paused under power or heat stress |
| `--rollout-guard ns/name[,...]` | a rollout paused while a change is not permitted |
| `--contain-namespaces ns[,...]`, `--contain-cpu-m` | an agent namespace held to a processor budget |
| `--security-configmap ns/name` | a security hold read from a ConfigMap key `hold` |
| `--strict-replicas` | level 4: the governor sets replica counts, the HPA kept as the fast reflex upward |
| `--pod-reflex-writes` | the fast pod reflex may raise an HPA's replica floor itself |
| `--rapl-cmd`, `--cpufreq-cmd`, `--cpufreq-policy-root`, `--cpufreq-require-schedutil`, `--cpu-max-khz` | the processor's own frequency governor under its power meter (section 8.9) |
| `--gpu-query-cmd`, `--gpu-power-cmd`, `--gpu-max-w`, `--gpu-temp-limit` | a graphics card's power reading and limit through commands (chapter 22 uses the card governor instead) |
| `--thermal-model` | the heat muscle's state from the harness's heat model (for modelled runs only) |
| `--give-back-only` | the cloud's struck rule in one flag (14 to 31 October): every switch above off whatever else is passed, the replica ceiling 0, the dial at the tune-down, the node pool the only knob; nothing passes it since 31 October, and it leaves with the next engine version |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

## K.4 The factory tune

`--allow`, `--verdict-samples`, `--verdict-every-s`, `--verdict-recheck-s`, `--verdict-trial-s`, `--compass-tau`, `--compass-center`, `--cruise-after-s`, `--idle-demand`, `--coast-step`, `--min-target-change`, `--headroom`, `--reflex-window-s`, `--strict-window-s`, `--latency-window-s`, `--slo-clear-s`, `--convey-on`, `--convey-off`, `--cap-min`, `--cap-min-change-m`, `--cap-headroom`, `--batch-load-max`, `--batch-power-max`, `--pace-high`, `--pace-low`, `--pace-heat`, `--rightsize-headroom`, `--rightsize-min-m`, `--coldstart-idle-s`, `--convey-every-s`.

Each has its shipped value as its default, and the receipts of an installation are read on those values. Two of them bear on planning and are stated here for that reason only. `--headroom` (half again, by default) is the spare the governor keeps above the processor requests of the pods when it counts the machines they need: it never parks a machine that would leave the pods' requests, plus that spare, without a place (section 25.2). `--allow` (2%) is the most a machine given back on trial may add to the service's response time before the trial takes it back (section 20.1).

Every setting that names a time is stated in seconds (the `-s` settings) and keeps its time at any decision period (`--interval`). The older count forms, `--verdict-every`, `--verdict-recheck`, `--cruise-after`, `--strict-window`, `--slo-clear` and `--coldstart-idle`, are still read, as a number of decisions, and win where they are given. Whatever the period, the node pool gives back at most one machine a minute and adds past the scheduling floor at most one step a minute, the conveyance runs every `--convey-every-s`, and a decision that falls behind its slot is skipped and written in the audit (`behind_s`), never run in a burst: since omni-v16 the period changes only how often the governor looks.

## K.5 The card governors' settings

Two governors run on a graphics card (chapter 22), and neither takes a setting from K.1 to K.4. The **two-wire card governor** (`python3 -m omni_controller.gpu_compass`) reads the card's draw, clocks, utilization and temperature through `nvidia-smi` and writes the clock ceiling and the power limit under the compass law with the verdict; the **one-wire power governor** (`python3 -m omni_controller.gpu_governor`) writes the power limit alone, with the speed lock. Both are factory-tuned: an installation writes the first group and leaves the rest as shipped. The card governors' counts (`--verdict-every`, `--verdict-recheck`, `--slo-clear`) are in decisions at their own 2-second period (8 decisions is 16 s; 900 is 30 minutes).

| Setting (both governors) | Default | What it is |
|---|---|---|
| `--mode` | `watch` | `watch` decides and records, writes nothing; `cap` writes |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting (both governors) | Default | What it is |
|---|---|---|
| `--gpus` | `0` | the card indexes governed, comma separated |
| `--smi` | `nvidia-smi` (or `NVIDIA_SMI` in the environment) | the client the governor reads and writes through |
| `--interval` | 2.0 s | the decision period on a card |
| `--duration` | 0 (until killed) | seconds to run |
| `--audit` | `gpu_compass_audit.jsonl`; `gpu_audit.jsonl` | the audit file (Appendix L.5) |
| `--kill-file` | `/tmp/omni-gpu-kill` | the file whose presence turns the governor OFF (section 22.6) |
| `--latency-file`, `--slo-ms` | none; 0 (no line) | the response-time feed and the line (section 22.2) |
| `--latency-window-s` | 5 s (two-wire); 30 s (one-wire) | seconds of response times the position reads |
| `--floor-w` | 0 (off) | the declared envelope's lowest watts: the power limit never under it |

| Setting (the two-wire governor only) | Default | What it is |
|---|---|---|
| `--dial` | `max` (or `OMNI_DIAL`) | the max: the clock ceiling never stepped below the top while the card serves; `save`: the tune-down (section 4.2) |
| `--race-util` | 0.95 | the utilization at which work is waiting: race, never pace |
| `--learn-samples` | 15 | busy readings of the card on its own before the compass may lower anything |
| `--clock-min-share` | 0 (the card's own lowest supported clock) | the clock ceiling's cover, as a share of the top |
| `--allow` | 0.02 | the most a ceiling step may add to the card's own time on a request |
| `--verdict-samples`, `--verdict-every`, `--verdict-recheck` | 30; 8; 900 | requests measured at native and at the trial step; decisions between trials; decisions before a refused step is tried again |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting (the two-wire governor only) | Default | What it is |
|---|---|---|
| `--up-gain`, `--down-gain` | 0.10; the law's | share of the top clock moved per unit of force, up and down |
| `--lid-headroom` | 0.10 | the lid above the draw the power limit takes |
| `--min-change-mhz`, `--min-change-w` | 15 MHz; 3 W | the smallest changes written |

| Setting (the one-wire governor only) | Default | What it is |
|---|---|---|
| `--headroom` | 0.3 | the limit never below the draw × (1 + headroom) |
| `--min-share` | 0.70 | the limit never below this share of the start limit (0: off) |
| `--util-gate`, `--util-band` | 0.5; 0.1 | the smoothed utilization at which the start limit returns, and the band below it at which the cap resumes |
| `--baseline-file`, `--speed-gain` | none (the lock off); 0.01 | the speed lock: the baseline from `tools/gpu_baseline.py`, and its line (at least this much faster than the baseline) |
| `--lock-boost`, `--lock-margin`, `--lock-step`, `--lock-hold-s`, `--lock-window-s`, `--lock-floor` | 5; 0.08; 0.02; 10 s; 60 s; 0.5 | the lock's largest step down (a multiple of `--lock-step`); the margin under the line within which watts are spent; the share of the start limit given up a decision; the seconds between two steps down; the response-time window the lock reads; the share of the start limit the lock never holds the limit under |
| `--min-change-w` | 5 W | the smallest change written |
| `--temp-limit` | 83 °C | the temperature read as heat 1.0 (section 22.5) |
| `--slo-clear` | 3 | decisions inside the line before a breach is counted clear |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix L. The Audit, Record by Record

The audit (`audit.jsonl`, section 21.4) is one JSON object a line. Every object carries `time` (seconds since the Unix epoch, from the governor's clock) and one key that names its kind. The records an implementer reads in a cloud are below, in the order a decision writes them; the fields not listed are the engine's own state, recorded for the owner's analysis and not needed to operate the governor.

## L.1 Every decision

| Record (its key) | When | Fields an implementer reads |
|---|---|---|
| `verdict` | when a give-back trial begins, ends or is abandoned | `verdict`: `trial`, `step allowed`, `step refused: the muscle is slower there`, `trial abandoned (service not calm)` or `trial abandoned (too little work)`; `step` (machines given back on trial); `machines_given_back_allowed` (how many may stay given back); `machines_native` (the machines the cluster held when the governor started); on `step allowed` and `step refused`, `reading` (`GAIN`, `STEADY`, `LOSS` or `past the allowance`, the trial's reading against the reference) and `interval_ln` (the 95% intervals behind it, in log units) |
| `cruise` | when work has waited for a place for several decisions, and when it stops | `cruise`: `on` or `off`; `pending` (pods waiting); while cruise is on, every machine is in service and the governor stands back |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Record (its key) | When | Fields an implementer reads |
|---|---|---|
| `authority` | every decision | `senses.blind` (each sense, `true` when it is blind), `senses.latency_age_s` (seconds since the feed's newest row), `proprioception` (for the machines, the drift between the count last commanded and the count now in service, as a share of the command: 0 means the command landed), `node_gate.ok` and `node_gate.reason` (whether the release gate would let a machine go, and why not) |
| `decision` | every decision | `nodes_observed` (machines in service), `nodes_recommended` (machines the governor wants in service), `pending` (pods waiting for a place), `latency_p95_ms` (the feed's 95th percentile over its window), `slo_clean` (the line met over the last decisions), `mode`; wherever a service is sensed, `hpa_target_recommended` |
| `write` | each command the governor runs | `write` (the command, as its argument list), `why` (`node pool size` from a decision, `scheduling floor (pending pods)` from a fast check, `reset: restore node pool` from a reset), `dry_run` |
| `refused` | each command that exits non-zero | `refused.nodes` (the count asked), `refused.now` (the count in service), `refused.code` (the exit code), `why`, `pool_top` |
| `decision_ms` | after every decision | the decision's own duration in milliseconds |

## L.2 Other records

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Record | When | Fields |
|---|---|---|
| `error` | a decision or a fast check fails (the API unreachable, a read refused) | `error` (the exception), `stderr` (the client's last output), `consecutive_failures` |
| `decision` with a text value | `killed` (the kill file is present: every setting handed back, nothing written), `restore only: every setting handed back` (the watchdog's restore), `master switch: OFF; every setting handed back; exiting` (`SIGTERM` or the master switch) | the text itself |
| `overhead` | when the governor's process exits | `cpu_s` (processor seconds of the governor and every command it ran), `wall_s`, `cores_mean` (their ratio: the governor's own cost in cores) |
| `idle_floor` | when the service is idle and the pool goes to its floor | `from`, `to`, `demand`, `at_idle` |
| `compass` | every decision | the engine's own reading, for the owner's analysis |

## L.3 Six searches that answer most questions

```
A=/var/log/omni/audit.jsonl
jq -c 'select(.write) | {time, why, cmd: (.write | join(" ")), dry_run
    ↪ }' $A              # every command, with its reason
jq -c 'select(.verdict) | {time, verdict, step, machines_given_back_al
    ↪ lowed}' $A          # every trial and its outcome
jq -c 'select(.authority) | select([.authority.senses.blind[]?] | any)
    ↪  | {time, senses: .authority.senses}' $A   # blind
jq -c 'select(.authority.proprioception.nodes? > 0) | {time, drift: .a
    ↪ uthority.proprioception}' $A          # unlanded
jq -c 'select(.error or .refused)' $A
    ↪                     # failures and refusals
jq -c 'select(.convey or .quota) | {time, convey, quota, machines, hos
    ↪ t_busy, p95_ms}' $A   # the processor lever and the quota
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

A drift reading on one decision is normal: the command was given at the end of the last decision, and the machine script takes seconds. A drift that stays above zero across several decisions is the one-writer question of section 21.8.

## L.4 The records of the other muscles

The muscles of levels 2 to 7 (section 7.3) write their own records, in a cloud as on site where they are wired. Each is one JSON object with `time`, as in L.1; the fields an implementer reads:

| Record | When | Fields |
|---|---|---|
| `quota` | once a conveyance pass, in a namespace whose ResourceQuotas the governor's role cannot read (section 6.5) | `quota`: `no budget known: the raise proceeds, Kubernetes admission is the authority`; `namespace`; `reason` (`quota read unavailable: <the error's class>`) |
| `convey` = `held` | a conveyance pass on a shared host at or over 75% busy, or unread, that would otherwise have raised a limit | `machines` (the nodes held), `host_busy` (the machine's busy share; `null` when unread), `shared_host` |
| `convey` = `engaged` or `released` | conveyance turning on (the 95th percentile at or over `--convey-on` × the line, or the feed blind) or off (under `--convey-off` × the line) | `p95_ms` (`null` while blind), `slo_ms`, `blind` |
| `write` with `why` beginning `convey:` | each pod CPU limit written: `convey: record original CPU limit` (the operator's limit annotated once, before the first raise); `convey: <node> idle CPU to its <n> serving pod(s), limit <m>m in place`; `convey: response time calm, operator's limit <m>m in place` | as `write` in L.1 |
| `error` beginning `convey resize rejected:` | Kubernetes refused a resize (the quota, admission, or the pod gone) | the error's class and its first 400 characters; a refused limit is never counted as given |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Record | When | Fields |
|---|---|---|
| `pod_reflex_reading` | each fast check that reads a sensed autoscaler's floor | the deployment and the floor it needs; `replicas` (now); `busy` (the host's busy share) |
| `pod_reflex` | a fast check that raises, or would raise, a floor (`--pod-reflex-writes`) | each deployment's new floor; `busy`; `R_ms` (the response time read); `S_ms` (the floor the line allows) |
| `cpu_pstate_snapshot` | once, at the start, on a machine whose processor ceilings the governor holds (section 8.9) | every frequency policy's ceiling as found |
| `cpu_pstate_write` | each ceiling written | the ceiling written; `why` (`cpu_pstate: ceiling <c> (cap <cap>, envelope [<lo>, <hi>], …)`) |
| `cpu_pstate_restore` | at reset or OFF | the ceilings restored from the snapshot; `why`: `reset: exact pre-Omni CPU ceilings` |
| `error` beginning with a muscle's name | a muscle's read or write failed | the error's class and its text; the decision goes on, the muscle holds |

## L.5 The card governors' records

The card governors (chapter 22; Appendix K.5) keep their own audit files, one JSON object a line with `time`, and are read the same way. The records, in the order a run writes them:

| Record | When | Fields an implementer reads |
|---|---|---|
| `snapshot` | once, at the start, before any write | per card: `limit_w` (the power limit read at start, the limit every reset returns to), `min_limit_w`, `enforced_w` (the limit the card obeys, where the driver reports it), the clock top, the power-management state; `mode`; `engine` (`compass, two wires`); the two-wire governor also writes `law` (its profile), `verdict` (the allowance, the samples, the spacing, the recheck) and `covers` (the clock range and the |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Record | When | Fields an implementer reads |
|---|---|---|
| | | power range nothing it computes can leave) |
| `refused` | at the start, in `cap` mode, when the card's power management is not enabled | the text: a written limit would not bind, so no authority is taken and the governor exits |
| `decision` | every decision | per card, the reading and the step: utilization, draw, the position in the band, the ceiling and the lid chosen, the verdict's state and the deepest step allowed; `hold: last write not read back` while a write has not landed |
| `would_write`, `would_clock_write` | in `watch` mode, or after another writer appeared, each write that would have been made | the `nvidia-smi` command as its argument list; `why`; `withheld` when another writer owns the limit |
| `write`, `clock_write` | each power limit, clock ceiling (`-lgc floor,ceiling`) or clock reset (`-rgc`) written in `cap` mode | the command; `why` (the step's reason, `restore`, or `reset: the limit read at start`); for a clock write `rc` and `stderr` |
| `actuator` | after each power-limit write | `gpu`, `requested_w`, `t_requested`, `rc`, `stderr`; on success `readback_w`, `enforced_w`, `t_readback`, `realized` (the read-back equals the request), `override` (the card enforces less than the limit), `delay_s`; on failure `write_failed` (the command) and the run ends with `fatal` |
| `actuator_realized` | a later decision, when a write that had not read back at once has landed | `gpu`, `requested_w`, `readback_w`, `enforced_w`, `delay_s` |
| `foreign_writer` | the limit read differs from what the governor wrote and from what it found: another writer | `gpu`, `reads_w`, `expected_w` (`pending_w` where a write was in flight); `action`: `observe only from now on` and both wires left to the other writer, for the rest of the run |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Record | When | Fields an implementer reads |
|---|---|---|
| `verdict` | when a ceiling trial begins, ends or is abandoned (two-wire governor) | as in L.1; `deepest_step` (the deepest ceiling step allowed) |
| `restored` | at the end, after the kill switch, OFF or the duration | per card, the limit read back; `start` (the limit read at the start); `ok` (every card back at its start, or another writer owned the limit); `writes` (how many the governor made); `foreign_writer`; `restore_write_failed` (any restore write the card refused) |
| `fatal` | a write the card refused | `write failed: <the command and its error>`; the arm ends here and never carries on as if watching |
| `error` | a decision that failed for any other reason | the error's class and text; the next decision is taken on schedule |

A card run is read from `snapshot` to `restored`: `ok` true in `restored`, no `foreign_writer`, no `fatal`, is a run in which every setting was handed back; `foreign_writer` true is a run in which the governor stood back from the first foreign reading, and the card's settings from then on are the other writer's.

APPENDIX

# Appendix M. The Three Clouds Side by Side

The same installation on each provider, task by task. The commands are those of chapters 16 to 18 and 27, condensed; the chapters give the context and the choices. Every Kubernetes command (`kubectl ...`) is the same on all three.

## M.1 The cluster

| Task | Azure (AKS) | Google Cloud (GKE) | Amazon Web Services (EKS) |
|---|---|---|---|
| Check the allowance for the machine | `az vm list-usage -l <region> -o` | `gcloud compute regions describe` | Service Quotas console, EC2 "Running |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Azure (AKS) | Google Cloud (GKE) | Amazon Web Services (EKS) |
|---|---|---|---|
| family | `table` | `<region>` (quotas) | On-Demand Standard instances" (vCPUs) |
| Create the cluster with its system pool | `az aks create ... --nodepool-name system --nodepool-taints CriticalAddonsOnly=true:NoSchedule --nodepool-labels omni-role=system` | `gcloud container clusters create ... --node-taints CriticalAddonsOnly=true:NoSchedule --node-labels omni-role=system` | `eksctl create cluster -f cluster.yaml` with a `system` managed node group carrying the label and the taint |
| Add a work pool under the autoscaler, labelled | `az aks nodepool add ... --labels omni-role=work --enable-cluster-autoscaler --min-count <n> --max-count <top>` | `gcloud container node-pools create work0 ... --enable-autoscaling --min-nodes <n> --max-nodes <top> --node-labels omni-role=work` | a `work0` managed node group with `labels: {omni-role: work}` and the autoscaler's discovery tags, and the Cluster Autoscaler installed by Helm (or Karpenter with a NodePool carrying the label) |
| The autoscaler's scale-down profile | `az aks update --cluster-autoscaler-profile scale-down-unneeded-time=<t> ...` | `gcloud container clusters update --autoscaling-profile balanced` or `optimize-utilization` | the Cluster Autoscaler's flags (`--scale-down-unneeded-time` and others), or Karpenter's `consolidationPolicy` and `consolidateAfter` |
| Credentials for `kubectl` | `az aks get-credentials -g <rg> -n <cluster>` | `gcloud container clusters get-credentials <cluster> --zone <zone>` | `aws eks update-kubeconfig --name <cluster> --region <region>` (eksctl writes it) |
| metrics-server | built in | built in | installed from its release manifest, placed on the system pool |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Azure (AKS) | Google Cloud (GKE) | Amazon Web Services (EKS) |
|---|---|---|---|
| Delete everything | `az group delete -n <rg> --yes` | `gcloud container clusters delete <cluster> --zone <zone>` | `eksctl delete cluster -f cluster.yaml` |

## M.2 The image and the registry

| Task | Azure | Google Cloud | Amazon |
|---|---|---|---|
| The registry | `az acr create -g <rg> -n <acr> --sku Standard`, then `az aks update --attach-acr <acr>` | `gcloud artifacts repositories create omni-compass --repository-format=docker --location=<region>` | `aws ecr create-repository --repository-name omni-compass` (and `omni-compass-cloud`) |
| Build the shipped and the cloud image | `az acr build -t omni-compass:<v> -f deploy/Dockerfile .` and `az acr build -t omni-compass-cloud:<v> -f deploy/Dockerfile.cloud --build-arg BASE=<acr>.azurecr.io/omni-compass:<v> .` | `docker build` and `docker push` of both, the second with `--build-arg BASE=<repo>/omni-compass:<v>` | as Google, with `aws ecr get-login-password ... \| docker login` first |
| Tag immutability | `az acr repository update --write-enabled false` per tag, or the registry's retention and lock settings | immutable tags on the repository | `aws ecr put-image-tag-mutability --image-tag-mutability IMMUTABLE` |
| Signature admission (section 27.3) | Image Integrity (Ratify with Azure Policy) | Binary Authorization | Kyverno `verifyImages` or the Sigstore policy controller |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

## M.3 Identity, network and records

| Task | Azure | Google Cloud | Amazon |
|---|---|---|---|
| The governor's cloud credential | none | none | none |
| The cluster's own credential for making machines | the cluster's managed identity (Azure's) | GKE's own service agent (Google's) | the Cluster Autoscaler's IAM role for service accounts, or Karpenter's role (the operator's, created by eksctl) |
| Network policy enforcement | Azure Network Policy Manager, Calico or Cilium (`--network-policy` at creation) | GKE Dataplane V2 or Calico | the VPC CNI's network policy agent, or Calico |
| A persistent volume for the audit | storage class `managed-csi` | storage class `standard-rwo` | storage class `gp3` through the EBS CSI driver |
| The API server's audit log | diagnostic setting, category `kube-audit-admin`, to Log Analytics | Cloud Audit Logs, Admin Activity, in Cloud Logging | control plane logging, type `audit`, to CloudWatch Logs |
| The governor's audit, shipped | the Azure Monitor agent (Container insights) | Cloud Logging's agent (on by default) | the CloudWatch agent or Fluent Bit |

## M.4 The bill

| Task | Azure | Google Cloud | Amazon |
|---|---|---|---|
| Where machine cost lives | Cost Management, cost analysis or export | the Cloud Billing export to BigQuery | the Cost and Usage Report, Cost Explorer |
| Separating the governed pools | the cluster's node resource group, and a tag on each work pool | GKE cost allocation by namespace and node-pool label | a cost allocation tag on each node group's instances, activated in Billing |
| The paired run from GitHub (Appendices F to H) | the `aks-metered` workflow | the `gke-metered` workflow | the `eks-metered` workflow |

Product names, billing granularity and console paths are the providers' own and change; the provider's current documentation is the authority for each, and an installation's plan (chapter 25) records the terms it used.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix N. The Paired Run from GitHub: Every Input

Phase 5's paired run can be made from the repository's own workflows, on GitHub's runners, against the company's cloud account (Appendices F to H set up the account and the repository secrets). Each workflow builds a fresh cluster for every arm of every repetition, runs native and native with Omni-Compass on top in rotation on the same load, prices the machines that existed every 15 seconds, deletes every cluster it made whatever happens, and writes the receipt to the run's summary and as an artifact. The inputs below are the workflows' own; a run's title names its load and its dial.

## N.1 The inputs every cloud shares

| Input | Default | What it is |
|---|---|---|
| `reps` | 5 | paired repetitions; one runs at a time |
| `arms` | `native compass` | the two arms, in rotation order: native, and native with Omni-Compass on top |
| `duration_s` | 900 | measured seconds per arm |
| `price_per_node_hour` | the family's list price on the day the workflow was written | the price the bill rows use; check the provider's price list on the day |
| `load_steps` | empty (a steady load) | load-generator replicas per step, in order, for a burst or a day-shaped load (for example `1 6 1 8 1 6`) |
| `max_workers` | 4 | the work pool's top; the cloud's autoscaler runs it from 1 to this many |
| `hpa_max` | empty (the manifest's 10) | the service's replica ceiling, the same in both arms; size it with the fleet (nine 200-millicore pods fit a two-processor machine) |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Input | Default | What it is |
|---|---|---|
| `organism`, `load_max`, `organism_scale` | empty, 6, 1 | one of the repository's seven organisms (the spine among them) with this cluster inside it, its peak load, and its copies on one clock (1, 10 or 100); empty runs the service alone |
| `dial` | `save` | the dial of Omni-Compass on top: `save`, the tune-down, for every bill result; `max` is never read as a bill result (section 20.2) |

Every cloud workflow runs the whole engine at the max (`dial max`, its default since 31 October; chapter 20).

## N.2 The inputs of each cloud

| Workflow | Input | Default | What it is |
|---|---|---|---|
| `aks-metered` | `location` | `eastus` | the Azure region |
| | `vm_size` | `Standard_D2s_v4` | the work machines' size |
| | `tier` | `free` | the clusters' control-plane tier, `free` or `standard` (outside the bill; the way past free-tier capacity refusals) |
| | `worker_pools`, `pool_prices` | empty | several work pools, one machine family each with its own top (`size:max,...`), and each pool's list price (`work0=<price>;work1=<price>;...`): the way past a per-family allowance |
| | `replica_ceiling` | empty | how far the governor may raise the service's replica cap while it binds and the line is threatened; empty reads it live (what the machines can hold, one pod under), as on kind; 0 keeps the operator's cap |

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Workflow | Input | Default | What it is |
|---|---|---|---|
| gke-metered | project | the GCP_PROJECT secret | the Google Cloud project |
| | zone | us-central1-a | the zone |
| | machine | e2-standard-2 | the work machines' type |
| eks-metered | region | us-east-1 | the AWS region |
| | instance | m5.large | the work machines' instance type |

The repository's secrets each workflow reads are AZURE_CREDENTIALS (Azure), GCP_CREDENTIALS and GCP_PROJECT (Google Cloud), and AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY (Amazon). They are set once in the repository's settings, never printed, and used only to build and delete the run's own clusters; the governor inside the clusters never sees them (section 15.5).

Before a Google Cloud or Amazon set starts, cloud-check runs for that cloud at no cost and stops before anything is built: the stored key's shape (for Google, its length and its missing parts only, never a character of it), the login, the account's plan, the machine type in the zone or region, and every quota a set needs (Google: processors in the region and in all regions, solid-state disk and external addresses; Amazon: the on-demand processor quota, which the check asks Amazon to raise when it is short). The conductor starts a set only after that cloud's newest check passed, and starts a check itself when there is none, the last one failed an hour ago or more, or the last pass is a day old; so a broken key, an account that cannot rent or a short quota spends no set, and what the check prints names what to fix (Appendices G and H).

## N.3 A second Azure workflow, for machines already paid for

aks-metered-full runs the same paired design with the governor's full wiring on the service (the pod autoscaler's target, the pods' processor limits and the modelled power sense). Since 31 October aks-metered itself runs the same full wiring (chapter 20), so the two differ only in their inputs: aks-metered-full's are the shared ones above, with a burst of 1 4 1 6 1 9 and three repetitions of 600 seconds by default, and its receipt is read like any other cloud set's.

## N.4 Reading the run

The run's summary carries the receipt, row by row, read by the rules of chapter 12 and section 20.6: GAIN, STEADY or LOSS for each judged row, with its 95% interval, and the shown rows beside them. The artifact live-reps-aks (or the cloud's equivalent) holds the same table as Markdown and JSON, with the repository's notice, licence and disclosures attached. A receipt is read only against the engine version of the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

commit it ran on (`python3 tools/omni_version.py --commit <sha>`, chapter 13), which each repetition's job prints at its start.

APPENDIX

# Appendix O. The License

The license under which this book and the software are made available, as it stands in the file `LICENSE` at the root of the repository.

```
Omni-Compass Evaluation License, version 1.0 (revised 11 October 2026)
SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0

Copyright (c) 2026 Dubra Software Inc. / The Omni-Compass LLC. All rig
    ↪ hts reserved.
All patents, copyrights and trademarks filed in the USA. www.omni-comp
    ↪ ass.com

THIS IS NOT AN OPEN-SOURCE LICENSE. The Software is proprietary. It is
    ↪  published so that it can be evaluated, reproduced
and audited, and for no other purpose. It is licensed, not sold. Evalu
    ↪ ation and simulation are free of charge; every

other use, and in particular every commercialization or monetization o
    ↪ f any kind, requires a signed, paid Omni-Compass
Enterprise License.

ACCEPTANCE. By downloading, copying, forking, installing, running, sca
    ↪ nning or otherwise using any part of the Software,

You accept this license. If You do not accept it, You may not use the
    ↪ Software. If You use the Software on behalf of an
organization, You represent that You have authority to bind it, and "Y
    ↪ ou" includes that organization and its
Affiliates.
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
1. DEFINITIONS

   1.1 "Licensor" means Dubra Software Inc. / The Omni-Compass LLC.
   1.2 "Software" means everything Licensor publishes or makes availab
    ↪ le under this license, in any form: the source
       code and any object code; the Omni-Compass engine, the compass

    ↪ law, the governors, controllers, brains, shields,
       plugs and wiring; the harnesses, tools, scripts, workflows, con
    ↪ figurations, presets, catalogs, organisms and realms;
       the documentation, the manual, the book, the specifications, th
    ↪ e preregistrations and the receipts; the benchmarks,
       data, tables, reports and Results published by Licensor; and ev

    ↪ ery update, version and copy of any of them.
   1.3 "Mechanism" means the methods, processes, control laws, algorit
    ↪ hms, architectures and systems embodied in or
       described by the Software, including everything described in th
    ↪ e patent applications Licensor has filed.
   1.4 "Evaluation" means internal use by You, free of charge, solely

    ↪ to test, study and assess the Software in
       contemplation of an Enterprise License: reading it; running its
    ↪  tests, verifiers, benchmarks and simulations; and
       running it in observe, watch, dry-run or shadow mode, or in a t
    ↪ est environment, on systems You own or control,
       where it writes to no system that serves production traffic, re

    ↪ al users, customers, patients or passengers, and to
       no power system, vehicle, machine or physical process outside a
    ↪  test bench.
   1.5 "Simulation" means running the Software against modelled, recor
    ↪ ded or synthetic systems, including the organisms,
       realms, traces and simulators published with the Software.

   1.6 "Production Use" means any use that is not Evaluation or Simula
    ↪ tion, including any use in which the Software, the
       Mechanism or any output of either controls, governs, tunes, siz
    ↪ es, schedules or otherwise influences a system that
       serves production workloads or real users, or any physical proc
    ↪ ess outside a test bench.
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   1.7 "Commercial Use" means any use for, or in connection with, comm
    ↪ ercial advantage, revenue, cost reduction in
       operations, or monetary or other compensation, including consul
    ↪ ting, integration, managed, outsourcing, training,
       benchmarking-for-hire and advisory services, and any use by or
    ↪ for a for-profit operation beyond Evaluation.

   1.8 "Hosted Service" means making the Software, the Mechanism or an
    ↪ y of their functionality available to any third
       party over a network or as a service, including software-as-a-s
    ↪ ervice, platform-as-a-service, a managed or
       outsourced service, a cloud marketplace offering, or a feature
    ↪ of another product or service.

   1.9 "Derivative" means any modification, adaptation, translation, p
    ↪ ort, extension or re-implementation of the Software
       or of the Mechanism, and any work that incorporates, is based o
    ↪ n, or is built using the Software as a reference.
   1.10 "Results" means the measurements, tables, receipts, logs, repo
    ↪ rts and other outputs produced by running the

       Software.
   1.11 "Enterprise License" means a written agreement for the Softwar
    ↪ e signed by Licensor and a licensee, under which
       the licensee pays the fees it sets out; it may include order fo
    ↪ rms, riders and statements of work.
   1.12 "Marks" means the names "Omni-Compass" and "OmniCompass", the

    ↪ Omni-Compass compass rose with its ring of Greek
       letters, and every related name, logo and trade dress of Licens
    ↪ or.
   1.13 "Affiliate" means any entity that controls, is controlled by,
    ↪ or is under common control with You, where control
       means ownership of more than half of the voting interests or th

    ↪ e power to direct management.
   1.14 "You" means the individual or entity exercising rights under t
    ↪ his license, with its Affiliates where the
       Acceptance paragraph above includes them.

2. WHAT THIS LICENSE ALLOWS
```

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
   2.1 Copyright license. Subject to Your compliance with this license
    ↪ , Licensor grants You a personal, worldwide,
       royalty-free, non-exclusive, non-transferable, non-sublicensabl
    ↪ e and revocable license to download, copy, read,
       run and modify the Software solely for Evaluation and Simulatio

    ↪ n, and to reproduce and publish Results as section
       4 allows.
   2.2 Patent license. Subject to the same conditions, Licensor grants
    ↪  You a license of the same scope, under the patent
       rights Licensor holds or will hold in the filed applications an
    ↪ d any patent issuing from them, solely to the

       extent needed to carry out Evaluation and Simulation with the S
    ↪ oftware. No patent license is granted for any
       Production Use, Commercial Use or Hosted Service, for any Deriv
    ↪ ative used outside Evaluation or Simulation, or for
       any implementation of the Mechanism other than the Software its
    ↪ elf.

   2.3 Forks. You may keep a copy or a fork of the Software on a code-
    ↪ hosting service, publicly or privately, only for
       Evaluation and Simulation, with this license and every notice i
    ↪ ntact. A fork confers no right beyond this license.
   2.4 Reservation. Every right not expressly granted is reserved by L
    ↪ icensor. No right, title, license or interest is

       granted by implication, estoppel, exhaustion or otherwise.

3. WHAT THIS LICENSE DOES NOT ALLOW

   Unless an Enterprise License expressly permits it, You may not, and
    ↪  may not let or help anyone else:

   (a) make any Production Use or Commercial Use of the Software, the
    ↪ Mechanism or any Derivative;
   (b) commercialize or monetize the Software, the Mechanism, any Deri
    ↪ vative or any Result in any way, directly or
       indirectly, including by charging for access, services, support
    ↪ , results, savings or outcomes they produce;
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   (c) offer the Software, the Mechanism or any Derivative as, or as p
    ↪ art of, a Hosted Service;
   (d) sell, resell, license, sublicense, lease, rent, lend, distribut
    ↪ e, publish as a package or binary, or otherwise
       transfer the Software or any Derivative, except forks under sec
    ↪ tion 2.3;

   (e) incorporate the Software, any part of it or any Derivative into
    ↪  any product, service, platform, appliance, image,
       firmware or offering;
   (f) use the Software, the Mechanism, the documentation or the Resul
    ↪ ts to design, build, train, tune, benchmark for
       sale, or improve any product or service that competes with Omni

    ↪ -Compass, or to re-implement the Mechanism;
   (g) use the Software or the Results to train, fine-tune or evaluate
    ↪  any machine-learning model intended to reproduce,
       imitate or substitute for the Mechanism;
   (h) remove, alter or obscure any notice, the SPDX identifiers, the
    ↪ patent and trademark notices, this license, the

       NOTICE or DISCLOSURES.md, or circumvent, disable or work around
    ↪  any limit, switch or safeguard in the Software,
       including the kill switch and the hand-back to native settings;
   (i) connect the Software so that it writes to a safety-critical sys
    ↪ tem, as section 10 describes;
   (j) use the Marks, except to refer to Omni-Compass accurately as TR

    ↪ ADEMARKS.md allows; or
   (k) use the Software in violation of any law, or in breach of secti
    ↪ on 11.

4. REPRODUCING AND PUBLISHING RESULTS

   You may reproduce the benchmarks published with the Software and pu
    ↪ blish Your own Results, provided that You: (a)
   name the Software's engine version and the exact commit You ran; (b
    ↪ ) report every measured row, the unfavourable ones
   included, with the evidence class the Software assigns to it; (c) m
    ↪ ake Your complete raw files and configuration
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   available to Licensor on request; (d) do not state or imply any end
    ↪ orsement, certification or affiliation by
   Licensor; and (e) carry the notice at the top of README.md. Publish
    ↪ ing Results is not Commercial Use so long as no
   compensation is received for it, and it grants no right beyond this
    ↪  license.

5. THE OMNI-COMPASS ENTERPRISE LICENSE

   5.1 Every Production Use, Commercial Use, Hosted Service, distribut
    ↪ ion, incorporation, OEM or embedded use, and every
       other commercialization or monetization of the Software, the Me

    ↪ chanism or any Derivative requires an Enterprise
       License, signed before the use begins and paid as it provides.
    ↪ Moving from Evaluation to Production Use requires
       an Enterprise License before the first write to any production
    ↪ system.
   5.2 An Enterprise License sets its own scope, fees, term, support,

    ↪ warranties, indemnities and limits
       (ENTERPRISE_LICENSE.md summarizes the standard terms; the signe
    ↪ d agreement alone is binding). For its licensee and
       its scope, a signed Enterprise License governs where it differs
    ↪  from this license.
   5.3 Enterprise licensing: The Omni-Compass LLC, main office infrast

    ↪ ructure, toll-free direct 1-888-339-9551, system route asset 1-8
    ↪ 88-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.o
    ↪ mni-compass.com.

6. OWNERSHIP AND INTELLECTUAL PROPERTY

   6.1 Licensor and its licensors own the Software, the Mechanism, the
    ↪  Marks and all intellectual property rights in
       them, including in every Derivative to the extent it embodies t
    ↪ hem. Nothing in this license transfers ownership.
   6.2 All patents, copyrights and trademarks covering the Omni-Compas
    ↪ s engine, its mathematics and its software have
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
       been filed in the United States by Dubra Software Inc. / The Om
    ↪ ni-Compass LLC. No filing number is stated here, and no grant,
       registration or approval is claimed; the status of every filing
    ↪  is held by the office where it was filed. Every
       person who downloads, copies, forks, scans or runs the Software
    ↪  is on notice of these filings and of this

       license (PATENTS.md, TRADEMARKS.md).
   6.3 Any use outside this license, without an Enterprise License, is
    ↪  unlicensed and infringes Licensor's copyright and
       other rights, in addition to breaching this license.

7. PATENT AND RIGHTS DEFENSE

   If You or any of Your Affiliates bring or join any claim, countercl
    ↪ aim or proceeding alleging that the Software or
   the Mechanism infringes, misappropriates or violates any patent, co
    ↪ pyright or other intellectual property right, or
   challenge the validity or enforceability of any of Licensor's right

    ↪ s in them, every license granted to You under this
   license ends automatically on the date the claim or challenge is fi
    ↪ led.

8. FEEDBACK AND CONTRIBUTIONS

   8.1 If You give Licensor comments, suggestions, ideas, reports or o
    ↪ ther feedback about the Software, You grant
       Licensor a perpetual, irrevocable, worldwide, royalty-free righ
    ↪ t to use, disclose and exploit it for any purpose,
       without obligation of any kind.
   8.2 Contributions are accepted only under the contributor license a

    ↪ greement (.github/CLA.md).

9. CHANGES

   Licensor reserves the right to change the Software, its documentati
    ↪ on, its results and these terms at any time,
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   without notice. A changed version of these terms applies to every c
    ↪ opy of the Software obtained, and to every use
   made, after it is published in the Software's repository; continued
    ↪  use after publication is acceptance of the
   changed terms. Licensor may discontinue the Software, or this licen
    ↪ se for future copies, at any time.

10. SAFETY

   The Software is not designed, tested or certified for safety-critic
    ↪ al use. Under this license it may not write to
   any system whose failure could cause death, injury, environmental d

    ↪ amage or significant property damage, including
   vehicles and aircraft, medical devices, power-grid protection and d
    ↪ ispatch, industrial safety systems and life-safety
   systems; such systems may be studied only in Simulation. Any live c
    ↪ onnection to a physical system requires an
   Enterprise License, a written safety case accepted by the system's

    ↪ owner, and every review that system's regulator or
   owner requires. The machines' own controls and protections stay in
    ↪ place; the kill switch must remain operable at all
   times.

11. EXPORT CONTROL, SANCTIONS AND LAWFUL USE

   You will comply with every export-control, sanctions, anti-corrupti
    ↪ on and other law that applies to the Software and
   to Your use of it, including the U.S. Export Administration Regulat
    ↪ ions and the sanctions administered by the U.S.
   Treasury's Office of Foreign Assets Control. You represent that You

    ↪  are not located in, organized under the laws of,
   or ordinarily resident in a country or region subject to comprehens
    ↪ ive U.S. sanctions, and that You are not on, or
   owned or controlled by a party on, any U.S. restricted-party list.
    ↪ You will not use the Software for any end use
   prohibited by those laws, including the design, development or prod
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ uction of weapons of mass destruction.

12. DATA AND PRIVACY

   The Software contains no telemetry. It sends nothing to Licensor an
    ↪ d makes no network connection of its own except to

   the systems You point it at. Your data and Your Results stay on You
    ↪ r systems and are Yours, subject to section 4. If
   You send Licensor any material (for example in an issue or an e-mai
    ↪ l), You represent that You may lawfully do so and
   that it contains no personal data or confidential information of an
    ↪ y third party.

13. THIRD-PARTY COMPONENTS

   Files that carry a different license notice (for example tests/thir
    ↪ d_party/) and third-party data sets (for example
   the PlanetLab traces under fleet/traces/ and k8s_controlplane/trace

    ↪ s/) remain under their own terms
   (THIRD_PARTY_NOTICES.md). Third-party software the Software runs on
    ↪  or connects to (Kubernetes, KEDA, Karpenter,
   NVIDIA drivers, databases and others) is licensed by its owners und
    ↪ er their own terms.

14. TERM AND TERMINATION

   14.1 This license lasts until it ends under this section or section
    ↪  7.
   14.2 It ends automatically, without notice, if You breach it. If Yo
    ↪ ur breach is Your first, was inadvertent, and is

       not a breach of sections 3(a) to 3(g) or section 10, Your lice
    ↪ nse is reinstated if You cure the breach within
       thirty days after You learn of it or after Licensor notifies Y
    ↪ ou, whichever comes first.
   14.3 Licensor may end this license for any person at any time by wr
    ↪ itten notice, and for all persons for future
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        copies under section 9.
   14.4 When this license ends, You must stop all use of the Software,
    ↪  delete or destroy every copy and Derivative in Your
        possession or control, and, on request, certify in writing tha
    ↪ t You have done so.
   14.5 Sections 1, 3, 4(b) to 4(e), 6, 7, 8, 11, 12, 15 to 18 and 19
    ↪ survive the end of this license.

15. NO WARRANTY

   TO THE FULLEST EXTENT PERMITTED BY LAW, THE SOFTWARE, THE MECHANISM
    ↪  AND THE RESULTS ARE PROVIDED "AS IS" AND "AS
    ↪ AVAILABLE", WITH ALL FAULTS AND WITHOUT WARRANTY OR CONDITION OF AN
    ↪ Y KIND, EXPRESS, IMPLIED OR STATUTORY, INCLUDING ANY
   WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITL
    ↪ E, NON-INFRINGEMENT, ACCURACY, QUIET ENJOYMENT, OR
   THAT THE SOFTWARE WILL BE ERROR-FREE OR UNINTERRUPTED. NO RESULT IS
    ↪  A PROMISE OF ANY OUTCOME ON ANY OTHER SYSTEM;
    ↪ MODELLED RESULTS (EVIDENCE CLASS S) ARE STATEMENTS ABOUT THE MODEL,
    ↪  NOT MEASUREMENTS (DISCLOSURES.md). YOU USE THE
   SOFTWARE AT YOUR OWN RISK AND ARE RESPONSIBLE FOR ITS USE ON YOUR S
    ↪ YSTEMS.

16. LIMITATION OF LIABILITY

   TO THE FULLEST EXTENT PERMITTED BY LAW, LICENSOR, ITS MEMBERS, MANA
    ↪ GERS, EMPLOYEES, CONTRACTORS AND LICENSORS SHALL NOT
   BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXE
    ↪ MPLARY OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF
   PROFITS, REVENUE, SAVINGS, DATA, USE OR GOODWILL, OR ANY COST OF SU
    ↪ BSTITUTE GOODS OR SERVICES, ARISING OUT OF OR
   RELATING TO THE SOFTWARE, THE MECHANISM, THE RESULTS OR THIS LICENS
    ↪ E, UNDER ANY THEORY OF LIABILITY, EVEN IF ADVISED OF
   THEIR POSSIBILITY. LICENSOR'S TOTAL LIABILITY FOR ALL CLAIMS SHALL
    ↪ NOT EXCEED ONE HUNDRED U.S. DOLLARS (USD 100). THESE
   LIMITS APPLY EVEN IF A REMEDY FAILS OF ITS ESSENTIAL PURPOSE.
```

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
17. INDEMNITY

   You will defend, indemnify and hold harmless Licensor and its membe
    ↪ rs, managers, employees and contractors from and
   against every claim, loss, liability, damage, cost and expense (inc
    ↪ luding reasonable attorneys' fees) arising out of
   or relating to (a) Your use of the Software outside this license, (
    ↪ b) any Production Use, Commercial Use or Hosted
   Service without an Enterprise License, (c) Your Derivatives, (d) Yo
    ↪ ur Results as You publish them, or (e) Your
   violation of any law or third-party right.

18. U.S. GOVERNMENT END USERS

   The Software is "commercial computer software" and "commercial comp
    ↪ uter software documentation" as those terms are
   used in 48 C.F.R. 2.101, 12.212 and 227.7202. A U.S. Government end
    ↪  user acquires only the rights set out in this
   license, consistent with 48 C.F.R. 12.212 and 227.7202-1 through 22
    ↪ 7.7202-4.

19. GENERAL

   19.1 Governing law. This license is governed by the laws of the Sta
    ↪ te of Delaware and the federal laws of the United
        States, without regard to conflict-of-laws rules. The United N
    ↪ ations Convention on Contracts for the
        International Sale of Goods does not apply.
   19.2 Venue. The state courts of the State of Delaware and the Unite
    ↪ d States District Court for the District of
        Delaware have exclusive jurisdiction over any dispute arising
    ↪ out of or relating to this license, and You consent
        to their personal jurisdiction. Licensor may seek
        injunctive or other equitable relief in any court of competent
    ↪  jurisdiction, without posting a bond, to protect
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        its intellectual property or confidential information; a breac
    ↪ h of sections 3 or 10 would cause Licensor
        irreparable harm.
   19.3 Enforcement costs. In any action to enforce this license, the
    ↪ prevailing party may recover its reasonable
        attorneys' fees and costs.

   19.4 Assignment. You may not assign or transfer this license or any
    ↪  right under it, by operation of law or otherwise;
        any attempt is void. Licensor may assign this license freely.
   19.5 Notices. Notices to Licensor go to The Omni-Compass LLC by tel
    ↪ ephone at its main office infrastructure (toll-free direct 1-888
    ↪ -339-9551; system route asset 1-888-DEX-YLL-1, landline 321-746-

    ↪ 4411 registered in Florida) or through www.omni-compass.com. Lic
    ↪ ensor may give notice to
        You through the Software's repository or at any address You ha
    ↪ ve given it.
   19.6 Severability and waiver. If any provision is held unenforceabl
    ↪ e, it is enforced to the maximum extent permitted and

        the rest of this license remains in effect. No failure or dela
    ↪ y in enforcing any provision is a waiver of it.
   19.7 Entire license. This license, with the NOTICE and DISCLOSURES.
    ↪ md, is the entire agreement between You and
        Licensor about its subject; for a licensee and its scope, a si
    ↪ gned Enterprise License governs where it differs.

   19.8 Interpretation. "Including" means "including without limitatio
    ↪ n". Headings are for convenience only. The
        English text of this license controls over any translation.
   19.9 Copies. Every copy, fork, export, report, archive or printout
    ↪ of any part of the Software must carry this
        LICENSE, the NOTICE and DISCLOSURES.md unchanged.

All rights reserved. Revision history: version 1.0 (evaluation and sim
    ↪ ulation only; first published 2026); revised
10 October 2026 (the notice in one wording everywhere); revised 11 Oct
    ↪ ober 2026 (definitions, patent license for
evaluation, hosted-service and machine-learning restrictions, results
```

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ publication, patent and rights defense, safety,
export control, data, U.S. Government end users, term, cure, indemnity
    ↪ , Delaware governing law and venue).

Enterprise licensing: The Omni-Compass LLC, main office infrastructure
    ↪ , toll-free direct 1-888-339-9551, system route asset 1-888-DEX-
    ↪ YLL-1, landline 321-746-4411 registered in Florida, www.omni-com
    ↪ pass.com
```

APPENDIX

# Appendix P. Licensing: Questions and Answers

Plain answers to the questions buyers, engineers and their lawyers ask. The binding terms are in `LICENSE`; where this page and `LICENSE` differ, `LICENSE` governs. Every declaration is in `DISCLOSURES.md`.

**Is Omni-Compass open source?** No. The source is published so that it can be evaluated, reproduced and audited. It is licensed under the Omni-Compass Evaluation License 1.0 (`LicenseRef-OmniCompass-Evaluation-1.0`), which permits evaluation and simulation only.

**What may I do without a commercial license?** Evaluation and simulation, free of charge (`LICENSE`, sections 1.4, 1.5 and 2): read the code; run `verify.py`, the simulations and the benchmarks; reproduce the published results and publish your own, every row included (`LICENSE`, section 4); run Omni-Compass in watch, dry-run or shadow mode, or in a test environment, on systems you own or control, where it writes to nothing that serves production traffic or real users and to no physical process outside a test bench. A limited patent license comes with it, for evaluation only (`LICENSE`, section 2.2).

**What needs the Omni-Compass Enterprise License?** Every commercialization or monetization of any kind; production use; operating any system beyond evaluation; offering it, or its function, as a hosted, managed or software-as-a-service offering; redistributing it; incorporating it, or any part or derivative of it, into a product, service, appliance or firmware; using it or its results to build or train a competing product (`LICENSE`, sections 3 and 5). The Enterprise License is a written agreement signed by The Omni-Compass LLC and paid for; its standard terms are summarized in `ENTERPRISE_LICENSE.md` (production, hosted service, OEM and embedded, managed service provider, research).

**May I run it as a service for others?** Not without an Enterprise License with a hosted-service rider (`LICENSE`, section 3(c); `ENTERPRISE_LICENSE.md`, section 1).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**May I connect it to a grid, a plant, a vehicle or a medical device?** Not under the evaluation license: such systems may be studied only in simulation. A live connection to any physical system needs an Enterprise License and a written safety case accepted by the system's owner (`LICENSE`, section 10).

**How is the Enterprise License priced?** Against the measured gain on the customer's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock). Terms are set in each signed agreement; no price stated anywhere in this repository is an offer.

**Are patents involved?** All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by Dubra Software Inc. / The Omni-Compass LLC (`PATENTS.md`). The evaluation license grants no patent license beyond evaluation.

**May I use the name or the compass rose?** Only to refer to Omni-Compass accurately (`TRADEMARKS.md`).

**May I contribute?** Contributions require the contributor license agreement (`.github/CLA.md`): every contribution is assigned to The Omni-Compass LLC, with a patent license from the contributor.

**Does Omni-Compass send data anywhere?** No. The software contains no telemetry and makes no network connection of its own except to the systems an operator points it at (a Kubernetes API, `nvidia-smi`, a meter command).

**What do scanners report?** The license is declared in machine-readable form for Black Duck, FOSSA, Snyk and REUSE (`pyproject.toml`, `REUSE.toml`, `.fossa.yml`, `.snyk`, `sbom/`); third-party components are listed in `THIRD_PARTY_NOTICES.md`.

**Can the terms change?** Yes. The Omni-Compass LLC reserves the right to change the software, its documentation, its results and these terms at any time, without notice; a changed version applies to copies obtained and uses made after it is published (`LICENSE`, section 9). A signed Enterprise License governs its own term.

**Which law governs?** The laws of the State of Delaware and the federal laws of the United States, with exclusive venue in the Delaware state courts and the United States District Court for the District of Delaware (`LICENSE`, section 19). The Omni-Compass LLC is based in Florida.

**Who do I contact?** The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix Q. Third-Party Notices

Omni-Compass uses the following third-party components. Each remains under its own license; nothing in the Omni-Compass Evaluation License changes those terms. Installed versions are those resolved from `requirements.txt`.

| Component | Use | License |
|---|---|---|
| NumPy | numerical arrays | BSD-3-Clause |
| Matplotlib | charts in the reports | Matplotlib License (PSF-based, BSD-compatible) |
| ReportLab (open-source edition) | the printable book and reports | BSD-3-Clause |
| Numba | compiled inner loops of the motion-axis plant | BSD-2-Clause |
| PyTorch (on GPU machines only, not installed by `requirements.txt`) | the GPU workload of the benchmark | BSD-3-Clause |
| `tests/third_party/hpa_independent.py` | an independently written HPA reference used unmodified in tests | its own terms, stated in the file |
| Kokoro-82M v1.0 text-to-speech model and its voice `af_heart` (hexgrad/Kokoro-82M; the int8 ONNX files of thewh1teagle/kokoro-onnx), run by kokoro-onnx; FFmpeg | the recorded voice of the live engine's guided tour (`docs/engine/tour/`, made by `tools/tour_voice.py`): only the clips it read are kept, never the model | Apache-2.0 (the model and its voices); MIT (kokoro-onnx); FFmpeg's own LGPL/GPL, used as a tool and not shipped |
| Liberation Serif and Liberation Sans fonts (embedded in the PDF book) | typesetting | SIL Open Font License 1.1 |
| DejaVu Sans and DejaVu Sans Mono fonts (embedded in the PDF book) | typesetting | Bitstream Vera / DejaVu license (free) |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. All patents, copyrights and trademarks filed in the USA.*

| Component | Use | License |
|---|---|---|
| PlanetLab / CoMon VM CPU traces, March 2011 (`fleet/traces/`, `k8s_controlplane/traces/`; source in their `PROVENANCE.json`) | held-out demand for the modelled fleet | their own terms (public research data); no Omni-Compass copyright is claimed in them |
| Google cluster-usage traces 2011 (`clusterdata-2011-2`), from which `results/traces/google2011/` is derived (`docs/TRACES_PREREGISTRATION.md`) | the public demand trace on the real cluster | Google's published terms for the traces; no Omni-Compass copyright is claimed in the trace itself |
| Data-centre footage behind the live engine page (`docs/engine/wall/*.webm`, `*.mp4`, `*.jpg`), cut from *B-roll of Wikimedia Foundation servers at CyrusOne in Carrollton, Texas - photographed in 2015* by the Wikimedia Foundation, via Wikimedia Commons (`docs/engine/wall/README.md` lists every change) | the page's moving background, decoration only | CC BY-SA 3.0 (`LICENSES/CC-BY-SA-3.0.txt`); the adapted loops and stills are shared under the same licence |

Kubernetes, kind, `kubectl`, the NVIDIA driver and `nvidia-smi` are not distributed with Omni-Compass; it calls them where an operator has installed them. Their names are the property of their owners (`DISCLOSURES.md`, section 1).

APPENDIX

# Appendix R. The Enterprise License: Standard Terms

This page summarizes the standard terms of the Omni-Compass Enterprise License, the paid license that every production use, commercial use, hosted service, distribution, OEM or embedded use, and every other commercialization or monetization of Omni-Compass requires (`LICENSE`, section 5). It is a summary for

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

evaluation and planning. It is not an offer, and it binds no one: only an Enterprise License signed by The Omni-Compass LLC and the licensee, with its order form, grants any right, and that signed agreement controls over this page in every respect.

## R.1 What can be licensed

| License | What it allows |
|---|---|
| **Production** | The licensee runs Omni-Compass on top of its own systems (clusters, fleets, cards, sites, plants), on premises or in its own cloud accounts, within the scope in the order form |
| **Hosted service** | The licensee offers Omni-Compass's function to its own customers as a hosted, managed or software-as-a-service offering, under a hosted-service rider (per customer, per managed unit or revenue share) |
| **OEM and embedded** | The licensee ships Omni-Compass inside its own product, appliance, firmware or platform, under an OEM rider (per unit or per installed base) |
| **Managed service provider** | The licensee operates Omni-Compass for named third parties under a managed-service rider |
| **Research** | A university or public laboratory uses Omni-Compass beyond evaluation for non-commercial research and publication, under a research rider |

**Scope** is set in each order form: the licensed systems and their size (machines, cores, cards, sites, muscles or managed units), the fields of use, the territory, the named sites or accounts, and the engine versions covered.

## R.2 Fees

Fees are set in each order form: a subscription, a usage-based fee, a share of the measured value captured, or a combination, with any minimum commitment. Where fees track value, the value is measured on the licensee's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock), by the method the order form names. Payment terms, taxes and late charges are as the order form states. No price, rate or benchmark in this repository is an offer.

## R.3 What the licensee receives

1. The licensed engine at a named, sealed version (its fingerprint, `tools/omni_version.py`), and each later version the subscription covers.
2. The wiring kits and the plug for the licensed systems, the manual and the book, and the receipts tooling.
3. Support and maintenance at the tier in the order form: response targets, named contacts, and corrections of defects that reproduce on the licensed version.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

4. Installation through the manual's levels: read-only first, the wire check, a watch arm equal to native, then writing levels one at a time, each with its receipt.

## R.4 Warranties

1. **Conformance.** For the warranty period in the order form, the licensed version performs materially as its documentation describes. The remedy is correction or, failing that, a refund of the fees paid for the nonconforming version for the affected period.
2. **The safeguards.** The kill switch hands every setting back to the value read before Omni-Compass's first write; the shield keeps every write inside the cover the operator granted; Omni-Compass never fights another writer.
3. **No other warranty.** No outcome, saving or result on the licensee's systems is warranted. Published results are evidence about the systems they ran on (`DISCLOSURES.md`, section 2).

## R.5 Intellectual property

1. The Omni-Compass LLC keeps all right, title and interest in Omni-Compass, the Mechanism and the marks. The licensee receives the rights its order form lists, for its term, and no others.
2. **Patent license.** The Enterprise License includes a license under The Omni-Compass LLC's patent rights in the filed applications, and in any patent issuing from them, for the licensed scope.
3. **Infringement indemnity.** The Omni-Compass LLC defends the licensee against a third-party claim that the licensed version, used as licensed, infringes that party's intellectual property, and pays the damages finally awarded or the settlement it agrees, subject to prompt notice, control of the defense and the usual exclusions (modifications, combinations, unlicensed use, versions superseded by an offered fix). Remedies may include a license, a modification or a refund.
4. **Confidential filings.** The patent application number and other confidential information are disclosed to a licensee under the agreement's confidentiality terms.

## R.6 Limits of liability

Each party's liability is limited to the fees paid or payable in the twelve months before the claim, and neither party is liable for indirect, incidental, special, consequential or punitive damages, except for the exclusions the signed agreement states (for example, breach of confidentiality, the infringement indemnity, and the licensee's breach of the license scope).

## R.7 Confidentiality, data and security

1. **Confidentiality.** Mutual, for the term and a period after it, covering non-public information either party gives the other.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

2. **Data.** Omni-Compass contains no telemetry. The licensee's data, readings, settings and receipts stay on the licensee's systems and remain the licensee's. Where the licensee chooses to share data for support, a data-processing agreement applies.
3. **Security.** A software bill of materials ships with every version (`sbom/`); vulnerabilities are handled under `SECURITY.md`; the licensee may run the verifier (`python3 verify.py`) and the seal checks on every delivered version.

## R.8 Safety and operation

1. The licensee remains the operator of its systems. The machines' own controls and protections stay in place; Omni-Compass sets only values they already accept.
2. Any connection to a physical system (grid, plant, vehicle, robot, medical device, facility) requires a written safety case accepted by the system's owner and every review its regulator requires; safety-critical systems are licensed only on those terms.
3. The kill switch stays operable at all times.

## R.9 Compliance, audit and insurance

1. Each party complies with export-control, sanctions and anti-corruption law.
2. The licensee reports usage against the licensed scope as the order form provides, and The Omni-Compass LLC may verify it on reasonable notice, no more than once a year.
3. Insurance, where the order form requires it, is carried by each party at the limits stated there.

## R.10 Term, renewal and transition

1. Term and renewal are as the order form states.
2. On termination or expiry, the licensee stops use, every setting Omni-Compass changed is handed back to native (the hand-back is built into the engine), and every copy is returned or destroyed; transition assistance is available at the rates in the order form.
3. Source-code escrow for the licensed version is available under a three-party escrow agreement.

## R.11 Governing law and precedence

The signed agreement states its governing law and venue; the standard is the law of the State of Delaware, with venue in the Delaware state courts and the United States District Court for the District of Delaware. Its order of precedence is: the order form, then any rider, then the Enterprise License terms, then the documentation. This page and the evaluation license (`LICENSE`) are superseded by the signed agreement for the licensee and its scope.

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## R.12 How to obtain one

The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com. A license starts with an evaluation on the licensee's own systems (`LICENSE`, section 1.4): watch mode, the wire check, and paired receipts on its own load.

APPENDIX

# Appendix S. Patents and Trademarks

## S.1 Patents

**All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by Dubra Software Inc. / The Omni-Compass LLC.**

The filings cover, among other things, the Omni-Compass engine (the six-state closed law and its bounded control command held through the integration step), the compass law (the service position pulled to the middle of its band by a bounded push and pull, with fail-up at the wall), the antagonist up/down wiring of each muscle, the plug contract (one restore point, read-back, the one-writer rule), the two-wire governance of a processor (clock ceiling and power limit), the two-way nervous system that grants and withdraws authority per organ, and one governor over many machines and realms on one clock.

### *S.1.1 What has been filed*

| Application | a nonprovisional utility patent application under 35 U.S.C. 111(a), filed with the United States Patent and Trademark Office |
|---|---|
| Title of the invention | **The Omni-Compass** |
| Inventor | Alan John Dubra, Kissimmee, Florida, United States |
| Drawing sheets | 15 |
| Application data sheet signed | 20 September 2026 (form PTO/AIA/14, 37 CFR 1.76) |
| Publication | normal eighteen-month publication requested |
| Application number | withheld here on purpose; the filing is verifiable with the USPTO and is shown to a licensee under a signed Omni-Compass Enterprise License |

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

The application data sheet names no assignee; the application stands in the inventor's name. Any assignment to The Omni-Compass LLC is recorded with the USPTO separately (37 CFR part 3).

No patent license is granted by publishing this repository, by the evaluation license, or by any act of The Omni-Compass LLC other than a signed, paid Omni-Compass Enterprise License. Use outside the evaluation license is infringement. Everyone who downloads, forks, scans or runs this software is on notice of these filings.

## S.2 Trademarks

**All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by Dubra Software Inc. / The Omni-Compass LLC.**

### *S.2.1 The marks*

"Omni-Compass", "OmniCompass", the Omni-Compass compass rose with its ring of Greek letters, and the related names, logos and trade dress (the "Marks") are trademarks of The Omni-Compass LLC; trademark applications have been filed in the United States. Neither the evaluation license (`LICENSE`) nor publication of this repository grants any license to the Marks. A license to use them is granted only in a signed Omni-Compass Enterprise License, and only as it states.

### *S.2.2 What is allowed without a license*

- Referring to Omni-Compass by name, accurately and in plain text, to say what it is, that you evaluated it, or that a result was produced with it, with the engine version and commit (`LICENSE`, section 4).
- Linking to the repository or to www.omni-compass.com.

### *S.2.3 What is not allowed*

- Using any Mark, or anything confusingly similar, in the name of a product, service, company, domain, account, package, repository, fork or derivative.
- Using the logo or the compass rose, or altering any Mark.
- Suggesting endorsement, sponsorship, certification, partnership or affiliation by The Omni-Compass LLC.
- Using a Mark in advertising, comparative claims or sales material without a signed license.

### *S.2.4 Other companies' marks*

Kubernetes is a trademark of The Linux Foundation; NVIDIA is a trademark of NVIDIA Corporation; KEDA and Karpenter are projects of their maintainers; other names belong to their owners. Their mention describes compatibility or a system Omni-Compass was tested with; it states no affiliation, endorsement or certification (`DISCLOSURES.md`, section 1).

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *S.2.5 Reporting misuse*

The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida, www.omni-compass.com.

APPENDIX

# Appendix T. Security Policy

## T.1 Reporting a vulnerability

Report a suspected vulnerability privately to The Omni-Compass LLC by telephone at its main office infrastructure (toll-free direct 1-888-339-9551; system route asset 1-888-DEX-YLL-1, landline 321-746-4411 registered in Florida) or through www.omni-compass.com, never in a public issue, pull request or discussion. Include the version and commit, what you saw, how to reproduce it, and its likely impact. We acknowledge reports, keep the reporter informed, and coordinate disclosure: a fix or a mitigation is published before the details are, and the reporter is credited if they wish. Please give us a reasonable time to fix before any disclosure, do not access data that is not yours, and do not disrupt any system; good-faith research that follows this policy will not be pursued by us.

## T.2 What is in scope

The engine, the controllers, the plugs and wiring, the harnesses, the workflows and the verifier in this repository. Third-party software Omni-Compass runs on or connects to (Kubernetes, KEDA, Karpenter, NVIDIA drivers, databases) is reported to its own maintainers.

## T.3 How Omni-Compass is built to fail safe

- The governor issues control actions only through the shield (`omnicompass/shield.py`, `cpp/src/shield.cpp`), inside the cover the operator granted, and never fights another writer.
- It snapshots every setting once, before its first write, and the kill switch (`python3 tools/omni_switch.py off`) hands every setting back; a dead-man lease hands them back if the governor stops.
- It runs under least privilege: on Kubernetes, a service account with `kubectl auth can-i` receipts for what it can and cannot do (`deploy/kind/rbac-omni.yaml`).
- The watchdog acts only on registry records its own user wrote that no other user can change, and never signals a process number the system gave to another process (`tools/omni_switch.py`). Put the

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

switch and the registry in a directory only the governors' user can write (`OMNI_MASTER_OFF`, `OMNI_REGISTRY`); `status` warns when they are not.

- It contains no telemetry and makes no network connection of its own except to the systems an operator points it at.
- The threats of misuse, the guards against each and what is still open are set out in `docs/THREAT_MODEL.md`.
- Every release carries a software bill of materials (`sbom/`) and a seal of the engine and its twins (`results/SEAL.json`, `python3 verify.py`).
- The capture script (`fleet/capture/kube_capture.sh`) is read-only; no component writes to a live cluster unless the operator starts it in a writing mode.

BACK MATTER

# Index

Every term of the glossary where this book defines or discusses it, every muscle family where it is named, every file of the repository and every command-line setting this book names, with the pages they stand on. A plain mention of a common word is not an entry: the index points to where a thing is explained. A term whose pages would run past its line keeps the pages that fit, where it is defined and first used; after those it is throughout the book.

-

--active-nodes-only . . . . . 125, 570
--allow . . . . . 21, 44, 51, 572, 573
--audit . . . . . 125, 568, 573
--baseline-file . . . . . 574
--batch . . . . . 49, 571
--batch-load-max . . . . . 572
--batch-pace . . . . . 49, 571
--batch-power-max . . . . . 572
--bind . . . . . 143, 147
--build-arg . . . . . 126, 217, 583

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

--cap-deployments . . . . . 43, 49, 102, 124, 179, 570
--cap-headroom . . . . . 572
--cap-min . . . . . 572
--cap-min-change-m . . . . . 572
--cert . . . . . 147
--clock-min-share . . . . . 573
--closure . . . . . 125, 570
--coast-step . . . . . 572
--coldstart-deployments . . . . . 49, 571
--coldstart-idle . . . . . 572
--coldstart-idle-s . . . . . 42, 572
--coldstart-signal . . . . . 571
--compass-center . . . . . 20, 572
--compass-tau . . . . . 572
--contain-cpu-m . . . . . 571
--contain-namespaces . . . . . 49, 571
--convey-every-s . . . . . 42, 572
--convey-off . . . . . 572, 578
--convey-on . . . . . 572, 578
--cooling-cmd . . . . . 26, 51, 74, 569
--cooling-max-c . . . . . 21, 569
--cooling-min-c . . . . . 21, 569
--cooling-restore-c . . . . . 21, 51, 569
--cpu-max-khz . . . . . 571
--cpufreq-cmd . . . . . 571
--cpufreq-policy-root . . . . . 26, 51, 571
--cpufreq-require-schedutil . . . . . 571
--cruise-after . . . . . 572
--cruise-after-s . . . . . 44, 572

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

--dial . . . . . 19, 51, 124, 161, 179, 195, 568, 573
--down-gain . . . . . 574
--dry-run . . . . . 126, 146, 181, 193, 204, 209, 566, 568
--duration . . . . . 150, 573
--fail-cap . . . . . 149
--failed . . . . . 149
--floor-interval . . . . . 125, 179, 570
--floor-w . . . . . 573
--give-back-only . . . . . 44, 571
--gpu-max-w . . . . . 571
--gpu-power-cmd . . . . . 571
--gpu-query-cmd . . . . . 571
--gpu-temp-limit . . . . . 571
--gpus . . . . . 573
--headroom . . . . . 155, 572, 574
--idle-demand . . . . . 572
--interval . . . . . 21, 42, 44, 51, 125, 146, 148, 149, 179, 570, 572, 573
--iterations . . . . . 570
--key . . . . . 147
--kill-file . . . . . 125, 568, 573
--kubectl . . . . . 569
--latency-file . . . . . 49, 119, 121, 125, 150, 567, 573
--latency-window-s . . . . . 572, 573
--law . . . . . 49, 124, 179, 570
--learn-samples . . . . . 573
--lid-headroom . . . . . 574
--lock-boost . . . . . 574
--lock-floor . . . . . 574
--lock-hold-s . . . . . 574

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

--lock-margin . . . . . 574
--lock-step . . . . . 574
--lock-window-s . . . . . 574
--max-count . . . . . 107, 111
--max-node-step . . . . . 125, 179, 570
--max-nodes . . . . . 125, 179, 186, 567
--min-change-mhz . . . . . 574
--min-change-w . . . . . 574
--min-count . . . . . 107
--min-nodes . . . . . 125, 154, 157, 179, 181, 183, 193, 567
--min-share . . . . . 574
--min-target-change . . . . . 572
--mode . . . . . 49, 109, 124, 134, 139, 179, 206, 565, 570, 572
--native . . . . . 41, 146, 568
--network-policy . . . . . 584
--no-api-proxy . . . . . 105, 570
--node-restore-cmd . . . . . 49, 125, 567
--node-scale-cmd . . . . . 27, 125, 162, 567
--once . . . . . 150
--out . . . . . 150
--own-every-s . . . . . 42, 569
--own-replicas . . . . . 2, 42, 102, 104, 568
--own-target . . . . . 42, 569
--pace-heat . . . . . 572
--pace-high . . . . . 572
--pace-low . . . . . 572
--pod-reflex-writes . . . . . 571, 579
--port . . . . . 143
--power-cmd . . . . . 26, 51, 74, 125, 165, 569

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

--prometheus . . . 149
--quantile . . . 149
--quantiles . . . 149
--race-util . . . 573
--rapl-cmd . . . 26, 571
--reflex-window-s . . . 44, 572
--replica-ceiling . . . 20, 108, 125, 568
--replicas . . . 133
--restore-only . . . 570
--rightsize-deployments . . . 49, 571
--rightsize-headroom . . . 572
--rightsize-min-m . . . 572
--rollout-guard . . . 49, 571
--scale-down-delay-after-add . . . 158
--scale-down-unneeded-time . . . 158, 582
--scale-down-utilization-threshold . . . 158
--scan-interval . . . 158
--security-configmap . . . 49, 571
--sensed . . . 102, 122, 124, 179, 567
--served . . . 149
--site-limit-w . . . 21, 165, 569
--slo-clear . . . 572, 574
--slo-clear-s . . . 36, 42, 44, 78, 572
--slo-ms . . . 20, 31, 50, 51, 118, 121, 125, 179, 199, 214, 567, 573
--smi . . . 573
--speed-gain . . . 574
--strict-replicas . . . 50, 571
--strict-window . . . 572
--strict-window-s . . . 44, 572

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

--temp-limit . . . 574
--thermal-model . . . 571
--tier . . . 110
--trust-uid . . . 78
--unit-ms . . . 149
--up-gain . . . 574
--util-band . . . 574
--util-gate . . . 574
--verdict-every . . . 572, 573
--verdict-every-s . . . 44, 572
--verdict-recheck . . . 572, 573
--verdict-recheck-s . . . 44, 572
--verdict-samples . . . 572, 573
--verdict-trial-s . . . 44, 572
--window . . . 148, 149

.

.github/workflows/aks-metered.yml . . . 210

A

Active profile . . . 24
Admission policy . . . 168
Agriculture & Irrigation . . . 536
AI Inference Serving . . . 513
AI Training . . . 531
Alert minutes . . . 78
Automotive EV & Mobile Powertrain . . . 508
Aviation & Autonomous Flight . . . 510

B

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Band . . . . . 76
Billable room . . . . . 155
Blind . . . . . 36, 41, 119
Blockchain & Ledger Nodes . . . . . 527
Buffer pool . . . . . 71
Building & Critical Environment HVAC . . . . . 535

C

Cache & Memory Services . . . . . 524
Candidate profile . . . . . 24
Card . . . . . 19, 51, 136, 138, 140, 219, 572, 579
CI/CD & Build Farms . . . . . 517
Cloud VM & Capacity . . . . . 511
Cluster autoscaler . . . . . 154, 163, 195
Commerce & Payment Systems . . . . . 510
Compass . . . . . 601
Container Resources . . . . . 525
Content Delivery & Edge Cache . . . . . 529
Cooling; Chillers & Thermodynamics . . . . . 534
Cordon . . . . . 189
cpp/src/shield.cpp . . . . . 609
Cross-Cluster; Multi-Region & Edge . . . . . 512
Cruise . . . . . 6, 65
Custom profile . . . . . ix, 23

D

Data Analytics & ETL . . . . . 532
Data path . . . . . xxxiii
Database & Transactions . . . . . 509
deploy/Dockerfile . . . . . 108, 167, 209

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

deploy/Dockerfile.cloud . . . . . 126, 148, 209, 565
deploy/grafana/omni-compass.json . . . . . 144
deploy/install/omni-compass-cloud.yaml . . . . . 44, 126, 179, 209, 566
deploy/install/omni-compass.yaml . . . . . 104, 179, 209, 565
deploy/install/rbac-cloud.yaml . . . . . 102, 126, 209, 565
deploy/kind/demo.yaml . . . . . 109
deploy/kind/rbac-omni.yaml . . . . . 49, 82, 102, 209, 609
deploy/prometheus/omni-compass-rules.yaml . . . . . 145
deploy/rbac-own-pods.yaml . . . . . 42, 102
deploy/rbac-target.yaml . . . . . 48, 209
Dial . . . . . 19, 124
Distributed Cluster Managers . . . . . 533
District Heating & Cooling . . . . . 537
docs/AMAZON_SETUP.md . . . . . 210
docs/AZURE_SETUP.md . . . . . 57, 210
docs/BENEFIT_SHEET.md . . . . . xxxi
docs/DOSSIER.md . . . . . xxvii
docs/engine/check_live_engine.py . . . . . xxx
docs/engine/export_harness_trace.py . . . . . 12
docs/engine/export_whole.py . . . . . xxviii, xxix
docs/engine/monitors.json . . . . . xxviii
docs/engine/OMNI_COMPASS_LIVE_ENGINE.html . . . . . xxviii, xxx
docs/engine/PAGE_WORD_FOR_WORD.md . . . . . xxviii
docs/engine/wall/*.webm . . . . . 603
docs/engine/wall/README.md . . . . . 603
docs/EVIDENCE_LEDGER.md . . . . . xxiii, xxvii
docs/FACTORY_PROFILE.md . . . . . ix, xxv, 18, 23, 95, 200
docs/GOOGLE_SETUP.md . . . . . 210
docs/GPU_PREREGISTRATION.md . . . . . xxiii, xxvi, xxvii

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

docs/INDUSTRY_LANDSCAPE.md . . . . . xxxiii, 205
docs/K8S_COMPASS_PREREGISTRATION.md . . . . . xxiii, xxvii, 543, 547, 551
docs/MUSCLE_CATALOG.md . . . . . xxviii, 16, 18, 210
docs/OMNI_COMPASS_LIVE_ENGINE_MANUAL.md . . . . . xxviii
docs/OMNI_COMPASS_MANUAL.md . . . . . xxvii
docs/OMNI_COMPASS_MANUAL.pdf . . . . . xxvii
docs/OMNI_V18.md . . . . . 547, 551
docs/OMNI_V22.md . . . . . 42
docs/PILOT_PROTOCOL.md . . . . . 195
docs/REALMS.md . . . . . 219
docs/REALMS_PREREGISTRATION.md . . . . . xxiii, xxvii
docs/REALM_MUSCLES.md . . . . . 16, 18
docs/SOLO_PREREGISTRATION.md . . . . . 3
docs/STATE_OF_PLAY.md . . . . . xxvii, 18
docs/THE_SPINE.md . . . . . 219
docs/THE_TUNE.md . . . . . xxv, 124, 129, 151, 195, 217
docs/THREAT_MODEL.md . . . . . 610
docs/TRACES_PREREGISTRATION.md . . . . . 603
docs/WHOLE_SYSTEM_PREREGISTRATION.md . . . . . xxviii, xxix
docs/wiring_recipes.csv . . . . . 8, 16, 17, 35, 222
docs/WORK_PER_BILL.md . . . . . xxxii
DPU SmartNIC & Programmable IO . . . . . 530
Dry run . . . . . 126, 192

E

Elevators & Vertical Transport . . . . . 542
Energy Storage & Microgrid . . . . . 508
Engine version . . . . . 91, 198

F

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Facility & Grid Optimization . . . . . 508
Factory profile . . . . . ix, xxv, xxxiii, 18, 23
Factory tune . . . . . xxv, 18, 572
Fail up . . . . . 36, 41
Feed . . . . . 118, 119, 121, 136, 141, 148, 188, 191, 565
Financial Markets & Trading Systems . . . . . 516
fleet/capture/kube_capture.sh . . . . . 610
Four arms . . . . . xxx, xxxii

G

GAIN . . . . . xi, xiv, xvii, xviii, xix, xxi, 13, 44, 91, 93, 165
Game Servers & Real-Time Interactive . . . . . 516
Governor . . . . . x, xii, 13, 41, 71, 77, 78, 104, 108, 113, 117, 121, 122, 130
GPU Fabric & RDMA . . . . . 530
GPU Hardware . . . . . 513
Grid Transmission & Distribution . . . . . 539
Gut: Data-plane Intake & Waste . . . . . 540

H

Healthcare Critical Environments . . . . . 535
Host CPU & Memory . . . . . 526
HPC & Distributed Compute . . . . . 532

I

Idle at once . . . . . 6
Industrial PLC & Process Automation . . . . . 534

K

Kill switch . . . . . 6, 79
Kubernetes Dynamic Device Allocation . . . . . 512
Kubernetes Placement & Scheduling . . . . . 521

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Kubernetes Workload Scaling . . . . . 523

L

Lease . . . . . 131
Line . . . . . xvii, 27, 78, 118, 121, 140, 145, 182, 191, 544, 565
Linux Kernel & OS Control . . . . . 525
LOSS . . . . . xi, xiv, xvii, xviii, xix, xxi, 44, 91, 93, 196

M

Marine Propulsion & Vessel Automation . . . . . 520
Media Streaming & CDN . . . . . 527
Medical Imaging & Clinical Systems . . . . . 515
Messaging & Streaming . . . . . 518
Mining & Mineral Processing . . . . . 537
Muscle . . . . . xx, xxiv, 3, 7, 13, 19, 25, 32, 50, 52, 76, 81, 217, 219, 220

N

Native . . . . . xii, xx, xxxiii, 2, 3, 16, 40, 61, 136, 507, 508, 509, 510, 511
Network Routing & Switching . . . . . 529
Node Fleet & Karpenter-Class Control . . . . . 514

O

Observability & Telemetry . . . . . 522
Oil & Gas Pipelines . . . . . 536
Omni-Compass direct . . . . . xx, 2
omnicompass/muscle_sdk.py . . . . . 32
omnicompass/shield.py . . . . . 609
One pedal . . . . . xi, xxxiv
OpenShift & Machine API . . . . . 517

P

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.
All patents, copyrights and trademarks filed in the USA.

Parked machine . . . . . 162
PDU; UPS & Electrical Distribution . . . . . 519
Pharmaceutical & Food Manufacturing . . . . . 539
Plug . . . . . 25, 26, 27, 222
Ports & Maritime Logistics . . . . . 533
Power Generation & Turbine Control . . . . . 537
Power given . . . . . x, xiv, xviii, xix, xxi, xxiv, 19, 89, 91, 93

Q

Quantum Computing Control Simulation . . . . . 518

R

Rail Traction & Train Control . . . . . 542
realms/catalog_v5.csv . . . . . 8, 13, 210, 218
realms/muscle_cards_v5.csv . . . . . xxv, 20, 210, 219
Receipt . . . . . xxi, 32, 44, 84, 88, 124, 196
Reliability; Security & Recovery . . . . . 522
Renewable Generation & Inverter Control . . . . . 538
Requested; commanded; acknowledged . . . . . 81
Restore factory profile . . . . . xxxiii
results/gpu/run-20261002T082232Z/GPU_REPS.md . . . . . xxiii
results/gpu/V21_A100_CAP_ABC.md . . . . . xxx
results/gpu/V23_A10_CAP_FULL_ABC.md . . . . . xxx
results/gpu/V23_A10_FAST_ABC.md . . . . . xxx
results/live/AKS_*.md . . . . . xxxi
results/live/AKS_STEADY_100_OMNI_V18.md . . . . . xxx
results/OMNI_INDEX.md . . . . . ix, xxix, xxxi
results/OVERSIGHT.md . . . . . 195
results/realms_v5/REALMS_V5.json . . . . . 222
results/realms_v5/REALMS_V5.md . . . . . 17, 47

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

results/scenarios/SCENARIOS_20.json . . . . . xxx
results/scenarios/SCENARIOS_20.md . . . . . 13
results/SEAL.json . . . . . xxviii, 610
results/TWO_BILLS.md . . . . . xxix, xxx, xxxi, xxxii, 205
results/VERIFY_RECEIPT.txt . . . . . 105
results/WHOLE_SYSTEM.md . . . . . ix, xxviii, xxix, xxxi
Return to native . . . . . xxxiii
Robotics Fleet & Warehouse Automation . . . . . 511
Robotics Motion Control . . . . . 507
Room . . . . . 154, 155, 157, 159
Runtime & Application . . . . . 520

S

scripts/cpu_power_run.sh . . . . . 210
scripts/dispatch_grid.py . . . . . 17
scripts/gpu_8card.sh . . . . . 139
scripts/gpu_paired.sh . . . . . 139
scripts/gpu_rented_run.sh . . . . . 28, 84, 139, 210
scripts/kind_bench.sh . . . . . 2, 82, 86
scripts/kind_nodepool.sh . . . . . 123, 162, 209
scripts/kind_paired.sh . . . . . 84, 206, 210
scripts/latency_probe.py . . . . . 45, 46, 119, 209
scripts/pilot_shadow.sh . . . . . 48
Search; Indexing & Vector DB . . . . . 524
Semiconductor Fab & Precision Manufacturing . . . . . 540
Serverless & Functions (FaaS) . . . . . 521
Service Mesh & API Reliability . . . . . 519
Skin: Perimeter & Admission . . . . . 528
Snapshot . . . . . 38

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Spacecraft & Flight Software . . . . . 541
STEADY . . . . . xi, xiv, xvii, xviii, xix, 44, 93
Storage Block/File/Object . . . . . 528
System pool . . . . . 106, 112

T

Telecom RAN & Edge Radio . . . . . 526
tests/test_muscle_sdk.py . . . . . 35
tests/third_party/hpa_independent.py . . . . . 602
The runs disagree . . . . . xiii, xviii, xix, xxi, 93
tools/confirm_abc.py . . . . . 88, 94, 210
tools/engine_runs.py . . . . . xxviii
tools/gpu_baseline.py . . . . . 574
tools/gpu_wire_check.py . . . . . 51, 210
tools/gpu_workload.py . . . . . 51, 210
tools/heavy_hitters.py . . . . . 223
tools/kafka_abc.py . . . . . 207
tools/live_reps.py . . . . . 84, 87, 88, 210
tools/omni_control.py . . . . . 145
tools/omni_exporter.py . . . . . 143
tools/omni_switch.py . . . . . 210, 609
tools/omni_version.py . . . . . 82, 209, 604
tools/pgbench_abc.py . . . . . 207
tools/prom_feed.py . . . . . 148, 209
tools/readings.py . . . . . 91
tools/redis_abc.py . . . . . 207
tools/run_hil.py . . . . . 17
tools/run_orchestra.py . . . . . 12
tools/run_realms_v5.py . . . . . 17

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

tools/run_scale.py . . . . . 17
tools/seal.py . . . . . 209
tools/store_brain.py . . . . . 42
tools/sysbench_abc.py . . . . . 207
tools/tour_voice.py . . . . . 602
tools/tune_cards.py . . . . . 219
tools/tune_study.py . . . . . xxv
tools/wall_meter.py . . . . . 138, 163
tools/whole_system.py . . . . . xxviii, 13
tools/ycsb_abc.py . . . . . 207

V

Virtualization & Hypervisor . . . . . 515

W

Watchdog . . . . . 131
Water Wastewater & Pumping . . . . . 540
Wire check . . . . . xvii, 28, 31, 77, 138
Work Admission & Demand Shaping . . . . . 541
Work pool . . . . . 106, 112
Workflow; Logistics & Fulfillment . . . . . 531

BACK MATTER

# Contact

Licensing, pilots, evaluations and the Omni-Compass Enterprise License:

**The Omni-Compass LLC**

Owner and developer: AJ Dubra

Main office infrastructure: toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1

*© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Landline, registered in Florida: 321-746-4411

www.omni-compass.com

Repository: github.com/The-Omni-Compass-LLC/The-Omni-Compass-LLC

Any use beyond evaluation and simulation requires a signed, paid Omni-Compass Enterprise License. Write to The Omni-Compass LLC before any production use.

---

© 2026 Dubra Software Inc. / The Omni-Compass LLC. All rights reserved. **Evaluation and Simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from Dubra Software Inc. / The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. `SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0` All patents, copyrights and trademarks filed in the USA. www.omni-compass.com